

Fusion Compiler™

User Guide

中文学习译本

原版版本: V-2023.12-SP3, May 2024

Synopsys, Inc.

原文版权归 Synopsys, Inc. 所有 (Version V-2023.12-SP3, May 2024)。
本文件为非官方个人学习译本, 仅供学习参考, 不得用于商业传播。
命令名、选项名、文件名等技术标识符保持英文原文不译。

目录

关于本用户指南	ii
本版本新增内容	ii
相关产品、文档与商标	iii
排版约定	iii
客户支持	iii
包容性与多样性声明	iv
第一章 使用 Fusion Compiler 工具	1
1.1 物理综合设计流程概览	2
1.2 形式验证	2
1.3 布局布线设计流程概览	2
1.4 Fusion Compiler 相关概念	5
1.4.1 功耗意图概念	5
1.4.2 多重图案化概念	7
1.5 使用 3DIC Compiler 用户界面	8
1.5.1 启动 CLI	9
1.5.2 退出工具	10
1.6 输入 fc_shell 命令	10
1.6.1 中断或终止命令处理	10
1.6.2 获取命令信息	10
1.7 使用应用选项	11
1.8 使用变量	11
1.9 查看 Man 页	12
1.10 使用 Tcl 脚本	12
1.11 通过检查点向脚本添加变更	13
1.11.1 定义检查点	14
1.11.2 配置检查点	14
1.11.3 查询检查点与行为	16
1.11.4 查看检查点历史	17
1.12 使用 Setup 文件	17
1.13 使用命令日志文件	17
1.14 启用多核处理	17
1.14.1 配置多线程	18

1.14.2	配置分布式处理	18
1.14.3	报告与移除多核配置	19
1.14.4	并行运行任务	19
1.14.5	在本地主机并行运行命令	19
第二章	准备设计	21
2.1	定义搜索路径	22
2.2	设置库	22
2.2.1	处理设计库	22
2.2.2	设置参考库	23
2.2.3	库配置	24
2.2.4	限制库单元使用	25
2.2.5	限制所用目标库	26
2.3	指定库子集限制	27
2.4	使用 Pin Access Checker 实用程序	28
2.5	分析库	29
2.5.1	库单元族中冗余、离群、等价与缺口简介	30
2.5.2	识别库单元族中的冗余、离群、等价与缺口	32
2.5.3	比较库	36
2.6	读取设计	39
2.6.1	以 SystemVerilog、Verilog 或 VHDL 格式读取 RTL 文件	40
2.6.2	使用 -autoread 选项读取设计	41
2.6.3	使用 VCS 命令行选项读取设计	41
2.6.4	读取 Verilog 门级网表文件	42
2.6.5	读取混合的门级网表与 RTL 文件	42
2.6.6	在 RTL 代码中嵌入 Tcl 命令	43
2.7	缓解设计不匹配	44
2.8	处理设计（门级网表与 block 操作补充）	44
2.9	导入 Floorplan 信息	45
2.9.1	读取 DEF 文件	45
2.10	设置多电压设计	50
2.10.1	应用多电压功耗意图	50
2.10.2	准备电源网络	52
2.10.3	定义电压区域	54
2.10.4	插入多电压单元	62
2.10.5	控制多电压单元的布局	63
2.10.6	启用多电压 Net 的改进缓冲	64
2.10.7	分析多电压信息	64
2.11	指定时序约束与设置	64
2.12	指定逻辑设计规则约束	65
2.13	指定时钟门控约束	66
2.14	控制时钟门控延迟	67
2.14.1	集成时钟门控延迟估计	67

2.14.2 用户指定的时钟门控延迟	68
2.15 为布局与合法化指定物理约束	71
2.15.1 定义 Keepout Margins	71
2.15.2 定义基于区域的布局障碍	73
2.15.3 定义布局 Bounds	77
2.15.4 查询布局 Bound	80
2.15.5 移除布局 Bound	80
2.15.6 定义布局 Attraction	80
2.15.7 定义合法化单元间距约束	81
2.15.8 报告单元间距约束	83
2.15.9 移除单元间距约束	83
2.16 指定布局设置	83
2.16.1 在早期阶段使用不精确约束进行布局	84
2.16.2 为时钟门控单元自动生成 Group Bound	84
2.16.3 控制布局密度	84
2.16.4 控制布局期间的拥塞驱动重构	85
2.16.5 降低拥塞	85
2.16.6 布局时考虑宽单元密度	86
2.16.7 布局时考虑单元 Pin 的影响	86
2.16.8 布局时考虑时钟 Net 非默认布线规则带来的拥塞影响	86
2.16.9 布局时考虑时序阵列中时钟门控单元的影响	87
2.16.10 布局时考虑合法化影响	87
2.16.11 布局时考虑 DFT 连接	87
2.16.12 布局时考虑动态功耗 QoR	87
2.16.13 执行 IR-Drop 感知布局	87
2.16.14 布局时分散 Repeater 单元	90
2.17 指定合法化设置	90
2.17.1 合法化期间最小化大位移	90
2.17.2 合法化期间优化 Pin Access	91
2.17.3 启用高级 PG Net 检查	91
2.17.4 启用高级合法化算法	91
2.17.5 设置 Variant 感知合法化	92
2.17.6 检查库单元是否可合法放置	94
2.18 控制单元、Net、Pin 与 Port 的优化	95
2.18.1 优化期间保留单元与 Net	95
2.18.2 将优化限制为仅做单元尺寸调整	96
2.18.3 优化期间保留网络	97
2.18.4 标记时钟网络	97
2.18.5 禁用设计规则检查 (DRC)	97
2.18.6 尺寸调整期间保留 Pin 名称	99
2.18.7 保留已有层次的 Port	99
2.18.8 隔离输入与输出 Port	99
2.18.9 修复多 Port Net	100

2.18.10	控制向 Module、层次单元与 Voltage Area 添加新单元	101
2.18.11	为优化指定单元名前缀	101
2.19	指定布线前优化设置	102
2.19.1	控制 place_opt 的 DFT 优化	102
2.19.2	指定布线前优化的寄生参数估计设置	103
2.19.3	指定布线前优化的自动 Via Ladder 插入设置	104
2.19.4	在高利用率区域启用面积回收	105
2.19.5	启用高级逻辑重构	105
2.20	设置功耗相关功能	106
2.20.1	标注开关活动	106
2.20.2	为 place_opt 与 clock_opt 启用功耗优化	109
2.20.3	通过限制 LVT 单元百分比改善良率	110
2.20.4	更新 Activity 以改善功耗优化	111
2.20.5	启用电源完整性功能	112
2.21	指定布线资源	114
2.21.1	指定全局层约束	114
2.21.2	指定 Net 专用层约束	115
2.21.3	指定时钟树层约束	116
2.21.4	启用基于 Level 的时钟非默认布线规则	118
2.21.5	为层设置优选布线方向	119
2.22	使用 Early Data Check Manager 处理设计数据	119
2.23	应用 Mega-Switch 命令设置	120
2.23.1	应用先进工艺节点所需设置	121
2.23.2	应用高性能核所需设置	121
2.23.3	应用改善特定 QoR 指标所需设置	122
第三章	物理综合	123
3.1	执行统一物理综合	123
3.1.1	使用 compile_fusion 命令	124
3.1.2	执行预检查	125
3.1.3	编译期间生成验证检查点	126
3.2	控制映射与优化	127
3.2.1	优化期间取消分组或保留层次	127
3.2.2	控制边界优化	129
3.2.3	控制数据通路优化	130
3.2.4	控制 MUX 优化	134
3.2.5	控制时序映射	137
3.2.6	控制寄存器复制	142
3.2.7	控制寄存器合并	143
3.2.8	选择性移除或保留常量与无负载寄存器	144
3.2.9	报告优化寄存器的交叉探测信息	145
3.2.10	控制高扇出 net 综合	146
3.3	执行多比特优化	146

3.3.1	执行集成多比特寄存器优化	147
3.3.2	从特定单比特寄存器创建多比特寄存器 bank	149
3.3.3	指定多比特寄存器命名风格	150
3.3.4	报告多比特寄存器	151
3.3.5	识别为何未执行多比特 Banking	151
3.3.6	改善 Banking 比率	153
3.3.7	在 GUI 中查看多比特组件	153
3.3.8	映射组合多比特单元	154
3.4	运行并发时钟与数据优化	155
3.4.1	Useful Skew	155
3.4.2	控制并发时钟与数据优化	155
3.5	执行测试插入与扫描综合	158
3.5.1	扫描综合流程	158
3.5.2	最小化扫描寄存器异步置位与复位线上的毛刺	160
3.6	指定性能、功耗与面积改善设置	160
3.6.1	启用高努力时序模式	160
3.6.2	启用增强延迟优化以改善总负松弛	160
3.6.3	启用高努力面积模式	161
3.6.4	启用嵌入式面积优化流程	161
3.7	执行设计分析	161
3.7.1	报告命令与示例	161
3.7.2	衡量结果质量	166
3.7.3	对比 QoR 数据	168
3.7.4	在批处理模式下报告逻辑级数	175
3.7.5	查询特定消息 ID	177
第四章	时钟门控	178
4.1	时钟门控简介	179
4.1.1	工具插入时钟门控的命名约定	180
4.1.2	编译日志文件中的时钟门控	181
4.1.3	时钟门控级数与阶段	182
4.2	时钟门控先决条件	183
4.2.1	时钟门控条件	183
4.2.2	时钟门控使能条件	184
4.2.3	时钟门控建立条件	184
4.3	设置时钟门控	184
4.3.1	设置时钟门控风格	185
4.3.2	设置时钟门控选项	185
4.3.3	对设计对象启用或禁用时钟门控	186
4.3.4	时钟门控使能源选择	186
4.3.5	用户驱动的使能排除	187
4.3.6	报告时钟门控选项	188
4.4	时钟门控流程	189

4.4.1	在多电压设计中插入时钟门控	190
4.4.2	在 RTL 设计中插入时钟门控	190
4.5	替换时钟门控	191
4.6	控制时钟门控延迟	191
4.6.1	集成时钟门控延迟估计	192
4.6.2	用户指定时钟门控延迟	193
4.6.3	set_clock_latency 命令	193
4.6.4	set_clock_gate_latency 命令	194
4.7	控制时钟门控级数	196
4.7.1	时钟门控多级扩展	196
4.7.2	时钟门控折叠	197
4.7.3	控制工具插入的时钟门控级数	197
4.7.4	基于活动的优化	197
4.8	合并时钟门控	198
4.9	设置时钟门控变换	198
4.9.1	使用时钟门控变换的示例	199
4.10	为时钟门控设置布线规则	200
4.11	时钟门控与多比特寄存器	200
4.12	布局感知时钟门控	200
4.13	基于扇入的时序时钟门控	201
4.13.1	设置时序时钟门控选项	202
4.13.2	报告时序时钟门控	202
4.14	报告时钟门控结果	203
4.15	报告时钟门控使能信号	204
4.16	时钟门控效率报告	204
4.17	自门控优化	207
4.17.1	自门控流程	209
4.17.2	自门控的库要求	209
4.17.3	自门控设置	209
4.17.4	设置自门控选项	210
4.17.5	控制自门控对象的选择	210
4.17.6	报告与查询自门控单元	211
4.18	DFT wrapper 时钟门控的特殊命名与查询	212
第五章	时钟树综合	214
5.1	CTS 前提条件	214
5.2	定义时钟树	215
5.2.1	推导时钟树	215
5.2.2	定义时钟树例外	218
5.2.3	限制时钟网络上的优化	220
5.2.4	跨 Mode 复制时钟树例外	221
5.2.5	由理想时钟延迟推导时钟树例外	221
5.2.6	处理存在平衡冲突的端点	222

5.3	验证时钟树	223
5.4	设置时钟树设计规则约束	224
5.5	指定 CTS 设置	224
5.5.1	指定时钟树 References	225
5.5.2	设置 Skew 与 Latency 目标	226
5.5.3	启用局部 Skew 优化	226
5.5.4	指定 CTS 主 Corner	227
5.5.5	阻止综合特定时钟	227
5.5.6	保留既有时钟树	227
5.5.7	启用时钟树降功耗技术	227
5.5.8	启用线长感知的功耗驱动搬移	228
5.5.9	降低电迁移	228
5.5.10	处理不准确约束	228
5.5.11	定义时钟单元间距规则	229
5.5.12	创建 Skew Groups	229
5.5.13	定义时钟单元名前缀	229
5.5.14	初始 CTS 使用全局路由器	229
5.5.15	为时钟网络指定约束	230
5.5.16	为特定端点构建更快时钟树	230
5.5.17	改进延迟驱动的时钟门分裂	230
5.5.18	降低时钟网络上的信号完整性影响	231
5.5.19	指定时钟延迟调整设置	231
5.5.20	报告时钟树设置	231
5.6	实现时钟树并执行 CTS 后优化	232
5.6.1	独立时钟树综合	232
5.6.2	用 clock_opt 综合、优化并布线时钟树	233
5.6.3	控制并发时钟与数据优化	234
5.6.4	分裂时钟单元	238
5.6.5	平衡不同时钟树之间的 Skew	238
5.6.6	使用机器学习数据的基于全局布线优化	239
5.6.7	布线时钟树	240
5.6.8	在 CTS、优化与时钟布线中插入 Via Ladder	240
5.6.9	CTS 后将时钟标记为 Propagated	240
5.6.10	执行布线后时钟树优化	241
5.6.11	执行电压优化	241
5.6.12	将时钟树标记为已综合	241
5.6.13	移除时钟树	241
5.7	实现多源时钟树	243
5.7.1	多源时钟树结构简介	243
5.7.2	实现常规多源时钟树	244
5.7.3	使用集成 Tap Assignment 的常规多源时钟树	244
5.7.4	仅含 H-Tree 全局结构的常规多源时钟树	245
5.7.5	实现 MSCTS 多重 H-Tree	245

5.7.6	实现功耗感知柔性 H-Tree	245
5.7.7	实现结构型多源时钟树	245
5.7.8	使用集成子树综合的 SMSCTS	246
5.7.9	改善 SMSCTS 与 CCD 的相关性	246
5.7.10	插入时钟驱动器	246
5.7.11	综合全局时钟树	247
5.7.12	创建时钟 Strap	247
5.7.13	布线到时钟 Strap	248
5.7.14	分析 Clock Mesh	249
5.7.15	自动 Tap 插入与 H-Tree 综合	249
5.7.16	指定 Tap Assignment 选项与设置	250
5.7.17	构建本地时钟子树结构	250
5.8	分析时钟树结果	250
5.8.1	生成时钟树 QoR 报告	250
5.8.2	报告时钟树功耗	251
5.8.3	创建时钟网络引脚集合	251
5.8.4	分析时钟时序	251
5.8.5	在 GUI 中分析时钟树	252
第六章	布线与布线后优化	253
6.1	Zroute 简介	254
6.2	基本 Zroute 流程	254
6.3	布线前提条件	255
6.4	定义 Via	255
6.4.1	从 LEF 读取 Via 定义	256
6.4.2	创建 Via 定义	256
6.5	插入 Via Ladder	257
6.5.1	定义 Via Ladder 规则	258
6.5.2	为电迁移 Via Ladder 生成规则	258
6.5.3	为性能 Via Ladder 生成规则	259
6.5.4	基于约束的 Via Ladder 插入	259
6.6	检查可布线性	261
6.7	布线约束	262
6.7.1	定义 Routing Blockage	262
6.7.2	定义 Routing Guide	264
6.7.3	推导 Routing Guide	265
6.7.4	控制 Block 边界周围布线	266
6.7.5	在特定区域内布线 (Routing Corridor)	267
6.7.6	使用非默认布线规则 (NDR)	267
6.7.7	控制 Off-Grid 布线	270
6.7.8	Must-Join Pin 布线	270
6.7.9	控制 Pin 连接与 Tapering	271
6.7.10	设置重布线模式	272

6.7.11	定义与使用 Routing Corridor (详述)	272
6.7.12	布线层资源与冻结层 (章内相关)	272
6.7.13	天线规则与冗余 Via (章内要点)	273
6.8	布线应用选项	273
6.9	时钟网络布线	273
6.10	关键网络布线	274
6.11	次级电源与地 Pin 布线	274
6.11.1	验证属性	274
6.11.2	设置约束并布线	274
6.12	信号网络布线	275
6.12.1	全局布线	275
6.12.2	轨道分配	276
6.12.3	详细布线	276
6.12.4	自动布线	277
6.13	网络屏蔽	277
6.13.1	定义并分配屏蔽规则	278
6.13.2	布线前屏蔽	278
6.13.3	信号布线中的软屏蔽规则	278
6.13.4	布线后屏蔽与增量屏蔽	278
6.14	执行布线后优化	279
6.14.1	布线后逻辑优化	279
6.14.2	使用 hyper_route_opt	280
6.14.3	修复 Pin Access 引起的 DRC	280
6.15	分析并修复信号电迁移违例	280
6.16	执行 ECO 布线	281
6.17	在 GUI 中布线	281
6.18	清理已布线网络	282
6.19	分析布线结果	282
6.19.1	拥塞报告与拥塞图	282
6.19.2	用 Zroute 做设计规则检查	283
6.19.3	签核 DRC、外部工具与 LVS	283
6.19.4	报告布线结果与线长	283
6.19.5	DRC 查询命令	284
6.20	保存布线信息	284
6.21	推导掩模颜色	284
6.22	插入与移除 Cut Metal 形状	284
第七章	芯片收尾与可制造性设计	286
7.1	插入 Tap Cell	286
7.1.1	使用 create_tap_cells 命令	287
7.1.2	使用 create_cell_array 命令	289
7.1.3	插入 Exterior Tap Wall	290
7.1.4	插入 Interior Tap Wall	290

7.1.5	插入 Tap Mesh	291
7.1.6	插入 Dense Tap Array	292
7.2	执行 Boundary Cell 插入	292
7.2.1	指定 Boundary Cell 插入要求	292
7.2.2	Boundary Cell 插入期间创建 Routing Guide	295
7.2.3	Boundary Cell 插入期间创建 Placement Blockage	295
7.2.4	报告 Boundary Cell 插入要求	295
7.2.5	移除 Boundary Cell 插入要求	296
7.2.6	插入 Boundary Cell	296
7.2.7	验证 Boundary Cell 布局	296
7.3	查找并修复天线违例	297
7.3.1	定义天线规则	297
7.3.2	设置天线模式	302
7.3.3	指定天线属性	303
7.3.4	分析并修复天线违例	304
7.3.5	详细布线期间插入二极管	305
7.4	插入冗余 Via	306
7.4.1	在时钟网上插入冗余 Via	306
7.4.2	在信号网上插入冗余 Via	307
7.4.3	报告冗余 Via 率	310
7.5	优化线长与 Via 数量	311
7.6	减小临界面积	311
7.6.1	执行 Wire Spreading	311
7.6.2	执行 Wire Widening	312
7.7	插入金属-绝缘体-金属电容	312
7.8	插入 Filler Cell	313
7.8.1	标准 Filler Cell 插入	313
7.8.2	控制标准 Filler Cell 插入	314
7.8.3	检查 Filler Cell DRC 违例	314
7.8.4	修复剩余 Mask 设计规则违例	315
7.8.5	随机 Filler Cell 替换	319
7.8.6	用特定 Filler 邻接标准单元	319
7.8.7	控制基于单元的 Filler 插入	319
7.8.8	基于阈值电压的 Filler 插入	320
7.8.9	控制基于阈值电压的 Filler 插入	320
7.8.10	移除阈值电压 Filler 信息	320
7.9	插入 Metal Fill	320
第八章	IC Validator In-Design	322
8.1	运行 IC Validator In-Design 命令前的准备工作	322
8.1.1	搭建 IC Validator 运行环境	323
8.1.2	启用 IC Validator 多核处理	323
8.1.3	定义 IC Validator In-Design 命令使用的 Layer Mapping	325

8.2	执行签核级设计规则检查	325
8.2.1	运行 <code>signoff_check_drc</code> 命令	326
8.2.2	在 GUI 中交互式检查签核设计规则	331
8.3	自动修复签核 DRC 违例	334
8.3.1	生成 Autofix 配置文件	334
8.3.2	设置签核 DRC 修复的应用选项	335
8.3.3	运行 <code>signoff_fix_drc</code> 命令	336
8.3.4	自动修复双重曝光奇环违例	338
8.3.5	自动修复设计规则汇总报告	338
8.4	通过电源网增强改善实例电压降	339
8.4.1	标准电源网增强	340
8.4.2	设置电源网增强的应用选项	341
8.4.3	时序驱动电源网增强	341
8.4.4	引导式电源网增强	342
8.4.5	移除电源网增强图形	342
8.5	使用 IC Validator In-Design 插入 Metal Fill	343
8.5.1	设置签核 Metal Fill 插入的应用选项	343
8.5.2	执行 Metal Fill 插入	344
8.5.3	签核 Metal Fill 结果文件	353
8.5.4	查询 Metal Fill	353
8.5.5	在 GUI 中查看 Metal Fill	354
8.5.6	报告金属密度	354
8.5.7	移除 Metal Fill	355
8.5.8	修改 Metal Fill	357
8.5.9	执行真实 Metal Fill 提取	357
8.6	自动修复孤立 Via	357
8.6.1	设置修复孤立 Via 的应用选项	358
8.6.2	运行 <code>signoff_fix_isolated_via</code> 命令	358
第九章	使用 Custom Router 布线	360
9.1	在 Fusion Compiler 工具中使用 Custom Router	361
9.2	使用 Custom Router 前的准备	361
9.3	定义布线约束	362
9.3.1	定义总线布线风格	363
9.3.2	创建差分组	363
9.3.3	在网络上插入屏蔽	364
9.3.4	定义网络优先级	365
9.3.5	定义最小走线长度	365
9.3.6	定义匹配走线长度	367
9.4	管理约束组	370
9.5	使用 Custom Router 应用选项	371
9.5.1	总线布线选项	371
9.5.2	网格贴合选项	373

9.5.3	差分对选项	373
9.5.4	屏蔽选项	374
9.5.5	单环长度补偿	374
9.5.6	平衡模式选项	374
9.6	使用 Custom Router 布线	375
9.7	网络屏蔽	376
9.8	检查布线结果	376
9.9	使用混合布线流程	376
9.10	使用 DDR 布线流程	377
第十章	相对布局物理数据通路	379
10.1	相对布局技术概述	380
10.1.1	相对布局的主要收益	380
10.2	相对布局流程	380
10.3	创建相对布局组	381
10.4	向组中添加对象	382
10.4.1	添加 Leaf Cell	382
10.4.2	指定 Leaf Cell 的方向	382
10.4.3	添加硬宏单元	383
10.4.4	层次化相对布局组	383
10.4.5	添加 Blockage	385
10.4.6	在预定义区域内自由添加单元	385
10.5	设置相对布局组选项	386
10.5.1	锚定相对布局组	386
10.5.2	列内 Leaf Cell 对齐	387
10.5.3	控制组内 Tiling 方式	389
10.5.4	相对布局组方向	389
10.5.5	Keepout Margin	390
10.5.6	处理布局中的固定单元	390
10.5.7	允许非相对布局单元	391
10.5.8	控制相对布局单元的优化	391
10.5.9	控制 Legalize 时的移动量	392
10.5.10	处理 Tap Column 与相对布局组重叠	392
10.6	修改相对布局组结构	392
10.7	为时钟 Sink 生成相对布局组	392
10.8	相对布局组的布局与合法化	393
10.8.1	存在 Obstruction 时的相对布局	393
10.8.2	对已布局设计做相对布局组合法化	393
10.8.3	在已布局设计中新增相对布局组	394
10.9	分析相对布局组	395
10.9.1	布局前检查约束	395
10.9.2	分析相对布局组的可布局性	395
10.9.3	报告相对布局约束违例	395

10.9.4 查询相对布局组	396
10.9.5 在 GUI 中分析相对布局	396
10.10 保存相对布局信息	396
10.11 相对布局命令汇总	396
第十一章 层次化实现	399
11.1 使用 Abstract 进行层次化综合	400
11.1.1 全芯片设计的划分与规划	400
11.1.2 综合子块	403
11.1.3 综合顶层	404
11.2 使用 ETM 进行层次化综合	404
11.2.1 使用 ETM 运行层次化综合可行性分析流程	405
11.2.2 使用 ETM 运行层次化综合流程	405
11.2.3 使用 ETM 的层次化综合示例脚本	406
11.3 层次化综合流程的预检查	407
11.4 提供块级测试模型	407
11.5 指定块级功耗意图	408
11.6 Abstract 视图概述	408
11.7 创建 Abstract 视图	409
11.7.1 创建含功耗信息的 Abstract	410
11.7.2 为信号电迁移分析创建 Abstract	411
11.7.3 处理多层物理层次	411
11.7.4 为最小脉宽分析创建 Abstract	412
11.8 报告 Abstract 纳入原因	412
11.9 创建 Abstract 之后修改 Block	413
11.10 创建 Frame 视图	413
11.11 在顶层链接 Abstract 视图	413
11.12 将 Block 从 Abstract 切换为 Design 视图	414
11.13 链接带多个 Label 的子块	415
11.14 从顶层指定 Block 的可编辑性	415
11.15 使用 Abstract 做顶层收敛前的准备	416
11.16 检查含 Abstract 设计的顶层收敛问题	417
11.16.1 用 Early Data Check Manager 处理设计数据	419
11.17 使用 Abstract 视图执行顶层收敛	424
11.18 创建 ETM 与 ETM Cell Library	425
11.18.1 在 Fusion Compiler 工具中创建 ETM 与 ETM Cell Library	425
11.18.2 在 PrimeTime 工具中创建 ETM	426
11.18.3 在 Library Manager 工具中创建 ETM Cell Library	426
11.19 在顶层链接 ETM	427
11.20 使用 ETM 执行顶层收敛	428
11.21 透明层次优化	428
11.21.1 访问库	429
11.21.2 设置透明层次优化	429

11.21.3 在 compile_fusion final_place 阶段执行 THO	430
11.21.4 在 compile_fusion final_opto 阶段执行 THO	430
11.21.5 compile_fusion 之后重新预算	431
11.21.6 在 route_opt 或 hyper_route_opt 期间执行 THO	431
第十二章 RedHawk 与 RedHawk-SC Fusion	432
12.1 使用 RedHawk-SC Fusion 运行轨分析	433
12.2 RedHawk Fusion 与 RedHawk-SC Fusion 概述	434
12.2.1 RedHawk Fusion 与 RedHawk-SC Fusion 数据流	435
12.2.2 RedHawk/RedHawk-SC Fusion 分析流程	436
12.2.3 在后台运行 RedHawk Fusion 命令	436
12.2.4 报告并检查后台进程状态	436
12.3 设置可执行程序	437
12.4 指定 RedHawk 与 RedHawk-SC 工作目录	437
12.5 为轨分析准备设计与输入数据	438
12.5.1 生成轨分析脚本文件	440
12.5.2 支持 RedHawk-SC 定制 Python 脚本文件	441
12.6 将理想电压源指定为 Tap	444
12.6.1 验证 Tap 并查找无效 Tap	446
12.6.2 移除 Tap	447
12.7 缺失 Via 与未连接 Pin 检查	448
12.7.1 设置检查选项	449
12.7.2 检查缺失 Via 与未连接 Pin	449
12.7.3 查看错误数据	450
12.7.4 修复缺失 Via	451
12.8 使用多个轨场景运行轨分析	452
12.8.1 为轨分析指定设计场景	452
12.8.2 创建并指定用于轨分析的轨场景	452
12.8.3 示例	453
12.9 执行压降分析	455
12.9.1 查看压降分析结果	456
12.10 执行 PG 电迁移分析	457
12.10.1 查看 PG 电迁移分析结果	457
12.11 执行最小路径电阻分析	459
12.11.1 查看最小路径电阻分析结果	460
12.11.2 使用鼠标工具追踪最小路径电阻	461
12.12 执行有效电阻分析	462
12.13 执行分布式 RedHawk Fusion 轨分析	463
12.13.1 许可证要求	463
12.13.2 启用分布式处理的步骤	463
12.14 电压驱动电源开关单元尺寸调整	465
12.15 使用宏模型	466
12.15.1 生成宏模型	466

12.15.2 创建 Block Context	468
12.16 执行签核分析	470
12.17 写出分析与检查报告	471
12.17.1 显示 Block 级轨结果	473
12.17.2 生成基于实例的分析报告	474
12.17.3 生成基于几何的分析报告	475
12.18 在 GUI 中显示地图	475
12.18.1 示例	476
12.19 在 GUI 中显示 ECO 形状	478
12.19.1 脚本示例	478
12.20 电压热点分析	479
12.20.1 生成热点	480
12.20.2 报告热点	480
12.20.3 移除热点	482
12.20.4 电压热点分析示例	482
12.21 查询属性	483
第十三章 ECO 流程	484
13.1 时序或功能变更的通用 ECO 流程	485
13.2 Freeze Silicon ECO 流程	485
13.3 Signoff ECO 流程	486
13.4 增量 Signoff ECO 流程	487
13.5 ECO Fusion 流程	488
13.6 ECO Fusion 电源完整性流程	489
13.7 手动实例化备用单元	489
13.8 自动添加备用单元	490
13.9 添加可编程备用单元	491
13.10 使用 eco_netlist 命令进行 ECO 更改	492
13.11 使用网表编辑命令进行 ECO 更改	493
13.11.1 使用 ECO 脚本进行网表编辑	493
13.12 调整单元尺寸	493
13.12.1 撤销尺寸调整期间所做的更改	494
13.13 在 net 上添加缓冲器	494
13.14 在已布线 net 上添加缓冲器	494
13.14.1 指定 net 名称、缓冲器类型及其位置	495
13.14.2 控制缓冲器的添加方式	498
13.14.3 为多电压设计指定设置	498
13.14.4 为 Freeze Silicon ECO 流程指定设置	498
13.15 优化 net 扇出	499
13.16 报告放置 ECO 单元的可用 site	499
13.17 识别并撤销非最优 ECO 更改	499
13.18 放置 ECO 单元	500
13.18.1 使用 place_eco_cells 时控制布局	500

13.18.2 使用 <code>place_eco_cells</code> 时控制合法化	501
13.18.3 以最小物理影响 (MPI) 放置 ECO 单元	502
13.19 将 ECO 单元放置并映射到备用单元	502
13.19.1 为可编程备用单元指定映射规则	502
13.19.2 将 ECO 单元映射到特定备用单元	502
13.19.3 将 ECO 单元映射到逻辑等效备用单元	503
13.20 更新 ECO 单元的电源 <code>net</code>	503
13.21 记录对版图所做的更改	504
13.22 执行成组中继器插入与放置的先决条件检查	504
13.23 添加一组中继器	505
13.23.1 定义一组中继器	506
13.23.2 将中继器列表分组	506
13.23.3 为中继器组设置约束	507
13.23.4 添加电压区域感知的成组中继器	507
13.23.5 报告分配给中继器组的约束	507
13.23.6 移除中继器组的约束	507
13.23.7 布线前放置成组中继器	508
13.23.8 执行沿布线的中继器放置	508
13.23.9 为多比特寄存器放置成组中继器	508
13.23.10 指定中继器组位置	509
13.23.11 允许中继器组位于宏之上	509
13.23.12 指定中继器组的切割间距与切割距离	510
13.23.13 指定中继器组的水平与垂直间距	510
13.23.14 指定用作中继器的库单元	510
13.23.15 避免中继器与既有 <code>tap</code> 单元重叠	510
13.23.16 成组中继器插入期间避免串扰	510
13.23.17 预览中继器组	511
13.23.18 取消中继器放置	511
13.23.19 移除中继器组	511
13.24 查询成组中继器	511
13.24.1 执行自动分组流程	512
13.24.2 执行手动分组流程	512
13.24.3 单元输入模式	513
13.25 交换变体单元	513
13.25.1 设置变体单元约束	513
13.25.2 设置应用选项	514
13.25.3 对变体单元分组	514
13.25.4 运行并放置成组中继器	515
13.25.5 故障排除	515
13.26 修复多电压违例	520
13.26.1 修复缓冲器	520
13.26.2 修复二极管	521

关于本用户指南

About This User Guide

Synopsys Fusion Compiler 工具提供完整的 netlist-to-GDSII 或 netlist-to-OASIS® 设计方案，将专有的设计规划（design planning）、物理综合（physical synthesis）、时钟树综合（clock tree synthesis）与布线（routing）贯穿逻辑与物理实现全流程。

本指南描述 Fusion Compiler 的实现与集成流程。关于工具的更多信息，请参阅下列配套文档：

- *Library Manager User Guide*
- *Fusion Compiler Design Planning User Guide*
- *Fusion Compiler Data Model User Guide*
- *Fusion Compiler Timing Analysis User Guide*
- *Fusion Compiler Graphical User Interface User Guide*
- *Fusion Compiler Multivoltage User Guide*
- *Fusion Compiler Power Analysis User Guide*

本用户指南面向使用 Fusion Compiler 工具实现设计的工程师。要使用该工具，需要具备物理设计与综合技能，并熟悉：

- 物理设计原理
- Linux 操作系统
- 工具命令语言（Tcl）

本前言包括以下各节：

- 本版本新增内容
- 相关产品、文档与商标
- 排版约定
- 客户支持
- 包容性与多样性声明

本版本新增内容

关于新功能、增强与变更、已知限制，以及已解决的 Synopsys Technical Action Requests（STAR）信息，请参阅 SolvNetPlus 站点上的 *Fusion Compiler Release Notes*。

相关产品、文档与商标

有关 Fusion Compiler 工具的更多信息，请参阅 Synopsys SolvNetPlus 支持站点上的文档：<https://solvnetplus.synopsys.com>

您可能还希望参阅以下相关 Synopsys 产品文档：

- Design Compiler®
- IC Validator
- PrimeTime Suite®
- StarRC™

排版约定

Synopsys 文档使用下列约定。

表 1: 文档排版约定

约定	说明
Courier	表示语法，例如 <code>write_file</code>
Courier italic	表示语法中的用户自定义值，例如 <code>write_file design_list</code>
Courier bold	表示示例中用户原样输入的内容，例如 <code>prompt> write_file top</code>
紫色高亮	• 在示例中表示特别关注的信息； • 在命令语法中表示默认值，例如 <code>include_enclosing = true false</code>
[]	表示语法中的可选参数，例如 <code>write_file [-format fmt]</code>
...	表示参数可按需重复，例如 <code>pin1 pin2 ... pinN</code>
 	表示在多个备选中选择其一，例如 <code>low medium high</code>
\	表示命令行续行
/	表示目录层次
粗体	表示带有关联操作的图形用户界面（GUI）元素
Edit > Copy	表示菜单命令路径，例如打开 Edit 菜单并选择 Copy
Ctrl+C	表示组合键，例如按住 Ctrl 再按 C

客户支持

客户支持通过 SolvNetPlus 提供。

访问 SolvNetPlus

SolvNetPlus 站点包含技术文章知识库以及对 Synopsys 工具常见问题的解答，并可访问软件下载、文档与技术支持等在线服务。

访问地址: <https://solvnetplus.synopsys.com>

如提示, 请输入用户名与密码。若尚无 Synopsys 账号, 请按页面说明注册。若使用 SolvNetPlus 时需要帮助, 请点击右上角菜单栏中的 **REGISTRATION HELP**。

联系客户支持

请访问 <https://solvnetplus.synopsys.com>。

包容性与多样性声明

Synopsys 致力于营造包容的环境, 使每位员工、客户与合作伙伴都感到受欢迎。公司正在审查并从产品及面向客户的材料中移除排他性用语; 内部亦推进从工程与工作环境中去除带有偏见的语言, 包括嵌入软件与 IP 中的术语。同时, Synopsys 努力确保 Web 内容与软件应用可供不同能力的用户使用。由于部分 IP 实现仍遵循尚在审查中的业界标准规范, 软件或文档中可能仍可见非包容性用语。

Chapter 1

使用 Fusion Compiler 工具

Working With the Fusion Compiler Tool

Fusion Compiler 工具支持以下功能：

- 提取（**extraction**）与时序分析（**timing analysis**）
- 物理综合与优化（**physical synthesis and optimization**）
- 布局与优化（含相对布局 **relative placement**）
- 时钟树综合（**clock tree synthesis**）
- 布线（**routing**）
- 芯片收尾（**chip finishing**）
- 层次化设计的顶层收敛（**top-level closure**）
- 工程变更指令（**ECO**）
- 报告（**reporting**）
- ASCII 输出接口

工具以 Verilog 网表、时序约束、逻辑与物理库以及晶圆厂工艺数据为输入；输出可为 OASIS 或 GDSII 格式的版图，或可供第三方路由器使用的已布局网表 DEF（**Design Exchange Format**）文件。也可随时将设计输出为二进制设计数据库，或以 ASCII 文件形式输出（Verilog、DEF 与时序约束）。

Fusion Compiler 还提供统一的：

- 数据模型（**data model**）：使库、设计数据、约束与设计意图可在整条实现流程中共享，并面向超大规模设计、以较小内存占用为目标进行架构；
- 图形用户界面（**GUI**）：支持时序路径、功耗意图、原理图与版图的无缝可视化分析，并在 RTL 与网表之间提供完整交叉探测（**cross-probing**）；
- 设计不匹配缓解基础设施（**design mismatch mitigation**）：提供用户可控手段，以应对不完整或不匹配的库与设计数据。

本章包括以下主题：

- 物理综合设计流程概览
- 形式验证 (Formal Verification)
- 布局布线设计流程概览
- Fusion Compiler 相关概念
- 使用 3DIC Compiler 用户界面
- 输入 `fc_shell` 命令
- 使用应用选项 (application options)
- 使用变量
- 查看 man 页
- 使用 Tcl 脚本
- 通过检查点 (checkpoints) 向脚本添加变更
- 使用 setup 文件
- 使用命令日志文件
- 启用多核处理

参见

有关在 Fusion Compiler 中处理设计数据，请参阅 *Fusion Compiler Data Model User Guide*。

1.1 物理综合设计流程概览

Physical Synthesis Design Flow Overview

图 1.1 给出基本的物理综合设计流程。

1.2 形式验证

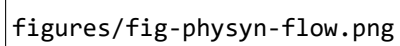
Formal Verification

Formality 工具使用形式化技术证明或否定两个设计的功能等价性，可执行 RTL-to-RTL、RTL-to-gate 与 gate-to-gate 验证。功能等价性检查不考虑静态时序验证。从 Fusion Compiler 到 Formality 的典型数据流包括：综合侧的 RTL/netlist、参考库 (NDM)、工艺数据、TLU+、UPF、可选 DEF/Tcl、可选 SDC，以及 SVF 文件；再配合 Design (NDM)、Verilog、UPF 等输入 Formality。

1.3 布局布线设计流程概览

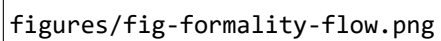
Place and Route Design Flow Overview

图 1.3 给出使用 Fusion Compiler 的基本布局布线设计流程。



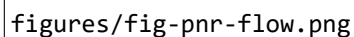
figures/fig-physyn-flow.png

图 1.1: Fusion Compiler 物理综合流程



figures/fig-formality-flow.png

图 1.2: Fusion Compiler 到 Formality 的形式验证数据流



figures/fig-pnr-flow.png

图 1.3: Fusion Compiler 布局布线流程

运行 Fusion Compiler 布局布线流程的步骤如下：

1. 设置库并准备设计数据，见第 二 章「准备设计」。

2. 执行设计规划与电源规划。

此时创建 floorplan，以确定设计尺寸、创建边界与 core 区域、为标准单元布局创建 site rows、设置 I/O pad，并创建电源计划。

更多信息见 *Fusion Compiler Design Planning User Guide*。

3. 执行布局与优化。

使用 place_opt 命令。该命令面向并解决设计的时序收敛：通过增强的布局与综合技术，迭代生成叶单元（leaf cells）的合法化布局与优化后的设计。还可补充功耗优化、布局面积回收、降低拥塞，以及最小化时序与设计规则违例。

4. 执行时钟树综合与优化。

使用 clock_opt 命令。Fusion Compiler 的 CTS 与内嵌优化可处理复杂时钟树问题（如 blockage 规避、布线前与布线后数据相关性等）。时钟树优化通过缓冲器/门的尺寸调整与搬移、电平调整、重构、插入延时、插入 dummy load，以及平衡时钟间延时等方式，改善 skew 与 insertion delay。详见第 五 章。

5. 执行布线与布线后优化，见第 六 章。

工具使用 Zroute 进行全局布线、track assignment、详细布线、拓扑优化与 ECO 布线。布线后优化使用 route_opt。对多数设计，默认设置即可；必要时可优化布线模式、降低串扰，或按特殊需求定制。

6. 执行芯片收尾与可制造性设计（DFM），见第 七 章。

可在流程各阶段应用芯片收尾、DFM 与可制造性（DFY）能力，以应对制造中的工艺设计问题。

7. 保存设计。

1.4 Fusion Compiler 相关概念

Fusion Compiler Concepts

本节介绍工具中的以下概念：

- 功耗意图（Power Intent）概念
- UPF 流程
- 多重图案化（Multiple-Patterning）概念

1.4.1 功耗意图概念

Power Intent Concepts

UPF 语言定义了一组命令，用于规定电子系统的低功耗设计意图。通过 UPF，可描述供电网络、开关、隔离、保持（retention）以及与芯片功耗管理相关的其他方面。同一套低功耗规范应贯穿设计、分析、验证与实现流程。Synopsys 工具遵循正式的 IEEE 1801 UPF 标准。

UPF 规定设计的功耗需求，但不显式规定如何实现这些需求：它说明如何为各设计元素创建供电网络、供电网络之间的行为关系，以及如何扩展逻辑功能以支持对设计元素的动态电源开关；不包含布局或布线信息。

在 UPF 中，**power domain**（电源域）是逻辑层次中共享一组供电需求的元素集合。默认情况下，域内所有逻辑元素使用相同的主供电（**primary supply**）与主地（**primary ground**）；也可为域定义其他电源。电源域在物理版图上通常实现为连续的 **voltage area**，但这并非语言强制要求。

每个电源域有 **scope**（作用域）与 **extent**（范围）：**scope** 是被指定为该域根的逻辑层次；**extent** 是属于该域、共享相同供电需求的逻辑元素集合。**scope** 是定义域的层次，且是域内元素的祖先；**extent** 则是实际属于该域的元素集合。

每一 **scope**（层次）都有 **supply net**（供电网络）与 **supply port**（供电端口）。**supply net** 在给定电源域中承载供电电压或地；跨越多个电源域的 **supply net** 称为在多个域中「复用」。**supply port** 是相邻层次（父/子模块）之间的供电连接点；跨层次的 **supply net** 必须经过 **supply port**。

Supply set（供电网集合）是供电网络的抽象集合，包含 **power** 与 **ground** 两个供电功能。**Supply set** 与域无关：在创建该 **supply set** 的 **scope** 内，其 **power** 与 **ground** 可供任何电源域使用；但各电源域也可限制其对本域内 **supply set** 的使用。可在 RTL 级用 **supply set** 定义功耗意图，从而在尚不知实际 **supply net** 名称时即可综合设计。在物理实现（布局布线）之前，必须对 **supply set** 进行 **refine**（细化），即与实际 **supply net** 关联。

Supply set handle 是为电源域创建的抽象 **supply set**。默认情况下，电源域具有主供电、默认隔离供电与默认保持供电的 **supply set handle**。借助这些 **handle**，即使尚未为该域创建任何 **supply set**、**supply net** 与 **supply port**，也可综合设计。在物理实现前，须将 **supply set handle** **refine** 到实际 **supply set**，再将 **supply set** **refine** 到实际 **supply net**。

Power switch（电源开关）用于接通/关断某条 **supply net** 的供电，具有输入供电网、可开关的输出供电网，以及至少一个控制开关的输入信号；也可有多个控制输入与一个或多个应答输出。**Power state table** 列出设计中各电源域允许的电压值与开关状态组合。

当逻辑信号从一个电源域进入另一个电压明显不同的电源域时，必须插入 **level shifter**（电平移位单元）以转换摆幅。当逻辑信号离开可开关电源域并进入另一电源域时，必须插入 **isolation cell**（隔离单元），以便在关机期间产生确定逻辑值。若两域电压差异显著，接口单元需在上电时做电平移位、在关机时做隔离；能同时完成两者的单元称为 **enable level shifter**。

在具有电源开关的域中，关机期间需保持数据的寄存器必须实现为 **retention register**（保持寄存器），其具有独立的 **always-on** 供电（有时称 **backup supply**），在域主供电关闭时保持数据稳定。

UPF 流程

UPF Flows

Fusion Compiler 同时支持传统 UPF 流程与 golden UPF 流程。Golden UPF 是维护多电压功耗意图的可选方法：在综合、物理实现与验证全过程中使用原始「golden」UPF 文件，并配合 Fusion Compiler 生成的 supplemental UPF 文件。

figures/fig-upf-flows.png

图 1.4: 传统 UPF 与 Golden UPF 流程

Golden UPF 流程在整条链路中保持并使用同一份原始 golden UPF；工具将功耗意图变更写入单独的 supplemental UPF。下游与验证工具使用 golden + supplemental 的组合，而不是单一的 UPF' 或 UPF'' 文件。

其优势包括：

- golden UPF 在流程中保持不变，保留原始结构、注释与通配命名；
- 可使用工具相关条件语句在不同工具中执行不同任务（传统 UPF-prime 流程中会丢失）；
- 功耗意图变更易于在 supplemental 文件中追踪；
- 可选地用 Verilog 网表存储全部 PG 连接信息，从而不必在 UPF 中使用 `connect_supply_net`，显著简化并缩小 UPF。

在加载 UPF 之前，用下列应用选项启用 golden UPF 流程：

```
fc_shell> set_app_options -as_user_default \
-list {mv.upf.enable_golden_upf true}
```

加载 supplemental UPF 时，对 `load_upf` 使用 `-supplemental` 选项。更多信息见 SolvNet 文章 1412864, *Golden UPF Flow Application Note*。

1.4.2 多重图案化概念

Multiple-Patterning Concepts

在 20 nm 及以下工艺节点，现有光刻工具难以印制所需几何图形。为此采用 多重图案化：将版图掩模划分为两张或更多掩模，以提高制造间距、分辨率与可印制性。图 1.5 给出双图案化示例 (MASK A / MASK B)。



图 1.5: 双图案化示例

使用多重图案化时，必须能将版图分解为满足多重图案化间距要求的两张或多张掩模。若某区域中相邻图形对数为奇数，且每对间距均小于多重图案化最小间距，则出现违例，称为 **odd cycle** (奇环)，见图 1.6。

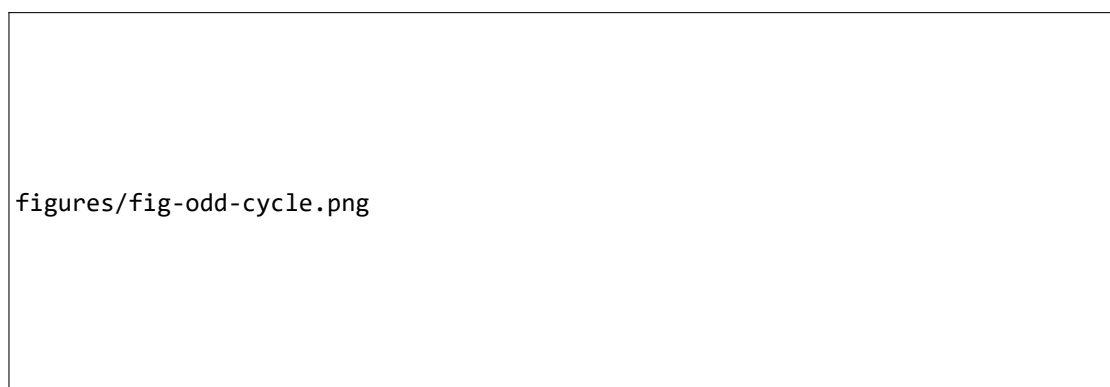


图 1.6: 奇环违例

若环中任一对间距大于多重图案化最小间距，则无违例，版图可分解（例如图 1.7 中 B 与 C 间

距 x 足够大)。

figures/fig-no-odd-cycle.png

图 1.7: 无奇环违例

Fusion Compiler 在布局与布线中考虑多重图案化间距要求并防止奇环，使版图利于双图案化。一般仅在最底层金属（multiple-patterning layers）上执行双图案化；这些层上的金属形状（无论布线还是标准单元/宏内金属）须满足更严格间距；其他层不必。

多重图案化影响布局布线全流程。根据标准单元库，可走 **uncolored** 或 **precolored** 流程：

- **Uncolored** 流程：单元到边界有足够间距，布局时不会产生多重图案化违例。此类库称 correct-by-construction library（多数多重图案化库属此类）。工具为 pin 与 net shape 决定合适的 mask 设置。
- **Precolored** 流程：单元内部金属形状已分配 mask（常称 color），此类库称 precolored library。工具必须考虑这些分配，避免布局违例，并据此决定 pin/net 的 mask。Mask 分配在工具中表现为 **mask constraints**，须在开始布局布线前正确设置。

掩模约束

Mask Constraints

Mask constraints 表示采用多重图案化技术的 block 中物理 pin 与 net 金属形状的掩模要求，驱动布局与布线以保证版图合规。

注意

Mask constraints 仅用于 precolored 流程；uncolored 流程不需要。

可对时序关键 net（net shapes、routing rules、routing blockages）以及 via 设置 mask constraints。对 net，约束定义在 mask_constraint 属性；对 via，定义在 lower_mask、upper_mask、cut_mask。支持取值见表 1.1。

1.5 使用 3DIC Compiler 用户界面

Working With the 3DIC Compiler User Interfaces

Fusion Compiler 在 Linux 上的 X Window 环境中运行，同时提供 shell 命令行界面（CLI）与 GUI。会话期间 CLI 始终可用；可在 shell 或 GUI 中启动/退出会话，也可在会话中打开或关闭 GUI。

表 1.1: Mask Constraint 取值

属性值	说明
same_mask	掩模颜色尚未确定。带此属性的形状与任意已着色金属形状之间至少须满足多重图案化最小间距。
mask_one	形状为 mask1 色；与其他 mask1 形状之间至少须满足多重图案化最小间距。
mask_two	形状为 mask2 色；与其他 mask2 形状之间至少须满足多重图案化最小间距。
mask_three	形状为 mask3 色；与其他 mask3 形状之间至少须满足多重图案化最小间距。
any_mask	形状未着色；仅需满足标准最小间距，多重图案化最小间距规则不适用。

工具使用 Tcl（EDA 业界广泛采用）。可用 Tcl 通过可复用过程与脚本扩展 `fc_shell` 命令语言（见 *Using Tcl With Synopsys Tools*）。

本节说明如何用命令行启动与退出工具：

- 启动 CLI
- 退出 Fusion Compiler

GUI 用法见 *Fusion Compiler Graphical User Interface User Guide*。另见：启动 GUI、关闭 GUI。

1.5.1 启动 CLI

Starting the CLI

CLI 是纯文本环境，在提示符下输入命令，常用于脚本、批处理与一键流程。启动前请确保 `bin` 目录已在 `$PATH` 中。

启动方式：在 Linux shell 中输入

```
% fc_shell
```

启动时可附加选项，例如：

- `-file script_file_name`: 执行脚本
- `-x command`: 执行一条命令
- `-output_log_file file_name`: 创建会话日志
- `-help`: 列出可用选项（不启动 shell）

启动时工具依次：

1. 创建命令日志文件；
2. 读取并执行 `setup` 文件；
3. 执行命令行 `-file/-x` 指定的脚本或命令；
4. 显示程序头与 `fc_shell>` 提示符。

参见

使用命令日志文件；使用 `setup` 文件；使用 Tcl 脚本。

1.5.2 退出工具

Exiting the Fusion Compiler Tool

可用 `exit` 或 `quit` 随时结束会话。

注意

从命令行退出时，工具不会保存已打开的 block。

1.6 输入 `fc_shell` 命令

Entering `fc_shell` Commands

与工具交互使用基于 Tcl 并带有 Fusion Compiler 扩展的 `fc_shell` 命令。命令语言提供类似 Linux shell 的变量、条件执行与流程控制。你可以：

- 在 `fc_shell>` 提示符下交互输入；
- 在 GUI 控制台命令行交互输入；
- 运行包含 `fc_shell` 命令的 Tcl 脚本。

输入命令、选项或文件名时，可在字符足以唯一确定名称后按 `Tab` 自动补全；若有多个候选，工具会列出并由方向键与 `Enter` 选择。

若需复用命令行输出中的内容，可选中后移到命令行并用鼠标中键粘贴。

运行命令时，输出（含处理信息、警告与错误）会回显到 `fc_shell`；若 GUI 打开，也会出现在控制台日志视图。默认非分页模式，输出可能滚动。要启用分页，将 `sh_enable_page_mode` 设为 `true`。

1.6.1 中断或终止命令处理

Interrupting or Terminating Command Processing

若选项或命令有误，可中断处理并留在 `fc_shell` 中：按 `Ctrl+C`。

某些命令/进程不可中断，须在系统级终止 `fc_shell`；终止进程或 shell 时不保存任何数据。

使用 `Ctrl+C` 时注意：

- 若正在处理脚本且中断其中一条命令，脚本处理被中断，后续脚本命令不再执行；
- 若在命令响应中断前连续按三次 `Ctrl+C`，`fc_shell` 会被中断并以如下信息退出：

Information: Process terminated by interrupt.

1.6.2 获取命令信息

Getting Information About Commands

在线资源包括：

- 命令帮助（`command help`）
- Man pages

显示命令帮助

Displaying Command Help

命令帮助可以是命令的简要说明，或其所支持选项与参数的列表。

- 简要说明：`help` 后跟命令名，例如

```
fc_shell> help report_timing
```

- 选项列表：命令名加 `-help`，例如

```
fc_shell> report_timing -help
```

1.7 使用应用选项

Using Application Options

应用选项（application options）控制工具行为，命名约定为：

`category[.subcategory].option_name`

其中 `category` 为受影响引擎名；部分类别还有子类别。

作用域分为：

- **Block** 作用域：仅作用于设置所在的 `block`，保存在设计库中，跨会话持久。
- **Global** 作用域：作用于所有 `block`，但仅在当前会话有效，不写入设计库；每个 `fc_shell` 会话都需重新指定。可将全局设置加入 `.synopsys_fc.setup`。

列出可用应用选项：`get_app_options`（默认可列出全部；可提供 `pattern` 限制）。

```
fc_shell> get_app_options
fc_shell> get_app_options timer.*
```

生成报告：`report_app_options`。详细说明见 *Fusion Compiler Data Model User Guide* 中的 Application Options。

参见

使用 `setup` 文件。

1.8 使用变量

Using Variables

一般而言，Fusion Compiler 通过应用选项而非应用变量修改默认行为；但仍支持用户定义的 Tcl 变量，以及少量应用变量（如 `search_path`）。

列出变量及其值：`printvar`。

```
fc_shell> printvar *
fc_shell> printvar search_path
```

参见

定义搜索路径（Defining the Search Path）。

1.9 查看 Man 页

Viewing Man Pages

显示命令或应用选项的 man 页：man 后跟名称。例如：

```
fc_shell> man report_timing
```

也可查看应用选项摘要页：

- 类别摘要：man category_options，例如

```
fc_shell> man timer_options
```

- 子类别摘要：man category.subcategory_options，例如

```
fc_shell> man route.common_options
```

- 命令相关摘要：man command_options，例如

```
fc_shell> man report_timing_options
```

在 fc_shell 中输入 man 时，man 页显示在 shell 中（GUI 打开时亦在控制台日志）；在 GUI 控制台输入时，显示在 GUI man 页查看器中。

1.10 使用 Tcl 脚本

Using Tcl Scripts

可用 Tcl 脚本完成例行、重复或复杂任务：将 Fusion Compiler 命令序列放入文本文件即可；任何 Fusion Compiler 命令都可在脚本中执行。

Tcl 中以 # 开头的行为注释，例如：

```
# This is a comment
```

编写脚本详见 *Using Tcl With Synopsys Tools*。运行脚本的方式：

- 启动时使用 fc_shell -file;
- 在命令行使用 source;
- GUI 中选择 File > Execute Script。

命令出错时工具会抛出 TCL_ERROR 并立即停止脚本。若要容忍错误并继续执行，可：

- 对可能出错的命令用 Tcl catch 检查 TCL_ERROR;
- 或在脚本中将 sh_continue_on_error 设为 true。

参见

启动 CLI；通过检查点向脚本添加变更。

1.11 通过检查点向脚本添加变更

Adding Changes to a Script With Checkpoints

在 block 上做实验时，可能以两种方式修改 golden flow 脚本：

- 流程变更：设置应用选项、修改约束或其他改进 QoR 的改动；
- 生成报告：增加报告或在流程新位置报告，便于调试。

多数流程通过少量静态可改文件施加变更。典型 place_opt 流程示例：source pre_place_opt_settings.tcl 做流程改动，末尾 source generate_reports.tcl 统一报告：

```
open_block ./design:init_design
source ./scripts/pre_place_opt_settings.tcl
remove_buffer_trees -all
place_opt -from initial_place -to initial_drc
create_placement -incremental -timing_driven -congestion
place_opt -from initial_drc -to initial_opto
place_opt -from final_place -to final_opto
source ./scripts/generate_reports.tcl
```

复杂改动有时需要直接改 golden 脚本，这在许多设计环境中不被鼓励或很难做，尤其当需在多阶段、多 block 或多组实验中实施时。

检查点（**checkpoint**）系统可简化该过程，使你无需修改 golden flow 脚本即可实验：在 golden flow 的重要步骤处定义 checkpoint，包裹相关命令。默认仅运行所包裹代码——单纯插入 checkpoint 不会改变流程。但可将 checkpoint 与「在包裹代码之前 / 之后 / 替代」执行的流程变更或报告关联；这些关联定义在可移植配置文件中，从而把 golden 脚本与某次运行的变更隔离。

同一配置文件可复制到各设计的 run 目录，从而在不修改各设计 golden 脚本的情况下统一施加变更。多用户项目中，每人可有独立的 checkpoint.config.tcl。

检查点系统还支持：

- 比非 checkpoint 脚本更精确地插入流程变更与报告；
- 以单一真相来源查看对默认 golden flow 的全部变更；
- 报告已运行 checkpoint 的历史，含当时执行的变更/报告；
- 报告各 checkpoint 的 runtime 与 memory；
- 生成唯一且描述性的报告名。

一般步骤：

1. 在脚本中插入 checkpoints；
2. 配置：定义要关联的流程变更/报告，并与 checkpoints 关联；
3. 运行脚本。

1.11.1 定义检查点

Defining Checkpoints

使用 `eval_checkpoint`，指定唯一名称并包裹代码块。

注意：

- 若新定义名称与已有 `checkpoint` 重名，运行遇到时工具会自动加唯一后缀（如 `placement` → `placement2`）；
- 不要嵌套 `checkpoint`：工具虽不阻止定义，但运行时忽略嵌套 `checkpoint`，只执行最外层（嵌套内的 Tcl 仍会执行，只是仿佛没有 `checkpoint`）。

示例：为 golden flow 各主要步骤包裹 `checkpoint`：

```
open_block ./design.nlib:init_design
source ./scripts/pre_place_opt_settings.tcl

eval_checkpoint remove_buffers {
    remove_buffer_trees -all
}

eval_checkpoint place_opt_to_initial_drc {
    place_opt -from initial_place -to initial_drc
}

eval_checkpoint incr_placement {
    create_placement -incremental -timing_driven -congestion
}

eval_checkpoint place_opt_to_initial_opto {
    place_opt -from initial_drc -to initial_opto
}

eval_checkpoint place_opt_to_final_opto {
    place_opt -from final_place -to final_opto
}
```

默认仅运行包裹代码；关联行为见下一小节。

1.11.2 配置检查点

Configuring Checkpoints

要使 `checkpoint` 影响流程，须将其与「之前 / 之后 / 替代」执行的流程变更或报告关联。

定义检查点行为

Defining Checkpoint Behaviors

可定义两类行为：**reports** 与 **actions**。Report 用于生成工具支持的报告或自定义报告；Action 用于在 `checkpoint` 包裹代码之前、之后或替代执行任意流程变更（例如在某 `checkpoint` 前设置应用选

项)。

这些行为定义在检查点系统自动 source 的 `checkpoint.config.tcl` 中。

定义 Checkpoint Reports 使用 `create_checkpoint_report`，指定唯一名称与生成报告的 Tcl:

```
create_checkpoint_report timing {
    set name [get_current_checkpoint -name]
    set pos  [get_current_checkpoint -position]

    report_qor -nosplit > ./checkpoint/$name.$pos.qor.rpt
    report_qor -summary -nosplit >> ./checkpoint/$name.$pos.qor.rpt

    report_timing -nosplit -max_paths 10 \
        > ./checkpoint/$name.$pos.path.rpt
}
```

注意用 `get_current_checkpoint -name/-position` 为报告命名：触发的 checkpoint 名以及 before/after 位置会体现在文件名中（例如 `checkpoint_A.after.qor.rpt`）。

再如写出全部非默认应用选项：

```
create_checkpoint_report app_options {
    set name [get_current_checkpoint -name]
    set pos  [get_current_checkpoint -position]
    report_app_options -non_default \
        > ./checkpoint/$name.$pos.app_options.rpt
}
```

定义 Checkpoint Actions 使用 `create_checkpoint_action`。例如启用基于全局布线的 buffering:

```
create_checkpoint_action gropto {
    set_app_options -name place_opt.initial_drc.global_route_based \
        -value true
}
```

高努力拥塞缓解示例（用 `get_current_checkpoint -script` 取回原包裹命令并追加选项）：

```
create_checkpoint_action placer_high_effort_cong {
    set placer_command [get_current_checkpoint -script]
    set cong_option "-congestion_effort high"
    eval $placer_command $cong_option
}
```

关联检查点与行为

Associating Checkpoints and Checkpoint Behaviors

定义 checkpoint 与 behavior 后，须用关联命令使运行遇到 checkpoint 时执行行为。

关联 **Reports** 使用 `associate_checkpoint_report`:

- `-enable`: 报告名;
- `-before` / `-after`: 在指定 `checkpoint` 之前/之后运行;
- 多个 `checkpoint` 可列名或用 `*` 通配。

示例:

```
associate_checkpoint_report -enable timing \
    -after { place_opt_to_initial_opto place_opt_to_final_opto }

associate_checkpoint_report -enable app_options -before *
```

关联 **Actions** 使用 `associate_checkpoint_action`:

- `-enable`: action 名;
- `-before` / `-after` / `-replace`: 之前、之后或替代包裹代码。

示例:

```
associate_checkpoint_action -enable gropto \
    -before place_opt_to_initial_drc

associate_checkpoint_action -enable placer_high_effort_cong \
    -replace incr_placement
```

同一 **Checkpoint** 关联多个行为时的执行顺序

1. 配置为 `before` 的 `reports`;
2. 配置为 `before` 的 `actions`;
3. `checkpoint` 包裹代码, 或配置为 `replace` 的 `actions`;
4. 配置为 `after` 的 `actions`;
5. 配置为 `after` 的 `reports`。

同一位置有多个 `report/action` 时, 按 `associate_checkpoint_*` 关联的先后顺序执行。

1.11.3 查询检查点与行为

Querying Checkpoints and Checkpoint Behaviors

使用 `get_checkpoint_data`:

```
fc_shell> get_checkpoint_data -list_names
fc_shell> get_checkpoint_data -name place_opt_to_initial_drc
fc_shell> get_checkpoint_data -list_reports
fc_shell> get_checkpoint_data -list_actions
fc_shell> get_checkpoint_data -report app_options
```

1.11.4 查看检查点历史

Viewing Your Checkpoint History

进入 checkpoint 目录：其中 checkpoint_history.rpt 记录每次运行遇到的 checkpoint 及 run-time/memory；该目录也存放写出的 checkpoint reports。

清除历史：

- `reset_checkpoints`：清空整个 checkpoint 目录（含报告）；清空前会在 run 目录保存带时间戳的副本；
- `reset_checkpoints -from <name>`：从指定 checkpoint 起（含）从历史中移除后续条目，但不删除已写出的报告文件。

1.12 使用 Setup 文件

Using Setup Files

启动时工具自动执行 `.synopsys_fc.setup` 中的命令。会在家目录与项目目录（启动时的当前工作目录）中查找，读取顺序为：

1. 家目录中的 `.synopsys_fc.setup`
2. 项目目录中的 `.synopsys_fc.setup`

Setup 文件可包含初始化应用选项、设置 GUI 选项等基本任务。例如：

- 在家目录 setup 中设置定义工作环境的应用选项；
- 在设计目录 setup 中设置影响某 block 处理的项目/block 专用选项。

1.13 使用命令日志文件

Using the Command Log File

命令日志记录工具处理的命令，包括 setup 文件命令与应用选项设置。默认写入调用 `fc_shell` 的目录下的 `fc_command.log`。

可在 `.synopsys_fc.setup` 中设置 `sh_command_log_file` 修改变量名；须在启动工具之前修改。若 setup 未含该变量，工具自动创建 `fc_command.log`。

每次会话会覆盖命令日志；要保存请移动或重命名。可用于：生成某实现策略的脚本、记录物理实现过程、记录遇到的问题。

1.14 启用多核处理

Enabling Multicore Processing

若干功能支持多核处理：多线程、分布式处理或并行命令执行。并行可显著缩短周转时间。

注意

单台机器有一个或多个 CPU，每个 CPU 有一个或多个核；可用总核数 = CPU 数 × 每 CPU 核数。

使用多核时，每 16 个并行任务需要 1 个 Fusion Compiler license（例如 17 个并行任务需 2 个 license）。

多数情况下用 `set_host_options` 配置。相关主题：配置多线程、配置分布式处理、报告配置、移除配置；层次设计上对多 block 并行跑同一任务；在本地主机并行运行检查/报告命令。

1.14.1 配置多线程**Configuring Multithreading**

多线程在同一机器的多核上并行，共享单一进程内存映像；每个并行任务称为 thread。为最佳性能，线程数应不超过可用核数。

支持由 `set_host_options` 配置多线程的命令包括：`place_opt`、`clock_opt`、启用 advanced legalizer 时的 `check_legality`、`insert_via_ladders`、`route_auto`/`route_global`/`route_track`/`route_detail`/`route_eco`、`route_opt`、`signoff_check_drc`、`signoff_fix_drc`、`signoff_create_metal_fill`、`signoff_fix_isolated_via`、`write_def` 等。

注意

单线程布线结果确定；多线程时因任务划分不同，布线结果不确定。全局布线支持确定性多核模式：将 `route.global.deterministic` 设为 on。

默认所有命令单线程。对支持多线程的命令，将 `set_host_options -max_cores` 设为大于 1 且不超过可用核数的值；该值作用于所有支持多线程的命令。

启用后，即使超过可用核数，命令仍会创建并使用指定线程数——过度订阅会降低性能。最佳实践：每可用核不超过一个线程。例如双 CPU、每 CPU 三核：

```
fc_shell> set_host_options -max_cores 6
```

1.14.2 配置分布式处理**Configuring Distributed Processing**

分布式处理使用多台机器；每个进程有独立内存映像。每个并行任务称为 process。可用进程数不应超过各主机核数之和。

支持分布式的命令包括：`analyze_rail`、`create_placement -floorplan`、`signoff_check_drc`、`signoff_fix_drc`、`signoff_create_metal_fill`、`signoff_fix_isolated_via` 等。

可指定：作业提交命令（`-submit_command`，默认 `rsh`）、主机列表、最大进程数（`-num_processes`）。可用 `-name` 命名配置以便选用或删除。例如用 `qsub`：

```
fc_shell> set_host_options -name dp_config \
-submit_command [list qsub -P bnormal -cwd]
```

1.14.3 报告与移除多核配置

Reporting / Removing Multicore Configurations

报告: `report_host_options`。移除: `remove_host_options` (`-all` 或 `-name`)。

1.14.4 并行运行任务

Running Tasks in Parallel

要对设计中多个 block 高效执行同一任务, 可用 `run_block_script`: 接受 Tcl 脚本并应用到指定 block。例如, 在层次化流程中并行编译子 block:

1. 编写在各 block 上运行的脚本(可用 `compile_fusion` 分阶段编译, 并 `create_abstract/create_frame/save_block` 等);
2. 用 `set_host_options` 配置分布式处理, 再对 block 列表调用 `run_block_script` (可指定 `-work_dir` 存放数据与日志)。

用 `-run_order top_down` 或 `bottom-up` 控制处理顺序: `top-down` 时, 子 block 须等全部父 block 处理完后再处理; `bottom-up` 时先处理子 block 再处理父 block。默认采用 `bottom-up`。

1.14.5 在本地主机并行运行命令

Running Commands in Parallel on Your Local Host

可通过后台 (`redirect -bg`) 或前台并行 (`parallel_execute`) 运行检查与报告命令, 以缩短 runtime。这些技术不要求超出父进程所需的额外 license, 也不要求配置分布式处理。

建议:

- 并行前先跑 `update_timing`, 否则各需更新时序的命令会各自独立跑;
- 子进程变量传回父进程时, 须在子进程写入文件, 再由父进程读取。

后台运行命令

Running Commands in the Background

使用 `redirect -bg: fc_shell` 立即返回执行下一条命令。父进程 `exit` 时会等待所有 `redirect -bg` 完成。

用 `list_commands -bg` 查看支持的命令。可跑单条命令或仅含支持命令的 Tcl; 脚本内若再含 `redirect -bg`, 其 `-bg` 会被忽略。最多同时两个后台任务, 超出则排队。

用 `redirect -max_cores` 指定后台可用核数; 父进程可用核数 (由 `set_host_options -max_cores` 指定) 会相应减少。

```
fc_shell> set_host_options -max_cores 8
fc_shell> redirect -bg -max_cores 3 -file bg_log.out \
    {source bg_script.tcl}
```

报告后台任务: `report_background_jobs`; `-reset` 可省略已完成任务。

并行运行命令

Running Commands in Parallel

使用 `parallel_execute`，在并行列表中最长命令完成后返回父进程，适合批处理脚本：

```
fc_shell> update_timing
fc_shell> parallel_execute {
    report_cmd1 log_file1
    report_cmd2 log_file2
    report_cmd3 log_file3
}
```

用 `-list_allowed_commands` 查看支持的命令。

参见

本章后续章节将展开设计准备、布局优化、CTS、布线等实现细节。命令与应用选项以原版 `man` 页及 `SolvNetPlus` 为准。

Chapter 2

准备设计

Preparing the Design

Fusion Compiler 工具使用设计库（design library）来存储设计及其相关库信息。本节说明如何创建设计库，以及如何准备并保存设计。

这些步骤在以下主题中说明：

- 定义搜索路径（Defining the Search Path）
- 设置库（Setting Up Libraries）
- 使用 Pin Access Checker 实用程序
- 分析库（Analyzing Libraries）
- 读取设计（Reading the Design）
- 缓解设计不匹配（Mitigating Design Mismatches）
- 导入 floorplan 信息
- 设置多电压设计（Setting Up Multivoltage Designs）
- 指定时序约束与设置
- 指定逻辑设计规则约束
- 指定时钟门控约束（Specifying Clock Gating Constraints）
- 为布局与合法化指定物理约束
- 指定布局设置
- 指定合法化设置
- 控制单元、net、pin 与 port 的优化
- 指定布线前优化设置
- 为功耗相关功能做准备

- 指定布线资源
- 使用 Early Data Check Manager 处理设计数据
- 应用 Mega-Switch 命令设置

2.1 定义搜索路径

Defining the Search Path

Fusion Compiler 工具使用搜索路径（search path）查找以相对路径指定、或未指定路径的文件。

要用 `set_app_var` 命令将 `search_path` 应用变量设为按顺序查找文件的目录列表。工具查找文件时，从 `search_path` 中最左侧的目录开始，使用找到的第一个匹配文件。

也可用 Tcl 的 `lappend` 命令把目录追加到默认搜索路径（即启动工具时所在目录）。例如：

```
fc_shell> lappend search_path ./mylibdir
```

2.2 设置库

Setting Up Libraries

block（块）是物理与功能设计数据的容器。**design library**（设计库）是一组相关 **block** 的集合，并附带适用于该 **block** 集合的工艺数据。芯片设计由一个或多个 **block** 组成，这些 **block** 常存储在不同设计库中。设计库使用在较低层库中定义的 **block** 实例，这些较低层库称为 **reference libraries**（参考库）。一个设计库也可作为另一设计库的参考库。

关于设置库，见以下主题：

- 处理设计库（Working With Design Libraries）
- 设置参考库（Setting Up Reference Libraries）
- 库配置（Library Configuration）
- 限制库单元使用（Restricting Library Cell Usage）
- 限制所用目标库（Restricting the Target Libraries Used）
- 指定库子集限制（Specifying Library Subset Restrictions）

2.2.1 处理设计库

Working With Design Libraries

可用绝对路径、相对路径或不指定路径，通过下列命令创建、打开、查询、保存或关闭设计库：

- `create_lib`

该命令在内存中创建库，并将其设为当前库。创建新设计库时必须指定库名。库名中不允许使用斜杠（/）与冒号（:）。下列命令用相对路径创建 `my_libA` 库：

```
fc_shell> create_lib ../my_lib_dir/my_libA
{my_libA}
```

- **open_lib**

打开指定库，将其设为当前库，并打开其全部关联参考库。打开库意味着将其载入内存并使其 block 可访问。下例打开磁盘上已保存的 my_libA:

```
fc_shell> open_lib my_libA
Information: Loading library file '/usr/lib/StdCells.ndm' (FILE-007)
Information: Loading library file '/usr/lib/RAMs.ndm' (FILE-007)
Information: Loading library file
'/usr/lib/PhysicalOnly.ndm' (FILE-007)
{my_libA}
```

- **current_lib**

默认情况下，最近打开的库为当前库。可用 **current_lib** 显式将任一已打开库设为当前库。例如:

```
fc_shell> current_lib my_libA
{my_libA}
```

- **save_lib**

创建或修改库时，变更仅保存在内存中。要保存到磁盘，使用本命令。例如:

```
fc_shell> save_lib lib_A
Saving library 'lib_A'
1
```

- **close_lib**

不再需要访问某库中的数据时，可用 **close_lib** 关闭。关闭前请务必保存库中的更改。例如:

```
fc_shell> close_lib
Closing library 'lib_A'
1
```

此外，可用 **current_lib**、**get_libs** 与 **report_lib** 命令查询设计库。

参见

更多信息见 *Fusion Compiler Data Model User Guide* 中的 Design Libraries 主题。

2.2.2 设置参考库

Setting Up Reference Libraries

创建设计库时，可用 **create_lib** 的 **-ref_libs** 选项为其指定参考库列表；也可随时用 **set_ref_libs** 更改参考库列表。

用下列命令指定、重新绑定（rebind）并报告参考库:

- **create_lib -ref_libs**

可用 **create_lib** 指定新设计库与其较低层参考库之间的关系。例如:

```
fc_shell> create_lib lib_B \
-ref_libs {../LIBS/lib_c ../STND/stdhvt.ndm} ...
```

```
{lib_B}
```

- **set_ref_libs -ref_libs**

对已有设计库，先打开库，再用 **set_ref_libs** 指定参考库。例如：

```
fc_shell> current_lib
{lib_B}
fc_shell> set_ref_libs \
    -ref_libs {../LIBS/lib_C ../STND/stdhvt.ndm}
../LIBS/lib_C ../STND/stdhvt.ndm
```

- **report_ref_libs**

用 **report_ref_libs** 报告设计库的参考库。例如：

```
fc_shell> create_lib lib_A -ref_libs \
    {../libs/SCLL.ndm ../libs/SCHH.ndm ../BLOCKS/MACROS}
{lib_A}
fc_shell> report_ref_libs...
Name Path Location
-----
*+ SCLL ../libs/SCLL.ndm /remote/project/libs/SCLL.ndm
*+ SCHH ../libs/SCHH.ndm /remote/project/libs/SCHH.ndm
* MACROS ../BLOCKS/MACROS /remote/project/BLOCKS/MACROS
"*" = Library currently open
"+" = Library has technology information
```

- **set_ref_libs -rebind**

当参考库列表失效时（例如把参考库移到新位置），需要重新绑定参考库。使用 **-rebind** 选项，会把 **search_path** 变量所指定的各参考库路径重新绑定到当前已载入内存的库。例如：

```
fc_shell> current_lib
{lib_A}
fc_shell> set_app_var search_path { . ../REFLIBS ../CLIBS }
. ../LIBS ../BLOCKS
fc_shell> set_ref_libs -rebind
../REFLIBS/lib_C ../REFLIBS/lib_D ../CLIBS/stdhvt.ndm}
```

重新绑定库不会影响设计库中已有 **block** 的绑定。要用更新后的参考库列表重新绑定这些 **block**，对 **link_block** 使用 **-rebind** 选项。

2.2.3 库配置

Library Configuration

库配置允许你指定将哪些厂商库用作当前设计的参考库。通过 **search_path** 与 **link_library** 变量指定技术文件、物理库与逻辑库，再用 **create_lib** 或 **set_ref_libs** 组装单元库。

在库配置过程中：

- Fusion Compiler 工具自动调用 Library Manager，无需用户干预即可生成单元库，如下图所示：



figures/fig-lib-config.png

图 2.1: 库配置中生成单元库

- 工具将生成的单元库保存到磁盘，并加入设计库的参考库列表。
- 这些单元库与在 Library Manager 中进行库准备时所创建的单元库相同。

参见

更多信息见 *Fusion Compiler Data Model User Guide* 中的 **Configuring Cell Libraries** 主题。

2.2.4 限制库单元使用

Restricting Library Cell Usage

默认情况下，对 block 执行优化或时钟树综合时，Fusion Compiler 可使用单元库中全部可用库单元。要限制单元使用，在 block 已载入内存后使用 `set_lib_cell_purpose`。该命令具有 block 作用域，并在工具会话间保持；重新打开 block 时会恢复这些设置。

要用 `-include` 与 `-exclude` 选项指定工具可以使用或不可以使用库单元的方式。这两个选项均可接受下列一个或多个取值：`all`、`cts`、`hold`、`optimization`、`none`。

- `-include` 在指定库单元上设置 `included_purposes` 属性。
- `-exclude` 在指定库单元上设置 `excluded_purposes` 属性。

注意

若库单元带有 `dont_use` 属性，则被排除在所有用途之外，效果等同于对该单元执行 `set_lib_cell_purpose -include none`。

执行优化（含 `setup`、`hold` 与逻辑 DRC 修复）时，工具可使用 `included purpose` 为 `optimization`、`hold` 或二者兼有的库单元。执行时钟树综合时，可使用 `included purpose` 为 `cts` 的库单元。

例如，禁止某组库单元用于所有用途：

```
fc_shell> set_lib_cell_purpose -include none lib_cells
```

仅允许某组库单元用于时钟树综合：

```
fc_shell> set_lib_cell_purpose -include none lib_cells
fc_shell> set_lib_cell_purpose -include cts lib_cells
```

允许某组库单元用于除时钟树综合外的所有用途：

```
fc_shell> set_lib_cell_purpose -exclude cts lib_cells
```

2.2.5 限制所用目标库

Restricting the Target Libraries Used

默认情况下，优化期间工具可从目标库中选择任意库单元。某些情况下，你可能希望限制用于时钟树综合与优化的库单元。例如，希望在某些设计 block 的优化中排除特定双高度（double-height）单元。

要限制用于时钟树综合与优化的库单元：

1. 用 `set_target_library_subset` 指定库子集。

默认情况下，库子集限制适用于：

- 顶层 block 及其全部子 block。要对特定子 block 设置，使用 `-objects` 选项。
- 时钟路径与数据路径二者。要仅对时钟或数据路径设置，分别使用 `-clock` 或 `-data`。

还可进一步限制目标库子集设置：

- 用 `-dont_use` 指定库中不应使用的单元列表。
 - 用 `-only_here` 指定这些库除指定对象外，不得用于任何其他对象。
2. 将 `opt.common.enable_target_library_subset_opt` 应用选项设为 1，以启用目标库子集。

设置目标库子集时请注意：

- 子集限制适用于层次单元，但不适用于叶子单元（leaf cells）。
- 命令对指定 block 及其层次中的子设计强制子集限制，但已设置不同子集限制的子设计除外。
- 较低层指定的子集会覆盖较高层指定的子集。

例如，假设设计逻辑层次如图 2.2，希望在优化与时钟树综合中实现如下库限制：

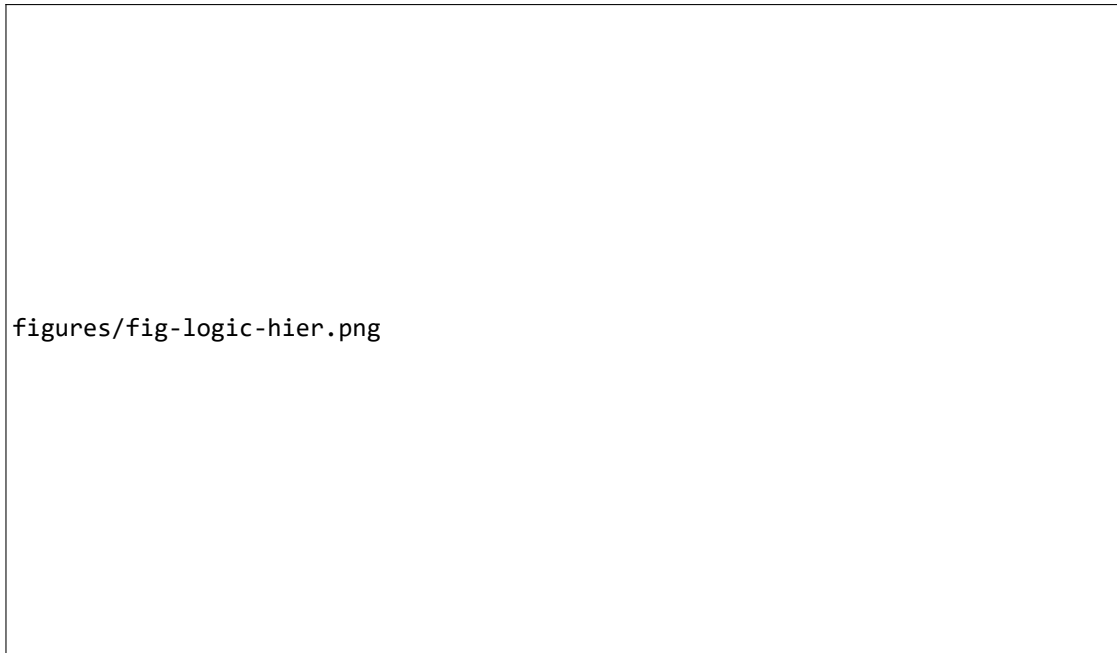
- 对 Sub1 block 及其子 block SubA、SubB，仅使用名为 LVT_lib 的库中的单元。
- 设计其余部分不得使用该库中的单元。

为此使用下列设置：

```
fc_shell> set_target_library_subset -objects {top/Sub1} \
    -only_here [get_lib_cells LVT_lib/*] [get_libs LVT_lib]
fc_shell> set_app_options \
    -name opt.common.enable_target_library_subset_opt -value 1
```

除上述设置外，假设还指定：

```
fc_shell> set_lib_cell_purpose -include cts \
    {HVT_lib/buf1 HVT_lib/buf2 LVT_lib/buf1 LVT_lib/buf2}
```



figures/fig-logic-hier.png

图 2.2: 设计的逻辑层次

则在时钟树综合期间于时钟网络上添加缓冲器时，工具会：

- 对名为 Sub1 的 block 及其子 block，使用 LVT_lib 中的 buf1、buf2；
- 对设计其余部分，使用 HVT_lib 中的 buf1、buf2。

报告目标库子集

Reporting Target Library Subsets

要查明已为顶层 block 或层次单元定义了哪些目标库子集,使用 `report_target_library_subset`。

由 `report_cells`、`report_timing` 等报告命令生成的报告中，对由 `-dont_use` 或 `-only_here` 指定的单元会附带 `td` 属性。

移除目标库子集

Removing Target Library Subsets

要从顶层 block 或层次单元移除目标库子集限制，使用 `remove_target_library_subset`。

2.3 指定库子集限制

Specifying Library Subset Restrictions

可将时序单元（`sequential cells`）以及已例化组合单元（`instantiated combinational cells`）的映射与优化限制到参考库与库单元的某个子集。要指定一个或多个库子集限制，先使用 `define_libcell_subset`，再使用 `set_libcell_subset`。

注意

库子集限制适用于叶子单元（leaf cells），不适用于层次单元（hierarchical cells）。

要在叶子单元上设置库子集限制：

1. 用 `define_libcell_subset` 定义库单元子集。
库单元被分入子集后，不再可用于优化期间的一般映射，而仅用于时序单元与已例化组合单元。
2. 用 `set_libcell_subset` 限制时序单元与已例化组合单元的优化。
优化期间，工具使用步骤 1 中定义的库子集来优化时序单元（已映射与未映射）以及已映射的组合单元。

下例中，`define_libcell_subset` 将 `SDFLOP1` 与 `SDFLOP2` 库单元归入名为 `special_flops` 的子集，随后 `set_libcell_subset` 将叶子单元 `LEAF1` 的映射限制到该 `special_flops` 库子集：

```
fc_shell> define_libcell_subset \  
-libcell_list "SDFLOP1 SDFLOP2" -family_name special_flops  
fc_shell> set_libcell_subset \  
-object_list "HIER1/LEAF1" -family_name special_flops
```

报告库子集限制

Reporting Library Subset Restrictions

要报告为时序单元与已例化组合单元指定的库子集，使用 `report_libcell_subset`。

移除库子集限制

Removing Library Subset Restrictions

要移除库子集，使用 `remove_libcell_subset`。

2.4 使用 Pin Access Checker 实用程序

Using Pin Access Checker Utility

Pin access checker（PAC）实用程序帮助你在库开发阶段识别 pin 可访问性问题。这有助于改进单元版图——库最终确定并发布给布局布线团队后，将无法再修改单元版图。

布线 DRC 会在布线后识别 pin 访问问题。当单元邻接（abutted）或放置过近时，两个或多个相邻 pin 会争夺公共布线资源，从而引起布线 DRC。

PAC 实用程序的好处包括：

- 生成带有多种邻接单元布局排列的布局布线数据库。
- 生成布局布线数据库，并根据报告评估所建库的可布局性与 pin 可访问性。

PAC 执行下列检查：

- 单元布局
- 每个库单元 pin 的可访问 track 数量

- 布局后的合法化检查
- 布线后的设计规则检查
- 上述全部检查（含 PG 布线）

可用 `create_pin_check_lib` 根据输入的技术文件与参考库创建或打开新库。PAC 在该库中创建并存储 block。必须通过下列应用选项与 `create_pin_check_lib` 提供至少三项输入：

- 技术文件：用 `-technology` 提供技术文件信息。
- 参考库：用 `-ref_libs` 提供参考库信息。
- 金属层布线方向：在 Tcl 脚本中用 `set_attribute [get_layers Mx] routing_direction horizontal/vertical` 定义布线方向。必须用 `pin_check.place.preplace_option_file` 应用选项指定该脚本名。

要启用 `pin access check` 实用程序，可使用 `check_libcell_pin_access`。可用该命令与 `pin_check.*` 应用选项启用下列 pin 访问检查：

- Legalization
- Router
- 自动推导单元间距规则（Automatic derivation of cell spacing rule）

对 `check_libcell_pin_access` 使用 `-mode`，通过指定不同 mode 执行多种检查。

```
create_pin_check_lib myTest.nlib -ref_libs ../lib/refLib.ndm \
-technology ../tf/refTech.tf

set_app_options -as_user_default -name \
pin_check.place.preplace_option_file -value

check_libcell_pin_access -mode analyze_lib_cell

check_libcell_pin_access -mode analyze_lib_pin

check_libcell_pin_access -mode design_under_test
```

若此前使用基于 Tcl 的旧版本，可用 `translate_pin_check_ui` 将旧脚本转换为新的 Fusion Compiler 命令：

```
translate_pin_check_ui -from old_script.tcl -to new_script.tcl
```

2.5 分析库

Analyzing Libraries

Fusion Compiler 提供下列库分析能力，有助于达成设计的性能、功耗与面积目标：

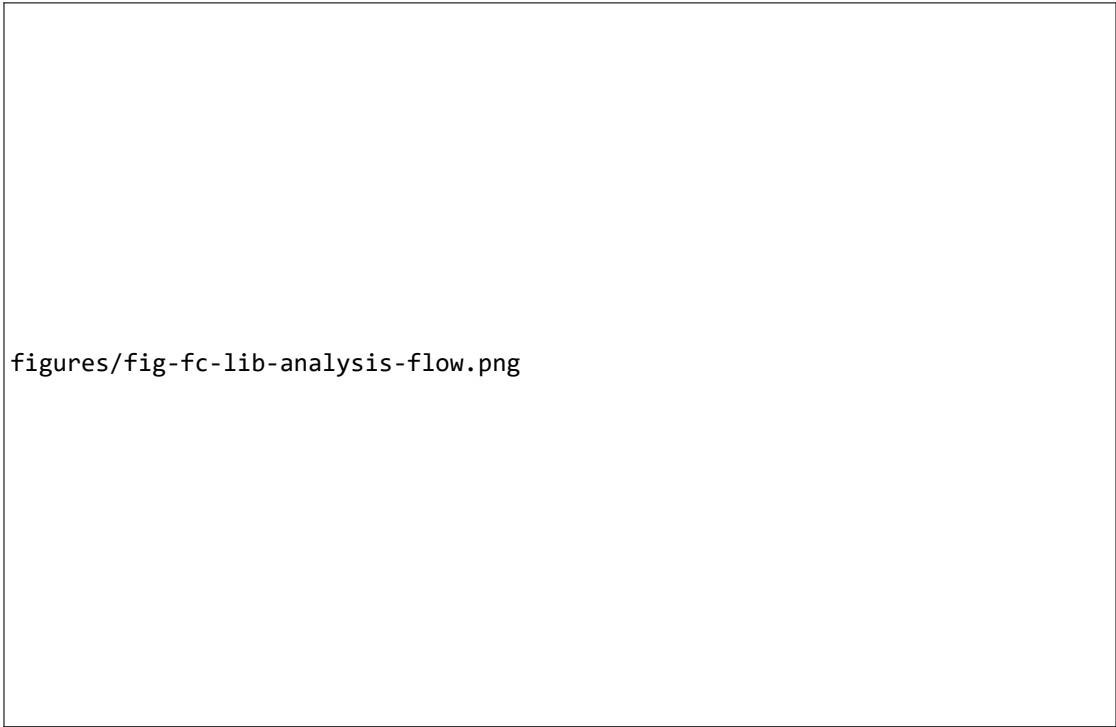
- 识别库单元族（library cell families）中的冗余（redundant）、离群（outlier）与等价（equivalent）单元，可在后续实现流程步骤中将其排除在优化之外。
- 识别库单元族中的缺口（gaps），可作为反馈提供给库供应商，以改进参考库。

- 比较不同库或同一库不同版本中等价单元的延迟、功耗、面积及其他特性，有助于确定适合设计实现的库。

库单元族是共享某些共同属性的单元集合。工具可基于下列方式识别库单元族：

- 用户标识的库单元属性，例如功能（function）、阈值电压（threshold voltage）等；
- 用户定义的库单元词法属性（lexical attributes）与设置；
- 用户指定的自定义库单元族。

下图给出执行库分析的高层流程。



figures/fig-fc-lib-analysis-flow.png

图 2.3: 库分析流程

关于不同类型库分析流程的更多信息，见下列主题：

- 库单元族中冗余、离群、等价与缺口简介
- 识别库单元族中的冗余、离群、等价与缺口
- 比较库

2.5.1 库单元族中冗余、离群、等价与缺口简介

Introduction to Redundancies, Outliers, Equivalences, and Gaps in Library Cell Families

当功能相同的单元族按驱动强度（drive strength）递增排序时，特定单元属性值应单调递增或递减。例如，按驱动强度递增排序时，单元面积应单调递增，功耗应递增，而延迟（在相同输出负载与输入 slew 下）应递减。

下图给出功能相同的单元族中，单元面积相对时序的单调分布。

若某单元的全部度量（如面积、延迟、功耗等）均等于或劣于另一单元，则该单元被视为冗余（redundant），如下图所示（功能相同的单元族中单元面积相对时序的图）。



figures/fig-fc-lib-monotonic.png

图 2.4: 库单元族中的单调分布



figures/fig-fc-lib-redundant.png

图 2.5: 库单元族中的冗余单元

若某单元的度量相对期望值有很大偏差，则该单元被视为离群（outlier），如下图所示（功能相同的单元族中单元面积相对时序的图）。



图 2.6: 库单元族中的离群单元

若单元在所有工艺、电压与温度（process, voltage, and temperature, PVT）取值下全部属性均相同，则这些单元被视为等价（equivalent），如下图所示（功能相同的单元族中单元面积相对时序的图）。

当相邻两个驱动强度之间存在很大差异时，即存在缺口（gap），如下图所示（功能相同的单元族中单元面积相对时序的图）。

2.5.2 识别库单元族中的冗余、离群、等价与缺口

Identifying Redundancies, Outliers, Equivalences, and Gaps in Library Cell Families

要识别单元族中的冗余、离群、等价与缺口，执行下列步骤：

1. 用 `create_libset` 创建待分析库的集合（称为 **libset**），例如：

```
fc_shell> create_libset -name myLibset -libs {abcd1 abcd2}
```

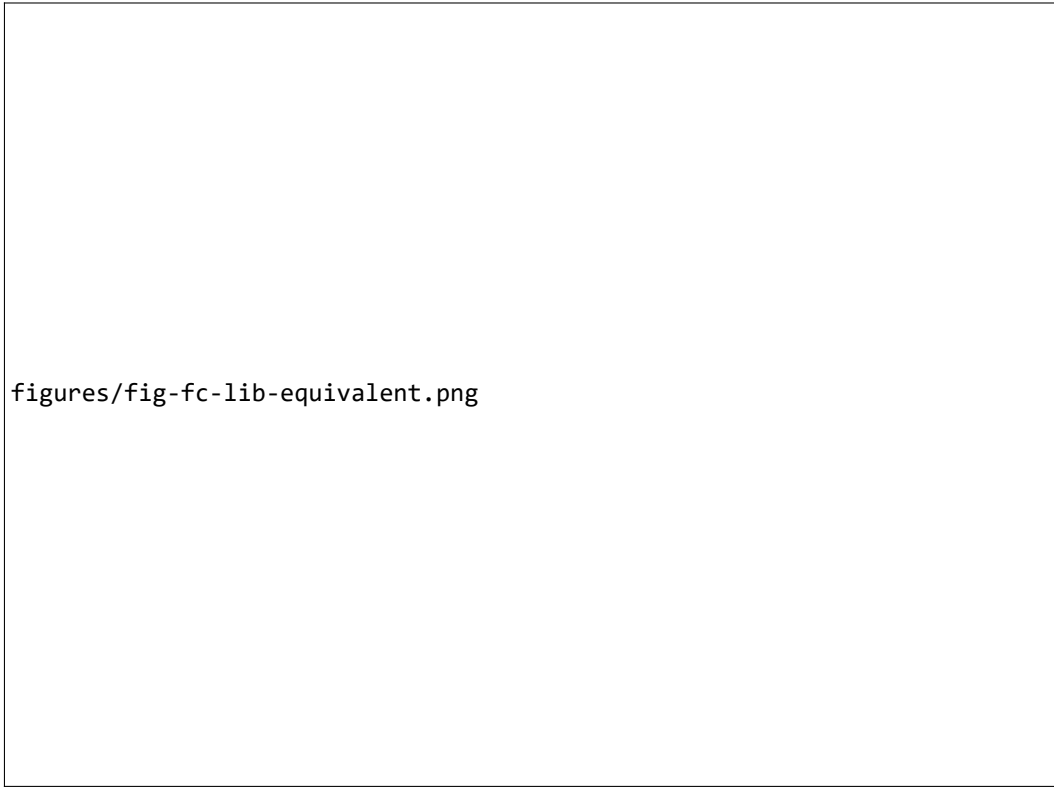
执行库分析时，可指定 **libset** 名，而不必指定库列表。此外，工具在分析期间生成报告、绘图等时也使用 **libset** 名，而非库名列表。

可将同一单元库包含在多个 **libset** 中。

2. 用 `create_environment` 设置库分析环境。

`create_environment` 将预定义的 **libset** 与工艺标签（process label）、工艺编号（process number）、电压与温度关联，随后用于分析库。

可用下列方法之一指定分析环境的工艺标签、工艺编号、电压与温度：



figures/fig-fc-lib-equivalent.png

图 2.7: 库单元族中的等价单元



figures/fig-fc-lib-gap.png

图 2.8: 库单元族中的缺口

- 用 `-ppvt` 直接指定工艺标签、工艺编号、电压与温度，例如：

```
fc_shell> create_environment -name env1 -libset myLibset \
    -ppvt {{SS1p, 1.0, 0.6, 125}}
```

- 用 `-op_conds` 指定参考库中的预定义工作条件，例如：

```
fc_shell> create_environment -name env2 -libset myLibset \
    -op_conds SS1p6125
```

- 用 `-corners` 指定 corner，由工具从中推断工艺标签、工艺编号、电压与温度，例如：

```
fc_shell> create_environment -name env3 -libset myLibset \
    -corners nworst
```

- 用 `-auto_infer_design` 指定由工具从当前设计关联的 scenario 与 UPF 推断工艺标签、工艺编号、电压与温度，例如：

```
fc_shell> create_environment -name env4 -libset myLibset \
    -auto_infer_design
```

3. (可选) 为库单元指定命名约定，并用词法属性执行词法分类 (lexical classification)。

库单元命名约定有助于工具在后续流程步骤中识别单元族。

例如，假定库中某单元名为 `AND2D4WSULVT`。该名称是 `AND2`、`D4`、`WS` 与 `ULVT` 字符串的拼接，分别表示与某词法属性关联的单元特性，如下表所示：

表 2.1: 示例库命名约定的词法属性

列	词法属性	允许模式	示例单元取值
1	<code>lexical_cell_prefix_name</code>	<code>[a-zA-Z0-9]*</code>	<code>AND2</code>
2	<code>lexical_drive1_name</code>	<code>D[0-9]*</code>	<code>D4</code>
3	<code>lexical_well_substrate_bias_architecture</code>	<code>WS</code>	<code>WS</code>
4	<code>lexical_device_threshold_voltage</code>	<code>ULVT SVT LVT MVT</code>	<code>ULVT</code>

可用 `set_lib_cell_naming_convention` 为该库指定命名约定，例如：

```
fc_shell> set_lib_cell_naming_convention -library $lib \
    -column 1 -pattern {[a-zA-Z0-9]*} -attribute \
    "lexical_cell_prefix_name"
fc_shell> set_lib_cell_naming_convention -library $lib \
    -column 2 -pattern {D[0-9]*} -attribute "lexical_drive1_name"
fc_shell> set_lib_cell_naming_convention -library $lib \
    -column 3 -pattern {WS} -attribute \
    "lexical_well_substrate_bias_architecture"
fc_shell> set_lib_cell_naming_convention -library $lib \
    -column 4 -pattern {ULVT|SVT|LVT|MVT} -attribute \
    "lexical_device_threshold_voltage"
```

注意

预定义词法属性的完整列表见 `set_lib_cell_naming_convention` 命令的 man page。

指定命名约定后，用 `classify_lib_cell_attributes` 执行词法分类，例如：

```
fc_shell> classify_lib_cell_attributes -libset myLibset
```

要用 `report_lib_cell_classification_attributes` 报告词法分类结果。

要用 `report_lexical_unclassified_lib_cells` 报告在给定命名约定下未被词法分类的库单元。若存在未分类单元，请补充命名约定并重新运行词法分类。

4. (可选) 用 `set_use_for_library_analysis` 将特定单元排除在库分析之外。

默认情况下，工具在库分析期间排除仅物理单元 (physical-only cells)。但你可排除更多单元，例如带有 `dont_use` 设置的库单元：

```
fc_shell> set exclude_cells [filter_collection [get_lib_cells $libs/*] \
    "dont_use == true"]
fc_shell> set_use_for_library_analysis $exclude_cells false
```

要用 `report_lexical_ignored_lib_cells` 报告被排除的单元。

5. 用 `identify_lib_cell_families` 识别全部库单元族。

要用 `report_lib_cell_families -all_initial` 报告工具识别的库单元族。

要手动定义库单元族，使用 `define_lib_cell_family`。要用 `report_lib_cell_families -all_userdef` 报告这些用户定义的单元族。

6. 用 `run_library_analysis` 分析库，例如：

```
fc_shell> run_library_analysis -name myLibset_run1 -environment env1
```

用 `-environment` 指定的环境名必须对应先前用 `create_environment` 创建的环境。

7. 用 `find_library_redundancies` 识别库单元族中的冗余，例如：

```
fc_shell> find_library_redundancies -analysis myLibset_run1
```

用 `-analysis` 指定的分析名必须对应先前用 `run_library_analysis` 执行的分析。

除冗余单元外，`find_library_redundancies` 还会识别库单元族中的离群单元与等价单元。

8. 用 `find_library_gaps` 识别库单元族中的缺口，例如：

```
fc_shell> find_library_gaps -analysis myLibset_run1
```

用 `-analysis` 指定的分析名必须对应先前用 `run_library_analysis` 执行的分析。

9. 用 `report_library_analysis` 报告库分析期间识别的库单元缺口、冗余、离群与等价，例如：

```
fc_shell> report_library_analysis -analysis myLibset_run1 \
    -analysis_type redundancy -detail
fc_shell> report_library_analysis -analysis myLibset_run1 \
    -analysis_type outlier -detail
fc_shell> report_library_analysis -analysis myLibset_run1 \
```

```
-analysis_type equivalent -detail
fc_shell> report_library_analysis -analysis myLibset_run1 \
-analysis_type gap -detail
```

10. 用 `gui_plot_lib_cells_attributes` 生成绘图，例如：

```
fc_shell> gui_plot_lib_cells_attributes -libset myLibset \
-lib_cell_type all -x_attr_name leakage -y_attr_name \
avgs1_fall_delay \
-chart_type scatter
```

2.5.3 比较库

Comparing Libraries

要比较库，执行下列步骤：

1. 用 `create_libset` 创建待分析库的集合（libset），例如：

```
fc_shell> create_libset -name myLibset -libs {abcd1.db abcd2.db}
```

执行库分析时，可指定 libset 名，而不必指定库列表。此外，工具在分析期间生成报告、绘图等时也使用 libset 名，而非库名列表。

可将同一单元库包含在多个 libset 中。

2. 用 `create_environment` 设置库分析环境。

`create_environment` 将预定义的 libset 与工艺标签、工艺编号、电压与温度关联，随后用于分析库。

可用下列方法之一指定分析环境的工艺标签、工艺编号、电压与温度：

- 用 `-ppvt` 直接指定工艺标签、工艺编号、电压与温度，例如：

```
fc_shell> create_environment -name env1 -libset myLibset \
-ppvt {{SS1p, 1.0, 0.6, 125}}
```

- 用 `-op_conds` 指定参考库中的预定义工作条件，例如：

```
fc_shell> create_environment -name env2 -libset myLibset \
-op_conds SS1p6125
```

- 用 `-corners` 指定 corner，由工具从中推断工艺标签、工艺编号、电压与温度，例如：

```
fc_shell> create_environment -name env3 -libset myLibset \
-corners nworst
```

- 用 `-auto_infer_design` 指定由工具从当前设计关联的 scenario 与 UPF 推断工艺标签、工艺编号、电压与温度，例如：

```
fc_shell> create_environment -name env4 -libset myLibset \
-auto_infer_design
```


3. (可选) 为库单元指定命名约定，并用词法属性执行词法分类。

库单元命名约定有助于工具在后续流程步骤中识别单元族。

例如，假定库中某单元名为 AND2D4WSULVT。该名称是 AND2、D4、WS 与 ULVT 字符串的拼接，分别表示与某词法属性关联的单元特性，如下表所示：

表 2.2: 示例库命名约定的词法属性（比较库）

列	词法属性	允许模式	示例单元取值
1	lexical_cell_prefix_name	[a-zA-Z0-9]*	AND2
2	lexical_drive1_name	D[0-9]*	D4
3	lexical_well_substrate_bias_architecture	WS	WS
4	lexical_device_threshold_voltage	ULVT SVT LVT MVT	ULVT

可用 `set_lib_cell_naming_convention` 为该库指定命名约定，例如：

```
fc_shell> set_lib_cell_naming_convention -library $lib \
  -column 1 -pattern {[a-zA-Z0-9]*} -attribute \
  "lexical_cell_prefix_name"
fc_shell> set_lib_cell_naming_convention -library $lib \
  -column 2 -pattern {D[0-9]*} -attribute "lexical_drive1_name"
fc_shell> set_lib_cell_naming_convention -library $lib \
  -column 3 -pattern {WS} -attribute \
  "lexical_well_substrate_bias_architecture"
fc_shell> set_lib_cell_naming_convention -library $lib \
  -column 4 -pattern {ULVT|SVT|LVT|MVT} -attribute \
  "lexical_device_threshold_voltage"
```

注意

预定义词法属性的完整列表见 `set_lib_cell_naming_convention` 命令的 man page。

指定命名约定后，用 `classify_lib_cell_attributes` 执行词法分类，例如：

```
fc_shell> classify_lib_cell_attributes -libset myLibset
```

要用 `report_lib_cell_classification_attributes` 报告词法分类结果。

要用 `report_lexical_unclassified_lib_cells` 报告在给定命名约定下未被词法分类的库单元。若存在未分类单元，请补充命名约定并重新运行词法分类。

4. (可选) 用 `set_use_for_library_analysis` 将特定单元排除在库分析之外。

默认情况下，工具在库分析期间排除仅物理单元。但你可排除更多单元，例如带有 `dont_use` 设置的库单元：

```
fc_shell> set exclude_cells [filter_collection [get_lib_cells $libs/*] \
  "dont_use == true"]
fc_shell> set_use_for_library_analysis $exclude_cells false
```

要用 `report_lexical_ignored_lib_cells` 报告被排除的单元。

5. 用 `identify_lib_cell_families` 识别全部库单元族。

要用 `report_lib_cell_families -all_initial` 报告工具识别的库单元族。

要手动定义库单元族，使用 `define_lib_cell_family`；要用 `report_lib_cell_families -all_userdef` 报告这些用户定义的单元族。

6. 用 `run_library_analysis` 分析库，例如：

```
fc_shell> run_library_analysis -name myLibset_run1 -environment env1
```

用 `-environment` 指定的环境名必须对应先前用 `create_environment` 创建的环境。

7. 用 `compare_libraries` 比较库。

此时必须用 `-ref_lib` 与 `-target_lib` 指定要比较的两个库。此外，必须用 `-ref_lib_analysis` 与 `-target_lib_analysis` 为每个库指定先前用 `run_library_analysis` 执行的库分析运行名，例如：

```
fc_shell> compare_libraries -ref_lib $rvt_lib -ref_lib_analysis sa \
-target_lib $lvt_lib -target_lib_analysis sa -verbose -plot
```

比较库时，可能遇到下列与参考库、目标库单元名称匹配相关的情况：

- 参考库与目标库中的单元名可完全匹配，例如同一库的两个不同版本。此时无需额外词法分类。
- 参考库与目标库可具有相同命名约定，但其中一个库的词法属性较少。
例如，假定参考库中名为 `ND2D1SVT` 的单元与目标库中名为 `ND2D1` 的单元匹配。此时目标库单元名不含 `lexical_device_threshold_voltage` 属性，如下表所示。

表 2.3: 参考库与目标库词法属性差异（属性缺失）

词法属性	参考库	目标库
lexical_cell_prefix_name	ND2	ND2
lexical_drive1_name	D1	D1
lexical_device_threshold_voltage	SVT	—

在此情况下，比较两个库时应忽略 `lexical_device_threshold_voltage` 属性。为此，在运行 `compare_libraries` 之前使用下列应用选项设置：

```
fc_shell> set_app_options -name libra.complib.match_full_cell_name \
-value false
fc_shell> set_app_options -name \
libra.complib.match_lexical_device_threshold_voltage -value false
fc_shell> compare_libraries -ref_lib $rvt_lib -ref_lib_analysis sa \
-target_lib $lvt_lib -target_lib_analysis sa -verbose -plot
```

注意

控制库比较期间名称匹配的应用选项完整列表见 `compare_libraries` 命令的 `man page`。

- 参考库与目标库可具有相似命名约定，但词法属性取值可能并不相同。
例如，假定参考库中名为 `ND2D1SVT` 的单元与目标库中名为 `nand2_svth_x1` 的单元匹配。此时两库具有相同词法属性，但取值相似而不完全相同，如下表所示。
在此情况下，必须创建包含下列信息的词法属性映射文件：

表 2.4: 参考库与目标库词法属性差异（取值不同）

词法属性	参考库	目标库
lexical_cell_prefix_name	ND2	nand2
lexical_drive1_name	D1	x1
lexical_device_threshold_voltage	SVT	svth

```
lexical_cell_prefix_name      ND2  nand2
lexical_drive1_name           D1   x1
lexical_device_threshold_voltage SVT svth
```

然后，运行 `compare_libraries` 时必须用 `-lex_attr_mapping` 指定该词法属性映射文件，例如：

```
fc_shell> set_app_options -name libra.complib.match_full_cell_name \
-value false
fc_shell> compare_libraries -ref_lib $rvt_lib -ref_lib_analysis sa \
-target_lib $lvt_lib -target_lib_analysis sa \
-lex_attr_mapping attribute_map_file -verbose -plot
```

- 参考库与目标库的命名约定完全不同。
- 在此情况下，必须创建包含待比较单元对的自定义映射文件，例如：

```
INVD2BWP      INVD1BWP HVT
INVD2BWP      BUFFD4BWP HVT
ND2D2BWP      ND2D4BWP HVT
DFQD4BWP      SDFQD4BWP HVT
```

然后，运行 `compare_libraries` 时必须用 `-custom_pairs` 指定该映射文件，例如：

```
fc_shell> set_app_options -name libra.complib.match_full_cell_name \
-value false
fc_shell> compare_libraries -ref_lib $rvt_lib -ref_lib_analysis sa \
-target_lib $lvt_lib -target_lib_analysis sa \
-custom_pairs custom_map_file -verbose -plot
```

2.6 读取设计

Reading the Design

读取设计之前，必须创建或打开与该设计关联的设计库，见「设置库」（Setting Up Libraries）。工具可读取 SystemVerilog、Verilog 或 VHDL 格式的 RTL 设计，以及 Verilog 格式的门级网表（gate-level netlists）。

用下列方法读取设计：

- 以 SystemVerilog、Verilog 或 VHDL 格式读取 RTL 文件
- 使用 `-autoread` 选项读取设计

- 使用 VCS 命令行选项读取设计
- 读取 Verilog 门级网表文件
- 读取混合的门级网表与 RTL 文件

默认情况下，工具读取 RTL 或门级网表文件时，会在当前设计库中创建一个 block，并增加其打开计数。工具通过识别在指定文件中未被其他模块例化的模块来确定 block 的顶层模块，并以该顶层模块名作为 block 名。

用下列命令处理 block：

- `create_block`：创建 block
- `open_block`：打开已有 block
- `current_block`：设置或报告当前 block
- `save_block`：保存 block
- `close_blocks`：关闭 block

参见

参见：Blocks; Working With Design Libraries。

2.6.1 以 SystemVerilog、Verilog 或 VHDL 格式读取 RTL 文件

Reading RTL Files in SystemVerilog, Verilog, or VHDL Format

使用 `analyze` 与 `elaborate` 命令，随后使用 `set_top_module`。

`analyze` 会根据 `-hdl_library` 选项提供的库名，在磁盘上自动创建 HDL 模板库（HDL template libraries）。HDL 模板库的位置由 `hdlin.hdl_library.default_dirname` 应用选项控制。未指定 `-hdl_library` 时，默认库名为 `WORK`。要用 `hdlin.hdl_library.default_name` 指定不同的默认库名。

注意

控制 RTL 读取行为的全部应用选项均带有 `hdlin` 前缀。

`elaborate` 构建指定模块，但不链接设计的其余部分。设计链接只能在整个设计都已在内存中之后执行，因此 `elaborate` 不执行链接。这允许在对整个设计执行一次链接之前运行多次 `elaborate`。顶层模块必须是已 `elaborate` 的模块之一。

用 `set_top_module` 完成设计链接并设置顶层模块。顶层模块作为参数传给 `set_top_module`。顶层模块必须是先前用 `elaborate` 已 `elaborate` 的模块。`set_top_module` 将指定模块设为顶层设计，链接整个设计，并创建一个供后续综合流程使用的单一 block。

下列脚本用模板库读取 VHDL 文件并创建名为 `top` 的 block。无需指定磁盘上模板库的位置；`analyze` 会根据 `-hdl_library` 自动创建：

```
analyze -format vhd1 -hdl_library BOT_HDL_LIB bot.vhd
analyze -format vhd1 -hdl_library MID_HDL_LIB mid.vhd
analyze -format vhd1 top.vhd
elaborate top
```

```
set_top_module top
```

若顶层设计被分析到默认库以外的 HDL 模板库，应用 `-hdl_library` 为顶层设计提供 HDL 模板库名。例如：

```
analyze -format vhd1 -hdl_library BOT_HDL_LIB bot.vhd
analyze -format vhd1 -hdl_library MID_HDL_LIB mid.vhd
analyze -format vhd1 -hdl_library TOP_HDL_LIB top.vhd
elaborate -hdl_library TOP_HDL_LIB top
set_top_module top
```

也可选用 `define_hdl_library` 指定磁盘上 HDL 模板库的位置。例如：

```
define_hdl_library BOT_HDL_LIB -path ./TEMPLATES/BOT_HDL_LIB
define_hdl_library MID_HDL_LIB -path ./TEMPLATES/MID_HDL_LIB
define_hdl_library WORK -path ./TEMPLATES/WORK
analyze -format vhd1 -hdl_library BOT_HDL_LIB bot.vhd
analyze -format vhd1 -hdl_library MID_HDL_LIB mid.vhd
analyze -format vhd1 top.vhd
elaborate top
set_top_module top
```

2.6.2 使用 `-autoread` 选项读取设计

Reading Designs Using the `-autoread` Option

要让工具自动读取 RTL 文件列表，对 `analyze` 指定 `-autoread`，如下例所示。指定该选项时，用于分析 RTL 文件的 RTL 语言由文件扩展名自动确定。要用下列应用选项控制文件扩展名：`hdlin.autoread.verilog_extensions`、`hdlin.autoread.sverilog_extensions`、`hdlin.autoread.vhdl_extensions` 以及 `hdlin.autoread.exclude_extensions`。

```
set DESIGN_NAME top
analyze -autoread -top ${DESIGN_NAME} ${RTL_FILE_LIST}
elaborate ${DESIGN_NAME}
elaborate -hdl_library TOP_HDL_LIB top
set_top_module ${DESIGN_NAME}
```

2.6.3 使用 VCS 命令行选项读取设计

Reading Designs Using the VCS Command-line Options

带 VCS 命令行选项的 `analyze` 提供与 VCS 仿真脚本的兼容性，并便于读取大型设计。使用 VCS 命令行选项时，工具通过在指定库中搜索被引用设计并加载这些设计，自动解析已例化设计的引用（references）。

要读取包含大量 HDL 源文件与库的设计，对 `analyze` 指定 `-vcs`。必须将 VCS 命令行选项括在双引号中。

例如：

```
analyze -vcs "-verilog -y mylibdir1 +libext+.v -v myfile1 \
+incdir+myincludedir1 -f mycmdfile2" top.v
```

```
elaborate ${DESIGN_NAME}
set_top_module ${DESIGN_NAME}
```

要在同一次 `analyze` 中读取指定扩展名的 SystemVerilog 文件与 Verilog 文件，使用 `-vcs "+systemverilogext+ext"`。此时文件不得包含任何 Verilog 2001 风格。

例如，下列命令分析扩展名为 `.sv` 的 SystemVerilog 文件以及 Verilog 文件：

```
analyze -format verilog -vcs "-f F +systemverilogext+.sv"
elaborate ${DESIGN_NAME}
set_top_module ${DESIGN_NAME}
```

2.6.4 读取 Verilog 门级网表文件

Reading Verilog Gate-Level Netlist Files

用 `read_verilog` 通过专用 Verilog 网表读取器读取 Verilog 网表文件。该命令不能用于读取 Verilog RTL 文件。读取 RTL 文件应仅使用 `analyze` 与 `elaborate`。读取网表后使用 `set_top_module` 指定顶层模块并链接设计以创建 block。

```
read_verilog top.netlist.v
set_top_module top
```

当文件含有多个独立模块时，用 `-top` 标识顶层模块。例如：

```
read_verilog -top top2 netlists.v
set_top_module top2
```

要指定不同于 `top_module_name.design` 的新 block 名，使用 `-design`。例如：

```
read_verilog -top my_top -design desA ../my_data/my_des.v
set_top_module my_top
```

2.6.5 读取混合的门级网表与 RTL 文件

Reading Mixed Gate-Level Netlists and RTL Files

可以读取 Verilog 网表与 RTL 文件的组合。用 `read_verilog` 读取 Verilog 网表；用 `analyze` 与 `elaborate` 读取 RTL 文件。全部网表文件与 RTL 文件读入工具后，用 `set_top_module` 链接设计以创建用于综合的 block。

下例说明如何读取网表与 RTL 的混合。如本例所示，带参数 `elaborate` 顶层模块时，顶层模块名也会包含参数化模块名。可用下列应用选项控制参数化模块的命名风格：`hdlin.naming.template_naming_style`、`hdlin.naming.template_parameter_style` 以及 `hdlin.naming.template_separator_style`。

```
# Read netlist files
read_verilog NETLIST/bot.v
# Analyze design files
analyze -format sverilog -hdl_library MID RTL/mid.sv
analyze -format sverilog          RTL/top.svd
# Elaborate top design module
```

```
elaborate -parameters "N=8,M=3" top
# Set the top level module to resolve references
set_top_module top_N8_M3
```

2.6.6 在 RTL 代码中嵌入 Tcl 命令

Embedding Tcl Commands in RTL Code

要在 RTL 代码中嵌入 Tcl 命令，使用 `fc_tcl_script_begin` 与 `fc_tcl_script_end` 指令。下列 Tcl 命令可嵌入 RTL 设计：

- `set_attribute`
- `set_ungroup`
- `set_size_only`
- `set_dont_touch`
- `set_dont_retime`
- `set_implementation`
- `set_optimize_registers`
- `get_modules`
- `get_cells`
- `get_pins`
- `get_nets`
- `get_ports`

下列 RTL 示例包含嵌入的 Tcl 命令，对名称前缀为 `iPort1.dout_reg` 的全部寄存器施加 `size_only` 设置：

```
module bot ( myIntf1.datIn iPort1 );
  always @(posedge iPort1.clk or negedge iPort1.rst)
  begin
    if (iPort1.rst == 1'b0)
      begin
        iPort1.dout <= '0;
      end
    else begin
      iPort1.dout <= iPort1.iData1 + iPort1.iData2;
    end
  end
end
// synopsys fc_tcl_script_begin
// set_size_only [get_cells iPort1.dout_reg*]
// synopsys fc_tcl_script_end
endmodule
```

2.7 缓解设计不匹配

Mitigating Design Mismatches

在早期设计开发中，频繁的设计更新可能导致数据不完整或不一致，例如 block 级设计与顶层设计对其引用之间的 pin 不匹配。要让工具即使存在某些类型的设计不匹配（design mismatch）仍能成功链接 block，可为该 block 设置不匹配配置（mismatch configuration）。

默认情况下，工具无法链接带有不一致或不匹配数据的任何 block。设置不匹配配置后，工具会忽略或修复若干不同类型的不匹配数据，并继续链接过程。以不匹配数据链接的 block 仅可用于可行性分析（feasibility analysis）。

参见

关于如何缓解设计不匹配，见 *Fusion Compiler Data Model User Guide*。

2.8 处理设计（门级网表与 block 操作补充）

Working With Designs

注意

Fusion Compiler 推荐优先使用上一节「读取设计」中的 `read_rtl/analyze/elaborate` 等流程。下列内容保留门级网表读取与 block 操作要点，与原 ICC2 流程兼容。

在多数情况下，在 Fusion Compiler 中处理设计时，使用设计库中某一版本（block）的 design view。本手册中，**block** 指设计库中的特定设计版本（不论顶层设计还是子设计）；**design** 泛指正在处理的设计。除非另有说明，二者均指 design view。

Fusion Compiler 以 Verilog 格式读取设计。Verilog 网表文件是单个文件中的结构级或门级设计。要将设计读入 Fusion Compiler：

1. 创建或打开与该设计关联的设计库。

- 若设计库尚不存在，用 `create_lib` 创建。
- 若设计库已存在，用 `open_lib` 打开。

2. 用 `read_verilog` 读取设计的 Verilog 网表文件。

默认情况下，工具读取 Verilog 网表时会在当前设计库中创建一个 block，并增加其打开计数。工具通过识别在指定 Verilog 文件中未被其他模块例化的模块来确定 block 的顶层模块，并以该顶层模块名作为 block 名。

工具还会为 block 创建默认电源域（power domain）。对多电压设计，在指定功耗意图（power intent）时该默认电源域会被替换，见「加载并应用 UPF 信息」。

用下列命令处理 block：

- `create_block`：创建 block
- `open_block`：打开已有 block
- `current_block`：设置或报告当前 block

- `save_block`: 保存 block
- `close_blocks`: 关闭 block

参见

参见: Blocks; Working With Design Libraries。

2.9 导入 Floorplan 信息

Importing the Floorplan Information

Floorplan 包含物理约束, 例如 `core` 区域与形状、`port` 位置、宏单元位置与方向等, 这些是执行物理综合与优化所必需的。

若 block 已有 floorplan, 按「读取 DEF 文件」所述将其作为 DEF 文件读入。

若 block 尚无 floorplan, 可按 *Fusion Compiler Design Planning User Guide* 所述执行设计规划并生成 floorplan。

2.9.1 读取 DEF 文件

Reading DEF Files

要从 DEF 文件读取 floorplan 信息, 使用 `read_def`:

```
fc_shell> read_def block.def
```

注意

尽可能使用 DEF v5.8 或更高版本, 因为该版本比早期版本支持更多类型的物理对象与障碍物 (obstructions)。

默认情况下, `read_def` 命令会:

- 将 floorplan 信息标注到当前 block。
要标注到其他 block, 用 `-design` 指定 block 名。

- 保留已有 floorplan 信息。

在增量 (incremental) 模式下:

- 布局区域根据当前 `core` 区域以及 DEF 中的 `site rows` 导入;
- 只能有一个取值的物理约束会被最新 DEF 中的值覆盖, 例如 `port` 位置与宏单元位置会被覆盖;
- 可累积取值的物理约束会重新计算, 即 `core` 区域可根据已有值与最新 DEF 中的 `site row` 定义重新计算。不同 DEF 文件中的 `placement keepout` 会累积, 最终 `keepout` 几何在综合期间由内部计算。

要在标注 DEF 中的 floorplan 信息之前先清除已有 floorplan, 使用 `-no_incremental`。该模式下, 布局区域根据 DEF 中的 `site rows` 导入。

- 对宏单元与 port 使用基于规则的名称匹配（rule-based name matching）。

基于规则的名称匹配利用工具的智能名称匹配能力自动解决名称差异。启用时，默认下列字符视为等价：

- 层次分隔符 { / _ . }

例如，名为 a.b_c/d_e 的单元会自动与 DEF 中的字符串 a/b_c.d/e 匹配。

- 总线记号 { [] _ () }

例如，名为 a [4] [5] 的单元会自动与 DEF 中的 a_4__5_ 匹配。

要禁用基于规则的名称匹配并要求 DEF 与 block 之间精确名称匹配，将 `file.def.rule_based_name_matching` 设为 `false`。

参见

更多信息见 *Fusion Compiler Data Model User Guide* 中的“Rule-Based Name Matching”。

- 忽略 DEF 中存在但 block 中不存在的任何对象（PG 对象除外）。

要允许从 DEF 创建新的非 PG 对象，用 `-add_def_only_objects` 指定要创建的对象类型。指定下列一个或多个关键字：

- **cells**：工具创建仅存在于 DEF 中的单元，并按 DEF 定义连接其电源与地 pin；即使 DEF 中定义了信号、时钟或 tie pin 连接，也不会连接这些 pin。工具也不会创建新的层次单元；DEF 中指定的任何层次必须已在 block 中存在。
- **nets**：工具创建仅存在于 DEF 中的信号、时钟与 tie net，并将它们连接到 DEF PINS 段中指定的 port；即使 DEF 中定义了到网表中其他 port 或 pin 的连接，也不会连接。工具不创建新的层次 net；DEF 中指定的任何层次必须已在 block 中存在。
- **ports**：工具创建仅存在于 DEF 中的信号、时钟与 tie port，并将它们连接到 DEF PINS 段中指定的 net。
- **all**：工具创建仅存在于 DEF 中的非 PG 单元、net 与 port，效果等同于同时指定 **cells**、**nets** 与 **ports**。

修复 Site 名称不匹配

Fixing Site Name Mismatches

若 DEF 中使用的 site 名与技术文件中定义的 site 名不匹配，用 `-convert_sites` 指定 site 名映射。例如，若 DEF 使用名为 CORE 的 site，而技术文件仅定义名为 unit 的 site，可用下列命令在读取 DEF 时转换 site 名：

```
fc_shell> read_def -convert_sites { {CORE unit} } block.def
```

验证 DEF 文件

Validating DEF Files

要在将 floorplan 信息标注到 block 之前分析输入 DEF，用 `-syntax_only` 启用仅检查（check-only）模式。该模式提供关于 DEF 正确性与完整性的诊断信息，不会向 block 标注任何 floorplan 信息。

```
fc_shell> read_def -syntax_only block.def

fc_shell> check_duplicates -remove
```

`check_duplicates -remove` 可确保因执行 `read_def` 而不会在 `block` 中留下重复的 `shape` 与 `via`。

从 **DEF** 文件提取的物理约束

Physical Constraints Extracted From the DEF File

`read_def` 从 DEF 提取物理约束信息并标注到 `block`。但仅提取并标注下列物理约束：

- Placement Area（布局区域）
- Port Locations（Port 位置）
- Cell Locations（单元位置）
- Placement Blockages（布局障碍）
- Site Rows
- Routing Tracks（布线 track）
- Placement Bounds（布局 bound）
- Routing Blockages（布线障碍）
- Preroutes（预布线）

要在 GUI 中目视检查所提取的物理约束，使用 `layout view`。从 DEF 提取的全部物理约束会自动加入 `layout view`。

布局区域

布局区域根据 `site array` 信息计算。

Port 位置

对 DEF 中指定了位置的每个 `port`，工具在 `block` 中对应 `port` 上设置该位置。

注意

若 DEF 不含 `port` 位置信息，工具会从 `pad` 单元的位置继承 `port` 位置。

```
PINS 2 ;
  -Out1 + NET Out1 + DIRECTION OUTPUT + USE SIGNAL +
    LAYER M3 (0 0) (4200 200) + PLACED (80875 0) N;
  -Sel0 + NET Sel0 + DIRECTION INPUT + USE SIGNAL +
    LAYER M4 (0 0) (200 200) + PLACED (135920 42475) N;
END PINS
```

支持名称已更改以及多层的 `port`。

```
# Example 2: DEF Port Locations With Changed Names and Multiple Layers
PINS 2 ;
    - sys_addr\[23\].extra2 + NET sys_addr[23] + DIRECTION INPUT +USE
    SIGNAL
    + LAYER METAL4 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
    - sys_addr[23] + NET sys_addr[23] + DIRECTION INPUT + USE SIGNAL +
    LAYER
    METAL3 ( 0 0 ) ( 820 5820 ) + FIXED ( 1587825 2744180 ) N ;
END PINS
```

```
# Example 3: DEF Port Orientation Information
PINS 1;
    - OUT + NET OUT + DIRECTION INPUT + USE SIGNAL
    + LAYER m4 ( -120 0 ) ( 120 240 )
    + FIXED ( 4557120 1726080 ) S ;
END PINS
```

单元位置

对 DEF 中指定了位置且带有 **FIXED** 属性的每个单元，工具在 block 中对应单元上设置该位置。Example 4 给出 DEF 宏单元位置与方向信息，其中字母 E、W 分别表示东向与西向旋转。

```
# Example 4: DEF Cell Location Information
COMPONENTS 2 ;
    - macro_cell_abx2 + FIXED ( 4350720 8160 ) E ;
    - macro_cell_cdy1 + FIXED ( 4800 8160 ) W ;
END COMPONENTS
```

布局障碍（Placement Blockages）

read_def 导入 DEF 中定义的 hard、soft 与 partial 布局障碍。

注意

DEF 5.7 之前的版本不支持 partial blockage。此外，若 floorplanning 工具生成的是 DEF 5.6，需手动添加 #SNPS_SOFT_BLOCKAGE pragma 以指定 soft blockage，见 Example 7。

```
# Example 5: DEF Hard Placement Blockage Information
BLOCKAGES 50 ;
...
    - PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 )
...
END BLOCKAGES
```

```
# Example 6: DEF Version 5.7 Soft Placement Blockage Information
BLOCKAGES 50 ;
...
    - PLACEMENT + SOFT RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...

```

```
END BLOCKAGES
```

```
# Example 7: DEF Version 5.6 Soft Placement Blockage Information
BLOCKAGES 50 ;
...
- PLACEMENT RECT ( 970460 7500 ) ( 3247660 129940 ) ; #SNPS_SOFT_BLOCKAGE
...
END BLOCKAGES
```

```
# Example 8: DEF Partial Placement Blockage Information
BLOCKAGES 50 ;
...
- PLACEMENT + PARTIAL 80 RECT ( 970460 7500 ) ( 3247660 129940 ) ;
...
END BLOCKAGES
```

Site Rows

DEF 中的 site row 信息定义布局区域。

```
# Example 9: DEF Site Row Information
ROW ROW_0 core 0 0 N DO 838 BY 1 STEP 560 0;
```

布线 Tracks

DEF 中的 track 信息为基于标准单元的设计定义布线网格。该信息可用于布线；track 支持还可改善拥塞评估与报告，使其更接近布线结果。

```
# Example 10: DEF Routing Track Information
TRACKS X 330 DO 457 STEP 660 LAYER METAL1 ;
TRACKS Y 280 DO 540 STEP 560 LAYER METAL1 ;
```

布局 Bounds

若 DEF 中存在定义 bound 的 REGIONS，read_def 会导入这些布局 bound。此外，若相关 GROUP 中有单元附着到该 region，会在这些单元与 block 中的单元之间进行模糊单元匹配；匹配到的单元按下列方式附着到 bound：

- 若 block 中已有与 DEF 同名的 region，则相关 group 中的单元在增量模式下通过 add_to_bound 附着到该 region。
- 若 block 中尚无该 region，则通过 create_bound 以 DEF 中的同名创建；相关 group 中匹配到的单元也会附着。

```
# Example 11: DEF Placement Bound Information
REGIONS 1 ;
- c20_group ( 201970 40040 ) ( 237914 75984 ) + TYPE FENCE ;
END REGIONS
GROUPS 1 ;
- c20_group
  cell_abc1
```

```

    cell_sm1
    cell_sm2
+ SOFT
+ REGION c20_group ;
END GROUPS

```

布线障碍（Routing Blockages）

布线障碍的提取方式与布局障碍类似。

预布线（Preroutes）

工具提取 DEF 中定义的 preroute。

```

# Example 12: DEF Preroute Information
SPECIALNETS 2 ;
- vdd
+ ROUTED METAL3 10000 + SHAPE STRIPE ( 10000 150000 ) ( 50000 * )
+ USE POWER ;
...
END SPECIALNETS

```

2.10 设置多电压设计

Setting Up Multivoltage Designs

设置多电压设计时需完成下列任务：

- 应用多电压功耗意图（Applying the Multivoltage Power Intent）
- 准备电源网络（Preparing the Power Network）
- 定义电压区域（Defining Voltage Areas）
- 插入多电压单元（Inserting Multivoltage Cells）
- 控制多电压单元的布局
- 启用多电压 net 的改进缓冲
- 分析多电压信息

2.10.1 应用多电压功耗意图

Applying the Multivoltage Power Intent

关于应用多电压功耗意图，见：

- 加载并应用 UPF 信息
- 为仅物理单元指定 UPF 约束
- 保存 UPF 信息

加载并应用 UPF 信息

Loading and Applying UPF Information

要加载功耗意图并将其应用到多电压设计：

1. 用 `load_upf` 读取 UPF 文件。

```
fc_shell> load_upf block.upf
```

2. 若使用 golden UPF 流程且有来自 Design Compiler 的名称映射文件，用 `read_name_map` 读取该文件。

```
fc_shell> read_name_map block.nmf
```

3. (可选) 验证 UPF 一致性，并识别网表、floorplan 与功耗意图之间的 PG 冲突。

用 `resolve_pg_nets -check_only` 验证 UPF 一致性并识别 PG 冲突。该命令会识别问题，并报告在提交 (`commit`) 功耗意图时为解决问题所做的更改。若希望以不同方式解决问题，可在运行 `commit_upf` 之前用手动编辑命令解决。

4. 用 `commit_upf` 提交功耗意图。

```
fc_shell> commit_upf
```

`commit_upf` 对 UPF 一致性执行全局检查；解决网表、floorplan 与 UPF 规范之间的 PG 冲突；并将功耗策略 (power strategies) 与已有多电压单元关联。关于将功耗策略与已有多电压单元关联，见「将功耗策略与已有多电压单元关联」。

5. 用 `report_mv_path` 报告为多电压单元所做的关联。

若工具未能关联某些多电压单元，该命令会报告失败原因。必须成功提交功耗意图后，才能继续设计流程。

注意

成功运行 `commit_upf` 之后，若再尝试使用额外的 UPF 命令 (下列除外: `set_related_supply_net`、`connect_supply_net`、`set_design_attributes`、`set_port_attributes`、`find_objects`、`set_scope`)，工具会发出错误。要在运行 `commit_upf` 后修改功耗意图，必须先用 `reset_upf` 移除现有 UPF 规范，再重新加载功耗意图。

为仅物理单元指定 UPF 约束

Specifying UPF Constraints for Physical-Only Cells

仅物理单元 (physical-only cells) 的 UPF 约束，在读入不支持仅物理单元的其他工具 (如验证工具) 时可能引发问题。因此，请用下例所示语法将仅物理单元的 UPF 约束包起来：

```
if {[info exists snps_handle_physical_only] && \
    $snps_handle_physical_only} {

    /* Supply net connections for filler cells */
    connect_supply_net VDDS -ports [get_pins FILLER*/VDD]
```

```
}

```

默认情况下，Fusion Compiler 中 `snps_handle_physical_only` 变量为 `true`。因此加载并提交 UPF 时，这些约束会应用到仅物理单元。

用 `save_upf` 保存 UPF 时，工具对用户指定的与工具推导的仅物理单元 UPF 约束使用相同语法。因此，未将 `snps_handle_physical_only` 设为 `true` 的工具会忽略这些 UPF 约束。

要创建仅包含特定类型单元命令的 UPF 文件，对 `save_upf` 使用 `-include` 或 `-exclude`。这些选项接受如 `diode_cells`、`pad_cells`、`physical_only_cells` 等参数。完整参数列表见 `save_upf` 的 `man` 页。

对指定单元类型的命令过滤适用于 `connect_supply_net`、`set_related_supply_net`、`create_power_domain` 与 `set_isolation`。可对 `block` 级与全芯片 UPF 文件应用命令过滤。

保存 UPF 信息

Saving UPF Information

物理实现期间，工具会更新设计的功耗意图。要用 `save_upf` 生成更新后的 UPF 文件，供设计流程后续步骤使用。

`save_upf` 根据输入 UPF 文件分离 UPF 命令，并添加注释标明输入 UPF 文件及其加载时间。

假设按下列脚本加载、提交并保存 UPF：

```
load_upf top_1.upf
load_upf top_2.upf
load_upf top_3.upf
commit_upf
save_upf top.upf

```

生成的 `top.upf` 包含如下信息：

```
## Start - Load_upf top_1.upf on Wed Oct 26 17:03:18 2016
...
...
## End - Load_upf
## Start - Load_upf top_2.upf on Wed Oct 26 17:03:49 2016
...
...
## End - Load_upf
## Start - Load_upf top_3.upf on Wed Oct 26 17:04:22 2016
...
...
## End - Load_upf

```

2.10.2 准备电源网络

Preparing the Power Network

关于为物理实现准备电源网络，见：

- 创建逻辑电源与地连接
- 创建浮空逻辑供电网（Floating Logical Supply Nets）

创建逻辑电源与地连接

Creating Logical Power and Ground Connections

读入设计后，必须确保电源与地 net 与设计各单元的电源、地及 tie-off pin 之间存在逻辑连接。若设计尚无这些连接，用 `connect_pg_net` 创建。该命令可为单电压与多电压设计中的叶子单元、层次单元与仅物理单元创建逻辑电源与地连接。

创建逻辑电源与地连接之前，必须解决网表、floorplan 与 UPF 规范之间的任何 PG 冲突。

- 对多电压设计，冲突在提交功耗意图时解决，见「加载并应用 UPF 信息」。
- 对单电压设计，必须运行 `resolve_pg_nets` 解决冲突。注意：单电压设计的 UPF 规范即 `read_verilog` 生成的默认电源域。

若设计含有未映射（unmapped）实例，工具会发出信息消息，表明仅连接已映射实例。

`connect_pg_net` 有两种工作模式：

- 自动（**Automatic**）

自动模式下，命令从 UPF 规范推导全部电源与地 net、电源与地 pin、tie-off pin 及连接。若供电网尚不存在，工具会创建。

要在自动模式下创建逻辑电源与地连接，对 `connect_pg_net` 使用 `-automatic`。

- 手动（**Manual**）

手动模式下，命令按你的指定建立连接。若指定 pin 或 port 已有连接，工具先移除已有连接再创建指定连接。工具会验证连接是否与功耗意图匹配；若发现不匹配，发出警告但仍创建连接。

要在手动模式下创建逻辑电源与地连接，用 `-net` 指定电源或地 net，并将要连接到该 net 的 pin 与 port 作为命令参数。例如，将 VDD 电源 net 连接到所有名为 vdd 的 pin，将 VSS 地 net 连接到所有名为 vss 的 pin：

```
fc_shell> connect_pg_net -net VDD [get_pins */vdd]
fc_shell> connect_pg_net -net VSS [get_pins */vss]
```

要仅将 PG net 连接到电源与地 pin，使用 `-pg`；要仅连接到 tie-off pin，使用 `-tie`。默认若两者均未指定，工具连接到全部电源、地与 tie-off pin。这些选项不能与对象列表或 `-create_nets_only`、`-net` 一起使用。

无论使用何种命令选项，工具总会创建构建完整 PG 网表结构所需的连接，例如顶层 PG port 与中间层次 PG pin。

创建浮空逻辑供电网

Creating Floating Logical Supply Nets

浮空逻辑供电网是未连接到 block 中任何逻辑 pin 或 port 的电源或地 net。例如，电源规划期间创建的电源 feedthrough net 即为浮空供电网。

要用 `connect_pg_net` 创建浮空供电网，按下列步骤：

1. 在 UPF 文件中用 `supply_net_pg_type` 属性指定供电网类型，如下例：

```
...
create_supply_net VDD
...
...
```

```
set_design_attributes -elements VDD \
    -attribute supply_net_pg_type power
...
```

2. 在用 `connect_pg_net` 创建供电网之前，将 `mv.pg.create_floating_pg` 设为 `true` 以启用浮空供电网，如下例脚本：

```
...
...
# Apply the UPF
load_upf $upf_input_file_name
commit_upf
...
...
# create logical supply nets
set_app_options -as_user_default \
    -name mv.pg.create_floating_pg -value true
connect_pg_net -automatic
...
check_mv_design
```

对域无关（domain-independent）供电网，工具在当前 UPF scope 的最顶层层次中创建逻辑供电网；对域相关（domain-dependent）供电网，在该域的最顶层层次中创建。

下列情况下工具不创建逻辑供电网：

- 连接与用 `supply_net_pg_type` 指定的供电网类型冲突；
- 供电网已连接到某实例的 PG pin；
- `connect_pg_net` 未在当前或父物理层次中指定。

2.10.3 定义电压区域

Defining Voltage Areas

电压区域（voltage area）是与电源域关联的单元的物理布局区域。对多电压设计，电源域在 UPF 规范中定义。对单电压设计，工具在读取 Verilog 网表时创建默认电源域，并将其与默认电压区域关联，该默认电压区域由 block 的 core 区域构成。

布局器将电压区域视为排他性 move bound（exclusive move bound）：必须将电压区域内的单元放在指定区域内，且必须将所有其他单元放在电压区域之外。电压区域可以是矩形或直角多边形（rectilinear）；此外可以不相交、嵌套或重叠。对重叠电压区域，各电压区域的有效形状由电压区域 shape 的堆叠顺序（stacking order）决定。

要用 `create_voltage_area` 定义电压区域。定义时至少须指定与该电压区域关联的电源域，使用 `-power_domains`。可指定一个或多个电源域；但所有指定电源域必须具有相同的主供电网（primary supply net）。若指定单个电源域，电压区域名称由电源域名推导；若指定多个电源域，必须用 `-name` 为电压区域指定名称。

电压区域由一个或多个矩形或直角多边形 shape 组成，这些 shape 可以邻接、不相交或重叠。要用 `-region` 定义这些 shape 的边界：创建新电压区域时用 `create_voltage_area`；向已有电压区域

添加 shape 时用 `create_voltage_area_shape`。注意: `create_voltage_area` 可一次指定一个或多个 shape, 而每次 `create_voltage_area_shape` 只能指定单个 shape。

- 指定矩形边界时, 用下列格式给出左下角与右上角:

```
{{llx lly} {urx ury}}
```

- 指定直角多边形边界时, 用下列格式给出各顶点坐标:

```
{{x1 y1} {x2 y2} {x3 y3} {x4 y4} ...}
```

若电压区域由多个邻接或重叠的 shape 组成, 可根据 shape 的堆叠顺序将其合并为最少的一组不相交 shape。关于如何合并电压区域 shape, 见「合并电压区域 Shape」。

工具也用堆叠顺序解决来自不同电压区域的重叠 shape。关于解决重叠电压区域, 见「解决重叠电压区域」。

为确保电压区域边界不发生短路, 可为电压区域定义 guard band, 作为环绕电压区域的硬 keepout 边距。若为电压区域 shape 定义了 guard band, guard band 计入该 shape 的有效边界, 但不计入电压区域的有效布局面积。关于定义 guard band, 见「定义 Guard Band」。

要用 `set_voltage_area` 修改已有电压区域, 见「修改电压区域」。

多电压设计通常存在运行期间关断与上电的电源域, 同时其他电源域保持上电。处理关断域时, 关断部分中某些单元可能需要持续保持活动, 例如实现保持寄存器、隔离单元、保持控制路径与隔离使能路径。这些单元称为 always-on 单元。要在电压区域内为 always-on 单元定义特殊布局区域 (always-on well), 在电压区域边界内定义排他性 move bound。关于定义排他性 move bound, 见「定义 Move Bounds」。

创建电压区域后, 运行 `check_mv_design` 以验证设计无多电压违例。

合并电压区域 Shape

Merging Voltage Area Shapes

要将电压区域 shape 合并为最少的一组不相交 shape, 对 `create_voltage_area`、`create_voltage_area_shape` 或 `set_voltage_area` 使用 `-merge_regions`。

- 使用 `create_voltage_area -merge_regions` 时, 工具合并用 `-region` 指定的 shape。
- 使用 `create_voltage_area_shape -merge_regions` 时, 工具合并用 `-region` 指定的 shape 与指定电压区域的已有 shape。
- 使用 `set_voltage_area -merge_regions` 时, 工具合并指定电压区域的全部已有 shape。

工具根据堆叠顺序合并电压区域 shape。默认堆叠顺序为定义 shape 的顺序, 最后定义的 shape 在最上层。合并后的 shape 替换一组邻接或重叠 shape 中的顶层 shape; 该组中的其他 shape 被移除, 不再与该电压区域关联。

例如, 假设用下列命令创建由三个矩形 shape 组成的电压区域, 如图 2.9 左侧所示:

```
fc_shell> create_voltage_area -power_domains {PD1} \
    -region { {{0 0} {10 10}} {{10 0} {30 10}} {{15 5} {35 15}} }
{PD1}
fc_shell> get_voltage_area_shapes -of_objects PD1
{VOLTAGE_AREA_SHAPE_1 VOLTAGE_AREA_SHAPE_2 VOLTAGE_AREA_SHAPE_3}
```

使用 `-merge_regions` 合并这些 `shape` 后，电压区域变为单个直角多边形，如图 2.9 右侧所示。合并后的电压区域 `shape` 名为 `VOLTAGE_AREA_SHAPE_3`，即创建电压区域时最后定义的 `shape`。

```
fc_shell> set_voltage_area PD1 -merge_regions
Information: Merging abutted and overlapping shapes in voltage_area
'PD1'. (NDMUI-154)
1
fc_shell> get_voltage_area_shapes -of_objects PD1
{VOLTAGE_AREA_SHAPE_3}
```

figures/fig-merge-va.png

图 2.9: 合并电压区域 Shape

要用 `report_voltage_areas -verbose` 报告电压区域 `shape` 的堆叠顺序。

要用 `set_voltage_area_shape` 修改电压区域 `shape` 的堆叠顺序，见「修改堆叠顺序」。

解决重叠电压区域

Resolving Overlapping Voltage Areas

若来自两个或多个电压区域的 `shape` 完全或部分重叠，工具用 `shape` 的堆叠顺序确定各电压区域的有效形状。默认堆叠顺序为定义 `shape` 的顺序，最后定义的在最上层。工具将重叠区域分配给与顶层 `shape` 关联的电压区域。与合并同一电压区域内的 `shape` 不同，解决不同电压区域之间的重叠时，工具不会移除任何 `shape`；仅解释方式发生变化。

例如，假设要创建如图 2.10 所示的嵌套电压区域。

figures/fig-nested-va.png

图 2.10: 嵌套电压区域

要生成这些有效电压区域，必须先指定外层电压区域，再指定内层电压区域，使内层电压区域的 shape 位于上层：

```
fc_shell> create_voltage_area -power_domains PD1 \
    -region { {0 0} {30 30} }
{PD1}
fc_shell> create_voltage_area -power_domains PD2 \
    -region { {10 10} {20 20} }
{PD2}
fc_shell> get_attribute -objects [get_voltage_areas PD2] \
    -name effective_shapes
{{10.0000 10.0000} {20.0000 20.0000}}
fc_shell> get_attribute -objects [get_voltage_areas PD1] \
    -name effective_shapes
{{0.0000 0.0000} {10.0000 0.0000} {10.0000 20.0000} {20.0000 20.0000}
 {20.0000 10.0000} {10.0000 10.0000} {10.0000 0.0000} {30.0000 0.0000}
 {30.0000 30.0000} {0.0000 30.0000}}
```

若先指定内层电压区域，则外层电压区域的 shape 在上层并遮蔽内层电压区域，因而被工具忽略，如下例：

```
fc_shell> create_voltage_area -power_domains PD2 \
    -region { {10 10} {20 20} }
{PD2}
fc_shell> create_voltage_area -power_domains PD1 \
    -region { {0 0} {30 30} }
{PD1}
fc_shell> get_attribute -objects [get_voltage_areas PD2] \
    -name effective_shapes
fc_shell> get_attribute -objects [get_voltage_areas PD1] \
    -name effective_shapes
{{0.0000 0.0000} {30.0000 30.0000}}
```

要用 `report_voltage_areas -verbose` 报告堆叠顺序；用 `set_voltage_area_shape` 修改堆叠顺序，见「修改堆叠顺序」。

修改堆叠顺序

Modifying the Stacking Order

可用 `set_voltage_area_shape` 修改电压区域 shape 的堆叠顺序：

- `-raise`: 将 shape 上移一位
- `-lower`: 将 shape 下移一位
- `-top`: 将 shape 移到最顶层
- `-bottom`: 将 shape 移到最底层
- `-above`: 将 shape 移到另一 shape 正上方
- `-below`: 将 shape 移到另一 shape 正下方

定义 Guard Band

Defining Guard Bands

Guard band 定义环绕电压区域的硬 keepout 边距，其中不能放置任何单元（包括电平移位单元与隔离单元）。Guard band 保证不同电压区域中的单元彼此分离，使电源规划不会引入短路。

默认电压区域没有 guard band。要用 `create_voltage_area` 或 `create_voltage_area_shape` 的 `-guard_band`，为 `-region` 中每个 shape 指定水平与垂直 guard band 宽度。水平 guard band 宽度适用于电压区域的全部垂直边；垂直 guard band 宽度适用于全部水平边。

注意

若同时使用 `-merge_regions`，必须为合并后的每个不相交 shape 指定 guard band 宽度。通常仅在待合并的全部 shape 邻接或重叠因而合并为单个 shape 时使用该选项。

电压区域 shape 的有效边界包含其 guard band；但其有效可布局面积不包含 guard band。

例如，图 2.11 显示由下列命令定义的 PD1 电压区域周围的 guard band：

```
fc_shell> create_voltage_area -power_domains PD1 \
-region {{0 0} {30 0} {30 10} {40 10} {40 30} {20 30} {20 25} {0 25}} \
-guard_band { {3 1} }
```

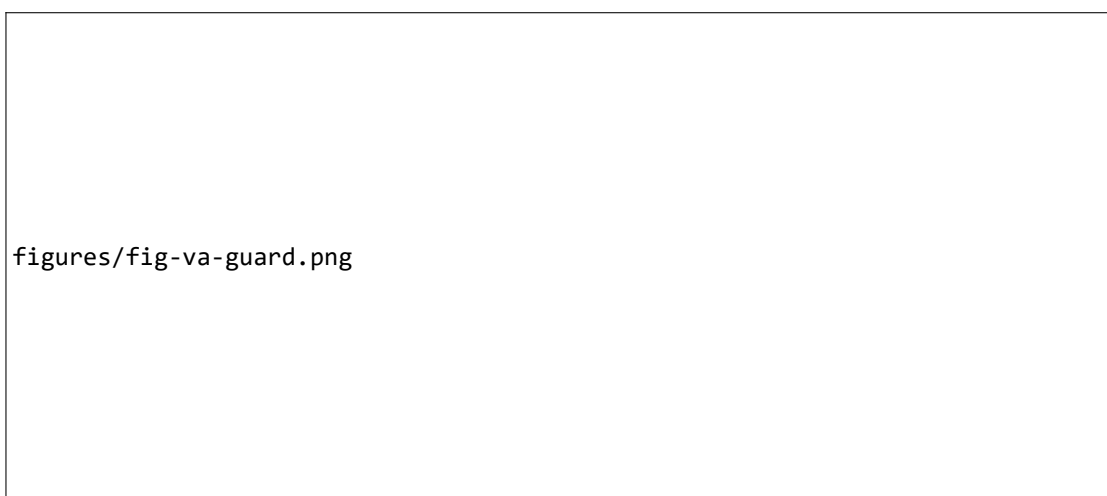


图 2.11: 电压区域 Guard Band

对同一电压区域关联的邻接或重叠 shape，工具按「合并电压区域 Shape」所述合并 shape，再将顶层 shape 定义的 guard band 应用到合并后的 shape。

例如，假设对图 2.9 右侧所示的电压区域 shape 用下列命令定义 guard band：

```
fc_shell> create_voltage_area -power_domains {PD1} \
-region { {{0 0} {10 10}} {{10 0} {30 10}} {{15 5} {35 15}} } \
-guard_band { {1 1} {3 3} {3 1} }
{PD1}
```

此时有效 guard band 与图 2.11 所示相同，即合并后 shape 所定义的 guard band。

对不同电压区域关联的重叠 shape，工具用有效边界按「解决重叠电压区域」所述解决 shape。顶层 shape 保留其布局面积与 guard band；较低层 shape 的有效布局面积与 guard band 不包含重叠区域。

注意：若邻接 shape 带有 guard band，由于有效边界包含 guard band，它们不再邻接而是重叠。

例如，假设对图 2.10 所示电压区域用下列命令定义 guard band：

```
fc_shell> create_voltage_area -power_domains PD1 \
    -region {{0 0} {30 30}} -guard_band { {2 2} }
{PD1}
fc_shell> create_voltage_area -power_domains PD2 \
    -region {{10 10} {20 20}} -guard_band { {2 2} }
{PD2}
```

此时 PD1 的有效布局面积会因 PD2 的有效边界（含其 guard band）而减小，如图 2.12。

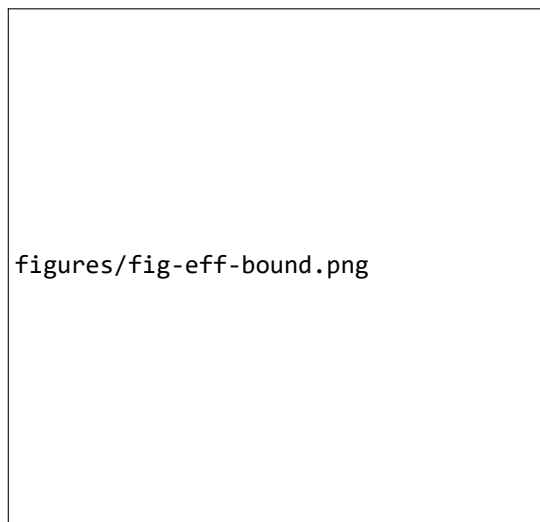


图 2.12: 重叠电压区域的有效边界

定义 Gas Station

Defining Gas Stations

Gas station 是为尽量减少在物理 feedthrough 路径上使用双轨（dual-rail）缓冲器而创建的小型电压区域。Gas station 中仅允许单轨中继器，且仅优化步骤可向 gas station 添加单元。

在设计规划阶段用 create_voltage_area 创建 gas station。工具会自动识别 gas station，并在绕线迂回代价与使用双轨单元代价之间权衡以使用它们。

用 create_voltage_area 的 -nwell 与 -pwell 为电压区域指定 n-well 与 p-well 供电网。为 gas station 电压区域定义 well 供电可为不同设计风格使用 gas station 提供灵活性。若未对这些选项赋值，则假定 well bias 与域供电值相同。

report_voltage_areas 会列出 n-well 与 p-well 供电网，无论是否显式设置。

查询电压区域

Querying Voltage Areas

可查询关于电压区域的下列信息：

- 当前 block 中的电压区域：用 get_voltage_areas 创建当前 block 中电压区域的集合（含默认电压区域）。

- 电压区域的详细信息：用 `report_voltage_areas` 显示详细信息（含默认电压区域）。要用 `-verbose` 包含构成各电压区域的 `shape` 信息。
- 电压区域的有效布局面积：查询电压区域的 `effective_shapes` 属性。
- 电压区域的有效 guard band：查询电压区域的 `effective_guard_band_boundaries` 属性。也可对单个电压区域 `shape` 查询该属性。

修改电压区域

Modifying Voltage Areas

创建电压区域后，可做下列修改：

- 更改与电压区域关联的电源域：对 `set_voltage_area` 使用 `-add_power_domains` 与 `-remove_power_domains`。
- 更改电压区域名称：对 `set_voltage_area` 使用 `-name`。
- 更改电压区域 region：用 `create_voltage_area_shape` 添加 shape；用 `remove_voltage_area_shapes` 移除 shape。

控制电压区域中的物理 Feedthrough Net

Controlling Physical-Feedthrough Nets in Voltage Areas

若 `net` 的一个或多个段位于某电压区域的逻辑层次中，则该 `net` 对该电压区域为 `native`。若 `net` 必须在物理上布线经过非 `native` 电压区域，则它是该电压区域的物理 `feedthrough net`，如下图所示。

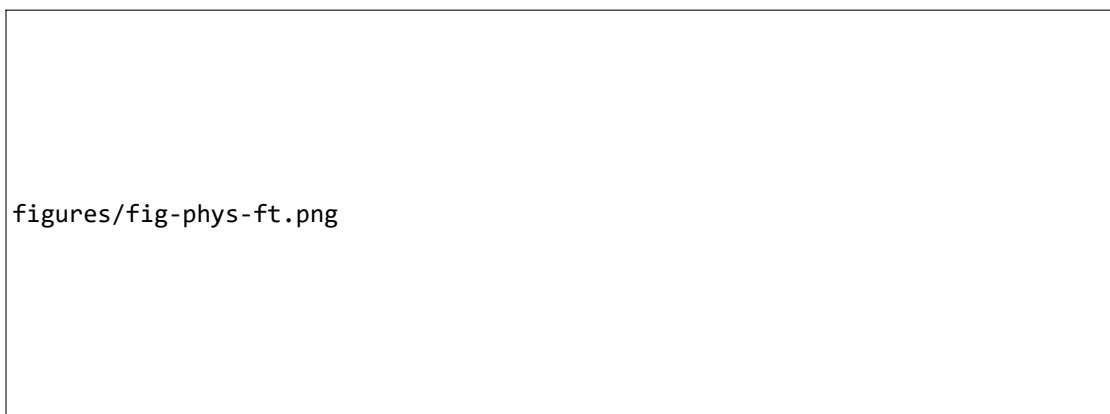


图 2.13: 电压区域的物理 Feedthrough Net

默认允许电压区域中存在物理 `feedthrough net`。要禁止某电压区域中的物理 `feedthrough net`，用 `create_voltage_area_rule -allow_pass_through false` 定义电压区域规则，例如：

```
fc_shell> create_voltage_area_rule -allow_pass_through false \
-name VA1_rule -voltage_areas VA1
```

有此规则时，N1 net 会绕开 VA1 电压区域，如图 2.14。

默认情况下，优化不会在电压区域的物理 `feedthrough net` 上插入缓冲器。在物理 `feedthrough net` 上插入缓冲器可能导致缓冲器的逻辑视图与物理视图不匹配。工具通过支持下列优化类型来解决此类不匹配：



figures/fig-phys-ft-off.png

图 2.14: 对电压区域禁用物理 Feedthrough Net

- 物理 **feedthrough** 缓冲（**Physical-feedthrough buffering**）

期间工具会：

1. 在其驱动或负载的逻辑层次中插入缓冲器，并用 power-domain-on-instance（PDOI）约束将非 native 电压区域的电源域应用到该缓冲器；
2. 将缓冲器放在非 native 电压区域内。

要允许物理 feedthrough 缓冲，使用 `create_voltage_area_rule -allow_physical_feedthrough true`，例如：

```
fc_shell> create_voltage_area_rule \
    -allow_physical_feedthrough true \
    -name VA1_rule -voltage_areas VA1
```

- 逻辑 **feedthrough** 缓冲（**Logical-feedthrough buffering**）

期间工具会：

1. 在与非 native 电压区域对应的逻辑层次中添加缓冲器；
2. 将缓冲器放在非 native 电压区域内。

要允许逻辑 feedthrough 缓冲，使用 `create_voltage_area_rule -allow_logical_feedthrough true`。要用 `-include_logical_feedthrough_hierarchy` 或 `-exclude_logical_feedthrough_hierarchy` 分别指定允许或不允许 feedthrough 缓冲器的逻辑层次。

下例为 VA1 启用逻辑 feedthrough 缓冲，并将 feedthrough 缓冲器限制为仅 U1 与 U2 层次单元：

```
fc_shell> create_voltage_area_rule \
    -allow_logical_feedthrough true \
    -include_logical_feedthrough_hierarchy {U1 U2}\
    -name VA1_rule -voltage_areas VA1
```

下例为 VA2 启用逻辑 feedthrough 缓冲，并排除 U3 层次单元：

```
fc_shell> create_voltage_area_rule \
    -allow_logical_feedthrough true \
    -exclude_logical_feedthrough_hierarchy {U3}\
    -name VA2_rule -voltage_areas VA2
```

注意

要向下层 block 添加逻辑 feedthrough 缓冲器，工具必须为该 block 添加新的边界 port。因此，用 `set_freeze_ports` 冻结 block 边界会阻止工具向该 block 添加逻辑 feedthrough 缓冲器。

figures/fig-ft-buf-diff.png

图 2.15: 物理与逻辑 Feedthrough 缓冲后逻辑视图的差异

要用 `create_voltage_area_rule` 配合 `-default_rule` 创建适用于所有尚无特定规则的电压区域的默认规则，例如：

```
fc_shell> create_voltage_area_rule -default_rule \
    -allow_physical_feedthrough true
```

若某电压区域既无特定规则也无默认规则，则该电压区域的 `feedthrough` 缓冲由 `opt.common.allow_physical_feedthrough` 应用选项控制。

要用 `report_voltage_area_rules` 报告电压区域规则；用 `remove_voltage_area_rules` 移除规则。

移除电压区域

Removing Voltage Areas

要用 `remove_voltage_areas` 从 block 移除电压区域。要用 `-all` 移除全部电压区域；要移除特定电压区域，指定电压区域名称。

2.10.4 插入多电压单元

Inserting Multivoltage Cells

多电压设计在电源域接口处需要特殊的多电压单元，例如电平移位单元与隔离单元。电平移位单元用于工作在不同电压电平的电源域之间；隔离单元用于处于不同状态（关断相对 `always-on` 或上

电) 的电源域之间。通常多电压单元在逻辑综合期间插入; 也可在 Fusion Compiler 中插入。

插入电平移位单元

Inserting Level Shifters

电平移位单元作为工作在不同电压电平的电源域之间的接口, 确保驱动的输出跳变能使接收单元切换, 即使接收端工作在不同电压电平。

注意

若 block 使用 Design Compiler 综合, 自动电平移位插入作为 `compile_ultra` 命令的一部分完成。

要在当前 block 中插入电平移位单元, 使用 `create_mv_cells`。工具按 UPF 规范中定义的单元映射与策略插入电平移位单元。插入时, 工具在电平移位单元上设置 `size_only` 属性, 并在 port 到电平移位单元的 net 上设置 `dont_touch` 属性。

默认情况下, 命令仅从 generic library 插入单元。若对 `create_mv_cells` 使用 `-mapped`, 则仅从用户提供的逻辑库插入单元; 若无合适单元则报错。

若工具未插入电平移位单元, 可用 `analyze_mv_design -level_shifter` 获取更多信息。若用 `-through` 指定 net 或 pin, 报告会列出该 net 或 pin 驱动端与负载端的电源域及相关供电, 以及说明为何未插入电平移位单元的错误消息。报告包含针对特定插入点的错误, 以及适用于经 `-through` 指定的 net 或 pin 整条路径的错误。

插入隔离单元

Inserting Isolation Cells

隔离单元用于有选择地关断电源域电压接口的输入侧; 它们不进行电平移位。隔离单元应在 RTL 级例化, 以避免形式验证错误。也可使用 Fusion Compiler 插入。

要在当前 block 中插入隔离单元, 使用 `create_mv_cells`。工具按 UPF 规范中定义的单元映射与策略插入隔离单元。插入时, 工具在隔离单元上设置 `size_only` 属性, 并在 port 到隔离单元的 net 上设置 `dont_touch` 属性。

默认情况下, 命令仅从 generic library 插入单元。若对 `create_mv_cells` 使用 `-mapped`, 则仅从用户提供的逻辑库插入单元; 若无合适单元则发出错误消息。

将功耗策略与已有多电压单元关联

Associating Power Strategies With Existing Multivoltage Cells

运行 `associate_mv_cells` 或 `commit_upf` 时, Fusion Compiler 会自动将功耗策略与已有多电压单元关联。若自动关联不正确, 可用 `set_power_strategy_attribute` 手动修改关联。要用 `get_power_strategies` 确定某电源域的功耗策略。

2.10.5 控制多电压单元的布局

Controlling the Placement of Multivoltage Cells

连接多电压单元 (如电平移位单元或隔离单元) 到另一电压区域中至少一个单元的 net, 称为多电压 net。要缩短多电压 net 长度, 将 `place.coarse.enable_advanced_mv_cell_placement` 设为

`true`，以启用高级多电压单元布局。

启用后，工具通过将多电压单元放得更靠近电压区域边界来缩短多电压 `net`。为优化多电压 `net`，工具使用双轨缓冲器。缩短多电压 `net` 可减少优化期间使用的双轨缓冲器数量，并防止因缺乏双轨缓冲器而导致多电压 `net` 未优化。

2.10.6 启用多电压 Net 的改进缓冲

Enabling Improved Buffering for Multivoltage Nets

多电压 `net` 是在逻辑或物理上跨越多个电压区域的 `net`。可将 `opt.buffering.enable_hierarchy_mapping` 设为 `true`，以在布线前优化期间对这类 `net` 修复逻辑 DRC 违例时启用改进的缓冲技术。

该应用选项影响 `place_opt` 与 `clock_opt`。启用后可减少用于修复逻辑 DRC 违例的缓冲器数量，但可能略微增加总负松弛（`total negative slack`）或 `hold` 违例数。

2.10.7 分析多电压信息

Analyzing Multivoltage Information

要用 `check_mv_design` 确保设计无多电压设计违例。

可用 `check_mv_design` 限制要检查的规则类型。例如，`-isolation` 指定检查隔离策略与隔离单元规则；`-pg_pin` 指定检查与 PG pin 相关的规则。

要用表 2.5 所列命令报告设计的多电压信息。

表 2.5: 报告多电压信息的命令

目的	使用的命令
报告多电压信息摘要	<code>report_mv_design</code>
报告多电压单元信息	<code>report_mv_cells</code>
报告多电压库单元信息	<code>report_mv_lib_cells</code>
报告带多电压约束的路径及相关多电压单元	<code>report_mv_path</code>
报告电源域	<code>report_power_domains</code>
检查指定电源域是否等价	<code>check_equivalent_power_domains</code>
显示等价电源域列表	<code>get_equivalent_power_domains</code>
报告电压区域	<code>report_voltage_areas</code>

2.11 指定时序约束与设置

Specifying Timing Constraints and Settings

时序约束描述设计的时序要求。时序约束的关键部分是准确指定的时钟及其效应（如 `latency` 与 `uncertainty`）。指定时钟后，工具自动约束由这些时钟驱动的寄存器之间的路径。但可通过指定时序例外（`timing exceptions`）改变这些时序路径的默认行为，例如不应分析的路径（`false paths`）、需要多个时钟周期的路径（`multicycle paths`）等。此外，可通过为输入与输出 port 指定 `input/output delay` 来约束边界时序路径。

Fusion Compiler 使用片上偏差（on-chip variation, OCV）模式进行时序分析，以建模芯片上工作条件变化的影响。该模式执行保守的时序分析，允许最小与最大延迟同时作用于不同路径。对 setup 检查，发射时钟路径与数据路径使用最大延迟，捕获时钟路径使用最小延迟；对 hold 检查，发射时钟路径与数据路径使用最小延迟，捕获时钟路径使用最大延迟。

一个 block 可能在多种条件（如不同温度与电压）下工作，并可能处于多种功能模式。时序分析中，每组条件由一个 **corner** 表示，每个功能模式由一个 **mode** 表示。**scenario** 是用于时序分析与优化的 corner 与 mode 的组合。开始处理 block 之前，必须定义用于该 block 的 mode、corner 与 scenario，以及延迟计算模型与所用布线层。所指定的布线层信息用于时序分析期间的 RC 估计。

详细信息见：

- 时钟与时钟效应： *Fusion Compiler Timing Analysis User Guide* 中的 “Defining Clocks”；
- 时序路径例外与边界路径约束： 同上指南中的 “Constraining Timing Paths”；
- Mode、corner 与 scenario： 同上指南中的 “Defining Modes, Corners, and Scenarios”；
- 工作条件与 OCV 相关设置： 同上指南中的 “Specifying Operating Conditions”；
- RC 估计与提取的寄生信息： 同上指南中的 “Performing Parasitic Extraction”。

2.12 指定逻辑设计规则约束

Specifying Logical Design Rule Constraints

最小电容、最大电容与最大 transition 是设计必须满足才能按预期工作的逻辑设计规则约束。它们是在逻辑库中指定的工艺相关限制。但可用表 2.6 中的约束命令指定更严格的设计规则约束。

优化期间，Fusion Compiler 会设法满足设计规则约束，即使这意味着违反时序、功耗与面积等优化约束——这些设计规则约束具有更高优先级。优化后，可用表 2.6 中的报告命令识别 block 中的设计规则约束违例。

表 2.6: 设计规则相关命令

目的	使用的命令
为输入 port、库单元 pin、叶子单元 pin、时钟或 block 指定允许的最小电容	set_min_capacitance
为输入 port、库单元 pin、叶子单元 pin、时钟或 block 指定允许的最大电容	set_max_capacitance
为输入 port、库单元 pin、叶子单元 pin、时钟或 block 指定允许的最大信号 transition 时间	set_max_transition
移除用户指定的最小电容约束	remove_min_capacitance
移除用户指定的最大电容约束	remove_max_capacitance
移除用户指定的最大 transition 约束	remove_max_transition
报告最小电容约束违例	report_constraints -min_capacitance
报告最大电容约束违例	report_constraints -max_capacitance
报告最大 transition 约束违例	report_constraints -max_transition

注意

Fusion Compiler 不遵从最大 fanout 设计规则约束。但可用 `opt.common.max_fanout` 应用选项为 block 中的数据路径指定最大 fanout。这是软优化约束。优化期间工具会设法确保数据路径单元驱动的扇出不超过指定的最大 fanout。

2.13 指定时钟门控约束

Specifying Clock Gating Constraints

默认情况下，工具在执行 `analyze` 与 `elaborate` 期间识别已有的时钟门控单元。在 `compile` 期间，若你未指定任何时钟门控约束，工具会在叶子级再插入一级时钟门控单元，并用默认时钟门控设置实现时钟门控逻辑。不能禁用时钟门控。

在设置时钟门控约束之前，先将设计与库载入内存，然后用下列命令设置约束：

- `set_clock_gate_style`
要选择集成时钟门控单元的类型，指定下列选项：

表 2.7: `set_clock_gate_style` 选项

选项	说明
<code>-test_point</code>	指定相对集成时钟门控单元的测试控制位置。有效取值为 <code>none</code> 、 <code>before</code> 与 <code>after</code> 。
<code>-observation_output</code>	指定观察输出 pin。
<code>-target</code>	指定要被时钟门控的目标时序元件。有效取值为 <code>pos_edge_flip_flop</code> 与 <code>neg_edge_flip_flop</code> 。
<code>-objects</code>	指定时钟门控所作用的对象。

- `set_clock_gating_options`
要定义所插入时钟门控单元的网表结构，指定下列选项：

表 2.8: `set_clock_gating_options` 选项

选项	说明
<code>minimum_bitwidth</code>	指定一起门控的最小位数。默认值为 3。
<code>max_fanout</code>	指定同一时钟门控单元所门控的最大单元数。默认值为无穷大（ <code>infinite</code> ）。

- `set_clock_gating_objects`
要控制指定对象的时钟门控，并覆盖 `compile` 期间设置的默认时钟门控行为，指定下列选项。对象可以是层次单元、寄存器、电源域或模块。
- 默认值
时钟门控优化期间，工具使用下列设置默认值：

表 2.9: set_clock_gating_objects 选项

选项	说明
-include	指定按时钟门控风格与时钟门控选项进行门控的对象列表。
-exclude	指定从时钟门控中排除的对象列表。
-clear	指定移除给定对象列表的全部时钟门控包含或排除设置。
-reset	指定移除先前由 set_clock_gating_objects 设置的全部条件。

表 2.10: 时钟门控默认设置

设置	默认值
最大扇出（Maximum fanout）	无穷大（Infinite）
最小位宽（Minimum bit-width）	3
时钟门控单元（Clock-gating cell）	对正沿与负沿触发触发器均使用基于锁存器（latch-based）的单元。若仅有一种类型可用，工具在时钟门控单元两侧插入反相器。

参见

更多信息见 Setting Up Clock Gating。

2.14 控制时钟门控延迟

Controlling Clock-Gate Latencies

综合期间，Fusion Compiler 假定时钟为理想时钟。理想时钟在时钟网络上不产生延迟。作出该假定是因为真实时钟网络延迟要到时钟树综合之后才可知。实际上时钟并非理想，时钟网络上存在非零延迟。对带时钟门控的设计，寄存器处的时钟网络延迟与时钟门控单元处的时钟网络延迟不同。寄存器与时钟门控单元之间的时钟网络延迟差异，会使时钟门控单元使能输入处的 setup 条件约束更紧。

工具可通过下列方式获得时钟门控延迟：

- 集成延迟估计（Integrated latency estimation）
默认情况下，工具在整个流程中估计并更新时钟门控延迟。估计的时钟延迟比用户指定的时钟延迟更准确。
- 用户指定延迟（User-specified latency）
可对特定实例禁用集成延迟估计，并手动指定延迟值。

2.14.1 集成时钟门控延迟估计

Integrated Clock-Gate Latency Estimation

默认情况下，工具在整个 place_opt 命令流程中估计并更新时钟门控延迟。这确保在时钟树综合之前、时钟仍为理想时，工具在数据路径优化与 useful-skew 优化中使用准确的时钟门控延迟。

若计划对设计执行结构性多源时钟树综合（structural multisource clock tree synthesis），请将 `opt.clock_latency_estimation.estimate_smscts_subtrees` 设为 `true`，以确保工具估计适合结构性多源时钟子树的时钟门控延迟。

为提高估计时钟门控延迟的准确性，在运行 `place_opt` 之前指定时钟树综合设置，包括时钟树缓冲器库单元列表、时钟树反相器库单元列表以及时钟非默认布线规则（NDR）。

估计的时钟延迟比用户指定的更准确。此外，工具在布局与多比特 `banking` 等步骤之后会更新估计的时钟延迟，而用户指定的延迟是静态值。

要阻止工具估计特定时钟门控的延迟，用 `set_attribute` 在其时钟 `pin` 上设置 `dont_estimate_clock_latency` 属性，例如：

```
fc_shell> set_attribute \
[get_pins {ICG21/CK}] dont_estimate_clock_latency true
```

为获得最佳结果，请勿将下列功能与集成时钟门控延迟估计一起使用：

- Trial clock tree synthesis: 将 `place_opt.flow.trial_clock_tree` 设为 `false` 以禁用。
- Clock-gate optimization: 将 `place_opt.flow.optimize_icgs` 设为 `false` 以禁用。
- 用户指定延迟: 用户指定延迟由 `set_clock_latency`、`set_clock_gate_latency`、`set_clock_gating_check -setup` 或 `set_path_margin -setup` 设置。

工具不会修改你在时钟门控单元上指定的任何 `set_clock_latency` 约束。工具估计的时钟门控延迟值存储为相对用户指定值的偏移（offset）。可用 `write_script -format icc2` 查看偏移值，并在生成输出中查找 `set_clock_latency -offset` 命令。若未设置任何延迟值，报告的偏移等于估计延迟值。

由 `write_sdc` 生成的 SDC 文件不会捕获估计的延迟偏移值。

若时钟门控及其被门控寄存器的相对位置在时钟树综合期间不变，延迟估计最准确。要保持集成时钟门控的布局，在时钟树综合之前将 `cts.compile.enable_cell_relocation` 设为 `auto`，以启用时钟网络单元的自动重定位。

2.14.2 用户指定的时钟门控延迟

User-Specified Clock-Gate Latency

不能禁用通用的集成时钟门控延迟估计；但可通过在时钟门控单元的时钟 `pin` 上设置 `dont_estimate_clock_latency` 属性，对特定时钟门控禁用该功能。本节说明如何对这些时钟门控手动指定延迟值。

用 `set_clock_latency` 或 `set_clock_gate_latency` 指定时钟网络延迟。`set_clock_gate_latency` 可用于门级与 RTL 设计。

用 `set_clock_gate_latency` 指定的时钟延迟，在 `compile_fusion` 插入时钟门控单元时标注到寄存器。但若在编译后修改时钟门控上的延迟值，必须用 `apply_clock_gate_latency` 手动将延迟值应用到已有时钟门控单元。

注意

用 `set_clock_gate_latency` 修改时钟门控延迟后，若再用 `compile_fusion` 编译设计，则不必再用 `apply_clock_gate_latency` 应用延迟值；工具会在编译期间标注指定值。

要用 `reset_clock_gate_latency` 移除先前由 `set_clock_gate_latency` 或 `apply_clock_gate_latency` 在时钟门控单元上设置的时钟延迟信息。该命令移除指定时钟上的时钟延迟值；若未指定时钟，则移除全部时钟门控单元上的时钟延迟值。

`set_clock_latency` 命令

The `set_clock_latency` Command

用 `set_clock_latency` 为特定时钟门控单元指定时钟网络延迟。

在图 2.16 中，`lat_cgtoreg` 是从时钟门控单元时钟 `pin` 到被门控寄存器时钟 `pin` 的估计延迟；`lat_reg` 是到无时钟门控的寄存器时钟 `pin` 的估计时钟网络延迟。

figures/fig-clk-lat-cg.png

图 2.16: 带时钟门控设计的时钟延迟

对设计中由特定时钟驱动的全部寄存器（有门控或无门控）的时钟 `pin`，对 `set_clock_latency` 使用 `lat_reg` 值。对全部时钟门控单元的时钟 `pin`，使用 `lat_reg` 与 `lat_cgtoreg` 之差。由于设置延迟值的目的是计入寄存器与时钟门控单元之间不同的时钟网络延迟，获得合理准确的差值（`lat_cgtoreg`）很重要。除非要用这些值处理与时钟门控无关的时钟网络延迟问题，否则绝对值相对次要。

`set_clock_gate_latency` 命令

The `set_clock_gate_latency` Command

使用 `compile_fusion` 时，时钟门控在编译过程中插入。要在工具插入时钟门控单元之前指定时钟网络延迟，使用 `set_clock_gate_latency`。该命令允许你将时钟门控单元的时钟网络延迟指定为时钟域、时钟门控级数（`stage`）以及时钟门控单元扇出的函数。你指定的延迟在 `compile_fusion` 插入时钟门控单元时标注到这些单元。可用 `apply_clock_gate_latency` 手动将延迟值标注到设计中已有的时钟门控单元。

`set_clock_gate_latency` 接受下列参数：

- `-clock clock_list`
- `-stage clock_gate_stage`
- `-fanout_latency fanout_list`

`fanout_list` 是指定扇出范围与延迟减量的元组列表。下例创建三个扇出范围：1 到 5 延迟为 0.5；6–15 为 0.8；16 到无穷大为 0.6。

```
set_clock_gate_latency -stage 1 \
    -fanout_latency {{1-5 0.5} {6-15 0.8} {16-inf 0.6}}
```

要为被时钟门控的寄存器指定时钟延迟值，对 `-stage` 使用值 0。由于是为被门控寄存器指定延迟，`-fanout_latency` 应为 `1-inf`（1 到无穷大），且延迟为寄存器的绝对时钟延迟值，例如：

```
set_clock_gate_latency -clock CLK -stage 0\
    -fanout_latency {1-inf 1.0}
```

若未指定 `-clock`，设置适用于设计中全部时钟。用 `set_clock_latency` 设置的时钟延迟优先级更高，不会被覆盖。

图 2.17 给出时钟门控级数与扇出示例。

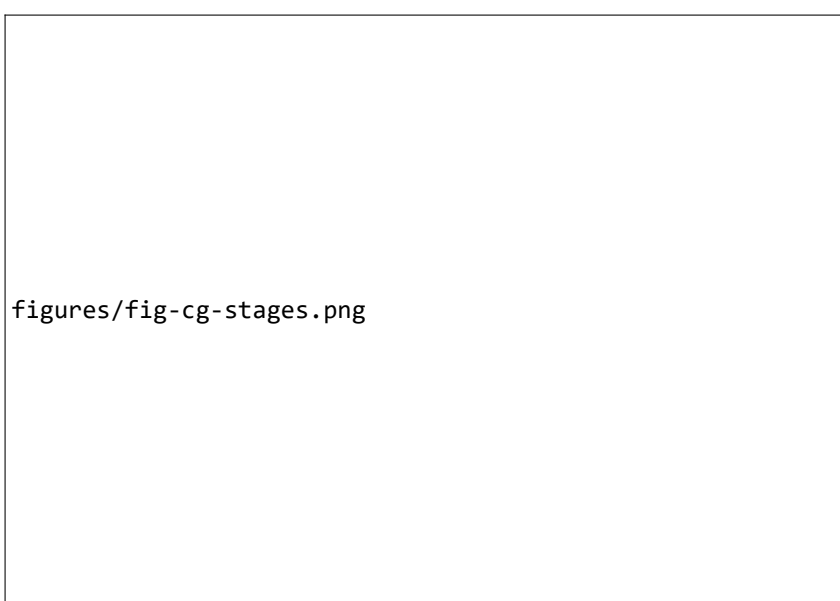


图 2.17: 时钟门控级数与延迟计算

下列命令为图 2.17 设置延迟值：

```
set_clock_gate_latency -stage 0 -fanout_latency {{1-inf 1.0}}
set_clock_gate_latency -stage 1 -fanout_latency {{1-inf 0.3}}
set_clock_gate_latency -stage 2 -fanout_latency {{1-inf 0.2}}
set_clock_gate_latency -stage 3 -fanout_latency {{1-inf 0.1}}
```

图 2.18 给出另一扇出与延迟计算示例。

下列命令为图 2.18 指定延迟与扇出：

```
set_clock_gate_latency -stage 0 \
    -fanout_latency {{1-inf 1.6}}
set_clock_gate_latency -stage 1 \
    -fanout_latency {{1-20 0.6} {21-inf 0.8}}
set_clock_gate_latency -stage 2 \
    -fanout_latency {{1-20 0.2} {21-inf 0.3}}
set_clock_gate_latency -stage 3 \
```



图 2.18: 变化扇出下的延迟计算

```
-fanout_latency {{1-30 0.1} {31-65 0.8} {66-inf 0.18}}
set_clock_latency "0.4" {uicg_a/CK}
```

注意：对时钟门控 uicg_a，用 set_clock_latency 赋值为 0.4 的延迟不会被覆盖。

2.15 为布局与合法化指定物理约束

Specifying Physical Constraints for Placement and Legalization

布局与合法化期间，floorplan 信息决定单元放置位置。下列主题说明如何指定影响布局与合法化的额外物理约束：

- 定义 Keepout Margins
- 定义基于区域的布局障碍（Area-Based Placement Blockages）
- 定义布局 Bounds（Placement Bounds）
- 定义布局吸引（Placement Attractions）
- 为合法化定义单元间距约束

2.15.1 定义 Keepout Margins

Defining Keepout Margins

Keepout margin 是 block 中固定单元边界周围的区域（如图 2.19 阴影部分），其中不放置其他单元。

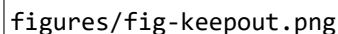


图 2.19: 布局 Keepout Margins

Outer keepout margin 是单元边界外的区域；**inner keepout margin** 是单元边界内的区域。固定单元各侧 keepout margin 的宽度可以相同或不同，取决于如何定义。此外，keepout margin 可定义为 hard 或 soft。将单元排除在此类区域之外可避免堵塞与 net 绕行，并获得更好的 QoR。

要用 `create_keepout_margin` 定义 keepout margin。默认创建 hard keepout margin；要创建 soft keepout margin，使用 `-type soft`。

定义 Outer Keepout Margin

Defining an Outer Keepout Margin

可在硬宏、层次单元或叶子单元上定义 outer keepout margin。定义时，可显式指定 keepout 距离；对硬宏，也可让工具根据宏 pin 数量推导 keepout 距离。

要显式指定 outer keepout margin，用 `-outer` 为各侧指定边距。按 `{lx by rx ty}` 格式指定左、下、右、上边距。值为 0 表示该侧无 keepout margin。

例如，为名为 `my_macro` 的宏创建各侧边距为 10 的 hard outer keepout margin：

```
fc_shell> create_keepout_margin -outer {10 10 10 10} my_macro
```

要让工具根据硬宏的 pin 数量推导 outer keepout 距离，用 `-tracks_per_macro_pin` 指定 track 与 pin 之比。使用该选项时，工具根据 track 宽度、宏 pin 数量以及指定的 track-to-pin 比计算 keepout margin，该比通常设为接近 0.5 的值。较大值产生较大 keepout margin。推导的 keepout margin 始终为 hard；`-type` 设置被忽略。推导的边距受 `-min_padding_per_macro` 与 `-max_padding_per_macro` 指定的最小与最大值约束。

例如，以 track-to-pin 比 0.6、最小 keepout 距离 0.1、最大 keepout 距离 0.2 为名为 `my_macro` 的宏推导 outer keepout margin：

```
fc_shell> create_keepout_margin -tracks_per_macro_pin 0.6 \
-min_padding_per_macro 0.1 -max_padding_per_macro 0.2 my_macro
```

定义 Inner Keepout Margin

Defining an Inner Keepout Margin

可在层次单元上定义 inner keepout margin，但不能在硬宏或叶子单元上定义。定义 inner keepout margin 时必须显式指定 keepout 距离。

要用 `-inner` 为各侧指定边距，格式为 `{lx by rx ty}`。值为 0 表示该侧无 keepout margin。

例如，为名为 `my_hcell` 的层次单元创建各侧边距为 10 的 hard inner keepout margin：

```
fc_shell> create_keepout_margin -inner {10 10 10 10} my_hcell
```

2.15.2 定义基于区域的布局障碍

Defining Area-Based Placement Blockages

基于区域的布局障碍是矩形区域，其中不能放置单元，或对单元类型或数量加以限制。Fusion Compiler 支持下列类型的基于区域的布局障碍：

- **Hard**：在粗布局、优化与合法化期间阻止在指定区域内放置标准单元与硬宏。
- **Hard macro**：在粗布局、优化与合法化期间阻止在指定区域内放置硬宏。
- **Soft**：在粗布局期间阻止在指定区域内放置标准单元与硬宏，但允许优化与合法化在指定区域内放置单元。
- **Partial**：在粗布局期间限制指定区域内的单元密度，对优化与合法化无影响。

要用 `create_placement_blockage` 定义布局障碍。至少须指定布局障碍的坐标。要创建矩形布局障碍，用 `-boundary` 指定矩形左下角与右上角坐标；要创建直角多边形布局障碍，用 `-boundary` 指定多边形坐标。也可通过将边界多边形指定为 `geo_mask` 或物理对象集合来创建多个布局障碍。若结果区域解析为多个非连续多边形，命令为每个多边形创建一个布局障碍。

默认情况下，`create_placement_blockage` 创建 hard 布局障碍。要创建其他类型，用 `-type` 指定障碍类型。单次 `create_placement_blockage` 只能创建一种类型的布局障碍。

可用 `-name` 为布局障碍指定名称，以便按名查询或移除。

定义 Hard 布局障碍

Defining a Hard Placement Blockage

要定义 hard 布局障碍，指定边界，并可选择指定名称。

例如，在角点为 (10,20) 与 (100,200) 的矩形区域内创建 hard 布局障碍：

```
create_placement_blockage -boundary {10 20 100 200}
```

注意

Hard 布局障碍在布局、合法化、优化与时钟树综合期间均被遵从。

Hard 布局障碍也可在 DEF 中定义，如 Example 13。

```
# Example 13: Placement Blockages in DEF
BLOCKAGES 2 ;
- PLACEMENT
  RECT ( 0 327600 ) ( 652740 327660 ) ;
- PLACEMENT
  RECT ( 0 327600 ) ( 652740 327660 ) ;
END BLOCKAGES
```

定义 **Hard Macro** 布局障碍

Defining a Hard Macro Placement Blockage

要定义 hard macro 障碍，指定边界、类型（`-type hard_macro`），并可选择指定名称。

例如，定义角点为 (120,75) 与 (230,200) 的 hard macro 障碍：

```
create_placement_blockage -boundary {120 75 230 200} \
  -type hard_macro
```

注意

Hard macro 布局障碍在布局、合法化与优化期间被遵从。这是硬宏布局所遵从的唯一布局障碍类型。

定义 **Soft** 布局障碍

Defining a Soft Placement Blockage

要定义 soft 障碍，指定边界、类型（`-type soft`），并可选择指定名称。

例如，定义角点为 (120,75) 与 (230,200) 的 soft 障碍：

```
create_placement_blockage -boundary {120 75 230 200} \
  -type soft
```

Soft 障碍阻止初始布局在指定区域内放置单元，但允许合法化、优化与时钟树综合这样做。然而，在布局与优化之后的后续增量布局中，工具可将合法化、优化与时钟树综合期间添加的单元移出 soft blockage 区域。要防止此行为，使用下列应用选项设置：

```
set_app_options -name place.coarse.enable_enhanced_soft_blockages \
  -value true
```

注意

Soft 布局障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义 **Partial** 布局障碍

Defining a Partial Placement Blockage

要定义 partial 障碍，指定边界、类型（`-type partial`）、障碍百分比（`-blocked_percentage`），并可选择指定名称。

例如，定义最大允许单元密度为 60%（blocked percentage 为 40）、角点为 (10,20) 与 (100,200) 的 partial 障碍：

```
create_placement_blockage -boundary {10 20 100 200} \
  -type partial -blocked_percentage 40
```

要允许无限制使用 partial blockage 区域，将障碍百分比指定为 0（-blocked_percentage 0）。

注意

Partial 布局障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义预定义类别的障碍

Define a Blockage of a Predefined Category

类别障碍（category blockage）是一种特殊的 partial 障碍，用于控制预定义单元类别在 partial blockage 内的布局。

要创建类别障碍：

1. 用 define_user_attribute 为类别定义用户属性，并用 set_attribute 将其应用到受影响的单元参考或实例。

要阻止单元放入类别障碍，将属性设为 true；要允许放入，设为 false。

定义与应用属性时遵守下列规则：

- 一个障碍只能由单个属性控制；
- 多个障碍可由同一属性控制；
- 一个单元可有多个属性，从而影响其在多个类别障碍中的布局；
- 同一属性可应用到参考与实例；
- 若单元实例与其参考的属性不同，实例上的属性优先。

2. 用 create_placement_blockage 配合 -type category、-blocked_percentage 与 -category 创建类别障碍。-category 的参数为用户属性名。

下例定义阻止某类单元放入其中的障碍：

```
define_user_attribute -type boolean \
  -classes lib_cell dr_cells
set_attribute [get_lib_cells mylib/dr*] dr_cells true
create_placement_blockage -boundary {{4300 2400} {4500 2700}} \
  -type category -blocked_percentage 45 -category dr_cell
```

下例定义仅允许某类单元放入其中的障碍：

```
define_user_attribute -type boolean \
  -classes lib_cell non_iso
set_attribute [get_lib_cells mylib/*] non_iso true
set_attribute [get_lib_cells mylib/iso*] non_iso false
create_placement_blockage -boundary {{4300 2400} {4500 2700}} \
  -type category -blocked_percentage 30 -category non_iso
```

注意

类别障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义排除寄存器的障碍

Defining Blockages That Exclude Registers

要定义排除寄存器的 **partial** 障碍，指定边界、类型（**-type register**）、障碍百分比（**-blocked_percentage**），并可选择指定名称。

例如，定义排除寄存器、但对其他单元允许 50% 单元密度的 **partial** 障碍：

```
create_placement_blockage -boundary {{10 20} {100 200}} \
    -type register -blocked_percentage 50
```

注意

寄存器障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义排除相对布局组的障碍

Defining Blockages That Exclude Relative Placement Groups

要定义排除相对布局组（relative placement groups）的 **partial** 障碍，指定边界、类型（**-type rp_group**）、障碍百分比（**-blocked_percentage**），并可选择指定名称。

例如，定义排除相对布局组、但对所有其他单元允许 100% 单元密度的 **partial** 障碍：

```
create_placement_blockage -boundary {{10 20} {100 200}} \
    -type rp_group -blocked_percentage 0
```

注意

相对布局组障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义仅允许相对布局单元的障碍

Defining Blockages That Allow Relative Placement Cells Only

要定义仅允许相对布局单元的 **partial** 障碍，指定边界、类型（**-type allow_rp_only**）、障碍百分比（**-blocked_percentage**），并可选择指定名称。

例如，定义仅允许相对布局单元、最大允许单元密度 80%（**blocked_percentage** 为 20）、角点为 (10, 20) 与 (100, 200) 的 **partial** 障碍：

```
create_placement_blockage -name rp_only -type allow_rp_only \
    -boundary {10 20 100 200} -blocked_percentage 20
```

这些障碍允许工具放置相对布局组（通常是大型结构），而不干扰其他单元的布局。工具布局并合法化相对布局组后，可移除这些障碍并允许工具在其中放置其他单元。

注意

仅相对布局单元障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

定义仅允许缓冲器的障碍

Defining Blockages That Allow Buffers Only

要定义仅允许缓冲器的 **partial** 障碍，指定边界、类型（**-type allow_buffer_only**）、障碍百分比（**-blocked_percentage**），并可选择指定名称。

例如，定义仅允许放置缓冲器与反相器、单元密度 30% 的 **partial** 障碍：

```
create_placement_blockage -boundary {{10 20} {100 200}} \
    -type allow_buffer_only -blocked_percentage 70
```

注意

仅缓冲器障碍在布局期间被遵从，但在合法化、优化或时钟树综合期间不被遵从。

查询布局障碍

Querying Placement Blockages

要用 **get_placement_blockages** 返回当前 **block** 中匹配特定条件的布局障碍集合。

移除布局障碍

Removing Placement Blockages

要用 **remove_placement_blockages** 从 **block** 移除布局障碍。要用 **-all** 移除全部布局障碍；要移除特定布局障碍，指定布局障碍名称。

2.15.3 定义布局 Bounds**Defining Placement Bounds**

布局 **bound** 是控制成组单元布局的约束。它允许你将单元分组以最小化线长，并将单元放在最合适的位置。例如，你可能希望为时钟门控单元或极关键时序单元组定义 **bound**，以保证它们不会因其他逻辑的布局而被打散。

表 2.11 列出 Fusion Compiler 支持的布局 **bound** 类型。

要用 **create_bound** 定义布局 **bound**。定义时必须用 **-name** 指定其名称。

一般还须指定要包含在 **bound** 中的单元与 **port**。若包含层次单元，则子设计中的全部单元属于该 **bound**。但也可创建空 **bound**，之后用 **add_to_bound** 指定内容；可用 **remove_from_bound** 从 **bound** 移除对象。

还必须指定创建所需 **bound** 类型所需的选项。下列主题说明如何创建各类 **move bound**：

- 定义 Move Bounds
- 定义 Group Bounds

表 2.11: 布局 Bound 类型

Bound 类型	说明
Soft move bound	工具在粗布局期间设法将 move bound 中的单元放在指定区域内，但不保证单元一定放在 bound 内。Soft move bound 在粗布局期间被遵从，合法化期间不被遵从。
Hard move bound	工具必须将 move bound 中的单元放在指定区域内。Hard move bound 在粗布局与合法化期间均被遵从，但时钟树综合期间不被遵从。
Exclusive move bound	工具必须在粗布局与合法化期间将指定单元放在指定区域内，并将所有其他单元放在排他性 move bound 之外。排他性 move bound 中新添加缓冲器的布局由 opt.buffering.exclusive_hard_bound_buffering_mode 控制。默认在缓冲期间，若能改善 QoR，工具可向排他性 hard move bound 添加新单元。若将该应用选项设为 0，工具遵从排他性 hard move bound。
Soft group bound	工具设法将 group bound 中的单元放在浮动区域内；但不保证单元一定放在 bound 内。
Hard group bound	工具必须将 group bound 中的单元放在浮动区域内，其实际坐标由工具确定。
Dimensionless group bound	工具根据用于使 group bound 内单元彼此靠近的努力程度，确定 group bound 的形状与位置。

- 查询布局 Bounds
- 移除布局 Bounds

定义 Move Bounds

Defining Move Bounds

Move bound 是放置一组单元的固定区域。它由一个或多个矩形或直角多边形 shape 组成，这些 shape 可以邻接、不相交或重叠。要用 -boundary 定义这些 shape 的边界：创建新 move bound 时用 create_bound；向已有 move bound 添加 shape 时用 create_bound_shape。注意：create_bound 可一次指定一个或多个 shape，而每次 create_bound_shape 只能指定单个 shape。

- 指定矩形边界时，用下列格式给出左下角与右上角：

```
{{llx lly} {urx ury}}
```

- 指定直角多边形边界时，用下列格式给出各顶点坐标：

```
{{x1 y1} {x2 y2} {x3 y3} {x4 y4} ...}
```

Move bound 可以是 hard、soft 或 exclusive。

- 定义 soft move bound 的语法：

```
create_bound -name name [-type soft] [-effort effort_level] \  
-boundary {coordinates} [bound_objects]
```

默认 effort 为 ultra; 也可指定 low、medium 或 high。

例如, 为 INST_1 单元实例定义左下角 (100,100)、右上角 (200,200) 的矩形 soft move bound:

```
fc_shell> create_bound -name b1 -boundary {100 100 200 200} INST_1
```

- 定义 hard move bound 的语法:

```
create_bound -name name -type hard -boundary {coordinates} \
    [bound_objects]
```

例如, 为 INST_1 定义矩形 hard move bound:

```
fc_shell> create_bound -name b2 -type hard \
    -boundary {100 100 200 200} INST_1
```

- 定义 exclusive move bound 的语法:

```
create_bound -name name -exclusive -boundary {coordinates} \
    [bound_objects]
```

例如, 为 INST_1 定义矩形 exclusive move bound:

```
fc_shell> create_bound -name b3 -exclusive \
    -boundary {100 100 200 200} INST_1
```

要用 create_bound_shape 向已有 move bound 添加 shape; 用 remove_bound_shapes 从已有 move bound 移除 shape。

定义 Group Bounds

Defining Group Bounds

Group bound 是放置一组单元的浮动区域。Group bound 可以是 hard、soft 或 dimensionless。

- 定义 soft group bound 的语法:

```
create_bound -name name [-type soft] [-effort effort_level] \
    [-dimensions {width height} bound_objects]
```

默认 effort 为 ultra; 也可指定 low、medium 或 high。

例如, 为 INST_1 与 INST_2 定义宽 100、高 100 的 soft group bound:

```
fc_shell> create_bound -name b4 -dimensions {100 100} \
    {INST_1 INST_2}
```

- 定义 hard group bound 的语法:

```
create_bound -name name -type hard -dimensions {width height} \
    [bound_objects]
```

例如, 为 INST_1 与 INST_2 定义宽 100、高 100 的 hard group bound:

```
fc_shell> create_bound -name b5 -type hard -dimensions {100 100} \
    {INST_1 INST_2}
```

- 定义 dimensionless group bound 的语法：

```
create_bound -name name [-effort effort_level] \
[bound_objects]
```

默认 effort 为 medium；也可指定 low、high 或 ultra。

例如，为 INST_1 与 INST_2 单元实例定义无量纲 group bound，并要求工具以 high effort 使单元在 bound 内更紧密放置，可用：

```
fc_shell> create_bound -name b6 -effort high {INST_1 INST_2}
```

2.15.4 查询布局 Bound

Querying Placement Bounds

要报告 block 中的布局 bound，使用 report_bounds 命令。

要返回当前 block 中匹配指定条件的布局 bound 集合，使用 get_bounds 命令。

要返回与一个或多个 move bound 关联的 bound shape 集合，使用 get_bound_shapes 命令。

2.15.5 移除布局 Bound

Removing Placement Bounds

要从 block 中移除布局 bound，使用 remove_bounds 命令。要移除 block 中全部布局 bound，使用 -all 选项；要移除特定布局 bound，指定 bound 名称。

2.15.6 定义布局 Attraction

Defining Placement Attractions

布局 attraction 是一种约束，用于指定一大组单元应放置在布局区域的同一邻近范围内。它是软约束，工具在初始布局时会考虑；但在后续增量布局阶段影响较小。若要对较小单元组施加更严格的布局约束，请用 create_bound 创建 move bound 或 group bound。

定义布局 attraction 使用 create_placement_attraction 命令。例如，指定子块 add1 与 mult1 中的单元放在同一邻近范围：

```
fc_shell> set add1_cells [get_flat_cells add1/*]
fc_shell> set mult1_cells [get_flat_cells mult1/*]
fc_shell> create_placement_attraction -name add1_mult1 \
"$add1_cells $mult1_cells"
```

可用 -region 选项指定：

- 放置单元的区域：给出区域左下角与右上角坐标，例如：

```
fc_shell> set U1_cells [get_flat_cells U1/*]
fc_shell> create_placement_attraction -name U1 \
-region {{0 0} {2000 1500}} $U1_cells
```

- 沿某直线放置单元（例如宏单元边缘）：给出直线两端坐标，例如：

```
fc_shell> set U2_cells [get_flat_cells U2/*]
fc_shell> create_placement_attraction -name U2 \
    -region {{1000 200} {1000 800}} $U2_cells
```

- 围绕某位置放置单元（例如 port 位置）：给出该位置坐标，例如：

```
fc_shell> set U3_cells [get_flat_cells U3/*]
fc_shell> create_placement_attraction -name U3 \
    -region {{0 700}} $U3_cells
```

下表列出用于修改、报告与移除布局 attraction 的其他命令。更多信息见各命令 man 页。

表 2.12: 与布局 Attraction 相关的命令（原书 Table 5）

目的	使用的命令
定义布局 attraction	create_placement_attraction
向已有布局 attraction 添加单元	add_to_placement_attraction
从已有布局 attraction 移除单元	remove_from_placement_attraction
报告布局 attraction	report_placement_attractions
查找已有布局 attraction	get_placement_attractions
移除已有布局 attraction	remove_placement_attractions

参见

定义 Move Bounds。

2.15.7 定义合法化单元间距约束

Defining Cell Spacing Constraints for Legalization

单元间距约束控制标准单元与另一标准单元或边界之间的间距（芯片边界、硬宏、硬宏 keep-out margin、硬布局 blockage，或 voltage area guard band）。将库单元分配到组（边界属于预定义组 SNPS_BOUNDARY），再定义这些组中单元之间的所需间距。

默认情况下，合法化期间标准单元之间没有间距约束。为提升良率，可定义标准单元之间，或标准单元与边界之间的有效间距。

注意

标准单元与电源/地 net 之间间距约束的支持，取决于该 net 在 block 中的表示方式。若电源/地 net 定义为完整 blockage，合法化器与 check_legality 命令会忽略标准单元与该电源/地 net 之间的间距规则违例。若定义为部分 blockage，则合法化器与 check_legality 会检查标准单元与电源/地 net 之间的间距规则违例。

单元间距约束的实现方式是：在库单元左右两侧（假定单元为 north 朝向）附着类似属性的 label，并规定这些 label 之间的无效间距。

定义单元间距约束的步骤：

1. 使用 `set_placement_spacing_label` 命令，为有间距约束的库单元添加 label。

每个 label 须指定：

- label 名称（`-name` 选项）
- 应用该 label 的库单元（`-lib_cells` 选项）
- 应用该 label 的库单元侧面（`-side` 选项，可取 `right`、`left` 或 `both`）

label 定义可累加；同一 label 可用于某些单元的右侧与另一些单元的左侧。例如，将名为 `X` 的 label 赋给 `cellA` 的右侧，以及 `cellB`、`cellC` 的左侧：

```
fc_shell> set_placement_spacing_label -name X \
        -lib_cells {cellA} -side right
fc_shell> set_placement_spacing_label -name X \
        -lib_cells {cellB cellC} -side left
```

也可为一侧分配多个 label。例如，将 `Y` 与 `Z` 赋给 `cellB` 的右侧：

```
fc_shell> set_placement_spacing_label -name Y \
        -lib_cells {cellB} -side right
fc_shell> set_placement_spacing_label -name Z \
        -lib_cells {cellB} -side right
```

2. 使用 `set_placement_spacing_rule` 命令定义 label 之间的间距要求。

每条规则须指定：

- 受约束的 label（`-labels` 选项）
每次 `set_placement_spacing_rule` 必须恰好指定两个 label。可为 `set_placement_spacing_label` 定义的任意 label，或预定义的 `SNPS_BOUNDARY` label（包含芯片边界、硬宏边界、硬宏 keepout margin、硬布局 blockage 与 voltage area guard band）。两个 label 可相同也可不同。
- 无效间距范围，以 unit tile 个数计。

例如，要求 label `X` 与 `Y` 之间至少有一个 unit tile（不可 abut）：

```
fc_shell> set_placement_spacing_rule -labels {X Y} {0 0}
```

要求 `X` label 与任意边界之间至少有一个 unit tile：

```
fc_shell> set_placement_spacing_rule \
        -labels {X SNPS_BOUNDARY} {0 0}
```

要求两个 `X` label 之间不能恰好间隔两个 unit tile：

```
fc_shell> set_placement_spacing_rule -labels {X X} {2 2}
```

要求 label `X` 与 `Z` 的间距小于 2 个或大于 4 个 unit tile：

```
fc_shell> set_placement_spacing_rule -labels {X Z} {2 4}
```

注意

Caution: 单元间距约束不会随 block 保存；仅作用于当前会话，每个会话都必须重新定义。

2.15.8 报告单元间距约束

Reporting Cell Spacing Constraints

报告单元间距规则使用 `report_placement_spacing_rules` 命令。该命令报告当前会话中定义的单元间距 `label`、其所应用的库单元（及侧面），以及为其定义的规则。

2.15.9 移除单元间距约束

Removing Cell Spacing Constraints

移除间距规则使用 `remove_placement_spacing_rules` 命令，必须指定要移除的规则：

- 移除全部间距规则：使用 `-all` 选项。
- 移除特定 `label` 及其关联的全部规则：用 `-label` 指定该 `label`。
- 移除两 `label` 之间的规则：用 `-rule` 指定与该规则关联的两个 `label`。

2.16 指定布局设置

Specifying Placement Settings

工具在布线前流程的多个阶段执行粗布局（`coarse placement`）。以下主题说明如何控制粗布局：

- 在早期阶段使用不精确约束进行布局
- 为时钟门控单元自动生成 Group Bound
- 控制布局密度
- 控制布局期间的拥塞驱动重构
- 降低拥塞
- 布局时考虑宽单元密度
- 布局时考虑单元 Pin 的影响
- 布局时考虑时钟 Net 非默认布线规则带来的拥塞影响
- 布局时考虑时序阵列中时钟门控单元的影响
- 布局时考虑合法化影响
- 布局时考虑 DFT 连接
- 布局时考虑动态功耗 QoR
- 执行 IR-Drop 感知布局
- 布局时分散 Repeater 单元

2.16.1 在早期阶段使用不精确约束进行布局

Performing Placement With Inaccurate Constraints at Early Stages

设计周期早期，布局约束可能不精确，导致工具在粗布局时退出。若因区域过小或被 blockage 覆盖而过利用率，仍希望继续布局，可将应用选项 `place.coarse.handle_early_data` 设为 `true`。启用后工具会发出如下警告示例：

```
Warning: Utilization of move_bound_0 is 109%. (PLACE-089)
Warning: Placement continues with over utilized regions in the design.
(PLACE-083)
Warning: Overutilized regions for move_bound_0 are modified by removal of
partial and full blockages. (PLACE-084)
```

启用该功能后，若某区域利用率超过 90%，工具会打印警告。要更改该阈值，使用应用选项 `place.coarse.utilization_warning_threshold`。

2.16.2 为时钟门控单元自动生成 Group Bound

Generating Automatic Group Bounds for Clock Gating Cells

工具可自动为集成时钟门控单元及其驱动的时序单元生成 group bound。设置如下：

```
fc_shell> set_app_options -name place.coarse.icg_auto_bound \
-value true
```

启用后，工具在布局开始时创建自动 group bound，并在布局结束时移除。已属于其他 group bound 的单元不会纳入自动 group bound。

要限制自动 bound 可包含的最大 fanout 数，使用 `place.coarse.icg_auto_bound_fanout_limit`，例如：

```
fc_shell> set_app_options \
-name place.coarse.icg_auto_bound_fanout_limit -value 30
```

默认 fanout 限制为 40。

2.16.3 控制布局密度

Controlling the Placement Density

可按下表控制 block 的布局密度。

指定最大密度或最大利用率时，取值应在 1 与 block 总体利用率之间。例如，若利用率 40%，可在 1 与 0.4 之间选择，如下：

```
fc_shell> set_app_options -name place.coarse.max_density -value 0.6
fc_shell> set_app_options \
-name place.coarse.congestion_driven_max_util -value 0.8
```

若不指定最大密度或最大利用率，且 `place.coarse.auto_density_control` 未设为 `false`，工具会根据设计流程阶段自动推导数值。

默认情况下 `place.coarse.auto_density_control` 已启用且为 `enhanced`，有助于改善总功耗与线长。

表 2.13: 控制布局密度的应用选项（原书 Table 6）

目的	使用的应用选项
指定最大密度，控制非拥塞驱动布局时单元可放置的密集程度	<code>place.coarse.auto_density_control</code> (默认 <code>-enhanced</code> ；若未将 <code>place.coarse.auto_density_control</code> 设为 <code>false</code> ，工具在流程中按预设计划工作。禁用该选项或手动将 <code>place.coarse.max_density</code> 设为 0 会使单元在 <code>core</code> 区域均匀铺开)
指定最大利用率，控制高拥塞区域周围低拥塞区域的单元可放置密集程度，以便高拥塞区单元可向外摊开以降低拥塞	<code>place.coarse.congestion_driven_max_util</code> (默认 0.93)

注意

无需禁用 `place.coarse.auto_density_control` 即可覆盖 `max_density` 与 `congestion_driven_max_util`；用户设置始终优先于工具推导值。

布局全程工具会打印 `PLACE-027` 信息消息，说明所用取值：

Information: Automatic density control has selected the following settings: max_density 0.60, congestion_driven_max_util 0.77. (PLACE-027)

工具将最大密度与拥塞驱动最大利用率设置独立应用于每个可放置区域（如 `voltage area` 或 `exclusive move bound`），而非对整个 `block` 取平均。

2.16.4 控制布局期间的拥塞驱动重构

Controlling Congestion-Driven Restructuring During Placement

默认情况下，工具在下列阶段对 `net` 进行重构以降低拥塞：

- `create_placement` 命令（使用 `-timing_driven` 时除外）
 - `place_opt` 命令的 `initial_place` 阶段
- 可如下控制拥塞驱动重构：
- 将 `place.coarse.cong_restruct_effort` 设为 `low`、`medium`（默认）、`high` 或 `ultra` 以指定 `effort`。
 - 将 `place.coarse.cong_restruct_depth_aware` 设为 `true`，防止路径深度增加超过三级逻辑。

2.16.5 降低拥塞

Reducing Congestion

要降低 `block` 拥塞，可使用：

- 将 `place.coarse.congestion_layer_aware` 设为 `true`，按层分别考虑拥塞并提高粗布局中拥塞缓解的精度。

默认工具在布局时合并所有层的拥塞信息。按层考虑后，工具可识别并修复 pin 密度高、但下层资源不足以完成 pin 连接的区域。

- 将 `place.coarse.increased_cell_expansion` 设为 `true`，根据局部布线资源需求在布局时扩大单元的虚拟面积。
在布线资源短缺区域增大虚拟面积，可降低单元密度与拥塞。
默认工具在水平方向扩展单元，有助于降低垂直布线层拥塞。若拥塞主要在水平布线层，将 `place.coarse.congestion_expansion_direction` 设为 `both`（默认为 `horizontal`）。
默认工具用全局布线拥塞图识别需扩展的高拥塞区，但不考虑 `soft routing rule` 带来的拥塞。要考虑 `soft routing rule` 的拥塞效应，将 `route.global.export_soft_congestion_maps` 设为 `true`。

2.16.6 布局时考虑宽单元密度

Considering Wide Cell Density During Placement

若单元无法跨垂直电源 strap，且单元宽度超过电源 strap 间距的一半，则两条电源 strap 之间只能放置一个此类单元。因此布局时必须尽量降低此类宽单元密度，以确保合法化时无需大位移即可放置。
对含宽单元的先进工艺节点，将 `place.coarse.wide_cell_use_model` 设为 `true`，以在布局中启用宽单元建模。

2.16.7 布局时考虑单元 Pin 的影响

Considering the Effects of Cell Pins During Placement

当用 `set_technology -node` 将工艺节点设为 7、7+、5、s5 或 s4 时，可将 `place.coarse.pin_cost_aware` 设为 `true`（默认 `false`），使布局使用工艺相关的 pin-cost 模型。启用后，工具用该模型预测访问各 pin 所需布线资源，以改善 pin 可达性。
对其余工艺节点，可将 `place.coarse.pin_density_aware` 设为 `true`（默认 `false`）以控制布局时的最大局部 pin 密度。

表 2.14: 控制布局期间 Pin 影响的设置（原书 Table 7）

set_technology -node	pin_cost_aware	pin_density_aware	工具行为
7, 7+, 5, s5, or s4	true	true 或 false	执行工艺相关的 pin-cost 感知布局
7, 7+, 5, s5, or s4	false	true	执行 pin 密度感知布局
7, 7+, 5, s5, or s4	false	false	布局时不考虑单元 pin 影响
其他任意设置	true 或 false	true	执行 pin 密度感知布局
其他任意设置	true 或 false	false	布局时不考虑单元 pin 影响

2.16.8 布局时考虑时钟 Net 非默认布线规则带来的拥塞影响

Considering the Congestion Effects Due to the Nondefault Routing Rules of Clock Nets

为改善时序并降低串扰，时钟 net 常大量使用非默认布线规则（NDR）。这些规则需要额外空间，可能在时钟布线后增加拥塞。
要将时钟 net NDR 的影响纳入布局考虑，将 `place.coarse.ndr_area_aware` 设为 `true`。

2.16.9 布局时考虑时序阵列中时钟门控单元的影响

Considering the Effects of Clock Gating Cells of Sequential Arrays During Placement

时序阵列（sequential array）常见于存储类器件，是按二维阵列排列的一组寄存器；同一行寄存器的时钟 pin 通常由单个时钟门控单元驱动。

要将时序阵列的时钟门控单元纳入布局考虑，将 `place.coarse.seq_array_icg_aware` 设为 `true`，有助于降低时钟 net 引起的拥塞。

2.16.10 布局时考虑合法化影响

Considering Legalization Effects During Placement

为减少合法化时大量单元被大幅搬移，粗布局应了解合法化限制。例如，因电源 strap 位置，某宽单元不能放在特定位置。

要将合法化效应纳入布局，将 `place.coarse.enhanced_legalizer_driven_placement` 设为 `true`。适用于合法化时大量单元有小到中等位移的 block；可用合法化报告的 RMS 位移值判断是否降低了位移。

2.16.11 布局时考虑 DFT 连接

Considering DFT Connections During Placement

要在仍优化功能布局的同时，增强工具对 DFT 连接的感知以改善 DFT 逻辑布局，将 `place.coarse.enable_dft_modeling` 设为 `auto`（默认 `false`）。

2.16.12 布局时考虑动态功耗 QoR

Considering the Dynamic Power QoR During Placement

要在布局时考虑动态功耗 QoR，执行：

1. 按「标注开关活动」所述，为设计标注 switching activity。
 2. 用 `set_scenario_status -dynamic_power true` 确保至少一个 scenario 启用动态功耗优化。
 3. 用下列方法之一，为 `create_placement`、`refine_placement`、`place_opt` 或 `clock_opt` 启用功耗驱动布局：
 - 将 `place.coarse.enhanced_low_power_effort` 设为 `none`、`low`、`medium` 或 `high`（默认 `low`），以启用动态功耗驱动布局。
- 动态功耗驱动布局中，工具尝试改善时序关键 net 的时序与功耗，以及其他 net 的功耗，从而在不影响时序 QoR 的前提下改善功耗 QoR。

2.16.13 执行 IR-Drop 感知布局

Performing IR-Drop-Aware Placement

布局时，工具可用单元的电压（IR）drop 值识别高功耗密度区域，并摊开高压降单元以降低该区域功耗密度。

执行 IR-drop 感知布局的步骤：

1. 对 block 进行布局并合法化。
2. 设置 RedHawk Fusion，并用 `analyze_rail -voltage_drop` 做静态或动态压降分析，例如：

```
fc_shell> source redhawk_setup.tcl
fc_shell> analyze_rail -voltage_drop static -nets {VDD VSS}
```

更多信息见「执行压降分析」。

3. 将 `place.coarse.ir_drop_aware` 设为 `true`，启用 IR-drop 感知布局。
4. (可选) 按「控制 IR-Drop 感知布局」指定附加设置。
5. 重新运行布局。

注意

执行 IR-drop 感知布局需要 Digital-AF 与 SNPS_INDESIGN_RH_RAIL license key。

控制 IR-Drop 感知布局

Controlling IR-Drop-Aware Placement

将 `place.coarse.ir_drop_aware` 设为 `true` 后，工具按单元总数创建三类单元。

figures/fig-ch2b-ir-drop-categories.png

图 2.20: IR-Drop 感知布局的默认单元类别

默认按图 2.20 所示准则选择三类单元：

- 上层类别：压降最高的约 1% 单元；布局时摊开力度最大。

- 中层类别：接下来约 5% 单元；摊开力度小于上层。
- 下层类别：其余单元；工具不对其进行摊开。

可用 `place.coarse.ir_drop_default_target_high_percentage` 与 `place.coarse.ir_drop_default_target_low_percentage` 更改上层与中层的百分比。例如，上层取前 2%，中层取接下来 6%：

```
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_high_percentage \
-value 2.0
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_low_percentage \
-value 6.0
```

也可不用单元总数百分比,而用压降相对电源电压的百分比定义类别:将 `place.coarse.ir_drop_default_target_high_population` 与 `place.coarse.ir_drop_default_target_low_population` 设为 `false`（默认为 `true`）。

例如，压降大于电源电压 8% 的单元归入上层，4%–8% 归入中层：

```
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_high_population \
-value false
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_high_percentage \
-value 8.0
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_low_population \
-value false
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_low_percentage \
-value 4.0
```

也可混合：上层用单元总数百分比，中层用压降相对电源电压百分比，或反之。例如上层取前 2% 单元，中层取压降大于电源电压 4% 的后续单元：

```
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_high_population \
-value true
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_high_percentage \
-value 2.0
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_low_population \
-value false
fc_shell> set_app_options \
-name place.coarse.ir_drop_default_target_low_percentage \
-value 4.0
```

可用 `set_placement_ir_drop_target` 为特定 voltage area 设置约束。例如，将 VA1 中前 1.5% 单元归入上层、接下来 4.5% 归入中层：

```
fc_shell> set_placement_ir_drop_target VA1 high 1.5
fc_shell> set_placement_ir_drop_target VA1 low 4.5
```

将 VA2 中压降大于电源电压 10% 的单元归入上层，5%–10% 归入中层：

```
fc_shell> set_placement_ir_drop_target VA2 high 10 -irdrop
fc_shell> set_placement_ir_drop_target VA2 low 5 -irdrop
```

获取、报告与重置基于 voltage area 的约束，分别使用 `get_placement_ir_drop_target`、`report_placement_ir_drop_target` 与 `reset_placement_ir_drop_target`。

2.16.14 布局时分散 Repeater 单元

Spreading Repeater Cells During Placement

若 block 在宏单元或 blockage 边缘/角落有大量 repeater 链（buffer、inverter 或流水线寄存器），工具可能沿边缘/角落聚集单元从而加剧拥塞。可在运行 `create_placement`、`place_opt` 或 `clock_opt` 前将 `place.coarse.spread_repeater_paths` 设为 `true`；工具会沿正交方向摊开 repeater，以降低边缘与角落拥塞。

2.17 指定合法化设置

Specifying Legalization Settings

工具在设计流程多个阶段执行合法化。以下主题说明如何控制合法化：

- 合法化期间最小化大位移
- 合法化期间优化 Pin Access
- 启用高级 PG Net 检查
- 启用高级合法化算法
- 设置 Variant 感知合法化
- 检查库单元是否可合法放置

2.17.1 合法化期间最小化大位移

Minimizing Large Displacements During Legalization

要最小化合法化时单元大位移，可启用：

- 朝向优化：将 `place.legalize.optimize_orientations` 设为 `true`。
工具会考虑翻转单元朝向以减小合法化位移。
- Stream placement：将 `place.legalize.stream_place` 设为 `true`。
工具将许多单元小幅移动，避免单颗单元大幅移动。可用 `place.legalize.stream_effort` 与 `place.legalize.stream_effort_limit` 进一步控制。

2.17.2 合法化期间优化 Pin Access

Optimizing Pin Access During Legalization

对先进工艺节点，为改善高 pin 密度区域的可布线性，可通过重分布单元启用 pin 优化：将 `place.legalize.optimize_pin_access_using_cell_spacing` 设为 `true`。

2.17.3 启用高级 PG Net 检查

Enabling Advanced PG Net Checks

对 12 nm、7 nm 或更小等先进节点，可将 `place.legalize.enable_advanced_prerouted_net_check` 设为 `true`，启用单元与预布线 PG net 之间的高级物理 DRC 检查。

启用后，可用 `place.legalize.advanced_layer_access_check` 与 `place.legalize.advanced_libpin_access_c` 进一步控制标准单元 pin 的可达性检查；对多数设计，默认行为已适用。

2.17.4 启用高级合法化算法

Enabling Advanced Legalization Algorithms

要将用于 2D 规则检查与单元交互的高级合法化算法启用（可缩短合法化 runtime），将 `place.legalize.enable_advanced_legalizer` 设为 `true`。

要指定未自动检测的附加高级合法化规则，设置 `place.legalize.enable_advanced_legalizer_rules`。

启用高级合法化算法后，默认在 pin access 优化时对齐垂直 pin shape 以改善可布线性。也可通过 `place.legalize.optimize_pin_access_strategies` 指定附加策略，见下表。

表 2.15: `place.legalize.optimize_pin_access_strategies` 设置（原书 Table 8）

目的	使用的设置
对齐水平 pin shape	<code>horizontal_align</code>
将单元移离高 pin 密度区域	<code>avoid_high_pin_density</code>
同时做以上两者	<code>horizontal_align</code> <code>avoid_high_pin_density</code>

高级合法化与合法性检查期间，工具可同时检查多单元与 PG net shape 之间的 DRC。启用：将 `place.legalize.enable_multi_cell_pnet_checks` 设为 `true`。

要在最后一次 `route_opt` 或 `route_auto` 之后启用多单元检查，将下列应用选项设为 `true`（须先启用 `place.legalize.enable_multi_cell_pnet_checks`），有助于在考虑 QoR 的同时改善 runtime：

- `place.legalize.enable_multi_cell_access_check`：考虑邻接单元与附近预布线 PG net shape 对单元 pin access 的影响。默认 `false`。
- `place.legalize.enable_multi_cell_track_capacity_check`：分析该单元 pin 与邻接单元的 track 容量。Track capacity 估计该区域布线 track 是否足以访问全部指定 pin。启用本选项前须先启用 `place.legalize.enable_multi_cell_access_check`。默认 `false`。

启用多线程高级合法化：

1. 将 `place.legalize.enable_threaded_advanced_legalizer` 设为 `true`。

2. 按「配置多线程」完成多线程配置。

执行多线程高级合法性检查：

1. 将 `place.legalize.enable_advanced_legalizer` 设为 `true`。

2. 按「配置多线程」完成多线程配置。

3. 用 `check_legality` 执行合法性检查。

2.17.5 设置 Variant 感知合法化

Setting Up for Variant-Aware Legalization

某些库提供功能等效、但可在不同位置合法化的单元集合。工具可通过用 `variant` 替换单元来合法化，而不必搬移单元。

下图示出 `pin` 位置不同的功能等效单元。使用 `variant` 感知合法化时，若单元 `pin` 未对齐 `track`，工具可尝试用其 `variant` 使 `pin` 对齐 `track`。

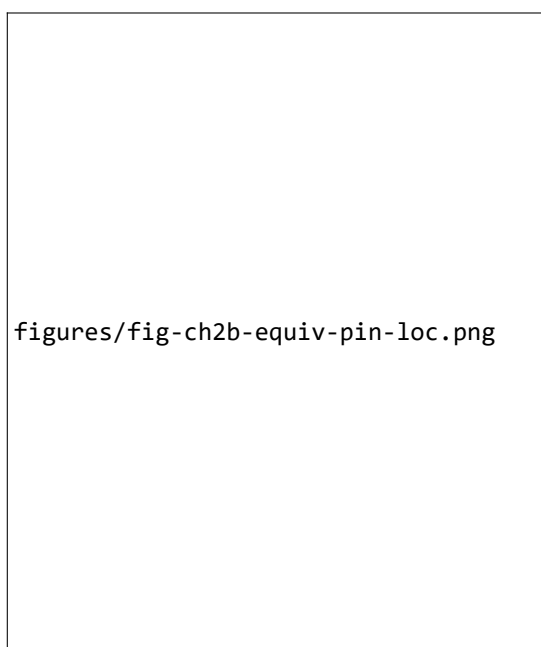


图 2.21: Pin 位置不同的等效单元

下图示出 `pin` 颜色不同的功能等效单元，须对齐同色 `track`。若 `pin` 未对齐对应颜色 `track`，工具可尝试使用不同颜色 `pin` 的 `variant` 并对齐到合适颜色的 `track`。

设置参考库与设计以支持 `variant` 感知合法化，需完成：

- 定义等效单元组
- 启用 Variant 感知合法化

定义等效单元组

Defining Equivalent Cell Groups

Library Compiler 支持从 Liberty 源文件创建 `variant-ready` 库。Liberty 中的 `physical_variant_cells` 属性定义单元的 `variant`；若该属性存在，编译后的库即为 `variant ready`。

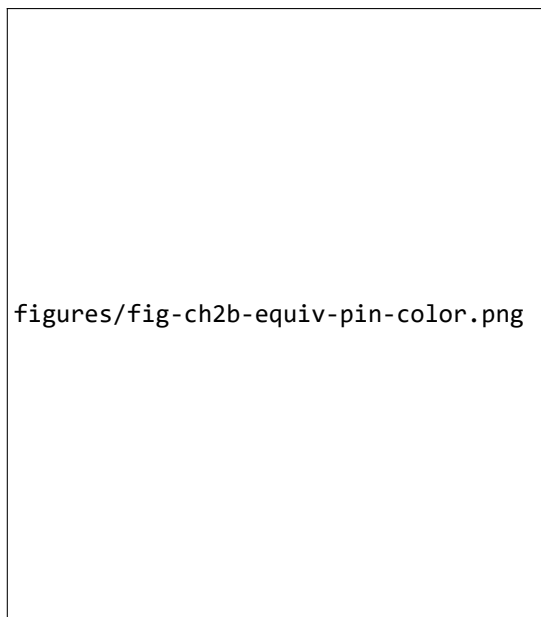


图 2.22: Pin 颜色不同的等效单元

若逻辑库非 variant ready，可在 Library Manager 的库准备阶段用 `create_cell_groups` 定义等效单元组。例如向已有参考库添加等效单元组：

```
lm_shell> create_workspace -flow edit LIB.ndm
lm_shell> create_cell_group -name A [get_lib_cells MY_LIB/A_*]
lm_shell> create_cell_group -name B [get_lib_cells MY_LIB/B_*]
lm_shell> check_workspace
lm_shell> commit_workspace
```

在 Library Manager 或 Fusion Compiler 中，可用 `report_cell_groups` 报告库中的等效单元组；该命令既报告 variant-ready 逻辑库 Liberty 中定义的组，也报告 Library Manager 中用 `create_cell_groups` 定义的组。

启用 **Variant** 感知合法化

Enabling Variant-Aware Legalization

将 `place.legalize.enable_variant_aware` 设为 `true` 以启用。启用后，工具优先用 variant 交换违例单元来修复合法化 DRC，而非搬移；若无 variant，再移到合法位置。

Variant 感知合法化中的 pin-track 与 pin-color 对齐，须将下列应用选项设为 `true`：

- `place.legalize.enable_prerouted_net_check`
- `place.legalize.enable_pin_color_alignment_check`

并用 `place.legalize.pin_color_alignment_layers` 指定要对齐的层。

2.17.6 检查库单元是否可合法放置

Checking if Library Cells Are Legally Placeable

指定全部布局与合法化约束及设置后，可用 `analyze_lib_cell_placement -lib_cells` 检查特定库单元能否在 `block` 中合法放置。

- 用 `-region` 将分析限制在 `core` 的特定区域（默认搜索整个 `core`）。
- 用 `-trials` 限制分析的 `site` 数量（默认随机分析 1000 个 `site`）。设为 0 则分析全部空闲 `site`，可能增加 `runtime`。
- 用 `-no_pdc` 或 `-no_adv` 在分析时忽略物理设计约束或高级设计规则。
- 用 `-threshold` 仅报告未达指定阈值的相对布局组：仅当单元可放置 `site` 占所分析 `site` 的百分比低于阈值时才报告。
- 用 `-max_cells` 限制报告单元数（默认 100，从通过率最差的库单元开始）。

若 `block` 中某些库单元通过率极低，合法化时会导致大位移与更长 `runtime`。此时应检查电源计划、单元版图与合法化设置以提高通过率。

下列示例分析当前 `block` 中有实例的全部库单元：

```
fc_shell> analyze_lib_cell_placement -lib_cells [add_to_collection \
  -unique "" [get_attribute [get_cells -physical_context] ref_phys_block]]
```

Analyzing 215 lib cell(s).

Warning: Routing direction of metal layer P0 is neither "horizontal" nor "vertical". PDC checks will not be performed on this layer. (PDC-003)

PDC app_options settings =====

```
place.legalize.enable_prerouted_net_check: 1
place.legalize.num_tracks_for_access_check: 1
place.legalize.use_eol_spacing_for_access_check: 0
place.legalize.allow_touch_track_for_access_check: 1
place.legalize.reduce_conservatism_in_eol_check: 0
place.legalize.preroute_shape_merge_distance: 0.0
```

Layer M1: cached 0 shapes out of 869 total shapes.

Layer M2: cached 773 shapes out of 773 total shapes.

Cached 41757 vias out of 99466 total vias.

0.0%...2.0%...4.0%...6.0%...8.0%...10.0%.....

Lib Cell	Pass Rate
saed32_lvt_lsup:LSUPX1_LVT.frame	0.4450
saed32_lvt_lsup:LSUPX8_LVT.frame	0.4780
saed32_lvt_lsup:LSUPX2_LVT.frame	0.4810
saed32_lvt_lsup:LSUPX4_LVT.frame	0.5040
.....	
saed32_rvt_std:HADDX1_RVT.frame	0.8600
saed32_lvt_std:OA21X1_LVT.frame	0.8610

2.18 控制单元、Net、Pin 与 Port 的优化

Controlling the Optimization of Cells, Nets, Pins, and Ports

以下主题说明如何控制单元、net、pin 与 port 的优化：

- 优化期间保留单元与 Net
- 将优化限制为仅做单元尺寸调整
- 优化期间保留网络
- 标记时钟网络
- 禁用设计规则检查（DRC）
- 尺寸调整期间保留 Pin 名称
- 保留已有层次的 Port
- 隔离输入与输出 Port
- 修复多 Port Net
- 控制向 Module、层次单元与 Voltage Area 添加新单元
- 为优化指定单元名前缀

2.18.1 优化期间保留单元与 Net

Preserving Cells and Nets During Optimization

要用 `set_dont_touch` 在优化期间保留设计对象。该命令对当前设计中的单元、net、reference 与子设计设置 `dont_touch` 属性，防止优化时修改或替换。

下表说明工具如何在优化期间保留各类设计对象。

表 2.16: 优化期间 `dont_touch` 对各设计对象的行为

设计对象	工具行为
单元	将单元排除在优化之外。
Net	保留各 pin 到该 net 的连接。
Module、设计、库单元	阻止优化所有引用它们的实例。
层次单元或 module	将该单元及其全部子单元排除在优化之外。

使用 `set_dont_touch` 时请注意：

- 该命令阻止层次 ungrouping。
- 子单元必须继承其父 module 的同一 `dont_touch` 属性。例如，父 module 已标 `dont_touch` 时，不能单独移除子单元上的该属性。
- 仅对完全映射（fully mapped）的逻辑使用，包括单元、module、设计，以及周围逻辑已完全映射的 net。

- dont_touch 优先级高于 boundary_optimization 与 size_only。

报告带 dont_touch 的设计对象，使用 report_dont_touch。

下列命令在单元 A 上设置 dont_touch:

```
fc_shell> set_dont_touch [get_cells A] true
```

移除该属性可用 remove_attributes，或将 set_dont_touch 设为 false:

```
fc_shell> remove_attributes [get_cells A] dont_touch
fc_shell> set_dont_touch [get_cells A] false
```

下表汇总可对 module、单元、net 与库单元设置、移除或报告 dont_touch 的命令支持情况。Yes 表示支持，No 表示不支持。

表 2.17: dont_touch 属性的命令支持（原书 Table 9）

命令	Module	单元（实例）	Net	库单元
set_dont_touch	Yes	Yes	Yes	Yes
set_attribute	Yes	Yes	Yes	Yes
define_user_attribute	No	No	No	No
get_attribute	Yes	Yes	Yes	Yes
remove_attributes	Yes	Yes	Yes	Yes
report_dont_touch	Yes	Yes	Yes	No
set_dont_touch_unmapped_object	No	No	Yes	NA

2.18.2 将优化限制为仅做单元尺寸调整

Restricting Optimization to Cell Sizing Only

要允许优化期间仅对单元或实例做尺寸调整，使用 set_size_only。该命令对指定单元或实例设置 size_only 属性，防止除 sizing 外的修改或替换。

查询该属性可用 report_attributes、get_attribute 或 report_cells；移除可用 remove_attributes，或将 set_size_only 设为 false。

在特定单元上设置:

```
fc_shell> set_size_only [get_cells cell_name] true
```

查询:

```
fc_shell> get_attribute [get_cells cell_name] size_only
```

报告当前设计全部 size_only 信息:

```
fc_shell> report_size_only -all
```

2.18.3 优化期间保留网络

Preserving Networks During Optimization

可用 `set_dont_touch_network` 在优化期间保留时钟树等网络。该命令对时钟、pin 或 port 设置 `dont_touch`，防止网络传递扇出中的单元与 `net` 被修改或替换。

默认同时保留时钟路径与 `clock-as-data` 路径，并在网络层次中传播 `dont_touch`。仅保留时钟路径时指定 `-clock_only`；从网络移除该属性时指定 `-clear`。

在 CLK 时钟网络上设置：

```
fc_shell> set_dont_touch_network [get_clocks CLK]
```

仅对 CLK 时钟路径：

```
fc_shell> set_dont_touch_network [get_clocks CLK] -clock_only
```

移除：

```
fc_shell> set_dont_touch_network [get_clocks CLK] -clear
```

2.18.4 标记时钟网络

Marking the Clock Networks

若 block 尚无时钟树，应使用 `ideal clock` 进行优化。将全部时钟网络标为 `ideal`：

```
foreach_in_collection mode [all_modes] {
  current_mode $mode
  set_ideal_network [all_fanout -flat -clock_tree]
}
```

为建模时钟树对布局的影响，还应为每个时钟用 `set_clock_uncertainty`、`set_clock_latency` 与 `set_clock_transition` 定义 `uncertainty`、`latency` 与 `transition` 约束。

在执行时钟树综合前，必须用 `remove_ideal_network` 移除时钟树扇出上的 `ideal` 设置。

2.18.5 禁用设计规则检查（DRC）

Disabling Design Rule Checking (DRC)

可用 `set_auto_disable_drc_nets` 禁用时钟、常数、scan enable 与 scan clock net 上的设计规则检查。

用该命令的选项控制当前设计中特定 `net` 的 DRC，见表 2.18。

例如，对全部此类 `net` 启用 DRC：

```
fc_shell> set_auto_disable_drc_nets -none
```

对连接时钟与常数的 `net` 禁用 DRC：

```
fc_shell> set_auto_disable_drc_nets \
-on_clock_network true -constant true
```

表 2.18: set_auto_disable_drc_nets 的选项（原书 Table 10）

选项	作用
-none	对全部 net 启用 DRC。
-all	对全部时钟、常数、scan enable 与 scan clock net 禁用 DRC。
-constant true false	对常数 net 禁用（true）或启用（false）DRC。
-on_clock_network true false	对时钟网络禁用或启用 DRC。
-scan true false	对 scan enable 与 scan clock net 禁用或启用 DRC。

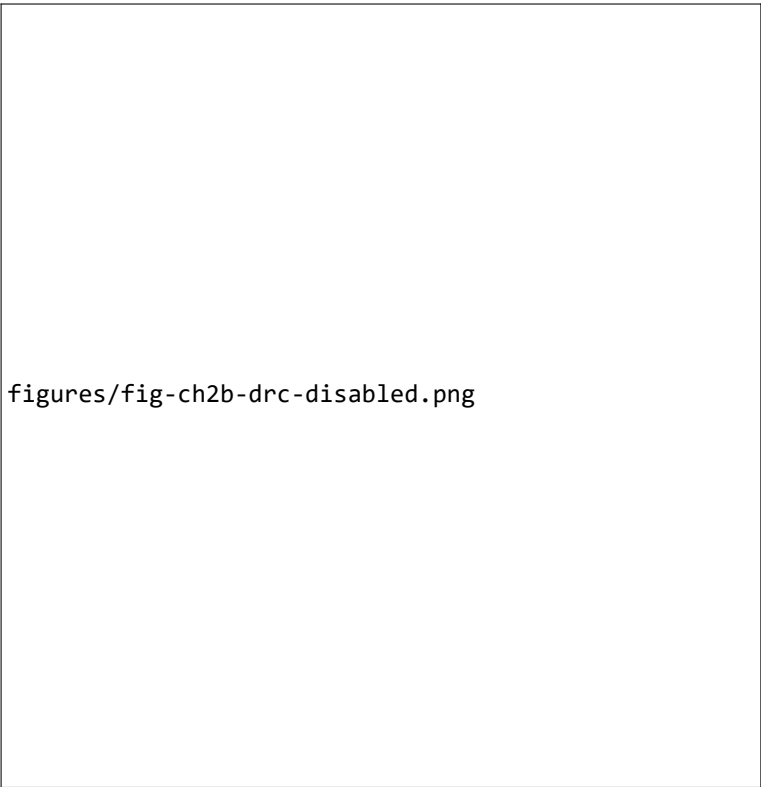


图 2.23: 已禁用 DRC 的时钟与常数 Net

2.18.6 尺寸调整期间保留 Pin 名称

Preserving Pin Names During Sizing

默认情况下，若结果电路功能等价，优化可在 sizing 时更改叶子单元的 pin 名称。Fusion Compiler 会自动更新内部约束版本以反映异常约束中涉及的新 pin 名。若发生此情况，用于约束 block 的原始约束不能再用于 signoff；必须从 Fusion Compiler 写出约束并随结果 block 一并使用。

该行为为优化引擎选择单元与改善代价函数提供最大灵活性。可通过设置 `opt.common.preserve_pin_names` 限制 sizing，使约束在优化中保持不变。默认值为 `never`，可取 `never` 或 `always`。

限制为仅 pin 名等价的单元：

```
fc_shell> set_app_options \
-name opt.common.preserve_pin_names -value always
```

恢复默认：

```
fc_shell> set_app_options \
-name opt.common.preserve_pin_names -value never
```

2.18.7 保留已有层次的 Port

Preserving Ports of Existing Hierarchies

优化与时钟树综合期间，工具可在已有层次 block 上创建新 port。要阻止此行为，使用 `set_freeze_ports`，并将对应单元实例的 `freeze_clock_port` 属性设为 `true`。

可用 `-clock`、`-data` 或 `-all` 分别阻止添加时钟 port、数据 port 或两者。例如，阻止在 MBX22 上创建额外时钟 port：

```
fc_shell> set_freeze_ports -clock [get_cells MBX22] true
```

报告 freeze-port 设置使用 `report_freeze_ports`。

2.18.8 隔离输入与输出 Port

Isolating Input and Output Ports

可隔离输入与输出 port 以提高时序模型精度。在指定输入或输出 port 插入隔离逻辑，使用 `set_isolate_ports`。隔离逻辑可以是 buffer 或一对 inverter；默认放置 buffer 以隔离：

- 输入 port 与其扇出网络
- 输出 port 与其驱动

使用 `-force` 时，工具确保 `-driver` 指定的库单元不会被 sizing。

工具不会在下列 port 上放置隔离逻辑：

- 双向 port
- 定义为时钟源或电源 pin 的 port
- 连接到带 `dont_touch` 属性的 net 的 port

示例

Example

下列命令插入隔离逻辑：

- 输入 port in3: buffer
- 输出 port out1: buffer
- 输出 port out3: LIBCELL_BUFF 库单元
- 输出 port out4: 一对 inverter

```
fc_shell> set_isolate_ports {in3 out1}
fc_shell> set_isolate_ports -driver LIBCELL_BUFF out3 -force
fc_shell> set_isolate_ports -type inverter out4
```

figures/fig-ch2b-port-isolation.png

图 2.24: Port 隔离逻辑插入前后对比

2.18.9 修复多 Port Net

Fixing Multiple-Port Nets

多 port net 包括：

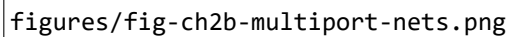
- Feedthrough net: 输入 port 直接馈入输出 port
- 连接到多个输出 port 的 net（逻辑等价输出）

门级网表中以 assign 语句表示的多 port net，可能在后续流程导致设计规则违例。默认优化期间工具不修复多 port net。

要将 opt.port.eliminate_verilog_assign 设为 true 以修复（默认 false）。修复时工具依次：

1. 尽可能重连，使每条 net 只连到一个层次 port。

工具优先重连以避免不必要的 buffering；因默认启用 boundary optimization，也会跨层次重连。



figures/fig-ch2b-multiport-nets.png

图 2.25: 多 Port Net 类型示意

2. 对未能通过重连修复的 net 插入 buffer 或 inverter。

2.18.10 控制向 **Module**、层次单元与 **Voltage Area** 添加新单元

Controlling the Addition of New Cells to Modules, Hierarchical Cells, and Voltage Areas

控制向下列对象添加新单元:

- **Module** 或层次单元: 使用 `set_allow_new_cells`
- **Voltage area**: 在 `create_voltage_area_rule` 中使用 `-allow_new_cells`

可用这些设置阻止向 abut 设计的顶层层次与 voltage area 添加新单元。设置保存在设计库中, 并在整个实现流程中被工具遵守。

阻止向顶层 module 添加:

```
fc_shell> set_allow_new_cells \
[get_attribute [current_block] top_module] false
```

阻止向 M1 module 添加:

```
fc_shell> set_allow_new_cells [get_modules M1] false
```

阻止向 U22 层次单元添加:

```
fc_shell> set_allow_new_cells [get_cells U22] false
```

阻止向 VA1 voltage area 添加:

```
fc_shell> create_voltage_area_rule -name VA1_rule \
-allow_new_cells false -voltage_areas VA1
```

2.18.11 为优化指定单元名前缀

Specifying a Cell Name Prefix for Optimization

可用 `opt.common.user_instance_name_prefix` 为优化期间在数据 net 上添加的单元指定名前缀。

下列示例为数据 net 上添加的单元指定名前缀:

- `place_opt` 期间: `PO_`
- `clock_opt` 期间: `CO_`
- `route_opt` 期间: `RO_`

```
fc_shell> set_app_options \  
-name opt.common.user_instance_name_prefix -value "PO_"  
fc_shell> place_opt  
fc_shell> set_app_options \  
-name opt.common.user_instance_name_prefix -value "CO_"  
fc_shell> clock_opt  
fc_shell> set_app_options \  
-name opt.common.user_instance_name_prefix -value "PO_"  
fc_shell> route_opt
```

为时钟树综合期间在时钟网络上添加的单元指定名前缀,使用 `cts.common.user_instance_name_prefix`, 见「为时钟单元定义名前缀」。

2.19 指定布线前优化设置

Specifying Settings for Preroute Optimization

为性能、功耗与面积（PPA）优化设计是工具的主要目标之一。以下主题说明如何在布线前阶段指定性能、功耗与面积优化设置：

- 控制 `place_opt` 的 DFT 优化
- 指定布线前优化的寄生参数估计设置
- 指定布线前优化的自动 Via Ladder 插入设置
- 在高利用率区域启用面积回收
- 启用高级逻辑重构

2.19.1 控制 `place_opt` 的 DFT 优化

Controlling DFT Optimization for the `place_opt` Command

若 block 含 scan chain, 默认 `create_placement`、`place_opt` 与 `clock_opt` 会执行物理 DFT 优化。初始布局时, 工具忽略 scan chain, 专注功能 net 的 QoR; 初始布局后, 基于初始布局对 scan chain 重新分区 (repartition) 并重排序 (reorder), 以进一步改善 QoR。

利用物理信息对 scan chain 重分区与重排序可：

- 缩短 scan chain 线长
- 最小化拥塞并改善可布线性

对含 scan chain 的 block 运行上述命令时, block 必须包含 SCANDEF 信息。用 `read_def` 标注 SCANDEF:

```
fc_shell> read_def ./myscan.def
```

注意

若此前已在 block 上标注 scan chain 信息，且 SCANDEF 中某 chain 与已有 scan chain 同名，则保留已有 chain 定义。

SCANDEF 文件指定 scan 路径所用的 scan chain 组件及其 pin，以及重排序约束。为获最佳效果，应使用 TestMAX DFT 做 DFT 插入，并用 Design Compiler topographical mode 生成 SCANDEF。DFT 插入见 *TestMAX DFT User Guide*；生成 SCANDEF 见 *Design Compiler User Guide*。

工具在 DFT 优化中使用 SCANDEF 信息。可按下表控制 DFT 优化。

表 2.19: 控制 DFT 优化的设置（原书 Table 11）

目的	使用的设置
对无 SCANDEF 信息的 block 运行 create_placement、place_opt 或 clock_opt	将 place.coarse.continue_on_missing_scandef 设为 true
禁用 DFT 优化	将 opt.dft.optimize_scan_chain 设为 false
禁用 scan chain 重分区	将 opt.dft.do_repartition 设为 false
禁用基于时序的 scan chain 重排序	将 opt.dft.timing_friendly_reorder 设为 false
重排序时考虑 scan 路径上的 hold 违例（CTS 后使用，以减少 scan 路径 hold 违例）	将 opt.dft.clock_aware_scan_reorder 设为 true
重分区与重排序时考虑 scan 路径上的 hold 违例（CTS 后使用）	将 opt.dft.clock_aware_scan_repartition_reorder 设为 false
DFT 优化期间保留层次 port（启用后不做 timing-friendly 或 clock-aware DFT 优化）	将 opt.dft.hier_preservation 设为 true

报告 DFT 优化相关应用选项设置，使用 report_app_options。

对已布局 block 执行独立 DFT 优化，使用 optimize_dft。

2.19.2 指定布线前优化的寄生参数估计设置

Specifying Parasitic Estimation Settings for the Preroute Optimization

以下主题说明如何在布线前阶段控制寄生参数估计：

- 启用基于全局布线层（GRLB）的布线前优化
- 为布线前优化启用 Route-Driven Estimation（RDE）

启用基于全局布线层（GRLB）的布线前优化

Enabling Global-Route-Layer-Based (GRLB) Preroute Optimization

为改善与布线后阶段的相关性，可在 place_opt 与 clock_opt 期间启用基于全局布线层的 RC 估计。启用后，工具对全部 net 使用全局布线，识别每单位电阻最合适的层，并将 net 约束到最小/最大层以进行布线前 RC 估计。

用 opt.common.use_route_aware_estimation 启用：

- **auto**: 仅当不同布线层的每单位电阻存在差异时启用
- **true**: 无论层间每单位电阻是否差异均启用

若启用该功能，在执行布线前须用 `remove_route_aware_estimation` 移除全部基于全局布线的估计。

为布线前优化启用 **Route-Driven Estimation (RDE)**

Enabling Route-Driven Estimation (RDE) for Preroute Optimization

为改善与布线后相关性，工具可执行全局布线、基于全局布线做提取，并在优化中使用这些寄生信息。

若检测到设计所用工艺为小于 16 nm 的先进节点，默认启用该功能。对任意工艺启用：将 `opt.common.enable_rde` 设为 `true`。

启用后，工具在 `place_opt` 与 `clock_opt` 的 `final_opt` 阶段执行 `route-driven` 寄生估计。寄生信息存入设计库，供后续优化步骤使用。启用后，应在设计流程后续全部布线前优化步骤中保持启用。

Route-driven estimation 期间，工具遵守 `set_extraction_options` 的 `-early_cap_scale`、`-late_cap_scale`、`-early_res_scale`、`-late_res_scale` 所指定的电容/电阻缩放因子。

启用 RDE 时，工具忽略用于启用 GRLB RC 估计的 `opt.common.use_route_aware_estimation` 设置。

2.19.3 指定布线前优化的自动 Via Ladder 插入设置

Specifying Automatic Via Ladder Insertion Settings for Preroute Optimization

Via ladder 是从 pin 层向上延伸到上层布线层的堆叠 via。优化期间使用 via ladder 可改善设计的性能与电迁移鲁棒性。工具可在 `place_opt` 与 `clock_opt` 期间为时序关键路径上的单元 pin 自动插入 via ladder。

布线前优化中执行 via ladder 插入：

1. 确保已按「定义 Via Ladder 规则」定义 via ladder 规则。
2. 用 `set_via_ladder_candidate` 为特定库 pin 指定可用 via ladder，见「为库 Pin 指定 Via Ladder 候选」。
3. (可选) 将 `opt.common.enable_via_ladder_insertion` 设为 `true`，为关键路径启用高性能与电迁移 via ladder 插入。
4. (可选) 将 `route.global.insert_gr_via_ladders` 设为 `true`，在带 via ladder 约束的 pin 上启用基于全局布线的 via ladder 插入。
5. 用 `place_opt` 或 `clock_opt` 执行优化。

为库 Pin 指定 Via Ladder 候选

Specifying Via Ladder Candidates for Library Pins

用 `set_via_ladder_candidate` 指定布线前优化中某库 pin 可用的有效 via ladder。每次命令只能指定一个 via ladder；要为同一库 pin 指定多个候选，多次调用，例如：

```
fc_shell> set_via_ladder_candidate [get_lib_pins lib1/AND2/A1] \
-ladder_name "VP4"
fc_shell> set_via_ladder_candidate [get_lib_pins lib1/AND2/A1] \
-ladder_name "VP2"
```

同一库 pin 上多个 via ladder 的指定顺序表示插入优先级；应从最高优先级开始指定。上例中 VP4 优先于 VP2。

注意

若同一 pin 同时适用 pattern_must_join 属性与 via ladder 候选，且两种方法估计延时相同时，via ladder 优先于基于 pattern 的 must-join 连接。

要指定某库 pin 需要电迁移 via ladder，将 pin 属性 is_em_via_ladder_required 设为 true：

```
fc_shell> set_attribute [get_lib_pins lib1/INV4/A] \
is_em_via_ladder_required true
```

若将该属性设为 true，必须用 set_via_ladder_candidate 将某个电迁移 via ladder 指定为候选。可通过下列方式之一将 via ladder 标识为电迁移 via ladder：

- 在工艺文件中定义 via 规则时使用 forElectromigration=1，例如：

```
ViaRule "EMVP1" {
    ...
    forElectromigration=1
}
```

- 将 for_electro_migration 属性设为 true：

```
fc_shell> set_attribute [get_via_rules EMVP1] \
for_electro_migration true
```

参见

定义 Via Ladder 规则。

2.19.4 在高利用率区域启用面积回收

Enabling Area Recovery in Regions of High Utilization

对因高利用率区域而无法合法化的设计，可将 opt.common.small_region_area_recovery 设为 true，让工具在高利用率区域执行面积回收。

启用后，工具在 place_opt 与 clock_opt 期间执行面积回收，但可能轻微劣化时序 QoR。

2.19.5 启用高级逻辑重构

Enabling Advanced Logic Restructuring

为改善面积、时序与功耗 QoR，可启用高级逻辑重构。对 place_opt 与 clock_opt 的 final_opto 阶段，用 opt.common.advanced_logic_restructuring_mode 启用，见下表。

表 2.20: `opt.common.advanced_logic_restructuring_mode` 设置（原书 Table 12）

目的	使用的设置
不执行重构	<code>none</code> （默认）
改善面积 QoR	<code>area</code>
改善时序 QoR	<code>timing</code>
改善功耗 QoR	<code>power</code>
改善面积与时序 QoR	<code>area_timing</code>
改善时序与功耗 QoR	<code>timing_power</code>
改善面积与功耗 QoR	<code>area_power</code>
改善面积、时序与功耗 QoR	<code>area_timing_power</code>

- 重构时，工具不会：
- 跨逻辑层次重构
 - 考虑带 `dont_touch`、`size_only` 或 `fixed` 属性的单元
 - 在线长增加时接受结果
- 但若设计无拥塞或布线问题,可将 `opt.common.advanced_logic_restructuring_wirelength_costing` 设为 `medium` 或 `none`（默认 `high`），使工具在线长增加时仍接受重构结果。

注意

该功能需要 Digital-AF license key。

2.20 设置功耗相关功能

Setting Up for Power-Related Features

以下主题说明为功耗相关功能设置设计所需的任务：

- 标注开关活动（Switching Activity）
- 为 `place_opt` 与 `clock_opt` 启用功耗优化
- 通过限制低阈值电压（LVT）单元百分比改善良率
- 更新 Activity 以改善功耗优化
- 启用电源完整性功能

2.20.1 标注开关活动

Annotating the Switching Activity

执行低功耗布局与动态功耗优化时，若为设计标注 `switching activity`，可获得更好的功耗节省。
标注方式：

- 用 `read_saif` 读取开关活动文件 (SAIF)。

Switching activity 与 scenario 相关；使用该命令时，确保当前 scenario 已启用动态功耗优化。

- 用 `set_switching_activity` 设置 switching activity。

可用 `-mode`、`-corner` 或 `-scenario` 为特定 mode、corner 或 scenario 设置；并确保所指定（或对应）的 scenario 已启用动态功耗优化。

若未指定 switching activity，工具对主输入与 black box 输出应用默认 toggle rate，并在设计中传播。

报告 block 的 switching activity 使用 `report_activity`；移除特定 net、pin、port 或整个 block 的 switching activity 使用 `reset_switching_activity`。

结合 **Name-Mapping** 文件使用 **RTL Switching Activity**

Using RTL Switching Activity With a Name-Mapping File

要在 Fusion Compiler 中使用 RTL SAIF 文件，须先在 Design Compiler 中完成：

1. 用 `read_saif` 读入 RTL SAIF。
2. 优化前用 `saif_map -start` 创建 name-mapping 数据库，跟踪优化中对象名变更。
3. 执行优化及其他设计变更。
4. 用 `saif_map -write_map` 生成将 RTL 对象名映射到门级对象的 name-mapping 文件。

然后在 Fusion Compiler 中：

1. 用 `saif_map -read_map` 读入 Design Compiler 的 name-mapping 文件，创建 name-mapping 数据库并开始跟踪对象名变更。
若不读 name-mapping 文件、但希望跟踪 Fusion Compiler 内的网表变更，可用 `saif_map -start` 创建数据库并开始跟踪。
2. (可选) 按「控制名称映射」手动修改名称映射。
3. 用 `read_saif` 读入同一 RTL SAIF（不影响 name-mapping 数据库）。
4. 执行优化及其他设计变更。

控制名称映射

Controlling the Name Mapping

在 SAIF-map 流程中，name-mapping 数据库跟踪优化期间 pin、port、单元与 net 名称变更，保存各对象的原始名称，使优化后仍可应用原始 SAIF。

可用 `saif_map` 的选项手动控制数据库中的名称映射，见下表。

缩放开关活动

Scaling the Switching Activity

若 SAIF 文件中的时钟频率与 Fusion Compiler 中不同，可按下列步骤缩放 switching activity：

1. 用 `read_saif` 读入 SAIF。

表 2.21: 手动控制名称映射的选项（原书 Table 13）

目的	saif_map 选项
为对象指定名称映射并覆盖数据库中可能已有的映射	-set_name
向数据库已有映射追加名称映射	-add_name
将 -set_name 或 -add_name 指定的映射应用于逻辑取反对象	与 -set_name/-add_name 联用 -inverted
读入 SAIF 时用 name rule 更改对象名	-change_name
报告全部手动名称映射设置	-report
移除特定对象的名称映射设置	-remove_name
清除全部手动创建的名称映射与 name rule 以重置	-reset
从数据库检索名称映射设置	-get_saif_names、-get_object_names

2. 将 power.use_generated_clock_scaling_factor 设为 on 以启用缩放。
3. 用 set_power_clock_scaling 缩放 switching activity。须指定：
 - 与待缩放 switching activity 关联的时钟对象
 - 用 -period 指定 SAIF 中使用的时钟周期，或用 -ratio 指定 SAIF 周期与 Fusion Compiler 周期之比

另可用 -scenario 指定应用缩放的 scenario。

下列示例读入 SAIF、启用缩放，将 CLK1/CLK2 关联活动按比例 5 缩放，将 CLK3 按比例 2 缩放：

```
fc_shell> read_saif top.saif
fc_shell> set_app_options -list \
    {power.use_generated_clock_scaling_factor true}
fc_shell> set_power_clock_scaling -ratio 5 {CLK1 CLK2}
fc_shell> set_power_clock_scaling -ratio 2 {CLK3}
```

若对同一时钟再次运行 set_power_clock_scaling，工具会在已缩放结果上继续缩放。
使用该命令时，工具仅缩放由 read_saif 应用的 switching activity；不缩放：

- 由 set_switching_activity 应用的 switching activity
- block abstract 内的 switching activity

缩放后的 switching activity 在设计中持久保存，可用 write_saif 写出并用于后续流程。

为供电网络指定开关概率

Specifying Switching Probability for Supply Nets

block 的漏电功耗会按其 supply net 的开关概率缩放，该概率表示 supply net 处于 on 状态的时间比例。用 set_supply_net_probability -static_probability 为一个或多个 supply net 指定开关概率。默认应用于当前 scenario 及对应 mode 与 corner；用 -modes、-corners 或 -scenarios 指定作用范围。获取用 get_supply_net_probability，重置用 reset_supply_net_probability。

默认工具经电源开关传播 supply net activity，并根据 UPF 电源开关约束确定开关后 supply net 的静态概率。例如：

```
create_power_switch my_switch \
  -output_supply_port {vout VDDS} \
  -input_supply_port {vin VDD} \
  -control_port {ms_sel ctrl1} \
  -control_port {ms_ctrl ctrl2} \
  -on_state {on vin {ms_ctrl && !ms_sel}}
```

工具根据电源开关处于 on 的概率推导连接到开关输出的 VDDS supply net 静态概率，依据包括：

- -on_state 指定的布尔函数 (ms_ctrl && !ms_sel)，以及对应控制 port 所连 net (ctrl1、ctrl2) 的 switching activity (静态概率)
- -on_state 中输入供电 port 所连 supply net (VDD) 的开关概率

下列应用选项控制是否根据供电开关活动缩放动态与漏电功耗：

- power.scale_dynamic_power_at_power_off: 是否缩放动态功耗。默认 false (不缩放)。
- power.scale_leakage_power_at_power_off: 是否缩放漏电功耗。默认 true (缩放)。

2.20.2 为 place_opt 与 clock_opt 启用功耗优化

Enabling Power Optimization for the place_opt and clock_opt Commands

Fusion Compiler 可优化动态与静态 (漏电) 功耗。

- 动态功耗：设计对象 (单元、pin、net) 中电压或逻辑翻转所耗散的能量，与翻转次数及频率成正比。
- 静态 (漏电) 功耗：电路无翻转时仍耗散的能量，亦称 leakage power，取决于器件特性。主要贡献是亚阈值漏电。在较低工艺节点，漏电在总功耗中占显著份额。

以下主题说明如何启用工具执行的各类功耗优化：

- 执行传统漏电功耗优化
- 执行动态功耗优化
- 执行总功耗优化

执行传统漏电功耗优化

Performing Conventional Leakage-Power Optimization

布线前优化阶段设置传统漏电功耗优化：

1. 用 set_scenario_status -leakage_power true 确保至少一个 scenario 启用漏电功耗优化；并用 set_qor_strategy -metric leakage_power | total_power 决定功耗流程目标。
2. (可选) 用 set_qor_strategy -mode 将 effort 设为 balanced 或 extreme_power (默认 balanced)，作为功耗流程的 effort 控制。

使用这些设置后，工具在 place_opt 与 clock_opt 期间执行传统漏电功耗优化，但不做动态功耗优化。

执行动态功耗优化

Performing Dynamic-Power Optimization

布线前优化阶段设置动态功耗优化：

1. 按「标注开关活动」为设计标注 switching activity。
2. 用 `set_scenario_status -dynamic_power true` 确保至少一个 scenario 启用动态功耗优化。

使用这些设置后，工具在 `place_opt` 与 `clock_opt` 期间执行动态功耗优化，但不做漏电功耗优化。

执行总功耗优化

Performing Total-Power Optimization

总功耗优化在优化中同时考虑漏电与动态功耗代价。设置步骤：

1. 设置漏电功耗优化：用 `set_scenario_status -leakage_power true` 确保至少一个 scenario 启用。
2. 设置动态功耗优化：
 - (a) 按「标注开关活动」标注 switching activity。
 - (b) 用 `set_scenario_status -dynamic_power true` 确保至少一个 scenario 启用。

使用该设置后，工具在 `place_opt` 与 `clock_opt` 期间执行总功耗优化。

2.20.3 通过限制 LVT 单元百分比改善良率

Improving Yield By Limiting the Percentage of Low-Threshold-Voltage (LVT) Cells

Fusion Compiler 可执行基于 LVT 百分比的优化。限制 block 中 LVT 单元百分比可改善良率；也可在早期设计阶段用于评估 RTL 质量。

设置基于 LVT 百分比的优化：

1. 指定 LVT 库单元组，例如：

```
fc_shell> remove_attributes -quiet \
[get_lib_cells -quiet */*] threshold_voltage_group
fc_shell> set_attribute [get_lib_cells -quiet *lvt*/*] \
threshold_voltage_group LVT
fc_shell> set_threshold_voltage_group_type \
-type low_vt LVT
```

确保所指定的 LVT 库单元：

- 已用 `set_lib_cell_purpose` 定义有效 purpose，并包含在 `set_target_library_subset` 定义的相应 target-library subset 中
- 未设置 `dont_touch` 或 `dont_use`

2. 用 `set_multi_vth_constraint -low_vt_percentage` 指定 LVT 单元百分比上限。

以单元数量百分比计用 `-cost cell_count`；以单元面积百分比计用 `-cost area`。

重置用 `reset_multi_vth_constraint`，报告用 `report_multi_vth_constraint`。

设置完成后，运行 `place_opt`、`clock_opt` 或 `route_opt` 时，工具对 block 中数据路径单元使用该 LVT 百分比约束。为尽量减小 QoR 扰动，`route_opt` 期间的基于 LVT 百分比优化力度有限，因此应尽早在此流程中启用。

注意

LVT 单元延时更小，但漏电功耗更高。限制 LVT 数量可降低漏电；要进一步降低漏电，请在流程各阶段启用漏电功耗优化。

2.20.4 更新 Activity 以改善功耗优化

Updating Activity for Improved Power Optimization

Fusion Compiler 中的 PrimePower In-Design 流程使用 RTL FSDB 的输入波形信息，执行基于时间的分析以更新 activity，提高功耗估计精度。

用 `set_indesign_primepower_options` 设置 In-Design 流程的 PrimePower 分析选项。必须用 `-fsdb` 指定 RTL activity 文件，用 `-pwr_shell` 指定 PrimePower 可执行文件路径。其他选项可指定 FSDB 细节、分布式/并发处理设置、scenario 与库设置，以及输出文件。

用 `update_indesign_activity` 执行分析，包括：

- 调用 PrimePower，读入网表、库、PVT、约束、寄生、必要 name mapping、UPF 等设计数据。库细节来自 Fusion Compiler 内存中的设计数据库。
- 将 RTL FSDB 读入 PrimePower，执行基于时间的分析以生成门级 SAIF。
- 将门级 SAIF 读回 Fusion Compiler。

启动 `update_indesign_activity` 前，将功耗 scenario 设为当前 scenario；activity 对当前 scenario 刷新。

若用 `set_indesign_primepower_options` 的 `-scenarios` 指定多个 scenario，则从 PrimePower 刷新的门级 SAIF 会读回所有指定 scenario。

PrimePower In-Design UI 允许用原生 ZeBu 数据库 (ZTDB) 传递输入波形。`set_indesign_primepower_options` 的 `-ztdb` 选项配置 PrimePower In-Design，以用 `update_indesign_activity` 创建脚本；该脚本使用 PrimePower 的 `read_ztdb` 做基于时间的分析，并生成自动回标到 Fusion Compiler 的门级 SAIF。

指定分布式分析

Specifying Distributed Analysis

为加快 runtime，可使用分布式分析（PrimePower 中默认禁用）。

在 `set_indesign_primepower_options` 中指定：

- `-num_processes`：启动的主机数，须大于 1。
- `-submit_command` 或 `-host_names`：主机配置。
- `-max_cores`：最大核数，默认 4。

选项有效时工具打印：

Information: PrimePower is being called in distributed mode

2.20.5 启用电源完整性功能

Enabling the Power Integrity Features

Fusion Compiler 提供下列技术保障设计电源完整性：

- 动态功耗整形（Dynamic Power Shaping, DPS）：利用并发时钟与数据优化（useful skew）降低瞬态功耗。
- 压降感知布局：用单元电压（IR）drop 值识别高功耗密度区，并摊开高压降单元以降低该区功耗密度。

设置推荐电源完整性流程：

1. 将 `opt.common.power_integrity` 设为 `true` 以启用。启用后，Fusion Compiler 在不同阶段使用：
 - (a) `place_opt` 的 `final_opto` 阶段：动态功耗整形（DPS）
 - (b) `clock_opt`：压降感知布局
2. （可选）将 `opt.common.power_integrity_effort` 设为 `low`，降低电源完整性 effort，使工具不以牺牲时序 QoR 为代价降低压降。
默认为 `high`：默认工具尝试将压降降至电源电压的 8% 以下，即使牺牲时序 QoR。
3. （可选）用 `opt.common.ir_drop_threshold` 指定最大压降阈值（默认电源电压的 8%）。若将 `opt.common.power_integrity_effort` 设为 `low`，工具忽略该阈值。
4. 按「设置动态功耗整形」指定 DPS 设置。
5. 按「设置压降感知布局」指定相关设置。
6. （可选）启用 IR 驱动 sizing：用 RedHawk 动态压降分析结果识别压降违例相关单元，并尝试用更小漏电电流的单元替换。除将 `opt.common.power_integrity` 设为 `true` 外，还将 `clock_opt.flow.enable_irdrivenopt` 设为 `true`。

也可不使用推荐电源完整性流程设置，而按「手动启用动态功耗整形与压降感知布局」为 `place_opt` 或 `clock_opt` 手动启用。

设置动态功耗整形

Setting Up for Dynamic Power Shaping

1. 确保为多电源 net 指定了 UPF 设置。
2. 确保已创建并激活 `setup` 与 `hold scenario`。
3. （可选）如下控制 DPS 期间的功耗分析与优化：
 - 用 `ccd.dps.focus_power_scenario` 指定关注的 `scenario`。默认关注全部活动 `scenario`；设计有大量活动 `scenario` 时，指定一两个可缩短 runtime，例如：

```
fc_shell> set_app_options -name \
    ccd.dps.focus_power_scenario -value {{S1 S2}}
```

- 用 `ccd.dps.use_case_type` 控制内部功耗分析类型。
默认 `autovector`：以 20% 翻转触发器概率运行内部功耗分析，且所有触发器每周期有效时钟；不建模时钟门控。
设为 `activity` 时，工具根据 `activity` 选择翻转概率与时钟门控模型。
- 用 `ccd.dps.optimize_power_target_modes` 定义总体动态功耗优化目标：
 - `global_peak`：降低全局峰值电流
 - `stepped_psgs`（默认）：在设计上步进窗口降低局部峰值电流，以降低压降违例
- 用 `ccd.dps.optimize_setup_tradeoff_level` 指定功耗与 `setup` 时序的折中：`low`、`medium`（默认）或 `high`。
- 用 `ccd.dps.optimize_hold_tradeoff_level` 指定功耗与 `hold` 时序的折中：`low`、`medium`（默认）或 `high`。

还可用 `ccd.dps.use_cases`、`ccd.dps.auto_targets`、`ccd.dps.auto_target_constraint_parameters` 等更复杂的应用选项进一步控制；详见对应 `man` 页。

设置压降感知布局

Setting Up for Voltage-Drop-Aware Placement

1. 用 `rail.analysis_type` 指定压降分析类型。有效值：`static`、`dynamic`（默认）、`dynamic_vcd`、`dynamic_vectorless`。
2. 按「执行压降分析」指定 RedHawk 压降分析所需设置。

手动启用动态功耗整形与压降感知布局

Manually Enabling Dynamic Power Shaping and Voltage-Drop-Aware Placement

手动启用：

- 对 `place_opt` 启用 DPS：

```
fc_shell> set_app_option -name opt.common.power_integrity \
    -value false
fc_shell> set_app_option -name place_opt.flow.enable_dps \
    -value true
```

- 对 `place_opt` 启用压降感知布局：

```
fc_shell> set_app_option -name opt.common.power_integrity \
    -value false
fc_shell> set_app_option -name place_opt.flow.enable_irap \
    -value true
```

- 对 `clock_opt` 启用压降感知布局：

```
fc_shell> set_app_option -name opt.common.power_integrity \
-value false
fc_shell> set_app_option -name clock_opt.flow.enable_irap \
-value true
```

2.21 指定布线资源

Specifying the Routing Resources

可为 **block**（全局层约束）、特定 **net**（**net** 专用层约束）以及未综合的时钟 **net**（时钟树层约束）指定最小与最大布线层。若同时指定全局层约束与 **net** 专用层约束，**net** 专用约束覆盖全局约束。

除约束布线层外，还可为每层指定优选布线方向。布线层约束用于 RC 估计、拥塞分析与布线。因其同时影响 RC 估计、拥塞分析与布线，应在对 **block** 执行布局前设置。

以下主题说明如何指定布线层约束、优选布线方向与 **via ladder**：

- 指定全局层约束
- 指定 Net 专用层约束
- 指定时钟树层约束
- 启用基于 Level 的时钟非默认布线规则
- 为层设置优选布线方向

2.21.1 指定全局层约束

Specifying the Global Layer Constraints

用 **set_ignored_layers** 指定全局层约束。默认全局层约束用于 RC 估计、拥塞分析，并作为布线的软约束。可用下列选项设置约束并更改默认行为。

- 用 **-min_routing_layer** 与 **-max_routing_layer** 指定最小与最大布线层（使用工艺文件中的层名）。

例如，布线、RC 估计与拥塞分析使用 M2 至 M7：

```
fc_shell> set_ignored_layers \
-min_routing_layer M2 -max_routing_layer M7
```

- 仅允许为 pin 连接使用超出最小/最大布线层的层：将 **route.common.global_min_layer_mode** 与 **route.common.global_max_layer_mode** 设为 **allow_pin_connection**。
- 将约束改为硬约束：将上述两应用选项设为 **hard**。
- 用 **-rc_congestion_ignored_layers** 指定 RC 估计与拥塞分析中额外忽略的层。

所指定层必须在最小与最大布线层之间。例如，布线用 M2–M7，RC 估计与拥塞分析用 M3–M7：

```
fc_shell> set_ignored_layers \
-min_routing_layer M2 -max_routing_layer M7 \
-rc_congestion_ignored_layers {M2}
```

- 更改已有层约束时，只需重新设置该选项。
重置某选项仅覆盖该选项现有值，其他选项不变。
例如，已将最小层设为 M2、最大层设为 M7，要将最大层改为 M8 并保留其他设置：

```
fc_shell> set_ignored_layers -max_routing_layer M8
```

报告全局层约束

Reporting Global Layer Constraints

用 `report_ignored_layers` 报告被忽略的层，例如：

```
fc_shell> report_ignored_layers
*****
Report : Ignored Layers
Design : my_design
Version: J-2014.12
Date   : Wed Oct 22 15:58:23 2014
*****
Layer Attribute          Value
-----
Min Routing Layer        M2
Max Routing Layer        M7
RC Estimation Ignored Layers  P0 M1 M2 M8 M9 MRDL
1
```

移除全局层约束

Removing Global Layer Constraints

用 `remove_ignored_layers` 移除全局层约束，必须指定下列一个或多个选项：

- `-min_routing_layer`: 移除最小布线层设置；同时移除低于该层、用于 RC 估计与拥塞分析的忽略层。
- `-max_routing_layer`: 移除最大布线层设置；同时移除高于该层、用于 RC 估计与拥塞分析的忽略层。
- `-all`: 移除最小与最大布线层之间、用于 RC 估计与拥塞分析的忽略层。
- `-rc_congestion_ignored_layers layer_list`: 移除指定的 RC/拥塞忽略层（须在最小与最大布线层之间）。

2.21.2 指定 Net 专用层约束

Specifying Net-Specific Layer Constraints

用 `set_routing_rule` 指定 net 专用层约束；用 `-min_routing_layer` 与 `-max_routing_layer` 指定最小与最大布线层（工艺文件层名）。

例如，布线 n1 时使用 M2 至 M7：

```
fc_shell> set_routing_rule [get_nets n1] \  
-min_routing_layer M2 -max_routing_layer M7
```

默认 net 专用最小层约束为软约束，最大层约束为硬约束。可如下更改默认行为：

- 用 `route.common.net_min_layer_mode` 与 `route.common.net_max_layer_mode` 更改全部 net 专用层约束的默认强度。
设为 `soft` 表示软约束，`allow_pin_connection` 允许仅对 pin 连接使用更低层，`hard` 表示硬约束。
- 用 `set_routing_rule` 定义约束时的 `-min_layer_mode` 与 `-max_layer_mode` 更改单个 net 专用约束的强度（取值同上）。
- 用 `route.common.net_min_layer_mode_soft_cost` 与 `route.common.net_max_layer_mode_soft_cost` 设置全部 net 专用软约束违例代价。
取值为 `low`、`medium`（默认）或 `high`。代价控制路由器避免违例的 effort；`high` 可减少违例但增加 runtime，`low` 可缩短 runtime 但可能增加违例。
- 用 `set_routing_rule` 的 `-min_layer_mode_soft_cost` 与 `-max_layer_mode_soft_cost` 为单个 net 专用约束设置软约束违例代价（取值同上）。

移除 Net 专用布线层约束

Removing Net-Specific Routing Layer Constraints

用 `set_routing_rule -clear` 移除 net 专用布线层约束。注意：该命令也会移除赋给指定 net 的任何非默认布线规则。

2.21.3 指定时钟树层约束

Specifying Clock-Tree Layer Constraints

用 `set_clock_routing_rules` 指定时钟树层约束；用 `-min_routing_layer` 与 `-max_routing_layer` 指定最小与最大布线层（工艺文件层名）。

默认该命令将指定的层约束赋给全部时钟树。限制赋给范围的选项见表 2.22。

表 2.22: 限制时钟树层约束（原书 Table 14）

将层约束赋给	使用的选项
特定时钟树	-clocks <i>clocks</i>
连接到时钟 root 的 net ¹	-net_type root
连接到一个或多个时钟 sink 的 net ¹	-net_type sink
时钟树中的内部 net（除 root 与 sink 外全部） ¹	-net_type internal
特定时钟 net	-nets <i>nets</i>

¹ 可与 `-clocks` 联用进一步限制赋值；不可与 `-nets` 联用。

图 2.26 示出 CTS 后时钟树的 root、internal 与 sink net。默认 root-net 层约束应用于从时钟 root 起、直至时钟树分支到扇出大于 1 之前的全部单扇出时钟 net；此后直至 sink net 应用 internal-net 层约束。

figures/fig-ch2b-clock-net-types.png

图 2.26: Root、Internal 与 Sink 时钟 Net 类型

例如，布线 CLK1 的 root net 时使用 M4 至 M7：

```
fc_shell> set_clock_routing_rules -clocks CLK1 -net_type root \  
-min_routing_layer M4 -max_routing_layer M7
```

识别 rootnet 时可用 `set_clock_tree_options -root_ndr_fanout_limit` 指定传递扇出上限。例如，传递扇出超过 300 个 sink 的时钟 net 视为 root net：

```
fc_shell> set_clock_tree_options -root_ndr_fanout_limit 300
```

图 2.27 示出指定传递扇出上限为 300 时同一时钟树的 root、internal 与 sink net。

figures/fig-ch2b-root-fanout.png

图 2.27: 用扇出限制选择 Root Net

计算传递扇出以识别 root net 时，工具仅计入有效时钟 sink，不包括 ignore pin。若被识别为 root 的 net 短于 10 micron，工具对该 net 使用 internal-net 层约束。

注意

用较小的 `set_clock_tree_options -root_ndr_fanout_limit` 值会增加被赋 `root-net` 层约束的时钟 `net` 数量，可能加剧布线拥塞。

移除该传递扇出限制，使用 `remove_clock_tree_options -root_ndr_fanout_limit`。

2.21.4 启用基于 Level 的时钟非默认布线规则**Enabling Level-Based Clock Nondefault Routing Rule**

时钟树综合期间，可按时钟树不同 level 为 `net` 分配连续的非默认布线规则。工具仅在初始 CTS 时实施变更，并按下列方式分配 NDR：

- 仅连接到纯 sink 的 `net`：sink NDR
- 同时连接到 sink 与中间单元（repeater 或时钟门）的 `net`：internal NDR
- 从 root 级直至下游分支点的 `net`：root NDR

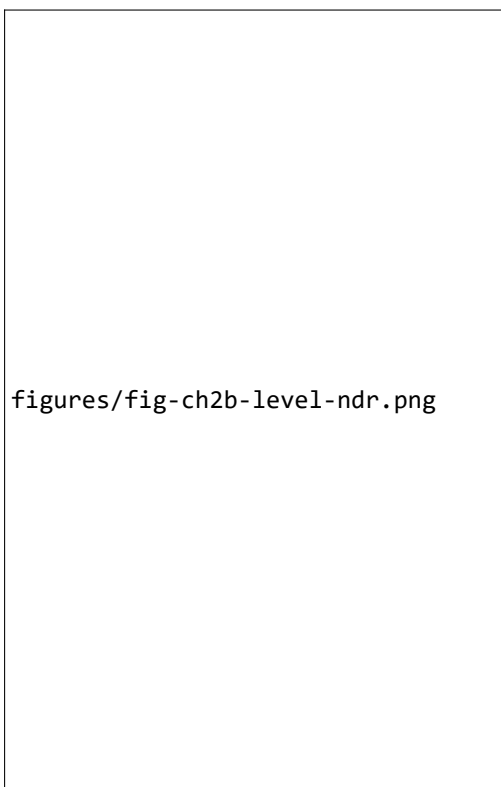


图 2.28: 基于 Level 的时钟非默认布线规则

如图所示，CTS 后：

- 未启用基于 level 的时钟 NDR：n1 与 n3 因至少连接一个 sink，编译时钟树后须获得 sink NDR；这会使不同 level 上出现不连续的 NDR。
- 启用基于 level 的时钟 NDR：时钟树路径上全部 NDR 须按拓扑从时钟 root 到 sink 排序，即 Root NDR > Internal NDR > Sink NDR。因此 n1、n2 赋 internal NDR，n3 赋 sink NDR。

启用该功能：

```
fc_shell> set_app_options -list {cts.compile.topological_ndr true}
```

2.21.5 为层设置优选布线方向

Setting the Preferred Routing Direction for Layers

Fusion Compiler 要求为工艺文件中定义的布线层指定优选布线方向。通常该信息在单元库中定义。要设置或更改某层的优选布线方向，按下列语法设置其 `routing_direction` 属性：

```
set_attribute -objects layers  
-name routing_direction  
-value vertical | horizontal
```

用工艺文件中的层名指定布线层。该属性设置的层方向仅作用于当前 block。

例如，将 M5 与 M7 的优选方向设为 vertical：

```
fc_shell> set_attribute -objects [get_layers {M5 M7}] \  
-name routing_direction -value vertical
```

注意

用 `create_routing_guide -switch_preferred_direction` 在 routing guide 覆盖区域内更改优选方向的设置，会覆盖 `routing_direction` 属性设置。

报告一个或多个布线层的用户定义优选方向，使用 `get_attribute`；移除则使用 `remove_attributes`。

参见

准备布线层。

2.22 使用 Early Data Check Manager 处理设计数据

Handling Design Data Using the Early Data Check Manager

设计周期早期迭代中，设计数据可能不完整或不正确，从而妨碍探索性实现等。Fusion Compiler Early Data Check Manager 允许指定工具在下列实现任务中如何处理数据检查：

- 设计规划
- 多电压设计实现
- 优化
- 布局
- 合法化
- 含物理层次设计的层次化实现

表 2.23: Early Data Check Manager 命令（原书 Table 15）

目的	使用的命令
指定一个或多个数据检查的处理策略	set_early_data_check_policy
获取数据检查违例报告	report_early_data_checks
获取可在其他命令中使用的违例 Tcl 集合	get_early_data_check_records
将设置保存到文件供日后使用	write_early_data_check_config
清除数据检查设置以便下一轮迭代	remove_early_data_check_records

下表列出用于控制数据检查处理方式及生成相关信息的 Early Data Check Manager 命令。

应在开始实现前指定数据检查处理策略。

下列示例指定对 SCANDEF 信息检查采用严格策略，因此因缺少 SCANDEF，工具不会继续执行 create_placement:

```
fc_shell> set_early_data_check_policy -policy strict \  
-check place.coarse.missing_scan_def  
fc_shell> create_placement  
Information: Policy for early data check 'place.coarse.missing_scan_def'  
is 'error'. (EDC-001)  
Error: No valid scan def found. (PLACE-042)  
Information: Ending 'create_placement' (FLW-8001)  
fc_shell> report_early_data_checks -check place.coarse.missing_scan_def  
Check Policy Strategy Fail Count  
-----  
place.coarse.missing_scan_def error 1  
-----
```

下列示例对所有与粗布局相关的数据检查采用宽松策略，因此即使无 SCANDEF，工具仍继续 create_placement:

```
fc_shell> set_early_data_check_policy -policy lenient \  
-check place.coarse.*  
fc_shell> create_placement  
Information: Policy for early data check 'place.coarse.missing_scan_def'  
is 'tolerate'. (EDC-001)  
Continuing without valid scan def.  
Start transferring placement data.  
Creating placement from scratch.  
...
```

更多信息见 *Fusion Compiler Data Model User Guide*。

2.23 应用 Mega-Switch 命令设置

Applying Mega-Switch Command Settings

可用 mega-switch 命令，以单一命令为工具流程不同阶段应用适用于特定情形的设置：

- 应用先进工艺节点所需设置

- 应用高性能核所需设置
- 应用改善特定 QoR 指标所需设置

2.23.1 应用先进工艺节点所需设置

Applying Required Settings for Advanced Technology Nodes

要对 12 nm 或 7 nm 工艺节点应用特定于布局、合法化、布线与提取的设置, 使用 `set_technology` 的 `-node 12` 或 `-node 7`。

使用该命令时, 在布局、优化与布线前执行:

1. 加载并 link 设计。
2. 用 `set_technology -node` 应用工艺节点所需设置, 例如:

```
fc_shell> set_technology -node 7
```

3. (可选) 用 `set_technology -report_only` 报告工艺节点设置。
4. 应用设计专用应用选项或命令设置, 以覆盖 `set_technology` 的通用设置。

用 `save_lib` 保存设计库时, 工艺节点设置一并保存。

注意

用 `set_technology` 指定工艺节点后, 不可再更改。

该命令支持哪些 ASIC 工艺, 请联系 Synopsys Support。

2.23.2 应用高性能核所需设置

Applying Required Settings for High Performance Cores

可用 `set_hpc_options` 应用为实现高性能核最佳 QoR 所需的工具设置。所应用设置影响布局、合法化、优化、时钟树综合、布线、提取与时序分析。

- 列出支持的高性能核类型与实现阶段: 用 `-list`
- 指定高性能核类型: 用 `-core`
- 指定实现流程阶段: 用 `-stage`

若设计使用 12 nm 或 7 nm 节点, 须在运行 `set_hpc_options` 前先用 `set_technology` 应用节点专用设置, 示例如下:

```
set_technology -node 7
set_hpc_options -core A72 -stage clock_opt_cts
...
clock_opt -from build_clock -to route_clock
...
set_hpc_options -core A72 -stage clock_opt_opto
...
clock_opt -from final_opto
```

```

...
set_hpc_options -core A72 -stage route_auto
...
route_auto
...
set_hpc_options -core A72 -stage route_opt
...
route_opt
...

```

2.23.3 应用改善特定 QoR 指标所需设置

Applying Required Settings for Improving Specific QoR Metrics

要用 `set_qor_strategy -stage pnr` 应用改善特定 QoR 指标所需的工具设置。

用 `-metric` 指定要改善的 QoR 指标。为获得最佳：

- 时序：使用 `timing`
- 漏电功耗与时序：使用 `leakage_power`
- 总功耗与时序：使用 `total_power`。
为进一步改善总功耗（以 `runtime` 为代价），可与 `-metric total_power` 联用 `-mode extreme_power`。
用 `-mode` 指定设计流程目标模式。支持取值：
 - `balanced`：默认模式，针对 `-metric` 指定的目标指标优化设计。
 - `early_design`：适用于设计流程早期、侧重短周转时间原型的阶段。
该模式可以 QoR 为代价缩短 `runtime`。
 - `extreme_power`：建议用于以 `runtime` 为代价进一步改善总功耗。
仅应与 `-metric leakage_power` 或 `-metric total_power` 联用。若与 `-metric timing` 联用，工具忽略该 `mode` 并使用 `balanced`。

使用该命令时，工具应用影响优化、布局、合法化、时钟树综合、布线、提取与时序分析的应用选项设置。

查看所需设置可用：

- 用 `-output` 生成包含这些设置的 Tcl 脚本
- 用 `-report_only` 报告设置
- 用 `-diff_only` 仅报告未设为所需值的设置

Chapter 3

物理综合

Physical Synthesis

Fusion Compiler 工具将 RTL 描述综合为优化后的门级设计，并对其进行优化，以在速度、面积与功耗方面取得最佳结果质量（quality of results, QoR），同时支持早期设计探索与原型（prototyping）。

本节主题包括：

- 执行统一物理综合（Performing Unified Physical Synthesis）
- 控制映射与优化（Controlling Mapping and Optimization）
- 执行多比特优化（Performing Multibit Optimization）
- 运行并发时钟与数据优化（Running Concurrent Clock and Data Optimization）
- 执行测试插入与扫描综合（Performing Test Insertion and Scan Synthesis）
- 指定性能、功耗与面积改善设置（Specifying Settings for Performance, Power, and Area Improvement）
- 执行设计分析（Performing Design Analysis）

3.1 执行统一物理综合

Performing Unified Physical Synthesis

Fusion Compiler 工具执行统一物理综合（unified physical synthesis）。其收益包括：

- 缩短运行时间
统一物理综合精简布局与优化的迭代，并利用综合期间构建的初始布局与缓冲树，以获得最佳运行时间。
- 改善结果质量（QoR）
统一的布线前（preroute）优化框架在优化算法中使用一致的代价（cost），从而改善 QoR。逻辑重构（logic restructuring）、并发时钟与数据优化（concurrent clock and data optimization）、多比特映射（multibit mapping）以及高级合法化器（advanced legalizer）等最佳技术，在整个布线前流程中共享。

Fusion Compiler 编译同时支持自顶向下 (top-down) 与自底向上 (bottom-up, 层次化) 编译策略。在自顶向下编译中, 所有环境与约束设置均相对于顶层设计定义。自顶向下编译提供一键式 (push-button) 方法, 并自动处理块间依赖, 但要求所有设计必须同时驻留在内存中。在层次化编译流程中, 先对设计子块采用自顶向下方式开始优化, 再将子块逐级并入更高层块, 直至顶层设计完成。

以下主题说明如何执行统一物理综合:

- 使用 `compile_fusion` 命令
- 执行预检查 (Performing Prechecks)
- 编译期间生成验证检查点 (Generating Verification Checkpoints During Compilation)

3.1.1 使用 `compile_fusion` 命令

Using the `compile_fusion` Command

要执行统一物理综合, 使用 `compile_fusion` 命令。

`compile_fusion` 命令包含以下阶段:

1. `initial_map`

此阶段工具映射设计并执行面积优化。

2. `logic_opt`

此阶段工具执行基于逻辑的延迟优化。

3. `initial_place`

此阶段工具合并时钟门控 (clock-gating) 逻辑并执行粗布局 (coarse placement)。

4. `initial_drc`

此阶段工具移除已有缓冲树, 并执行高扇出 net 综合 (high-fanout-net synthesis) 与电气 DRC 违例修复。

5. `initial_opto`

此阶段工具执行物理优化与增量布局。

6. `final_place`

此阶段工具执行最终布局以改善时序与拥塞。

7. `final_opto`

此阶段工具执行最终优化与合法化, 以改善时序、功耗与逻辑 DRC。

运行 `compile_fusion` 时, 默认执行全部阶段。若只运行其中部分阶段, 用 `-from` 指定起始阶段, 用 `-to` 指定结束阶段 (之后停止)。若不指定 `-from`, 工具从 `initial_place` 阶段开始; 若不指定 `-to`, 工具一直运行到 `final_opto` 阶段完成。

将 `compile_fusion` 拆分为多个阶段, 可在阶段之间执行不同任务, 例如更改约束或设置、生成分析报告、执行设计规划 (design planning) 或 DFT 插入等其他实现任务。若在阶段之间对手工修改设计, 请确保修改符合该阶段所期望的设计状态。例如, `initial_map` 之后设计应已映射, 因此不要在该阶段之后向设计引入未映射逻辑。

可以重复某一阶段; 但请确保按正确顺序运行所有阶段。下例运行 `compile_fusion` 直至完成 `initial_opto`, 进行分析并更改应用选项设置, 然后从 `initial_opto` 再次启动:


```
fc_shell> compile_fusion -to initial_opto
fc_shell> report_qor
fc_shell> set_app_option \
    -name opt.common.advanced_logic_restructuring_mode -value timing
fc_shell> compile_fusion -from initial_opto
```

在下列情况下，`compile_fusion` 之后设计中可能仍含未映射单元（unmapped cells）：

- 库中缺少所需单元
- 用户对所需库单元施加了限制

当设计含未映射单元时，`compile_fusion` 会继续编译，并为未映射单元提供估计面积。

未映射单元标有 `is_unmapped` 属性。要在网表中查询未映射单元，使用：

```
fc_shell> get_cells -hierarchical -filter "is_unmapped==true"
```

注意下列限制：

- DesignWare 算子（operators）不会被缓解（mitigated）。
- 缓解后仅布局信息保存在数据库中；布线信息不保存。

3.1.2 执行预检查

Performing Prechecks

Fusion Compiler 可执行编译预检查（compile prechecks），以防止编译失败，并在遇到不完整（或不一致）的设置或约束问题时，使 `compile_fusion` 能快速以错误消息退出。该能力默认开启；也可用 `compile_fusion -check_only` 启用这些检查。

`compile_fusion` 与 `compile_fusion -check_only` 执行下表所列预检查，并据此发出错误消息。勾选标记（X）表示执行该预检查。

编译预检查示例

Compile Precheck Examples

以下两例分别展示启用与未启用编译预检查时的编译日志。设置 `-check_only` 时，`compile_fusion` 对任何失败的预检查条件发出错误消息。

```
----- Begin compile flow -----
(FLW-3001)
->Compile in progress:   1.0%                (FLW-3778)
Error: Layer M1 does not have a preferred direction (OPT-1008)
...
Error: Layer MRDL does not have a preferred direction (OPT-1008)
...
Warning: Technology layer 'M1' setting 'routing-direction' is not valid
(NEX-001)
...
Warning: Technology layer 'MRDL' setting 'routing-direction' is not valid
(NEX-001)
```

表 3.1: 编译流程期间执行的预检查

代码	消息	默认编译预检查	compile_fusion -check_only
OPT-3000	设计未链接或存在链接问题。	X	X
OPT-3001	链接后单元未解析。	X	X
OPT-3002	单元为物理层次，不允许用于编译。	X	X
OPT-3003	单元为逻辑层次，不允许用于编译。	X	X
OPT-3005	设计不存在。	X	X
OPT-3006	当前实例层次不是顶层。	X	X
OPT-2001	无法初始化 scenario 设置。	X	
OPT-2002	corner 缺少寄生信息或缺少 TLUPlus 信息。	X	
OPT-4000	设计没有 floorplan 或有效 floorplan。	X	X
OPT-4001	设计没有有效 die area。	X	X
OPT-4002	Macro 没有布局信息。	X	X
OPT-4004	设计 port 没有布局信息。	X	X
OPT-4010	连接到 port 的 pad 没有布局信息。	X	X
OPT-4008	设计未定义 site rows。	X	X
OPT-4009	设计未定义 tracks。	X	X
OPT-1008	层没有首选方向（preferred direction）。	X	
OPT-1006	库分析失败——找不到 buffer 与 inverter。	X	X
OPT-1007	库分析失败——无工艺数据。	X	X
OPT-4012	设计有 1 个重复项。运行 check_duplicates 获取更多细节。	X	

代码 3.1: 启用编译预检查的编译日志（原书 Example 14）

```
----- End logic optimization -----
(FLW-3001)
->Compile in progress: 27.0% (FLW-3778)
->Compile in progress: 28.0% (FLW-3778)
->Compile in progress: 28.5% (FLW-3778)
Information: Corner max_corner: no PVT mismatches. (PVT-032)
Warning: Technology layer 'M1' setting 'routing-direction' is not valid
(NEX-001)...
Warning: Technology layer 'MRDL' setting 'routing-direction' is not valid
(NEX-001)
Information: Design Average RC for design top (NEX-011)
```

代码 3.2: 未启用编译预检查的编译日志（原书 Example 15）

3.1.3 编译期间生成验证检查点

Generating Verification Checkpoints During Compilation

可在 compile_fusion 期间生成中间验证检查点（verification checkpoints），供 Formality 工具使用，以提供综合设计状态的网表快照并验证中间网表。验证检查点使 Fusion Compiler 与 Formality 能在中间网表上同步，从而提高完成率并改善 QoR。

要启用验证检查点，使用 `set_verification_checkpoints` 命令，如下例：

```
fc_shell> set_verification_checkpoints
Enabling checkpoint "ckpt_logic_opt"
Enabling checkpoint "ckpt_pre_map"
```

启用该功能后，默认情况下 `compile_fusion` 在映射前与逻辑优化期间生成验证检查点，对应阶段名为 `ckpt_pre_map` 与 `ckpt_logic_opt`。也可仅在一个阶段启用验证检查点，如下例：

```
fc_shell> set_verification_checkpoints {ckpt_logic_opt}
Disabling checkpoint "ckpt_pre_map"
Enabling checkpoint "ckpt_logic_opt"
```

在每个验证检查点，工具自动生成检查点网表，并在 Formality 验证所用的 `.svf` 文件中写入 `guide_checkpoint` 命令。

3.2 控制映射与优化

Controlling Mapping and Optimization

以下主题说明如何定制并控制 `compile_fusion` 命令在 `initial_map`、`logic_opt` 与 `initial_drc` 阶段所执行的映射与优化：

- 优化期间取消分组或保留层次（Ungrouping or Preserving Hierarchies During Optimization）
- 控制边界优化（Controlling Boundary Optimization）
- 控制数据通路优化（Controlling Datapath Optimization）
- 控制 MUX 优化（Controlling MUX Optimization）
- 控制时序映射（Controlling Sequential Mapping）
- 控制寄存器复制（Controlling Register Replication）
- 控制寄存器合并（Controlling Register Merging）
- 选择性移除或保留常量与无负载寄存器
- 报告优化寄存器的交叉探测信息
- 控制高扇出 net 综合

3.2.1 优化期间取消分组或保留层次

Ungrouping or Preserving Hierarchies During Optimization

取消分组（ungrouping）将给定层次级别的子设计合并到父单元或父设计中。它移除层次边界，使 Fusion Compiler 能通过减少逻辑级数改善时序，并通过共享逻辑改善面积。默认情况下，`compile_fusion` 自动取消分组未设置 `dont_touch` 属性的逻辑层次。

优化期间，`compile_fusion` 执行下列类型的自动分组：

- 基于面积的自动取消分组 (**area-based automatic ungrouping**)

在初始映射之前，命令估计未映射层次的面积并移除小的子设计。由于自动取消分组在早期阶段执行，优化上下文更好；此外，跨已取消分组的层次可启用数据通路提取 (datapath extraction)。这些因素改善面积与时序 QoR。

- 基于延迟的自动取消分组 (**delay-based automatic ungrouping**)

命令沿关键路径取消分组层次，主要用于时序优化。

关于 DesignWare 与数据通路单元的取消分组，见「数据通路实现」(Datapath Implementation)。

可通过将 `compile.flow.autoungroup` 应用选项设为 `false` 禁用自动取消分组；默认为 `true`。但这样做可能影响 QoR。因此，更好的做法是限制取消分组，而非禁用它，可采用下列方法：

控制特定类型层次的自动取消分组

Controlling the Automatic Ungrouping of Specific Types of Hierarchies

要控制特定类型层次的自动取消分组，使用 `set_autoungroup_options` 命令。

下例指定：带 UPF 与 SDC 约束的块的父层次应保留，且其他层次的自动取消分组从第三级开始：

```
fc_shell> set_autoungroup_options -keep_parent_hierarchies UPF
fc_shell> set_autoungroup_options -keep_parent_hierarchies SDC
fc_shell> set_autoungroup_options -start_level 3
```

`set_autoungroup_options` 命令是累积的，设置保存在设计库中。

要报告自动取消分组或保留的层次信息，使用 `report_ungroup` 命令。

控制特定对象的自动取消分组

Controlling the Automatic Ungrouping of Specific Objects

要强制或阻止特定单元实例或模块在优化期间被取消分组，对 `set_ungroup` 使用 `true` 或 `false`。该命令为指定对象设置相应的 `ungroup` 属性。若对模块设置该属性，则引用该模块的所有单元均受影响。

注意

`set_ungroup` 的优先级高于 `set_autoungroup_options`。

下例脚本设置特定模块在优化期间取消分组，并保留特定单元实例：

```
# Set modules A and B to be ungrouped
set_ungroup [get_modules {A B}] true
# Set cells u_C and u_D to be preserved
set_ungroup [get_cells {u_C u_D}] false
# Query the ungroup attribute
get_attribute -name ungroup -objects [get_modules {A B}]
get_attribute -name ungroup -objects [get_cells {u_C u_D}]
compile_fusion
```

3.2.2 控制边界优化

Controlling Boundary Optimization

可跨用户指定的逻辑边界控制边界优化（boundary optimization），并使物理块的接口边界与 RTL 模块接口匹配。边界优化允许在块实现期间使用 RTL floorplanning DEF 文件。

下图示出：在禁用与启用边界优化时，常量、相等-相反逻辑（equal-opposite logic）、逻辑相位（logic phase）以及无负载 port 如何穿越逻辑边界传播。

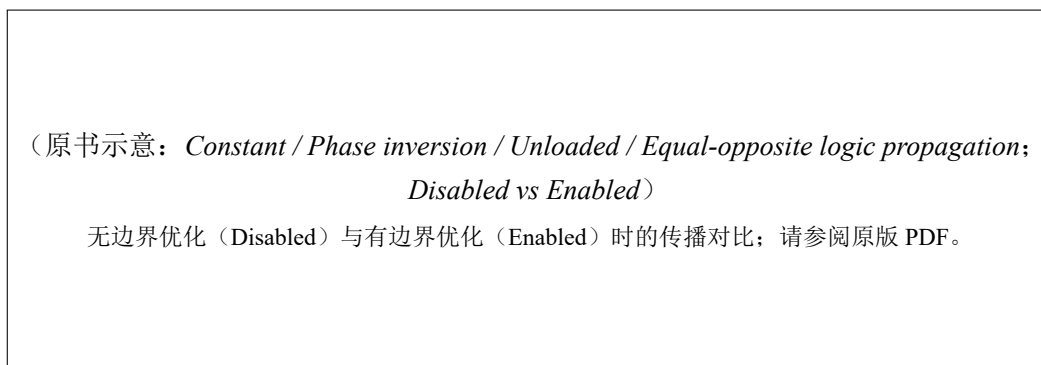


图 3.1: 边界优化启用与禁用时的传播行为

全局控制

Global Control

默认情况下，编译期间启用所有类型的边界优化。要全局控制边界优化，使用 `compile.flow.boundary_optimization` 应用选项。

将 `compile.flow.boundary_optimization` 设为 `false` 时：

- 工具禁用相等-相反逻辑传播与相位反转（phase inversion）；
- 常量与无负载 port 仍继续跨层次传播。

要全局禁用常量传播与无负载传播，将 `compile.flow.constant_and_unloaded_propagation_with_no_boundary_optimization` 设为 `false`；默认为 `true`。该设置不影响对象级规范。

控制特定类型的边界优化

Controlling Specific Types of Boundary Optimization

要控制层次单元 pin、层次单元或块的边界优化级别，使用 `set_boundary_optimization` 命令。工具按下列优先级顺序确定优化期间使用的设置：

1. 施加在层次单元实例 pin 上的设置
2. 施加在层次单元实例上的设置（同时作用于其 pin）
3. 施加在模块上的设置（同时作用于其全部单元与 pin）

下表列出可用 `set_boundary_optimization` 指定的边界优化级别，以及各级别启用的边界优化类型。

表 3.2: 边界优化级别与对应执行的边界优化类型

边界优化级别	常量传播	无负载传播	相等-相反逻辑传播	相位反转
all	Enabled	Enabled	Enabled	Enabled
auto	Enabled	Enabled	Disabled	Disabled
none	Disabled	Disabled	Disabled	Disabled

可用 `-constant_propagation`、`-unloaded_propagation`、`-equal_opposite_propagation` 与 `-phase_inversion` 选项控制对象允许的具体边界优化类型。这些选项与用 `all`、`auto` 或 `none` 指定的边界优化级别互斥，不能在同一次命令调用中对同一对象混用。若对同一对象多次使用 `set_boundary_optimization`，效果为累加。

根据启用或禁用的边界优化类型，工具将指定对象的只读属性 `constant_propagation`、`unloaded_propagation`、`equal_opposite_propagation` 与 `phase_inversion` 设为 `true` 或 `false`。若你：

- 启用全部四种类型，工具将只读属性 `boundary_optimization` 设为 `true`；
- 至少禁用一种类型，工具将只读属性 `boundary_optimization` 设为 `false`。

要报告边界优化设置，使用 `report_boundary_optimization`；要移除它们，使用 `remove_boundary_optimization`。

下例：

- 禁用 U1 单元实例的全部边界优化；
- 禁用 U2 单元实例的相等-相反逻辑传播与相位反转；
- 禁用 U3/in1 单元 pin 的常量传播与相位反转。

```
fc_shell> set_boundary_optimization [get_cells U1] none
fc_shell> set_boundary_optimization [get_cells U2] auto
fc_shell> set_boundary_optimization [get_pins U3/in1] \
    -constant_propagation false
fc_shell> set_boundary_optimization [get_pins U3/in1] \
    -phase_inversion false
```

控制相位反转

Controlling Phase Inversion

要防止边界优化将 `inverter` 移过层次边界,将 `compile.optimization.enable_hierarchical_inverter` 设为 `false`。默认该应用选项为 `true`，允许相位反转。仅当边界优化启用时，该应用选项才有效。

3.2.3 控制数据通路优化

Controlling Datapath Optimization

数据通路设计常用于含大量数据处理的应用，如 3-D、多媒体与数字信号处理（DSP）。默认运行 `compile_fusion` 时启用数据通路优化，并自动链接标准库与 DesignWare Foundation 库。

数据通路优化期间，工具执行下列任务：

- 共享（或撤销共享）数据通路算子
- 使用进位存储算术（carry-save arithmetic）技术
- 提取数据通路
- 对提取的数据通路执行高层算术优化
- 探索可能涉及不同资源共享配置的更优解
- 在资源共享与数据通路优化之间权衡

本节主题：

- 数据通路提取（**Datapath Extraction**）
将加法、减法与乘法等算术算子变换为数据通路块。
- 数据通路实现（**Datapath Implementation**）
用数据通路生成器为提取的组件生成最佳实现。
- 优化数据通路（**Optimizing Datapaths**）

数据通路提取

Datapath Extraction

数据通路提取将加法、减法与乘法等算术算子变换为数据通路块，供数据通路生成器实现。该变换通过使用进位存储算术改善 QoR。

进位存储算术不完全传播进位，而以中间形式存储结果。进位存储加法器（carry-save adder）比传统进位传播加法器（carry-propagate adder）更快，因为其延迟与位宽无关。这些加法器面积也显著小于进位传播加法器，因为进位不使用全加器。

下例给出表达式 $a * b + c + d = z$ 的代码。如图 3.2 所示，该表达式的传统实现使用三个进位传播加法器，而进位存储算术仅需一个进位传播加法器与两个进位存储加法器。图中还给出两种实现的时序与面积数字。

```
module dp (a, b, c, d, e);  input  [15:0] a, b;
    input  [31:0] c, d;
    output [31:0] Z;
    assign Z = (a * b) + c + d;
endmodule
```

（原书 Figure 30: *Conventional Carry-Propagate Adder Versus Carry-Save Adder*）

请参阅原版 PDF 插图。

图 3.2: 传统进位传播加法器与进位存储加法器对比

工具支持提取下列组件：

- 可合并到同一进位存储算术树中的算术算子
- 作为数据通路一部分提取的算子：*、+、-、>、<、<=、>=、==、!= 以及 MUX
- 可变移位算子（Verilog 的 <<、>>、<<<、>>>; VHDL 的 sll、sr1、sla、sra、rol、ror）
- 带位截断的运算

仅当这些组件彼此直接相连（即组件之间无非算术逻辑）时，数据通路流程才能提取它们。请注意：

- 有符号与无符号算子混合提取仅允许用于加法器树；
- 已实例化的 DesignWare 组件不能被提取。

数据通路实现

Datapath Implementation

所有提取的数据通路元素与综合算子（synthetic operators）均由 Fusion Compiler 中的 DesignWare 数据通路生成器实现。该生成器使用智能生成技术执行下列任务：

- 使用上下文驱动优化实现数据通路块
- 针对给定时序上下文重新审视高层优化决策
- 考虑逻辑库特性

数据通路实现可使用下列命令与应用选项：

- **set_datapath_architecture_options**

该命令控制为算术与移位算子生成数据通路单元所用的策略。下例指定使用 Booth 编码架构生成多路选择器：

```
fc_shell> set_datapath_architecture_options -booth_encoding true
```

- **set_implementation**

该命令为除法器及具有指定名称的综合库单元实例指定所用实现。下例为已实例化的 DesignWare 单元 U1 指定使用 cla3 实现：

```
fc_shell> set_implementation cla3 U1
```

- **set_synlib_dont_use**

该命令阻止使用特定实现。下例指定不使用 rpl 实现：

```
fc_shell> set_synlib_dont_use \
{dw_foundation/DW_div/cla standard/DW_div/rpl}
```

- **compile.datapath.ungroup**

默认情况下，时序优化期间所有 DesignWare 单元与数据通路块均被取消分组。要禁用取消分组，将 compile.datapath.ungroup 从默认 true 改为 false。例如：

```
fc_shell> set_app_options \
-name compile.datapath.ungroup -value false
```


优化数据通路

Optimizing Datapaths

下列脚本禁用带数据通路门控元素 `add_3` 的 DP_OP 单元的取消分组：

```
# Disable ungrouping of DP_OP with datapath gating
set_datapath_gating_options -ungroup false [get_cells add_3]
```

分析数据通路提取

Analyzing Datapath Extraction

要从数据通路优化获得最佳 QoR，应确保编译期间从 RTL 代码提取尽可能大的数据通路块。要在编译前检查数据通路块将如何从 RTL 提取，使用 `analyze_datapath_extraction` 命令。该命令分析设计的算术内容并提供反馈，以便按需改进 RTL 代码。

下例显示数据通路报告：

```
Cell : DP_OP_123_2304_10297_J1
-----
Current Implementation : str(area,speed)
Contained Operations : add_30(test.v:30) add_30_2(test.v:30)
                      mult_36(test.v:36)
Multiplier Arch       : benc_radix4

      Data
Var    Type  Class    Width  Expression
-----
PI_0   PI    Unsigned    6
PI_1   PI    Unsigned    1
PI_2   PI    Unsigned   24
T16    IF0   Unsigned    7  PI_0 + PI_1 (test.v:30)
PO_0   PO    Unsigned   30  PI_2 * T16 (test.v:36)
```

要报告设计中使用的资源与数据通路块，使用 `report_resources`。要报告实现概览，对 `report_resources` 指定 `-summary`。例如：

```
fc_shell> report_resources -hierarchy
```

控制数据通路门控

Controlling Datapath Gating

下图示出数据通路门控优化期间生成的 DP_OP 单元输入的信号跳变。要控制数据通路门控，使用 `set_datapath_gating_options` 命令。

下列脚本禁用数据通路元素 `add_3` 上的数据通路门控：

```
# Disable datapath gating on particular operator
set_datapath_gating_options -enable false [get_cells add_3]
```

(原书示意: *DP_OP* 单元输入信号跳变)
请参阅原版 PDF 插图。

图 3.3: 数据通路门控优化中 *DP_OP* 单元的输入信号跳变

3.2.4 控制 MUX 优化

Controlling MUX Optimization

Fusion Compiler 可将 HDL 代码中表示多路选择器的组合逻辑直接映射到目标逻辑库中的单个多路选择器 (MUX) 或 MUX 单元树。

多路选择器通常用 `if` 与 `case` 语句建模。为实现多路选择器逻辑, HDL Compiler 使用 *SELECT_OP* 单元, 再由 Fusion Compiler 映射到逻辑库中的组合逻辑或多路选择器。

(原书示意: *User-specified / Tool evaluated; MUX gates vs area-costed MUX or non-MUX; analyze/elaborate → Infers Select_OP → compile_fusion*)
请参阅原版 PDF 插图。

图 3.4: MUX 优化流程示意

选择与多路复用逻辑

Selecting and Multiplexing Logic

Fusion Compiler 对基于控制信号选择数据信号的逻辑推断 *SELECT_OP* 单元。*SELECT_OP* 单元映射到组合逻辑。

以下主题说明 *SELECT_OP* 推断:

- 推断 *SELECT_OP* (Inferring *SELECT_OP*s)

推断 *SELECT_OP*

Inferring *SELECT_OP*s HDL Compiler 使用 *SELECT_OP* 组件实现由 `if` 与 `case` 语句隐含的条件运算。图 3.5 示出 8-bit 数据信号的 *SELECT_OP* 单元实现示例。

取决于设计约束, Fusion Compiler 用组合逻辑或逻辑库中的多路选择器单元实现 *SELECT_OP*。关于 *SELECT_OP* 推断的更多信息, 见 HDL Compiler 文档。

(原书 *Figure 31: SELECT_OP Implementation for an 8-bit Data Signal*)

请参阅原版 PDF 插图。

图 3.5: 8-bit 数据信号的 SELECT_OP 实现

figures/fig-verilog-select-op.png

图 3.6: Verilog 输出——SELECT_OP 与选择逻辑

库要求

Library Requirements

逻辑库中需要有多路选择器单元，并且需要使用单元库中的多路复用逻辑。

以下各节说明为实现多路选择器优化所需的库单元要求，或为实现来自单元库的多路复用逻辑以控制 MUX 映射：

- 理解多路选择器优化的库单元要求
- 从单元库实现多路复用逻辑

理解多路选择器优化的库单元要求

Understanding Library Cell Requirements for Multiplexer Optimization 多路选择器优化要求逻辑库中至少存在一个 2:1 多路选择器单元。该单元的输入或输出可以反相。若库中不存在 2:1 多路选择器原语单元，将看到下列警告：

Warning: Target **library** does not contain any 2-1 multiplexer.
(OPT-853)

将创建来自目标库的 MUX_OP 单元实现，但未必是最佳实现。目标库中除下列情况外的所有多路选择器单元都可用于构造 MUX_OP 实现：

- 带使能的多路选择器单元
- 总线输出多路选择器
- 大于 32:1 的多路选择器

为使 Fusion Compiler 充分利用逻辑库中的多路选择器单元，请重新编译库，或向 ASIC 供应商获取用 V3.4a 或更高版本编译的库。

从单元库实现多路复用逻辑

Implementing Multiplexing Logic From Cell Library 逻辑综合期间，工具用下列方法之一从单元库实现多路复用逻辑：

- AND、OR 与 INV 门
- MUX 门

工具评估两种方法的面积影响，并选择面积最小的方案。

可通过在块中对应的 SELECT_OP WVGTECH 单元上施加 map_to_mux 属性，指定工具用 MUX 门实现特定多路复用逻辑，如例 3.3 所示。

```
# Apply the map_to_mux attribute on the SELECT_OP cell named C10
set_attribute -objects [get_cells C10] -name map_to_mux -value true

# Query if the attribute is set as required on C10
get_attribute -objects [get_cells C10] -name map_to_mux

#Run the compile step
```

```
compile_fusion

#Identify the cell created using the map_to_mux attribute
get_cells -hier -filter "map_to_mux==true && !infer_mux_override"
```

代码 3.3: 使用 map_to_mux 属性实现特定多路复用逻辑（原书 Example 16）

3.2.5 控制时序映射

Controlling Sequential Mapping

时序映射（sequential mapping）阶段包含两步：寄存器推断（register inferencing）与工艺映射（technology mapping）。术语寄存器同时指边沿触发寄存器与电平敏感锁存器。

寄存器推断是将寄存器的 RTL 描述翻译为称为 SEQGEN 的与工艺无关表示的过程。SEQGEN 在 elaboration 期间创建，通常在编译期间映射到触发器。工艺映射是将 SEQGEN 映射到指定目标逻辑库门的过程。这两步均由 compile_fusion 执行。

寄存器与锁存器均由 SEQGEN 单元表示，它是时序元件的与工艺无关模型，如下图所示：

（原书示意：SEQGEN——clear / preset / next_state / clocked_on / data_in / enable / synch_* → Q / QN）
请参阅原版 PDF 插图。

图 3.7: SEQGEN 单元模型

下表列出 SEQGEN 单元的引脚。仅使用其中一部分，取决于所推断的单元类型。未使用的引脚接到零。

表 3.3: SEQGEN 单元引脚

方向	名称	描述	单元类型
Input	clear	异步复位	Flip-flop 或 latch
	preset	异步置位	Flip-flop 或 latch
	next_state	同步数据	Flip-flop
	clocked_on	时钟	Flip-flop
	data_in	异步数据	Latch
	enable	时钟或异步使能	Latch
	synch_clear	同步复位	Flip-flop
	synch_preset	同步置位	Flip-flop
	synch_toggle	同步翻转	Flip-flop
	synch_enable	同步使能	Flip-flop
Output	Q	同相输出	Flip-flop 或 latch
	QN	反相输出	Flip-flop 或 latch

工艺映射期间，工具以 SEQGEN 为映射起点。时序映射器检查引脚连接以及库单元描述中的信息，以映射到逻辑库寄存器。

本节主题：

- 按 RTL 所述精确映射时序元件
- 扫描单元映射
- 同步复位或置位寄存器的映射
- 为库单元引脚指定同步输入类型
- 控制时序输出反相
- 阻止连接到寄存器 QN 引脚
- 将同步使能逻辑映射到 MUX
- 移位寄存器识别

按 RTL 所述精确映射时序元件

Mapping Sequential Elements Exactly as Described in RTL

要指定工具按 RTL 所述精确映射时序元件，将 `compile.seqmap.exact_map` 设为 `true`。启用该功能时，工具禁用映射到带反相输出或未使用端口的时序库单元。

扫描单元映射

Mapping of Scan Cells

在 test-ready 编译期间，工具用含可测性逻辑的触发器替换普通触发器。下图示出 test-ready 编译期间 D 触发器被替换为扫描寄存器的示例。这类架构（多路选择触发器，multiplexed flip-flop）在 D 触发器输入端加入 2 输入 MUX。MUX 的选择线启用两种模式——功能模式（正常数据输入）与测试模式（扫描数据输入）。本例中，扫描输入引脚为 `si`，扫描使能引脚为 `se`，扫描输出引脚 `so` 与功能输出引脚 `Q` 共享。

（原书示意： $D / si / Clk / se \rightarrow MUX \rightarrow Q / so$ ）

请参阅原版 PDF 插图。

图 3.8: Test-ready 编译中的扫描寄存器示例

运行 `compile_fusion` 时：

- 工具将所有时序单元直接映射到库中的扫描寄存器；
- 扫描引脚（如 `scan enable` 与 `scan out`）保持未连接；

- 扫描寄存器的扫描输出引脚用于扫描连接。

使用 `seqmap.bind_scan_pins` 应用选项可使扫描使能引脚保持无效。

若库中无扫描寄存器，工具用非扫描单元映射并发出警告。要对所有扫描寄存器设置 `dont_use` 属性以映射到非扫描单元。

同步复位或置位寄存器的映射

Mapping of Synchronous Reset or Preset Registers

Fusion Compiler 默认不推断同步复位或置位触发器。要推断带同步复位或置位引脚的寄存器，在 RTL 中使用 `sync_set_reset` Synopsys compiler 指令。HDL Compiler 随后将这些信号连接到 SEQGEN WVGTECH 单元的 `synch_clear` 与 `synch_preset` 引脚，以告知映射器这些是同步控制信号，并应尽可能靠近寄存器保留。

要启用工具推断同步复位/置位触发器，或将同步复位/置位逻辑保留在触发器附近，使用下列两种方法之一：

- `sync_set_reset` 指令
在 RTL 中指定 `//synopsys sync_set_reset` 指令。
- `hdlin.elaborate.ff_infer_sync_set_reset` 应用选项
当 RTL 不含该指令时，将该应用选项设为 `true`；默认为 `false`。
编译期间，工具根据库中的寄存器执行下列映射：
- 库含带同步复位（或置位）引脚的寄存器
工具将复位（或置位）`net` 连接到带专用复位引脚的寄存器的复位（或置位）引脚。
- 库不含带同步复位（或置位）引脚的寄存器
工具在数据输入端增加额外逻辑，以在无复位（或置位）引脚的寄存器上产生复位（或置位）条件。工具还尽量将该逻辑映射得尽可能靠近数据引脚。

下图示出映射到带与不带同步复位引脚的寄存器的示例。



图 3.9: 带/不带同步复位引脚的寄存器映射

即使库有带同步复位（或置位）引脚的寄存器，仍须在 RTL 中使用 `sync_set_reset` 指令，以便工具区分复位（或置位）信号与其他数据信号，并将复位信号尽可能靠近寄存器连接。要将复位信号靠近寄存器连接，将 `compile.seqmap.honor_sync_set_reset` 设为 `true`。

注意

电平敏感锁存器不会推断同步复位与置位信号。即使库有带同步复位或置位引脚的锁存器，锁存器上的同步复位或置位编码风格也始终导致数据信号上的组合逻辑。

异步复位或置位寄存器的映射

Mapping of Asynchronous Reset or Preset Registers

要启用工具映射带异步复位或置位信号的寄存器，必须：

- 在 RTL 中正确描述寄存器；
- 确保库含对应单元；
- 确保 `hdlin.elaborate.ff_infer_async_set_reset` 设为 `true`（默认）。

若库不含此类寄存器，工具将单元保留为未映射。

为库单元引脚指定同步输入类型

Specifying Synchronous Input Types for Library Cell Pins

映射时序单元时，工具使用库单元引脚的 `nextstate_type` 属性识别其同步引脚类型。该属性的有效值为 `data`、`preset`、`clear`、`load`、`scan_in` 与 `scan_enable`。

若逻辑库中未指定或不正确指定该属性，可用 `set_attribute` 手动指定，如下例：

```
fc_shell> set_attribute [get_lib_pins my_lib/dffsrst/RN] \
    nextstate_type clear
```

控制时序输出反相

Controlling Sequential Output Inversion

默认情况下，若能改善 QoR，`compile_fusion` 可将寄存器映射到输出极性与寄存器相反的时序库单元。这称为时序输出反相（*sequential output inversion*）。

当映射到仅有一类异步输入（`set` 或 `reset`）的目标库时，时序输出反相也很有用。此时，映射使用缺失异步输入的寄存器的唯一方法，是使用带相反类型异步输入的库单元并反相该单元输出，如下图所示。

（原书示意：*sequential output inversion*——*SET* 寄存器映射到 *RST + QN*）
请参阅原版 PDF 插图。

图 3.10: 映射期间的时序输出反相示例

注意

反相寄存器的信息写入验证指导（.svf）文件。验证带时序输出反相的块时，必须在 Formality 中包含该 .svf 文件。

要控制时序输出反相：

- 对整个块，使用 `compile.seqmap.enable_output_inversion` 应用选项（默认为 `true`）；
- 对特定层次或叶子单元，使用 `set_register_output_inversion` 命令。

用 `set_register_output_inversion` 指定的实例级设置会覆盖用 `compile.seqmap.enable_output_inversion` 指定的块级设置。

`compile_fusion` 允许将设计中的时序元件映射到输出相位反相的时序库单元。在某些情况下，工具可能推断出具有某一类异步控制的寄存器，而库中只有相反类型的引脚。此时，工具映射到相反类型的寄存器，并反相所有数据输入与输出。

阻止连接到寄存器 QN 引脚

Preventing Connections to Register QN Pins

时序映射期间，若能改善 QoR，工具可将寄存器映射到同时具有 Q 与 QN 引脚的库单元。但可通过将 `compile.seqmap.disable_qn_usage` 设为 `true`，阻止工具使用此类寄存器的 QN 引脚。

将同步使能逻辑映射到 MUX

Mapping of Synchronous Enable Logic to a MUX

当 RTL 含同步使能逻辑时，`compile_fusion` 将使能逻辑映射到带同步使能引脚的库单元。当此类库单元不可用时，`compile_fusion` 将推出的同步逻辑映射到 MUX，如下图所示。

（原书示意：同步使能逻辑映射到 MUX）
请参阅原版 PDF 插图。

图 3.11: 同步使能逻辑映射到 MUX

要启用映射到 MUX，在编译前对 SEQGEN 单元设置 `map_sync_enable_to_mux` 属性。例如：

```
fc_shell> set_attribute "A1/B/reg[30] mid/sub1/Q_reg[0]" \
-name map_sync_enable_to_mux -value true
fc_shell> set_attribute "mid3/apbif_reg/Q_reg[4]" \
-name map_sync_enable_to_mux -value true
fc_shell> compile_fusion
```

移位寄存器识别

Identification of Shift Registers

编译期间，工具可自动识别设计中的移位寄存器。`compile_fusion -to initial_opto` 将第一个寄存器映射到扫描单元，其余映射到非扫描单元，如下图所示。

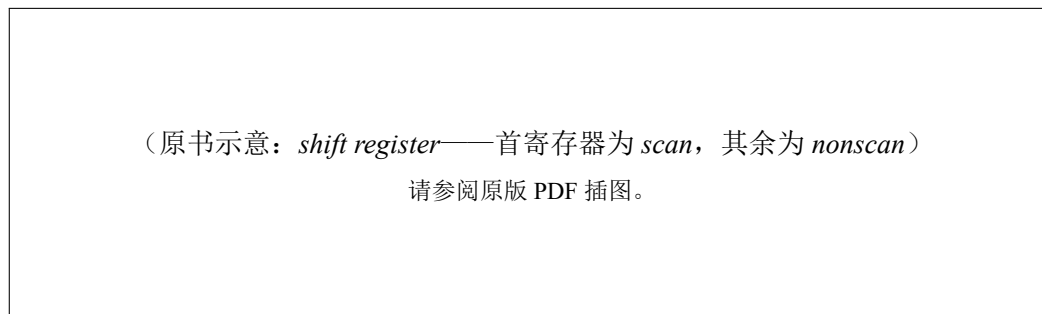


图 3.12: 移位寄存器识别与映射

该能力因扫描信号更少而改善时序设计面积并降低拥塞。要禁用该能力,将 `compile.seqmap.identify_shift_registers` 设为 `false`; 默认为 `true`。

3.2.6 控制寄存器复制

Controlling Register Replication

可通过运行 `compile_fusion -to logic_opt` 执行逻辑优化。要用 `set_register_replication` 标识希望工具在逻辑优化期间复制的寄存器。该命令在指定寄存器上设置 `register_replication` 属性，并通过平衡扇出改善时序。逻辑优化期间，指定寄存器被复制；使用 `-num_copies` 时，原寄存器的负载在新复制的寄存器间均匀分配。使用 `-max_fanout` 时，工具计算副本数，并尝试在不违反最大扇出约束的前提下均匀分配负载。

可用下列方法之一控制创建的副本数：

- 用 `-num_copies` 指定复制后（含原件）的寄存器总数；
- 用 `-max_fanout` 指定复制后寄存器的最大扇出。

若同时指定两个选项，则忽略 `-max_fanout`。

下例指定工具应复制 `regA` 与 `regB`，并为每个寄存器再创建一份副本：

```
fc_shell> set_register_replication -num_copies 2 regA
fc_shell> set_register_replication -num_copies 2 regB
```

图 3.13 示出优化前后的 `regA` 与 `regB`。

可用 `-include_fanin_logic` 或 `-include_fanout_logic` 将指定寄存器扇入或扇出中的逻辑一并包含。可指定寄存器扇入中的任一起点，或扇出中的任一终点。

下例指定工具应复制 `A_reg` 以及从该寄存器到 `U1` 单元路径上的逻辑：

```
fc_shell> set_register_replication -num_copies 2 A_reg \
-include_fanout_logic U1
```

要指定复制后必须仍由原寄存器驱动的终点寄存器或主输出 `port`（如图 3.14），使用 `-driven_by_original_register` 选项。

(原书 *Figure 33: Before and After Register Replication*)

请参阅原版 PDF 插图。

图 3.13: 寄存器复制前后对比

下例指定复制后 `r0` 必须驱动 `r1`、`r2` 与 `r3`:

```
fc_shell> set_register_replication -num_copies 3 r0 \  
-driven_by_original_register {r1 r2 r3}
```

(原书 *Figure 34: Using the -driven_by_original_register Option*)

请参阅原版 PDF 插图。

图 3.14: 使用 `-driven_by_original_register` 选项

3.2.7 控制寄存器合并

Controlling Register Merging

默认情况下，工具在优化期间执行寄存器合并（`register merging`）。要禁用寄存器合并，将 `compile.seqmap.enable_register_merging` 设为 `false`；默认为 `true`。哪些寄存器被合并的信息存储在 `.svf` 文件中，输出日志会显示关于寄存器合并的信息消息。

也可用 `set_register_merging` 基于属性设置控制寄存器合并。要在优化期间启用寄存器合并，对指定对象列表指定 `true` 以设置寄存器合并属性；要禁用，对指定对象列表指定 `false`。对象可包括叶子单元、层次单元与模块。

下例指定不合并实例名含 `u1/count_reg` 的寄存器：

```
fc_shell> set_register_merging "u1/count_reg*" false
```

下例指定不合并名为 `u1` 的实例的全部触发器：

```
fc_shell> set_register_merging "u1" false
```

3.2.8 选择性移除或保留常量与无负载寄存器

Selectively Removing or Preserving Constant and Unloaded Registers

为通过减小面积优化设计，工具可自动检测并移除常量寄存器与无负载寄存器。可按下列主题控制该行为：

移除常量寄存器

Removing Constant Registers

设计中某些寄存器可能因一个或多个输入引脚为常量而永不改变状态。这些常量可直接出现在输入端，也可由扇入逻辑优化后最终导致寄存器输入为常量。消除此类寄存器可显著改善面积。

默认情况下，`compile_fusion` 移除常量寄存器，并在编译日志中打印信息消息。下表列出可消除时序元件的情形。

表 3.4: 可消除的常量寄存器情形

寄存器类型	Data	Preset	Reset
简单常量数据	1 或 0		
Preset 与常量数据 1	1	X	
常量 preset	X	1	
Reset 与常量数据 0	0		X
常量 reset	X		1
Preset、reset 与常量数据 1；reset 始终无效	1	X	0
Preset、reset 与常量数据 0；preset 始终无效	0	0	X
一个输入为 0 的 2 输入 XOR 门	X		
D 与 Q 引脚直接相连的 retention 寄存器	1 或 0		

要为正在综合的整个块禁用常量寄存器移除，将 `compile.seqmap.remove_constant_registers` 从默认的 `true` 改为 `false`。要针对特定单元或模块控制常量寄存器移除，使用 `set_constant_register_removal`。

例如，下列设置阻止工具移除名为 `reg11` 的常量寄存器：

```
fc_shell> set_constant_register_removal reg11 false
```

下列设置全局禁用整个块的常量寄存器移除，但对名为 `sub5` 的模块除外：

```
fc_shell> set_app_option -name compile.seqmap.remove_constant_registers \
-value false

fc_shell> set_constant_register_removal [get_module sub5] true
```

已移除常量寄存器的信息写入验证指导 `.svf` 文件与编译日志。要报告编译期间移除的常量寄存器，使用 `report_constant_registers`。

移除无负载寄存器

Removing Unloaded Registers

优化期间，工具删除输出不驱动任何负载的寄存器。若单元未在设计中使用，与寄存器输入相关联的组合逻辑锥也可被删除。寄存器输出可能因电路冗余或常量传播而变为无负载。在某些已实

例化寄存器的设计中，输出可能本来就无负载。

默认情况下，`compile_fusion` 移除设计中的无负载寄存器，并在编译日志中打印信息消息。要为正在综合的块禁用该能力，将 `compile.seqmap.remove_unloaded_registers` 从默认 `true` 改为 `false`。要针对特定单元或模块控制无负载寄存器移除，使用 `set_unloaded_register_removal`。

例如，下列设置阻止工具移除名为 `reg22` 的无负载寄存器：

```
fc_shell> set_unloaded_register_removal reg22 false
```

下列设置全局禁用整个块的无负载寄存器移除，但对名为 `sub5` 的模块除外：

```
fc_shell> set_app_option -name compile.seqmap.remove_unloaded_registers \
-value false

fc_shell> set_unloaded_register_removal [get_module sub5] true
```

要报告编译期间移除的无负载寄存器，使用 `report_unloaded_registers`。

3.2.9 报告优化寄存器的交叉探测信息

Reporting Cross-Probing Information for Optimized Registers

`compile_fusion` 在寄存器优化期间合并寄存器，并移除常量与无负载寄存器。要在 `compile_fusion` 命令日志与 `report_transformed_registers` 命令输出中打印这些优化寄存器的 RTL 文件名与行号，将 `compile.seqmap.print_cross_probing_info_for_removed_registers` 设为 `true`。

下表显示不同类型寄存器优化期间打印的消息。

表 3.5: 寄存器优化期间打印的交叉探测信息消息

寄存器优化	信息消息
常量寄存器移除	Information: The register 'a/b_reg { {/user/test/design.v:49} }' is removed as constant 0 (SQM-4100)
无负载寄存器移除	Information: The register 'a/c_reg{ {/user/test/design.v:193} }' is removed because it is unloaded. (SQM-4101)
寄存器合并	Information: The register 'a_reg[1] { {/user/testcases/top.v:113} }' is removed because it is merged to 'a_reg[0]'. (SQM-4102)

下例输出显示 `report_transformed_registers` 报告的交叉探测信息：

```
Legend:
    c0r - Constant 0 Register Removed
    c1r - Constant 1 Register Removed
    ul  - Unloaded Removed
    inv - Inverted
    rep - Replicated
    mrg - Merged register
    mb  - Mutibit
    mbd - Mutibit debanked

Register      Transformation      Filename:
Line number
```

```

-----
-----
mega_shift_reg          c0r
  {{a/b/test1.v:112}}
Carry_Flag_reg          ul
  {{a/b/test2.v:117}}
mega_shift_reg[10]      mrg (mega_shift_reg[31])
  {{a/b/test3.v:121 }}

```

3.2.10 控制高扇出 net 综合

Controlling High-Fanout-Net Synthesis

工具在 `compile_fusion` 的 `initial_drc` 阶段执行高扇出 net 综合。要用全局布线 net 确定高扇出综合期间插入 `buffer` 的拓扑，将 `compile.initial_drc.global_route_based` 设为 `true`。在 `compile_fusion` 的 DRC 修复阶段完成高扇出综合之后，全局布线被删除。

基于全局布线的缓冲有利于路由通道高度拥塞的设计，例如 `macro bank` 之间的通道。该功能会增加 `compile_fusion` 的运行时间，因此仅应在路由通道高度拥塞的设计上启用。

3.3 执行多比特优化

Performing Multibit Optimization

Fusion Compiler 可将单比特寄存器或较小的多比特寄存器组合 (`bank`)，并替换为等效的更大多比特寄存器。例如，工具可将八个 1-bit 寄存器或四个 2-bit 寄存器 `bank` 组合，替换为一个 8-bit 寄存器 `bank` 或两个 4-bit 寄存器 `bank`。工具仅在单比特寄存器具有相同时序约束时才合并它们，并将时序约束复制到结果多比特寄存器。

(原书 *Figure 35: Replacing Multiple Single-Bit Register Cells With a Multibit Register Cell*)

请参阅原版 PDF 插图。

图 3.15: 用多比特寄存器单元替换多个单比特寄存器单元

用多比特寄存器替换单比特寄存器可减少：

- 因共享晶体管与优化的晶体管级版图而减少的面积；
- 时钟树总网线长度；
- 时钟树 `buffer` 数量与时钟树功耗。

若能改善总负松弛（total negative slack），工具也可将大多比特寄存器拆分（debank）为较小的多比特寄存器或单比特寄存器。

某些逻辑库具有混合驱动强度（mixed-drive-strength）的多比特寄存器，部分 bit 驱动强度高于其他 bit。对此类单元，若违例路径经过较低驱动强度的 bit，工具可对混合驱动强度多比特单元重新布线（rewire），使违例路径经过较高驱动强度的 bit。

3.3.1 执行集成多比特寄存器优化

Performing Integrated Multibit-Register Optimization

要在 `compile_fusion` 期间执行集成多比特寄存器优化：

1. 将 `compile.flow.enable_multibit` 设为 `true`，启用多比特寄存器优化。

启用后，默认情况下工具在 `compile_fusion` 的不同阶段执行下列多比特优化：

- 在 `initial_map` 阶段于 RTL 级组合单寄存器或较小寄存器 bank，以创建更大寄存器 bank。
要禁用，将 `compile.flow.enable_rtl_multibit_banking` 设为 `false`。
- 在 `logic_opt` 阶段拆分 RTL 级创建的多比特寄存器 bank（若 bank 中包含关键时序路径上的寄存器）。
要禁用，将 `compile.flow.enable_rtl_multibit_debanking` 设为 `false`。
- 在 `initial_opto` 阶段考虑布局创建多比特寄存器 bank。
要禁用，将 `compile.flow.enable_physical_multibit_banking` 设为 `false`。
可将 `compile.flow.enable_second_pass_multibit_banking` 设为 `true`，并将 `compile.flow.enable_physical_multibit_banking` 设为 `false`，从而在 `final_place` 而非 `initial_opto` 阶段启用基于布局的多比特 banking。
应仅在 `initial_opto` 或 `final_place` 之一启用基于布局的多比特 banking，不要两者都启用。
- 在 `initial_opto` 与 `final_opto` 阶段拆分多比特寄存器 bank（若能改善 TNS）。
要禁用，将 `compile.flow.enable_multibit_debanking` 设为 `false`。

2. （可选）。

3. 用 `set_multibit_options` 指定多比特寄存器 banking 设置。

例如，下列命令在 RTL-banking 阶段将 `reg[1]` 排除在多比特优化之外：

```
fc_shell> set_multibit_options -exclude [get_cells "reg[1]"] \
    -stage rtl
```

下列命令指定在 physical-banking 阶段，工具仅对连接到同一总线的寄存器做 banking：

```
fc_shell> set_multibit_options -bus_registers_only \
    -stage physical
```

4. （可选）按下表用应用选项控制多比特寄存器优化。
5. （可选）按「从特定单比特寄存器创建多比特寄存器 bank」所述，从特定单比特寄存器创建多比特寄存器 bank。
6. （可选）按「指定多比特寄存器命名风格」所述，控制多比特 banking 所用命名风格。

表 3.6: 控制多比特寄存器优化的通用应用选项

目的	设置此应用选项
Banking 时忽略未包含在 SCANDEF 信息中的 scan flip-flop	multibit.banking.enable_strict_scan_check 设为 true
Banking 时考虑带多个 scan-in 引脚与内部 scan-in 引脚的多比特寄存器单元	multibit.banking.lcs_consider_multi_si_cells 设为 true
启用由等效 integrated-clock-gating (ICG) 单元驱动的寄存器 banking	
启用后, 工具可能丢失施加在连接至 clock-gating 单元的 net 上的非默认布线规则。因此优化后请检查并重新应用这些规则。	multibit.banking.across_equivalent_icg 设为 true
启用功耗管理单元的时序驱动 banking	
工具跳过 fixed 单元以及标有 dont_touch、legalize_only 与用户指定 size_only 属性的单元	mv.cells.enable_multibit_pm_cells 设为 true
在物理 banking 与 debanking 期间包含带用户指定 size_only 设置的单元	multibit.common.exclude_size_only_cells 设为 false
为 compile_fusion 启用多比特寄存器 rewiring	compile.flow.enable_multibit_rewiring 设为 true
在 logic_opt 阶段执行 RTL-based 多比特重排时考虑翻转率 (toggle rate)	compile.flow.enable_toggle_aware_rtl_multibit_reordering 设为 true
启用 site-aware debanking、时钟 useful skew (CUS)、driver sizing, 以及对瓶颈多比特寄存器的早期 debanking	
多比特 debanking 功能影响 initial_opto (compile_fusion)、final_place 以及 final_opto (compile_fusion/place_opt) 阶段的 debanking。	
多比特 debanking 不影响 RTL debanking (在 compile_fusion 的 logic_opt 阶段启用)。	opt.common.enable_multibit_feature 设为 true

7. 用 `compile_fusion` 执行优化。

8. (可选) 报告多比特信息。

要用 `report_transformed_registers -multibit` 报告工具推断的多比特寄存器 bank，见「报告多比特寄存器」。

要用 `report_multibit` 报告多比特统计信息，例如单比特与多比特寄存器总数、多比特 banking 比率等。

关于如何确定为何未对特定单元执行多比特寄存器 banking，见「识别为何未执行多比特 Banking」。

9. (可选) 按「在 GUI 中查看多比特组件」所述，在 GUI 中分析多比特 banking 组件。

3.3.2 从特定单比特寄存器创建多比特寄存器 bank

Creating a Multibit Register Bank From Specific Single-Bit Registers

在对 RTL 设计运行 `compile_fusion` 之前，可用 `create_multibit` 从特定未映射的单比特寄存器创建多比特总线。随后运行 `compile_fusion` 时，在 `initial_map` 阶段，工具将 `create_multibit` 创建的总线中的单比特单元映射为多比特寄存器 bank。

用 `create_multibit -sort` 指定的单比特寄存器必须：

- 未映射；
- 类型相同；
- 位于同一逻辑层次；
- 不属于已有总线。

下例中，单比特单元按字母数字降序组合，即 `reg4`、`reg3`、`reg2`、`reg1`：

```
fc_shell> set_app_option -name compile.flow.enable_multibit -value true
fc_shell> create_multibit {reg2 reg4 reg1 reg3} -sort descending
fc_shell> compile_fusion
```

下例中，单比特单元按字母数字升序组合，即 `reg1`、`reg2`、`reg3`、`reg4`：

```
fc_shell> set_app_option -name compile.flow.enable_multibit -value true
fc_shell> create_multibit {reg2 reg4 reg1 reg3} -sort ascending
fc_shell> compile_fusion
```

下例中，单比特单元按指定顺序组合，即 `reg2`、`reg4`、`reg1`、`reg3`：

```
fc_shell> set_app_option -name compile.flow.enable_multibit -value true
fc_shell> create_multibit {reg2 reg4 reg1 reg3} -sort none
fc_shell> compile_fusion
```

注意

必须先将 `compile.flow.enable_multibit` 设为 `true`，再对未映射单比特寄存器运行 `create_multibit`。否则工具发出下列错误：

```
Error: create_multibit command failed: multibit flow is
disabled. (MBIT-068)
```

3.3.3 指定多比特寄存器命名风格

Specifying Naming Styles for Multibit Registers

Fusion Compiler 允许定制多比特寄存器的命名风格。可用本节所述应用选项修改默认命名风格，以匹配设计流程或第三方工具的要求。

指定名称分隔符

Specifying a Name Separator

创建多比特寄存器名称时，工具拼接单比特寄存器名称，并以 `_` 字符作为名称分隔符。例如，组合名为 `reg1` 与 `reg2` 的单比特寄存器时，多比特寄存器名为 `reg1_reg2`。

要更改默认名称分隔符，使用 `multibit.naming.multiple_name_separator_style`。

指定名称前缀

Specifying a Name Prefix

要为多比特寄存器名称指定前缀，使用 `multibit.naming.name_prefix`。

压缩层次名

Compacting Hierarchical Names

组合带层次名的单比特寄存器时，工具用单比特寄存器的完整层次名拼接多比特寄存器名。例如，组合 `A/B/P/reg1` 与 `A/B/Q/reg2` 时，多比特寄存器名为 `A/B/P/reg1_A/B/Q/reg2`。

要使多比特寄存器名更紧凑，将 `multibit.naming.compact_hierarchical_name` 设为 `true`，指定公共层次名不重复。此时组合 `A/B/P/reg1` 与 `A/B/Q/reg2`，多比特寄存器名为 `A/B/P/reg1_Q/reg2`。

为总线寄存器的连续位指定范围分隔符

Specifying a Range Separator for Contiguous Bits of a Bused Register

组合总线寄存器的连续位时，工具用 `:` 字符作为范围分隔符。例如，组合 `regA[0]`、`regA[1]` 与 `regA[2]` 时，多比特寄存器名为 `regA[2:0]`。

要更改范围分隔符，使用 `multibit.naming.range_separator_style`。

为总线寄存器的非连续位指定元素分隔符

Specifying an Element Separator for Noncontiguous Bits of a Bused Register

组合总线寄存器的非连续位时，工具用 `,` 字符作为元素分隔符。例如，组合 `regA[1]` 与 `regA[3]` 时，多比特寄存器名为 `regA[3,1]`。

要更改元素分隔符，使用 `multibit.naming.element_separator_style`。

指定组合多维寄存器数组元素时的展开命名风格

Specifying the Expanded Naming Styles When Combining Elements of a Multidimensional Register Array

要为组合多维寄存器数组元素所创建的多比特寄存器指定展开命名风格，使用 `multibit.naming.expanded_name_style`。更多信息见该应用选项的 `man` page。

3.3.4 报告多比特寄存器

Reporting Multibit Registers

运行 `compile_fusion` 之后，可用 `report_transformed_registers -multibit` 报告因多比特 banking 与 debanking 而变换的寄存器。

下例显示该命令生成的报告片段：

```
fc_shell> report_transformed_registers -multibit
...
*****
Attributes:
mb - multibit
mbd - multibit debanked
...
Register      Optimization
-----
MB_2_MB_1     mb (source:MB_2, MB_1)
MB_0_MB_3     mb (source:MB_0, MB_3)
MB_5, MB_6    mbd (source: MB_5_MB_6)
```

3.3.5 识别为何未执行多比特 Banking

Identifying Why Multibit Banking Is Not Performed

要用 `report_multibit` 报告多比特 banking 比率摘要、未 banking 或 debanking 单元的原因列表，以及每种原因的单元数，如下例：

```
fc_shell> report_multibit
*****
Report : report_multibit
...
...
*****

Total number of sequential cells:          4020
Number of single-bit flip-flops:           3149
Number of single-bit latches:              29
Number of multi-bit flip-flops:            842
Number of multi-bit latches:               0
```

```

Total number of single-bit equivalent cells:      853
(A)   Single-bit flip-flops:                     3149
(B)   Single-bit latches:                       29
(C)   Multi-bit flip-flops:                     5352
(D)   Multi-bit latches:                        0

Sequential cells banking ratio ((C+D)/(A+B+C+D)): 62.74%
Flip-flop cells banking ratio ((C)/(A+C)):        62.96%
BitsPerflop:                                     2.13

```

Reasons **for** sequential cells not mapping to multibit during RTL Banking:

Explanations:

```

r12: Cell is single bit because its parent multibit cell was debanked
due to improve timing (Number of cells: 91)
r31: Cell cannot be banked to multibit because it is assigned to use
single-bit lib cell (Number of cells: 63)

```

Reasons **for** multibit sequential cells not debanking to single bit cells during RTL Debanking:

Explanations::

```

r45: Multibit cell cannot be debanked because it is not critical
enough (Number of cells: 478)

```

要报告 banking 与 debanking 期间被忽略的全部单元, 对 report_multibit 使用 -ignored_cells 选项。

要报告可用于 banking 与 debanking 的兼容多比特与单比特库单元, 使用 check_multibit_library 命令。

下例生成 RTL banking 与 debanking 的兼容库单元报告:

```

fc_shell> check_multibit_library -stage RTL \
    -banking -debanking
*****
Report : check_multibit_library
Flow   : RTL BANKING
*****

-----
Single bit Lib cell      Compatibility      Multi bit Lib cell
-----
SB_reg1      PIN ORDER MISMATCH  MB_reg1
SB_reg2      COMPATIBLE          MB_reg2
-----

Singlebit with NO Multibit Equivalents
-----

SB_reg3

*****
Report : check_multibit_library

```

```

Flow      : RTL DEBANKING
*****
-----
Multi bit Lib cell      Compatibility      Single bit Lib cell
-----
MB_reg2                  COMPATIBLE          SB_reg2
MB_reg1                  PIN ORDER MISMATCH SB_reg1
-----
Multibit with NO Singlebit Equivalents
-----
MB_reg3

```

3.3.6 改善 Banking 比率

Improving the Banking Ratio

在 `initial_map` 阶段，工具可优先使用具有功能等效多比特库单元的时序库单元。运行 `compile_fusion` 之后，若 `report_multibit -ignored_cells` 表明许多单元因库中缺少多比特单元而未被 banking，可将 `compile.seqmap.prefer_registers_with_multibit_equivalent` 设为 `true` 并重新运行 `compile_fusion` 以启用该功能。但启用该功能可能降低设计 QoR。

3.3.7 在 GUI 中查看多比特组件

Viewing Multibit Components in the GUI

Fusion Compiler 使用 WordView 基础设施映射多比特组件。要在 GUI 中查看多比特组件，必须满足下列要求：

- 确保库以 `.ndm` 格式包含多比特组件；
- 通过设置 `compile.flow.enable_multibit` 启用多比特组件映射，如下：

```

fc_shell> set_app_options -name compile.flow.enable_multibit \
-value true

```

在 WordView 中查看多比特组件的步骤如下：

1. 用 `analyze` 与 `elaborate` 读取 RTL。
2. 运行 `compile_fusion -to initial_opto`。
工具在编译期间将总线寄存器映射到多比特组件。
3. (可选) 用 `report_cells` 检查多比特组件映射信息。例如，RTL 含 `reg_state` 总线寄存器：

```

module top (...);
output [2:0] reg_state;
always @(posedge clk)
    reg_state <= next_reg_state;
endmodule

```

报告显示 `reg_state` 总线寄存器被映射到 `MB_DFF` 多比特组件：

```
fc_shell> report_cells [get_cells *reg_state_reg* -hierarchical]
...
Cell                Reference    Library    Attributes
-----
reg_state_reg[2:0]  MB_DFF      my_mb_lib    n
```

3.3.8 映射组合多比特单元

Mapping Combinational Multibit Cells

组合多比特单元（combinational multibit cells）是库单元，单个单元实现多个逻辑功能。Fusion Compiler 工具：

- 将单比特组合单元 banking 到多比特组合单元；
- 按需 debank 多比特组合单元以改善 QoR。

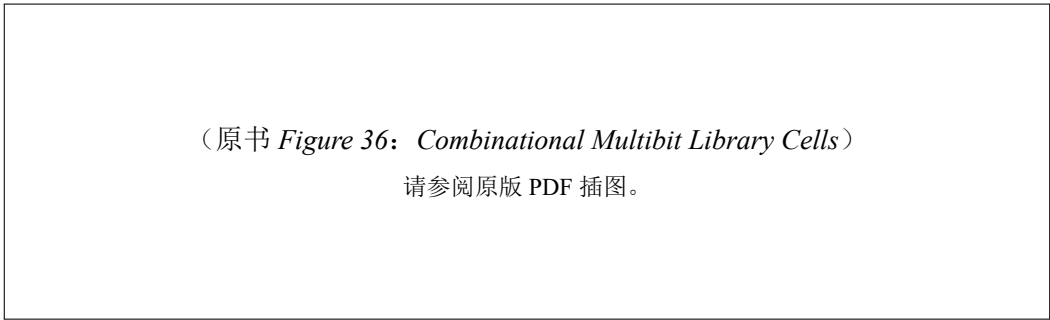


图 3.16: 组合多比特库单元

要在 initial_opto 与 final_opto 阶段启用组合多比特映射，用 set_app_options 设置 opt.common.enable_multibit_combinational_cells：

```
fc_shell> set_app_options -name \
    opt.common.enable_multibit_combinational_cells -value true
```

注意

默认情况下，工具会对多比特组合单元执行 debanking。

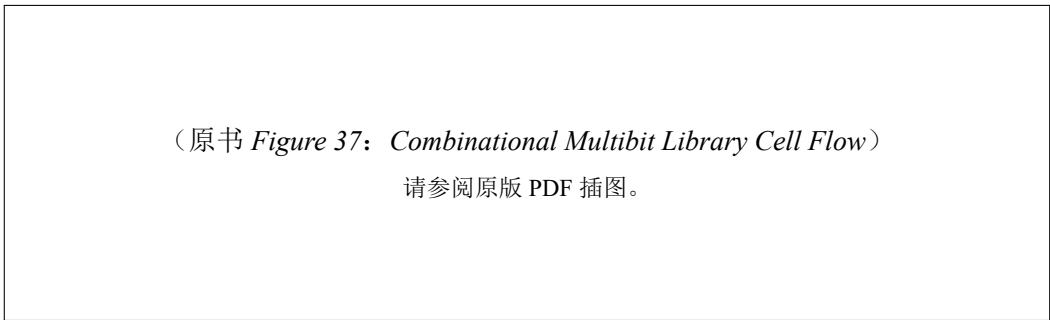


图 3.17: 组合多比特库单元流程

要打印为每个库 reference 创建的组合多比特单元，对 report_multibit 使用 -combinational 选项，如下例：

```
fc_shell> report_multibit -combinational

-----
#Outputs  Reference                Number of instances  Single-bit
                                           Equivalent
-----
2         combo_mbit_lib_cell1    257                 514
4         combo_mbit_lib_cell2    57                  228
-----
```

要检查组合多比特库单元的兼容单比特等效单元，对 check_multibit_library 使用 -combinational 选项，如下例：

```
fc_shell> check_multibit_library -combinational

-----
Combinational      Compatibility      Equivalent Singlebit
Multibit LibCell          LibCell
-----
combo_mbit_lib_cell1  Compatible          combo_sbit_lib_cell1
                        Don't use Mismatch  combo_sbit_lib_cell2
combo_mbit_lib_cell2  PVT mismatch        combo_sbit_lib_cell3
-----
```

3.4 运行并发时钟与数据优化

Running Concurrent Clock and Data Optimization

更改时钟 latency 可平衡相继时序路径级的松弛，从而优化时钟与数据路径。该能力称为并发时钟与数据（concurrent clock and data，CCD）优化，可减少总负松弛（TNS）、最差负松弛（WNS）、面积与漏电功耗，并改善物理综合与布局布线之间的相关性。

3.4.1 Useful Skew

Useful Skew

下图示出并发时钟与数据优化如何允许用时钟偏斜（clock skew）消除负松弛，并可能实现面积与漏电回收。

3.4.2 控制并发时钟与数据优化

Controlling Concurrent Clock and Data Optimization

默认情况下，工具在 compile_fusion 期间执行并发时钟与数据优化。可通过将 compile.flow.enable_ccd 设为 false 禁用；默认为 true。

并发时钟与数据优化期间：

- 工具使用理想时钟树，并为所有活动 scenario 更新时钟 latency 与 balance points;

(原书示意: *CCD / Useful Skew*——用时钟偏斜消除负松弛)

请参阅原版 PDF 插图。

图 3.18: 并发时钟与数据优化中的 Useful Skew

- 工具在 *delayed* 与 *advanced latency* 设置范围内, 调整设计中最关键时序元件的寄存器 *latency*。
默认最大 *delayed latency* 为 100 ps, 最大 *advanced latency* 为 300 ps;
- 所有 *path group* 均参与 *useful skew* 计算;
- 并发时钟与数据优化期间考虑所有边界时序路径。

注意

在增量编译 (*incremental compile*) 中, 启用并发时钟与数据优化无效。

为获得最佳结果, 还应:

- 即使在编译期间也启用 *hold scenario*, 以减少因并发时钟与数据优化插入时钟 *latency* 而导致的 *hold* 违例。
工具仅在并发时钟与数据优化中考虑 *hold scenario*, 而不适用于其他优化步骤;
- 为 *clock_opt* 命令启用并发时钟与数据优化。

控制时钟 **Latency**、**Path Group** 与边界路径

Controlling Clock Latencies, Path Groups, and Boundary Paths

要为 *useful skew* 计算指定最大 *delayed latency*, 将 *ccd.max_postpone* 设为特定值, 如下例:

```
fc_shell> set_app_options \
  ccd.max_postpone 50
```

要为 *useful skew* 计算指定最大 *advanced latency*, 将 *ccd.max_prepone* 设为特定值, 如下例:

```
fc_shell> set_app_options \
  ccd.max_prepone 100
```

要省略特定 *path group* 的 *useful skew* 计算, 用 *ccd.skip_path_groups* 指定一个或多个 *path group*, 如下:

```
fc_shell> set_app_options ccd.skip_path_groups path_group_list
```

要禁用边界时序路径的并发时钟与数据优化, 将 *ccd.optimize_boundary_timing* 从默认 *true* 改为 *false*。此后不会调整 I/O 路径上的优化以及边界寄存器的 *latency*, 以改善寄存器到寄存器时序。


```
fc_shell> set_app_options ccd.optimize_boundary_timing false
```

降低动态压降

Reducing Dynamic Voltage Drop

运行 `compile_fusion` 时，可通过将 `compile.flow.enable_dps` 设为 `true` 启用动态功耗整形 (dynamic power shaping) 优化，以降低块上的动态压降 (dynamic voltage drop)。工具在 `compile_fusion` 的 `final_opto` 阶段执行动态功耗整形优化。

启用该功能时，工具执行功耗分析，并用 `useful skew` 技术降低动态压降。可选地，可用 `ccd.dps.use_case` 定制动态功耗整形优化期间的功耗分析。

在布线前阶段指定优化目标

Specifying Optimization Targets at the Preroute Stage

用 `compile_fusion` 执行并发时钟与数据优化时，可为下列对象的 WNS 优化赋予更高优先级：

- 某些 path group——用 `ccd.targeted_ccd_path_groups` 指定，如下例：

```
fc_shell> set_app_options -name ccd.targeted_ccd_path_groups \
    -value {PG1 PG2}
```

若用该应用选项指定的 path group 同时也被 `ccd.skip_path_groups` 指定为跳过，则该 path group 在并发时钟与数据优化期间被跳过。

- 某些终点——用 `ccd.targeted_ccd_end_points_file` 指定包含终点的文件，如下例：

```
fc_shell> set_app_options \
    -name ccd.targeted_ccd_end_points_file \
    -value endpoint_targets.tcl
```

若同时指定 path group 与终点，工具仅优化指定 path group 中的指定终点。

- 最差的 300 条时序路径——将 `ccd.enable_top_wns_optimization` 设为 `true`。

传递到后端

Transferring to Back End

要将时钟 latency 与 balance points 从前端工具传递到后端工具，使用 `write_ascii_files` 命令。该命令写入 `design.MCMM` 目录，其中包含 Tcl 文件形式的时序上下文。例如，`design.MCMM/scenario_scenario.tcl` 文件含下列时钟 latency 设置：

```
set_clock_latency -clock [get_clocks {CLK}] -0.3 [get_pins {d_reg[4]/CP}]
set_clock_latency -clock [get_clocks {CLK}] 0.1 [get_pins {d_reg[7]/CP}]
```

`design.MCMM/mode_mode.tcl` 文件含下列 balance points：

```
set_clock_balance_points -clock [get_clocks {CLK}] \
    -balance_points [get_pins {d_reg[4]/CP}] \
    -consider_for_balancing true \
```

```

-rise -delay 0.3 -corners [get_corners {default}]
set_clock_balance_points -clock [get_clocks {CLK}] \
  -balance_points [get_pins {d_reg[7]/CP}] \
  -consider_for_balancing true \
-rise -delay -0.1 -corners [get_corners {default}]

```

3.5 执行测试插入与扫描综合

Performing Test Insertion and Scan Synthesis

Fusion Compiler 测试综合方案可将 DFT 能力透明地实现到 Synopsys 综合流程中，而不干扰功能、时序、信号完整性或功耗要求。

Fusion Compiler 的测试解决方案包含两个阶段：

- 扫描综合流程（Scan Synthesis Flow）
- 最小化扫描寄存器异步置位与复位线上的毛刺

两阶段测试插入的收益包括：

- 同时为逻辑与 DFT 综合获得更好的时序、面积、功耗与拥塞 QoR；
- 功耗意图（power intent）与 DFT 逻辑更好集成；
- 改善的拥塞优化，考虑 compressor 与 decompressor（codec）测试逻辑的布局；
- 消除为综合并布局 DFT 逻辑而进行的增量编译步骤。

3.5.1 扫描综合流程

Scan Synthesis Flow

为某些寄存器指定 DFT 插入相关的综合约束时，这些寄存器必须映射到非扫描版本，并涉及 core wrapper reuse 阈值。要指定 DFT 配置：

1. 执行 `compile_fusion -from initial_map -to logic_opt`。
2. 指定 DFT 配置，如 DFT 信号、链数、测试模式等。
3. 完成 DFT 插入。关于 DFT 插入的更多信息，见 *TestMAX DFT User Guide*。
4. 执行 `compile_fusion -from initial_place`。

注意

必须在导入 floorplan 信息之前创建扫描测试以及 scan-in 与 scan-out port。此外，floorplan 必须包含测试与扫描 port 的物理位置。

仅扫描脚本示例

Scan-Only Script Example

```
# Set up the Library
...
# Read the design
...
# Specify any registers that should not be mapped to scan equivalent, and
# also to be excluded from any scan chains
set_scan_element false <registers that should not be mapped to scan
equivalent>
# Run compile_fusion to logic_opto step
compile_fusion -to logic_opto
# DFT Specifications:
# Scan chain setup
set_scan_configuration -chain_count sc
# DFT signals
# Clocks and asynchronous resets
set_dft_signal -view existing -type ScanClock \
    -port clk -timing [list 45 55]
set_dft_signal -view existing -type Reset \
    -port reset_n -active_state 0
# Scan in, scan out, scan enable
set_dft_signal -view spec -type ScanDataIn -port SI
set_dft_signal -view spec -type ScanDataOut -port SO
set_dft_signal -view spec -type ScanEnable -port SE
# Scan Synthesis steps
create_test_protocol
dft_drc
preview_dft
insert_dft
# Continue from initial_place step
compile_fusion -from initial_place -to initial_opto
# Perform post dft_drc
dft_drc -test_mode Internal_scan
# Generate post dft_drc verbose report
report_dft_drc_violations -test_mode Internal_scan -rule all
# Write out Scandef
write_scan_def design.scan.def
# Write out test model
write_test_model -output design.ctl
# Save output for TetraMAX
write_test_protocol -output design.scan.spf \
    -test_mode Internal_scan
write_verilog -hierarchy all design.v
```

3.5.2 最小化扫描寄存器异步置位与复位线上的毛刺

Minimizing Glitches on Asynchronous Set and Reset Lines of Scan Registers

当设计在功能模式与扫描模式之间切换时，扫描寄存器的异步置位与复位线上可能出现毛刺 (glitch)，从而复位这些寄存器的状态。工具可通过在异步置位与复位线上添加门控逻辑来防止。

要启用该功能，必须用 `set_register_async_gating_pin -port` 指定控制门控逻辑的顶层 port。该控制 port 必须是 RTL 中预定义、且未连接到任何其他 net 的 port。要获取此类门控扫描寄存器的信息，在物理综合之后运行 `report_async_gated_registers`。

所添加的门控逻辑可以是 AND 或 OR 门，取决于扫描寄存器异步引脚的极性。添加门控逻辑后，Formality 需要额外信息以验证设计，Fusion Compiler 会生成相应的 .svf 文件，在功能模式下禁用门控逻辑。

注意

该功能仅支持 in-compile DFT 流程。

3.6 指定性能、功耗与面积改善设置

Specifying Settings for Performance, Power, and Area Improvement

以下主题说明可用于改善设计性能、功耗与面积 (PPA) 的设置：

- 启用高努力时序模式 (Enabling High-Effort Timing Mode)
- 启用增强延迟优化以改善总负松弛
- 启用高努力面积模式 (Enabling High-Effort Area Mode)
- 启用嵌入式面积优化流程

3.6.1 启用高努力时序模式

Enabling High-Effort Timing Mode

多数情况下，为编译启用高努力时序模式可获得更好的时序 QoR。要启用该模式，将 `compile.flow.high_effort_timing` 设为 1。

在编译期间运行高努力时序模式可能增加运行时间，因为工具会执行多轮布局与优化以获得最优时序结果。

3.6.2 启用增强延迟优化以改善总负松弛

Enabling Enhanced Delay Optimization to Improve Total Negative Slack

可通过 `path group` 让工具在优化期间专注修复更多违例路径，以改善块的总负松弛。但该技术可能增加运行时间并降低面积与功耗 QoR。

要启用改善总负松弛的增强延迟优化技术，将 `compile.flow.enhanced_delay_opto_for_pg` 设为 1。与大量使用 `path group` 改善总负松弛相比，该功能具有更好的运行时间以及更好的面积与功耗 QoR。

3.6.3 启用高努力面积模式

Enabling High-Effort Area Mode

对需要进一步减小面积的设计，将 `compile.flow.high_effort_area` 设为 `true`。

在编译期间运行高努力面积模式可能增加运行时间，因为工具会执行多轮布局与优化以获得最优面积结果。

3.6.4 启用嵌入式面积优化流程

Enabling the Embedded Area Optimization Flow

在 `compile_fusion` 的 `logic_opt` 阶段，作为初始步骤，工具执行时序优化以及 DesignWare 重选与取消分组。该初始步骤之后，工具可在继续 `logic_opt` 阶段其他优化技术之前，执行可选的嵌入式面积优化流程。

嵌入式面积优化流程包括专用取消分组、快速面积驱动逻辑重构，以及轻量时序优化。要启用该功能，将 `compile.flow.areaResynthesis` 设为 `true`。

该功能可减小含许多 DesignWare 层次的块的面积。

3.7 执行设计分析

Performing Design Analysis

使用 Fusion Compiler 生成的报告分析并调试设计。可在编译前后生成报告：编译前生成报告以检查属性、约束与设计规则是否正确设置；编译后生成报告以分析结果并调试设计。

本节包含下列主题：

- 报告命令与示例（Reporting Commands and Examples）
- 衡量结果质量（Measuring Quality of Results）
- 对比 QoR 数据（Comparing QoR Data）
- 在批处理模式下报告逻辑级数
- 查询特定消息 ID

3.7.1 报告命令与示例

Reporting Commands and Examples

下表列出部分用于报告 QoR、时序、面积等的报告命令。

report_area 示例

report_area Example

```
fc_shell> report_area -designware -hierarchy -physical
*****
Report : area
```

表 3.7: 常用报告命令

使用此命令	用途
report_area	报告当前设计的面积信息。 - 面积基于物理库。 - 不报告 port 与 net 数量。
report_qor	报告当前设计的 QoR 信息与统计。 - 信息基于物理库。
report_timing	报告当前设计的时序信息。
report_power	计算并报告设计或实例的动态与静态功耗。
report_clock_gating	报告设计的总时钟门控统计。
report_logic_levels	报告设计的逻辑级数。

```
Design : test
...
*****
Information: Base Cell (com): cell XNOR2ELL, w=3300, h=6270 (npin=3)
Information: Base Cell (seq): cell LPSDFE2, w=10560, h=6270 (npin=5)
Number of cells: 8
Number of combinational cells: 4
Number of sequential cells: 4
Number of macros/black boxes: 0
Number of buf/inv 0
Number of references 2
Combinational area: 49.66
Buf/Inv area: 0.00
Noncombinational area: 231.74
Macro/Black box area: 0.00
Total cell area: 281.40

-----
Hierarchical area distribution
-----
Global cell area          Local cell area
-----
Hierarchical cell  Absolute  Percent  Combi-  Noncombi-  Black-
                  Total      Total    national national  boxes
Design
-----
test              281.40    100.0    37.24    231.74    0.00
test
mult_10           12.41     4.4      12.41     0.00     0.00
DW_mult_uns_J1_H3_D1
-----
Total              49.66    231.74    0.00

Area of detected synthetic parts
```

Module	Implem.	Count	Area	Perc. of cell area
DW_mult_uns	pparch	1	12.41	4.4%
Total:		1	12.41	4.4%
Estimated area of ungrouped synthetic parts				
Module	Implem.	Count	Estimated area	Perc. of cell area
DW_mult_uns	pparch	3	48.28	17.2%
Total:		3	48.28	17.2%
Total synthetic cell area:			60.69	21.6% (estimated)
Core Area: 1767				
Aspect Ratio:			1.0902	
Utilization Ratio:			0.1593	
The above information was from the logic library.				
The following information was from the physical library:				
Total moveable cell area:			281.4	
Total fixed cell area:			0.0	
Total physical cell area:			281.4	
Core area:			0.000, 0.000, 40.260, 43.890	

report_qor 示例

report_qor Example

```
fc_shell> report_qor
*****
Report : qor
Design : top
...
*****
Scenario          'SC1'
Timing Path Group  'clk'
-----
Levels of Logic:           6
Critical Path Length:      0.28
Critical Path Slack:       -0.19
Critical Path Clk Period:  0.30
```

```

Total Negative Slack:                -0.80
No. of Violating Paths:              11
Worst Hold Violation:               -0.07
Total Hold Violation:               -0.13
No. of Hold Violations:             32.00
-----

Cell Count
-----
Hierarchical Cell Count:             2
Hierarchical Port Count:            24
Leaf Cell Count:                    287
Buf/Inv Cell Count:                 45
Buf Cell Count:                     4
Inv Cell Count:                     41
CT Buf/Inv Cell Count:              0
Combinational Cell Count:           208
Sequential Cell Count:              79
Macro Count:                        1
-----

Area
-----
Combinational Area:                  419.34
Noncombinational Area:              736.26
Buf/Inv Area:                       67.60
Total Buffer Area:                   9.66
Total Inverter Area:                 57.94
Macro/Black Box Area:               2337.90
Net Area:                           0
Net XLength:                        0
Net YLength:                        0
-----
Cell Area (netlist):                 3493.49
Cell Area (netlist and physical only): 3493.49
Net Length:                         0
-----

Design Rules
-----
Total Number of Nets:                363
Nets With Violations:                0
Max Trans Violations:               0
Max Cap Violations:                 0
-----

```

report_transformed_registers 示例

report_transformed_registers Example

report_transformed_registers 命令报告编译期间被修改或移除的寄存器的下列信息:

- 时序输出端口反相
- 被移除的常量寄存器及其常量值
- 用户保留的常量寄存器
- 被移除的无负载寄存器
- 用户保留的无负载寄存器
- 寄存器合并
- 寄存器复制
- 移位寄存器
- 多比特寄存器
- 寄存器优化摘要

仅对由编译或增量编译生成的已映射设计使用 `report_transformed_registers`。若对未映射设计运行该命令，则不报告任何信息。变换或优化寄存器的信息存储在库中，因此可在不同会话中用该命令查询相同信息。

例如：

```
fc_shell> report_transformed_registers
*****
Report: report_transformed_registers
Version: ...
Date: ...
*****
Legend:
C0p - constant 0 register preserved
C1p - constant 1 register preserved
C0r - constant 0 register removed
C1r - constant 1 register removed
ulp - preserved unloaded register
ulr - removed unloaded register
mrg - merged register
srh - shift register head
srf - shift register flop
mb - multibit
rep - replicated
inv - inverted

Register                                Optimization
-----
I_RISC_CORE/I_DATA_PATH/PSWL_Carry_reg  C0r
I_RISC_CORE/I_STACK_TOP/I3_STACK_MEM/Stack_Mem_reg[0][0] C0r
I_RISC_CORE/I_STACK_TOP/I3_STACK_MEM/Stack_Mem_reg[1][0] C0r
```

要报告每种寄存器变换类型的计数，指定 `-summary` 选项，如下例：

```
fc_shell> report_transformed_registers -summary
```

```
*****
```

```
Report: report_transformed_registers
```

```
Version: ...
```

```
Date: ...
```

```
*****
```

```
Legend:
```

```
C0p - constant 0 register preserved
```

```
C1p - constant 1 register preserved
```

```
C0r - constant 0 register removed
```

```
C1r - constant 1 register removed
```

```
ulp - preserved unloaded register
```

```
ulr - removed unloaded register
```

```
mrg - merged register
```

```
srh - shift register head
```

```
srf - shift register flop
```

```
mb - multibit
```

```
rep - replicated
```

```
inv - inverted
```

Register	Count

Constant Registers Deleted	12
Constant Registers Preserved	1
Unloaded Registers Deleted	101
Unloaded Registers Preserved	38
Shift Registers	170
Merged	12
Multibit	10
Inverted	15
Replicated	4

3.7.2 衡量结果质量

Measuring Quality of Results

要用 `create_qor_snapshot` 衡量设计当前状态的结果质量（QoR），并将质量信息存储到一组报告文件中。可用不同优化策略或在设计的不同阶段捕获 QoR 信息并对比结果质量。

`create_qor_snapshot` 从时序、设计规则、面积、功耗、拥塞等方面衡量并报告设计质量，并将质量信息存储到当前工作目录下名为 `snapshot` 的目录中的一组快照文件。要存储到不同目录，使用 `time.snapshot_storage_location`。

下例在 `snapshot` 目录中生成名为 `r1`、时序路径使用零线负载（zero wire load）的 QoR 快照：

```
fc_shell> create_qor_snapshot -name r1 -zero_wire_load
```

```
...
```

```
No. of scenario = 4
```

```
s1 = function
```

```
s2 = function_min
```

```

s3 = test
s4 = test_min
-----
---
WNS of each timing group:      s1      s2      s3      s4
-----
REGIN                        -0.34      -      -      -
fref_clk0                    2.34     3.25      -      -
pld_clk                      0.65     1.65      -      -
pll_fixed_clk_central       0.61     1.65      -      -
REGOUT                      0.39     0.44     1.18     1.72
...
-----
Setup WNS:                   -0.34     0.44     1.02     1.10     -0.34
Setup TNS:                   -130.82    0.00     0.00     0.00    -130.82
Number of setup violations:   1152      0        0        0        1152
Hold WNS:                    -0.55    -0.78    -0.30    -1.28    -1.28
...
-----
Area:                                1200539.606
Cell count:                                562504
Buf/inv cell count:                       145920
Std cell utilization:                       0.24
...

```

查询 QoR 快照

Querying QoR Snapshots

要检索并报告由 `create_qor_snapshot` 创建的快照，使用 `query_qor_snapshot`。该命令分析先前生成的 QoR 时序报告，并按指定过滤器以 HTML 格式显示结果。

下例中，`query_qor_snapshot` 分析存储在 `snapshot` 目录中的 `placeopt` 快照，然后报告最差负松弛介于 -2.0 ns 与 -1.0 ns 之间的时序路径：

```

fc_shell> create_qor_snapshot -name placeopt
fc_shell> query_qor_snapshot -name placeopt -filters "-wns -2.0,-1.0"

```

报告 QoR 快照

Reporting QoR Snapshots

要显示当前设计的 QoR 信息与统计，使用 `report_qor`。例如，下列命令显示存储在 `snapshot` 目录中的 `r1` QoR 快照：

```

fc_shell> report_qor_snapshot -name r1 -display

```

移除 QoR 快照

Removing QoR Snapshots

要从 `time.snapshot_storage_location` 指定的目录移除已有 QoR 快照，使用 `remove_qor_snapshot`。例如，下列命令移除存储在 `snapshot` 目录中的 `preroute` QoR 快照：

```
fc_shell> remove_qor_snapshot -name preroute
```

3.7.3 对比 QoR 数据

Comparing QoR Data

可按下列步骤生成基于 Web 的报告，以查看并用 QORsum 报告对比 QoR 数据：

1. (可选) 用 `set_qor_data_options` 配置后续步骤捕获并显示的 QoR 数据。

- 要用 `-leakage_scenario` 与 `-dynamic_scenario` 指定设计中最关键的功耗 scenario。
工具用你指定的功耗 scenario 生成 QORsum 报告中功耗 QoR 的高层摘要。若不指定这些选项，则对漏电与动态 scenario 均使用总功耗最高的活动功耗 scenario。
这些设置仅用于功耗 QoR 摘要。工具使用所有活动功耗 scenario 的功耗信息捕获并报告 QORsum 报告中的详细功耗信息。
- 要用 `-clock_name` 与 `-clock_scenario` 指定最关键的时钟名与时钟 scenario。
工具用你指定的时钟名与 scenario 生成 QORsum 报告中时钟 QoR 的高层摘要。若不指定这些选项，工具识别最关键时钟并用于时钟 QoR 摘要。
这些设置仅用于时钟 QoR 摘要。工具使用所有时钟生成 QORsum 报告中的详细时钟 QoR 信息。
- 要用 `-run_name` 指定在 QORsum 报告中标识该次运行的名称。
默认情况下，工具用数字命名每次运行，如 `Run1`、`Run2` 等。可用该选项为每次运行指定更有意义的名称。也可在步骤 3 用 `compare_qor_data` 生成 QORsum 报告时通过 `-run_names` 指定运行名称；若如此，工具忽略 `set_qor_data_options -run_name` 指定的运行名称。

下例指定漏电功耗 scenario、动态功耗 scenario、时钟 scenario 以及用于 QORsum 报告对应摘要的时钟：

```
fc_shell> set_qor_data_options \
-leakage_scenario S1 -dynamic_scenario S2 \
-clock_scenario S3 -clock_name sys_clk
```

2. 用 `write_qor_data` 为报告收集 QoR 数据。该命令捕获与时序、功耗、运行时间及其他度量相关的 QoR 数据。

可在希望捕获 QoR 数据的设计流程各阶段多次运行 `write_qor_data`。用 `-label` 并指定一个值，以标明在流程的哪个阶段捕获数据。

默认情况下，`write_qor_data` 捕获与已布局并优化设计相关的数据。可用 `-report_group` 指定五个支持值之一（`unmapped`、`mapped`、`placed`、`cts`、`routed`），以根据捕获数据时的流程状态调整所捕获的报告集。要更细粒度控制，用 `-report_list` 显式指定要生成的每个报告。

`write_qor_data` 从多数常用工具报告（如 `report_qor` 与 `report_power`）捕获标准 QoR 数据

集到磁盘。若有不属于该命令集合的额外自定义数据要捕获，用 `capture_qor_data` 捕获完全自定义的 QoR 数据或 GUI 布局图像，以包含在基于 Web 的 QORsum 报告中。这可用于补充 `write_qor_data` 捕获的标准 QoR 数据，或创建完全自定义的 QORsum 报告。相关命令包括：`define_qor_data_panel`（可在 QORsum 报告中创建用于查看 QoR 数据的新面板），以及 `set_qor_data_metric_properties`（可定制任一 QORsum 报告面板中任一度量列的属性，从而控制着色风格或对比数据时的着色阈值等）。

3. 用 `compare_qor_data` 生成 QORsum 报告。

该命令取得一次或多次 `write_qor_data` 运行所捕获的数据，并创建基于 Web 的报告以查看和对比这些结果。

必须用 `-run_locations` 指定上一步每次 `write_qor_data` 运行输出的位置。每个路径位置对应要对比的运行结果。

可用 `-run_names` 为 QORsum 报告中的运行指定特定名称。若如此，工具忽略用 `set_qor_data_options -run_name` 指定的运行名称。默认情况下，工具用数字命名每次运行，如 `run1`、`run2` 等。

可用 `-output` 指定输出目录。默认写入名为 `compare_qor_data` 的目录。要用 `-force` 覆盖已有输出数据；默认不覆盖。

4. 用 `view_qor_data` 查看生成的 QORsum 报告。

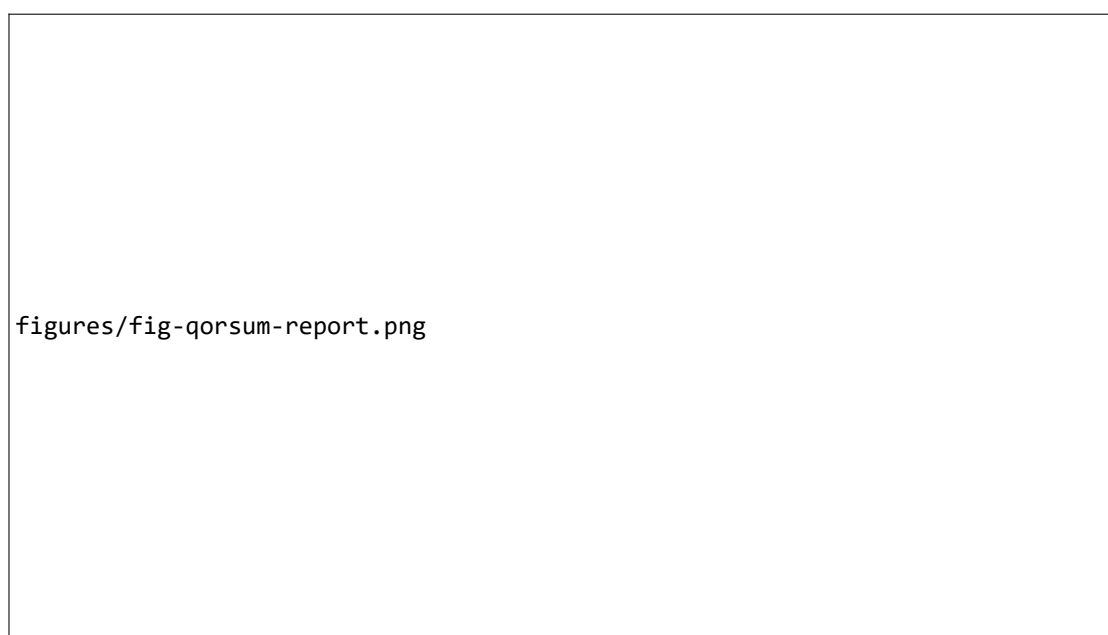


图 3.19: QORsum 报告

关于探索对比数据的更多信息，见下列主题：

- 设置基线运行（Setting Your Baseline Run）
- 更改 QoR 显示风格
- 排序与过滤数据
- 探索详细对比数据

设置基线运行

Setting Your Baseline Run

默认情况下，第一行数据是基线运行（baseline run），其他所有运行均与之对比。基线运行的名称以金色高亮，以标记为“golden”或基线结果。

非基线单元格的着色表示相对基线的差异方向与程度：

- 红色表示相对基线变差；绿色表示相对基线改善；
- 较浅着色表示相对基线差异较小；较深着色表示相对基线差异较大。

要将光标悬停在列标题上，可查看各着色对应的 delta 阈值。

要更改基线运行，点击 Runs 按钮，并从 Baseline Run 列选择新的基线运行，如下图所示：

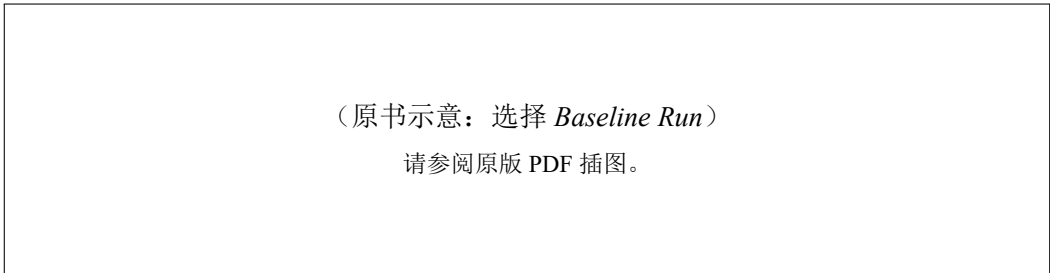


图 3.20: 更改基线运行

参见

更改 QoR 显示风格；排序与过滤数据；探索详细对比数据。

更改 QoR 显示风格

Changing the QoR Display Style

默认情况下，对比表显示所有运行的 QoR 值。例如，下列运行的 NVE 数为 2374。

在表中任意处右键单击，可循环切换不同数据显示风格。可按下列方式查看数据：

- 相对基线的百分比 delta；
- 相对基线的绝对 delta（以斜体显示）。

例如，作为百分比 delta，NVE 数显示比基线少 2.7% 的失败终点；作为绝对 delta，NVE 数显示比基线少 65 个失败终点。

参见

排序与过滤数据；探索详细对比数据。

排序与过滤数据

Sorting and Filtering the Data

可对运行数据排序与过滤，以揭示结果中的模式并确定要进一步探索的参数。

关于排序与过滤数据，见下列主题：

- 排序数据（Sorting the Data）
- 过滤度量（Filtering Metrics）
- 过滤运行（Filtering Runs）
- 分析示例（Example Analysis）

参见

探索详细对比数据。

排序数据

Sorting the Data 点击任一列标题，按该度量排序数据。第一次点击升序，第二次点击降序。

例如，要在表顶部显示最差 TNS、底部显示最佳 TNS，点击 TNS 列标题；再点一次则顶部为最佳、底部为最差。

（原书示意：按 TNS 升序/降序排序）
请参阅原版 PDF 插图。

图 3.21: 按 TNS 排序

过滤度量

Filtering Metrics 要控制表中显示哪些度量，点击 Metrics 按钮，并在 Metrics 对话框中相应勾选或取消度量。

例如，要仅显示 setup 时序、网表面积与单元计数，按图 3.22 选择度量。

（原书 Figure 39: Metrics Dialog Box）
请参阅原版 PDF 插图。

图 3.22: Metrics 对话框

过滤运行

Filtering Runs 要控制表中显示哪些运行，点击 **Runs** 按钮，并从 **Visible Runs (Summary)** 列相应勾选或取消探索运行。

例如，要在表中仅显示前四个运行，按图 3.23 选择运行。

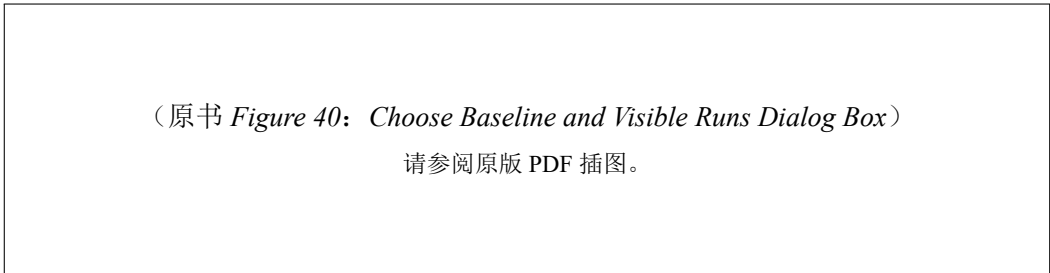


图 3.23: 选择基线与可见运行对话框

分析示例

Example Analysis 下例演示如何对数据排序与过滤，以缩小希望进一步探索的运行范围。假设打开图 3.24 所示对比报告，并将 `util60_aspect1%1_layerM9` 运行设为基线运行。



图 3.24: 对比报告

可先查看 **TNS**，并将数据按最佳到最差 **TNS** 排序，如下图所示。注意最佳 **TNS** 运行具有 **M9** 顶层金属，最差具有 **M7** 顶层金属。这表明限制金属层对时序有显著影响。

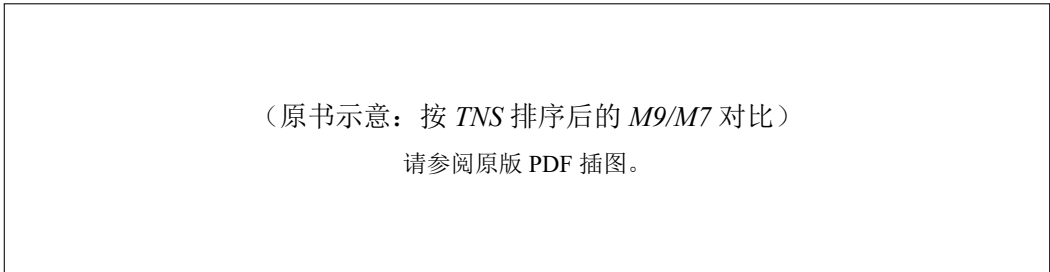


图 3.25: 按 TNS 排序后的结果

然后可通过关闭 **M7** 运行的可见性，将分析限制到 **M9** 运行，如下图所示。

在获得最佳 **TNS** 运行后，可通过按 **GRCOverflow** 列排序（最差溢出在顶部）对比其拥塞，如下图所示。注意较高利用率运行比低利用率运行更拥塞。可关闭较高利用率运行的可见性，将分析限

(原书示意：仅显示 *M9* 运行)
请参阅原版 PDF 插图。

图 3.26: 限制到 M9 运行

制到低利用率运行，如图 3.28 所示，从而得到更好满足时序与拥塞目标、且规模可控的探索运行子集。

(原书示意：按 *GRCOverflow* 排序)
请参阅原版 PDF 插图。

图 3.27: 按拥塞排序

(原书 *Figure 42: Displaying Lower-Utilization Runs*)
请参阅原版 PDF 插图。

图 3.28: 显示低利用率运行

探索详细对比数据

Exploring the Detailed Comparison Data

启动 QORsum 报告时，应用打开 QOR Summary 表，汇总各运行的高层时序、功耗与拥塞度量。该视图及其他摘要视图帮助你排序与过滤数据，以定位要进一步探索的运行。

详细表面板提供比摘要表更深的数据洞察。例如，摘要表可能给出流程各主要阶段设计时序（WNS、TNS、NVE）的高层概览；详细表可能显示基于 *path group* 的更深入时序信息，给出设计中每个 *scenario* 与 *path group* 的时序数字。这只是一例。QORsum 报告中还有许多详细表面板，可显示设计中每个库单元实例的统计（实例数、单元类型、总面积），或每条已布线金属层上的线长等信息。

详细表可一次查看最多六个不同数据集。数据集是为特定运行与流程阶段/标签捕获的一组数据。例如，可同时查看三次不同运行的 *route_opt* 结果；或查看一次运行，但观察设计流程不同阶段（*place_opt*、*clock_opt*、*route_opt*）结果的进展，如下图所示。

(原书 *Figure 43: Path Group Details*)

请参阅原版 PDF 插图。

图 3.29: Path Group Details

流程以各度量下的列并排显示。例如，WNS 列组（以及每个度量的列组）显示同样的六列，标记为 1、2、5、6、9、10。这些列代表各流程的结果，并赋予流程 ID 号（而非完整流程名），以免列过宽。每个度量下的第一列是基线，其余五列为与之对比的测试流程。表上方显示图例：流程 ID 在金色框（基线）或蓝色框（测试流程）中，后跟流程名。点击 **Legend** 按钮可展开或折叠图例。也可在点击 **Runs** 按钮打开的“Choose Baseline and Visible Runs”对话框中看到流程 ID。

详细表视图一次最多显示六个运行（基线运行，加上 Choose Baseline and Visible Runs 对话框中选中的另外前五个运行）。要更改所显示的运行：

1. 点击 **Runs** 按钮，打开 Choose Baseline and Visible Runs 对话框。
2. 从 **Visible Runs (Summary)** 列相应勾选或取消运行。这会同步选择或取消 **Visible flows (Detailed)** 列中的对应项。

由于详细表可能显示数百甚至数千行数据，一次无法查看超过六个数据集。但若希望查看加载到 QORsum 报告中的所有运行与所有流程阶段/标签上这些度量的更多细节，有一种称为 **Focused View**（聚焦视图）的交替视图。可通过点击报告顶部的 **Focused View** 切换按钮，或点击详细表左侧冻结列来访问。在聚焦视图中，可查看详细表某一行数据在所有运行与所有流程阶段上的结果。例如，在基于 **path group** 的时序详细表中，可能只想聚焦 **scenarioA** 中 **path_group1** 的信息。点击该 **path group** 与 **scenario** 对应行即可打开其 **Focused View**。

沿用同一 **path-group** 时序示例，聚焦视图一次显示一个 **path group**。可通过报告顶部下拉菜单选择不同 **scenario** 或 **path group** 名称来更改焦点。见图 3.30。

(原书 *Figure 44: Focused View Drop-down Menu*)

请参阅原版 PDF 插图。

图 3.30: Focused View 下拉菜单

过滤详细数据

Filtering the Detailed Data 详细视图提供过滤器以聚焦特定数据。某些视图有自动显示的默认过滤器。例如，下列详细视图有默认的 **scenario** 与 **path group** 过滤器：**Path Type Details**、**Path Group**

Details 与 Logic Level Details。

可通过下列方式修改默认过滤器：

- 点击过滤器名称前的 X 符号移除过滤器；
- 更改过滤器值；
- 点击过滤器上除 X 符号或值字段外的任意处，以启用或禁用过滤器。

也可对详细数据应用自定义过滤器。要创建自定义过滤器：

1. 点击 Filter 按钮显示 Add Filter 字段。
2. 通过定义过滤条件并选择要包含在结果中的数据集来定义过滤器。

要定义过滤器，选择列与比较类型，然后指定比较值。

- 数值比较：选择 =、!=、<、<=、> 或 >=，并在 Value 字段指定数值；
- 字符串比较：选择 contains，并在 Value 字段指定字符串；
- 正则表达式比较：选择 regexp，并在 Value 字段指定正则表达式；
- 基于所选列可用值过滤：选择 enum，Value 字段会填充含可用值的下拉菜单，然后启用一个或多个显示值。启用时值前显示勾选。要启用或禁用某值，点击或用上下箭头导航高亮该值，再按 Enter 切换状态。也可在 Value 字段输入字符串以过滤下拉菜单中的可用值。要点击 X 符号取消已选值。

3. 点击绿色勾选标记应用过滤器。

例如，要按显示 func@cworst scenario 中的 path group 过滤 Path Group Details 视图，按下列方式定义过滤器（原书有对应示意图）。

（原书示意：按 *scenario* 过滤 *Path Group Details*）

请参阅原版 PDF 插图。

图 3.31: 过滤 Path Group Details

3.7.4 在批处理模式下报告逻辑级数

Reporting Logic Levels in Batch Mode

作为在 GUI 中显示逻辑级数直方图的替代，可在命令行或脚本（批处理模式）中用 report_logic_levels 报告逻辑级数。

注意

必须有已映射设计才能运行逻辑级数报告。

可用该命令报告：

- 整个设计或选定 path group 的逻辑级数；
- 所有类型路径的逻辑级数，包括所有 scenario 中的同时钟路径、跨时钟路径、不可行路径与多周期路径。

报告在摘要、逻辑级数分布与逻辑级数路径报告各节中包含逻辑分布与时序信息。默认不报告 buffer 与 inverter。

要报告逻辑级数：

1. (可选) 用 `set_analyze_rtl_logic_level_threshold` 设置不同于默认的逻辑级数阈值。

默认情况下，工具根据路径所需延迟值推导逻辑级数阈值。

- 为整个设计设置逻辑级数阈值：

```
fc_shell> set_analyze_rtl_logic_level_threshold threshold_value
```

- 为特定 path group 设置逻辑级数阈值：

```
fc_shell> set_analyze_rtl_logic_level_threshold \
    -group path_group threshold_value
```

path group 设置覆盖整个设计的全局阈值。

- 重置所有阈值：

```
fc_shell> set_analyze_rtl_logic_level_threshold -reset
```

- 重置特定 group 设置：

```
fc_shell> set_analyze_rtl_logic_level_threshold \
    -group path_group -reset
```

2. (可选) 通过设置 `shell.synthesis.logic_level_report_summary_format` 指定报告摘要节中的字段。默认含五个字段: "group period wns num_paths max_level"。最多可指定八个字段。例如：

```
fc_shell> set_app_options \
    -name shell.synthesis.logic_level_report_summary_format \
    -value "group num_paths max_level min_level avg_level"
...
```

3. (可选) 通过设置 `shell.synthesis.logic_level_report_group_format` 指定报告 path group 节中的字段。默认含五个字段: "slack ll llthreshold startpoint endpoint"。最多可指定十个字段。例如：

```
fc_shell> set_app_options \
    -name shell.synthesis.logic_level_report_group_format \
    -value "slack ll ll_buf_inv clockcycles startclk endclk"
...
```

4. 用 `report_logic_levels` 报告逻辑级数。下例生成 `clk_i` path group 的逻辑级数报告：

```
fc_shell> report_logic_levels -group clk_i
```

3.7.5 查询特定消息 ID

Querying Specific Message IDs

要查询上一步命令期间出现的特定信息、警告或错误消息 ID，可使用 `check_error` 命令。

按下列步骤设置并查询消息 ID：

1. 用全局作用域应用选项 `shell.common.check_error_list` 指定要查询的消息 ID：

```
fc_shell> set_app_options -name shell.common.check_error_list \  
-value {DES-001 CMD-005}  
shell.common.check_error_list {DES-001 CMD-005}
```

2. 以 `link` 命令作为上一步命令的示例：

```
fc_shell> link  
Error: Current block is not defined. (DES-001)
```

3. 用 `check_error` 查询 `link` 命令步骤期间发出的指定消息 ID：

```
fc_shell> check_error -verbose  
{DES-001}  
1
```

结果显示 `link` 命令步骤期间发出了 DES-001 消息 ID。若未发出任何指定消息 ID，`check_error` 返回 0。

Chapter 4

时钟门控

Clock Gating

RTL 时钟门控 (RTL clock gating) 是 Fusion Compiler 工具提供的一种功耗优化功能。这是一种高层次优化技术: 通过向并非始终使能、且具有同步加载使能 (synchronous load-enable) 或显式反馈环路的寄存器添加时钟门控, 可显著节省功耗。工具通过对同一层次内或跨层次共享的公共使能信号进行提取, 对触发器实施门控。该技术通过降低寄存器时钟输入上的翻转活动并消除多路选择器, 大幅降低动态功耗, 也可带来更小的芯片面积。

默认情况下, 在 `compile_fusion` 命令期间会按指定的时钟门控设置执行时钟门控。若未指定任何时钟门控约束, 则使用默认约束。虽然不能禁用时钟门控, 但可使用本节中的时钟门控命令, 阻止在特定块或整个设计中插入时钟门控。

工具在 `analyze` 与 `elaborate` 命令期间识别已有的时钟门控单元, 并在叶子级再插入一级时钟门控单元。

更多信息见下列主题:

- 时钟门控简介 (Introduction to Clock Gating)
- 时钟门控先决条件 (Clock-Gating Prerequisite Conditions)
- 设置时钟门控 (Setting Up Clock Gating)
- 时钟门控流程 (Clock Gating Flows)
- 替换时钟门控 (Replacing Clock Gates)
- 控制时钟门控延迟 (Controlling Clock-Gate Latencies)
- 控制时钟门控级数 (Controlling the Number of Clock-Gate Levels)
- 合并时钟门控 (Merging Clock Gates)
- 设置时钟门控变换 (Setting Clock Gating Transformations)
- 为时钟门控设置布线规则 (Setting Routing Rules for Clock Gates)
- 时钟门控与多比特寄存器 (Clock Gating and Multibit Registers)
- 布局感知时钟门控 (Placement-Aware Clock Gating)

- 基于扇入的时序时钟门控（Fanin-Based Sequential Clock Gating）
- 报告时钟门控结果（Reporting Clock-Gating Results）
- 报告时钟门控使能信号（Reporting Clock-Gate Enable Signals）
- 时钟门控效率报告（Clock Gate Efficiency Reporting）
- 自门控优化（Self-Gating Optimization）
- DFT wrapper 时钟门控的特殊命名与查询（Special Naming and Querying for DFT Wrapper Clock Gates）

4.1 时钟门控简介

Introduction to Clock Gating

时钟门控适用于同步加载使能寄存器，即共享同一时钟与同步控制信号的触发器。同步控制信号包括同步加载使能、同步置位（synchronous set）、同步复位（synchronous reset）与同步翻转（synchronous toggle）。

同步加载使能寄存器表现为带有反馈环路的寄存器，可在多个周期内保持同一逻辑值。对同步加载使能寄存器应用时钟门控，可降低寄存器组（register bank）反复加载时所需的功耗。

图 4.1 给出使用多路选择器与反馈环路的简单寄存器组实现。

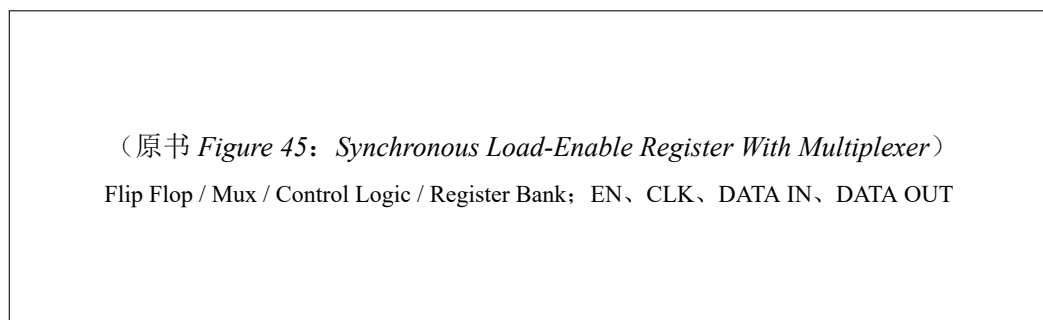


图 4.1: 带多路选择器的同步加载使能寄存器

当同步加载使能信号（EN）为逻辑 0 时，寄存器组被禁用。在此状态下，电路用多路选择器将寄存器组中每个存储元件的 Q 输出反馈到 D 输入。当 EN 为逻辑 1 时，寄存器使能，允许新值加载到 D 输入。

这些反馈环路可能不必要地消耗功耗。例如，若同一值在多个时钟周期内被重新加载到寄存器（EN 等于 0），寄存器组及其时钟 net 在寄存器组取值不变时仍消耗功耗；多路选择器也会消耗功耗。时钟门控通过在寄存器的时钟 net 上插入门控，消除图 4.1 所示的反馈 net 与多路选择器。

注意

应用时钟门控技术时，工具将生成时钟（generated clocks）视为与已定义时钟类似。

时钟门控单元选择性地阻止时钟边沿，从而使被门控的时钟信号不再对门控寄存器打拍。

图 4.2 给出基于锁存器的时钟门控单元，以及相对时钟信号 CLK 的波形。

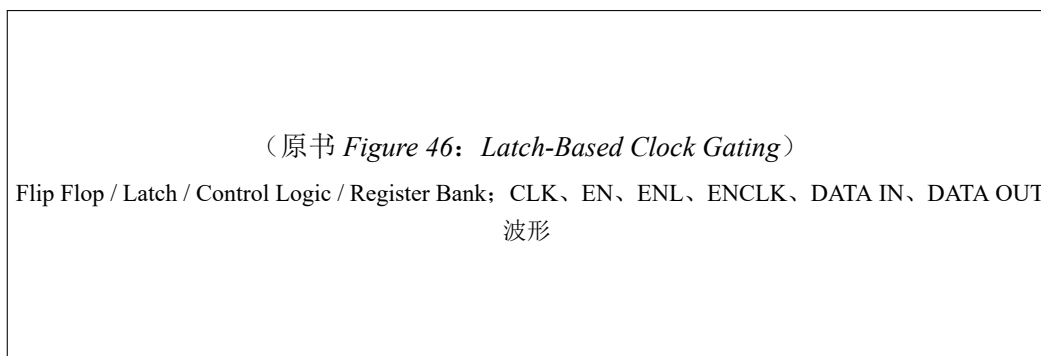


图 4.2: 基于锁存器的时钟门控

寄存器组的时钟输入 ENCLK 由 AND 门门控开或关。ENL 是控制门控的使能信号；它来自图 4.1 所示多路选择器上的 EN 信号。寄存器组由 ENCLK 信号的上升沿触发。

锁存器防止 EN 信号上的毛刺传播到寄存器的时钟 pin。当二输入 AND 门的 CLK 输入为逻辑 1 时，若无锁存器，EN 上的任何毛刺都可能传播并破坏寄存器时钟信号。锁存器消除了这种可能，因为它在时钟为逻辑 1 时阻塞信号变化。

在基于锁存器的时钟门控中，AND 门在时钟后沿之后保持时钟信号取值，从而阻塞不必要的时钟脉冲。例如，对于由 HDL 构造推断的上升沿时钟触发器，时钟门控在时钟下降沿之后将门控时钟强制为 0。

通过控制寄存器组的时钟信号，可消除在多个时钟周期内向寄存器重新加载同一值的需要。时钟门控在寄存器组的时钟网络中插入时钟门控电路，从而消除不必要的寄存器活动。

时钟门控可：

- 降低时钟网络功耗耗散
- 放宽数据通路时序
- 通过消除反馈多路选择器环路降低拥塞

对于具有大型寄存器组的设计，时钟门控可通过减少设计中的门数来节省功耗与面积。但对于较小的寄存器组，向时钟树增加逻辑的开销，可能不及消除少量反馈 net 与多路选择器所节省的功耗划算。

4.1.1 工具插入时钟门控的命名约定

Naming Convention for Tool-Inserted Clock Gates

Fusion Compiler 工具插入的时钟门控使用跨运行一致的命名约定，格式如下：

`prefix_base-name[_index]`

- 前缀默认为 `clock_gate`。要更改前缀，设置 `compile.clockgate.clock_gate_name_prefix` 应用选项。
- 基名 (base name) 是该时钟门控所门控寄存器中出现最频繁的实例名。基名不含总线位索引。
- 若需解决时钟门控之间的命名冲突，工具会添加下划线分隔符与从 0 开始的整数索引。

例如，若工具为名为 `out_reg` 的寄存器插入时钟门控，则该时钟门控命名为 `clock_gate_out_reg`。若工具为名为 `out_reg` 的寄存器再插入第二个时钟门控（可能在同一总线的不同位上），则命名为

clock_gate_out_reg_0。

但若工具为名为 out_reg_1 的寄存器插入时钟门控，为避免与 out_reg 相关时钟门控名冲突，插入的时钟门控命名为 clock_gate_out_reg_1_0。

4.1.2 编译日志文件中的时钟门控

Clock Gating in the Compile Log File

编译日志文件包含时钟门控信息，可帮助调试简单的设置问题。

默认情况下，时钟门控信息显示在下列阶段之后：

- initial_map 阶段中的时钟门控插入
 - logic_opto 阶段中的时钟门控重构
 - initial_opto 阶段中的时钟门控重构
 - 若已启用，initial_opto 阶段中的自门控插入
- 所报告信息包括：
- 带有 dont_touch 属性的已有时钟门控数量（该属性会阻止优化，可能导致不理想结果）
 - 具有 CTS purpose 的时钟门控库单元数量（插入所需）
 - 时钟门控度量，例如元件数与被门控寄存器数
 - 未能门控的寄存器原因汇总

若启用了自门控或基于扇入的时序时钟门控功能，其度量会单独显示。

包含自门控的运行，其编译日志示例如下：

Number of Pre-Exsting Clock Gates with dont_touch Attribute: 0			
Number of ICG library cells with CTS purpose: 12			

Tool Gated Register Summary			

Clock Gating Type	Number of Clock Gates	Register Count	Bitwidth
			Equivalent

Regular Clock Gating	1	2	2
Self Gating	0	0	0

Regular Clock Gating Ungated Register Summary			

Ungated Reason	Count	Bitwidth	

Enable is Constant One	2	2	

Self Gating Ungated Register Summary			

Ungated Reason	Count	Bitwidth	

Register is clock gated, and ineration		
is set to none	2	2
The tool as not able to find enough registers		
that are compatible to this register		
in order to be gated.	2	2

代码 4.1: 含自门控的编译日志示例

4.1.3 时钟门控级数与阶段

Clock-Gate Levels and Stages

工具在时钟门控命令与选项中对时钟门控级数（level）与阶段（stage）使用下列定义：

- 时钟门控级数与时钟门控的扇入相关。
 - Level 0 指时钟源。
 - Level 1 指时钟源下游的第一个时钟门控。
 - Level 2 指由 level 1 时钟门控驱动的时钟门控。
 - Level N 指由 level N – 1 时钟门控驱动的时钟门控。
- 时钟门控阶段与时钟门控的扇出相关。
 - Stage 0 指由时钟门控驱动的寄存器。
 - Stage 1 指仅直接（或经缓冲器、反相器或层次过渡）驱动寄存器的时钟门控。此外，无法确定阶段的时钟门控（例如无负载时钟门控）也赋为 stage 1。
 - Stage N 指直接（或经缓冲器、反相器或层次过渡）至少驱动一个 stage N – 1 时钟门控的时钟门控。

工具分别统计全部时钟门控与工具插入时钟门控的级数与阶段。

图 4.3 给出含两个已有时钟门控（P1、P2）与三个工具插入时钟门控（T1、T2、T3）的简单电路。表 4.1 列出各时钟门控的级数与阶段。

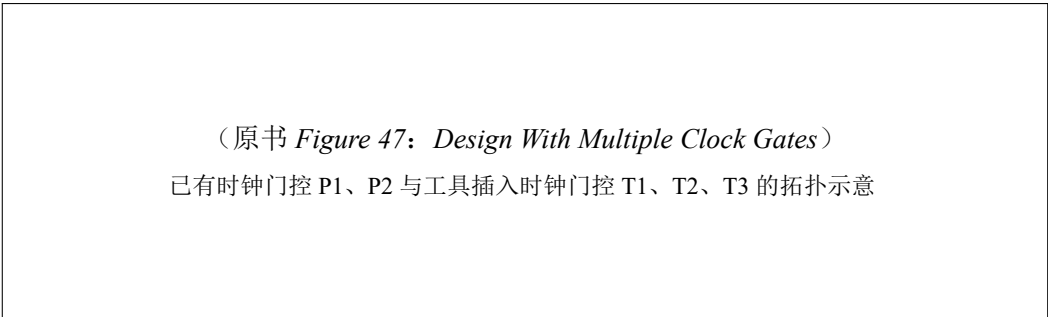


图 4.3: 含多个时钟门控的设计

表 4.1: 时钟门控级数与阶段

对象	类型	Level (全部)	Level (工具插入)	Stage (全部)	Stage (工具插入)
P1	已有时钟门控	1	0	4	2
P2	已有时钟门控	2	0	3	2
T1	工具插入时钟门控	3	1	2	2
T2	工具插入时钟门控	4	2	1	1
T3	工具插入时钟门控	3	1	1	1

4.2 时钟门控先决条件

Clock-Gating Prerequisite Conditions

在对寄存器的时钟信号实施门控之前，Fusion Compiler 工具会检查若干时钟门控条件是否满足。仅当全部时钟门控条件均满足时，工具才插入时钟门控：

- 电路表现出同步加载使能功能。
- 电路满足建立（setup）条件。
- 寄存器组或寄存器组集合满足用 `set_clock_gating_options -minimum_bitwidth` 指定的最小位数。默认最小位宽为 3。

时钟门控完成后，已门控与未门控寄存器组的时钟门控条件状态会出现在时钟门控报告中。关于时钟门控报告，见“报告时钟门控结果”。

4.2.1 时钟门控条件

Clock-Gating Conditions

寄存器须满足下列条件，工具才会对其时钟信号实施门控：

- 使能条件（**Enable condition**）
若寄存器组的同步加载使能信号为常量逻辑 1、可化简为逻辑 1，或为逻辑 0，则该条件为 **false**，电路不被门控。若同步加载使能信号不是常量逻辑 1 或 0，则该条件为 **true**，并继续检查建立条件。使能条件是工具检查的第一个条件。更多信息见“时钟门控使能条件”。
- 建立条件（**Setup condition**）
该条件仅适用于无锁存器（latch-free）时钟门控。使能信号必须来自与被门控寄存器使用同一时钟的寄存器。仅当寄存器满足使能条件时才检查建立条件。更多信息见“时钟门控建立条件”。
- 宽度条件（**Width condition**）
宽度条件是对具有等价使能信号的寄存器或寄存器组进行门控的最小位数。默认值为 3。可用 `set_clock_gating_options` 的 `-minimum_bitwidth` 选项设置宽度条件。仅当寄存器同时满足使能条件与建立条件时才检查宽度条件。

4.2.2 时钟门控使能条件

Clock-Gating Enable Condition

寄存器或时钟门控的使能条件是设计中 **net** 的组合函数。寄存器的使能条件表示必须将时钟信号传递到该寄存器的状态；时钟门控的使能条件对应于必须将时钟传递到其扇出寄存器的状态。工具使用寄存器的使能条件进行时钟门控插入。

使能条件用 **net** 的布尔表达式表示。例如：

```
module TEST (en1, en2, en3, in, clk, dataout);
  input en1, en2, en3, clk;
  input [5:0] in;
  output [5:0] dataout;
  reg [5:0] dataout;
  wire enable;
  assign enable = (en1 | en3) & en2;
always @( posedge clk ) begin
  if( enable )
    dataout <= in;
  else
    dataout <= dataout;
end
endmodule
```

代码 4.2: 使能条件 Verilog 示例

在此例中，寄存器组 `dataout_reg*` 的使能条件可表示为 `en1 en2 + en3 en2`。

4.2.3 时钟门控建立条件

Clock-Gating Setup Condition

要执行时钟门控，工具要求寄存器组的使能信号与其时钟同步。这就是建立条件。

对于基于锁存器或集成时钟门控，无论使能信号与时钟是否处于同一时钟域，工具都可插入时钟门控。若使能信号与寄存器组位于不同时钟域，必须确保两个时钟域同步，且时钟门控单元的建立与保持时间满足时序要求。

一个例外是：使用 `set_input_delay` 命令指定时钟的主输入 **port**。在此情况下，输入 **port** 与该时钟同步，建立条件为真。

4.3 设置时钟门控

Setting Up Clock Gating

时钟门控功能仅支持基于锁存器的集成时钟门控单元。工具插入的时钟门控单元没有 **wrapper** 层次。

时钟门控优化使用下列默认值：

- 最大扇出为无穷大。
- 最小位宽为 3。

- 对正沿与负沿触发触发器均使用基于锁存器的预控制（precontrol）单元作为时钟门控单元。若仅有正沿或负沿时钟门控单元可用，工具在时钟门控单元两侧插入反相器。
- 必须先加载设计与库，才能设置时钟门控约束。设置时钟门控的步骤如下：

- 加载设计与库
- 用 `set_clock_gate_style` 命令设置风格
- 用 `set_clock_gating_options` 命令设置选项
- 用 `set_clock_gating_objects` 命令指定要包含或排除出时钟门控的对象
- 用下列命令允许其他类型的时钟门控单元用于优化：

```
fc_shell> set_lib_cell_purpose -include cts [get_lib_cells ...]
```

更多信息见下列主题：

- 设置时钟门控风格（Setting the Clock-Gating Style）
- 设置时钟门控选项（Setting Clock-Gating Options）
- 对设计对象启用或禁用时钟门控（Enabling or Disabling Clock Gating on Design Objects）
- 时钟门控使能源选择（Clock-Gating Enable Source Selection）
- 用户驱动的使能排除（User-Driven Enable Exclusion）
- 报告时钟门控选项（Report Clock Gating Options）

4.3.1 设置时钟门控风格

Setting the Clock-Gating Style

用 `set_clock_gate_style` 命令选择要使用的集成时钟门控单元类型。可用选项如下：

表 4.2: set_clock_gate_style 选项

选项	说明
-test_point	指定相对集成时钟门控内锁存器的测试点位置。有效取值为 none、before（默认）与 after。
-observation_output	指定库时钟门控单元必须具有观察输出 pin。有效取值为 true 与 false。
-target	指定目标时序元件类型。有效取值为 pos_edge_flip_flop 与 neg_edge_flip_flop。默认为两者皆可（both）。
-objects	时钟门控所作用的对象。

4.3.2 设置时钟门控选项

Setting Clock-Gating Options

用 `set_clock_gating_options` 命令定义所插入时钟门控单元的网表结构。可用选项如下：

表 4.3: set_clock_gating_options 选项

选项	说明
-minimum_bitwidth	指定可门控的最小累计位数。默认值为 3。
-max_fanout	指定同一时钟门控单元所门控的最大单元数。默认值为无穷大。
-objects	指定应用时钟门控选项的对象列表。设计对象可以是层次实例、电源域与设计。

4.3.3 对设计对象启用或禁用时钟门控

Enabling or Disabling Clock Gating on Design Objects

可通过覆盖时钟门控风格设置的全部必要条件，对某些设计对象启用或禁用时钟门控。set_clock_gating_objects 命令指定在 compile_fusion 期间应启用或禁用时钟门控的设计对象。

下例排除子设计 ADDER 中的全部寄存器，但保留 out1_reg 组。out1_reg 组按指定的时钟门控风格进行门控：

```
fc_shell> set_clock_gating_objects -exclude ADDER \  
-include ADDER/out1_reg[*]
```

下例设置、然后移除由 -include、-exclude 与 -clear 选项指定的包含与排除准则：

```
fc_shell> set_clock_gating_objects -include ADDER/out1_reg[*] \  
-exclude ADDER/out2_reg[*]  
fc_shell> set_clock_gating_objects \  
-clear {ADDER/out1_reg[*] ADDER/out2_reg[*]}
```

4.3.4 时钟门控使能源选择

Clock-Gating Enable Source Selection

默认情况下，工具从下列来源之一提取时钟门控使能条件：

- 直接连接到时序元件同步使能 pin 的信号，如图 4.4 所示
- 时序元件输入与输出数据 pin 反馈回路上的使能条件，如图 4.5 所示



图 4.4: 直接连接的使能信号

使用时钟门控时，可指定使能逻辑的信号源。set_clock_gating_objects 的 -enable_source 选项作用于同一命令中用 -force 或 -include 指定的全部对象。若使用 -enable_source，则不能使用 -clear、-reset 或 -exclude 选项。

(原书 *Figure 49: Enable Signal in a Feedback Loop*)

图 4.5: 反馈回路中的使能信号

`-enable_source` 选项可为下列之一:

- none
- enable_pin_only
- feedback_loop_only
- both (默认)
- prefer_enable_pin
- prefer_feedback_loop

例如, 下列命令插入始终使能的时钟门控单元:

```
set_clock_gating_objects -force [get_cells u_dfx/input_b_reg*] \
-enable_source none
```

下列命令在存在同步使能 pin 时使用其使能信号; 否则工具使用寄存器反馈回路上的使能:

```
set_clock_gating_objects -include [get_cells u_dfx/input_c_reg*] \
-enable_source prefer_enable_pin
```

4.3.5 用户驱动使能排除

User-Driven Enable Exclusion

`set_clock_gating_enable` 命令提供接口, 用于指定要从时钟门控使能中排除的信号。
`report_clock_gating -ungated` 命令会报告因使能排除而未被门控的寄存器的新原因。

`set_clock_gating_enable` 的选项与语法与对应的 **Design Compiler** 命令相同。用此命令控制时钟门控插入期间用于使能时钟门控的信号, 并可说明为何某信号不应作为时钟门控的使能条件。

`set_clock_gating_enable` 命令的语法如下:

```
status set_clock_gating_enable
[-exclude objects_to_exclude]
[-undo objects_to_remove_exclusion]
```

下图给出使能排除前的电路:

在此步骤中, 从所有用户指定的排除信号向前遍历, 直至到达端点 (主输出 port、时序单元输入 port——包括时钟门控单元、黑盒等)。若端点是时钟门控使能 pin, 则路径上的全部 pin 被标记临时设计属性, 供后续步骤使用。

下图给出使能排除后的电路:

（原书示意：使能排除前的电路）
从用户指定的排除信号向前遍历直至端点

图 4.6: 使能排除前的电路

（原书示意：使能排除后的电路）

图 4.7: 使能排除后的电路

4.3.6 报告时钟门控选项

Report Clock Gating Options

`report_clock_gating_options` 命令报告当前设计所用的时钟门控设置。命令输出包含一张表，报告每个相关对象的最小位宽与最大扇出。也可仅打印为某些对象设置的选项。

该命令可用参数如下：

- **No split:** 默认情况下，长名称会拆成多行以便阅读。启用此选项后名称打印在单行。
- **Objects:** 要查询的层次对象列表。忽略设计中其余对象；对传入的非层次单元，分别以 CGT-2500 与 CGT-2501 警告消息报错。

`report_clock_gating_options` 命令的输出如图 4.8 所示：

figures/fig-cg-rpto-opts.png

图 4.8: `report_clock_gating_options` 命令输出

当未设置任何门控选项，或设计仅设置了特定选项时，表在单行中显示已分配选项，以及可能通过 `-objects` 参数传入的其他对象。

图 4.9 给出未对特定对象设置选项时 `report_clock_gating_options` 的输出。

figures/fig-cg-rpto-no-obj.png

图 4.9: 未对特定对象设置选项时的命令输出

图 4.10 给出未对特定对象设置选项、但将设计与 `b0` 均作为 `objects` 参数传入时的输出。

figures/fig-cg-rpto-params.png

图 4.10: 无特定对象选项但作为参数传入时的命令输出

当多个对象用相同选项设置时，它们出现在同一行；若未使用 `-nosplit` 参数，名称列会分成多行；否则全部占单行。

```
report_clock_gating_options -objects {design, obj1, obj2, obj3, obj4}
```

figures/fig-cg-rpto-same.png

图 4.11: 多个对象设置相同选项时的命令输出

图 4.12 给出与图 4.11 相同场景、但使用 `-nosplit` 参数时的情况：

```
report_clock_gating_options -objects {design, obj1, obj2, obj3, obj4} \
-nosplit
```

4.4 时钟门控流程

Clock Gating Flows

Fusion Compiler 工具在 `compile_fusion` 命令期间插入时钟门控单元。下列主题描述时钟门控流程：

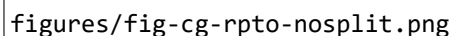


图 4.12: 使用 -nosplit 时的命令输出

- 在多电压设计中插入时钟门控 (Inserting Clock Gates in Multivoltage Designs)
- 在 RTL 设计中插入时钟门控 (Inserting Clock Gates in an RTL Design)

4.4.1 在多电压设计中插入时钟门控

Inserting Clock Gates in Multivoltage Designs

在多电压设计中，设计的不同层次可能具有不同工作条件，并使用不同的目标库子集。在多电压设计中插入时钟门控单元时，工具根据指定的时钟门控风格，以及与设计层次单元工作条件匹配的工作条件，选择合适的库单元。若未指定时钟门控风格，工具使用默认风格：测试点为 **before**，观察输出为 **false**。

若工具找不到同时适合时钟门控风格与工作条件的库单元，会发出警告消息且不插入时钟门控单元。要检查是否有可用于时钟门控插入的集成时钟门控单元，使用 `check_clock_gate_library_cell_availability` 命令。

4.4.2 在 RTL 设计中插入时钟门控

Inserting Clock Gates in an RTL Design

要在 RTL 设计中插入时钟门控逻辑并用该逻辑综合设计，按下列步骤操作：

1. 读入 RTL 设计。
2. (可选) 用 `insert_dft` 命令向设计插入测试单元。
3. 用 `compile_fusion` 命令编译设计。

编译过程中，工具对符合时钟门控资格的寄存器插入时钟门控。默认情况下，`compile_fusion` 在时钟门控插入期间使用时钟门控默认设置，并遵守逻辑库中指定的建立、保持及其他约束。要覆盖库中指定的建立与保持值，在编译设计前使用 `set_clock_gating_check` 命令。默认设置适用于大多数设计。

`compile_fusion` 命令会按需自动连接集成时钟门控单元的 scan enable 与 test port 或 pin。

4. 用 `report_clock_gating` 命令报告设计中的寄存器与时钟门控单元。用 `report_power` 命令获取时钟门控插入后设计所用动态功耗信息。

下例给出使用默认设置的典型命令序列：

```
fc_shell> read_verilog design.v
fc_shell> create_clock -period 10 -name CLK
fc_shell> compile_fusion
fc_shell> insert_dft
fc_shell> report_clock_gating
```

```
fc_shell> report_power
```

4.5 替换时钟门控

Replacing Clock Gates

若希望用库中的集成时钟门控单元替换 RTL 中全部与工艺无关的时钟门控，在执行 `compile_fusion` 之前使用 `replace_clock_gates` 命令。`replace_clock_gates` 命令识别时钟网络中充当时钟门控的组合元件并加以替换。

使用 `replace_clock_gates` 时，工具在选择库单元时遵守 `set_clock_gate_style`、`set_lib_cell_purpose` 与 `set_target_library_subset` 命令指定的约束。

当工具识别组合元件时，这些元件必须只有两个输入 `pin`（时钟 `pin` 与使能信号 `pin`）以及一个输出 `pin`。若不满足这些准则，该单元不是替换候选。下表列出 WVGTECH 实例及其替换单元。

表 4.4: 组合单元与集成时钟门控替换对照

RTL 中的组合单元	集成时钟门控替换
AND 门	上升沿触发的基于锁存器的时钟门控
OR 门	下降沿触发的基于锁存器的时钟门控
NAND 门	下降沿触发的基于锁存器的时钟门控。时钟输入 <code>pin</code> 反相。
NOR 门	下降沿触发的基于锁存器的时钟门控。两个输入 <code>pin</code> 均反相。

新时钟门控单元按下列名称推断：

```
rep_clock_gate_+<instance_name_of_combinational_cell>
```

工具将已替换的时钟门控报告为工具插入的时钟门控。

4.6 控制时钟门控延迟

Controlling Clock-Gate Latencies

综合期间，Fusion Compiler 工具假定时钟为理想时钟（`ideal clock`）。理想时钟经时钟网络无延迟。作此假定是因为真实时钟网络延迟要到时钟树综合之后才可知。实际上时钟并非理想，经时钟网络存在非零延迟。对于含时钟门控的设计，寄存器处的时钟网络延迟与时钟门控单元处不同。寄存器与时钟门控单元之间时钟网络延迟的差异，会使时钟门控单元使能输入的建立条件约束更紧。

工具可通过下列方式获得时钟门控延迟：

- 集成延迟估计（**Integrated latency estimation**）
默认情况下，工具在整个流程中估计并更新时钟门控延迟。估计的时钟延迟比用户指定的时钟延迟更准确。
- 用户指定延迟（**User-specified latency**）
可对特定实例禁用集成延迟估计，并手动指定延迟值。

4.6.1 集成时钟门控延迟估计

Integrated Clock-Gate Latency Estimation

默认情况下，工具在整个 `compile_fusion` 命令流程中估计并更新时钟门控延迟。这确保在时钟树综合之前时钟仍为理想时，工具在数据通路优化与有用偏斜（`useful-skew`）优化期间使用准确的时钟门控延迟。

若计划对设计执行结构化多源时钟树综合（`structural multisource clock tree synthesis`），请将 `opt.clock_latency_estimation.estimate_smscts_subtrees` 应用选项设为 `true`，以确保工具估计适合结构化多源时钟子树的时钟门控延迟。

为提高估计时钟门控延迟的准确性，在运行 `compile_fusion` 之前指定时钟树综合设置，包括时钟树缓冲器库单元列表、时钟树反相器库单元列表以及时钟非默认布线规则。

估计的时钟延迟比用户指定的时钟延迟更准确。此外，工具在布局与多比特组库（`multibit banking`）等步骤之后更新估计的时钟延迟，而用户指定延迟是静态值。

要阻止工具估计特定时钟门控的延迟，用 `set_attribute` 命令在其时钟 `pin` 上设置 `dont_estimate_clock_latency` 属性，如下例：

```
fc_shell> set_attribute \
[get_pins {ICG21/CK}] dont_estimate_clock_latency true
```

为获得最佳结果，请勿将下列功能与集成时钟门控延迟估计一起使用：

- 试探时钟树综合（`Trial clock tree synthesis`）
要禁用此功能，将 `compile.flow.trial_clock_tree` 应用选项设为 `false`。
- 时钟门控优化（`Clock-gate optimization`）
要禁用此功能，将 `compile.flow.optimize_icgs` 应用选项设为 `false`。
- 布局感知时钟门控（`Placement-aware clock gating`）
无论 `compile.clockgate.physically_aware` 应用选项如何设置，启用时钟门控延迟估计时工具都会自动禁用此功能。
- 用户指定延迟
用户指定延迟由 `set_clock_latency`、`set_clock_gate_latency`、`set_clock_gating_check-setup` 或 `set_path_margin -setup` 命令设置。

工具不会修改你在时钟门控单元上指定的任何 `set_clock_latency` 约束。工具估计的时钟门控延迟值存储为相对用户指定值的偏移（`offset`）。可用 `write_script -format icc2` 命令查看偏移值，并在生成输出中检查 `set_clock_latency -offset` 命令。若未设置任何延迟值，所报告偏移等于估计延迟值。

估计的延迟偏移值不会被捕获到 `write_sdc` 命令生成的 SDC 文件中。

若时钟门控及其门控寄存器的相对位置在时钟树综合期间不变，延迟估计最准确。要保留集成时钟门控的布局，在时钟树综合前将 `cts.compile.enable_cell_relocation` 应用选项设为 `auto`，以启用时钟网络单元的自动重定位。

4.6.2 用户指定时钟门控延迟

User-Specified Clock-Gate Latency

不能禁用总体的集成时钟门控延迟估计。但可通过在时钟 pin 上设置 `dont_estimate_clock_latency` 属性，对特定时钟门控禁用该功能。本节说明如何在这些时钟门控上手动指定延迟值。

用 `set_clock_latency` 或 `set_clock_gate_latency` 命令指定时钟网络延迟。`set_clock_gate_latency` 命令可用于门级与 RTL 设计。

用 `set_clock_gate_latency` 指定的时钟延迟，在 `compile_fusion` 命令插入时钟门控单元时标注到寄存器上。但若在编译后修改时钟门控上的延迟值，必须用 `apply_clock_gate_latency` 命令手动将延迟值应用到已有时钟门控单元。

注意

用 `set_clock_gate_latency` 修改时钟门控延迟后，若再用 `compile_fusion` 编译设计，无需用 `apply_clock_gate_latency` 应用延迟值；工具会在编译期间标注指定值。

要移除先前用 `set_clock_gate_latency` 或 `apply_clock_gate_latency` 在时钟门控单元上设置的时钟延迟信息，使用 `reset_clock_gate_latency` 命令。该命令移除指定时钟上的时钟延迟值。若不指定时钟，则移除全部时钟门控单元上的时钟延迟值。

4.6.3 `set_clock_latency` 命令

The `set_clock_latency` Command

用 `set_clock_latency` 命令为特定时钟门控单元指定时钟网络延迟。

在图 4.13 中，`lat_cgtoreg` 是从时钟门控单元时钟 pin 到门控寄存器时钟 pin 的估计延迟，`lat_reg` 是到无时钟门控寄存器时钟 pin 的估计时钟网络延迟。

figures/fig-clk-lat-cg.png

图 4.13: 含时钟门控设计的时钟延迟

对于设计中由特定时钟驱动的全部寄存器（已门控或未门控）时钟 pin，对 `set_clock_latency` 使用 `lat_reg` 值。对全部时钟门控单元的时钟 pin，对 `set_clock_latency` 使用 `lat_reg` 与 `lat_cgtoreg` 之差。由于设置延迟值的目的是计入寄存器与时钟门控单元之间不同的时钟网络延迟，重要的是获

得差值（`lat_cgtoreg`）的合理准确值。除非用这些值处理与时钟门控无关的时钟网络延迟问题，否则绝对值本身并不那么重要。

4.6.4 `set_clock_gate_latency` 命令

The `set_clock_gate_latency` Command

使用 `compile_fusion` 时，时钟门控在编译过程中插入。要在工具插入时钟门控单元之前指定时钟网络延迟，使用 `set_clock_gate_latency` 命令。该命令允许将时钟门控单元的时钟网络延迟指定为时钟域、时钟门控阶段以及时钟门控单元扇出的函数。你指定的延迟在 `compile_fusion` 插入时钟门控单元时标注到其上。也可用 `apply_clock_gate_latency` 命令将延迟值手动标注到设计中已有的时钟门控单元。

`set_clock_gate_latency` 命令接受下列参数：

- `-clock clock_list`
- `-stage clock_gate_stage`
- `-fanout_latency fanout_list`

`fanout_list` 是指定扇出范围与延迟减量的元组列表。下例创建三个扇出范围：1 到 5，延迟值 0.5；6–15，延迟值 0.8；16 到无穷大，延迟值 0.6。

```
set_clock_gate_latency -stage 1 \
    -fanout_latency {{1-5 0.5} {6-15 0.8} {16-inf 0.6}}
```

要为已时钟门控的寄存器指定时钟延迟值，将 `-stage` 选项设为 0。由于是为已时钟门控寄存器指定延迟值，`-fanout_latency` 的值应为 `1-inf`（1 到无穷大），且延迟是寄存器的绝对时钟延迟值，如下例：

```
set_clock_gate_latency -clock CLK -stage 0 \
    -fanout_latency {1-inf 1.0}
```

若不指定 `-clock`，设置应用于设计中全部时钟。

用 `set_clock_latency` 设置的时钟延迟优先级更高，不会被覆盖。

图 4.14 给出时钟门控阶段与扇出的示例。

下列命令为图 4.14 设置延迟值：

```
set_clock_gate_latency -stage 0 -fanout_latency {{1-inf 1.0}}
set_clock_gate_latency -stage 1 -fanout_latency {{1-inf 0.3}}
set_clock_gate_latency -stage 2 -fanout_latency {{1-inf 0.2}}
set_clock_gate_latency -stage 3 -fanout_latency {{1-inf 0.1}}
```

图 4.15 给出另一扇出与延迟计算示例。

下列命令为图 4.15 指定延迟与扇出：

```
set_clock_gate_latency -stage 0 \
    -fanout_latency {{1-inf 1.6}}
set_clock_gate_latency -stage 1 \
    -fanout_latency {{1-20 0.6} {21-inf 0.8}}
set_clock_gate_latency -stage 2 \
    -fanout_latency {{1-20 0.2} {21-inf 0.3}}
```



图 4.14: 时钟门控阶段与延迟计算



图 4.15: 变化扇出下的延迟计算

```
set_clock_gate_latency -stage 3 \
    -fanout_latency {{1-30 0.1} {31-65 0.8} {66-inf 0.18}}
set_clock_latency "0.4" {uicg_a/CK}
```

注意：对时钟门控 `uicg_a`，延迟值 0.4 由 `set_clock_latency` 赋值，不可被覆盖。

4.7 控制时钟门控级数

Controlling the Number of Clock-Gate Levels

时钟门控级数是时钟源与寄存器之间时钟门控的计数。标准时钟门控插入提供一级时钟门控插入。但设计中常含已有时钟门控，此时设计可能包含多级时钟门控。

可通过允许额外级数的时钟门控、限制时钟门控级数，或两者兼用，修改默认时钟门控插入操作。

4.7.1 时钟门控多级扩展

Clock-Gate Multilevel Expansion

手动确定最佳时钟门控级数很难。Fusion Compiler 工具可分析全部时钟门控，并插入额外时钟门控以利用共享使能信号。插入额外时钟门控可能改善功耗、面积或其他度量，取决于设计。若六个或更多时钟门控共享使能信号，工具插入额外一级时钟门控。新时钟门控再被考虑进一步级数扩展。因此，该操作期间工具可能插入多级时钟门控。

要允许工具扩展时钟门控级数，将 `compile.clockgate.enable_level_expansion` 应用选项设为 `true`（默认为 `false`），并将 `compile.clockgate.max_number_of_levels` 应用选项设为非零值（默认为 0）。此时，工具添加时钟门控，直至满足下列任一条件：

- 全部公共使能信号已识别并用于多级时钟门控扩展。
- 达到用户指定的最大时钟门控级数。

工具按下列方式命名新插入的时钟门控，取决于其使能函数被新时钟门控修改的那组时钟门控的状态：

- 若至少一个时钟门控是工具插入的，基名为 `clock_gate_m1`。名称后附加寄存器基名，类似于其他工具插入时钟门控的命名约定。
- 若全部时钟门控都是已有的，工具使用扇出最大的已有时钟门控名称，并附加 `_m1_pe`。

`compile.clockgate.max_number_of_levels` 应用选项指定允许的最大时钟门控级数。默认为 0，表示允许无穷多级。但要使用时钟门控级数扩展能力，必须将该应用选项设为非零值（此外还需将 `compile.clockgate.enable_level_expansion` 设为 `true`）。

若指定了时钟门控级数扩展，但允许的最大级数偏低，扩展操作无法充分利用工具分析公共使能信号的能力。

要对特定时钟门控启用或禁用时钟门控级数扩展，使用 `set_clock_gate_transformations -expand_levels` 命令。

下列使用说明适用：

- 新时钟门控继承由 `set_clock_gate_transformations`、`set_clock_gate_style`、`set_dont_touch`、`set_size_only`、`set_lib_cell_purpose` 与 `set_boundary_optimization` 命令设置的约束。

- 插入操作后更新时钟门控延迟估计。
- 当插入操作修改某时钟门控的使能信号时，该门控保留其原有“已有”或“工具插入”的标识。
- 若已达到最大级数，自门控不得再添加另一级时钟门控。
- 多比特单元内的集成时钟门控计入时钟门控级数，但不是级数扩展候选。
- 新时钟门控不继承时序例外。

4.7.2 时钟门控折叠

Clock-Gate Collapsing

`compile.clockgate.max_number_of_levels` 应用选项默认为 0，表示允许无穷多级。

可不启用时钟门控级数扩展而设置 `compile.clockgate.max_number_of_levels`。此时，若指定限值小于 `initial_map` 阶段之后存在的时钟门控级数，工具会折叠（collapse）时钟门控级数。折叠方式如下：

- 工具首先尝试折叠高于指定最大级数的时钟门控，从最高级（最靠近寄存器）的时钟门控开始。
- 若某特定时钟门控无法折叠，工具尝试折叠上游时钟门控，直至到达最低级（最靠近源）。折叠可跨层次进行。工具尽量保留父（上游）时钟门控。
- 工具移除其余不满足最大级数的全部时钟门控。

4.7.3 控制工具插入的时钟门控级数

Controlling Tool-Inserted Clock-Gate Levels

可指定工具应对全部时钟门控或仅工具插入时钟门控使用的最大级数。若为全部时钟门控指定值，必须知道 RTL 设计中已有多少级时钟门控。或者，可仅对工具插入时钟门控指定值。关于 Fusion Compiler 如何计数时钟门控级数的细节，见“时钟门控级数与阶段”。

要用下列互斥选项之一指定最大时钟门控级数，使用 `set_clock_gating_tree_options` 命令：

- `-max_total_levels` 指定全部时钟门控的最大级数。参数为大于或等于 1 的整数。
- `-max_tool_inserted_levels` 指定工具插入时钟门控的最大级数。参数为大于或等于 1 的整数。

要将时钟级数控制限制到特定时钟源，使用 `set_clock_gating_tree_options` 的 `-clocks` 选项，其参数为时钟列表。

若在尝试满足最大时钟级数约束时某些时钟门控不可折叠，要允许移除时钟门控（亦称 `ungating`），使用 `-ungate_if_not_collapsible` 选项。

4.7.4 基于活动的优化

Activity-Based Optimization

若加载了 SAIF 文件，Fusion Compiler 工具会分析翻转活动，以评估是否扩展工具插入的时钟门控。该分析可防止插入不必要的时钟门控。

例如，在图 4.16 中，时钟门控 A1 与 A2 各为两条路径所共有，因此是多级扩展候选。

(原书 *Figure 58: Before Clock-Gate Level Expansion*)

图 4.16: 时钟门控级数扩展前

(原书 *Figure 59: After Clock-Gate Level Expansion*)

图 4.17: 时钟门控级数扩展后

但若时钟门控 A2 始终使能, 其翻转率为 0、静态概率为 1。此时再增加一级时钟门控并无收益。最终配置如图 4.17 所示。

若启用时钟门控级数扩展, Fusion Compiler 默认执行活动驱动分析。若无 SAIF 文件, 将 `compile.clockgate.enable_activity_driven_level_expansion` 应用选项设为 `false` (默认为 `true`) 以禁用此功能。

若禁用时钟门控级数扩展, 工具忽略 `compile.clockgate.enable_activity_driven_level_expansion` 应用选项。

4.8 合并时钟门控

Merging Clock Gates

通过合并时钟门控, 工具试图在最大化时钟门控覆盖的同时最小化时钟门控数量。工具查找共享公共时钟与使能信号的时钟门控单元。该功能默认开启。

不同层次中时钟门控单元的扇出可移到公共祖先中的时钟门控单元, 前提是没有限制阻止 `port punching`。注意工具还会检查时钟门控风格与多电压约束。

带有 `dont_touch` 或 `size_only` 约束的时钟门控单元不被优化。

要禁用此功能, 使用 `set_clock_gate_transformations` 命令。

4.9 设置时钟门控变换

Setting Clock Gating Transformations

默认情况下, 综合设计时自动进行时钟门控优化。但你可定义设置, 以允许或阻止对特定对象的时钟门控优化。

工具不对时钟门控加以区分。每个时钟门控, 无论是已有还是工具插入, 都受优化约束。

限制时钟门控优化的最简单方式是使用 `set_dont_touch` 与 `set_size_only` 命令。这些命令通过阻止全部时钟门控的移除、合并与拆分, 限制所有时钟门控优化。

更精细的方法是使用 `set_clock_gate_transformations` 命令。用此命令可对每个集成时钟门控设置特定约束，并控制可对该时钟门控执行哪些操作。

`set_clock_gate_transformations` 命令提供下列选项：

- `-split_fanout`
 设为 `false` 时，即使扇出大于允许的最大值，也阻止扇出拆分。该设置也阻止布局感知时钟门控执行的任何拆分。
- `-merge_equivalents`
 设为 `true` 时，允许将时钟门控与等价时钟门控合并。
- `-collapse_levels`
 设为 `false` 时，阻止折叠冗余时钟门控。
- `-ungate_fanout`
 设为 `true` 时，指定时钟门控扇出保持未门控。
- `-reset`
 重置整个设计的全部时钟门控设置。

若在时钟门控实例或时钟门控库单元上设置了 `size_only` 或 `dont_touch` 属性，这些属性优先于 `set_clock_gate_transformations` 指定的任何约束。

4.9.1 使用时钟门控变换的示例

Example Using Clock Gating Transformations

对图 4.18 中的示例，假定设置下列限制：

```
set_clock_gating_options -minimum_bitwidth 6
set_clock_gate_transformations [get_cells uicg_b*] true \
    -ungate_fanout false
set_clock_gate_transformations [get_cells uicg_c*] false
```

(原书 Figure 60: Example of Clock Gating Restrictions)

图 4.18: 时钟门控限制示例

优化期间，工具因最小位宽违例（最小位宽为 6）移除 `uicg_a` 时钟门控。`uicg_b1` 与 `uicg_b2` 时钟门控被合并（假定它们有等价使能信号）。即使未满足最小位宽，也不允许 `ungating`，因此寄存器保持门控。`uicg_c1` 与 `uicg_c2` 时钟门控不允许任何变换，因此不对这些时钟门控执行优化。

4.10 为时钟门控设置布线规则

Setting Routing Rules for Clock Gates

在设置物理实现约束时，可为特定 `net` 设置非默认布线规则，以定义更严格（更宽）的线宽与间距要求，从而改善时序并降低串扰。要用 `create_routing_rule` 与 `set_routing_rule` 命令指定非默认布线规则。

但 `set_routing_rule` 命令不允许在工具插入时钟门控的输出 `net` 上标注，因为这些 `net` 要到执行 `compile_fusion` 之后才存在。

要指定自动应用到工具插入时钟门控输出 `pin` 所连接 `net` 的非默认布线规则，使用 `set_clock_gate_routing_rule` 命令。这些规则在 `compile_fusion` 命令之后应用到时钟门控。

例如：

```
create_routing_rule cg_rule...
set_clock_gate_routing_rule -rule cg_rule
...
compile_fusion
...
report_routing_rules
```

4.11 时钟门控与多比特寄存器

Clock Gating and Multibit Registers

在 Fusion Compiler 流程中，时钟门控与多比特寄存器映射协同工作。多比特打包比率（`multibit packing ratio`）优先于时钟门控。

下列命令启用多比特映射：

```
fc_shell> set_app_options -name compile.flow.enable_multibit -value true
```

不会插入时钟门控单元去门控预实例化的多比特寄存器。

关于时钟门控报告的更多信息，见“报告时钟门控结果”。

4.12 布局感知时钟门控

Placement-Aware Clock Gating

启用布局感知时钟门控时，工具执行下列优化：

- 复制具有时序关键使能的时钟门控
- 调整时钟门控，使其更靠近所门控的寄存器
- 为时钟门控单元自动标注时钟网络延迟

要启用此功能，将 `compile.clockgate.physically_aware` 应用选项设为 `true`。（默认为 `false`。）

图 4.19 示例给出使用与不使用物理感知时钟门控的差异。在该布局视图中，可见集成时钟门控（红色显示）在物理上更靠近其门控寄存器。

(原书 *Figure 61: Placement-Aware Clock Gating*)
 时钟门控单元红色；门控时钟 net 蓝色；Disabled vs Enabled 对比

图 4.19: 布局感知时钟门控

4.13 基于扇入的时序时钟门控

Fanin-Based Sequential Clock Gating

时序时钟门控是从数据通路中上游或下游时序元件提取并传播使能条件，以便为额外寄存器启用时钟门控插入的过程。Fusion Compiler 工具支持基于扇入的时序时钟门控：若传递扇入（transitive fanin）中的全部寄存器在上一时钟周期未改变状态，则某寄存器可在给定时钟周期被门控。

要启用此功能，将 `compile.clockgate.fanin_sequential` 应用选项设为 `true`（默认为 `false`）。该功能需要 FC-New-Tech-Power 许可证。

图 4.20 与图 4.21 说明基于扇入的时序时钟门控的一般概念。图 4.20 中的电路有寄存器组 R1，其传递扇入由寄存器组 R0 组成。组 R0 中全部寄存器共享同一使能条件 EN0，且两组寄存器均由时钟 CLK 打拍。若在某一时钟周期组 R0 中的寄存器被禁用（即使能信号 EN0 为 0），这些寄存器不改变取值。因此，在下一时钟周期，组 R1 中的寄存器也不改变取值，从而成为时钟门控候选。

(原书 *Figure 62: Before Clock Gating*)

图 4.20: 时钟门控前

图 4.21 给出时钟门控实现。首先为寄存器组 R0 插入时钟门控 CG1。工具创建新寄存器 RD1 以生成使能信号 EN0 的延迟版本。延迟信号驱动寄存器组 R1 的时序时钟门控 SEQCG1。

(原书 *Figure 63: After Clock Gating*)

图 4.21: 时钟门控后

Fusion Compiler 工具根据电路分析按需实现若干不同的时钟门控策略。例如，若扇入寄存器并不都具有相同的使能信号，工具会在使能信号的上游查找公共信号，并将这些信号用作所插入延迟

寄存器的输入。

时序时钟门控由 `compile_fusion` 命令插入。在 `initial_map` 阶段，工具分析寄存器的局部层次并按需插入时钟门控。在 `logic_opto` 阶段，工具跨层次边界进行分析并按需插入额外时钟门控。

因基于扇入的时序时钟门控而插入的时钟门控，名称以 `seq_clock_gate_fi_` 为前缀，其余部分遵循标准时钟门控命名约定。若因此功能插入延迟寄存器，寄存器名以 `reg_seqcg_` 为前缀，后接标准时钟门控命名约定（即被门控寄存器中最常见的基名）。

4.13.1 设置时序时钟门控选项

Setting Sequential Clock Gating Options

工具插入的时序时钟门控遵守 `set_clock_gate_style` 命令中的设置（该命令适用于全部时钟门控）。但下列命令专用于基于扇入的时序时钟门控：

- `set_fanin_sequential_clock_gating_options` 命令
该命令指定基于扇入的时序时钟门控允许门控的最小位宽与最大扇出。默认最小位宽为 3，默认最大扇出无限制。可指定设置所应用的设计、电源域与层次实例列表。
- `report_fanin_sequential_clock_gating_options` 命令
该命令列出设计对象的设置。
- `set_fanin_sequential_clock_gating_objects` 命令
该命令允许指定要包含进或排除出基于扇入的时序时钟门控插入的寄存器、层次单元、电源域或模块。
- `report_fanin_sequential_clock_gating_objects` 命令
该命令列出用 `set_fanin_sequential_clock_gating_objects` 显式设置的设置。

无论用 `set_fanin_sequential_clock_gating_options` 与 `set_fanin_sequential_clock_gating_objects` 做了何种设置，下列条件会阻止基于扇入的时序时钟门控插入：

- 寄存器有异步状态变化。
- 寄存器的传递扇入包含边沿触发触发器以外的对象。禁止的对象包括 `port`、锁存器以及有异步状态变化的寄存器。
- 从传递扇入寄存器提取的使能信号依赖于其输出 `pin`。
- 寄存器时钟信号的翻转率过低，时钟门控无收益。
- 传递扇入中的寄存器由不同基时钟控制，或由相反时钟边沿触发。
- 传递扇入中至少一个寄存器始终使能。
- 所分析寄存器或其传递扇入寄存器的次态函数依赖于 RTL 中含 `dont_care` 条件的逻辑。
- 所分析寄存器为多比特，且其切片不共享同一使能信号。

4.13.2 报告时序时钟门控

Reporting Sequential Clock Gates

若启用时序时钟门控，`report_clock_gating` 命令自动显示类似下表的汇总：

Sequential Clock Gating Summary			
		Bitwidth	
No of sequential clock gates	2		
No of fanin-based sequential clock gates	2 (100.0%)		
No of sequentially-gated registers	7 (21.2%)	7 (21.2%)	
No of fanin-based sequentially-gated regs	7 (21.2%)	7 (21.2%)	
No of registers not sequentially gated	26 (78.8%)	26 (78.8%)	
Total number of registers	33	33	

代码 4.3: 时序时钟门控汇总示例

用 report_clock_gating -ungated 命令报告未实现时序时钟门控的原因。报告类似如下：

fc_shell> report_clock_gating -ungated			
Not fanin-based sequentially gated registers			
Register	Reason for not gating	What's next?	
q2_reg[0]	Register contains object different than edge-triggered flip-flops in its transitive fanin	Check the register transitive fanin	
q2_reg[4]	Register contains object different than edge-triggered flip-flops in its transitive fanin	Check the register transitive fanin	
u/reg1_reg[2]	Register enable function is not a cover of the transitive fanin enable function	Check the register enable condition	
Total	7		

4.14 报告时钟门控结果

Reporting Clock-Gating Results

编译设计后，可用 report_clock_gating 命令检查结果。示例如图 4.22。

报告提供时钟门控统计汇总。“已有时钟门控（pre-existing clock gates）”指用户实例化的时钟门控。已有类别也包括由工具插入时钟门控门控的寄存器。这包括同时由已有与工具插入时钟门控单元门控的寄存器。



图 4.22: report_clock_gating 报告示例

默认情况下，工具报告多级统计，包括设计中时钟门控单元的最大级数。

时钟门控级数从时钟源向寄存器计数。

若设计中发现多比特触发器，会自动报告多比特组成（multibit composition）。

要报告未门控寄存器的细节，对 `report_clock_gating` 使用 `-ungated` 选项。要获取已门控寄存器的细节，使用 `-gated` 选项。

4.15 报告时钟门控使能信号

Reporting Clock-Gate Enable Signals

时钟门控的使能信号通常是由下列一个或多个参考（或不变）点组成的布尔表达式：

- 层次 port
- 时序输出 pin
- 黑盒输出 pin

要将时钟门控使能信号报告为布尔函数，使用 `report_clock_gating_enable_condition` 命令。必须指定要报告的时钟门控集合。

4.16 时钟门控效率报告

Clock Gate Efficiency Reporting

时钟门控效率报告功能通过考虑翻转率计算时钟门控效率。该基于翻转率的计算与功耗节省直接相关，可用于改善门控效率。

`report_clock_gating_efficiency` 命令报告用户指定时钟门控在每棵时钟树上的总体时钟门控效率。



figures/fig-cg-multibit-rpt.png

图 4.23: 多比特组成报告示例

命令语法如下:

```
report_clock_gating_efficiency
[-by_register]
[-by_clock_gate]
[-bin_size]
[-type clock_gate | self_gate | fanin_seqcg]
[-origin tool_inserted | pre_existing]
[-sortby name | stage | gating_efficiency]
```

下图给出 report_clock_gating_efficiency 命令报告:

该报告的列包括下列参数:

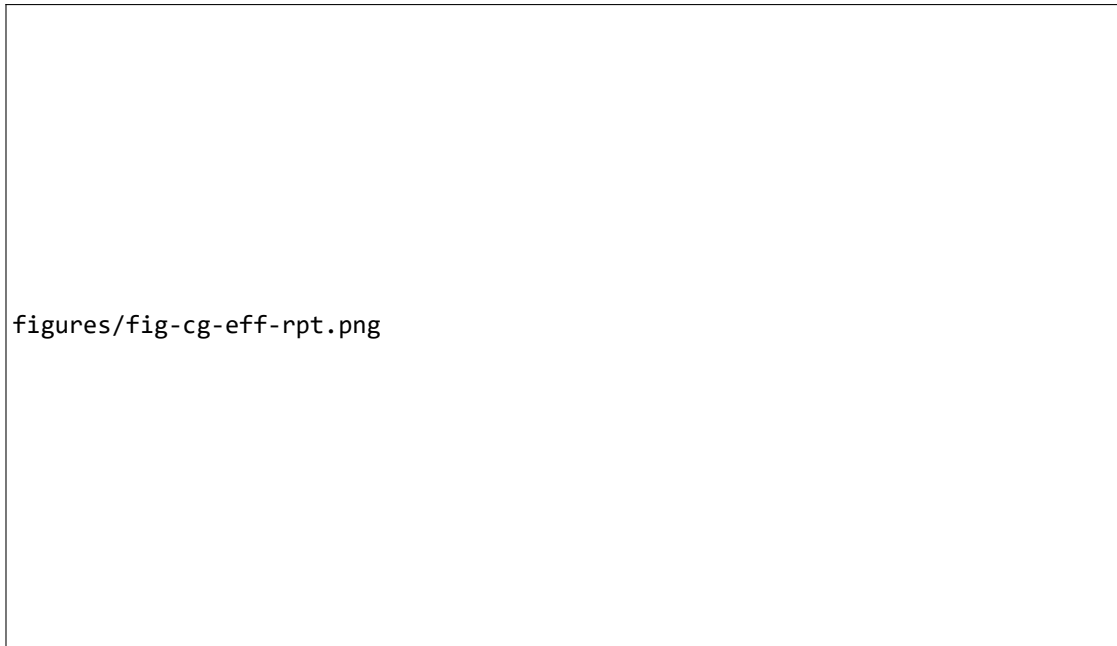
- **Toggle Savings Distribution:** 寄存器效率直方图, bin 大小为 5 (可修改)
- **Number of Registers:** 每个直方图 bin 中的寄存器数
- **Accumulated Percentage:** 迄今为止遇到的寄存器累计百分比
- **Number of Bitwidth:** 每个 bin 中寄存器数的按位等价
- **Accumulated Percentage (of the bitwidth):** 迄今为止遇到的按位寄存器等价累计百分比

报告可按下列方式细化:

- 用下列命令按 bin 大小报告效率:

```
report_clock_gating_efficiency -bin_size #numberofbin
```

- 用下列命令按寄存器报告:



figures/fig-cg-eff-rpt.png

图 4.24: report_clock_gating_efficiency 报告示例

```
report_clock_gating_efficiency -by_register
```

该报告的列包括：

- **Register:** 所考虑寄存器的名称
 - **Bitwidth:** 寄存器位宽
 - **Local Clock Toggle Rate:** 门控后寄存器时钟 pin 上的翻转率
 - **D Toggle Rate:** 寄存器 D pin 上的翻转率
 - **Q Toggle Rate:** 寄存器 Q pin 上的翻转率
 - **D/Clk Toggle Rate:** 相对本地时钟翻转率的 D 翻转率
 - **Q/Clk Toggle Rate:** 相对本地时钟翻转率的 Q 翻转率
 - **Current Gating Efficiency:** 按 $1 - (\text{local clock toggle}/\text{root clock toggle})$ 计算的寄存器效率
- 用下列命令按时钟门控报告：

```
report_clock_gating_efficiency -by_clock_gate
```

该报告的列包括：

- **Clock:** 时钟门控的根时钟
- **Clock Gating Stage:** 时钟门控阶段
- **Gating Element**
- **Number of Fanout Registers:** 时钟门控直接驱动的寄存器数
- **Number of Fanout Bitwidth:** 时钟门控直接驱动寄存器数的位宽等价

- **Toggle Savings:** 时钟门控的翻转节省
- **Clock Gating Efficiency:** 时钟门控效率
- **Receiver Clock Gate:** 所考虑时钟门控直接驱动的时钟门控名称

也可用下列命令按名称、效率或时钟门控阶段升序排序输出：

```
report_clock_gating_efficiency -sort_by [name | stage | gating_efficiency]
```

- **name:** 按字母序
- **stage:** 按时钟门控阶段
- **gating_efficiency:** 按时钟门控或寄存器门控效率

下图给出报告示例：

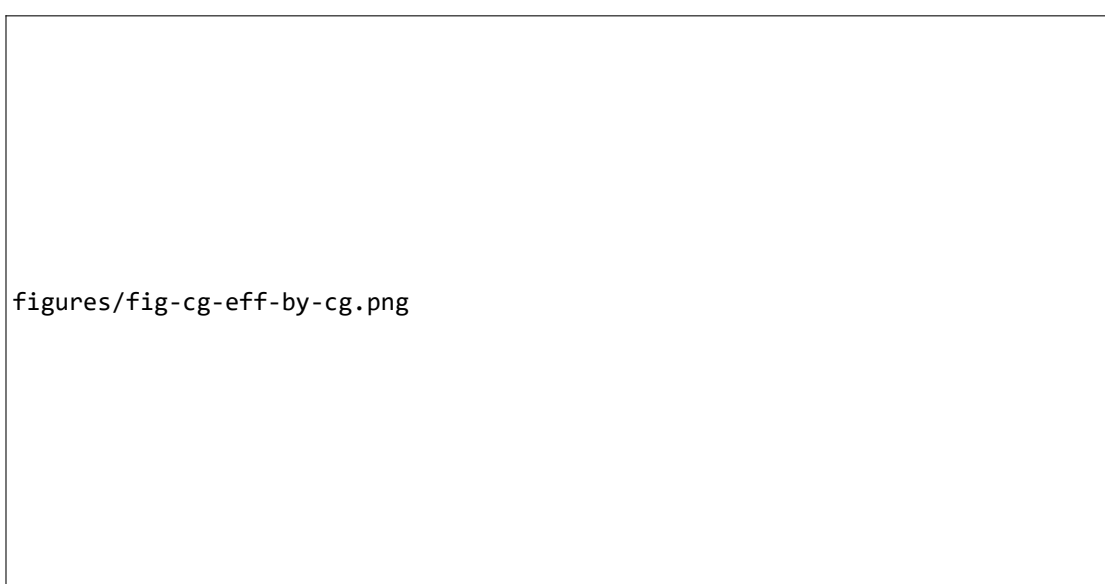


图 4.25: 时钟门控效率报告示例

4.17 自门控优化

Self-Gating Optimization

工具支持自门控优化，发生在 `compile_fusion` 命令期间。该功能在数据保持不变的时钟周期关闭某些寄存器的时钟信号，从而降低动态功耗。

无法从现有逻辑推断使能条件的寄存器，只能用自门控技术门控，不能用传统时钟门控。默认情况下，工具仅对尚未门控的寄存器支持自门控。可用 `set_self_gating_options` 命令允许对这些寄存器进行自门控。但对这些寄存器，时钟信号关闭的时间可能增加。

为确保 QoR 改善，自门控算法会考虑时序与功耗。若满足下列条件，则为寄存器插入自门控单元：

- 寄存器数据 pin 有足够时序裕量。对多 scenario 设计，算法考虑为建立启用的活动 scenario 中最差情况的时序。

(原书 *Figure 66: Example of a Self-Gating Cell*)

CLK / D / Q 自门控单元示意

图 4.26: 自门控单元示意

- 电路内部动态功耗降低。对多 scenario 设计，算法使用为动态功耗启用的活动 scenario 之间的平均内部动态功耗。

为最小化面积与功耗开销，可通过比较器单元树创建组合使能条件，使自门控单元在若干寄存器间共享。若自门控寄存器由同步置位或同步清零信号驱动，这些信号也会纳入使能信号构造，使电路功能保持不变。图 4.27 是跨两个寄存器（共 4 位）共享自门控单元的示例。注意其中一个自门控寄存器是多比特寄存器，另一个是单比特寄存器。工具也可对多比特寄存器组或单比特寄存器组进行自门控。

(原书 *Figure 67: Self-Gating Tree*)

CLK、D0/Q0、D1/Q1、D2/Q2、D/Q、EN 比较器树示意

图 4.27: 自门控树

工具不支持对下列类型的时序单元插入自门控：

- 电平敏感时序单元
- 电平敏感扫描设计寄存器
- 主从触发器
- 保持寄存器（retention registers）
- 属于移位寄存器的单比特与多比特寄存器
- 具有多条时钟路径的多比特寄存器

更多信息见下列主题：

- 自门控流程（Self-Gating Flow）
- 自门控的库要求（Library Requirements for Self-Gating）
- 自门控设置（Setting Up for Self-Gating）
- 报告与查询自门控单元（Reporting and Querying Self-Gating Cells）

4.17.1 自门控流程

Self-Gating Flow

图 4.28 说明用 `compile_fusion` 命令插入自门控的 Fusion Compiler 流程。

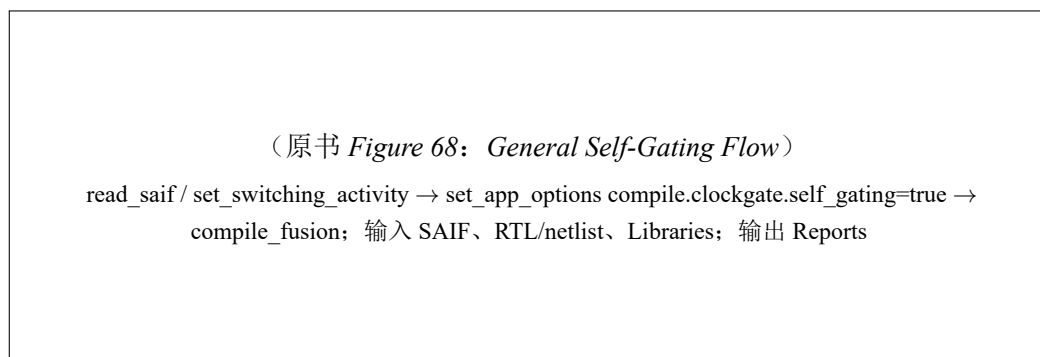


图 4.28: 一般自门控流程

4.17.2 自门控的库要求

Library Requirements for Self-Gating

要执行自门控，逻辑库应包含 AND 与 OR 门以构建自门控树。库还应包含 XOR、NAND 与 OR 门用于比较器门。库中的集成时钟门控单元必须具有下列设置：

- 时序单元：latch
- 控制点：before
- 控制信号：scan_enable
- 观察点：none

若库在对应工作条件下不含具有这些配置的单元，工具不插入自门控单元。若用 `set_clock_gate_style` 指定了与自门控兼容的集成时钟门控，自门控功能使用同一集成时钟门控，或与所指定最相似的时钟门控。

4.17.3 自门控设置

Setting Up for Self-Gating

要为块设置自门控，在运行 `compile_fusion` 之前执行下列步骤：

1. 将 `compile.clockgate.self_gating` 应用选项设为 `true` 以启用自门控。

```
fc_shell> set_app_options -name compile.clockgate.self_gating \  
-value true
```

2. 用 `read_saif` 或 `set_switching_activity` 命令施加翻转活动。
3. （可选）用 `set_self_gating_options` 命令指定自门控设置。更多信息见“设置自门控选项”。

4. (可选)用 `set_self_gating_objects` 命令覆盖工具选择自门控对象的默认行为。更多信息见“控制自门控对象的选择”。

4.17.4 设置自门控选项

Setting Self-Gating Options

要用下列选项指定自门控设置，使用 `set_self_gating_options` 命令：

- `-min_bitwidth bitwidth`
控制自门控组的最小尺寸。默认值为 4。
- `-max_bitwidth bitwidth`
控制自门控组的最大尺寸。默认值为 8。
- `-interaction_with_clock_gating interaction_type`
由用户实例化时钟门控单元门控的寄存器是自门控候选。有效参数为 `none`（跳过由工具插入时钟门控门控的寄存器）、`insert`（默认：在已门控寄存器上插入自门控）与 `collapse`（若工具插入时钟门控在同一层次则折叠它们）。
- `-objects object_list`
指定选项应应用到所列对象。支持的对象为层次实例、电源域与设计。默认情况下设置应用于整个设计。
- `-tree_type combinational_logic_type`
用指定逻辑类型构建门控使能树。有效参数为 `xor`、`nand`、`or` 与 `auto`（默认）。

要报告自门控选项，使用 `report_self_gating_options` 命令。

4.17.5 控制自门控对象的选择

Controlling the Selection of Self-Gating Objects

要覆盖选择自门控对象的默认行为，用下列选项使用 `set_self_gating_objects` 命令：

- `-force object_list`
指示工具对所列对象执行自门控，即使对象未通过内部检查。
- `-exclude object_list`
阻止工具对所列对象自门控，即使它们通过内部检查。
- `-include object_list`
指定所列对象应按 `set_clock_gating_options` 设置的插入规则进行自门控。这是设计中全部寄存器的默认设置。
- `-clear object_list`
移除所列对象上的全部包含、排除与强制准则。
- `-reset`
移除先前由 `set_self_gating_objects` 设置的全部配置。
- `-tree_type combinational_logic_type`
指定要使用的组合单元类型。有效参数为 `xor`（默认）、`or`、`nand` 与 `auto`。指定 `auto` 时，工具

根据翻转活动信息自动决定使用何种比较逻辑。这可能有助于在优化功耗的同时减小面积。必须与 `-include` 或 `-force` 选项一起使用。

要报告自门控对象，使用 `report_self_gating_objects` 命令。

4.17.6 报告与查询自门控单元

Reporting and Querying Self-Gating Cells

默认情况下，`report_clock_gating` 命令自动检测设计中自门控的存在，并在报告底部显示相应信息，如下例报告所示。

Clock Gating Summary		
Number of Clock gating elements	5	
Number of Tool-Inserted Clock gates	5 (100.00%)	
Number of Pre-Existing Clock gates	0 (0.00%)	
Number of Gated registers	48 (100.00%)	
Number of Tool-Inserted Gated registers	48 (100.00%)	
Number of Pre-Existing Gated registers	0 (0.00%)	
Number of Ungated registers	0 (0.00%)	
Total number of registers	48	
Max number of Clock Gate Levels	2	
Number of Multi Level Clock Gates	4	
Self Gating Summary		
Number of self-gating cells	3	
Number of self gated registers	20 (41.67%)	
Number of registers not self-gated	28 (58.33%)	
Total number of registers	48	

代码 4.4: 含自门控汇总的时钟门控报告

自门控单元在时钟门控汇总表中也计为时钟门控。在此例中，有两个时钟门控与三个自门控，共五个时钟门控元件。

可用 `get_clock_gates` 命令查找设计中的全部自门控。该命令返回时钟门控或自门控单元的集合。例如，下列命令返回门控由 CLK 打拍的寄存器的自门控单元：

```
fc_shell> get_clock_gates -clock CLK -type self-gate
```

4.18 DFT wrapper 时钟门控的特殊命名与查询

Special Naming and Querying for DFT Wrapper Clock Gates

DFT wrapper 时钟门控指仅驱动 DFT wrapper 单元的那些时钟门控。无论其来源如何，时钟门控引擎都会在全部 DFT wrapper 时钟门控名称前加上 `dft_wrapper_` 前缀。这有助于在自动测试图案生成（ATPG）期间识别驱动 wrapper 单元的特定时钟门控。

表 4.5: 命令及其期望返回值

命令	期望返回值
<code>get_clock_gates -gating_wrapper_cells_only</code> （返回仅驱动 wrapper 单元的全部时钟门控）	<code>dft_wrapper_ICG</code> 与 <code>dft_wrapper_clock_gate_reg</code>
<code>get_clock_gates -gating_wrapper_cells_only -origin tool_inserted</code> （仅返回仅驱动 wrapper 单元的工具插入时钟门控）	仅 <code>dft_wrapper_clock_gate_reg</code>
<code>get_clock_gates -gating_wrapper_cells_only -origin pre_existing</code> （仅返回仅驱动 wrapper 单元的已有时钟门控）	仅 <code>dft_wrapper_ICG</code>
<code>get_clock_gates {ICG clock_gate_reg} -gating_wrapper_cells_only</code> <code>get_clock_gates {ICG dft_wrapper_clock_gate_reg} -gating_wrapper_cells_only</code> （从 {} 括号内给定单元中返回仅驱动 wrapper 单元的时钟门控）	空字符串，表示这两个时钟门控都不门控 wrapper 单元
<code>get_clock_gates {*clock_gate_reg} -gating_wrapper_cells_only</code> （返回仅驱动 wrapper 单元且匹配 {} 括号内给定模式的时钟门控）	<code>dft_wrapper_clock_gate_reg</code>

此功能仅在启用 DFT wrapper 单元的时钟门控支持时有效。要为 DFT wrapper 单元启用时钟门控支持，将下列变量设为 `true`:

```
compile.clockgate.enable_dft_wrapper_cell_support
```

下图说明如何拆分时钟门控，使不同时钟门控可分别门控内部单元与 wrapper 单元：

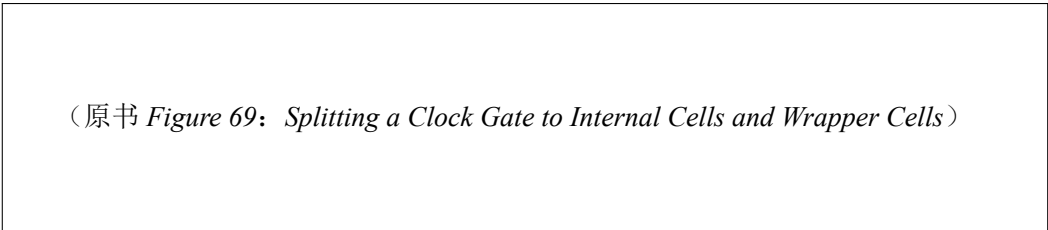


图 4.29: 将时钟门控拆分为内部单元与 wrapper 单元

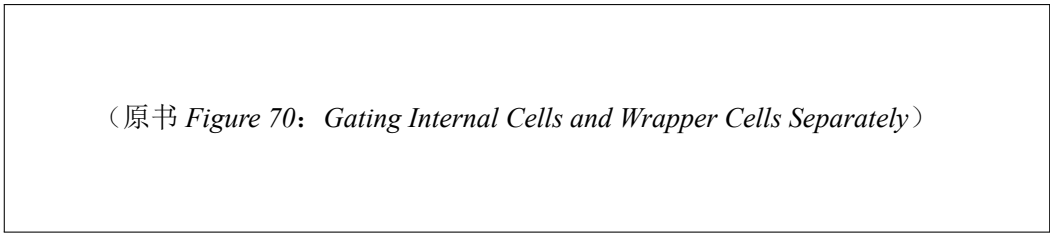


图 4.30: 分别门控内部单元与 wrapper 单元

注意

仅为门控 wrapper 单元而克隆的时钟门控，从其父时钟门控继承名称并附加 dft_wrapper_。这不适用于仅驱动 wrapper 单元的自门控名称。

表 4.6 说明不同类型时钟门控如何重命名。

表 4.6: 不同类型时钟门控如何重命名

时钟门控类型	原名	新名
工具插入	clock_gate_reg	dft_wrapper_clock_gate_reg
已有	icg_0	dft_wrapper_icg_0

对 get_clock_gates 命令使用 -gating_wrapper_cells_only 选项，可返回仅驱动 wrapper 单元的全部时钟门控（自门控除外）。该选项可与已有选项结合使用。

用 compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs 应用选项可限制（仅驱动 wrapper 单元的）已有时钟门控的重命名。该选项接受字符串列表作为值。名称匹配应用选项中任一子串的时钟门控不会被重命名。

注意

compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs 应用选项对 -gating_wrapper_cells_only 查询选项无影响。

考虑下例：

```
set_app_options -name \  
compile.clockgate.substring_list_for_dft_wrapper_dont_rename_icgs -value \  
{clk_gate_ctl clk_gate_snps}
```

这可防止对名称中含 clk_gate_ctl 或 clk_gate_snps 模式的任何已有时钟门控进行重命名，但仍可用 get_clock_gates -gating_wrapper_cells_only 查询它们。

Chapter 5

时钟树综合

Clock Tree Synthesis

要在 Fusion Compiler 中执行时钟树综合（CTS），请参阅以下主题：

- CTS 前提条件
- 定义时钟树
- 验证时钟树
- 设置时钟树设计规则约束
- 指定 CTS 设置
- 实现时钟树并执行 CTS 后优化
- 实现多源时钟树（Multisource Clock Trees）
- 分析时钟树结果

5.1 CTS 前提条件

Prerequisites for Clock Tree Synthesis

在对 block 运行 CTS 之前，应满足以下要求：

- 已用 `create_clock` 或 `create_generated_clock` 标识时钟源。
- block 已完成布局与优化。

使用 `check_legality -verbose` 确认布局合法。对布局不合法的 block 运行 CTS 可能导致运行时间过长、QoR 下降。

开始 CTS 前，估计的 QoR 应已满足要求，包括：

– 拥塞（Congestion）

若 CTS 前拥塞未解决，插入时钟树可能使拥塞恶化。若 block 拥塞，可用 `-congestion` 与 `-congestion_effort high` 重新运行 `create_placement`，但运行时间可能较长。

- 时序 (Timing)
- 最大电容 (Maximum capacitance)
- 最大转换时间 (Maximum transition time)
- 电源与地网已预布线 (prerouted)。
- 高扇出网络 (如 scan enable) 已用缓冲器综合。
- 已定义 active scenario。
默认情况下, 工具对所有为 setup 或 hold 分析启用的 active scenario 中的全部时钟进行综合与优化。

5.2 定义时钟树

Defining the Clock Trees

运行 CTS 前, 应分析 block 中每棵时钟树并确定:

- 时钟根 (clock root) 是什么;
- 所需的时钟 sink 与时钟树例外 (exceptions);
- 时钟树是否包含既有单元 (如 clock-gating 单元);
- 时钟树是否自收敛 (**convergent clock path**) 或与另一时钟树重叠 (**overlapping clock path**);
- 是否与其他时钟树存在时序关系 (如 interclock skew 要求)。

据此定义时钟树并确认工具得到的定义正确。工具根据 block 中已定义的时钟推导时钟树; 若不满足需求, 可按下列主题定义:

- 推导时钟树
- 定义时钟树例外
- 限制时钟网络上的优化
- 跨 mode 复制时钟树例外
- 由理想时钟延迟推导时钟树例外
- 处理存在平衡冲突的端点

5.2.1 推导时钟树

Deriving the Clock Trees

工具从时钟根出发, 沿传递扇出 (transitive fanout) 追踪至时钟端点以推导时钟树。一般在遇到时序单元或宏的时钟引脚时终止; 但对集成时钟门控 (ICG) 单元, 或其扇出驱动 generated clock 的时序单元, 工具会继续穿越。

若门控逻辑使用非单极性 (non-unate) 单元 (如 XOR/XNOR), 追踪时会同时使用正单极性与负单极性时序弧。若与 block 功能模式不符, 可用 set_case_analysis 将单元非时钟输入保持为常量, 迫使其进入 CTS 所需功能模式。

例如，图 5.1 所示门控逻辑中，可用下列命令使 XOR 门（U0）进入功能模式：

```
fc_shell> set_case_analysis 0 U0/B
```

（原书 *Figure 36: Non-Unate Gated Clock*）

请参阅原版 PDF 插图；可将截图放入 `figures/` 后用 `\includegraphics` 替换。

图 5.1: 非单极性门控时钟

识别时钟根

Identifying the Clock Roots

工具将 `create_clock` 定义的时钟源（输入端口或内部层次引脚）作为时钟根。

对由 `create_generated_clock` 定义的嵌套时钟树，工具将 `master-clock` 源视为时钟根，嵌套树端点视为该 `master-clock` 源的端点。例如如图 5.2 中，工具将 CLK 端口视为 `genclk1` 的时钟根。

（原书 *Figure 37: Nested Clock Tree With a Generated Clock*）

图 5.2: 带 generated clock 的嵌套时钟树

若 block 含 `generated clock`，须正确界定 `master-clock` 源，否则 skew 与时序 QoR 会变差：

- 若无法回溯到 `master-clock` 源，则无法将该 `generated clock` 的 sink 与其源的 sink 一并平衡；
- 若 `master-clock` 源并非由 `create_clock/create_generated_clock` 定义的时钟源，则无法为其或其源综合时钟树。

用 `check_clock_trees` 验证 `master-clock` 源定义，见「验证时钟树」。

指定时钟根时序特性

Specifying the Clock Root Timing Characteristics

为获得真实 CTS 结果，须正确建模时钟根时序特性：

- 若时钟根为无 I/O pad 的输入端口，须准确指定 `driving cell`。
若驱动过弱，工具可能插入额外缓冲以满足最大转换时间、最大电容等 DRC。
未指定 `driving cell`（或驱动强度）时，工具假定端口具有无限驱动能力。
例如，CLK1 为 CLK1 时钟树根，设驱动为 `mylib/CLKBUF`：

```
fc_shell> set_driving_cell -lib_cell mylib/CLKBUF [get_ports CLK1]
```

- 若时钟根为已插入 I/O pad 的输入端口，须准确指定输入转换时间。例如上升 0.3、下降 0.2：

```
fc_shell> set_input_transition -rise 0.3 [get_ports CLK1]
fc_shell> set_input_transition -fall 0.2 [get_ports CLK1]
```

识别时钟端点

Identifying the Clock Endpoints

推导时钟树时，工具识别两类端点：

- **Sink pins**（亦称 **balancing pins**）：用于延迟平衡。工具为所有 sink 赋予零插入延迟并用于平衡。CTS 中 sink 参与 DRC 与时钟时序（skew、insertion delay）的计算与优化。
- **Ignore pins**：排除在时钟时序计算与优化之外；仅参与 DRC 计算与优化。CTS 时在 ignore pin 前插入 guide buffer 以隔离；该点之外不再做 skew/insertion delay 优化，但仍做 DRC 修复。

工具将下列端点识别为 sink pins：

- 时序单元（latch 或 flip-flop）的时钟引脚（除非该单元驱动 generated clock）；
- 宏单元的时钟引脚。

工具将下列端点识别为 ignore pins：

- 位于另一时钟扇出中的时钟树源引脚。例如图 5.3 中，被驱动时钟 clk2 的源引脚是驱动时钟 clk1 的 ignore pin；被驱动时钟的 sink 不视为驱动时钟的 sink。

（原书 Figure 38: Cascaded Clock With Two Source Clocks）

图 5.3: 双源级联时钟

- 时序单元的非时钟输入引脚；
- 三态使能引脚；
- 输出端口；
- 定义不正确的时钟引脚（例如无触发沿信息，或无到输出的时序弧）；
- 用 set_case_analysis 保持常量的缓冲器/反相器输入引脚；

注意

若时钟源被 set_case_analysis 保持为常量，工具不会综合该时钟树。

- 无扇出、或无已启用时序弧的组合单元/ICG 输入引脚。

可在 GUI 中检查 sink/ignore 识别是否正确（见「在 GUI 中分析时钟树」）。若默认正确，时钟树定义即完成；否则先排查禁用时序弧与 case analysis（report_disable_timing、report_case_analysis），

删除导致错误定义的设置。必要时用 `set_clock_balance_points` 覆盖默认 `balancing/ignore` 设置，见「定义时钟树例外」。

5.2.2 定义时钟树例外

Defining Clock Tree Exceptions

时钟树例外是用户对工具为特定时钟推导的默认端点（`balancing` 与 `ignore pins`）所做的更改。工具支持：

- 用户定义 **sink pins**：在工具推导之外额外定义 sink（例如将工具推导的 `ignore pin` 定义为 sink）。见「定义 Sink Pins」。
- 用户定义插入延迟要求：为 sink（推导或用户指定）指定特殊 `insertion delay`。见「定义插入延迟要求」。
- 用户定义 **ignore pins**：排除工具推导为 sink 的端点（例如排除组合逻辑之后的所有分支）。见「定义 Ignore Pins」。

定义 Sink Pins

Defining Sink Pins

语法：

```
set_clock_balance_points
  -clock clock
  -consider_for_balancing true
  -balance_points pins
  [-corners corners]
```

`-clock` 可选；省略则对所有时钟生效。用户指定 sink 的默认 `insertion delay` 为 0；覆盖方法见「定义插入延迟要求」。默认应用于所有 corner；用 `-corners` 可限定。

例如将 U2/A 指定为当前 corner 下 CLK 的 sink：

```
fc_shell> set_clock_balance_points -clock [get_clocks CLK] \
  -consider_for_balancing true -balance_points [get_pins U2/A]
```

报告用 `report_clock_balance_points`（同时报告用户定义的 sink 与 ignore）。移除用 `remove_clock_balance_points`：

```
fc_shell> remove_clock_balance_points -clock [get_clocks CLK] \
  -balance_points [get_pins U2/A]
```

定义插入延迟要求

Defining Insertion Delay Requirements

用 `set_clock_balance_points` 的 `-delay` 覆盖 sink 默认零相位延迟。工具将指定延迟（正或负）加到到达该 sink 的计算插入延迟上。语法：

```
set_clock_balance_points
  [-clock clock]
  -delay delay
  [-rise] [-fall] [-early] [-late]
  -balance_points pins
  [-corners corners]
```

默认用于最长/最短路径以及上升/下降路径；用 **-rise/-fall/-early/-late** 控制适用范围。默认所有 corner；用 **-corners** 限定。

例如对 U2/CLK 在上升最短路径使用 2.0 ns：

```
fc_shell> set_clock_balance_points -clock [get_clocks CLK] \
  -rise -early -delay 2.0 -balance_points [get_pins U2/CLK]
```

用 **report_clock_balance_points** 报告用户定义插入延迟、带用户插入延迟的推导 sink，以及用户定义 ignore pins。

定义 Ignore Pins

Defining Ignore Pins

语法：

```
set_clock_balance_points
  [-clock clock]
  -consider_for_balancing false
  -balance_points pins
```

-clock 可选。对图 5.4 中 CLK 树，若要忽略 U2 之后的所有分支：

```
fc_shell> set_clock_balance_points -clock [get_clocks CLK] \
  -consider_for_balancing false -balance_points [get_pins U2/A]
```

(原书 *Figure 39: User-Defined Ignore Pin*)

图 5.4: 用户定义的 ignore pin

报告用 **report_clock_balance_points**。CTS 时工具添加 guide buffer 隔离 ignore pin；后续数据通路优化可修复 guide buffer 之外的 DRC。移除：

```
fc_shell> remove_clock_balance_points -clock [get_clocks CLK] \
  -balance_points [get_pins U2/CLK]
```

确保时钟树例外有效

Ensuring Clock Tree Exceptions are Valid

工具通过引脚属性 `is_clock_used_as_clock` 是否为 `true` 判断引脚是否在时钟网络上。若对属性为 `false` 的引脚用 `set_clock_balance_points` 指定例外，工具会接受但在 CTS 中忽略。

考虑图 5.5 的时钟网络：默认情况下，U2/A、U2/Y、FF3/D 的 `is_clock_used_as_clock` 为 `false`，U2/A 之后视为数据网络；U2/A 为 ignore pin，其扇出排除在 CTS 之外。CTS 会在 U2/A 前加 guide buffer；后续数据通路优化可修 DRC。

(原书 Figure 40: *Specifying Exceptions on Pins Beyond the Clock Network*)

图 5.5: 在时钟网络之外的引脚上指定例外

若希望 CTS 用时钟树约束一直修到 FF3/D 的 DRC：

1. 将 FF3/D 显式设为 ignore pin:

```
fc_shell> set_clock_balance_points -clock [get_clocks CLK] \
    -consider_for_balancing false -balance_points [get_pins FF3/D]
```

2. 用 `set_sense -clock_leaf` 强制将其视为时钟网络引脚:

```
fc_shell> set_sense -clock_leaf [get_pins FF3/D]
```

该命令会：将扇入到该 clock leaf 的时钟分支上所有引脚的 `is_clock_used_as_clock` 设为 `true`；将该 leaf 扇出上所有引脚设为 `false`，且时钟不继续传播。

检查属性：

```
fc_shell> get_attribute [get_pins FF3/D] is_clock_used_as_clock
```

5.2.3 限制时钟网络上的优化

Restricting Optimization on the Clock Network

可通过以下方式限制时钟网络部分区域的优化：

- **Don't touch:** 标识不做 CTS/优化的部分，可防止特定引脚之外被修改、特定 net 被缓冲、特定单元被改尺寸。见「设置 Don't Touch」。
- **Size-only:** 仅允许单元改尺寸；除非 placement status 为 fixed，否则仍可搬移。见「设置 Size-Only」。

设置 Don't Touch

Setting Don't Touch Settings

```
fc_shell> set_dont_touch_network -clock_only [get_pins pin_name]
```


-clock_only 仅作用于时钟网络；省略则同时作用于数据与时钟路径。清除：

```
fc_shell> set_dont_touch_network -clock_only [get_pins pin_name] -clear
```

也可对具体对象用 set_dont_touch：

```
fc_shell> set_dont_touch [get_cells cell_name] true
fc_shell> set_dont_touch [get_nets -segments net_name] true
```

报告：report_dont_touch -all。

注意

若单元 placement status 为 fixed，CTS 中按 don't touch 单元处理。

设置 Size-Only

Setting Size-Only Settings

```
fc_shell> set_size_only [get_cells cell_name] true
fc_shell> report_size_only -all
```

5.2.4 跨 Mode 复制时钟树例外

Copying Clock Tree Exceptions Across Modes

用 set_clock_tree_options 的 -copy_exceptions_across_modes、-from_mode、-to_mode，指定将例外从某一 mode 复制到一个或多个等价 mode。例如将 M1 的例外复制到 M2、M3：

```
fc_shell> set_clock_tree_options -copy_exceptions_across_modes \
    -from_mode M1 -to_mode {M2 M3}
```

5.2.5 由理想时钟延迟推导时钟树例外

Deriving Clock Tree Exceptions From Ideal Clock Latencies

若已对特定 sink 设置理想时钟延迟，可用 derive_clock_balance_points 将其转换为时钟树例外：把 set_clock_latency 在 sink 上的理想延迟转为 set_clock_balance_points。应在设计仍为理想时钟、CTS 之前运行。

默认行为：

- 为所有启用 setup/hold（或两者）的 active scenario 中的所有理想 primary clock 生成 balance point；不为 generated clock 生成。可用 -clocks/-corners 限定。
- 计算每个 primary ideal clock 的参考延迟 = set_clock_latency 在时钟上指定的理想 source 延迟 + network 延迟。可用 -reference_latency 指定不同参考值。
- 将推导的约束应用到设计；用 -output 可改为输出含 set_clock_balance_points 的文件。

推导的 balance point 按时钟与 corner 区分。须确保 CTS 所用各 mode、至少 worst corner 上对所有时钟正确设置了 set_clock_latency。

公式：

sink 处 clock balance point 延迟 = 时钟参考延迟 - sink 处总理想时钟延迟

其中总理想延迟 = 时钟 source latency + sink 的理想 network latency。

示例约束：

```
fc_shell> set_clock_latency -source 0.5 [get_clocks clk]
fc_shell> set_clock_latency 1.0 [get_clocks clk]
fc_shell> set_clock_latency 1.5 [get_pins reg1/CK]
fc_shell> set_clock_latency 0.5 [get_pins reg2/CK]
```

运行 derive_clock_balance_points 后得到：

```
fc_shell> set_clock_balance_points -delay -0.5 \
    -balance_points [get_pins reg1/CK] -clock clk -corners worst
fc_shell> set_clock_balance_points -delay 0.5 \
    -balance_points [get_pins reg2/CK] -clock clk -corners worst
```

为避免冲突，不要对已用该命令推导 balance point 的时钟再手动施加 balance point。

5.2.6 处理存在平衡冲突的端点

Handling Endpoints With Balancing Conflicts

某些 block 中存在结构上无法平衡的端点。工具可自动检测并推导 ignore pin 以解决冲突。对 synthesize_clock_trees 与 clock_opt 默认启用；设 cts.common.enable_auto_exceptions 为 false 可关闭。

可检测并修复的冲突类型：

- 含组合时钟路径模块的内部 **sink**：若模块同时含内部 sink 与组合时钟路径，无法平衡内部 sink；工具将其定义为 ignore pin。例如图 5.6(a) 网表对应 (b) 中带 check pin 的 ETM：ETM 内部 check pin 延迟来自 ETM 时序库，CTS 视其为 sink 并与顶层其他 sink 平衡；若到 check pin 为最短路径，顶层 CTS 无法在最短路径上插缓冲，导致大 skew。将其设为 ignore 可解决。

(原书 Figure 41: Module With Combinational Clock Path)

图 5.6: 含组合时钟路径的模块

- 无法同时平衡的 **sink**：两时钟驱动公共 sink，且其一因 generated clock 成为另一 sink 的 parent 时，无法同时将 parent、child 与公共 sink 平衡。工具将 parent 时钟引脚定义为 ignore pin。例如图 5.7 中，工具将 FF_gen/CLK 定义为 clkb 的 ignore pin。

(原书 *Figure 42: Clock Pins That Cannot Be Balanced Simultaneously*)

图 5.7: 无法同时平衡的时钟引脚

5.3 验证时钟树

Verifying the Clock Trees

综合前用 `check_clock_trees` 验证定义。例如：

```
fc_shell> check_clock_trees -clocks [get_clocks CLK]
```

未指定 `-clock` 时检查当前 block 全部时钟。检查项包括：

- 无 sink 的时钟（master 或 generated）；
 - 时钟网络成环；
 - 重叠时钟使多时钟到达同一寄存器，但未启用 `multiple-clocks-per-register` 传播；
 - 被忽略的时钟树例外；
 - 在输出引脚上定义的 stop/float pin；
 - 时钟树 reference 中使用了具有多条时序弧的缓冲器；
 - 导致空缓冲列表的情形；
 - generated clock 无有效 master-clock 源，包括：
 - `create_generated_clock` 中指定的 master 不存在；
 - master 未驱动 generated clock 的源引脚；
 - 源引脚由多时钟驱动，但部分 master 未在 `create_generated_clock` 中指定。
- 例如图 5.8：GEN_REG/Q 由 CLKA、CLKB 驱动，若仅指定 CLKA 为 master，则 GEN_CLK 无有效 master。

(原书 *Figure 43: Generated Clock With Invalid Master Clock Source*)

图 5.8: master 源无效的 generated clock

对多角多模（MCMM）设计，还检查：

- 冲突的 per-clock 例外设置；
- 冲突的 balancing 设置。

实现前应手动修复报告问题；每条消息有 man 页说明修复方法。修复全部问题可改善时钟树与时序 QoR。

5.4 设置时钟树设计规则约束

Setting Clock Tree Design Rule Constraints

CTS 支持以下设计规则约束：

- 最大电容：用 `set_max_capacitance` 的 `-clock_path` 指定。未指定时 CTS 默认为 0.6 pF。
- 最大转换时间：用 `set_max_transition` 的 `-clock_path` 指定。未指定时 CTS 默认为 0.5 ns。

默认作用于当前 mode 关联的所有 corner。对特定 mode：用 `get_clocks -mode`；对特定 corner：用约束命令的 `-corners`。须跨 mode 及其 corner 正确施加。

例如对当前 mode 所有 corner 的 CLK 路径设最大转换 0.20 ns：

```
fc_shell> set_max_transition 0.20 -clock_path [get_clocks CLK]
```

对不同 corner 设不同值：

```
fc_shell> set_max_transition 0.15 -corners corner1 \
-clock_path [get_clocks -mode mode1]
fc_shell> set_max_transition 0.10 -corners corner2 \
-clock_path [get_clocks -mode mode1]
```

对 mode 关联的不同 corner 设相同最大电容：

```
fc_shell> set_max_capacitance 0.6 \
-clock_path [get_clocks -mode mode2] \
-corners [get_corner \
[get_attribute [get_modes mode2] associated_corners]]
```

5.5 指定 CTS 设置

Specifying the Clock Tree Synthesis Settings

CTS 选项指导时钟树实现，包括以下主题：

- 指定时钟树 References
- 设置 Skew 与 Latency 目标
- 启用局部 Skew 优化
- 指定 CTS 主 Corner
- 阻止综合特定时钟
- 保留既有时钟树
- 启用时钟树降功耗技术
- 启用线长感知的功耗驱动搬移

- 降低电迁移（EM）
- 处理不准确约束
- 定义时钟单元间距规则
- 创建 Skew Groups
- 定义时钟单元名前缀
- 初始 CTS 使用全局路由器
- 为时钟网络指定约束
- 为特定端点构建更快时钟树
- 改进延迟驱动的时钟门分裂
- 降低时钟网络上的信号完整性影响
- 指定时钟延迟调整设置
- 报告时钟树设置

5.5.1 指定时钟树 References

Specifying the Clock Tree References

可用于构建时钟树的缓冲器/反相器，以及既有门的 reference cell，称为时钟树 references。用 `set_lib_cell_purpose -include cts` 指定。列表须满足：

- 至少含一个缓冲器或一个反相器；
- 包含既有门的库单元（否则无法在 CTS/优化中对其改尺寸）。可用 `derive_clock_cell_references` 自动推导非缓冲/反相既有单元的等价单元，见「为既有门推导 References」；
- 多电压且需 always-on 缓冲时，列表须含 always-on 单元；
- 列表中的库单元不得有 `dont_touch`（即便列入 reference 也不会被 CTS 使用）。

为确保仅使用指定单元、且这些单元不被 CTS 以外的优化使用：

```
fc_shell> set cts_cells list_of_cells
fc_shell> set_lib_cell_purpose -exclude cts [get_lib_cells]
fc_shell> set_lib_cell_purpose -include none [get_lib_cells $cts_cells]
fc_shell> set_lib_cell_purpose -include cts [get_lib_cells $cts_cells]
```

为既有门推导 References

Deriving Clock Tree References for Preexisting Gates

须将等价单元指定为 clock references，否则影响 QoR。`derive_clock_cell_references` 会为非缓冲/反相的既有时钟单元在等价库单元上设置 `valid_purpose`；不推导缓冲器与反相器的等价单元，须手动指定：

```
fc_shell> set_lib_cell_purpose -include cts \
    {tech_lib/clk_buf* tech_lib/clk_inv*}
fc_shell> derive_clock_cell_references
fc_shell> synthesize_clock_trees
```

用 `-output` 可生成 Tcl 脚本而非直接应用：

```
fc_shell> derive_clock_cell_references -output cts_leq_cells.tcl
fc_shell> set_lib_cell_purpose -include cts \
    {tech_lib/clk_buf* tech_lib/clk_inv*}
fc_shell> source cts_leq_cells.tcl
fc_shell> synthesize_clock_trees
```

限制所用目标库

Restricting the Target Libraries Used

用 `set_target_library_subset -clock` 可限制顶层或逻辑层次下级所用库；须将 `opt.common.enable_target_library_subset_opt` 设为 1。例如全局允许 HVT/LVT 的 `buf`，但 TSK_BLK 仅用 LVT_lib：

```
fc_shell> set_lib_cell_purpose -include cts \
    {HVT_lib/buf1 HVT_lib/buf2 LVT_lib/buf1 LVT_lib/buf2}
fc_shell> set_target_library_subset -clock {LVT_lib} \
    -objects [TOP/TSK_BLK]
fc_shell> set_app_options \
    -name opt.common.enable_target_library_subset_opt -value 1
```

5.5.2 设置 Skew 与 Latency 目标

Setting Skew and Latency Targets

默认 CTS 追求所有时钟的最佳 skew 与 latency，对低频时钟可能带来面积、功耗与运行时间开销。

- skew 目标: `set_clock_tree_options -target_skew`
- latency 目标: `set_clock_tree_options -target_latency`

默认作用于所有时钟与所有 corner；用 `-clocks/-corners` 限定。报告: `report_clock_tree_options`。
移除: `remove_clock_tree_options (-all 或相应选项)`。

5.5.3 启用局部 Skew 优化

Enabling Local Skew Optimization

默认最小化全局 skew（最长与最短时钟路径之差）。若最长/最短路径寄存器之间无时序路径，优化全局 skew 未必改善 QoR；优化局部 skew（时序路径 launch/capture 寄存器间最坏 skew）可改善 QoR。局部 skew 优化针对所有违例的 setup/hold 路径。

对 `synthesize_clock_trees` 与 `clock_opt` 的综合与优化阶段启用：

```
fc_shell> set_app_options -list {cts.compile.enable_local_skew true}
fc_shell> set_app_options -list {cts.optimize.enable_local_skew true}
```

启用后默认由工具推导有助于 QoR 的 skew 目标，并忽略 `set_clock_tree_options -target_skew`。若要阻止自动推导：

```
fc_shell> set_app_options \
-name cts.common.enable_auto_skew_target_for_local_skew -value false
```

5.5.4 指定 CTS 主 Corner

Specifying the Primary Corner for Clock Tree Synthesis

初始 CTS 默认识别时钟延迟最差的 corner，并在该 corner 的所有 mode 中插入缓冲以平衡延迟。用 `cts.compile.primary_corner` 指定主 corner。

5.5.5 阻止综合特定时钟

Preventing Specific Clocks From Being Synthesized

默认综合所有 `create_clock/create_generated_clock` 定义的时钟。用 `cts.common.skip_cts_clocks` 排除：

```
fc_shell> set_app_options \
-name cts.common.skip_cts_clocks -value CK_B
```

5.5.6 保留既有时钟树

Preserving Preexisting Clock Trees

默认 CTS 会移除所有既有时钟缓冲器与反相器。将 `cts.compile.remove_existing_clock_trees` 设为 `false` 可阻止。要保留特定既有缓冲/反相器，对其施加 `don't touch` 或 `size-only` 例外。

5.5.7 启用时钟树降功耗技术

Enabling Clock Tree Power Reduction Techniques

用 `cts.compile.power_opt_mode` 启用，见表 5.1。

表 5.1: `cts.compile.power_opt_mode` 设置

目的	设置
将 clock-gating 单元移近驱动以缩短高翻转输入网线长	gate_relocation
用聚类技术改善功耗	low_power_targets
同时使用门控搬移与聚类	all

5.5.8 启用线长感知的功耗驱动搬移

Enabling Wire Length Aware Power-Driven Relocation

工具支持改进的活动驱动时钟门搬移：控制线长劣化，并在搬移前考虑功耗影响。将 `cts.compile.power_opt_mode` 设为 `all` 或 `gate_relocation`。下列情况不搬移：

- 搬移 ICG 功耗收益很小或恶化功耗，同时恶化线长；
- 搬移线长影响大而功耗收益小。

5.5.9 降低电迁移

Reducing Electromigration

时钟单元功耗较高；局部聚集会提高电源轨电流密度，增加 EM 风险。可通过设置时钟单元间距，避免沿标准单元电源轨在垂直 strap 之间局部堆积：

1. 用 `set_clock_cell_spacing` 定义反相器、缓冲器、ICG 的间距规则（见「定义时钟单元间距规则」）；
2. 用 `synthesize_clock_trees` 或 `clock_opt` 执行 CTS；

注意

`balance_clock_groups` 也遵守时钟单元间距规则。见「平衡不同时钟树之间的 Skew」。

3. 用 `check_legality -verbose` 检查间距违例；设置合适约束后应无违例。

5.5.10 处理不准确约束

Handling Inaccurate Constraints During Clock Tree Synthesis

实现早期约束可能不准确或过严，直接 CTS 可能导致长运行时间或差 QoR。可设：

```
fc_shell> set_app_options \
    -name cts.common.enable_dirty_design_mode -value true
```

此时工具会：

- 忽略时钟网上的 `dont_touch/dont_touch_network`；
- 移除过紧的最大转换/最大电容约束；
- 移除小于 50 microns 的最大网长约束；
- 若水平间距 $> 3 \times \text{site height}$ 或垂直间距 $> 20 \times \text{site height}$ ，忽略时钟单元间距规则；
- 若 NDR 的 `width+spacing` $> 10 \times$ （默认 `width+spacing`），忽略 NDR；
- 报告大到足以增加时钟插入延迟的 `balance point` 延迟值。

5.5.11 定义时钟单元间距规则

Defining Clock Cell Spacing Rules

用 `set_clock_cell_spacing`；至少指定 x 或 y 方向最小间距（通常两者都指定）。

- `-x_spacing`：微米，通常依赖驱动强度与时钟频率；
- `-y_spacing`：通常小于半行高。

注意

间距过大因单元远离目标 sink 簇而增加 skew。

默认作用于所有时钟单元；用 `-lib_cells/-clocks` 限定。相邻两单元若同方向都有规则，间距为两者之和。可将 `cts.placement.cell_spacing_rule_style` 设为 `maximum`，改为取较大值而非求和。

`synthesize_clock_trees`、`balance_clock_groups`、`clock_opt` 均遵守这些规则。报告：`report_clock_cell_spacings`。移除：`remove_clock_cell_spacings`（默认全部；`-lib_cells` 可限定）。

5.5.12 创建 Skew Groups

Creating Skew Groups

若希望一组 sink 仅在组内平衡，用 `create_clock_skew_group`，以 `-objects` 指定 sink。可选：`-clock`、`-mode`（默认当前 mode）、`-name`。例如：

```
fc_shell> create_clock_skew_group -name sg1 \
    -objects {reg1/CP reg2/CP reg3/CP}
```

报告：`report_clock_skew_groups`。移除：`remove_clock_skew_groups`。

5.5.13 定义时钟单元名前缀

Defining a Name Prefix for Clock Cells

用 `cts.common.user_instance_name_prefix`：

```
fc_shell> set_app_options \
    -name cts.common.user_instance_name_prefix -value "CTS_"
```

数据网上优化（含 `clock_opt`）的单元前缀用 `opt.common.user_instance_name_prefix`。

5.5.14 初始 CTS 使用全局路由器

Using the Global Router During Initial Clock Tree Synthesis

默认初始 CTS 用虚拟路由器；复杂 floorplan 可能产生拥塞热点并在时钟布线后劣化 QoR。将 `cts.compile.global_route_aware_buffering` 设为 `true`，可在 `synthesize_clock_trees/clock_opt` 的初始 CTS 阶段使用全局路由器。

5.5.15 为时钟网络指定约束

Specifying Constraints for Clock Nets

默认用默认布线规则与可用布线层。可按「指定布线资源」设置时钟网最小/最大布线层，用于 RC 估计、拥塞分析与布线。为降低线延迟，可用宽线与高层金属（NDR）；须先定义规则并赋给时钟网（见「使用非默认布线规则」）。

5.5.16 为特定端点构建更快时钟树

Constructing Faster Clock Tree to Specific Endpoints

可为指定端点构建无额外绕线/延迟平衡的更快时钟树以改善 latency。须先为所需端点创建用户 skew group，再运行 `synthesize_clock_trees` 或 `clock_opt` 的 `build_clock` 阶段：

```
fc_shell> create_clock_skew_group -objects <endpoints>
```

注意

- 若快速端点之间有平衡要求，须放入同一 skew group；
- 若无平衡要求，须为每个端点单独建 skew group；
- 所需 skew group 端点的上游时钟逻辑与默认组或其他用户 skew group 共享，可能导致为平衡其他组端点而绕线。

（原书 Figure 44: *Faster Clock Tree to Specific Endpoints*）

图 5.9: 到特定端点的更快时钟树

若一门同时驱动 skew group 与非 skew group 端点，skew group 端点可能因非组端点的平衡要求而变慢。除建 skew group 外，还须用 `split_clock_cells -to` 沿路径克隆门，以从时钟源到目标端点建立独立路径（在 CTS 之前）：

```
fc_shell> split_clock_cells -to $endpoints
```

也可用 `split_clock_cells -to <$sinks> -latency_driven` 自动识别最优克隆方案；端点须同时用 `create_clock_skew_group` 建组。

5.5.17 改进延迟驱动的时钟门分裂

Improving Latency-Driven Clock-Gate Splitting

延迟驱动的 CG 分裂现可自动识别并评估不同克隆搬移，以改善插入延迟并满足最大级数约束。启用：将 `cts.icg.latency_driven_cloning` 设为 `true`。

(原书 *Figure 45: Latency Driven Clock-Gate Splitting*)

图 5.10: 延迟驱动的时钟门分裂

5.5.18 降低时钟网络上的信号完整性影响

Reducing Signal Integrity Effects on Clock Nets

可通过推导更大间距的 NDR 并赋给无拥塞区域的时钟网改善时序。启用：

```
fc_shell> set_app_options \
    -name cts.optimize.enable_congestion_aware_ndr_promotion -value true
```

新 NDR 间距取决于原规则：

- 原 NDR 间距 $\leq 2 \times$ 默认间距：加倍，名称加后缀 `_ext_spacing`；
- 原 NDR 间距 $> 2 \times$ 默认：增加一个默认间距，后缀同上；
- 默认规则：间距加倍，新名为 `default_rule_equivalent_ndr_double_spacing`。

5.5.19 指定时钟延迟调整设置

Specifying Settings for Clock Latency Adjustments

I/O 端口通常由真实或虚拟时钟约束。CTS 后须更新约束 I/O 的时钟延迟以保持边界约束准确。用 `clock_opt` 时工具自动更新；可用 `set_latency_adjustment_options` 控制。例如按 `PC_CLK` 更新 `VCLK1/VCLK2`，排除 `VCLK3`：

```
fc_shell> set_latency_adjustment_options \
    -reference_clock PC_CLK -clocks_to_update {VCLK1 VCLK2} \
    -exclude_clocks VCLK3
```

用 `synthesize_clock_trees` 时不会自动更新；CTS 后运行 `compute_clock_latency`（遵守上述设置）。

5.5.20 报告时钟树设置

Reporting the Clock Tree Settings

用 `report_clock_settings`。默认报告当前 block 所有时钟的配置（DRC、目标 skew/latency）、references、单元间距规则、NDR。用 `-clock` 限定时钟；用 `-type`：

- `configurations`：DRC、目标 skew/latency；
- `references`：时钟树 references；
- `spacing_rules`：单元间距；
- `routing_rules`：时钟网 NDR。

5.6 实现时钟树并执行 CTS 后优化

Implementing Clock Trees and Performing Post-CTS Optimization

执行 CTS 前应保存 block，以便必要时以同一起点 refinement 目标后重跑。可用方法：

- 独立 CTS（`synthesize_clock_trees`）
- 用 `clock_opt` 综合、优化并布线时钟树
- 控制并发时钟与数据（CCD）优化
- 分裂时钟单元
- 平衡不同时钟树之间的 Skew
- 使用机器学习数据的基于全局布线优化
- 布线时钟树
- 在 CTS、优化与时钟布线中插入 Via Ladder
- CTS 后将时钟标记为 `propagated`
- 执行布线后时钟树优化
- 执行电压优化
- 将时钟树标记为已综合
- 移除时钟树

5.6.1 独立时钟树综合

Performing Standalone Clock Trees Synthesis

```
fc_shell> synthesize_clock_trees
```

该命令执行：

1. **CTS：**此前对时钟网做虚拟布线并用 RC 估计确定时序；
2. **时钟树优化：**此前对时钟网做全局布线并用 RC 提取；优化中对被修改的时钟网做增量全局布线。

注意

该命令不对时钟树做详细布线。

默认作用于所有为 setup/hold 启用的 active scenario 中的全部时钟。用 `-clock` 限定，例如：

```
fc_shell> synthesize_clock_trees -clocks [get_clocks CLK]
```

CTS 后，工具将所有 active scenario 各 mode 中的时钟设为 `propagated`。

5.6.2 用 clock_opt 综合、优化并布线时钟树

Synthesizing, Optimizing, and Routing Clock Trees With the clock_opt Command

clock_opt 用一条命令完成综合、布线与进一步优化，阶段为：

1. build_clock: 综合并优化所有 active scenario 各 mode 的全部时钟；之后设为 propagated;
2. route_clock: 对已综合时钟网做详细布线;
3. final_opto: 进一步优化、时序驱动布局与合法化，然后全局布线并做广泛的基于全局布线的优化（含增量合法化与 route patching）。

用 -from/-to 限制连续阶段；省略 -from 从 build_clock 开始，省略 -to 跑到 final_opto 完成。例如仅综合优化并详细布线：

```
fc_shell> clock_opt -to route_clock
```

因 final_opto 做全局布线，运行前应设好所有路由器相关设置（布线规则、route guide、工艺节点应用选项等）。每个 block 应只跑一次 final_opto；完成后信号网均已全局布线，后续 route_auto/route_global/route_group 会跳过全局布线。为避免后续 track assignment/详细布线出错，clock_opt 后不要改动任何全局布线形状。

CTS 中考虑压降信息

Considering Voltage Drop Information During Clock Tree Synthesis

默认 clock_opt 插入时钟缓冲时不考虑压降，可能把缓冲放在高 PG 电阻处。可用 RedHawk Fusion 分析 PG 网络：

1. analyze_rail -min_path_resistance;
2. open_rail_result 加载结果;
3. 将 clock_opt.flow.enable_voltage_drop_aware 设为 true;
4. 运行 clock_opt。

优化中对关键网络使用 NDR

Using Nondefault Routing Rules for Critical Nets During Optimization

布线前优化中可对时序关键网使用 NDR，并以软约束引导路由器。对 clock_opt 启用：

```
fc_shell> set_app_options -name clock_opt.flow.optimize_ndr \
-value true
```

在 clock_opt 中执行 CCD 优化

Performing Concurrent Clock and Data Optimization During the clock_opt Command

在数据通路优化中利用正松弛调整寄存器时钟到达时间以改善时序 QoR，称为并发时钟与数据（CCD）优化。clock_opt 默认启用；将 clock_opt.flow.enable_ccd 设为 false 可关闭。可选控制步骤：限制延迟调整值、排除边界路径、排除特定 path group/scenario/sink、控制时序/hold 优化努力、控制 I/O 时钟延迟调整、用 report_ccd_timing 分析效果（详见「控制 CCD 优化」各小节）。

控制 **clock_opt** 中的多比特优化

Controlling Multibit Optimization Performed During the clock_opt Command

- 混合驱动强度多比特单元重连：违例路径经过较弱驱动位时，可重连使路径走较强驱动位。启用：`clock_opt.flow.enable_multibit_rewiring=true`。
- **CTS 后多比特 debanking**：在 `final_opto` 阶段，若不会引入 hold 违例则可 `debank` 以改善 QoR。启用：`clock_opt.flow.enable_multibit_debanking=true`。

在时钟网络上执行功耗或面积回收

Performing Power or Area Recovery on the Clock Network

启用 CCD 时，工具在 `final_opto` 对时钟单元与寄存器做时钟功耗回收。未启用 CCD 时可：

```
fc_shell> set_app_options \
-name clock_opt.flow.enable_clock_power_recovery \
-value power
```

此前须：用 `set_scenario_status` 的 `-dynamic_power/-leakage_power` 启用功耗优化 `scenario`；（可选）用 `read_saif` 提供翻转活动，否则用默认活动做动态功耗回收。面积回收则将值设为 `area`。

在 **clock_opt** 中执行 **IR-Drop** 感知布局

Performing IR-Drop-Aware Placement During the clock_opt Command

1. `clock_opt -to route_clock`;
2. 配置 RedHawk Fusion 并用 `analyze_rail -voltage_drop` 做静态/动态压降分析；
3. 将 `place.coarse.ir_drop_aware` 设为 `true`；
4. （可选）按「控制 IR-Drop 感知布局」做附加设置；
5. `clock_opt -from final_opto`。

5.6.3 控制并发时钟与数据优化

Controlling Concurrent Clock and Data Optimization

CCD 对 `place_opt` 与 `clock_opt` 默认启用；对 `route_opt` 须将 `route_opt.flow.enable_ccd` 设为 `true`。可控制：

- 限制延迟调整值
- 排除边界路径 / 特定 `path group` / `scenario` / `sink`
- 控制时序与 hold 优化努力
- 控制 I/O 时钟延迟调整
- 在 `route_opt` 中做动态压降驱动的 CCD
- 在布线前/布线后阶段指定优化目标

- 布线后启用缓冲移除
- 报告 CCD 时序；将 CCD offset 缩放到新 scenario
- 对 latch 与离散时钟门一并 skew

限制延迟调整值

Limiting the Latency Adjustment Values

- 提前量上限：ccd.max_prepone
- 推迟量上限：ccd.max_postpone

无默认值，单位为库时序单位。例如：

```
fc_shell> set_app_options -list {ccd.max_prepone 0.2}
fc_shell> set_app_options -list {ccd.max_postpone 0.1}
```

排除边界路径

Excluding Boundary Paths

默认对 block 内所有路径做 CCD。边界寄存器为输入端口传递扇出、输出端口传递扇入中的寄存器。将 ccd.optimize_boundary_timing 设为 false 可排除边界路径。可用 ccd.ignore_ports_for_boundary_identification 使部分端口的边界路径仍参与 CCD：

```
fc_shell> set_app_options -name ccd.optimize_boundary_timing \
    -value false
fc_shell> set_app_options \
    -name ccd.ignore_ports_for_boundary_identification \
    -value {IN_A OUT_A}
```

即便禁用边界寄存器 CCD，若改善共享时钟路径的内部寄存器 QoR，工具仍可优化其时钟树扇入锥。将 ccd.optimize_boundary_timing_upstream 设为 false 可阻止，但会严重限制 CCD 范围。上述设置影响 place_opt/clock_opt/route_opt。

排除特定 Path Groups / Scenarios / Sinks

Excluding Specific Path Groups, Scenarios, and Sinks

```
fc_shell> set_app_options -name ccd.skip_path_groups \
    -value {CLK1 {CLK2 scnA}}
fc_shell> set_app_options -name ccd.ignore_scenarios \
    -value {scnB}
```

排除特定 sink：

1. 对 sink 设 cts_fixed_balance_pin：

```
fc_shell> set_attribute -objects reg21/CK \
    -name cts_fixed_balance_pin -value true
```

2. 将 `ccd.respect_cts_fixed_balance_pins` 设为 `true`（阻止调整这些 sink 的延迟；其他 sink 的 CCD 仍可能间接改变其延迟）。设为 `upstream` 可阻止从这些 sink 到时钟根的时钟路径被改动。

控制时序与 **Hold** 优化努力

Controlling Timing and Hold Optimization Effort

- `ccd.timing_effort`: low/high, 默认 medium（影响 `clock_opt` 的 `final_opto` 与 `route_opt`）;
- `ccd.hold_control_effort`: medium/high/ultra, 默认 low。提高 hold 优先级会减少可修复的 setup 违例，仅在 hold 关键时使用。

控制 I/O 时钟延迟调整

Controlling the Adjustment of I/O Clock Latencies

`clock_opt` 做 CCD 时默认调整约束边界端口的 I/O 时钟延迟；随后运行 `compute_clock_latency` 时也使用 CCD 技术。不更新的情形包括：`ccd.adjust_io_clock_latency=false`、禁用边界路径 CCD、用 `ccd.skip_path_groups` 排除对应 path group、用 `set_latency_adjustment_options -exclude_clocks` 排除特定 I/O 时钟。

`route_opt` 中动态压降驱动的 CCD

Performing Dynamic-Voltage-Drop-Driven CCD During route_opt

将 `route_opt.flow.enable_voltage_drop_opt_ccd` 设为 `true`。工具用 RedHawk Fusion 识别动态压降热点与可搬移的 cell row，在优化中搬移或缩小尺寸以降低峰值动态压降且不伤害时序 QoR。可选阈值：

- `ccd.voltage_drop_voltage_threshold`: 相对供电电压的压降百分比；
- `ccd.voltage_drop_population_threshold`: 按最差违例选取的单元数百分比。

另可将 `route_opt.flow.enable_irdrivenopt` 与上述选项同时设为 `true`，启用 IR 驱动 sizing（用更小漏电单元替换）。

布线前阶段指定优化目标

Specifying Optimization Targets at the Preroute Stage

对 `place_opt/clock_opt` 的 CCD，可提高下列目标的 WNS 优先级：

- 特定 path group: `ccd.targeted_ccd_path_groups`（若同时在 `ccd.skip_path_groups` 中则仍跳过）；
- 端点文件: `ccd.targeted_ccd_end_points_file`（同时指定 path group 与端点时，仅优化指定组内的指定端点）；
- 最差 300 条路径: `ccd.enable_top_wns_optimization=true`。

布线后阶段指定优化目标

Specifying Optimization Targets at the Postroute Stage

1. 将 `route_opt.flow.enable_targeted_ccd_wns_optimization` 设为 `true`;
2. 用 `ccd.targeted_ccd_path_groups` 和/或 `ccd.targeted_ccd_end_points_file` 指定目标（至少其一）;
3. （可选）`ccd.targeted_ccd_select_optimization_moves`: `auto`（默认，含 `buffering`）、`size_only`、`equal_or_smaller_sizing`、`footprint_sizing`;
4. （可选）`ccd.targeted_ccd_wns_optimization_effort`: `high`（默认）/ `medium` / `low`;
5. （可选）`ccd.targeted_ccd_threshold_for_nontargeted_path_groups`: 非目标 `path group` 可被劣化的松弛阈值（默认正松弛可劣化至零）。

可同时启用常规 CCD（`route_opt.flow.enable_ccd`）与 `targeted` CCD；此时先常规后 `targeted`。示例脚本（设计已布线并做过布线后优化）：

```
open_lib design1
open_block blk.post_route
set_app_options -name route_opt.flow.enable_ccd -value false
set_app_options \
  -name route_opt.flow.enable_targeted_ccd_wns_optimization -value true
set_app_options -name ccd.targeted_ccd_path_groups -value clkA
set_app_options -name ccd.targeted_ccd_end_points_file -value clkA_ep.txt
set_app_options \
  -name ccd.targeted_ccd_select_optimization_moves -value size_only
set_app_options -name ccd.targeted_ccd_wns_optimization_effort \
  -value medium
route_opt
```

布线后启用缓冲移除

Enabling Buffer Removal at the Postroute Stage

```
fc_shell> set_app_options -name route_opt.flow.enable_ccd -value true
fc_shell> set_app_options -name ccd.post_route_buffer_removal -value true
fc_shell> route_opt
```

报告 CCD 时序

Reporting Concurrent Clock and Data Timing

用 `report_ccd_timing`。默认报告 `block` 中五个最关键端点寄存器的最差 `capture`（D-slack）与 `launch`（Q-slack）的 `setup/hold`。-`type`: `fanin/fanout`、`stage`、`chain`（见图 5.11）。默认报告 `setup`；-`hold` 报告 `hold`。-`scenarios` 限定 `scenario`；-`pins` 限定端点。可用 -`prepone`/-`postpone` 与 -`annotate_cell_delay`/-`annotate_net_delay` 做 `what-if` 分析（仅报告，不保存）。

(原书 Figure 46: Previous, Current, and Next Path Stages)

图 5.11: 前一、当前与下一路径阶段

将 **CCD Offset** 缩放到新 **Scenario**

Scaling CCD Offsets to New Scenarios for Reporting Timing

用 `copy_useful_skew` 将 CCD offset 复制到新 scenario，以获得一致的时序/QoR 报告。当 pin 的 CTS balance point offset 仅在部分 active scenario 中推导时，用该命令复制到无 offset 的 inactive/active scenario，并确保在计算新 offset 的所有 pin 与 scenario 上设置相应 `set_clock_latency`。先对有 balance point offset 的 scenario 取平均，再按各 scenario corner 的缩放因子缩放。

对 **Latch** 与离散时钟门一并 **Skew**

Skewing Latch and Discrete Clock Gates

在 `compile_fusion` 或 `place_opt` 的 `final_opto` 前将 `ccd.skew_opt_merge_dcg_cells_by_driver` 设为 `true`（默认 `false`），使 latch 与门控元件使用相同 offset，保持同级并改善离散时钟门的 hold。

5.6.4 分裂时钟单元

Splitting Clock Cells

```
fc_shell> split_clock_cells -cells [get_cells U1/ICG*]
```

不分裂的情形：无 DRC 违例；有 don't-touch/size-only/fixed-placement；须在电源域或 `set_freeze_ports` 标识的 block 边界 punch port。分裂后新单元命名为 < 原名 >_split_< 整数 >，并复制原单元全部设置与约束。也可用 `-loads` 指定由同一驱动驱动的负载集合：

```
fc_shell> set loads1 [get_pins I1/reg*/CK]
fc_shell> set loads2 [get_pins I2/reg*/CK]
fc_shell> split_clock_cells -loads [list $loads1 $loads2]
```

5.6.5 平衡不同时钟树之间的 Skew

Balancing Skew Between Different Clock Trees

参与延迟平衡的时钟集合称为 clock balance group；可定义多个组，并为组内时钟定义延迟 offset；合称 interclock delay balancing 约束。

注意

工具无法在 generated clock 与其他时钟之间平衡 skew。

步骤：生成约束（手动或自动）→ 运行平衡。

定义 **Interlock** 延迟平衡约束

Defining the Interlock Delay Balancing Constraints

用 `create_clock_balance_group`: 至少 `-name` 与 `-objects` (可用 `get_clocks -mode`)。默认目标为零 `offset`, 以组内最长插入延迟平衡。用 `-offset_latencies` 指定相对 `offset`; `-corner` 限定 `corner`。例如 `clk3` 比 `clk1/clk2` 少 100:

```
fc_shell> create_clock_balance_group -name group3 \
    -objects [get_clocks {clk1 clk2 clk3}] \
    -offset_latencies {0 0 -100}
```

报告: `report_clock_balance_groups`。移除: `remove_clock_balance_groups` (指定组或 `-all`)。

自动生成约束

Generating Interlock Delay Balancing Constraints Automatically

```
fc_shell> derive_clock_balance_constraints
```

将有 `interlock` 时序路径的时钟放入同一 `balance group`。可用 `-slack_less_than` 仅考虑松弛小于阈值的路径, 例如:

```
fc_shell> derive_clock_balance_constraints -slack_less_than -0.2
```

运行 **Interlock** 延迟平衡

Running Interlock Delay Balancing

```
fc_shell> balance_clock_groups
```

对 MCMM, 对所有 `active scenario` 执行。

5.6.6 使用机器学习数据的基于全局布线优化

Performing Global-Route-Based Optimization Using Machine Learning Data

对布线前/布线后时序相关性差的设计, 可在 CTS 后用详细布线后得到的 ML 数据做优化。术语: `Labels` (预测输出)、`Features` (影响输入)、`Model` (映射关系)、`Training` (学习过程)。

迭代 **N** (采集与训练):

1. 完成 `clock_opt` 的 `build_clock` 与 `route_clock`;
2. 设 `est_delay.ml_delay_opto_dir` 为输出目录;
3. 将 `est_delay.ml_delay_gre_mode` 设为 `feature`;
4. `clock_opt -from final_opto`;
5. 详细布线;
6. 详细布线后、布线后优化前: `estimate_delay -training_labels`;
7. `estimate_delay -train_model`;

8. 继续布线后优化与后续流程。

迭代 **N+1**（应用模型）：同样完成时钟综合与布线 → 指定 ML 目录 → 将 `est_delay.ml_delay_gre_mode` 设为 `enable` → `clock_opt -from final_opto` → 详细布线 → `estimate_delay -disable_model` → 继续流程。

须保证各步骤/迭代中 `active scenario` 相同；设计、环境或流程变更时须重建模型。

5.6.7 布线时钟树

Routing Clock Trees

```
fc_shell> route_group -all_clock_nets -reuse_existing_global_route true
```

`-reuse_existing_global_route true` 使详细路由器使用既有时钟全局布线，相关性更好。或：

```
fc_shell> clock_opt -from route_clock -to route_clock
```

5.6.8 在 CTS、优化与时钟布线中插入 Via Ladder

Inserting Via Ladders During CTS, Optimization, and Clock Routing

Via ladder 是从引脚层堆叠到上层、供路由器连接的通孔叠层，可降低通孔电阻，改善性能与 EM 鲁棒性。流程：

1. 定义 via ladder 规则；
2. 用 `set_via_ladder_candidate` 指定库引脚可用的 via ladder；
3. 用 `set_via_ladder_rules` 定义约束；
4. 将 `opt.common.enable_via_ladder_insertion` 设为 `true`；
5. `clock_opt -to build_clock`；
6. `insert_via_ladders`；
7. `clock_opt -from route_clock`。

5.6.9 CTS 后将时钟标记为 Propagated

Marking Clocks as Propagated After Clock Tree Synthesis

运行 `synthesize_clock_trees` 或 `clock_opt` 后，工具移除所有 `active scenario` 中已综合时钟的 `ideal` 设置，并将对应寄存器时钟引脚设为 `propagated`。若有 CTS 前未设为 `active` 的 `scenario`：

```
fc_shell> set_scenario_status -active true [all_scenarios]
fc_shell> synthesize_clock_trees -propagate_only
```

`-propagate_only` 仅移除 `ideal` 设置，不执行 CTS。

5.6.10 执行布线后时钟树优化

Performing Postroute Clock Tree Optimization

详细布线时钟/信号网后，因全局布线与详细布线差异以及耦合/串扰，时钟 QoR 可能下降。用 `synthesize_clock_trees -postroute`，并以 `-routed_clock_stage` 指明布线阶段：

```
fc_shell> synthesize_clock_trees -postroute \
-routed_clock_stage detail
fc_shell> synthesize_clock_trees -postroute \
-routed_clock_stage detail_with_signal_routes
```

注意

启用 CCD（含用该技术的功耗/面积回收）时，布线后时钟树优化不优化 latency/skew，仅修复时钟网络上的逻辑 DRC。

5.6.11 执行电压优化

Performing Voltage Optimization

电压优化尝试在更低电压下获得更好的 PPA。需要：不同电压的 scaled library panes；以及带 scaled library 设置的 signoff 方法。高层流程：

1. 原电压 place_opt；
2. 原电压 clock_opt；
3. define_scaling_lib_group 关联相关 library panes；
4. set_vopt_range 指定 corner 电压范围；
5. set_vopt_target 指定 TNS/功耗目标与容差（可用 -excludeIO）；
6. voltage_opt；
7. 优化电压下 route_auto；
8. 优化电压下 route_opt。

5.6.12 将时钟树标记为已综合

Marking Clock Trees as Synthesized

用 `mark_clock_trees` 防止工具修改既有时钟树，见表 5.2。工具遍历时钟树直至遇到例外引脚或默认 sink。

5.6.13 移除时钟树

Removing Clock Trees

`remove_clock_trees` 从根到 sink 遍历并移除缓冲器与反相器（保留带 dont_touch/size_only 的单元）。例外之外的单元仍视为时钟树一部分；clock-to-data 引脚之外视为数据通路不移除。同时

表 5.2: 使用 mark_clock_trees 命令

目的
仅标记特定时钟（默认标记所有 active scenario 各 mode 中 create_clock/create_generated_clock 定义的时钟）
标记为已综合（默认）
清除 synthesized 属性
施加 dont_touch
清除 dont_touch
传播 set_clock_routing_rules 的 NDR
传播 set_clock_cell_spacing 间距规则
将 sink 标记为 fixed
冻结时钟网布线

重置 CTS 相关属性。

注意

remove_clock_trees 不支持 clock mesh 网。

默认移除所有 mode 的所有时钟树；用 -clocks 限定（默认当前 mode；可用 get_clocks -mode）。结构对移除的影响见表 5.3。默认同时移除被删时钟网的详细布线形状；可将网的 shape_use 设为 user_route 以保留。

表 5.3: 时钟树移除行为

对象	对移除的影响
Boundary cell	移除
Don't touch net 上的单元	保留
Don't touch / Fixed cell	保留
Generated clock	若定义在缓冲/反相器引脚上则保留；遍历与移除继续穿过
Guide buffer	移除
ICG	保留；遍历继续穿过
Block abstraction model	保留；遍历继续穿过
Inverter	仅成对移除；单独反相器不移除
Isolation / Level shifter	保留
Three-state buffer	保留；遍历在此停止
例外之外添加的缓冲/反相器	移除

5.7 实现多源时钟树

Implementing Multisource Clock Trees

本节介绍多源时钟树结构类型及实现流程：

- 多源时钟树结构简介
- 实现常规多源时钟树
- 使用集成 Tap Assignment 的常规多源时钟树
- 仅含 H-Tree 全局结构的常规多源时钟树
- 实现 MSCTS 多重 H-Tree
- 实现功耗感知柔性 H-Tree
- 实现结构型多源时钟树（SMSCTS）
- 使用集成子树综合的 SMSCTS
- 改善 SMSCTS 与 CCD 的相关性
- 插入时钟驱动器；综合全局时钟树；创建时钟 strap；布线到 strap；分析 clock mesh
- 自动 Tap 插入与 H-Tree 综合；指定 Tap Assignment 选项；构建本地子树

5.7.1 多源时钟树结构简介

Introduction to Multisource Clock Trees Structures

多源时钟树对片上偏差更宽容，跨 corner 性能优于传统结构，由以下组成：

- 全局时钟结构：时钟根、通常为 H-tree 的全局时钟树、mesh drivers、clock mesh；
- 本地子树，驱动方式可为：
 - 由连接到 mesh 的预定义 tap drivers 驱动（图 5.12）——子树用常规 CTS（`synthesize_clock_trees/clock_opt`）构建，称常规多源时钟树；
 - 直接从 mesh 多点驱动（图 5.13）——保留用户结构并通过合并/分裂优化，称结构型多源时钟树。

（原书 Figure 47: Regular Multisource Clock Tree）

图 5.12: 常规多源时钟树

(原书 *Figure 48: Structural Multisource Clock Tree*)

图 5.13: 结构型多源时钟树

5.7.2 实现常规多源时钟树

Implementing a Regular Multisource Clock Tree

1. 指定时钟树约束与设置;
2. 多电压设计: 启用 `cts.multisource.enable_full_mv_support`; 启用 `opt.common.allow_physical_feedthrough` 运行 `check_mv_design` 并修复;
3. 启用 `cts.multisource.cell_em_aware`;
4. `create_clock_drivers` 插入 tap drivers;
5. `create_clock_straps` 创建 clock mesh;
6. `create_clock_drivers` 插入 mesh drivers;
7. `synthesize_multisource_global_clock_trees` 构建全局树;
8. `route_clock_straps` 连接 mesh 与 tap;
9. `analyze_subcircuit` 分析 mesh;
10. 指定 tap assignment 选项;
11. `synthesize_multisource_clock_taps` (合并等价时钟单元、将端点赋给最近 tap、沿路径分裂、复制 UPF/SDC/属性);
12. `clock_opt -from build_clock -to route_clock` 构建由 tap 驱动的本地子树, 并修复 ignore 例外之外的 DRC;
13. 分析时钟树结果。

5.7.3 使用集成 Tap Assignment 的常规多源时钟树

Implementing a Regular Multisource Clock Tree Using Integrated Tap Assignment

若在 `place_opt` 前插入 tap drivers, 工具可在布局优化中分配 sink 以改善总体时序与时钟门控路径。流程与常规类似, 但在 mesh 分析后: 将 `place_opt.flow.enable_multisource_clock_trees` 设为 true 并运行 `place_opt`, 再 `clock_opt -from build_clock -to route_clock`。

5.7.4 仅含 H-Tree 全局结构的常规多源时钟树

Implementing a Regular Multisource Clock Tree With an H-Tree-Only Global Clock Tree Structure

全局结构无 mesh, H-tree 直接驱动 tap (图 5.14)。用 `set_regular_multisource_clock_tree_options` 与 `synthesize_regular_multisource_clock_trees` 完成 tap 插入与 H-tree 综合, 再做 tap assignment 与 `clock_opt`。

(原书 Figure 49: Regular Multisource Clock Tree With an H-Tree-Only Global Clock Structure)

图 5.14: 仅含 H-Tree 全局结构的常规多源时钟树

5.7.5 实现 MSCTS 多重 H-Tree

Implementing MSCTS Multiple H-Tree

用 `set_regular_multisource_clock_tree_options` 的 `-htree_sections` 在 floorplan 不同区域构建多个全局树 (图 5.15), 可用对称 tap 配置或坐标指定 tap 位置。

(原书 Figure 50/51: MSCTS Multiple H-Tree)

图 5.15: MSCTS 多重 H-Tree

5.7.6 实现功耗感知柔性 H-Tree

Implementing Power Aware Flexible H-Tree

在柔性 H-Tree 综合中加入功耗感知以优化总时钟功耗与总体 latency。须至少有一个动态功耗 scenario 在 H-tree 综合时为 active; 多 scenario 时取最差 active scenario 做功耗代价。初始 tap 配置基于 floorplan、sink 分布、布局与布线约束等估计各子树 latency/功耗并比较。启用: `cts.multisource.power_aware_flexible_htree_synthesis=true`。

5.7.7 实现结构型多源时钟树

Implementing a Structural Multisource Clock Tree

子树由 mesh 或预定义驱动直接驱动; 通过合并、分裂、改尺寸与搬移优化, 同时保留用户定义的级数结构。主要步骤: MV/EM 设置 → `create_clock_straps` → 插入 mesh drivers → 综合全局树 → 用 `set_annotated_delay/set_annotated_transition` 建模 mesh 延迟 (避免子树综合时看到过大延

迟) → `synthesize_multisource_clock_subtrees` → `route_clock_straps` → `analyze_subcircuit` → `clock_opt -from build_clock -to route_clock`。

5.7.8 使用集成子树综合的 SMSCTS

Implementing a Structural Multisource Clock Tree Using Integrated Subtree Synthesis

用 `place_opt` 综合优化本地子树可改善时序 QoR。额外注意:可启用 `cts.multisource.enable_multi_corner_synthesis`。用 `set_multisource_clock_subtree_options` 指定 `-clock`、`-driver_objects` 及可选的 `-dont_merge_cells`、`-balance_levels/-target_level`、`-max_total_wire_delay`、`-enable_icg_reordering` 等;启用 `place_opt.flow.enable_multisource_clock_trees` 后运行 `place_opt`;布线到 strap 并分析 mesh 后,启用 `clock_opt.flow.enable_multisource_clock_trees` 再跑 `clock_opt` 以增量优化子树。

5.7.9 改善 SMSCTS 与 CCD 的相关性

Improving the Correlation Between the SMSCTS and CCD

将 `cts.multisource.enable_subtree_synthesis_aware_ccd` 设为 `true`,可在 `place_opt` 与 `clock_opt` 的 `build_clock` 阶段改善 CCD balance point 实现与 SMSCTS 后时序,降低长短路径违例。

5.7.10 插入时钟驱动器

Inserting Clock Drivers

插入前:指定全部 CTS 设置(含布线规则);可将 `cts.multisource.enable_clock_driver_snapping` 设为 `true` 使驱动与 mesh strap 对齐。`create_clock_drivers` 可用于 mesh drivers (常规/结构型)与 tap drivers (常规)。要点:

- `-loads`: 负载网或同网引脚/端口;
- `-location` 精确位置,或 `-boxes` 网格(可配 `-boundary`);
- `-lib_cells` 或 `-template`;
- mesh drivers 用 `-short_outputs` 将输出短接驱动 mesh;
- `-configuration` 可插入多层驱动形成 H-tree (图 5.16)。

(原书 Figure 52: Clock Drivers Placed in an H-Tree Structure)

图 5.16: 按 H-Tree 结构放置的时钟驱动器

插入后驱动标记为 `fixed` 与 `don't touch`;避免与 `fixed` 单元、`blockage`、宏重叠,但可与非 `fixed` 单元重叠。命令不做合法化,需 `legalize_placement` (多次插入后统一合法化以节省时

间)。布线 H-tree 时使用时钟网规则允许的最高层；为防驱动置于预布线下方导致引脚可达性问题，会为同层预布线创建临时 placement blockage，布线后删除。多行高单元放置困难时可将 `cts.multisource.enable_pin_accessibility_for_global_clock_trees` 设为 `false`。移除用 `remove_clock_drivers`。

多级物理层次:用 `set_editability` 启用下级编辑;设 `cts.multisource.enable_mlph_flow=true`;
插入驱动;用 `synthesize_multisource_global_clock_trees -roots -leaves -use_zroute_for_pin_connections`
布线。下级优先复用既有时钟端口，否则在允许时创建新端口 (`set_freeze_ports`)。

5.7.11 综合全局时钟树

Synthesizing the Global Clock Trees

`synthesize_multisource_global_clock_trees` 综合并详细布线 H-tree 风格全局树, 尽量最小化端点 skew。须指定 `-nets` 与 `-lib_cells`。默认用 Custom Router; 引脚连接 DRC 难解时用 `-use_zroute_for_pin_connections`; `-skip_pin_connections` 可在靠近引脚的最高可用金属层停止。也可仅对已综合结构做详细布线 (`-roots/-leaves`)。移除用 `remove_multisource_global_clock_trees`。

5.7.12 创建时钟 Strap

Creating Clock Straps

`create_clock_straps` 创建单层直线金属形状，可实现：

- **Clock mesh:** 水平与垂直层上的二维网格，交叉点用 via 连接 (图 5.17);
- **Clock spine:** 一维或二维；二维由一维 spine 连接正交方向多条 stripe，不同 spine 的 stripe 不相连，最小间距称 backoff (图 5.18)。

(原书 Figure 53: Clock Mesh Structure)

图 5.17: Clock Mesh 结构

(原书 Figure 54: Clock Spine Structure)

图 5.18: Clock Spine 结构

主要选项: `-net`、`-boundary`、`-keepouts` (`-allow_splitting false` 可禁止跨越 keepout 时分裂)、`-layers`、`-width`、`-grid` (单条距离或 start/end/step 迭代列表)、

-margins、-create_ends、-allow_floating、-type (user_route/stripe/detect) 、spine 的 -length/-backoff/-spine_direction、屏蔽相关 -bias/-bias_to_nets/-bias_margins、-clear。

Mesh 示例:

```
fc_shell> create_clock_straps -nets [get_nets clk1_mesh] \
  -layers {M7 M8} -widths {2.4 2.4} -types {stripe stripe} \
  -grids {{20 1200 100} {60 980 150}} -boundary {{0 0} {1200 980}}
```

5.7.13 布线到时钟 Strap

Routing to Clock Straps

route_clock_straps -nets 将驱动与负载连到 shape_use=stripe 的 strap (不连 user_route)。
-topology:

- fishbone (默认): 各驱动单独连最近 stripe; 多负载用 comb 连到 finger 再连 stripe (图 5.19)。可设 -fishbone_fanout/-fishbone_span/-fishbone_sub_span/-fishbone_layers (图 5.20、5.21)。
- comb: 每引脚直接连最近 stripe; 曼哈顿距离超过 comb distance (默认两全局布线格, 可用 route.common.comb_distance) 时用 Steiner 连最近 net shape (图 5.22)。适合大量引脚直接位于 stripe 下, 但堆叠 via 可能引起物理 DRC (图 5.23)。
- sub_strap: 在中间层创建平行于 stripe 的附加 strap, 减少堆叠 via (图 5.24)。须 -sub_strap_layers; 可选 -sub_strap_max_delay (默认 2 ps)。

(原书 Figure 55: Fishbone Topology)

图 5.19: Fishbone 拓扑

(原书 Figure 56: Fanout of a Finger)

图 5.20: Fishbone finger 的扇出

(原书 Figure 57: Span and Subspan)

图 5.21: Fishbone 的 span 与 subspan

(原书 *Figure 58: Comb Topology*)

图 5.22: Comb 拓扑

(原书 *Figure 59: Vias in the Comb Topology*)

图 5.23: Comb 拓扑中的 Via

5.7.14 分析 Clock Mesh

Analyzing the Clock Mesh

Mesh 需要比传统结构更高的时序精度。用 `analyze_subcircuit` 做晶体管级仿真并回标时序。前提: mesh 已详细布线; 有门级电路模型与晶体管模型; 可访问 NanoSim/FineSim/HSPICE。须指定 `-net` (或 `-to`) 与 `-name`; 多时钟时用 `-clock`。可选在端口/起点标注转换 (时钟须为 `propagated`)。步骤: RC 提取 → 生成 SPICE → 仿真 → 生成含 `set_disable_timing/set_annotated_delay/set_annotated_transition` 的注释文件 (多驱动 mesh 禁用除 anchor driver 外的驱动) → 应用注释。各步可用独立选项单独运行。

5.7.15 自动 Tap 插入与 H-Tree 综合

Performing Automated Tap Insertion and H-Tree Synthesis

1. 用 `set_regular_multisource_clock_tree_options` 指定 `-clock`、`-topology htree_only`、`-tap_boxes`、`-tap_lib_cells`、`-htree_lib_cells` 等 (库单元须已 `set_lib_cell_purpose -include cts`)。可报告/移除对应 options。
2. (可选) 启用柔性 H-tree: `cts.multisource.flexible_htree_synthesis=true`; 对称 tap: `cts.multisource.tap_selection=symmetric`。
3. `synthesize_regular_multisource_clock_trees` (阶段 `tap_synthesis` 与 `htree_synthesis`; 可用 `-from/-to`)。

(原书 *Figure 60: Vias in the Substrap Topology*)

图 5.24: Substrap 拓扑中的 Via

5.7.16 指定 Tap Assignment 选项与设置

Specifying Tap Assignment Options and Settings

运行 `synthesize_multisource_clock_taps` 前：

1. 移除不必要的 `dont_touch/size_only/fixed/locked`，以免阻碍合并/分裂；
2. 用 `set_multisource_clock_tap_options` 指定 `-clock`、`-driver_objects`、`-num_taps`（须等于驱动数）等；
3. （可选）用 `create_multisource_clock_sink_group` 保持 skew group 对象在同一 tap 下；
4. （可选）设置合并/分裂命名相关的 `cts.multisource.subtree_*` 前缀/后缀。

5.7.17 构建本地时钟子树结构

Building the Local Clock Subtree Structures

结构型多源树中，用 `synthesize_multisource_clock_subtrees` 综合由 mesh 直接驱动的本地子树。前置：建好全局结构；用真实 annotated delay/transition 建模 mesh；清除不必要的保护属性；用 `set_multisource_clock_subtree_options` 配置。可选：多角优化、忽略驱动上 `max_cap/max_fanout`（`cts.multisource.ignore_drc_on_subtree_driver`）、fishbone 子树布线（`cts.multisource.subtree_routing_mode`）等。

命令步骤：合并等价单元 → 分裂/改尺寸/搬移并 snap 第一级到 strap → 布线（默认可全局布线；启用 fishbone 则混合详细 fishbone 与全局 comb，见图 5.25）→ 基于布线细化。可用 `-from/-to` 运行 `merge/optimize/route_clock/refine`。由此构建的结构型子树在后续 CTS 命令中不会被更改或删除。

（原书 Figure 61: Detail Fishbone Routes and Global Comb Routes）

图 5.25: 详细 Fishbone 布线与全局 Comb 布线

5.8 分析时钟树结果

Analyzing the Clock Tree Results

综合后分析是否满足要求，通常包括：生成时钟树 QoR 报告；分析时钟时序；在 GUI 中验证时钟实例布局。

5.8.1 生成时钟树 QoR 报告

Generating Clock Tree QoR Reports

用 `report_clock_qor`，可生成：

- **Summary** (默认): latency、skew、DRC、面积、缓冲计数;
- **Latency** (-type latency): 每时钟最长/最短路径;
- **DRC violators** (-type drc_violators);
- **Robustness** (-type robustness, 须 -robustness_corner): 当前 corner 与指定 robustness corner 的 latency 比;
- **Balance groups** (-type balance_groups): 含指定与实际 latency offset;
- **Local skew** (-type local_skew): 每时钟/skew group 汇总及最大/最小局部 skew;
- **Power** (-type power): 按时钟/scenario 的漏电、内部、sink、网络翻转、动态与总功耗。

可用 -csv/-output 输出 CSV。直方图类型: -histogram_type 为 latency/transition/capacitance/local_skew/robustness。可用 -from/-to/-through、-clock、-skew_group、-mode、-corner、-scenario 限定范围 (-scenario 与 -mode/-corner 互斥)。

5.8.2 报告时钟树功耗

Reporting Clock Tree Power

用 report_clock_power。可用 -clocks/-modes/-corners/-scenario 限定; -type per_segment 或 -type per_subtree:

- **Segment**: 从一个 ICG 输出到其扇出中下一 ICG 或 sink 的缓冲树;
- **Subtree**: 从 ICG 一直到其扇出全部 sink 的时钟树 (图 5.26)。

(原书 Figure 62: Segments and Subtrees of a Clock Tree)

图 5.26: 时钟树的 Segment 与 Subtree

5.8.3 创建时钟网络引脚集合

Creating Collections of Clock Network Pins

get_clock_tree_pins 默认返回所有 scenario 全部时钟网上的引脚。可用 -clocks、-scenarios/-modes/-corners、-from/-through/-to、-filter 限定。例如:

```
fc_shell> set clk1_icg_pin \
[get_clock_tree_pins -filter is_on_ICG -clocks clk1]
```

5.8.4 分析时钟时序

Analyzing Clock Timing

用 report_clock_timing, 须指定 -type:

- **skew:** 单时钟局部 skew;
- **interclock_skew:** 时钟间 skew;
- **latency:** 延迟;
- **transition:** 转换时间。

5.8.5 在 GUI 中分析时钟树

Analyzing Clock Trees in the GUI

Fusion Compiler GUI 提供 Clock Tree Visual Mode, 可将时钟树信息叠加到版图视图, 或在原理图视图中查看结构。详见 *Fusion Compiler Graphical User Interface User Guide* 中的 Using Map and Visual Modes。

Chapter 6

布线与布线后优化

Routing and Postroute Optimization

本章介绍 Fusion Compiler 路由器 Zroute 的布线能力，以及布线后优化功能。Zroute 面向多核硬件架构，可高效处理 45 nm 及以下先进工艺设计规则与可制造性设计（DFM）任务。

本章包括以下主题：

- Zroute 简介（Introduction to Zroute）
- 基本 Zroute 流程（Basic Zroute Flow）
- 布线前提条件（Prerequisites for Routing）
- 定义 Via（Defining Vias）
- 插入 Via Ladder（Inserting Via Ladders）
- 检查可布线性（Checking Routability）
- 布线约束（Routing Constraints）
- 布线应用选项（Routing Application Options）
- 时钟网络布线（Routing Clock Nets）
- 关键网络布线（Routing Critical Nets）
- 次级电源/地 pin 布线（Routing Secondary Power and Ground Pins）
- 信号网络布线（Routing Signal Nets）
- 网络屏蔽（Shielding Nets）
- 执行布线后优化（Performing Postroute Optimization）
- 分析并修复信号电迁移违例（Analyzing and Fixing Signal Electromigration Violations）
- 执行 ECO 布线（Performing ECO Routing）
- 在 GUI 中布线（Routing Nets in the GUI）

- 清理已布线网络 (Cleaning Up Routed Nets)
- 分析布线结果 (Analyzing the Routing Results)
- 保存布线信息 (Saving Route Information)
- 推导掩模颜色 (Deriving Mask Colors)
- 插入与移除 Cut Metal 形状 (Inserting and Removing Cut Metal Shapes)

6.1 Zroute 简介

Introduction to Zroute

Zroute 包含五个布线引擎：全局布线 (global routing)、轨道分配 (track assignment)、详细布线 (detail routing)、ECO 布线与布线验证。全局布线、轨道分配与详细布线可通过任务专用命令或自动布线命令调用；ECO 布线与布线验证则使用任务专用命令。

Zroute 主要特性包括：

- 全布线步骤 (含全局布线、轨道分配、详细布线) 支持多核多线程；
- 真实连通模型：矩形相触即视为电气连通，无需导线中心线重合；
- 动态迷宫网格：可 off-grid 连接 pin，同时保留网格路由器的速度优势；
- 多边形管理器：识别多边形，并理解 DRC 面向多边形；
- 详细布线阶段并发优化设计规则、天线规则、导线与 via；
- 详细布线阶段并发插入冗余 via；
- 全局/轨道/详细布线内建 soft rule 支持；
- 时序与串扰驱动的全局布线、轨道分配、详细布线与 ECO 布线；
- 智能设计规则处理 (合并冗余违例、智能收敛)；
- 带层约束与非默认布线规则 (NDR) 的 net group 布线；
- 时钟布线；
- 布线验证；
- 以 soft rule 方式优化 DFM 与可制造性 (DFY)；
- 支持先进设计规则，如多重图案化 (multiple patterning)。

6.2 基本 Zroute 流程

Basic Zroute Flow

图 6.1 给出基本 Zroute 流程，包括时钟布线、信号布线、DFM 优化与布线验证。

(原书 *Figure 63: Basic Zroute Flow*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 6.1: 基本 Zroute 流程

6.3 布线前提条件

Prerequisites for Routing

运行 Zroute 前，block 与物理库须满足：

- 库要求：Zroute 从 `technology file` 获取全部设计规则信息，布线前须确保规则已在 `technology file` 中定义。详见 *Synopsys Technology File and Routing Rules Reference Manual*。
- Block 要求：
 - 电源/地网络已在设计规划后、布局前布线完成（见 *Fusion Compiler Design Planning User Guide*）；
 - 已完成时钟树综合与优化（见第 五 章）；
 - 估计拥塞可接受；
 - 估计时序可接受（约 0 ns slack）；
 - 估计最大电容与 `transition` 无违例。

后三项可通过 `check_routability` 验证布局可布线性（见「检查可布线性」）。

6.4 定义 Via

Defining Vias

路由器支持以下 via 定义类型：

- **Simple via** 与 **simple via array**：simple via 为单 cut via，由 cut 层及 cut/金属层矩形的高宽指定；simple via array 为多 cut 的 simple via。可用于：带 NDR 的时钟/信号布线、冗余 via 插入、带先进 via 规则的 PG 布线。
- **Custom via**：由任意 Manhattan 多边形集合构成的多 cut、异形 via；仅可用于冗余 via 插入。

Via 定义来源：

- 设计库关联 `technology file` 的 `ContactCode` 段；
- LEF 文件中的 `via rule GENERATE` 语句（定义 simple via array）；
- 用户自定义 via（`create_via_def`）。

若希望 Zroute 将某 via 定义用于信号布线，须满足：is_default 为 true，且 is_excluded_for_signal_routing 为 false。此外，Zroute 还会使用 technology file 的 via table 或 create_routing_rule 所定义 NDR 中显式指定的非默认 via。

6.4.1 从 LEF 读取 Via 定义

Reading Via Definitions from a LEF File

使用 read_tech_lef 读取 LEF 中 via rule GENERATE 语句。支持如下 LEF 语法：

```
via_rule_name GENERATE [DEFAULT]
  LAYER lower_layer_name
    ENCLOSURE lower_overhang1 lower_overhang2
  LAYER upper_layer_name
    ENCLOSURE upper_overhang1 upper_overhang2
  LAYER cut_layer_name
    RECT llx lly urx ury
    SPACING x_spacing BY y_spacing
```

不支持 WIDTH 与 RESISTANCE 语句。

6.4.2 创建 Via 定义

Creating a Via Definition

使用 create_via_def 创建 via 定义；创建后可在设计中任意处使用（类似 technology file 的 ContactCode）。用户 via 定义保存在设计库中，仅该设计可用。可定义 simple via、simple via array 与 custom via。

定义 Simple Via

Defining Simple Vias

```
create_via_def
  -cut_layer layer
  -cut_size {width height}
  -upper_enclosure {width height}
  -lower_enclosure {width height}
  [-min_rows number_of_rows]
  [-min_columns number_of_columns]
  [-min_cut_spacing distance]
  [-cut_pattern cut_pattern]
  [-is_default]
  [-force]
  via_def_name
```

无需指定 enclosure 层：工具根据 technology file 中与 via 层相邻的金属层自动确定。默认创建单 cut via；定义 via array 时指定 -min_rows/-min_columns 与 -min_cut_spacing。默认 cut pattern 为满阵列；可用 -cut_pattern 修改。覆盖已有定义须用 -force。

```
fc_shell> create_via_def design_via1_HV \
  -cut_layer VIA12 -cut_size {0.05 0.05} \
  -lower_enclosure {0.02 0.0} -upper_enclosure {0.0 0.2}

fc_shell> create_via_def design_via12 -cut_pattern "01 10"
```

用 report_via_defs 报告用户 via 定义。

定义 Custom Via

Defining Custom Vias

```
create_via_def
  -shapes { {layer {coordinates} [mask_constraint]} ... }
  [-lower_mask_pattern alternating | uniform]
  [-upper_mask_pattern alternating | uniform]
  [-force]
  via_def_name
```

-shapes 须为一个 via 层与两个金属 enclosure 层指定形状；每层可有多个形状。

```
fc_shell> create_via_def design_H_shape_via \
  -shapes { {VIA12 {0.035 -0.035} {0.100 0.035}}
           {VIA12 {-0.100 -0.035} {-0.035 0.035}}
           {METAL1 {-0.130 -0.035} {0.130 0.035}}
           {METAL2 {-0.100 -0.065} {-0.035 0.065}}
           {METAL2 {0.035 -0.065} {0.100 0.065}}
           {METAL2 {-0.100 -0.035} {0.100 0.035}} }
```

双图案化时，可对形状或 enclosure 层分配 mask constraint，二者不可同时使用：

- 对形状：在 -shapes 中用 mask_one/mask_two/mask_three；
- 对 enclosure 层：用 -lower_mask_pattern/-upper_mask_pattern (uniform 或 alternating)，创建 via 时再指定首个形状的 mask。

```
fc_shell> create_via_def VIA12 \
  -shapes { {M1 {-0.037 -0.010} {-0.017 0.01} mask_two}
           {M1 {0.017 -0.010} {0.037 0.01} mask_one}
           {VIA1 {-0.037 -0.01} {0.037 0.01} mask_two}
           {M2 {-0.044 -0.010} {0.044 0.010} mask_one} }
fc_shell> create_via -net n1 -via_def VIA12 \
  -origin {100.100 200.320}
```

6.5 插入 Via Ladder

Inserting Via Ladders

Via ladder（亦称 via pillar）是自 pin 层向上堆叠至上层、供路由器连接的堆叠 via。Via ladder 可降低 via 电阻，改善性能与电迁移鲁棒性。

使用 via ladder 前须：

- 在设计的工艺数据中定义 via ladder 规则（见「定义 Via Ladder 规则」）；
- 在全局布线之前插入 via ladder。

插入方法：

- 布线前优化中的自动插入；
- 基于约束的插入：insert_via_ladders；
- 手动插入：create_via_ladder。

参见

查询 Via Ladder；移除 Via Ladder；控制 Via Ladder 连接。

6.5.1 定义 Via Ladder 规则

Defining Via Ladder Rules

Via ladder 规则定义各级的行数与每行 cut 数，可在 technology file 中定义，或用 create_via_rule 创建。以下三种方式可创建等价规则：

方式 1：在 technology file 的 ViaRule 段中定义（含 cutLayerNameTbl、numCutRowsTbl、numCutsPerRowTbl、upperMetalMinLengthTbl、cutXMinSpacingTbl、cutYMinSpacingTbl、maxNumStaggerTracksTbl、forHighPerformance、forElectromigration 等）。详见 *Synopsys Technology File and Routing Rules Reference Manual*。

方式 2：命令行详细规格 + 属性：

```
fc_shell> create_via_rule -name VL1 -cut_layer_names {VIA1 VIA2} \
    -cut_names {CUT1A CUT2A} -cut_rows {2 2} -cuts_per_row {2 2}
fc_shell> set_attribute [get_via_rules VL1] \
    upper_metal_min_length_table {L1 L2}
fc_shell> set_attribute [get_via_rules VL1] \
    for_high_performance false
fc_shell> set_attribute [get_via_rules VL1] \
    for_electro_migration true
```

方式 3：先创建空规则再设属性（cut_layer_name_table_size、cut_layer_name_table、num_cuts_per_row_table 等）。

无论创建方式如何，工具对 via rule 一视同仁。可用 get_via_rules、remove_via_rules、report_via_rules、list_attributes -application -class via_rule，以及 write_tech_file 保存。

6.5.2 为电迁移 Via Ladder 生成规则

Generating Via Ladder Rules for Electromigration Via Ladders

可用 generate_via_ladder_template -config_file 根据 XML 配置生成规则与约束脚本，输出：

- 模板脚本（默认 `auto_gen_via_ladder_template.tcl`，可用 `-template_file` 改名）：含 `create_via_rule` 与 `set_attribute`；
- 关联脚本（默认 `auto_gen_via_ladder_association.tcl`，可用 `-association_file` 改名）：含 `set_via_ladder_candidate`，并将 `pin` 的 `is_em_via_ladder_required` 设为 `true`。

配置文件基本结构：

```
<EmRule>
  <Template name="rule_name">
    <Layer name="layer_name" row_number="cuts_per_row"
      [max_stagger_tracks="count"] ... />
  </Template>
  <Pin name="pin_name">
    <Template name="rule_name"/>
  </Pin>
</EmRule>
```

工具从 `technology file` 推导 `cut` 层、`cut` 名与各金属层最小长度。

注意

运行 `generate_via_ladder_template` 前须打开 `block`，以确保可访问工艺数据。

```
fc_shell> generate_via_ladder_template -config_file vl_config
```

6.5.3 为性能 Via Ladder 生成规则

Generating Via Ladder Rules for Performance Via Ladders

使用 `setup_performance_via_ladder` 生成性能 via ladder 规则与关联。默认：

- 生成至 M5 的规则（`-max_layer` 可改）；输出 XML（默认 `auto_perf_via_ladder_rule.xml`）与规则脚本（默认 `auto_perf_via_ladder_rule.tcl`）；也可用 `generate_via_rules_for_performance`；
- 将规则关联到无 `dont_use` 的库单元 `pin`（`-dont_use` 可包含 `dont_use` 单元；`-lib_pins` 可限定 `pin`）；输出关联脚本（默认 `auto_perf_via_ladder_association.tcl`）；也可用 `associate_performance_via_ladder`；
- 运行所生成的脚本。

6.5.4 基于约束的 Via Ladder 插入

Constraint-Based Via Ladder Insertion

步骤：

1. 定义 via ladder 规则；
2. 用 `set_via_ladder_rules` / `set_via_ladder_constraints` 定义约束；
3. `insert_via_ladders` 插入；
4. `verify_via_ladders` 验证。

定义 Via Ladder 约束

Defining Via Ladder Constraints

约束指定在哪些 pin 上插入何种模板；可定义全局与 instance 级约束，冲突时 instance 级优先。另可用 `set_via_ladder_candidate` 为特定 pin 指定候选（布线前优化在时序关键路径上插入时使用）。用 `report_via_ladder_constraints` 报告。

全局约束(via ladderrules):`set_via_ladder_rules`。映射选项:`-master_pin_map/-master_pin_map_file` (格式 `{{lib_cell/pin {via_ladder_list}}...}`)、`-default_ladders`。作用范围:`-all_instances_of`、`-all_clock_outputs`、`-all_clock_inputs`、`-all_pins_driving`。报告/移除: `report_via_ladder_rules`、`remove_via_ladder_rules`。

Instance 级约束:

```
fc_shell> set_via_ladder_constraints -pins {u1/i1 u2/i2} \
    {VL1 VL2 VL3}
```

移除用 `remove_via_ladder_constraints`。

插入 Via Ladder

Inserting Via Ladders

`insert_via_ladders` 默认行为与常用选项:

- 默认不删除已有 ladder；用 `-clean true` 先清除；
- 默认针对约束中全部 pin（信号与时钟）；用 `-nets` 限定网络；
- 每个目标 pin 插入一个 ladder；多模板时选用首个不导致 DRC 的模板；全导致 DRC 时默认选 DRC 代价最低者；
- `-ignore_routing_shape_drcs true`：检查时忽略详细布线形状；
- `-relax_line_end_via_enclosure_rule true`：放松线端 cut enclosure；
- `-relax_pin_layer_metal_spacing_rules true`：ladder 完全包在 pin 内时放松 pin 层间距；
- `-allow_drcs false`：禁止产生 DRC 的插入；
- `-strictly_honor_cut_table true`：仅用模板中的 cut；
- `-shift_vias_on_transition_layers true`：过渡层 cut 偏离 track 以改善 DRC；
- `-ndr_on_top_layer_only true`：仅顶层遵循非默认宽度；
- `-connect_within_metal/-connect_within_metal_for_via_ladder`：要求 pin 层上方 via 完全落在 pin（含延伸）内；
- `-allow_patching true`：允许 enclosure 打补丁以满足最小长度；
- `-auto_stagger true`：规则未定义最大 stagger track 时仍允许 stagger；
- `-verbose true/-user_debug true`：报告插入状态与失败详情（内部/硬/软 DRC）。

注意

若 pin 过小无法容纳任一指定 ladder，则不插入。

保护 Via Ladder**Protecting Via Ladders**

```
fc_shell> set_app_options -name design.enable_via_ladder_protection \
-value true
```

为 true 时，remove_shapes、remove_vias、remove_objects、set_attribute、set_via_def、add_to_edit_group 仅处理不属于 via ladder 的对象；属于 ladder 的对象会警告并跳过。

验证与更新**Verifying and Updating Via Ladders**

verify_via_ladders 检查 ladder 是否匹配约束并正确连到 pin；可用 -nets 限定。若插入时用了 -shift_vias_on_transition_layers true，验证时也须使用，否则会误报。-report_all_via_ladders false 可将每类报告限制为 40 条。

更新方式：refresh_via_ladders；或在 route_group/route_auto/route_eco 中由 route.auto_via_ladder.update_during_route（默认 true）自动更新。相关应用选项包括 route.auto_via_ladder.connect_within_metal、route.auto_via_ladder.auto_stagger、route.auto_via_ladder.report_all_via_ladders、route.auto_via_ladder.update_on_route_group_nets_only 等。

手动插入**Manual Via Ladder Insertion**

1. 用 create_shape/create_via 创建组成形状；
2. 用 create_via_ladder -shapes 创建 ladder（名称形如 VIA_LADDER_n）。

可选属性选项：-via_rule、-electromigration、-high_performance、-pattern_must_join、-pin。

查询：get_via_ladders、report_via_ladders。移除：

```
fc_shell> remove_via_ladders *
```

-nets 可限定网络。

6.6 检查可布线性

Checking Routability

布局完成后，用 check_routability 检查 block 是否具备详细布线条件。默认检查：

- 被阻塞的标准单元端口：物理 pin 均不可达则视为 blocked。可达判定：pin 内有伸向邻层的 via、pin 层上存在不少于搜索范围的路径，或较短路径止于邻层 via。默认搜索范围为 2 倍 layer pitch

(`-standard_cell_search_range`, 最大 10)。`-connect_standard_cells_within_pins true` 检查 via 是否完全在 pin 内。`-check_standard_cell_blocked_ports false` 可关闭。

- 被阻塞的顶层/宏端口: 默认搜索距离为 10 倍 pitch (`-blocked_range/-blocked_range_via_side`, 最大 40)。`-obey_direction_preference true` 要求沿优选方向可达。`-check_non_standard_cell_blocked_ports false` 可关闭。
- 越界 **pin** (`-check_out_of_boundary false` 可关);
- 最小网格违例 (`-check_min_grid false` 可关);
- 非法 **via** 定义 (未着色 via array 行列不超过 20; custom/非对称 simple via 不超过 1000 cut; via 定义总数不超过 65535——不可关闭);
- 非法 **real metal via cut blockage** (`-check_via_cut_blockage false` 可关);
- 最小宽度设置 (NDR 最小宽/屏蔽宽不超过 technology 最大宽——不可关闭)。

可选检查 (需显式打开) : `-check_pg_blocked_ports`、`-check_redundant_pg_shapes`、`-check_routing_track_space`、`-check_frozen_net_blocked_ports`、`-check_no_net_pins`、`-check_real_metal_blockage_overlap_pin`、`-check_lib_via_cut_blockage`、`-check_shield`、`-via_ladder`。

Pin 连接控制: `-obey_access_edges`、`-access_edge_whole_side`、`-report_no_access_edge`、`-allow_via_rotation`。

Blocked port 检查会考虑 `set_ignored_layers/set_routing_rule/set_clock_routing_rules` 的软/硬层约束及 `route.common.freeze_layer_by_layer_name` 等; `-honor_layer_constraints false` 可忽略层约束。

错误数据默认名为 `check_routability.err` (`-error_data` 可改), 随 block 保存; 可用 error browser 查看。

6.7 布线约束

Routing Constraints

布线约束在布线过程中提供指导。表 6.1 概述各类约束。

6.7.1 定义 Routing Blockage

Defining Routing Blockages

Routing blockage 定义特定层上禁止布线的区域,Zroute 视其为硬约束。用 `create_routing_blockage` 创建, 至少指定边界与层。

- 矩形: `-boundary { {llx lly} {urx ury} }`;
- 直角多边形: `-boundary { {x1 y1} {x2 y2} ... }`, 也可由 `geo_mask` 等物理对象并集生成 (多连通则生成多个 blockage);
- 层: `-layers` (金属、via 或 poly)。

表 6.1: 布线约束类型

指导内容	说明
控制布线层	<code>set_ignored_layers</code> 等, 见「指定布线资源」
更严格最小宽/间距	定义并应用 NDR (<code>create_routing_rule</code>)
控制穿越 voltage area	电压域布线规则
控制布线方向	层优选方向; 区域用 <code>preferred-direction-only</code> / <code>switch-preferred-direction routing guide</code>
限制非优选方向边数	<code>maximum-number-of-pattern routing guide</code>
控制 off-grid 布线	见「控制 Off-Grid 布线」
禁止特定 net 布线	<code>physical_status</code> 设为 <code>locked</code>
禁止特定区域布线	<code>routing blockage</code>
为顶层布线预留空间	<code>corridor routing blockage</code>
限制布线到特定区域	<code>routing corridor</code>
区域优先级	<code>access preference routing guide</code>
控制布线密度	<code>utilization routing guide</code>
鼓励 river routing	<code>single-layer river routing guide</code>
改善硬宏 pin 可布线性	<code>pin access routing guides</code>
控制 block 边界周围布线	<code>perimeter constraint objects</code>
Metal cut allowed/forbidden 规则	<code>boundary</code> <code>cell</code> 插入时自动创建, 或 <code>derive_metal_cut_routing_guides</code>
控制 pin 连接	<code>must-join</code> 、 <code>pin connection</code> 类型、 <code>pin tapering</code>
控制 via ladder 连接	见相关小节
限制重布线范围	<code>rerouting mode</code>

默认名称 `RB_objId`，阻止边界内所有网络；按 `real metal` 参与 RC 提取与 DRC，须满足最小间距。常用选项：`-name`、`-cell`、`-zero_spacing`（零间距且不作 `real metal`；防止 via 插入时须用此选项）、`-net_types`（`signal/clock/power/ground` 等）。

```
fc_shell> create_routing_blockage \  
-boundary { {0.0 0.0} {100.0 100.0} } \  
-net_types signal -layers M1  
fc_shell> create_routing_blockage \  
-boundary { {0.0 0.0} {100.0 100.0} } \  
-net_types signal -layers V1 -zero_spacing  
fc_shell> create_routing_blockage \  
-boundary { {0.0 0.0} {100.0 100.0} } \  
-net_types {power ground} -layers M2
```

为顶层布线预留空间：创建时加 `-reserve_for_top_level_routing`。块级实现时作普通 `blockage`；`frame view` 提取时移除以允许顶层布线。

查询：`get_routing_blockages`、`get_objects_by_location -classes routing_blockage`。移除：`remove_routing_blockages`（`-all` 移除全部 `corridor`）。

6.7.2 定义 Routing Guide

Defining Routing Guides

`Routing guide` 为区域提供布线指令。用户定义的 `guide` 由 `Zroute` 遵从，但 `Advanced Route` 工具不遵从。

注意

用户定义的 `routing guide` 由 `Zroute` 遵从；`Advanced Route` 工具不遵从。

用 `create_routing_guide` 创建，须指定矩形边界 `-boundary {{llx lly} {urx ury}}` 及用途相关选项。表 6.2 列出用户定义类型。

表 6.2: 用户定义 Routing Guide

用途	选项
仅在优选方向布线	<code>-preferred_direction_only</code>
切换优选方向	<code>-switch_preferred_direction</code>
限制特定布线模式出现次数	<code>-max_patterns</code>
控制布线密度	<code>-horizontal_track_utilization / -vertical_track_utilization</code>
区域优先级	<code>-access_preference</code>
鼓励 <code>river routing</code>	<code>-river_routing</code>

默认名称 `RD#n`；可用 `-name`、`-layers`、`-cell`。

控制布线方向

Using Routing Guides to Control the Routing Direction

```
fc_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
    -layers {M4} -preferred_direction_only
fc_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
    -layers {M1 M2} -switch_preferred_direction
```

注意

若 switch-preferred-direction 与 preferred-direction-only 重叠，前者优先。

限制非优选方向边数

Using Routing Guides to Limit Edges in the Nonpreferred Direction

```
fc_shell> create_routing_guide -boundary {{50 50} {200 200}} \
    -max_patterns {{M2 non_pref_dir_edge 2} {M4 non_pref_dir_edge 3}} \
    -layers {M2 M4}
```

当前支持的 pattern 为 non_pref_dir_edge。超过阈值时 Zroute 报告违例。

控制密度与优先级、River Routing

Density, Access Preference, and River Routing

```
fc_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
    -horizontal_track_utilization 50 -vertical_track_utilization 30
fc_shell> create_routing_guide -boundary {{0 0} {300 300}} \
    -access_preference {M1 {wire_access_preference {{0 0} {2 1}} 0.2}
        {via_access_preference {{0 0} {5 5}} 1.0}
        {via_access_preference {{40 40} {45 45}} 0.5}}
fc_shell> create_routing_guide -boundary {{0.0 0.0} {100.0 100.0}} \
    -river_routing -layers {M2}
```

强度 0–1；强度为 1 时视为硬约束，小于 1 的区域被忽略。用 report_routing_guides 报告。查询/移除：get_routing_guides、remove_routing_guides -all。

6.7.3 推导 Routing Guide

Deriving Routing Guides

Pin Access Routing Guide

Deriving Pin Access Routing Guides

在更小工艺节点复用硬宏时，可用 derive_pin_access_routing_guides 改善宏 pin 可达性：

```
fc_shell> derive_pin_access_routing_guides -cells myMacro \
    -layers {M2 M3} -x_width 0.6 -y_width 0.7
```

为指定宏在指定层创建 routing guide，并在宏周围创建带 pin 开孔的 metal blockage 及相邻 via 层 blockage。默认仅在优选方向开孔；-nonpreferred_direction 可在非优选方向也开孔；-pin_extension 可延伸开孔。命令会将宏 pin 的 is_rectangle_only_rule_waived 设为 true。

(原书 *Figure 64: Pin Access Routing Guides and Blockages*)

图 6.2: Pin Access Routing Guide 与 Blockage

移动宏后须重新生成；不可手动修改。

Metal Cut Routing Guide

Deriving Metal Cut Routing Guides

若 technology file 定义了 metal cut allowed / forbidden preferred grid extension 规则且已有 boundary cell, 可用:

```
derive_metal_cut_routing_guides -add_metal_cut_allowed
```

-check_only 可检查；错误数据默认 block_name.err (-error_view 可改)。

6.7.4 控制 Block 边界周围布线

Controlling Routing Around the Block Boundary

用 derive_perimeter_constraint_objects 沿边界放置金属形状、routing guide 与 blockage。边界变更后须重新生成。

- -perimeter_objects metal_preferred: 沿优选方向边插入浮空金属（连续形状或用 -stub 的金属 stub）；
- -short_metal_length/-metal_spacing: 垂直于非优选边的短金属；
- -perimeter_objects mprg_nonpreferred: 沿非优选边插入最大 pattern guide（阈值为 0）；
- 另可沿边界插入 routing blockage。

常用选项: -hierarchical、-layers、-spacing_from_boundary、-width_nonpreferred、-spacing_nonpreferred、-check_only。

(原书 *Figure 65: Continuous Metal Shape Perimeter Constraint Objects*)

图 6.3: 连续金属形状边界约束对象

(原书 *Figure 66: Metal Stub Perimeter Constraint Objects*)

图 6.4: Metal Stub 边界约束对象

(原书 *Figure 67: Metal Shape Perimeter Constraint Objects With Additional Short Shapes*)

图 6.5: 带短金属的边界约束对象

6.7.5 在特定区域内布线 (Routing Corridor)

Routing Nets Within a Specific Region

Routing corridor 将布线限制在指定区域。全局布线视其为硬约束；轨道分配与详细布线视其为软约束（为修 DRC 可能略微越界）。

用 `create_routing_corridor` 创建, `add_to_routing_corridor` 将 net 加入, `check_routing_corridors` 检查。

6.7.6 使用非默认布线规则 (NDR)

Using Nondefault Routing Rules

用 `create_routing_rule` 定义比 technology file 更严格的宽度、间距、via 间距、via 选择、mask、屏蔽等规则，再用 `set_routing_rule` 或 `set_clock_routing_rules` 赋给网络。

最小线宽与间距

Defining Minimum Wire Width and Spacing Rules

```
create_routing_rule rule_name \
  -widths {layer1 width1 ...} \
  -spacings {layer1 spacing1 ...}
```

(原书 *Figure 68: Routing Guide Perimeter Constraint Objects*)

图 6.6: Routing Guide 边界约束对象

(原书 *Figure 69: Using a Routing Corridor*)

图 6.7: 使用 Routing Corridor

也可定义 soft spacing (带 weight):

```
-soft_spacings { {layer spacing weight} ... }
```

(原书 *Figure 70: Ignoring Nondefault Spacing Rule Violations*)

图 6.8: 忽略非默认间距规则违例

为 soft spacing 分配修复努力:

```
fc_shell> set_app_options \
-name route.common.soft_rule_weight_to_effort_level_map \
-value { {low low} {medium medium} {high high} }
```

effort 取值: off/low/medium/high (low weight 不可用 medium/high effort)。

最小 Via 间距与非默认 Via

Via Spacing and Nondefault Vias

```
fc_shell> create_routing_rule via_spacing_rule \
-via_spacings {{V1 V1 2.3} {V1 V2 3.2}}
```

指定非默认 via: -cuts (由工具按 technology 选择) 或 -vias (显式列出 via 类型、cut 数与方向 R/NR)。

Mask 约束与屏蔽规则

Mask Constraints and Shielding Rules

```
fc_shell> create_routing_rule clock_mask1 \
-mask_constraints {M4 mask_one M5 mask_one}
fc_shell> create_routing_rule shield_rule \
-shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} \
-shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}
```


注意

为确保 DRC 收敛，双图案化 mask 约束应仅用于极少数时序关键网络。

报告: `report_routing_rules -verbose` (-output 可写出 Tcl)。移除: `remove_routing_rules`。
修改须先删后重建。

将 **NDR** 赋给网络

Assigning Nondefault Routing Rules to Nets

- `set_clock_routing_rules`: CTS 前赋给时钟网，CTS 会传播到新建时钟网；
- `set_routing_rule`: 赋给信号网，或 CTS 后的时钟网。

表 6.3 列出限制时钟规则分配的选项。

表 6.3: 限制时钟布线规则分配

赋给	选项
特定时钟树	-clocks
连到 clock root 的网络	-net_type root
连到 clock sink 的网络	-net_type sink
时钟树内部网络	-net_type internal
特定时钟网络	-nets

(原书 *Figure 71: Root, Internal, and Sink Clock Net Types*)

图 6.9: Root / Internal / Sink 时钟网络类型

```
fc_shell> set_clock_routing_rules -rules NDR1 -net_type root
fc_shell> set_clock_routing_rules -rules NDR2 -net_type internal
fc_shell> set_clock_routing_rules -rules NDR3 -net_type sink
fc_shell> set_clock_tree_options -root_ndr_fanout_limit 300
```

(原书 *Figure 72: Using a Fanout Limit for Selecting Root Nets*)

图 6.10: 用扇出限制选择 Root 网络

注意

较小的 `-root_ndr_fanout_limit` 会增加使用 root NDR 的时钟网数量，可能加剧拥塞。

多条 NDR 时优先级: `set_routing_rule > set_clock_routing_rules -nets > set_clock_routing_rules -clocks > 全局 set_clock_routing_rules`。

信号网示例:

```
fc_shell> set_routing_rule -rule WideMetal [get_nets CLK]
fc_shell> set_routing_rule -default_rule [get_nets CLK]
```

另有 `-no_rule`、`-clear`。报告: `report_routing_rules -of_objects`、`report_clock_routing_rules`。

6.7.7 控制 Off-Grid 布线

Controlling Off-Grid Routing

默认仅当 technology 中 `onWireTrack/onGrid` 为 1 时才要求对齐 track。可用以下选项覆盖:

```
fc_shell> set_app_options \
  -name route.common.wire_on_grid_by_layer_name \
  -value {{M2 true} {M3 true}}
fc_shell> set_app_options \
  -name route.common.via_on_grid_by_layer_name \
  -value {{V2 true}}
fc_shell> set_app_options \
  -name route.common.extra_via_off_grid_cost_multiplier_by_layer_name \
  -value {{M2 0.5}}
```

有效代价 = 基础代价 $\times (1 + \text{multiplier})$, multiplier 范围 0.0–20.0。

6.7.8 Must-Join Pin 布线

Routing Must-Join Pins

Must-join pin 的各 terminal 作为同一网络连接。默认随机结构 (图 6.11); 为改善电磁特性, 可用 pattern-based must-join (经 via ladder 能力实现, 图 6.12)。

(原书 Figure 73: Default Must-Join Pin Connection)

图 6.11: 默认 Must-Join 连接

多层结构 (图 6.13): 将 `route.auto_via_ladder.pattern_must_join_over_pin_layer` 设为 2。当库 pin 有 `pattern_must_join` 且运行 `route_auto/route_group/route_eco` 时自动使用。

```
fc_shell> set_app_options -name \
  route.auto_via_ladder.pattern_must_join_max_number_stagger_tracks \
  -value {3 0}
```

(原书 *Figure 74: Single-Level Pattern-Based Must-Join Pin Connection*)

图 6.12: 单层 Pattern-Based Must-Join

(原书 *Figure 75: Multi-Level Pattern-Based Must-Join Pin Connection*)

图 6.13: 多层 Pattern-Based Must-Join

无法创建时报告“Needs pattern must join pin connection”DRC。可用 `route.auto_via_ladder.update_pattern_mu` 控制是否在每次布线时更新。

6.7.9 控制 Pin 连接与 Tapering

Controlling Pin Connections and Pin Tapering

```
fc_shell> set_app_options \
  -name route.common.connect_within_pins_by_layer_name \
  -value {{M1 via_all_pins}}
```

Mode: off (默认)、via_standard_cell_pins、via_wire_standard_cell_pins、via_all_pins、via_wire_all_pins。

Pin tapering: 默认基于距离; 也可用层基 tapering (-taper_over_pin_layers/-taper_under_pin_layers 等)。二者互斥, 同时指定时采用层基设置。相关应用选项: route.detail.pin_taper_mode、route.detail.use_wide_wire_effort_level、route.detail.use_wide_wire_to_input_pin 等。

Via ladder 连接: 默认连到 ladder 最顶层。route.detail.via_ladder_upper_layer_via_connection_mode、route.detail.allow_default_rule_nets_via_ladder_lower_layer_connection 可调整。

(原书 *Figure 76: Pattern-Based Must-Join Pin Connection With Staggering*)

图 6.14: 带 Stagger 的 Pattern-Based Must-Join

(原书 Figure 77–79: *Pin Connection Restrictions*)

图 6.15: Pin 连接限制示意 (无限制 / via / via+wire)

6.7.10 设置重布线模式

Setting the Rerouting Mode

将 net 的 `physical_status` 设为 `locked` (禁止重布)、`minor_change` (仅小改) 或 `unrestricted` (默认)。

```
fc_shell> set_attribute -objects [get_nets net1] \
    -name physical_status -value locked
```

6.7.11 定义与使用 Routing Corridor (详述)

Defining and Using Routing Corridors

Routing corridor 将特定网络的全局布线限制在一组相连矩形所定义的区域内, 并可对每个矩形指定最小/最大布线层。主要用于在信号布线前布关键网。若 routing guide 与 corridor 冲突, corridor 优先。

```
fc_shell> create_routing_corridor -name corridor_a \
    -boundary { {10 10} {20 35} } \
    -min_layer_name M2 -max_layer_name M4
fc_shell> create_routing_corridor_shape -routing_corridor corridor_a \
    -boundary { {20 25} {40 35} } \
    -min_layer_name M2 -max_layer_name M4
fc_shell> add_to_routing_corridor corridor_a [get_nets {n1 n2}]
fc_shell> check_routing_corridors RC_0
```

也可按路径创建: `-path + -width`; 端帽样式 `-start_endcap / -end_endcap (flush/full_width/half_width)`。一个 net/supernet 只能属于一个 corridor, 且 corridor 须覆盖其全部 pin。

修改: `create_routing_corridor_shape`、`remove_routing_corridor_shapes`、`add_to_routing_corridor`、`remove_from_routing_corridor`。报告: `report_routing_corridors (-output 可写出 Tcl)`、`get_routing_corridors -of_objects`。移除: `remove_routing_corridors -all`。

6.7.12 布线层资源与冻结层 (章内相关)

Routing Layer Resources and Freeze Layers

全局/网络/时钟的最小最大布线层分别由 `set_ignored_layers`、`set_routing_rule` 与 `set_clock_routing_rules` 的 `-min_routing_layer / -max_routing_layer` 设置; 软/硬模式由 `route.common.global_min_layer_mode`、`route.common.net_min_layer_mode` 等控制。冻结层由

`route.common.freeze_layer_by_layer_name` 与 `route.common.freeze_via_to_frozen_layer_by_layer_name` 指定。电压域穿越与层优选方向的详细设置见第 2 章相关主题。

6.7.13 天线规则与冗余 Via（章内要点）

Antenna Rules and Redundant Vias

详细布线阶段并发处理天线规则，并在报告中给出天线违例与冗余 via 转换率。用 `check_routes` 的 `-antenna` 可启用/关闭天线检查。冗余 via 可在信号布线期间并发插入；详细布线结束报告含“Redundant via conversion report”（按层与 `weight` 的优化转换率）。更完整的天线修复与冗余 via 专项流程见原书交叉引用主题及第 6 章芯片收尾相关内容。

6.8 布线应用选项

Routing Application Options

工具提供影响全部三个布线引擎的选项，以及分别影响全局布线、轨道分配、详细布线的选项。详见 `man` 页：`route.common_options`、`route.global_options`、`route.track_options`、`route.detail_options`。

执行布线命令时，已设置（或工具代设）的选项会写入布线日志。要显示全部布线选项（含未设置者）：

```
fc_shell> set_app_options \
    -name route.common.verbose_level -value 1
```

6.9 时钟网络布线

Routing Clock Nets

在布线其余网络前先布时钟网，使用 `route_group`：

```
fc_shell> route_group -all_clock_nets
```

也可用 `-nets` 指定部分时钟网。该命令对时钟网依次执行全局布线、轨道分配与详细布线。

- 普通时钟网使用 A-tree 布线以最小化各 pin 到 driver 的距离；
- 时钟 mesh 默认 Steiner；设 `route.common.clock_topology` 为 `comb` 可改用 `comb`；
- 默认忽略已有全局布线；`-reuse_existing_global_route true` 可增量复用；
- 若已有详细布线则做增量详细布线；
- 默认最多 40 次 search-and-repair；`-max_detail_route_iterations` 可改。

注意

若违例已全部修复或判定无法再修，Zroute 会在达到最大迭代前提前停止。

6.10 关键网络布线

Routing Critical Nets

```
fc_shell> route_group -nets collection_of_critical_nets
fc_shell> route_group -from_file file_name
```

若网络关联 routing corridor，则在该区域内布线。全局布线视 corridor 为硬约束；轨道分配与详细布线视之为软约束。

默认忽略已有全局布线，复用 dangling wire 与已有详细布线。-reuse_existing_global_route true 做增量全局布线（与 corridor 不兼容）；-utilize_dangling_wires false 禁用 dangling 复用；-stop_after_global_route true 在全局布线后停止。

默认不修 soft DRC；可设：

```
fc_shell> set_app_options \
-name route.common.post_group_route_fix_soft_violations \
-value true
```

6.11 次级电源与地 Pin 布线

Routing Secondary Power and Ground Pins

步骤：

1. 确认标准单元 frame view 中次级 PG pin 属性正确；
2. 设置布线约束；
3. 用 route_group 布线。

6.11.1 验证属性

Verifying the Secondary Power and Ground Pin Attributes

必需属性：is_secondary_pg=true；port_type=power 或 ground。

```
fc_shell> report_attributes -application \
[get_lib_pins -of_objects reflib/cell/frame -all \
-filter "name == attr_name"]
```

6.11.2 设置约束并布线

Setting Constraints and Routing

可与信号布线类似设置层约束、单连接、route.common.connect_within_pins_by_layer_name、route.common.number_of_secondary_pg_pin_connections（簇内最大 pin 数，0 表示不限）、NDR 等。若要下次级 PG 与 tie-off 分离约束，设 route.common.separate_tie_off_from_secondary_pg 为 true。

Topology ECO 努力由 `route.detail.topology_eco_effort_level` 控制: `off` (默认) / `low` (1 次) / `medium` (4 次) / `high` (8 次)。

```
fc_shell> route_group -nets VDD1
fc_shell> add_buffer {TOP/U1001/Z} {libA/BUFHVT} \
    -new_cell_names mynewHVT
fc_shell> legalize_placement
fc_shell> connect_pg_net -automatic
fc_shell> route_group -nets {VDD2}
```

6.12 信号网络布线

Routing Signal Nets

信号布线前，全部时钟网须已布线且无违例。方法：

- 独立命令: `route_global`、`route_track`、`route_detail` (适合定制流程或大块分步)；每条命令开始读入、结束更新 block，不检查输入数据完整性；
- 自动布线: `route_auto` (依次全局、轨道、详细；适合验证收敛、拥塞与 DRC QoR)。

信号布线可插入冗余 via (见「在信号网上插入冗余 Via」)。

6.12.1 全局布线

Global Routing

布线前设置 `route.common_options` 与 `route.global_options`。默认非时序驱动。全局路由器将 block 划分为 global routing cell (默认宽度等于标准单元高度，与 row 对齐)，按 blockage、pin、track 计算容量与需求，报告 overflow。

先进节点可开 via 密度建模：

```
set_app_options -name route.global.via_cut_modeling -value true
```

两阶段: phase 0 初始布线；随后多轮 rip-up/reroute 减拥塞。Effort: `minimum/low/medium` (默认，最多 3 轮) / `high` (4) / `ultra` (6)。`route.global.force_full_effort` 可强制跑满轮次。

结束后库中保存 g-link/g-via (供后续步骤: 轨道分配与详细布线后移除) 与拥塞数据。默认仅硬拥塞；`route.global.export_soft_congestion_maps=true` 可保存软拥塞。

设计规划阶段探索: `route_global -floorplan true`; 层次化可用 `-virtual_flat all_routing` 或 `top_and_interface_routing_only`。

时序驱动全局布线

Timing-Driven Global Routing

```
set_app_options -name route.global.timing_driven -value true
```

流程: 算延时 → phase0 → 两轮减拥塞 → 更新时序 → 识别新关键网 → 再多轮重布关键网。Effort 与 `route.global.advance_node_timing_driven_effort` (`low/medium/high`) 共同决定重布轮数 (见表 6.4)。`route.global.timing_driven_effort_level` 默认 `high`，偏重时序 QoR。

表 6.4: 时序驱动全局布线重布轮数（示意）

-effort_level	advance=low	medium	high（默认）
low	2	2	2
medium（默认）	3	3	5
high	4	4	6
ultra	6	6	8

串扰驱动与增量全局布线

Crosstalk-Driven and Incremental Global Routing

```
set_app_options -name route.global.crosstalk_driven -value true
set_app_options -name time.si_enable_analysis -value true
# 增量:
route_global -reuse_existing_global_route true
```

6.12.2 轨道分配

Track Assignment

须先完成全局布线。route_track 为各全局布线分配实际 track，并重布重叠导线。完成后不再保留全局布线。

注意

轨道分配用实际金属形状替换全局布线后，block 中不再包含全局布线。

时 序/串 扰 驱 动: route.track.timing_driven、route.track.crosstalk_driven（及 time.si_enable_analysis）。

6.12.3 详细布线

Detail Routing

须先完成全局布线与轨道分配。详细路由器按分区查找 DRC，rip-up/reroute 修复，并并发处理天线规则、优化 via 数与线长。

默认：整块布线（-coordinates 可限区域）；首轮均匀分区、后续按违例分布调整；最多 40 轮（-max_number_iterations）；非时序驱动。相关选项:route.detail.optimize_partition_size_for_drc、route.detail.drc_convergence_effort_level、route.detail.force_max_number_iterations、route.detail.timing_driven、route.common.rc_driven_setup_effort_level、route.detail.repair_shorts_over、route.common.post_detail_route_fix_soft_violations。

增量详细布线须用 -incremental true（否则从 iteration 0 重开）：

```
fc_shell> route_detail -incremental true
```


注意

增量详细布线不修复 open net; 须用 ECO 布线 (route_eco)。

Early mode:set_qor_strategy -mode early -stage pnr。可用 route.detail.save_after_iterations 在指定轮次保存 block。route.detail.report_ignore_drc 可从汇总中排除特定 DRC。

6.12.4 自动布线

Routing Signal Nets by Using Automatic Routing

```
fc_shell> route_auto
```

默认忽略已有全局布线,顺序执行全局、轨道、详细,步骤间不保存。选项:-reuse_existing_global_route、-stop_after_track_assignment、-max_detail_route_iterations、-save_after_global_route/-save_after_track、-save_cell_prefix。

时序驱动:将 route.global.timing_driven、route.track.timing_driven、route.detail.timing_driven 均设为 true。串扰驱动: 另开 route.global.crosstalk_driven、 route.track.crosstalk_driven 与 time.si_enable_analysis。

6.13 网络屏蔽

Shielding Nets

Zroute 按屏蔽规则中的宽度与间距生成屏蔽导线。除同层屏蔽外，还可对上一层/下一层做同轴屏蔽（coaxial shielding，图 6.16），隔离更好但更耗布线资源。

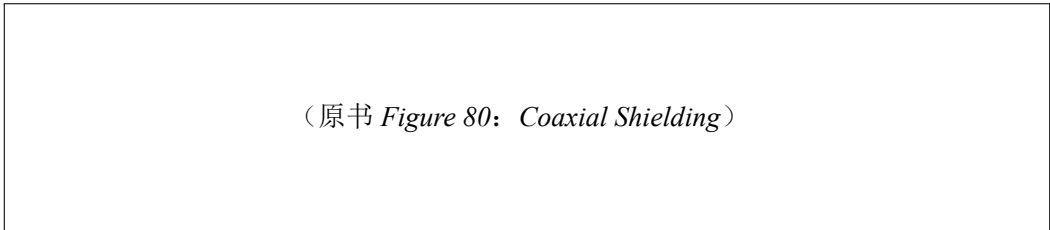


图 6.16: 同轴屏蔽

- 布线前屏蔽 (preroute): 覆盖更好，但可能造成信号布线拥塞；常用于关键时钟；
- 布线后屏蔽 (postroute): 对可布线性影响极小，保护较弱。

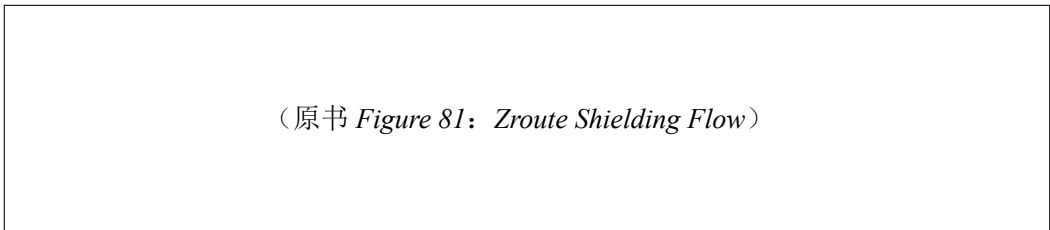


图 6.17: Zroute 屏蔽流程

6.13.1 定义并分配屏蔽规则

Defining and Assigning Shielding Rules

用 `create_routing_rule` 的 `-shield_widths/-shield_spacings` 定义；时钟网用 `set_clock_routing_rules`（仅 CTS 前），信号网用 `set_routing_rule`。建议仅对高频/关键时钟用屏蔽，低频时钟用双倍间距。

6.13.2 布线前屏蔽

Performing Preroute Shielding

时钟树布线后、信号布线前，用 `create_shields` 按规则加屏蔽。默认：对所有有屏蔽规则且未 frozen 的网；忽略短于 4 pitch 的导线（`route.common.min_shield_length_by_layer_name` 可按层改）；屏蔽接到地网（`-with_ground` 或 `route.common.shielding_nets`）；同层屏蔽。

同轴屏蔽：`-coaxial_above/-coaxial_below`，配合 `-coaxial_above_skip_tracks` 等或 `-coaxial_skip_tracks_on_layers` 或用户间距选项（不可混用）。`-align_to_shape_end`、`-preferred_direction_only` 等可进一步控制。

```
fc_shell> create_shields -nets $clock_nets \
    -coaxial_below true -coaxial_below_skip_tracks 0 \
    -with_ground VSS
```

命令报告基于网络与线长的平均屏蔽率。若与 `report_shields` 略有差异，以 `report_shields` 为准（通常 <3%，最多约 5%）。

6.13.3 信号布线中的软屏蔽规则

Soft Shielding Rules During Signal Routing

有屏蔽规则但未预屏蔽的网，Zroute 在全局/轨道/详细全程预留屏蔽空间，详细布线结束报告 soft spacing (shielding) 违例；并不真正插入屏蔽，须事后再跑 `create_shields`。仅预留同层屏蔽空间。若希望有 PG 可用时不预留，设 `route.common.allow_pg_as_shield=true`。

6.13.4 布线后屏蔽与增量屏蔽

Postroute and Incremental Shielding

布线后同样用 `create_shields`。若用 `-nets` 且无 NDR，则用 `technology` 默认宽间距。产生 DRC 时自动做最多 5 轮详细布线；仍有违例可 `route_detail -incremental true`；若破坏 tie-off，改用 `route_eco`。

增量屏蔽：将 `route.common.resield_modified_nets` 设为 `unshield` 或 `resield`（默认 off）。仅支持工具内修改的网络，DEF 外部修改不支持。

查询屏蔽形状：`get_shapes -shield_only`。报告：`report_shields (-per_layer true)`。布线中启用屏蔽检查：`route.common.check_shield=true`。

```
# 示例：时钟预屏蔽 + 关键网后屏蔽
create_routing_rule shield_rule \
    -shield_widths {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3} \
```

```
-shield_spacings {M1 0.1 M2 0.1 M3 0.1 M4 0.1 M5 0.1 M6 0.3}
set_clock_routing_rules -clocks CLK -rules shield_rule
synthesize_clock_trees
route_group -all_clock_nets -reuse_existing_global_route true
create_shields -nets $clock_nets -with_ground VSS
set_routing_rule -rule shield_rule $critical_nets
route_auto
create_shields -nets $critical_nets -with_ground VSS
```

6.14 执行布线后优化

Performing Postroute Optimization

两类优化：

- 逻辑优化：改善时序/面积/功耗 QoR，修逻辑 DRC，并做合法化与 ECO 布线——route_opt；
- 可布线性优化：加大单元间距以修 pin access 引起的布线 DRC——optimize_routability。

若两者都跑，先逻辑后可布线性。

6.14.1 布线后逻辑优化

Performing Postroute Logic Optimization

前置：设计已完全布线并合法化，逻辑/布线 DRC 不过多。用 check_legality、check_routes 验证。推荐流程：

1. compute_clock_latency 更新时钟延时；
2. 运行两次 route_opt（第一次合法化/ECO 可能影响时序，第二次再优化）。

route_opt 步骤：提取并更新时序（默认 PrimeTime 延时引擎，需 O-2018.06 及以后 Library Manager 生成的库；可用 check_consistency_settings）→ 启用的优化 → 合法化 → ECO 布线。

默认优化 data path 的 setup/hold/面积/逻辑 DRC。额外优化见表 6.5。

表 6.5: 启用 route_opt 额外优化的应用选项

优化	应用选项
并发时钟与数据（CCD）及时钟逻辑 DRC	route_opt.flow.enable_ccd=true
时钟逻辑 DRC（未开 CCD 时）	route_opt.flow.enable_cto=true
时钟树面积回收	route_opt.flow.enable_clock_power_recovery=area
时钟树功耗回收	同上 = power
功耗优化	route_opt.flow.enable_power=true
机器学习路径优化	route_opt.flow.enable_ml_opto=true 且 PBA=path

注意

`route_opt` 仅遵从 `route_opt` 应用选项；除 `opt.common.allow_physical_feedthrough` 外不遵从 `opt` 选项。

PBA: `time.pba_optimization_mode=path` (推荐) 或 `exhaustive`。ECO 阶段默认 5 轮详细布线 (`route.detail.eco_max_number_of_iterations`)；`route_opt.eco_route.mode=track` 可改做轨道分配。

签核时序违例可用 `set_route_opt_target_endpoints (-setup_timing/-hold_timing)` 指定目标后跑 `route_opt`，再 `-reset`。Size-only 模式：`route_opt.flow.size_only_mode` 取 `true_footprint/footprint/equal/equal_or_smaller`。

6.14.2 使用 `hyper_route_opt`

Performing Postroute Optimization Using `hyper_route_opt`

可用单次 `hyper_route_opt` 减少迭代次数；设置与 `route_opt` 相同。可选 Tcl 回调 `snps_hyper_route_opt_post_eco` 控制 metal fill、PG augmentation 等。之后仍可用 `set_route_opt_target_endpoints+route_opt` 收尾，或 `route_detail -incremental` 修剩余布线 DRC。

6.14.3 修复 Pin Access 引起的 DRC

Fixing DRC Violations Caused by Pin Access Issues

```
optimize_routability
# 预览:
optimize_routability -check_drc_rules
# 完成后移除 keepout:
optimize_routability -remove_keepouts
```

分析邻接单元违例，在违例侧加 `keepout` (默认 site 宽，`-keepout_width` 可指定；`-flip` 可尝试翻转)。须随后 `route_eco` 或 `-route`；`route.common.eco_fix_drc_in_changed_area_only=on` 可仅在变更区修 DRC。

6.15 分析并修复信号电迁移违例

Analyzing and Fixing Signal Electromigration Violations

信号电迁移由线宽缩小与高速开关导致电流密度升高引起，可造成短路或开路。流程：

1. `read_signal_em_constraints` 加载约束 (ITF 或 ALF)；
2. `read_saif` 或 `set_switching_activity` 标注开关活动；
3. `report_signal_em` 分析；
4. `fix_signal_em` 修复。

```
open_block block1_routed
read_signal_em_constraints em.itf
read_saif block1.saif
report_signal_em -violated -verbose > block1.signal_em.rpt
fix_signal_em
```

ITF: -encrypted 可读加密文件。ALF: -format ALF。约束以查找表存入设计库。

分析计算 average / RMS / peak 电流。相关应用选项见表 6.6（如 em.net_delta_temperature、em.net_global_rms_relaxation_factor、em.net_min_duty_ratio、em.net_use_waveform_duration、em.net_violation_rule_types 等）。

表 6.6: 信号电迁移分析应用选项（节选）

选项	默认	说明
em.net_delta_temperature	5.0	RMS 查找用 ΔT
em.net_duration_condition_for_peak	0.0	peak 检查 duration
em.net_global_rms_relaxation_factor	1.0	RMS 全局松弛因子
em.net_use_waveform_duration	false	是否用波形 duration

fix_signal_em 通过加宽金属的 NDR 并做 ECO 布线修复；每调用一轮，有残留须再跑。可用 -nets 限定。

6.16 执行 ECO 布线

Performing ECO Routing

修改网络后须用 route_eco 重连，顺序执行全局、轨道、详细布线。默认考虑时序与串扰（会提取并更新时序）；若不需要可关闭各引擎的 timing/crosstalk_driven 选项以加速。

默认：处理全部 open net（-nets 可限定）；忽略已有全局布线（-reuse_existing_global_route true 可增量）；复用 dangling wire（-utilize_dangling_wires false 可关）；最多 40 轮详细布线；在整块修硬 DRC。

- -open_net_driven true: 仅在 open net 邻域修 DRC；
- -reroute modified_nets_only 或 modified_nets_first_then_others；
- route.common.post_eco_route_fix_soft_violations=true 修 soft DRC。

报告变更网络时默认前 100 条；-max_reported_nets -1 报告全部。

6.17 在 GUI 中布线

Routing Nets in the GUI

在活动 layout 视图用 Create Route 工具交互布线（Edit 工具栏按钮，或 **Create > Route**）。默认忽略 routing blockage（可在 Mouse Tool Options 中改为遵从）；使用 technology 宽间距而忽略 NDR（同样可在面板中开启遵从 NDR）。

注意

Create Route 工具不遵从 `create_routing_guide` 定义的 routing guide; 也不使用 NDR 中的屏蔽宽/间距。

修改已布线网络的 GUI 工具: Area Push、Spread Wires、Stretch Connected、Quick Connect。面板上的 Help 可打开 man 页。

6.18 清理已布线网络

Cleaning Up Routed Nets

```
fc_shell> remove_redundant_shapes
fc_shell> remove_redundant_shapes -nets my_net \
    -remove_loop_shapes true
```

默认基于设计视图中已存 DRC 信息移除 dangling/floating 形状; `-initial_drc_from_input false` 则先跑 `check_routes`。可用 `-nets/-layers/-route_types` 限定。不改拓扑、不碰 terminal; 不处理 open net 或有 DRC 的网。`-remove_dangling_shapes false`、`-remove_floating_shapes false`、`-remove_loop_shapes true`、`-report_changed_nets true`。清理后应重新做 DRC 验证。

6.19 分析布线结果

Analyzing the Routing Results

6.19.1 拥塞报告与拥塞图

Generating a Congestion Report and Map

```
fc_shell> report_congestion
```

默认用库中拥塞图; 若无则跑 `congestion-map-only` 全局布线。选项: `-rerun_global_router`、`-boundary`、`-layers`、`-mode global_route_cell_edge_based`、`-include_soft_congestion_maps`。

GUI: **View > Map > Global Route Congestion**。图 6.18、图 6.19 示意拥塞图与 GRC 上的 overflow (如 18/9 表示需求 18、供给 9)。

(原书 Figure 82: Global Route Congestion Map)

图 6.18: 全局布线拥塞图

拥塞计算模式: 各层 overflow 之和 (默认, 忽略 underflow), 或总 demand 减总 supply (含 underflow, 结果更乐观)。

(原书 *Figure 83: Global Route Overflow on Global Routing Cell*)

图 6.19: Global Routing Cell 上的 Overflow

6.19.2 用 Zroute 做设计规则检查

Performing Design Rule Checking Using Zroute

```
fc_shell> check_routes
```

默认检查布线 DRC、open net、天线与 voltage area 违例（排除 user/frozen/PG 网）。可用 `-nets`、`-check_from_user_shapes`、`-check_from_frozen_shapes`、`-drc/-open_net/-antenna/-voltage_area` 可分别关闭。`-coordinates` 做区域检查（与 `-nets` 互斥；区域检查不做 open/天线/tie-to-rail）。

错误写入 `zroute.err`。可用 DRC query 命令或 error browser 分析。基于 `check_routes` 结果的增量详细布线：

```
fc_shell> route_detail -incremental true \
    -initial_drc_from_input true
```

6.19.3 签核 DRC、外部工具与 LVS

Signoff DRC, External Tools, and LVS

- 签核: `signoff_check_drc` / `signoff_fix_drc`（需 IC Validator 许可证）；
- 外部 Calibre: `read_drc_error_file` 导入扁平 Calibre 错误文件；
- LVS: `check_lvs` 检查 short/open/floating（`-checks`、`-nets`、`-max_errors`、`-treat_terminal_as_voltage_source`、`-open_reporting_detailed`、`-report_floating_pins` 等）。

(原书 *Figure 84: Layout With Power and Ground Terminals*)

图 6.20: 含电源/地 Terminal 的版图

6.19.4 报告布线结果与线长

Reporting Routing Results and Wire Length

`report_design -routing`: 各层信号/PG 形状数与线长、HV 分布、via 统计与 double via 转换率等（`-hierarchical` 可含整棵物理层次）。

`report_wirelength`: 按层与方向汇总线长（含 global/detail、Virtual、BestAvailable；支持斜线）。

6.19.5 DRC 查询命令

Using the DRC Query Commands

```
fc_shell> get_drc_error_data -all
fc_shell> open_drc_error_data zroute.err
fc_shell> get_drc_error_types -error_data zroute.err
fc_shell> get_drc_errors -error_data zroute.err \
  -filter {type_name == "Diff net spacing"}
```

属性示例: type_name、bbox、objects、shape。完整列表: list_attributes -application -class drc_error。

6.20 保存布线信息

Saving Route Information

```
fc_shell> write_routes
```

生成可复现金属形状与 via（含属性）的 Tcl 脚本；不包含 routing blockage（后者用 write_floorplan）。

6.21 推导掩模颜色

Deriving Mask Colors

布线命令通常会分配 mask 颜色。对未着色形状，用 derive_mask_constraint 从最近底层 track 推导（宽线占多 track 时取与下一空闲 track 相反颜色）。

```
fc_shell> derive_mask_constraint \
  [get_shapes -of_objects [get_nets net1]]
fc_shell> derive_mask_constraint -derive_cut_mask \
  [get_vias -within {{600 600} {700 700}}]
fc_shell> derive_mask_constraint -follow_pin_mask \
  [get_shapes -of_objects [get_nets net1]]
fc_shell> derive_mask_constraint -follow_wider_object_mask \
  [get_shapes -of_objects [get_nets net1]]
```

参见

第一章「多重图案化概念」。

6.22 插入与移除 Cut Metal 形状

Inserting and Removing Cut Metal Shapes

Cut metal 用于双图案化设计以减小线端最小间距。Cut metal 层及尺寸在 technology file 的 Layer 段由 cutMetalWidth、cutMetalHeight/cutMetalExtension 定义；与同 mask 号金属层对应（如 cut-

Metal1 ↔ metal1)。

布线定稿后：

```
fc_shell> create_cut_metals  
fc_shell> remove_cut_metals
```

插入条件：优选方向间距等于 cutMetalWidth、该处尚无 cut metal、金属层有对应 cut metal 层。可插在 net-net、net-PG、PG-obstruction 及子单元内金属之间，并传播颜色。

写出 DEF 须 write_def -version 5.8；写出 GDSII/OASIS 须 write_gds/write_oasis 的 -connect_below_cut_metal。布线变更后须先 remove_cut_metals 再重新插入。

Chapter 7

芯片收尾与可制造性设计

Chip Finishing and Design for Manufacturing

Fusion Compiler 工具提供芯片收尾（chip finishing）以及可制造性设计（DFM）与良率（yield）相关能力，可在设计流程各阶段应用，以应对芯片制造中的工艺设计问题。

有关芯片收尾与可制造性设计功能，请参阅以下主题：

- 插入 Tap Cell（Inserting Tap Cells）
- 执行 Boundary Cell 插入（Performing Boundary Cell Insertion）
- 查找并修复天线违例（Finding and Fixing Antenna Violations）
- 插入冗余 Via（Inserting Redundant Vias）
- 优化线长与 Via 数量（Optimizing Wire Length and Via Count）
- 减小临界面积（Reducing Critical Areas）
- 插入金属-绝缘体-金属电容（Inserting Metal-Insulator-Metal Capacitors）
- 插入 Filler Cell（Inserting Filler Cells）
- 插入 Metal Fill（Inserting Metal Fill）

7.1 插入 Tap Cell

Inserting Tap Cells

Tap cell 是带有 well tie、substrate tie 或二者兼有的特殊非逻辑单元。当库中多数或全部标准单元不含 substrate/well tap 时，通常需要使用此类单元。一般设计规则会规定标准单元内每个晶体管到 well 或 substrate tap 的最大允许距离。

在全局布局之前（floorplanning 阶段），可在 block 中插入 tap cell，形成二维阵列结构，以确保后续放置的标准单元均满足最大 diffusion-to-tap 距离限制。插入后应目视检查，确认所有标准单元可布局区域均已被 tap cell 妥善保护。

Fusion Compiler 提供两种插入 tap cell 阵列的方法：

- 基于 site row 以及 tap cell 之间的最大距离插入 tap cell 阵列
使用 `create_tap_cells` 命令，见「使用 `create_tap_cells` 命令」。
- 使用指定的 offset 与 pitch 插入 tap cell 阵列，并可选择在边界附近额外插入 tap cell
使用 `create_cell_array` 命令，见「使用 `create_cell_array` 命令」。

先进工艺节点常需额外插入 tap cell，以管理 substrate 与 well 噪声。在标准 tap cell 插入之后，还可使用以下能力：

- 插入 tap wall
Tap wall 是一行或一列无间隙线性排列的 tap cell。可在 block、macro 或 hard placement blockage 边界之外或之内插入。边界外的称为 exterior tap wall；边界内的称为 interior tap wall。
 - 插入 exterior tap wall：使用 `create_exterior_tap_walls`，见「插入 Exterior Tap Wall」。
 - 插入 interior tap wall：使用 `create_interior_tap_walls`，见「插入 Interior Tap Wall」。
- 插入 tap mesh
Tap mesh 是二维 tap cell 阵列，每个 mesh window 中有一个 tap cell。使用 `create_tap_meshes`，见「插入 Tap Mesh」。
- 插入 dense tap array
Dense tap array 的 tap 间距小于 `create_tap_cells` 所用间距。使用 `create_dense_tap_cells`，见「插入 Dense Tap Array」。

7.1.1 使用 `create_tap_cells` 命令

Using the `create_tap_cells` Command

要基于 site row 添加 tap cell 阵列，使用 `create_tap_cells`。必须指定用于插入的库单元名（`-lib_cell`）以及 tap cell 之间的最大距离（微米，`-distance`）。

例如：

```
fc_shell> create_tap_cells -lib_cell myreflib/mytapcell -distance 30
```

注意

若应用选项 `place.legalize.enable_pin_color_alignment_check` 为 `true`（默认 `false`），命令在插入时会确保 tap cell 内部 pin 与金属对齐到相应颜色的 routing track，并移动 tap cell 以避免颜色对齐违例。为避免因移动导致 signoff DRC 错误，在该应用选项为 `true` 时应略微减小 tap 距离。

默认情况下，`create_tap_cells` 在整个 block 的每一行插入 tap cell。工具从行左边缘开始，按指定 tap 距离确定后续位置。此外，若在相邻于 block 边界、hard macro 或 hard placement blockage 的每个行边缘的最小 tap 距离（指定距离的一半）内不存在 tap cell，工具会额外插入一个 tap cell。图 7.1 给出采用 every-row 插入模式时的默认布局；注意 hard macro 右侧会额外加入 tap cell，以确保距 hard macro 边缘在最小 tap 距离内存在 tap cell。

可修改以下默认行为方面：

- 镜像（MX）行所用库单元
使用 `-mirrored_row_lib_cell` 指定镜像行所用库单元。

(原书 *Figure 85: Default Tap Cell Placement*)

请参阅原版 PDF 插图；可将截图放入 `figures/` 目录后用 `\includegraphics` 替换。

图 7.1: 默认 Tap Cell 布局

- 插入图案

使用 `-pattern` 指定以下之一：

- `every_row` (默认): 每行都插入。此时 `-distance` 应约为工艺设计规则中最大 `diffusion-to-tap` 值的两倍。
- `every_other_row`: 仅在奇数行插入。`-distance` 应约为最大 `diffusion-to-tap` 值的两倍。
- `stagger`: 每行都插入, 偶数行相对奇数行偏移半个 `-offset`, 形成类似棋盘的图案。`-distance` 应约为最大 `diffusion-to-tap` 值的四倍。

- 相对行左边缘的偏移

使用 `-offset` 将图案起点向右平移指定微米距离。

- 固定单元的处理

默认在计算出的 `tap` 位置插入, 不论该处是否被固定单元占用。使用 `-skip_fixed_cells` 可避免在固定单元位置插入。默认将该固定单元视为 `blockage` 并在该处打断行, 从而可能在固定单元两侧各插入一个 `tap cell`。若要不打断行又避免重叠, 可同时使用 `-preserve_distance_continuity` 与 `-skip_fixed_cells`: 命令会平移 `tap cell` 以避免重叠; `-preserve_distance_continuity` 仅平移那些计算位置被所列库单元实例占用的 `tap cell`。

例如, 下列命令会平移计算位置被 `mylib/cell1` 固定实例占用的 `tap cell`:

```
fc_shell> create_tap_cells -lib_cell mylib/tap -distance 30 \
-skip_fixed_cells -preserve_distance_continuity mylib/cell1
```

使用 `-skip_fixed_cells` 时, 还可用 `-min_horizontal_periphery_spacing` 指定边界与位于 `blockage` 上方或下方的角部 `tap cell` 之间的最小水平间距 (以 `unit tile` 数为单位)。图 7.2 示出有无 `boundary cell` 时该选项对布局的影响。

- 额外 `tap cell` 的添加

可用下列一种或两种方法阻止额外插入：

- `-row_end_tap_bypass`: 声明 `boundary cell` 已含 `tap`, 故边界不再额外插入。注意命令不会验证 `block` 是否确有 `boundary cell`, 或其是否真含 `tap`。
- `-at_distance_only`: 仅允许在指定 `tap` 距离、半距离或四分之一距离 (仅 `stagger`) 处插入。注意使用该选项可能导致 DRC 违例。

- 插入区域

使用 `-voltage_area` 将插入限制在指定 `voltage area`。

(原书 *Figure 86: Tap Cell Boundary Row Spacing*)

请参阅原版 PDF 插图。

图 7.2: Tap Cell 边界行间距

- 命名约定

默认命名: `tapfiller!library_cell_name!number`。

用 `-separator` 可更改默认分隔符 “!”。

用 `-prefix` 可为特定运行加前缀, 命名变为: `tapfiller!prefix!library_cell_name!number`。

7.1.2 使用 `create_cell_array` 命令

Using the `create_cell_array` Command

要按特定布局添加 tap cell 阵列, 使用 `create_cell_array`。必须指定库单元名 (`-lib_cell`) 以及 x 、 y 方向的 pitch。

例如:

```
fc_shell> create_cell_array -lib_cell myreflib/mytapcell \
-x_pitch 10 -y_pitch 10
```

默认在整个 block 的指定网格每个位置插入 tap cell。工具从 core 区域左下角开始, 按 x/y pitch 确定后续位置。插入的 tap cell 会 snap 到 site row, 并以该行默认方向放置。若位置被固定单元、hard macro、soft macro 或 power-switch cell 占用, 则不在该处插入。

可修改以下默认行为:

- 插入区域

`-voltage_area`: 限制到指定 voltage area。

`-boundary`: 限制到指定区域。

若两者同时使用, 仅在重叠区域插入。

- 相对插入区域边界的偏移

`-x_offset`: 起点向右平移 (微米)。

`-y_offset`: 起点向上平移 (微米)。

- 插入图案

`-checkerboard`: 在网格每隔一个位置插入。

– `-checkerboard even`: 左下角放置 tap cell。

– `-checkerboard odd`: 左下角留空。

某些设计方法要求在棋盘图案中位于特定单元 (如 critical area breaker 或 boundary cell) 上方或下方的空位也插入 tap cell。可用 `-preserve_boundary_row_lib_cells` 指定相关库单元以启用

这些额外插入。

- **Snap** 行为

设置 `-snap_to_site_row false` 可禁用 snap 到 site row；此时默认方向为 R0。

- 方向

用 `-orient` 指定：R0、R90、R180、R270、MX、MY、MXR90 或 MYR90。

- 命名约定

默认：headerfooter__library_cell_name_R#_C#_number。

使用 `-prefix` 时：prefix__library_cell_name_R#_C#_number。

7.1.3 插入 Exterior Tap Wall

Inserting Exterior Tap Walls

Exterior tap wall 是放置在 block、macro 或 hard placement blockage 边界之外的一行或一列无间隙 tap cell。

使用 `create_exterior_tap_walls`，必须指定：

- **Tap wall 所用 tap cell** (`-lib_cell`)

默认以 R0 方向插入；可用 `-orientation` 指定其他方向。

若设计有 FinFET grid，选择 tap cell 时请遵循：

- 水平 tap wall：指定方向下的单元宽度须为 FinFET grid *x*-pitch 的整数倍；
- 垂直 tap wall：指定方向下的单元高度须为 FinFET grid *y*-pitch 的整数倍。

默认命名：extTapWall__library_cell_name_R#_C#_number。

使用 `-prefix` 时：prefix__library_cell_name_R#_C#_number。

- 插入侧 (`-side`)

每次只能指定一侧：top、bottom、right 或 left。多侧需多次运行。

- 插入区域 (`-bbox`)

指定包围盒左下与右上角：

```
-bbox {{llx lly}} {urx ury}}
```

包围盒必须覆盖指定侧上对象边缘的一部分。若覆盖该侧多条边，则沿最长边插入；若未覆盖任何边，则报错。

默认情况下，水平 tap wall 从包围盒左边缘开始并贴齐顶/底边界；垂直 tap wall 从包围盒底边缘开始并贴齐左/右边界。可用 `-x_margin` 与 `-y_margin` 指定边距。若含边距后的区域过小，则不插入。应确保插入区域不含 placement blockage、固定单元或 macro，否则可能导致 tap wall 不完整或缺失。

插入时命令不遵循标准单元 row/site，也不跨越对象边界。插入后会固定所插 tap cell 的布局。

7.1.4 插入 Interior Tap Wall

Inserting Interior Tap Walls

Interior tap wall 是放置在 block、macro、hard placement blockage 或非默认 voltage area 的标准单元布局区域边界之内的一行或一列 tap cell。

注意

仅可对具有单一 site 定义的设计插入 interior tap wall。

使用 `create_interior_tap_walls`，必须指定：

- **Tap cell** (`-lib_cell`)

须为与行高、site 宽匹配的单高 (`single-height`) 单元。

默认按行方向插入；可用 `-orientation` 更改。

默认命名：`tapfiller__library_cell_name_R#_C#_number`。

使用 `-prefix` 时：`prefix__library_cell_name_R#_C#_number`。

- 插入侧 (`-side`)

每次仅一侧；多侧需多次运行。

默认沿标准单元可布局区域指定侧的所有边无间隙放置。

- 用 `-bbox` 指定插入区域包围盒。命令沿包围盒所涵盖可布局区域在指定侧的最长边插入。对齐取决于该边段所含角点数量：

- 无角点：水平 wall 从左边、垂直 wall 从底边开始；

- 一个角点：从该角点开始；

- 指定侧上两个角点：水平从左边、垂直从底边开始。对含两角的水平 wall，末两单元间距可能缩短以对齐右角；空间不足时末单元可能不贴齐右角。

- 对水平 tap wall，可用 `-x_spacing` 指定单元间隙。若小于单元宽度则间距为 0；若非 site 宽整数倍则向下取整到 site 宽倍数。

插入时遵循标准单元 row 与 site；未定义则不插入。Placement blockage、固定单元与 macro 等障碍可能导致不完整或缺失的 tap wall。插入后固定布局。

7.1.5 插入 Tap Mesh

Inserting Tap Meshes

Tap mesh 是每个 mesh window 含一个 tap cell 的二维阵列。通常插在 P&R block 之外，也可插在完全位于 core 区域内或完全位于 core 区域外的任意区域。

使用 `create_tap_meshes`，必须指定：

- **Tap cell** (`-lib_cell`)

默认 R0；可用 `-orientation`。

默认命名：`headerfooter__library_cell_name_R#_C#_number`。

使用 `-prefix` 时：`prefix__library_cell_name_R#_C#_number`。

- 插入区域 (`-bbox`): `{{llx lly} {urx ury}}`

- **Mesh window** (`-mesh_window`)

以 *x*、*y* 方向 pitch 的整数值指定窗口大小：`-mesh_window {x_pitch y_pitch}`。

Mesh window 与插入区域左下角对齐。

插入时不遵循标准单元 row/site。在每个 mesh window 左下角放置 tap cell；若被阻挡，则取到该左下角曼哈顿距离最近的可用位置；若无可用位置则警告并跳过该窗口。插入后固定布局。

7.1.6 插入 Dense Tap Array

Inserting Dense Tap Arrays

Dense tap array 的 tap 间距小于 create_tap_cells 所用间距。

使用 create_dense_tap_cells，必须指定：

- **Tap cell** (-lib_cell)
须与运行 create_tap_cells 时相同。
默认命名：tapfiller!library_cell_name!number；可用 -separator、-prefix。
- **Tap 距离** (-distance)
必须小于 create_tap_cells 所用距离。
- **插入区域** (-bbox)

命令首先移除完全位于插入区域内的既有 tap cell，再按指定距离重新插入以形成更密阵列。为避免 tap 规则违例，应使用与 create_tap_cells 相同的插入图案。支持 -offset、-pattern、-skip_fixed_cells、-at_distance_only（详见 man 页）。插入后固定布局。

7.2 执行 Boundary Cell 插入

Performing Boundary Cell Insertion

在放置标准单元之前，可为 block 添加 boundary cell。Boundary cell 包括：加在单元行两端以及 core 区域、hard macro、blockage、voltage area 等对象边界周围的 end-cap cell；以及填充水平与垂直 end-cap 之间空隙的 corner cell。End-cap cell 通常为非逻辑单元（如电源轨去耦电容）。因工具接受任意标准单元作为 end-cap，须确保指定合适的单元。

插入 boundary cell 的步骤：

1. 用 set_boundary_cell_rules 指定插入要求（见「指定 Boundary Cell 插入要求」）。
2. 用 compile_boundary_cells 或 compile_targeted_boundary_cells 按规则插入（见「插入 Boundary Cell」）。
3. 用 check_boundary_cells 或 check_targeted_boundary_cells 验证布局（见「验证 Boundary Cell 布局」）。

每次运行仅支持单一 site 定义。compile_boundary_cells 与 compile_targeted_boundary_cells 根据 set_boundary_cell_rules 所指定库单元确定 site，并仅在使用该 site 定义的区域插入。若设计有多种 site 定义，须对每种各运行一次该流程。

7.2.1 指定 Boundary Cell 插入要求

Specifying the Boundary Cell Insertion Requirements

使用 set_boundary_cell_rules 指定：

- 用于 boundary cell 的库单元（见「指定 Boundary Cell 插入所用库单元」）；
- 放置规则（见「指定 Boundary Cell 放置规则」）；
- 插入单元的命名约定（见「指定 Boundary Cell 命名约定」）；
- 创建 routing guide 以遵守 metal cut allowed/forbidden preferred grid extension 规则（见「Boundary Cell 插入期间创建 Routing Guide」）；

注意

该功能仅由 `compile_boundary_cells` 支持，`compile_targeted_boundary_cells` 不支持。

- 创建 placement blockage 以遵守 minimum jog / minimum separation 规则（见「Boundary Cell 插入期间创建 Placement Blockage」）。

注意

该功能仅由 `compile_boundary_cells` 支持，`compile_targeted_boundary_cells` 不支持。

注意

`set_boundary_cell_rules` 指定的设置保存在设计库中。

参见

报告 Boundary Cell 插入要求：见「报告 Boundary Cell 插入要求」。

指定 **Boundary Cell** 插入所用库单元

Specifying the Library Cells for Boundary Cell Insertion

Boundary cell 包括左右上下边界的 end-cap，以及内/外角单元。可用 `set_boundary_cell_rules` 的下列选项为各类型指定不同库单元：

- `-left_boundary_cell`、`-right_boundary_cell`：分别指定左、右边界 end-cap 所用的单个库单元。
- `-top_boundary_cells`、`-bottom_boundary_cells`：分别指定顶、底边界 end-cap 的库单元列表。按指定顺序插入；若剩余空间小于当前单元，则插入列表中下一个能放入剩余空间的单元。
对垂直行 block，行自底向顶，故顶边界沿 block 左侧、底边界沿右侧。
对 double-back block 中的翻转行，顶边界单元用于底边界，底边界单元用于顶边界。
- `-top_tap_cell`、`-bottom_tap_cell`：分别指定顶、底边界行上的 tap cell，以确保端 cap 满足最大 diffusion-to-tap 限制。
- `-top_left_outside_corner_cell`、`-top_right_outside_corner_cell`、`-bottom_left_outside_corner_cell`、`-bottom_right_outside_corner_cell`：各外角位置的单个库单元。
- `-top_left_inside_corner_cells`、`-top_right_inside_corner_cells`、`-bottom_left_inside_corner_cells`、`-bottom_right_inside_corner_cells`：各内角位置的库单元列表。工具插入第一个与内角尺寸匹配的角单元；若无一精确匹配，则插入第一个可不违规则放置的单元。

图 7.3 示出含两个 hard macro/blockage 的水平行 block 上 boundary 与 corner 单元位置；行翻转时位置也会翻转。

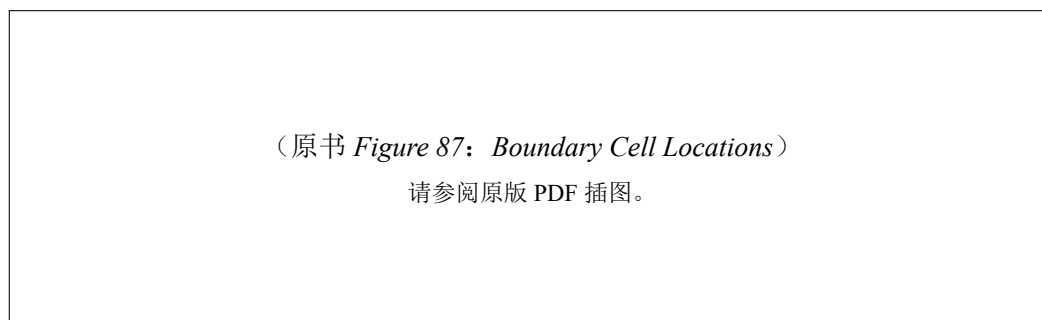


图 7.3: Boundary Cell 位置

指定 **Boundary Cell** 放置规则

Specifying Boundary Cell Placement Rules

默认：

- 运行 `compile_boundary_cells` 时，工具以默认方向在 core 区域、hard macro 与 hard placement blockage 周围放置 boundary cell；
- 运行 `compile_targeted_boundary_cells` 时，工具以默认方向在指定对象周围放置。

要翻转边界单元方向，可在 `set_boundary_cell_rules` 中使用一个或多个选项：

`-mirror_left_boundary_cell`、`-mirror_right_boundary_cell`、`-mirror_left_outside_corner_cell`、`-mirror_right_outside_corner_cell`、`-mirror_left_inside_corner_cell`、`-mirror_right_inside_corner_cell`、`-mirror_left_inside_horizontal_abutment_cell`、`-mirror_right_inside_horizontal_abutment_cell`。

不能翻转顶/底边界单元方向。

使用 `compile_boundary_cells` 时，还可修改：

- 翻转行上顶/底内角单元互换
`-do_not_swap_top_and_bottom_inside_corner_cell`：阻止在翻转行顶内角使用底内角单元、在底内角使用顶内角单元。
- 考虑 **voltage area**
`-at_va_boundary`：在 voltage area 边界两侧插入水平 boundary cell。
- 一单位 **tile** 间隙
`-no_1x`：防止插入产生一单位 tile 间隙。当行长等于两倍角单元宽加一单位 tile 宽时不插入；若等于两倍角单元宽则仍插入。
- **Tap cell** 间距
`-tap_distance`：顶/底边界行上 tap cell 间距（微米）。
- 短行上的插入
`-min_row_width`：行宽小于该阈值时不插入。
- 子 **block** 中的插入
`-insert_into_blocks`：递归在子 block 中插入（默认不插入）。

指定 **Boundary Cell** 命名约定

Specifying the Naming Convention for Boundary Cells

默认命名: `boundarycell!library_cell_name!number`。

用 `-separator` 更改分隔符;用 `-prefix` 加前缀后为:`boundarycell!prefix!library_cell_name!number`。

7.2.2 Boundary Cell 插入期间创建 Routing Guide

Creating Routing Guides During Boundary Cell Insertion

若 `technology file` 定义了 `metal cut allowed` 与 `forbidden preferred grid extension` 规则, 可用 `-add_metal_cut_allowed` 在插入期间创建相应 `routing guide`。此时 `compile_boundary_cells` 会创建:

- **Metal cut allowed routing guide:** 覆盖全部可布局 `site row` 所占区域, 再按垂直收缩因子缩小 (为最小 `site row` 高度的 50%);
- **Forbidden preferred grid extension routing guide:** 覆盖直至 `block` 边界的剩余区域。

注意

仅 `compile_boundary_cells` 支持; `compile_targeted_boundary_cells` 不支持。

7.2.3 Boundary Cell 插入期间创建 Placement Blockage

Creating Placement Blockages During Boundary Cell Insertion

若工艺有 `minimum jog` 或 `minimum separation` 要求, 可在 `set_boundary_cell_rules` 中配合下列选项, 由 `compile_boundary_cells` 创建 `placement blockage`:

- `-min_horizontal_jog`: 若 `macro` (含 `hard keepout`)、`hard placement blockage` 或 `voltage area` (含 `guard band`) 的水平边小于指定值, 则用 `compile_boundary_cells -add_placement_blockage` 创建 `blockage` 以防违例。
- `-min_vertical_jog`: 对垂直边同理。
- `-min_horizontal_separation`: 若连续水平布局区域小于指定值, 则创建 `blockage`。
- `-min_vertical_separation`: 对垂直方向同理。

注意

默认 `compile_boundary_cells` 会检查指定规则但不创建 `blockage`; 须使用 `-add_placement_blockage` 才会创建。该功能仅 `compile_boundary_cells` 支持。

7.2.4 报告 Boundary Cell 插入要求

Reporting the Boundary Cell Insertion Requirements

使用 `report_boundary_cell_rules` 报告由 `set_boundary_cell_rules` 指定的要求。仅报告用户指定设置, 不报告默认值。

7.2.5 移除 **Boundary Cell** 插入要求

Removing Boundary Cell Insertion Requirements

使用 `remove_boundary_cell_rules` 移除一项或多项由 `set_boundary_cell_rules` 指定的要求。

7.2.6 插入 **Boundary Cell**

Inserting Boundary Cells

按插入目标选择命令：

- 在 core 区域、hard macro 与 placement blockage 周围插入：`compile_boundary_cells`；
- 仅在一个或多个 voltage area 周围：`compile_boundary_cells -voltage_area`；
- 仅在特定对象周围：`compile_targeted_boundary_cells -target_objects`（可针对特定 macro、voltage area、voltage area shape、placement/routing blockage 与 core 区域）。

`set_boundary_cell_rules` 同时配置两种 `compile` 命令；部分规则仅被 `compile_boundary_cells` 遵守，详见「指定 **Boundary Cell** 插入要求」。

7.2.7 验证 **Boundary Cell** 布局

Verifying the Boundary Cell Placement

用 `compile_boundary_cells` 插入后，用 `check_boundary_cells` 验证。用 `-error_view` 可生成供 error browser 查看的错误数据文件。该命令检查：

- 缺失的 **boundary** 或 **corner cell**

验证整个边界周围有 **boundary/corner cell** 且无间隙。图 7.4 示出有效布局及因缺失单元导致的错误。

注意

对成熟节点，**boundary cell** 可能仅插在左、右侧；此时命令只验证这两侧无间隙。

- 不正确的 **boundary/corner cell**

验证所用库单元与 `set_boundary_cell_rules` 指定一致。

- 不正确的方向

验证各 **boundary cell** 方向符合允许方向。

- 短行与短边

验证每行 **boundary cell** 宽于 `-min_row_width`，且各 **blockage** 水平边大于 `-min_horizontal_jog`。

(原书 *Figure 88: Check for Missing Boundary or Corner Cells*)

请参阅原版 PDF 插图。

图 7.4: 检查缺失的 Boundary/Corner Cell

7.3 查找并修复天线违例

Finding and Fixing Antenna Violations

芯片制造中，栅氧易被静电放电损伤。多层金属化过程中导线累积的静电荷可能损坏器件或导致芯片完全失效。静电电荷泄放入器件的现象称为天线（antenna）或电荷收集天线问题。

为防止天线问题，工具验证每个输入 pin 的金属天线面积除以栅极面积小于晶圆厂给出的最大天线比：

$$\frac{\text{antenna-area}}{\text{gate-area}} < \text{max-antenna-ratio}$$

该检查基于 block 的全局与层相关天线规则，以及单元的天线属性。

天线流程步骤：

1. 定义天线规则；
2. 指定 pin 与 port 的天线属性；
3. 分析并修复天线违例。

7.3.1 定义天线规则

Defining Antenna Rules

天线规则定义如何计算 block 中网络的最大天线比，以及 pin 的天线比。必须用 `define_antenna_rule` 定义全局天线规则；也可用 `define_antenna_layer_rule` 指定层相关规则。无层规则时使用全局规则。

相关主题：

- 计算最大天线比
- 计算 Pin 的天线比

计算最大天线比

Calculating the Maximum Antenna Ratio

须指定：

- 二极管提供多少保护（diode protection mode）——`define_antenna_rule` 的 `-diode_mode`（必选），见「设置 Diode Protection Mode」；

- 在二极管保护下如何计算天线比（diode ratio vector）——define_antenna_layer_rule 的 -diode_ratio（必选），见「指定 Diode Ratio Vector」。

设置 Diode Protection Mode

Setting the Diode Protection Mode

Diode protection mode 规定二极管提供的保护程度。先进工艺设计常使用不同 gate oxide class 表示多种栅氧厚度。工具最多支持四种：gate class 0、1、2、3。无 gate class 数据的 pin 视为 class 0。对使用多种栅氧厚度的设计，须使用支持多厚度的模式：14、16、18。

用 define_antenna_rule -diode_mode 指定（必选）。表 7.1 定义各模式如何影响网络最大天线比；各二极管最大天线比的计算公式见表 7.3，其中使用 -diode_ratio 向量值。

表 7.1: Diode Protection Mode 设置（原书 Table 28）

模式	定义
0	二极管不提供任何保护。
1	二极管提供无限保护。此模式下最高金属层不检查天线违例，因为最高层形成时输入–输出连接已完成。
2	保护有限；多二极管时取各二极管最大天线比的最大值。
3	保护有限；多二极管时取各二极管最大天线比之和。
4	保护有限；多二极管时用各二极管 diode-protection 值之和计算最大天线比。
5	保护有限；用所有二极管的最大 diode-protection 值计算等效栅极面积。
6	保护有限；用所有二极管 diode-protection 值之和计算等效栅极面积。
7	保护有限；用最大 diode-protection 值计算等效金属面积。
8	保护有限；用 diode-protection 值之和计算等效金属面积。
9	保护有限；按最大二极管保护做缩放。
10	保护有限；按总二极管保护做缩放。
11	保护有限；按最大二极管保护做缩放。
12	保护有限；按总二极管保护做缩放。
13	已废弃。
14	保护有限；两套独立计算公式。支持 gate class。
15	仅内部使用。
16	保护有限；多二极管时用 diode-protection 值之和计算最大天线比。支持 gate class。
17	保护有限；用 diode-protection 值之和计算等效栅极面积；输入与输出 pin 分别计算。
18	保护有限；多二极管时用 diode-protection 值之和计算最大天线比。支持 gate class，且计算取决于 pin 是否连二极管。

指定 Diode Ratio Vector

Specifying the Diode Ratio Vector

用 `define_antenna_layer_rule -diode_ratio` 指定用于计算最大天线比的值。未指定时工具使用 `{0 0 1 0 0}`。实际用法取决于 `define_antenna_rule -diode_mode`。

表 7.2 给出各模式的向量格式；表 7.3 说明如何用向量值计算。其中：

- `dp`: 输出 pin 的 diode protection 值（见「指定天线属性」）；
- `layerMaxRatio`: 该层最大天线比，由 `define_antenna_layer_rule -ratio` 或全局规则的 `-metal_ratio/-cut_ratio` 指定。

示例见后文「模式 2/3/4」「模式 5/6」「模式 7/8」「模式 14 多栅氧厚度」各小节。

表 7.2: Diode Ratio Vector 格式（原书 Table 29）

模式	格式
2–10, 17	<code>{v0 v1 v2 v3 [v4]}</code> ；v4 为可选保护上限，省略视为 0（无上限）。
11, 12	<code>{{v0 v1 v2 v3 [v4]},{s0 s1 s2 s3 s4 s5}}</code> ；第二向量为缩放值。
14	每个 gate class 一个 10 值向量（共 4 个）。
16	每个 gate class 一个 8 值向量（共 4 个）。
18	每个 gate class 一个 12 值向量（共 4 个）。

模式 2、3、4 示例

Example for Diode Modes 2, 3, and 4

假设天线面积按单层表面面积（antenna mode 1）计算，M1 的 diode ratio vector 为 `{0.7 0.0 200 2000}`，`layerMaxRatio=400`，同一 M1 网连接二极管 A（保护值 0.5）、B（1.0）、C（1.5）；未指定 v4 时按 0。

```
fc_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \  
-metal_ratio 400 -cut_ratio 20  
fc_shell> define_antenna_layer_rule -mode 1 -layer "M1" \  
-ratio 400 -diode_ratio {0.7 0.0 200 2000}
```

各二极管最大天线比按表 7.3 与表 7.4 计算。

网络最大天线比：

- 模式 2：取各二极管最大值 = 2300；
- 模式 3：求和 = 400 + 2200 + 2300 = 4900；
- 模式 4：用保护值之和代入公式， $(0.5 + 1.0 + 1.5) * 200 + 2000 = 2600$ 。

模式 5、6 示例

表 7.3: 基于 Diode Mode 的天线比计算 (原书 Table 30)

模式	计算
0, 1	不使用
2, 3, 4	若 $dp > v0$ 且 $v4 \neq 0$: $antenna_ratio = \min(((dp+v1)*v2+v3), v4)$; 若 $dp > v0$ 且 $v4 = 0$: $(dp+v1)*v2+v3$; 若 $dp \leq v0$: $layerMaxRatio$
5	$antenna_ratio = metal_area / (gate_area + equi_gate_area)$; 若 $max_diode_protection > v0$ 且 $v4 \neq 0$: $equi_gate_area = \min(((max_diode_protection+v1)*v2+v3), v4)$; 若 $v4 = 0$: $(max_diode_protection+v1)*v2+v3$; 若 $max_diode_protection \leq v0$: $equi_gate_area = 0$
6	同 5, 但用 $total_diode_protection$
7	$antenna_ratio = (metal_area - equi_metal_area) / gate_area$; $equi_metal_area$ 由 $max_diode_protection$ 按与模式 5 相同的 $v0-v4$ 规则计算; 若 $equi_metal_area > metal_area$ 则取 $metal_area$
8	同 7, 但用 $total_diode_protection$
9	$antenna_ratio = scale * metal_area / gate_area$; 若 $max_diode_protection > v0$: $scale = \max(1 / ((max_diode_protection+v1)*v2+v3), v4)$; 否则 $scale = 1.0$
10	同 9, 但用 $total_diode_protection$
11	$antenna_ratio = scale * metal_area / gate_area$; 按 $max_diode_protection$ 相对 $v0-v4$ 的区间选取 $s0-s5$
12	同 11, 但用 $total_diode_protection$
13	N/A
14	连二极管: 两式同时成立 $(metal/via)/gate \leq (dp*v1+v2)$ 且 $(metal/via)/(v5*gate+v6*dp) \leq v7$; 不连二极管: $(metal/via)/gate \leq v3$ 且 $(metal/via)/(v8*gate) \leq v9$
15	N/A (内部使用)
16	允许最大天线比: 同模式 2/3/4 的 $dp/v0-v4$ 规则; 若 $gate-area \leq v5$: $antenna-ratio = antenna-area / (gate-area + v6*dp)$; 否则 $antenna-area / (gate-area + v6*dp + v7)$
17	同模式 6 (等效栅极面积), 输入/输出 pin 分别计算
18	连二极管: 同模式 16; 不连二极管: 按 $gate-area$ 与 $v8-v11$ 取允许最大比, $antenna-ratio = antenna-area / gate-area$

表 7.4: 各二极管最大天线比计算 (原书 Table 31)

二极管	保护值	最大天线比
A	0.5	400 (保护值 $< v0=0.7$, 用 $layerMaxRatio$)
B	1.0	2200 $((1.0 + 0.0) * 200 + 2000)$
C	1.5	2300 $((1.5 + 0.0) * 200 + 2000)$

Example for Diode Modes 5 and 6

假设天线面积按单层模式表面面积（antenna mode 1）计算，M1 的 diode ratio vector 为 {0 0 1 0}（默认），M1 的 layerMaxRatio 为 400，同一 M1 网络连接二极管 A（保护值 0.5）、B（1.0）、C（1.5）；未指定 v4 时按 0 处理。

```
fc_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \
-metal_ratio 400 -cut_ratio 20
```

因使用全局 layerMaxRatio 与默认向量，无需层相关规则。模式 5/6 的最大天线比为 $\text{metal_area} / (\text{gate_area} + \text{equi_gate_area})$ ，其中 $\text{equi_gate_area} = (\text{diode_protection} + v1) * (v2 + v3)$ 。

- 模式 5：用最大保护值 1.5，得 $\text{metal_area} / (\text{gate_area} + 1.5)$ ；
- 模式 6：用总保护值 $0.5 + 1.0 + 1.5 = 3.0$ ，得 $\text{metal_area} / (\text{gate_area} + 3.0)$ 。

模式 7、8 示例

Example for Diode Modes 7 and 8

假设单层表面面积模式，M1 向量为 {0.7 0.0 150 800}，layerMaxRatio=400，二极管保护值同前。

```
fc_shell> define_antenna_rule -mode 1 -diode_mode diode_mode \
-metal_ratio 400 -cut_ratio 20
fc_shell> define_antenna_layer_rule -mode 1 -layer "M1" \
-ratio 400 -diode_ratio {0.7 0.0 150 800}
```

模式 7/8 计算为 $(\text{metal_area} - \text{equi_metal_area}) / \text{gate_area}$ ：

- 模式 7：最大保护 1.5，得 $(\text{metal_area} - 1025) / \text{gate_area}$ ；
- 模式 8：总保护 3.0，得 $(\text{metal_area} - 1250) / \text{gate_area}$ 。

模式 14（多栅氧厚度）示例

Example for Diode Mode 14 With Multiple Gate Oxide Thicknesses

假设单层表面面积模式，设计含两种栅氧厚度。gate class 0 的 M1 向量为 {1 0 1e9 1e9 0 1 2 285 1 285}，gate class 1 为 {1 0 1e9 1e9 1 0 2 165 1 28}，layerMaxRatio=285。模式 14 每个 gate class 需 10 个值；即使仅用两类，定义时仍须给出全部 40 个值。未用 class 填很大比率以禁用检查。

```
fc_shell> define_antenna_rule -mode 1 -diode_mode 14 \
-metal_ratio 285 -cut_ratio 10
fc_shell> define_antenna_layer_rule -mode 1 -layer "M1" \
-ratio 285 -diode_ratio {1 0 1e9 1e9 0 1 2 285 1 285 \
                        1 0 1e9 1e9 1 0 2 165 1 28 \
                        1 0 1e9 1e9 0 1 2 1e9 1 1e9 \
                        1 0 1e9 1e9 0 1 2 1e9 1 1e9 }
```

向量各分量含义：

- V0: 公式 1 使用的栅极面积范围 (0= 当前 class; 1= 全部 class 总和);
- V1: 连二极管时公式 1 的 diode protection 乘数;
- V2: 连二极管时公式 1 的偏移;
- V3: 不连二极管时公式 1 的比率要求;
- V4: 公式 2 的栅极面积范围 (同 V0);
- V5-V7: 连二极管时公式 2 的栅极乘数、保护乘数与比率要求;
- V8-V9: 不连二极管时公式 2 的栅极乘数与天线比要求。

模式 14 公式(表 7.3 续):若连二极管,须同时满足 $(\text{metal}/\text{via area})/\text{gate-area} \leq (\text{dp} \cdot \text{v1} + \text{v2})$ 与 $(\text{metal}/\text{via area})/(\text{v5} \cdot \text{gate-area} + \text{v6} \cdot \text{dp}) \leq \text{v7}$; 若不连二极管,须同时满足 $(\text{metal}/\text{via area})/\text{gate-area} \leq \text{v3}$ 与 $(\text{metal}/\text{via area})/(\text{v8} \cdot \text{gate-area}) \leq \text{v9}$ 。

计算 Pin 的天线比

Calculating the Antenna Ratio for a Pin

工具支持多种天线比 (antenna-area/gate-area) 计算方式。通过指定下列信息控制:

- 如何计算天线面积 (antenna area mode);
- 计算时考虑哪些金属段 (antenna recognition mode)。

用 `define_antenna_rule` 与 `define_antenna_layer_rule` 的 `-mode` 指定 (二者的必选选项), 见「设置天线模式」。

7.3.2 设置天线模式

Setting the Antenna Mode

天线模式通过以下两方面控制天线比计算:

- 天线面积计算方式 (antenna area mode)
 - 表面面积: $W \times L$;
 - 侧壁面积: $(W + L) \times 2 \times \text{thickness}$ 。

注意

使用侧壁面积时, 须在 `technology file` 各 `Layer` 段中用 `unitMinThickness`、`unitNomThickness`、`unitMaxThickness` 定义金属厚度。

- 天线识别模式 (antenna recognition mode)
 - **Single-layer mode:** 仅考虑当前层金属段, 忽略下层。可布线性最好。
 $\text{antenna_ratio} = \text{该层相连金属面积} / \text{总栅极面积}$
 - **Accumulated-ratio mode:** 考虑当前层及与输入 pin 相连的下层段。
 $\text{antenna_ratio} = \text{当前层与下层各 single-layer 比率之和}$

- **Accumulated-area mode:** 考虑当前层与下层金属面积。
 $\text{antenna_ratio} = \text{当前及下层相连金属面积} / \text{总栅极面积}$

用 -mode 指定天线模式（必选）。表 7.5 列出取值。

表 7.5: 天线模式设置（原书 Table 32）

天线模式值	面积计算方式	天线识别模式
1	Surface	Single-layer
2	Surface	Accumulated-ratio
3	Surface	Accumulated-area
4	Sidewall	Single-layer
5	Sidewall	Accumulated-ratio
6	Sidewall	Accumulated-area

天线识别模式示例

Antenna Recognition Mode Example

图 7.5 给出含侧视的布局示例；表 7.6 列出各识别模式下的天线比。

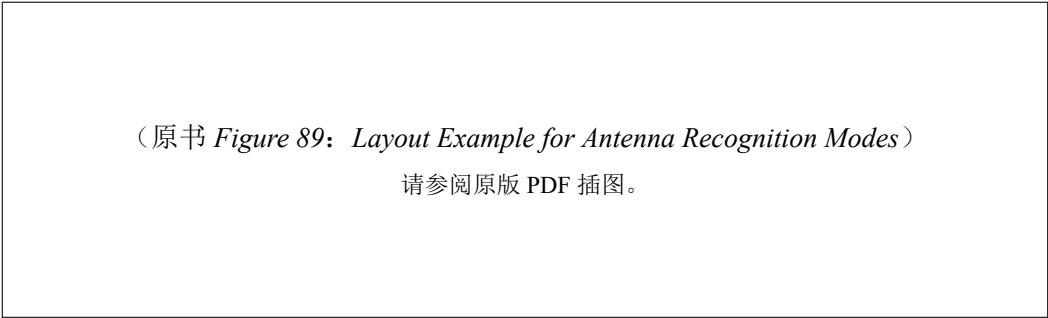


图 7.5: 天线识别模式布局示例

7.3.3 指定天线属性

Specifying Antenna Properties

标准单元与 hard macro 的天线属性一般在参考库的 frame view 中定义。可为未在库中定义属性的单元设置默认值：

- route.detail.default_diode_protection: 标准单元输出 pin 缺省 diode protection;
- route.detail.default_gate_size: 标准单元输入 pin 缺省 gate size;
- route.detail.default_port_external_antenna_area: 顶层 port 缺省天线面积;
- route.detail.default_port_external_gate_size: 顶层 port 缺省 gate size;
- route.detail.macro_pin_antenna_mode: macro pin 的天线处理方式;
- route.detail.port_antenna_mode: 顶层 port 的天线处理方式。

表 7.6: 天线识别模式与比率（原书 Table 33）

考虑的段	天线比
Single-layer	M1: $\text{Gate1}=\text{M1b}/\text{Gate1}$; $\text{Gate2}=\text{M1c}/\text{Gate2}$; $\text{Gate3}=\text{M1d}/\text{Gate3}$ 。 M2: $\text{Gate1}=\text{M2b}/\text{Gate1}$; $\text{Gate2}=\text{M2d}/\text{Gate2}$; $\text{Gate3}=\text{M2e}/\text{Gate3}$ 。 M3: $\text{Gate1,2}=(\text{M3b}+\text{M3c})/(\text{Gate1}+\text{Gate2})$; $\text{Gate3}=\text{M3d}/\text{Gate3}$ 。 M4: $\text{Gate1,2,3}=(\text{M4a}+\text{M4b})/(\text{Gate1}+\text{Gate2}+\text{Gate3})$ 。
Accumulated-ratio	M1 同 single-layer。 M2: $\text{Gate1}=\text{M1b}/\text{Gate1}+\text{M2b}/\text{Gate1}$ （Gate2/3 类似累加）。 M3/M4: 在下层比率上继续累加当前层 single-layer 比率。
Accumulated-area	M1 同 single-layer。 M2: $\text{Gate1}=(\text{M1b}+\text{M2b})/\text{Gate1}$ 等。 M3: Gate1,2 为多层金属面积之和 / (Gate1+Gate2); Gate3 类似。 M4: 全部金属面积 / (Gate1+Gate2+Gate3)。

层次化流程中若为 block 创建 abstraction，须用 derive_hier_antenna_property 从 block 提取天线信息供上一层使用。

注意

若 macro 并非所有层都定义了层次化天线属性，Zroute 视为数据不完整，跳过天线分析并给出 ZRT-311 警告。参见 SolvNet 文章 027178。

参见

在 hard macro 上标注天线属性（Annotating Antenna Properties on Hard Macro Cells）。

7.3.4 分析并修复天线违例

Analyzing and Fixing Antenna Violations

若设计库含天线规则，Zroute 在详细布线期间自动分析并修复天线违例。要禁用，将 route.detail.antenna 设为 false。天线规则与其他设计规则一样在详细布线中并发检查与修正，以减少迭代、缩短总运行时间。

默认行为：

- 对全部时钟与信号网检查并修复；可用 route.detail.skip_antenna_fixing_for_nets 禁止对特定网修复（仍分析并报告）；
- 不对电源/地网检查或修复；将 route.detail.check_antenna_on_pg 设为 true 可启用；
- 在第二次迭代（初始布线完成且基本 DRC 已修复后）开始修复；可用 route.detail.antenna_on_iteration 更改；
- 通过 layer hopping 修复：把大金属多边形拆到更高层，或（对最高层有限保护情形）下移到更低层。

Layer hopping 类型：

- 用更高层金属段打断天线：对 metal-N 违例，在靠近栅极处插入小段 metal-(N+1)，降低剩余 metal-N 比率。不适用于最高层且输出保护有限的情况。
- 下移到更低层：主要用于最高层违例；可能增加 RC 与时序延迟。

也可插入二极管修复。将 `route.detail.insert_diodes_during_routing` 设为 `true` 启用。要强制仅用二极管，将 `route.detail.hop_layers_to_fix_antenna` 设为 `false`。若两者皆启用，默认先尝试 layer hopping；要将偏好改为二极管，设 `route.detail.antenna_fixing_preference` 为 `use_diodes`。

删除修复天线所不需要的冗余二极管：`route.detail.delete_redundant_diodes_during_routing=true`。Spare 二极管不删除；详细布线插入的 port 保护二极管也不删除。

每轮详细布线结束会报告天线违例，例如：

DRC-SUMMARY:

```
@@@@@@ TOTAL VIOLATIONS =      506
@@@@@ Total number of instance ports with antenna violations = 1107
```

检查天线违例：

```
fc_shell> check_routes -antenna true
```

7.3.5 详细布线期间插入二极管

Inserting Diodes During Detail Routing

通过在靠近被保护栅极的网络上插入反向偏置二极管，为累积电荷提供放电通路，且正常工作时不应导通电流。

启用：`route.detail.insert_diodes_during_routing=true`。

控制选项：

- `route.detail.antenna_fixing_preference=use_diodes`：优先二极管而非 hopping；
- `route.detail.hop_layers_to_fix_antenna=false`：强制二极管修复；
- 默认可新增二极管或使用 spare；用 `route.detail.diode_preference` 设为 `new/spare/none`；
- `route.detail.diode_insertion_mode=new/spare/new_and_spare`：强制方法；

注意

要利用 spare 二极管，须在标准单元布局前/后、天线违例可能出现区域布线之前加入 spare。

- `route.detail.diode_libcell_names`：限制可用二极管（仅写单元名，勿含库名）；
- `route.detail.reuse_filler_locations_for_diodes=false`：禁止复用 filler 位置。

层次与 voltage area：

- Pin 违例：二极管插在与违例 pin 相同逻辑层次；
- 顶层 port 违例：默认顶层；若 port 属 voltage area，可将 `route.detail.use_lower_hierarchy_for_port_diodes` 设为 `true`。

详细布线后插入二极管

Inserting Diodes After Detail Routing

详细布线后可用 `create_diodes` 显式指定违例与约束。根据约束与 `route.detail.diode_insertion_mode`，插入新二极管或复用 `spare`。

须用列表指定：`port`、单元实例、二极管参考单元、数量、连接最高层、相对 `pin` 的最大距离。距离内无法插入/复用则跳过。

```
{port_name instance_name diode_reference number_of_diodes
  max_routing_layer max_routing_distance}
```

注意

将单元实例指定为顶层 `block` 名，可为顶层 `port` 插入二极管。

相关应用选项：`route.detail.diode_insertion_mode`、`route.detail.reuse_filler_locations_for_diodes`、`route.detail.use_lower_hierarchy_for_port_diodes`。

示例（CI 的 A port，2 个 Adiode，最高 M5，2.5 μm ）：

```
fc_shell> create_diodes {{A CI Adiode 2 M5 2.5}}
```

7.4 插入冗余 Via

Inserting Redundant Vias

冗余 via 插入是 Zroute 在整个布线流程中支持的重要 DFM 功能。各布线阶段并发优化 via 数量与线长。结果以冗余 via 转换率（单 via 转为冗余 via 的百分比）衡量；亦应关注未优化单 via 数量——越少通常对 DFM 越有利。

相关主题：

- 在时钟网上插入冗余 Via
- 在信号网上插入冗余 Via

7.4.1 在时钟网上插入冗余 Via

Inserting Redundant Vias on Clock Nets

Zroute 可在时钟布线期间或之后插入冗余 via。布线后插入见「布线后冗余 Via 插入」。

时钟布线期间步骤：

1. 用 `create_routing_rule` 在非默认布线规则中指定冗余 via（见「指定非默认 Via」）。选择时须兼顾 DFM 与可布线性。NDR 还可定义更严线宽/间距与 `tapering` 距离。
2. 用 `set_clock_routing_rules` 将 NDR 赋给时钟网：

```
fc_shell> set_clock_routing_rules -rules clock_via_rule
```

3. 运行 CTS：

```
fc_shell> synthesize_clock_trees
```

4. 布线时钟网：

```
fc_shell> route_group -all_clock_nets \
-reuse_existing_global_route true
```

全局布线预留冗余 via 空间，详细布线插入冗余 via。

7.4.2 在信号网上插入冗余 Via

Inserting Redundant Vias on Signal Nets

可采用：

- 布线后冗余 via 插入；
- 并发 soft-rule 冗余 via 插入；
- 接近 100% 冗余 via 插入。

一般先做布线后插入；若转换率 $\geq 80\%$ 可再尝试并发 soft-rule；若 $\geq 90\%$ 可再尝试接近 100% 插入。

注意

转换率升高会使设计规则更难收敛，并可能降低信号完整性；接近 100% 仅用于确需极高转换率的 block，且可能需调整 floorplan utilization 以留出空间。

相关主题：查看默认 Via 映射表；定义定制 Via 映射表；布线后插入；并发 soft-rule；接近 100%；时序保持；报告转换率。

查看默认 Via 映射表

Viewing the Default Via Mapping Table

默认 Zroute 从 technology file 读取默认 contact code 并生成优化映射表。查看：

```
fc_shell> add_redundant_vias -list_only true
```

多数情况下定制映射表优于默认表。若已用 add_via_mapping 创建定制表，该命令显示定制表。

示例（原书 Example 25）：

```
fc_shell> add_redundant_vias -list_only
...
Redundant via optimization will attempt to replace the following vias:
  VIA12SQ_C    -> VIA12SQ_C_2x1    VIA12SQ_C_2x1(r) VIA12SQ_C_1x2 ...
  VIA12SQ_C(r) -> VIA12SQ_C_2x1    VIA12SQ_C_2x1(r) VIA12SQ_C_1x2 ...
...
```

定义定制 **Via** 映射表

Defining a Customized Via Mapping Table

使用 `add_via_mapping`。至少用 `-from/-to` 指定源 via 与替换 via（须为 technology file 中的 via 或 `create_via_def` 创建的设计专用 via）。`-from` 须为 simple via 或 via array；`-to` 可为 simple、simple array 或 custom via。

细化选项：

- `-weight`：优先级 1–10（默认 1）；高权重优先使用；
- `-transform`：all（默认可旋转/翻转）、rotate、flip、none。

映射保存在设计库 via 映射表中。覆盖已有映射须 `-force`。

```
fc_shell> add_via_mapping \
  -from {VIA12 1x1} -to {VIA12T 2x1} -weight 5
fc_shell> add_via_mapping \
  -from {VIA12 1x1} -to {VIA12 2x1} -weight 1
```

用 `report_via_mapping` 查看（`-from/-to` 可过滤）。用 `remove_via_mappings` 移除（`-all/-from/-to`）。

使用 **Via** 映射表子集

Using a Subset of the Via Mapping Table for Redundant Via Insertion

通过应用选项按层选择权重组：

- `route.common.redundant_via_include_weight_group_by_layer_name`：仅用指定权重组；未列出的层不做插入；
- `route.common.redundant_via_exclude_weight_group_by_layer_name`：排除指定权重组；
- 两者同时设置时，在 include 集合中再排除 exclude 集合。

布线后冗余 **Via** 插入

Postroute Redundant Via Insertion

使用 `add_redundant_vias`：可将单 cut 换成多 cut 阵列、换成其他 contact code 的单 cut，或换成不同多 cut 阵列。插入时详细布线检查邻近 partition 的设计规则以减少违例。

默认对所有网插入；`-nets` 可限制网络。完成后重新检查并修复 DRC。

若转换率不足，可用 `-effort high`（约再提升 3–5%，通过移动 via 腾空间）；在 45 nm 及以下可能不利于光刻友好图案，此时宜改用并发 `soft-rule`。也可将 `route.detail.optimize_wire_via_effort_level` 设为 `high` 以减少单 via、缩短线长。

初始布线后插入完成后，设 `route.common.post_detail_route_redundant_via_insertion`，以便后续详细/ECO 布线后自动插入，维持转换率。

并发 **Soft-Rule** 冗余 **Via** 插入

Concurrent Soft-Rule-Based Redundant Via Insertion

在布线期间为冗余 via 预留空间以提高转换率；可用于初始与 ECO 布线。实际插入仍须事后运行 `add_redundant_vias`。

注意

预留空间会增加布线运行时间；仅当布线后方法转换率已 $\geq 80\%$ 且需进一步提高时使用。

步骤：

1. (可选) 定义 via 映射表；
2. 启用并发 `soft-rule`：

```
fc_shell> set_app_options \  
-name route.common.concurrent_redundant_via_mode \  
-value reserve_space
```

用 `route.common.concurrent_redundant_via_effort_level` 控制努力（默认 `low`；`high` 也影响详细布线，可能影响 DRC 收敛）。若在 `place_opt` 前启用，拥塞估计会考虑冗余 via。

ECO：设 `route.common.eco_route_concurrent_redundant_via_mode=reserve_space`，并用对应 `effort` 选项。

注意

ECO 期间使用可能影响时序与 DRC 收敛；一般仅当初始布线用了接近 100% 插入时才在 ECO 用 `soft-rule` 维持转换率。

3. 布线 `block`（预留空间并修复硬规则）；
4. 运行布线后 `add_redundant_vias`。

接近 **100%** 冗余 **Via** 插入

Near 100 Percent Redundant Via Insertion

将冗余 via 作为硬设计规则处理。仅支持初始布线，不支持 ECO；初始用此法后，ECO 应改用 `soft-rule` 以保持转换率。

注意

对拥塞 `block` 可能大幅增加运行时间；仅当 `soft-rule` 已达 $\geq 90\%$ 且仍需提高时使用。

```
fc_shell> set_app_options \  
-name route.common.concurrent_redundant_via_mode \  
-value insert_at_high_cost
```

布线期间插入冗余 via 并修复硬规则；优先级通常与其他硬规则相同，若详细布线 DRC 不收敛会自动放宽冗余 via 约束。

冗余 **Via** 插入时保持时序

Preserving Timing During Redundant Via Insertion

插入冗余 via 会改变时序：短网因电容增加变慢，长网因电阻下降变快。时序保持模式通过对关键网禁止插入来避免影响时序。

在 `add_redundant_vias` 上使用：

- `-timing_preserve_setup_slack_threshold`
- `-timing_preserve_hold_slack_threshold`
- `-timing_preserve_nets`

应在流程末尾、常规插入已收敛转换率与时序 QoR 之后使用；过早使用可能因关键网过多而严重降低转换率。

7.4.3 报告冗余 **Via** 率

Reporting Redundant Via Rates

插入后 Zroute 生成报告，包含：

- 非默认 via 的总优化转换率；
- 各层优化转换率（含 double via 与 DFM-friendly bar via），并分别相对总 via 数与可优化 routed via 数给出；

注意

使用 bar via 时优化转换率参考价值有限。

- 各层按 weight 的优化 via 分布（高于某权重的转换率须对各权重求和）；无权重记为“Un-optimized”；
- block 总 double via 转换率。

注意

`report_design` 的冗余 via 率与 `Zroute/check_routes` 不同：前者含 PG 与信号 via，后者仅信号 via，且基于冗余 via 映射。

示例（原书 Example 27）：

```
Total optimized via conversion rate = 96.94% (1401030 / 1445268 vias)
Layer V01          = 41.89% (490617 / 1171301 vias)
  Weight 10        = 9.64% (112869 vias)
  Weight 5         = 32.25% (377689 vias)
  ...
Total double via conversion rate = 46.69% (2158006 / 4622189 vias)
```

7.5 优化线长与 Via 数量

Optimizing Wire Length and Via Count

详细布线时，Zroute 在存在 DRC 违例的区域优化线长与 via 数量，但不优化无违例区域。

为提高良率，在详细布线与冗余 via 插入之后，可用 `optimize_routes` 独立优化线长与 via 数量。

默认按总体代价选择重布网络；对所选网决定重布全部形状或部分形状。用 `-nets` 指定网络；可用 `-reroute_all_shapes_in_nets` 强制重布该网全部形状。

默认最多 40 次详细布线迭代以修复优化后违例；可用 `-max_detail_route_iterations` 控制。

7.6 减小临界面积

Reducing Critical Areas

临界面积是这样一块区域：若随机颗粒缺陷中心落在该处，会导致电路失效、降低良率。导电缺陷造成短路，非导电缺陷造成开路。

相关主题：

- 通过 wire spreading 减小短路临界面积；
- 通过 wire widening 减小开路临界面积。

7.6.1 执行 Wire Spreading

Performing Wire Spreading

详细布线与冗余 via 插入后，可用 `spread_wires` 增大平均线间距，减小短路临界面积。默认对同层信号线：

- 在优选方向疏导半个 pitch（用 `-pitch` 以层 pitch 倍数修改，如 `-pitch 1.5`）；
- 最小 jog 长度为两倍层 pitch（`-min_jog_length`，层 pitch 整数倍）；
- 最小 jog 间距为最小层间距加半个层 pitch（`-min_jog_spacing_by_layer_name`，按层以微米指定）。

图 7.6 示出 jog 长度与间距的用法。

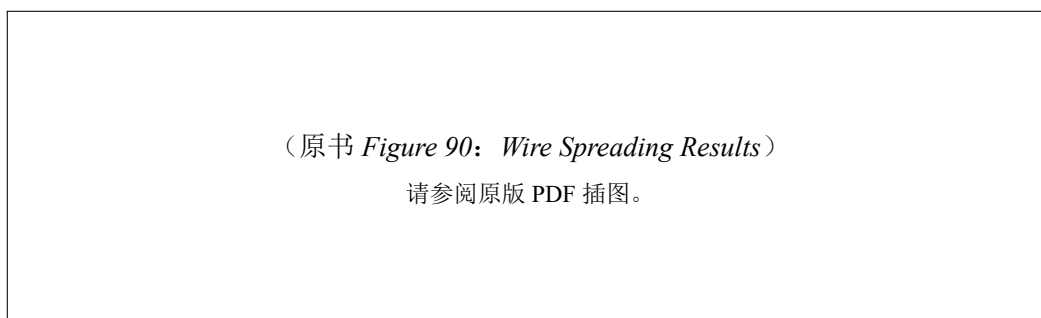


图 7.6: Wire Spreading 结果

```
fc_shell> spread_wires -min_jog_length 3 \
    -min_jog_spacing_by_layer_name {{M1 0.07} {M2 0.08}}
```

疏导后自动进行详细布线迭代修复 DRC。时序保持模式选项：`-timing_preserve_setup_slack_threshold`、`-timing_preserve_hold_slack_threshold`、`-timing_preserve_nets`。仅对 slack 不小于阈值的网、`-timing_preserve_nets` 所列网，以及同层两 pitch 内的相邻网执行疏导。

7.6.2 执行 Wire Widening

Performing Wire Widening

在详细布线、冗余 via 与 wire spreading 之后，用 `widen_wires` 增大平均线宽以减小开路临界面积。默认将所有线加宽到原宽 1.5 倍。可用 `-widen_widths_by_layer_name` 为每层定义最多五种候选宽度：

```
fc_shell> widen_wires \
    -widen_widths_by_layer_name {{M1 0.07 0.06} {M2 0.08 0.07}}
```

加宽会减小邻线间距，可能削弱 spreading 带来的短路临界面积改善。用 `-spreading_widening_relative_weight` 权衡：默认等权；0.0–0.5 偏向 widening（开路）；0.5–1.0 偏向 spreading（短路）。

加宽后进行详细布线迭代修复 DRC；加宽导线不触发 fat wire 间距规则。时序保持选项同 spreading；不对 slack 小于阈值或指定保护网执行加宽。

7.7 插入金属–绝缘体–金属电容

Inserting Metal-Insulator-Metal Capacitors

MiM 电容由两层专用导电层夹绝缘体构成，插在两层常规金属之间，见图 7.7。

（原书 Figure 91: Cross-Section View of MiM Capacitor Layers）
请参阅原版 PDF 插图（含 M8/MBOT/MTOP/M9 与 viaMimbottom/viaMimtop 等 maskName）。

图 7.7: MiM 电容层剖面示意

MiM 电容通常接在电源与地之间，以在电气噪声下维持供电稳定。尺寸常按电源规划的 power strap 几何定制。

用 `create_mim_capacitor_array` 插入阵列并连接到电源轨。至少指定 MiM 库单元及阵列 x/y 增量：

```
fc_shell> create_mim_capacitor_array \
    -lib_cell my_lib/mim_ref_cell \
    -x_increment 20 -y_increment 20
```

默认行为:

- 对整个 block 插入; 可用 `-boundary` 限制到单个矩形/直角多边形区域。插入时不考虑标准单元、macro、placement blockage 或 voltage area;
- 方向 R0; 可用 `-orientation` (作用于阵列全部单元);
- 命名: `mimcap!library_cell_name!number`; 可用 `-prefix`。

7.8 插入 Filler Cell

Inserting Filler Cells

为保证电源网连通, 可在标准单元行空隙中插入 filler; 亦常用于加入去耦电容以稳定供电。工具支持单高与多高 filler。

注意

插入前须用 `check_legality` 确认 block 已合法化。

两种主要方法:

- 有序 filler 参考列表 (标准流程): 按列表顺序在每个间隙插入第一个能放入的单元。若工艺要求特定 filler 邻接标准单元, 可先插入这些必选 filler, 再用标准流程填剩余空隙;
- 阈值电压规则 (VT-based): 按间隙两侧单元的 VT 类型用用户规则选 filler, 多用于成熟晶圆厂节点。

7.8.1 标准 Filler Cell 插入

Standard Filler Cell Insertion

使用 `create_stdcell_fillers` 插入, 用 `remove_stdcell_fillers_with_violation` 检查并移除违例 filler。插入时通过布局合法化器遵守先进节点规则与物理约束; 若 `place.legalize.enable_advanced_legalizer=true`, 使用先进合法化算法做 2D 规则与单元交互检查, 可缩短运行时间。

步骤:

1. 用 `check_legality` 确认合法。注意: `check_legality` 假定所有 `design_type==filler` 的标准单元都可用于填充, 而 `create_stdcell_fillers` 仅插入 `-lib_cells` 所列单元, 二者差异可能导致插入后仍有不可填间隙。
2. (可选) 用 `create_left_right_filler_cells` 为特定标准单元两侧插入必选 filler (见「用特定 Filler 邻接标准单元」)。
3. (可选) `set_host_options -max_cores n` 启用多核 (仅当 technology file 含最小阈值电压、OD 或 TPO 规则时生效)。
4. 用 `create_stdcell_fillers -lib_cells` 插入 metal filler (宜从大到小)。可用:

```
fc_shell> get_lib_cells ref_lib/* -filter "design_type==filler"
fc_shell> set FILLER_CELLS \
```

```
[get_object_name [sort_collection -descending \
  [get_lib_cells ref_lib/* -filter "design_type==filler"] area]]
```

5. connect_pg_net -automatic 连接 PG。
6. remove_stdcell_fillers_with_violation 移除违例 metal filler（可能需多次，因移除会暴露新违例）。
7. 再插入 nonmetal filler 并连接 PG。

参见

Generic ECO Flow for Timing or Functional Changes; 移除 Filler Cell。

7.8.2 控制标准 Filler Cell 插入

Controlling Standard Filler Cell Insertion

默认填充整个 block 水平标准单元行的全部空隙。可控制：

- 按最低漏电选择：-leakage_vt_order 按漏电递减给出 VT 层；-leakage_vt_check 仅检查既有 filler 是否可换成更低漏电单元（最多报告数由 chf.create_stdcell_fillers.max_leakage_vt_order_violation 控制）。须启用 advanced legalizer，且 technology file 中 VT 层 maskType=implant。
- 填充百分比：-utilization；用 -type_utilization 控制各类（如 ULVT/LVT DCAP）比例（总和 ≤100），可用 -fill_remaining 填剩余，-prefer_type_ordering 按组内从左到右优先。
- 区域：-bboxes、-voltage_area。
- 电源网违例：默认不查（省时）；metal filler 建议 -rules check_pnet。
- 防间隙：-smallest_cell_size 为 1/2/3 倍 unit site（advanced legalizer 开启时忽略）。
- **Hard placement blockage**：默认遵守；-ignore_hard_blockages 可忽略。
- 命名：xofiller!...；-prefix/-separator。
- 合法化错误：默认遇错失败；-continue_on_error 可强制完成。

示例：

```
fc_shell> create_stdcell_fillers -lib_cells $FILLER_CELLS \
  -type_utilization { {*/DCAP16*ULVT */DCAP8*ULVT ...} 70
                    {*/DCAP16*LVT */DCAP8*LVT ...} 30 } \
  -prefer_type_ordering -fill_remaining
```

7.8.3 检查 Filler Cell DRC 违例

Checking for Filler Cell DRC Violations

默认 remove_stdcell_fillers_with_violation 检查名称含 xofiller 的实例与顶层信号布线对象之间的 DRC，并移除违例者。可控制：

- -name：识别 filler 的字符串；

- -boundary: 检查区域;
- -check_between_fixed_objects true: 与所有邻接对象检查;
- -shorts_only true: 仅因与布线形状短路而移除;
- 双重图案化: 当 route.common.color_based_dpt_flow=true 且 technology file 有相应规则时检查;
- -check_only true: 只检查不移除, 写 block_fillers_with_violation.rpt。

7.8.4 修复剩余 Mask 设计规则违例

Fixing Remaining Mask Design Rule Violations

部分 mask 规则因出现概率低、计算重而未在插入时处理。插入后用 replace_fillers_by_rules -replacement_rule 修复。表 7.7 列出支持的规则与关键字（原书 Table 34）。

表 7.7: replace_fillers_by_rules 支持的设计规则（原书 Table 34）

设计规则	-replacement_rule	说明
End orientation	end_orientation	翻转最左/最右 filler 方向
Half row adjacency	half_row_adjacency	半高 filler 邻接限制
Illegal abutment	illegal_abutment	禁止某些 filler 邻接
Maximum horizontal length	od_horiztonal_distance	连续 filler 行水平长度
Maximum horizontal edge length	horizontal_edges_distance	特定层邻接对象水平边长
Maximum stacking for small fillers	small_filler_stacking	小 filler 堆叠高度
Maximum vertical edge length	max_vertical_constraint	堆叠 filler 垂直边长
Tap cell spacing	od_tap_distance	Tap OD 与邻接 OD 距离
Random	random	随机替换为定制 filler

End Orientation 规则

End Orientation Rule

对连续水平单元序列，修改最左/最右且属于目标列表的 filler 方向。须指定 -target_fillers, 并用 -left_end/-right_end (inverse、R0_or_MX、MY_or_R180)。-check_only 仅报告将改方向的单元。

Half Row Adjacency 规则

Half Row Adjacency Rule

按邻接标准单元类型（Inbound: 顶/底边界在半高处，需 ICIP; Regular: 全高边界，邻接侧不得有 ICIP）替换半高 filler。替换单元与原 filler 同高、同宽、同 VT；库提供带 ICIP 的 p/n 型。图 7.8; 表 7.8。

每个列表每种尺寸只能含一个单元。

（原书 *Figure 92: Half-Height Filler Cell Replacement With Half Row Adjacency Rule*）
请参阅原版 PDF 插图。

图 7.8: 半高 Filler 替换与半行邻接规则

表 7.8: 半高替换 Filler Cell（原书 Table 35）

邻接 inbound 标准单元	指定替换 filler 的选项
无	-p_none、-n_none
仅左	-p_left、-n_left
仅右	-p_right、-n_right
左与右	-p_left_right、-n_left_right
顶或底（仅 2X filler）	-p_left_center_right、-n_left_center_right

Illegal Abutment 规则

Illegal Abutment Rule

```
fc_shell> replace_fillers_by_rules -replacement_rule illegal_abutment \  
-replace_abutment { {FILL1a CFILL1a} {FILL1b CFILL1b} } \  
-illegal_abutment {FILLx FILLy}
```

-skip_fixed_cells 可忽略 physical_status 为 fixed/locked 的 filler。图 7.9。

（原书 *Figure 93: Fixing an Illegal Abutment Violation*）
请参阅原版 PDF 插图。

图 7.9: 修复非法邻接违例

最大水平边长规则

Maximum Horizontal Edge Length Rule

-replacement_rule horizontal_edges_distance, 须指定 -max_constraint_length、-layer, 以及 -refill_table 或 -constraint_fillers/-non_constraint_fillers。

```
fc_shell> replace_fillers_by_rules \  
-replacement_rule horizontal_edges_distance \  
-max_constraint_length 30 -layer M2 \  

```



```
-refill_table { {{VT1_FILL1}} {VT1_DECAP4 VT1_DECAP2}}
               {{VT2_FILL1}} {VT2_DECAP4 VT2_DECAP2}} }
```

最大水平长度规则

Maximum Horizontal Length Rule

-replacement_rule od_horiztonal_distance（原文关键字拼写如此）。示例限制连续 filler 水平长度 70 μm:

```
fc_shell> replace_fillers_by_rules \
-replacement_rule od_horiztonal_distance -max_constraint_length 70 \
-refill_table { {{VT1_FILL1}} {VT1_DECAP4 VT1_DECAP2}}
               {{VT2_FILL1}} {VT2_DECAP4 VT2_DECAP2}} }
```

图 7.10（原书 Figure 94）。

（原书 Figure 94：相关最大长度违例修复示意）
请参阅原版 PDF 插图。

图 7.10: 修复最大水平长度相关违例

小 Filler 最大堆叠规则

Maximum Stacking for Small Filler Cells Rule

-replacement_rule small_filler_stacking,指定 -max_constraint_length、-small_fillers、-replacement_fillers。超出高度时用大 filler 替换相邻 2–3 个小 filler；无法替换则重排。保持违例处 VT，非违例处 VT 可能改变且不检查。

```
fc_shell> replace_fillers_by_rules \
-replacement_rule small_filler_stacking -max_constraint_length 10 \
-small_fillers {FILL1 FILL2} -replacement_fillers {FILL3 FILL4}
```

图 7.11、7.12。

（原书 Figure 95: *Fixing Maximum Stacking Violation by Replacement*）
请参阅原版 PDF 插图。

图 7.11: 通过替换修复最大堆叠违例

(原书 *Figure 96: Fixing Maximum Stacking Violation by Reordering*)

请参阅原版 PDF 插图。

图 7.12: 通过重排修复最大堆叠违例

最大垂直边长规则

Maximum Vertical Edge Length Rule

-replacement_rule max_vertical_constraint。指定 -max_constraint_length、-constraint_fillers (默认所有标准单元受约束, -exception_cells 排除; -no_violation_along_exception_cells 以及 -non_constraint_fillers/-non_constraint_left_fillers/-non_constraint_right_fillers。

```
fc_shell> replace_fillers_by_rules \
  -replacement_rule max_vertical_constraint -max_constraint_length 180 \
  -constraint_fillers {DECAP8 DECAP4} -non_constraint_fillers {FILL1} \
  -exception_cells {BUF1 TAP1}
```

图 7.13。

(原书 *Figure 97: Fixing a Maximum Vertical Edge Length Violation*)

请参阅原版 PDF 插图。

图 7.13: 修复最大垂直边长违例

Tap Cell Spacing 规则

Tap Cell Spacing Rule

-replacement_rule od_tap_distance。指定 -tap_distance_range (site 数)、-tap_cells, 以及 -left_violation_tap/-right_violation_tap/-both_violation_tap。目标与替换 tap 须同尺寸。对无 OD 邻接单元用 -adjacent_non_od_cells 跳过。

```
fc_shell> replace_fillers_by_rules -replacement_rule od_tap_distance \
  -tap_cells {TAP} -tap_distance_range {3 3} \
  -left_violation_tap {TAP1} -right_violation_tap {TAPr} \
  -both_violation_tap {TAPb}
```

图 7.14。

(原书 Figure 98: *Fixing a Tap Cell Spacing Violation*)

请参阅原版 PDF 插图。

图 7.14: 修复 Tap Cell 间距违例

7.8.5 随机 Filler Cell 替换

Random Filler Cell Replacement

```
fc_shell> replace_fillers_by_rules -replacement_rule random \
    -random_replace { {FILL1 {FILL1a FILL1b FILL1c}}
                    {FILL2 {FILL2a FILL2b}} }
```

可用 `-skip_fixed_cells`。

7.8.6 用特定 Filler 邻接标准单元

Abutting Standard Cells With Specific Filler Cells

用 `create_left_right_filler_cells` 在特定标准单元左/右侧插入必选 filler，然后再做标准 filler 插入填剩余空隙。

`-lib_cells` 规则格式：

```
{standard_cells} {left_filler_cells} {right_filler_cells}
```

要求：至少一侧有 filler；filler 的 `design_type` 为 `lib_cell` 或 `filler`；与标准单元同 site；标准单元高度为 filler 高度整数倍；宜从大到小。通配符顺序与 `get_lib_cells` 一致。

```
fc_shell> create_left_right_filler_cells \
    -lib_cells {
        { {mylib/SC1} {mylib/LF4X mylib/LF2X} {mylib/RF2X mylib/RF1X} }
        { {mylib/SC2} {mylib/LF2X mylib/LF1X} {} } }
```

插入时检查合法位置，但不检查最小 VT、OD、TPO 等先进规则；紧贴标准单元插入，不考虑单元间距规则。

7.8.7 控制基于单元的 Filler 插入

Controlling Cell-Based Filler Cell Insertion

`create_left_right_filler_cells` 还可控制：

- 区域： `-boundaries`、`-voltage_areas`；
- 方向：默认按行允许方向；`-follow_stdcell_orientation` 跟随标准单元（翻转时左右互换）；
- 一单位 tile 间隙： `-rules no_1x`（此时勿在列表中包含宽度为 1 tile 的 filler）；

- 命名约定：同标准 filler (xofiller!...)。

7.8.8 基于阈值电压的 Filler 插入

Threshold-Voltage-Based Filler Cell Insertion

步骤：

1. 确保 VT 类型信息可用；
2. 用 `set_vt_filler_rule` 定义规则；
3. (可选) 为多种邻接 VT 组合分别设规则；
4. 用 `create_vtcell_fillers` 插入。

```
fc_shell> set_vt_filler_rule -vt_type {default default} \
    -filler_cells {myLib/filler2x myLib/filler1x}
fc_shell> set_vt_filler_rule -vt_type {vtA vtA} \
    -filler_cells {myLib/fillerA2x myLib/fillerA1x}
fc_shell> create_vtcell_fillers
```

7.8.9 控制基于阈值电压的 Filler 插入

Controlling Threshold-Voltage-Based Filler Cell Insertion

默认填充整个 block 水平标准单元行空隙。可控制 `-region`、`-voltage_area`，以及命名（默认 `xofiller!...`；`-prefix/-separator`）。

7.8.10 移除阈值电压 Filler 信息

Removing the Threshold-Voltage Filler Cell Information

```
fc_shell> create_vtcell_fillers -clear_vt_information
```

移除 **Filler Cell**

Removing Filler Cells

```
fc_shell> remove_cells [get_cells xofiller*]
```

仅移除有 DRC 违例的标准流程 filler：用 `remove_stdcell_fillers_with_violation`。

7.9 插入 Metal Fill

Inserting Metal Fill

布线后，可用金属导线填充 block 空隙以满足金属密度规则。插入前 block 应接近满足时序，且 DRC 很少或没有。

运行 `signoff_create_metal_fill`，详见第 八 章「使用 IC Validator In-Design 插入 Metal Fill」。

注意

运行 `signoff_create_metal_fill` 需要 IC Validator license。

Chapter 8

IC Validator In-Design

IC Validator In-Design

IC Validator In-Design 是 Fusion Compiler 与签核级物理验证工具 IC Validator 之间的紧密集成接口。借助这一接口，设计人员无需离开 `fc_shell` 环境、也无需手工导出版图数据，即可直接调用 IC Validator 的验证引擎完成一系列物理实现与验证任务。运行这些功能需要有效的 IC Validator 许可证。

本章围绕以下五类核心能力展开：

- 签核级设计规则检查（signoff design rule checking），核心命令为 `signoff_check_drc`；
- 对签核 DRC 检查出的违例进行自动修复，核心命令为 `signoff_fix_drc`；
- 基于电压降（IR drop）分析结果对电源网络进行增强，核心命令为 `signoff_create_pg_augmentation`；
- 插入满足金属密度规则的 metal fill，核心命令为 `signoff_create_metal_fill`；
- 检测并修复孤立 via（isolated via），核心命令为 `signoff_fix_isolated_via`。

这些命令统称为 IC Validator In-Design 命令。它们有一套共同的运行前准备流程（配置 IC Validator 执行环境、可选的多核/分布式处理设置、layer mapping 文件），也都遵循相似的「设置应用选项 → 保存 block 到磁盘 → 运行命令 → 查看结果文件」工作模式。掌握第一节所述的准备工作，是正确使用后续各命令的基础。

注意

运行任何 IC Validator In-Design 功能都需要一份有效的 IC Validator 许可证。

8.1 运行 IC Validator In-Design 命令前的准备工作

Preparing to Run IC Validator In-Design Commands

在调用任何一条 IC Validator In-Design 命令之前，通常需要完成三件事：告诉 Fusion Compiler 到哪里去找 IC Validator 可执行程序；根据需要配置多核/分布式运算以缩短验证时间；以及在工艺层号与代工厂 runset 层号不一致时提供 layer mapping 文件。下面分小节说明。

8.1.1 搭建 IC Validator 运行环境

Setting Up the IC Validator Environment

Fusion Compiler 本身并不内置 IC Validator 的验证引擎，而是在需要时以子进程方式调用外部的 IC Validator 可执行程序。因此第一步是通过环境变量 `ICV_HOME_DIR` 告知工具 IC Validator 的安装路径，通常将其写入 shell 的启动配置文件（如 `.cshrc`）中：

```
setenv ICV_HOME_DIR /root_dir/icv
set path = ($path $ICV_HOME_DIR/bin/LINUX.64)
```

注意

所指定的 IC Validator 版本必须与当前使用的 Fusion Compiler 版本相互兼容，两者不匹配可能导致运行失败或结果不可信。可使用 `report_versions` 命令核对版本兼容关系；具体的兼容版本对照信息，请查阅 SolvNet 上的 IC Validator 文档。

8.1.2 启用 IC Validator 多核处理

Enabling IC Validator Multicore Processing

默认情况下，IC Validator In-Design 命令使用单一进程完成设计规则检查等工作，这在大型 block 上可能非常耗时。为缩短运行时间，可以启用多核处理——包括多线程、跨主机分布式处理，以及若干子命令的并行执行。用户只需告诉工具在哪些主机上可以使用多少核心，IC Validator 会自行决定如何组合多线程与并行执行以取得最短运行时间。

启用多核处理的命令是 `set_host_options`。该命令默认对所有支持多核处理的 Fusion Compiler 命令生效；若只想让配置作用于 IC Validator In-Design 命令，需加上 `-target ICV` 选项。

可以采用以下三种方式之一（或组合）来启用 IC Validator 的多核处理：

- 直接指定若干台具体主机（Specific Hosts）；
- 借助作业调度系统（Job Scheduler），如 LSF 或 Grid Engine；
- 将上述两种方式混合使用（Hybrid）。

在指定主机上运行

Running IC Validator on Specific Hosts

若不指定主机名或提交协议，IC Validator 默认运行在当前 Fusion Compiler 所在的主机上。要在一台或多台指定主机上启用多核处理，需要用 `set_host_options` 指定：

- 主机名：作为命令的位置参数给出一个或多个主机名；
- 核心数：用 `-num_processes` 或 `-max_cores` 指定；若两者同时给出，则实际使用的核心数为二者的乘积。该核心数适用于所列出的每一台主机。

例如，让当前主机使用 4 个核心：

```
fc_shell> set_host_options -target ICV -num_processes 4
```

又如，让 `machineA` 与 `machineB` 各使用 8 个核心（共 16 个核心）：

```
fc_shell> set_host_options -target ICV \
    -num_processes 2 -max_cores 4 {machineA machineB}
```

借助作业调度系统运行

Running IC Validator Using a Job Scheduler

若希望通过作业调度系统动态申请计算资源，而不是写死具体主机名，可以指定：

- 提交协议（`-submit_protocol`）：
 - `lsf`：使用 LSF（Load Sharing Facility），要求 `bsub` 命令位于 Linux 路径中；
 - `grid`：使用 Univa Grid Engine（UGE，旧称 SGE / GRD），要求 `qsub` 命令位于 Linux 路径中。

同样，若不指定主机名或提交协议，IC Validator 默认运行在当前主机上。

- 申请的主机数：用 `-num_processes` 指定；
- 每台主机使用的核心数：用 `-max_cores` 指定；
- （可选）提交命令的附加参数：用 `-submit_command` 指定完整的提交命令字符串。

例如，通过 LSF 申请 4 台主机、每台 8 核（共 32 核）：

```
fc_shell> set_host_options -submit_protocol lsf -target ICV \
    -num_processes 4 -max_cores 8
```

为上述作业追加内存需求（1000MB）：

```
fc_shell> set_host_options -submit_protocol lsf -target ICV \
    -num_processes 4 -max_cores 8 \
    -submit_command {bsub -R rusage[mem=1000]}
```

通过 Grid Engine 申请等价资源：

```
fc_shell> set_host_options -submit_protocol grid -target ICV \
    -num_processes 4 -max_cores 8
```

为 Grid Engine 作业指定 project 名称（如 `bnormal`）：

```
fc_shell> set_host_options -submit_protocol grid -target ICV \
    -num_processes 4 -max_cores 8 -submit_command "qsub -P bnormal"
```

混合方式运行

Running IC Validator Using Hybrid Multicore Processing

在使用 LSF 时，可以将「固定使用的具体主机」与「按需向 LSF 申请的额外主机」结合起来。做法是：先按前述方法配置具体主机，再用 `-add_hosts` 选项追加 LSF 配置。这部分追加的资源既是「可选资源」也是「资源上限」——LSF 可能在运行的开始、中途或结束时才把这些主机交给作业，也可能完全不分配。

例如，先固定使用当前主机的 4 个核心，再尝试向 LSF 额外申请最多 4 台主机：


```
fc_shell> set_host_options -target ICV -max_cores 4
fc_shell> set_host_options -target ICV -add_hosts \
    -submit_protocol lsf -num_processes 4
```

参见

set_host_options 命令的通用说明，另见「Enabling Multicore Processing」一节。

8.1.3 定义 IC Validator In-Design 命令使用的 Layer Mapping

Defining the Layer Mapping for IC Validator In-Design Commands

Fusion Compiler 使用的技术文件（technology file）与代工厂提供的 runset 文件，两者对同一物理层的编号约定不一定相同。当二者层号不一致时，就需要一份 layer mapping 文件，把技术文件中的层号映射到 runset 文件所用的层号。该文件的位置通过应用选项 signoff.physical.layer_map_file 指定。

Layer mapping 文件每一行描述技术文件中的一个层与 runset 文件中对应层的映射关系，语法为：

```
tf_layer[:tf_purpose][:use_type][:mask_type]
    runset_layer[:runset_data_type]
```

各字段含义如下：

- **tf_layer**（必填）：技术文件中的层号；
- **tf_purpose**：技术文件中的 purpose 号；
- **use_type**：该几何图形在设计中的用途，取值可为 power、ground、signal、clock、boundary、hard_placement_blockage、soft_placement_blockage、routing_blockage、area_fill、track 之一；
- **mask_type**：多重曝光（multiple patterning）掩膜约束；金属层取值为 mask_one、mask_two、mask_three、mask_same，via 层还额外支持 MASK_FOUR 至 MASK_FIFTEEN；
- **runset_layer**（必填）：runset 文件中对应的层号；
- **runset_data_type**：runset 文件中对应的 data type 号。

一行中分号（;）之后的内容均视为注释，不影响映射关系。

8.2 执行签核级设计规则检查

Performing Signoff Design Rule Checking

签核级设计规则检查（signoff design rule checking，signoff DRC）在 Fusion Compiler 内部调用 IC Validator 引擎，依据代工厂提供的签核 runset 文件检查布线是否满足工艺规则。它主要有两种使用方式：

- 运行 signoff_check_drc 命令，对整个 block 做一次完整或增量式检查；
- 使用 GUI 中的 Live DRC 功能，在版图窗口中对当前显示区域做交互式检查。

典型流程是：完成布局布线之后先用 `signoff_check_drc` 做一次全芯片检查；之后每次 ECO 之后，既可以再次运行 `signoff_check_drc`（结合增量模式），也可以用 Live DRC 只对刚修改的局部区域做检查。

参见

自动修复签核 DRC 违例，见「自动修复签核 DRC 违例」一节。

8.2.1 运行 `signoff_check_drc` 命令

Running the `signoff_check_drc` Command

运行签核 DRC 检查的一般步骤为：

- 1. 按前一节所述搭建好 IC Validator 运行环境；
- 2. （可选）用 `set_host_options` 启用分布式/多核处理；
- 3. 设置本次检查所需的应用选项；至少必须通过 `signoff.check_drc.runset` 指定所用的代工厂 `runset`；
- 4. 用 `save_block` 把当前 block 状态保存到磁盘——IC Validator 读取的是磁盘上的版图数据，而非内存中的实时状态，因此运行前务必先保存；
- 5. 运行 `signoff_check_drc` 命令。

命令默认会依次完成三件事：把 block 加载进 IC Validator（加载规则见下文「读取 Block 的规则」）；执行签核 DRC 检查；生成错误数据文件（error data file）以及一系列 IC Validator 结果文件。随后既可以在 error browser 中查看违例列表，也可以把结果作为自动修复流程（`signoff_fix_drc`）的输入。此外还可以借助 ICV heat map 观察违例在版图上的密度分布。

参见

在 GUI 中交互式检查签核设计规则，见「在 GUI 中交互式检查签核设计规则」一节。

设置签核 DRC 检查的应用选项

Setting Options for Signoff Design Rule Checking

运行 `signoff_check_drc` 之前，可通过 `set_app_options` 设置表 8.1 所列应用选项来配置本次运行；用 `report_app_options` 可查看当前生效的设置。

表 8.1: 签核 DRC 检查的应用选项

应用选项	默认值	说明
<code>signoff.check_drc.runset</code> (必填)	N/A	指定 DRC 检查所用的代工厂 <code>runset</code> 。
<code>signoff.check_drc.auto_eco_threshold_value</code>	10	使用 <code>-auto_eco true</code> 做增量检查时允许的最大改动比例（百分比）。

应用选项	默认值	说明
signoff.check_drc.excluded_cell_types	(无)	指定跳过检查的单元类型，可取 lib_cell（标准单元）、macro（宏单元）、pad（I/O pad）、filler（填充单元）中的一个或多个。
signoff.check_drc.fill_view_read_if_updated	read	控制是否读取 fill 数据：默认仅当 fill 数据时间戳比 design view 新时才读取；设为 read 表示始终读取，设为 discard 表示从不读取。
signoff.check_drc.ignore_blockages_in_cells	true	控制对宏单元/I-O pad 只读取 frame view 中的 pin 信息（true）还是同时读取 blockage 与 pin 信息（false）。
signoff.check_drc.ignore_child_cell_errors	false	控制错误数据文件中是否同时包含顶层与子单元级违例（false）或只包含顶层违例（true）。
signoff.check_drc.max_errors_per_rule	1000	每条规则最多报告的违例数。
signoff.check_drc.read_design_views	否	指定需要读取 design view（而非 frame view）的参考单元；design view 能揭示被 frame view 抽象所掩盖的问题。
signoff.check_drc.read_layout_views	否	指定需要读取 layout view（而非 frame view）的参考单元；效果与读 GDSII/OASIS 数据一致但可缩短运行时间。
signoff.check_drc.run_dir	signoff_check_drc_dir	指定运行目录，用于存放 signoff_check_drc 产生的文件；可为相对路径（在当前目录下创建）或绝对路径。
signoff.check_drc.user_defined_options	(无)	追加到 IC Validator 命令行的额外参数字符串；工具不对该字符串做任何检查。
signoff.physical.layer_map_file	(无)	指定 layer mapping 文件名，用于在技术文件层号与 runset 层号不一致时做映射，见前一节。
signoff.physical.merge_exclude_libraries	(无)	指定哪些单元库中的单元不参与 signoff.physical.merge_stream_files 指定的 stream 数据替换。
signoff.physical.merge_stream_files	(无)	指定需要合并进当前 block 用于签核 DRC 检查的 stream（GDSII 或 OASIS）文件；使用后这些文件中的数据会替代对应单元库的数据。

读取 Block 的规则

Reading Blocks for Signoff Design Rule Checking

默认情况下，IC Validator 对顶层 block 和标准单元实例读取 design view，对宏单元与 I/O pad 单元实例只从 frame view 中读取 pin 信息。可以按需调整：

- 把 signoff.check_drc.ignore_blockages_in_cells 设为 false，可让工具同时读取宏单元/pad 的 frame view 中的布线 blockage 与 pin 信息；
- 通过 signoff.physical.merge_stream_files 指定 stream 文件，可让特定参考单元直接使用

GDSII/OASIS 版图数据（`signoff.physical.merge_exclude_libraries` 列出的库除外）；

- 通过 `signoff.check_drc.read_layout_views` 指定特定参考单元读取 layout view——其内容与生成它所用的 GDSII/OASIS 数据完全一致，但运行速度更快；
- 通过 `signoff.check_drc.read_design_views` 指定特定参考单元读取 design view。

注意

Layout view 与 design view 默认都不包含在单元库中；如需使用，须在库准备阶段分别把 `lib.workspace.save_layout_views`、`lib.workspace.save_design_views` 设为 true，详见 Library Manager User Guide。

子单元数据的读取优先级依次为：

1. `signoff.physical.merge_stream_files` 指定的 GDSII/OASIS 数据；
2. `signoff.check_drc.read_layout_views` 指定的 layout view；若找不到 layout view 而该单元又出现在 `signoff.check_drc.read_design_views` 列表中，则改读 design view；若二者均无，再退回读 frame view；若 frame view 也不存在，该单元在检查中被忽略；
3. `signoff.check_drc.read_design_views` 指定的 design view；找不到则退回 frame view，仍找不到则忽略该单元；
4. Frame view。

例如，对所有 block 都读取 design view：

```
fc_shell> set_app_options \
    -name signoff.check_drc.read_design_views -value {*}
fc_shell> signoff_check_drc
```

又如，只对顶层 block 以及名字以 mc 开头的所有参考单元实例读取 design view，其余子 block 仍读 frame view：

```
fc_shell> set_app_options \
    -name signoff.check_drc.read_design_views -value {mc*}
fc_shell> signoff_check_drc
```

签核设计规则检查的默认行为与常见定制

Signoff Design Rule Checking

不做任何额外设置时，签核 DRC 检查具有以下默认行为，均可按需覆盖：

- 检查所有单元类型（标准单元、filler、宏单元、I/O pad）。若要排除某些类型，设置 `signoff.check_drc.excluded_cell_types`，取值可为 `lib_cell`、`filler`、`macro`、`pad` 之一或多个。
- 对整个 block 做检查。
 - 若只想检查自上次运行以来发生改动的区域，使用 `-auto_eco true`。该选项仅当此前已运行过 `signoff_check_drc`，且自那次运行以来 block 的改动比例低于变化阈值（默认 20%，可通

过 `signoff.check_drc.auto_eco_threshold_value` 调整) 时才可用。默认将当前 block 与上一次 `signoff_check_drc` 所用版本比较; 如需与其他 block 比较, 用 `-pre_eco_design` 指定对比对象。增量检测所需信息来自名为 `block_sdrc.err` 的错误数据文件 (`block` 为当前 block 名或 `-pre_eco_design` 指定的名字)。

默认情况下, 增量检查报告的违例只覆盖本次改动区域; 若想把本次结果与此前的初始违例合并展示, 需在运行 `-auto_eco true` 之前将 `signoff.check_drc.merge_incremental_error_data` 设为 `true`。默认合并时读取的是 `signoff_check_drc.err` 文件中的初始违例; 如需换用其他文件, 通过 `signoff.check_drc.merge_base_error_name` 指定。

- 若只想检查特定区域, 用 `-coordinates` (限定区域) 和/或 `-excluded_coordinates` (排除区域) 指定; 两者可同时使用, 重叠部分不做检查。坐标格式为 `{{x1 y1} {x2 y2} ... {xn yn}}` (每组坐标之间要有空格)。
- 检查 **runset** 中定义的全部规则。用 `-select_rules` 按模式匹配只选中部分规则, 或用 `-unselect_rules` 排除部分规则; 两者可组合使用, 例如只想做布线本身的校验而跳过金属密度检查时, 可选中金属层相关规则、排除密度相关规则。规则名取自 `runset` 文件 `COMMENT` 段中的定义。
- 检查所有布线层。用 `-select_layers` 限定只检查指定布线层 (层名取自技术文件); 或用 `-check_all_runset_layers true` 让检查覆盖 `runset` 中的全部层, 包括非布线层——这通常用于在完成布线层检查后, 对整个 block 做一次快速的补充检查; 此模式下版图信息来自 `design view`。
- 每条规则最多报告 **1000** 条违例 (含顶层与子单元级)。可通过 `signoff.check_drc.max_errors_per_rule` 调整上限; 也可将 `signoff.check_drc.ignore_child_cell_errors` 设为 `true` 以忽略子单元级违例。

`signoff_check_drc` 会根据所给选项拼装出调用 IC Validator 的命令行; 如需追加其他 IC Validator 命令行参数, 可通过 `signoff.check_drc.user_defined_options` 指定 (工具不检查该字符串的合法性)。

注意

若打算把签核 DRC 结果作为自动修复流程的输入, 应在运行 `signoff_check_drc` 前把 `signoff.check_drc.ignore_child_cell_errors` 设为 `true`: Zroute 只能修复顶层违例, 这样设置可确保签核 DRC 只报告可被修复的违例。

签核 DRC 结果文件

Signoff DRC Results Files

`signoff_check_drc` 把输出文件写入运行目录 (由 `signoff.check_drc.run_dir` 指定, 默认 `signoff_check_drc_run`), 主要包括:

- 错误数据文件: 默认名为 `signoff_check_drc.err`, 存放于设计库中; 可用 `-error_data` 指定其他文件名。该文件对子单元级错误只在其中一个实例上标注, 便于定位底层错误来源。可用它在 `error browser` 中查看或展示违例。
- **IC Validator** 结果文件, 可用于调试签核 DRC 结果, 包括:

- `block.LAYOUT_ERRORS`——本次运行检测到的违例详情；
- `block.RESULTS`——本次运行结果摘要；
- `signoff_check_drc.log`——本次运行环境摘要；
- `icv_config_out`——顶层 `block` 与各单元库的路径信息；
- `icv_sdrc.conclude`——违例汇总报告；
- `layer.map`——本次运行自动生成的 layer mapping 文件；
- `signoff_check_drc.rc`——IC Validator runset 用到的环境变量；
- `./run_details` 目录——IC Validator 为本次运行产生的全部原始数据。

更多细节请参阅 IC Validator 文档。

在 **ICV Heat Map** 中查看违例

Viewing the Violations in an ICV Heat Map

ICV heat map 以密度图的形式展示签核 DRC 违例在 `block` 上的分布，便于快速定位问题密集的区域。

注意

该功能要求 IC Validator 版本不低于 P-2019.06，并需要 IC Validator NXT 许可证。

查看步骤：

1. 将应用选项 `signoff.check_drc.enable_icv_explorer_mode` 设为 `true`，开启 ICV heat map：

```
fc_shell> set_app_options \
    -name signoff.check_drc.enable_icv_explorer_mode -value true
```

2. 按前述方法运行签核 DRC 检查；
3. 在 GUI 中依次选择 `View > Map > ICV Heatmap`。

（原书 *Figure 99: ICV Heat Map*）

请参阅原版 PDF 插图。

图 8.1: ICV Heat Map 示例

在 heat map 视图中，可以：

- 勾选或取消特定违例类型的显示；
- 调节 Alpha（不透明度）百分比，控制色块的透明程度：设为 100% 时色块完全不透明，数值越低越透明；

- 调节 Bins 值，控制颜色分级的粒度；
- 勾选或取消特定层的显示；
- 在色块上右键选择 Set Style，自定义色块的显示样式。

(原书 Figure 100–109: Heat Map 显示与样式设置示例)
请参阅原版 PDF 插图。

图 8.2: Heat Map 显示选项与样式设置

此外，可将 Error Browser 与 heat map 联动：在 GUI 中打开 View > Error Browser，选中某次 DRC 运行产生的错误数据，再在 Error Browser 中选中具体违例，即可自动在 heat map 上高亮对应位置。

8.2.2 在 GUI 中交互式检查签核设计规则

Checking Signoff Design Rules Interactively in the GUI

Live DRC 是在 Fusion Compiler GUI 中执行交互式签核设计规则检查的功能，效果类似把 IC Validator 检查即时嵌入版图窗口。

注意

该功能要求 IC Validator 版本不低于 P-2019.06。

使用 Live DRC 的步骤：

1. 按前述方法搭建 IC Validator 环境；
2. 设置交互式检查所需的应用选项，至少要指定：
 - `signoff.check_drc_live.runset`：用于检查的代工厂 runset；
 - `signoff.physical.layer_map_file`：把技术文件层号映射到 runset 层号的 layer mapping 文件。
3. 确保版图窗口中显示了想要检查的对象。Live DRC 默认使用各单元的 frame view；若需要更精细的细节，可通过 `signoff.physical.merge_stream_files` 为特定单元指定 GDSII/OASIS 文件，例如：

```
fc_shell> set_app_options \
-name signoff.physical.merge_stream_files \
-value {stdcell.gds macro.oas}
```

4. 在 GUI 菜单栏空白处右键选择 DRC Tools，打开 DRC 工具条；

5. 通过 Edit > Options > ICV-Live，或点击 DRC 工具条上的齿轮图标，配置本次检查要使用的规则；若要整体排除密度或连接性相关规则，设置 `signoff.check_drc_live.exclude_command_class`；
6. 通过 Edit > ICV > Run ICV-Live on Current View，或点击工具条上的运行按钮，执行检查。

首次运行 Live DRC 时，工具需要处理 IC Validator runset（获取可用规则列表并缓存 runset）以及 `signoff.physical.merge_stream_files` 中指定的 GDSII/OASIS 文件，这一步可能耗时数分钟；只要层、规则、stream 文件不变，后续运行会直接复用缓存而明显加快。若希望提前完成这部分初始化，可预先运行 `signoff_init_live_drc` 命令。

检查结果同样显示在 Error Browser 中，可以逐条选中并交互式修复。

参见

命令方式的签核 DRC 检查，见「运行 `signoff_check_drc` 命令」一节。

控制用于检查的显示对象

Displaying Objects for Design Rule Checking

交互式检查时，IC Validator 实际检查的范围是当前版图窗口中显示的内容，再加上 1 微米的默认延展（extension）。可通过 `signoff.check_drc_live.halo` 调整该延展距离。

可通过 View Settings 与 Hierarchy Settings 面板控制窗口中显示的层、层级深度与单元类型：

- 打开 View Settings 面板：设置 Level 字段控制显示层级数；在 Layers 页签中勾选需要显示的层；
- 打开 Hierarchy Settings 面板：设置「View level」字段控制视图层级数；在「Expanding cell types」中勾选 Standard、Fill Cell、Block。



图 8.3: 用于控制 DRC 显示范围的设置面板

DRC 工具条

DRC Toolbar

DRC 工具条集中了配置和运行 Live DRC、以及在版图中查看已报告违例所需的按钮。

表 8.2: DRC 工具条按钮功能

按钮功能	说明
运行检查	使用 Live DRC 执行设计规则检查。

按钮功能	说明
选项设置	打开对话框查看和设置 Live DRC 应用选项。
规则配置	打开对话框配置本次检查所用的规则。
选择检查区域	在版图中框选本次检查所覆盖的区域。
恢复上次区域	将检查区域重置为上一次 Live DRC 运行所用的区域。
错误环境叠加开关	启用/关闭下层单元中导致违例的 GDSII/OASIS 图形叠加显示；仅当运行检查前已设置 <code>signoff.physical.merge_stream_files</code> 时可用。
清除/高亮全部违例	清除或高亮 Live DRC 检测到的全部违例。
逐条浏览违例	依次遍历当前规则下的各条违例。
缩放/平移跟随	控制在浏览违例时是否自动缩放或平移视图。
显示规则名	显示当前高亮违例所属的设计规则名称。

设置交互式设计规则检查的应用选项

Setting Options for Interactive Design Rule Checking

运行交互式检查之前，可通过表 8.3 所列应用选项配置本次运行。

表 8.3: 交互式设计规则检查的应用选项

应用选项	默认值	说明
<code>signoff.check_drc_live.check_all_visible_layers</code>	<code>all_layers</code>	控制是否只检查可见层上定义的规则；取值可为 <code>all_layers</code> （默认，覆盖所有层）、 <code>visible_involved</code> 、 <code>visible_only</code> 。
<code>signoff.check_drc_live.default_layers</code>	<code>all_layers</code>	指定带颜色掩膜的层写入 Live DRC 默认层时所用的层。
<code>signoff.check_drc_live.exclude_density_rules</code>	<code>{connectivity true}</code>	指定要从检查中排除的规则类型；默认排除密度与连接性规则。
<code>signoff.check_drc_live.halo</code>	（见说明）	指定 Live DRC 检查区域在当前显示范围基础上向外延展的距离（微米）。
<code>signoff.check_drc_live.hierarchy_level</code>	<code>visible</code>	指定 Live DRC 处理的最大层级深度。
<code>signoff.check_drc_live.keep_failing_results</code>	<code>false</code>	控制是否保留检查区域之外的违例；默认只保留完全落在检查区域内的违例。设为 <code>true</code> 时保留完全落在「裁剪区域」（检查区域加上 <code>halo</code> 一半宽度）内的违例；完全落在裁剪区域之外的违例始终被丢弃。
<code>signoff.check_drc_live.run_dir</code>	<code>signoff_check_drc_live</code>	指定运行目录。
<code>signoff.check_drc_live.runset</code> （必填）	（无）	指定用于检查的代工厂 <code>runset</code> 。
<code>signoff.check_drc_live.user_defined_options</code>	（无）	追加到 IC Validator 命令行的额外参数字符串。

应用选项	默认值	说明
<code>signoff.physical.layer_map_file</code> (必填)	无	指定 layer mapping 文件。
<code>signoff.physical.merge_stream_files</code>	files	指定合并进当前 block 供交互式检查使用的 stream 文件。

8.3 自动修复签核 DRC 违例

Automatically Fixing Signoff DRC Violations

Zroute 布线引擎可以利用 `signoff_check_drc` 产生的签核 DRC 结果，自动修复其中检测到的设计规则违例，核心命令是 `signoff_fix_drc`。

整体流程为：

1. 按前述方法搭建 IC Validator 环境；
2. 设置签核 DRC 检查所需的应用选项（见前一节）；
3. （可选）生成一份 `autofix` 配置文件，用来辅助修复过程；该文件定义了「层 → DRC 规则」的映射关系，`signoff_fix_drc` 只处理该文件中列出的规则。默认情况下命令会自动生成该配置；也可以手工生成，见下文「生成 Autofix 配置文件」；
4. 设置签核 DRC 修复所需的应用选项；
5. 用 `save_block` 保存 block 到磁盘——`signoff_fix_drc` 同样使用磁盘上的版图数据；
6. 运行 `signoff_fix_drc` 命令。

注意

若 block 采用了双重曝光（double-patterning）工艺，应先对除双重曝光规则外的其他布线规则做完整的签核 DRC 修复，再单独针对双重曝光的奇环（odd-cycle）违例做一轮修复，详见「自动修复双重曝光奇环违例」一节。

8.3.1 生成 Autofix 配置文件

Creating an Autofix Configuration File

Autofix 配置文件的每一行格式为：

```
"mask_layer_name" "full_IC_Validator_DRC_rule_comment"
```

以井号（#）开头的行为注释。

若此前已经运行过 `signoff_check_drc`，可以运行 `$ICV_HOME_DIR/contrib/generate_layer_rule_map.pl` 脚本自动生成配置文件，其语法为：

```
generate_layer_rule_map.pl
  -dplog IC_Validator_log_file
  -tech_file technology_file
  -o config_file_name
```

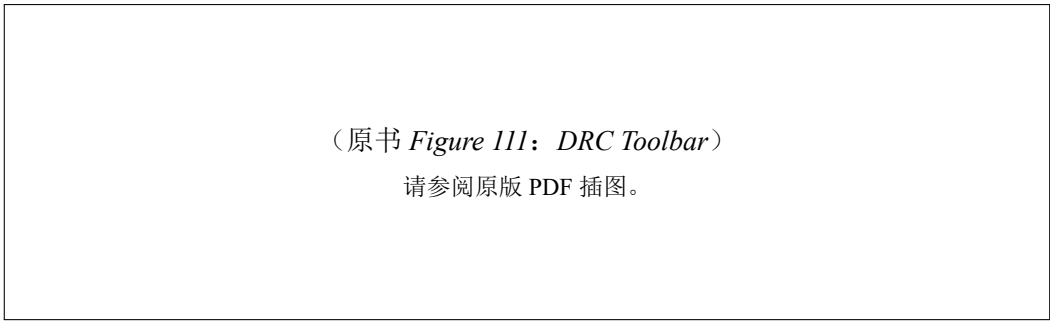


图 8.4: DRC 工具条

其中 `-dplog` 指定的日志文件是上一次 `signoff_check_drc` 运行生成的 `runset.dp.log`，位于签核 DRC 运行目录（`signoff.check_drc.run_dir` 指定，默认 `signoff_drc_run`）下的 `run_details` 子目录中。

生成配置文件后，用应用选项 `signoff.fix_drc.config_file` 指定其位置。

8.3.2 设置签核 DRC 修复的应用选项

Setting Options for Signoff DRC Fixing

运行 `signoff_fix_drc` 之前，可通过表 8.4 所列应用选项配置本次运行。

表 8.4: 签核 DRC 修复的应用选项

应用选项	默认值	说明
<code>signoff.fix_drc.advanced_guidance_for_rules</code>	<code>all</code>	控制是否使用高级布线引导；默认对所有规则启用（ <code>all</code> ），设为 <code>off</code> 可关闭该特性。
<code>signoff.fix_drc.check_drc</code>	<code>local</code>	控制修复后的 DRC 检查范围： <code>local</code> （默认，只检查受本次修复影响的区域）或 <code>global</code> （检查整个 <code>block</code> ）。
<code>signoff.fix_drc.custom_guidance</code>	无	设为 <code>dpt</code> 时，本次修复专门针对双重曝光奇环违例，详见后文。
<code>signoff.fix_drc.fix_detail_route_drc</code>	<code>global</code>	控制修复布线类 DRC 违例的范围： <code>global</code> （整个 <code>block</code> ）或 <code>local</code> （只在违例附近）。
<code>signoff.fix_drc.init_drc_error_dir</code>	无	指定复用已有签核 DRC 结果的目录；不设置时命令会先自行运行一次 <code>signoff_check_drc</code> 生成初始结果。
<code>signoff.fix_drc.last_run_fullchip</code>	<code>false</code>	控制最后一轮修复循环中的 DRC 检查是否覆盖整个 <code>block</code> （ <code>true</code> ）还是只覆盖受影响区域（ <code>false</code> ，默认）。
<code>signoff.fix_drc.max_detail_route_iterations</code>	5	签核 DRC 修复后进行搜索修复（ <code>search-and-repair</code> ）迭代的最大次数。
<code>signoff.fix_drc.max_errors_per_rule</code>	1000	每条规则最多尝试修复的违例数。
<code>signoff.fix_drc.run_dir</code>	<code>signoff_fix_drc_run</code>	运行目录。

应用选项	默认值	说明
<code>signoff.fix_drc.target_clock_networks</code>	<code>false</code>	控制修复面向数据网络（ <code>false</code> ，默认）还是时钟网络（ <code>true</code> ）。
<code>signoff.fix_drc.user_defined_options</code>	<code>options</code>	追加到 IC Validator 命令行的额外参数字符串。

8.3.3 运行 `signoff_fix_drc` 命令

Running the `signoff_fix_drc` Command

默认情况下，`signoff_fix_drc` 执行以下步骤：

1. 运行 `signoff_check_drc` 生成初始的签核 DRC 结果。若已有现成结果，可通过 `signoff.fix_drc.init_drc_error_db` 指定其所在目录（相对路径或绝对路径均可，相对路径以当前工作目录为基准），从而跳过重复检查。例如复用默认签核 DRC 运行目录的结果：

```
fc_shell> set_app_options \  
-name signoff.fix_drc.init_drc_error_db \  
-value signoff_check_drc_run  
fc_shell> signoff_fix_drc
```

注意

若要复用已有的签核 DRC 结果作为输入，运行 `signoff_check_drc` 时必须已按「生成自动修复流程输入」中所述设置了相应的应用选项。

2. 运行两轮修复循环（repair loop）来修复 DRC 违例。每一轮循环包含：
 - (a) 尝试修复上一次 IC Validator 签核 DRC 检查出的违例；
 - (b) 对修改后的 block 重新运行一次签核 DRC 检查。可用 `-max_number_repair_loop`（取值 1–10）调整循环次数；若某一轮结束后已无剩余违例，工具不会再继续执行后续循环。随着循环轮次增加，该轮修复所涉及的重布线范围也会相应扩大；可用 `-start_repair_loop` 提高起始循环序号来扩大范围，但这可能对 block 造成更大扰动，进而需要 Zroute 做更多额外的检查与修复。
3. 写出结果汇总报告，详见下文「自动修复设计规则汇总报告」。

修复 DRC 违例

Fixing DRC Violations

在每一轮修复循环中，`signoff_fix_drc` 尝试修复上一次签核 DRC 检查出的违例：

- 第一轮循环使用 `signoff.fix_drc.init_drc_error_db` 指定目录（未设置时为 `signoff_drc_run_init`）中的 IC Validator 数据；
- 后续循环使用上一轮循环产生、存放在 `signoff.fix_drc.run_dir` 指定目录（未设置时为 `signoff_fix_drc_run`）中的数据。

修复阶段的默认行为:

- 尝试修复上一次检查中的所有违例, 但违例数超过 1000 条的规则会被跳过;
- 默认在整个 `block` 范围内对数据网络做修复。可用 `-coordinates` (限定区域) 和/或 `-excluded_coordinates` (排除区域) 调整修复范围, 坐标格式同前;
- 可用 `-nets` 明确指定不应受影响的时序关键网络, 或用 `-timing_preserve_setup_slack_threshold` 让工具依据建立时间裕量阈值自动判定关键网络 (单位为库时间单位), 以避免修复动作破坏时序;
- 修复完成后执行 5 轮详细布线迭代以消除修复过程中新引入的 DRC 违例;
- 修改后的 `block` 保存为名为 `block_ADR_#` 的 `design view`。

可通过以下应用选项调整默认行为:

- `signoff.fix_drc.max_errors_per_rule`: 设定忽略某规则所需的违例数阈值;
- `signoff.fix_drc.fix_detail_route_drc`: 设为 `local` 可将修复范围限制在违例附近, 缩短运行时间;
- `signoff.fix_drc.target_clock_nets`: 设为 `true` 可让修复面向时钟网络而非数据网络;
- `signoff.fix_drc.max_detail_route_iterations`: 设定详细布线迭代次数 (0-1000)。

检查 DRC 违例

Checking for DRC Violations

修复循环中执行 DRC 检查时, `signoff_fix_drc` 的默认行为为:

- 对顶层 `block` 和标准单元实例读取 `design view`, 对宏单元/I-O pad 实例只读取 `frame view` 的 `pin` 信息 (具体调整方法见前文「读取 Block 的规则」);
- 初始检查之后, 仅对受本轮修复影响的区域做检查。若要在最后一轮循环中对整个 `block` 做检查, 将 `signoff.fix_drc.last_run_full_chip` 设为 `true` (可提升准确性但增加运行时间); 若要让所有循环都对整个 `block` 做检查, 将 `signoff.fix_drc.check_drc` 设为 `global` (准确性更高但运行时间可能显著增加);
- 不检查子 `block` 内部的违例;
- 排除不适合自动修复的规则: 如需明确指定要检查的规则, 可使用 `-select_rules / -unselect_rules`;
- 结果存放于 `signoff_fix_drc_run` 目录 (可用 `signoff.fix_drc.run_dir` 更改)。

注意

初始签核 DRC 运行的结果单独存放于 `signoff_drc_run_init` 目录中。

此外, 可通过 `signoff.fix_drc.user_defined_options` 向 IC Validator 命令行追加其他参数。

参见

执行签核级设计规则检查, 见「执行签核级设计规则检查」一节。

8.3.4 自动修复双重曝光奇环违例

Automatically Fixing Double-Patterning Odd-Cycle Violations

若 block 采用双重曝光工艺，非双重曝光布线规则与双重曝光布线规则必须分开进行签核 DRC 修复。步骤如下：

1. 先按「自动修复签核 DRC 违例」中描述的流程，修复非双重曝光相关的违例。运行 `signoff_check_drc` 与 `signoff_fix_drc` 时，用 `-unselect_rules` 排除双重曝光规则：

```
fc_shell> signoff_check_drc -unselect_rules {list_of_dpt_rules}
```

注意

具体的双重曝光规则名称需查阅代工厂提供的设计规则手册（DRM）。

2. 再针对双重曝光奇环违例做一轮专门的修复。运行 `signoff_fix_drc` 之前，把 `signoff.fix_drc.custom_guidance` 设为 `dpt`：

```
fc_shell> set_app_options \  
-name signoff.fix_drc.custom_guidance -value dpt
```

运行 `signoff_check_drc` 与 `signoff_fix_drc` 时，改用 `-select_rules` 只保留双重曝光规则：

```
fc_shell> signoff_check_drc -select_rules {list_of_dpt_rules}
```

3. 按「执行签核级设计规则检查」中描述的流程，重新做一次签核 DRC 检查以验证修复结果。

8.3.5 自动修复设计规则汇总报告

Summary Report for Automatic Design Rule Fixing

`signoff_fix_drc` 会在当前工作目录写出汇总报告文件 `result_summary.rpt`。下面是该报告的一个示例（数值仅作格式演示）：

```
Input DRC Error Database..... ./inputs/signoff_check_drc_run  
Limit to Cell..... my_design  
Maximum Errors/Command..... 1000  
  
SIGNOFF AUTOFIX DRC SUMMARY:  
DRC NAME      : TOTAL : PROCESSED : TARGETED : REMAINING : FIX RATE : COMMENT  
M0.W.3        :    1 :         1 :         0 :         0 :    --    : 1 ignored <#1>  
M4.L.3        :    4 :         4 :         0 :         0 :    --    : 4 ignored <#2>  
VIA1.S.10.3   :   10 :        10 :         2 :         0 : 100.0%   : 8 ignored <#2>  
TOTAL         :    :         :         2 :         0 : 100.0%   :  
  
NOTE:  
#1 : size of error shapes greater than threshold  
#2 : shapes associated with error are not Zroute modifiable
```

报告各列含义见表 8.5。

表 8.5: result_summary.rpt 各列说明

列名	说明
DRC NAME	runset 中该设计规则的名称。
TOTAL DRC	signoff_check_drc 检测到的该规则违例总数。
PROCESSED DRC	经过筛选流程、被判断是否可修复的违例数。
TARGETED DRC	筛选后自动修复实际尝试修复的违例数。
REMAINING TARGETED DRC	自动修复完成后仍然剩余的违例数。
FIX RATE	剩余目标违例数占目标违例总数的比例。
COMMENT	被忽略违例的原因说明；详细原因代码见报告末尾的 NOTE 部分。

8.4 通过电源网增强改善实例电压降

Improving Instance Voltage Drop by Augmenting the Power Grid

电源网增强（power grid augmentation, PG augmentation）是一种改善实例电压降（instance voltage drop）的手段：在电源/地网络上添加额外的金属图形，这些图形相当于并联电阻，从而降低标准单元所「看到」的电源网等效电阻。

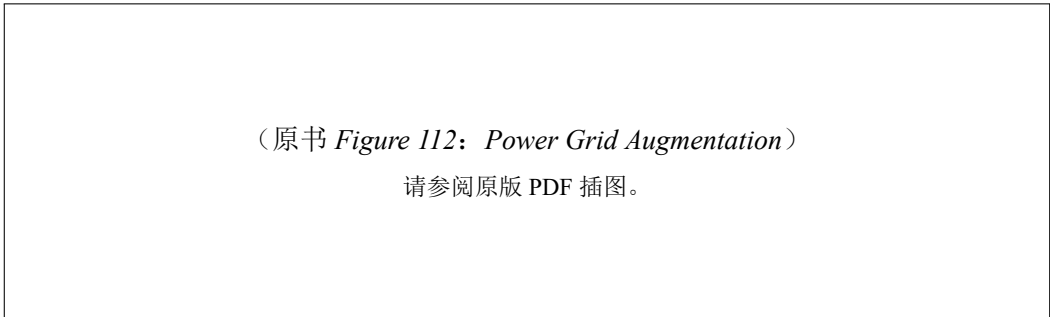


图 8.5: 电源网增强示意

Fusion Compiler 支持三种电源网增强方式：

- 标准电源网增强（**standard**）：在可行范围内尽可能多地插入增强图形，以最大程度降低实例电压降；
- 时序驱动电源网增强（**timing-driven**）：在指定区域插入增强图形，但会主动避开时序关键网络周边；
- 引导式电源网增强（**guided**）：只针对存在 IR 违例的单元，沿其到最小路径电阻 tap 点的路径插入增强图形。

三种方式都依赖 RedHawk Fusion 的分析结果来指导增强位置的选择，并最终由 IC Validator 完成增强图形的插入。

参见

RedHawk 与 RedHawk-SC Fusion 分析，另见「RedHawk and RedHawk-SC Fusion」一节。

8.4.1 标准电源网增强

Standard Power Grid Augmentation

标准电源网增强会尽可能多地插入增强图形以降低实例电压降。执行步骤：

1. 确认 block 已完成布线，且几乎没有（或完全没有）DRC 违例；
2. 用 RedHawk Fusion 运行一次电压降分析（增强前基线），其结果将用于指导本次增强；
3. 按前述方法搭建 IC Validator 环境；
4. （可选）用 `set_host_options` 启用分布式/多核处理；
5. 设置电源网增强所需的应用选项（见下文表 8.6）。至少必须指定：

- `signoff.create_pg_augmentation.power_net_name`: 电源网名称；
- `signoff.create_pg_augmentation.ground_net_name`: 地网名称。

若设计中存在多个电源网，需针对每个电源网各运行一次。

6. 用 `save_block` 保存 block 到磁盘；
7. 运行 `signoff_create_pg_augmentation` 命令执行增强，必须用 `-node` 指定所用的工艺节点；若因存在多个电源网而需要做增量式增强，后续运行时加上 `-mode add` 以在已有增强图形基础上追加，而非重新生成；
8. 再次运行 RedHawk Fusion 电压降分析，测量增强后的效果。

标准电源网增强运行时，`signoff_create_pg_augmentation` 会：

1. 在指定区域插入增强图形。插入过程中会遵循布线设计规则以避免产生新的 DRC 违例。默认对整个 block 做增强；如需限定范围，可组合使用以下选项：
 - `-coordinates_ground / -excluded_coordinates_ground`: 限定/排除地网增强的区域；
 - `-coordinates_power / -excluded_coordinates_power`: 限定/排除电源网增强的区域（若限定与排除区域重叠，则不在重叠部分增强）。

增强插入的图形带有 `shape_use` 属性值 `pg_augmentation`，可据此查询，例如：

```
fc_shell> get_shapes -filter {shape_use=~pg_augmentation}
```

2. 生成汇总报告，列出每个电源/地网在各层新增图形的分布，以及新增金属图形数、via 图形数与总图形数；
3. 将更新后的 block 保存到磁盘。

参见

另见「时序驱动电源网增强」「引导式电源网增强」「移除电源网增强图形」各节。

8.4.2 设置电源网增强的应用选项

Setting Options for Power Grid Augmentation

表 8.6: 电源网增强的应用选项

应用选项	默认值	说明
<code>signoff.create_pg_augmentation.(<u>否</u>)ground_net_name</code> (必填)		地网名称。
<code>signoff.create_pg_augmentation.(<u>否</u>)power_net_name</code> (必填)		电源网名称。
<code>signoff.create_pg_augmentations.(<u>否</u>)signoff.create_pg_augmentation.run_dir</code>		运行目录: 存放本次运行产生的文件。
<code>signoff.create_pg_augmentation.(<u>无</u>)user_defined_options</code>		追加到 IC Validator 命令行的额外参数字符串。

8.4.3 时序驱动电源网增强

Timing-Driven Power Grid Augmentation

时序驱动电源网增强在指定区域插入增强图形，但会绕开时序关键网络周边。该模式默认不开启。执行步骤与标准电源网增强大体相同，区别主要在于第 7 步：运行 `signoff_create_pg_augmentation` 时，需要额外指定以下选项之一或两者：

- `-nets`：显式指定关键网络；
- `-timing_preserve_setup_slack_threshold`：让工具依据建立时间裕量阈值自动判定关键网络（单位为库时间单位）。

同样需要用 `-node` 指定工艺节点；若涉及多电源网的增量增强，后续运行同样使用 `-mode add`。
时序驱动增强运行时，命令会：

1. 执行时序分析（含多角多模分析），以最小化对时序的影响；
2. 根据所给选项识别时序关键网络；
3. 移除已有的增强图形——默认移除整个 `block` 范围内的增强图形，若使用 `-mode add` 则保留已有图形；
4. 在指定区域插入增强图形，但避开关键网络周边；
5. 保存更新后的 `block`；
6. 生成汇总报告。

参见

另见「标准电源网增强」「引导式电源网增强」「移除电源网增强图形」各节。

8.4.4 引导式电源网增强

Guided Power Grid Augmentation

引导式电源网增强只针对存在 IR 违例的单元进行增强：对每个候选单元，沿其到最小路径电阻 tap 点之间的路径插入增强图形。由于只在特定局部区域添加图形，引导式增强所用的金属图形更少、占用的布线资源更小，因而对耦合电容的影响也更低。

执行步骤：

1. 确认 block 已完成布线，且几乎没有（或完全没有）DRC 违例；
2. 用 RedHawk Fusion 运行电压降分析与最小路径电阻分析，其结果将用于指导本次增强；
3. 按前述方法搭建 IC Validator 环境；
4. （可选）启用分布式/多核处理；
5. 用 `save_block` 保存 block 到磁盘——本命令同样依赖磁盘上的版图数据；
6. 运行 `fix_pg_wire` 命令执行引导式增强，用以下选项之一或两者指定处理范围：
 - `-number_pins`：指定按 IR 违例严重程度排序、需要处理的最差实例数量；
 - `-voltage_drop_threshold`：让工具依据电压降阈值自动判定需要处理的单元。

此外还必须指定：

- `-supply_nets`：需要增强的电源/地网名称；
- `-node`：工艺节点。

更多细节请参阅 `fix_pg_wire` 的 man page。

7. 再次运行 RedHawk Fusion 电压降分析，测量增强后的效果。

参见

另见「标准电源网增强」「时序驱动电源网增强」「移除电源网增强图形」各节。

8.4.5 移除电源网增强图形

Removing PG Augmentation Shapes

如需移除电源网增强图形，对 `signoff_create_pg_augmentation` 使用 `-mode remove` 选项，工具会移除当前 block 中的全部增强图形。

ECO 感知的增强图形自动移除

ECO-Aware Removal of PG Augmentation Shapes

若 ECO 之后某些电源网增强图形导致了 DRC 违例，可以用 `-mode remove_by_drc_auto` 选项自动移除这些违规图形（包括图形移除后产生的悬空连接）：

```
fc_shell> signoff_create_pg_augmentation -node <nodename> \
    -mode remove_by_drc_auto
```

8.5 使用 IC Validator In-Design 插入 Metal Fill

Inserting Metal Fill With IC Validator In-Design

完成布线之后，为满足大多数制造工艺要求的金属密度规则，通常需要在版图空白区域插入 fill 图形（metal fill / via fill）。在插入 metal fill 与 via fill 之前，block 应当已经接近满足时序要求，且几乎没有（或完全没有）DRC 违例。

插入步骤：

1. 按前述方法搭建 IC Validator 环境；
2. （可选）启用分布式/多核处理；
3. 设置 metal fill 插入所需的应用选项（见下文表 8.7）；
4. 用 save_block 保存 block 到磁盘——命令使用磁盘上的版图数据；
5. 运行 signoff_create_metal_fill 命令执行插入。

注意

signoff_create_metal_fill 会遵循 create_routing_blockage 定义的布线 blockage、不在其中插入 fill，但不会遵循 create_routing_guide 定义的布线引导线。若采用 pattern-based 方式插入且技术文件与 runset 文件的 blockage 层号不一致，需提供 layer mapping 文件。

插入过程中，IC Validator 会在 block 的 design view 内建立一个名为 block.FILL 的内部子设计，并把生成的 fill 单元（命名为 FILL_INST_#）放入其中；可用 get_fill_cells 命令查询这些单元。

插入完成后，还可以：

- 在 GUI 版图窗口中显示已插入的 metal fill；
- 修改已插入的 metal fill；
- 对含真实 metal fill 的版图执行寄生参数提取，用于时序分析。

8.5.1 设置签核 Metal Fill 插入的应用选项

Setting Options for Signoff Metal Fill Insertion

表 8.7: 签核 Metal Fill 插入的应用选项（节选常用项）

应用选项	默认值	说明
适用于所有插入方式的选项		
signoff.create_metal_fill.apply_nondefault_rules	false	是否在插入时遵循 nondefault 布线规则。
signoff.create_metal_fill.auto_eco_threshold_value	20	使用 -auto_eco true 做增量插入时允许的最大改动比例。
signoff.create_metal_fill.flat	false	强制使用 flat 模式（true）而非 runset 中指定的模式（false）。

应用选项	默认值	说明
signoff.create_metal_fill.read_design_views	{design_views}	指定读取 design view 而非 frame view 的参考单元。
signoff.create_metal_fill.read_layout_views	{layout_views}	指定读取 layout view 而非 frame view 的参考单元。
signoff.create_metal_fill.run_signoff_fill_run	{signoff_fill_run}	运行目录。
signoff.create_metal_fill.user_defined_options	(无)	追加到 IC Validator 命令行的额外参数字符串。
signoff.physical.merge_exclude_libraries	{libraries}	排除参与 stream 数据替换的单元库。
signoff.physical.merge_stream_files	{files}	合并进当前 block 的 stream 文件。
仅适用于 <i>pattern-based</i> 插入的选项		
signoff.create_metal_fill.runset	NotA	pattern-based 插入所用的代工厂 runset (track-based 插入不使用该选项)。
signoff.physical.layer_map_file	(无)	layer mapping 文件 (技术文件层号与 runset 层号不一致时需要)。
仅适用于 <i>track-based</i> 插入的选项		
signoff.create_metal_fill.max_density_threshold	(无)	确定各层的最大密度阈值，格式为 {layer_name max_density}; 仅用于 track-based 插入，且在 -mode add 下不生效。
仅适用于时序驱动插入 (<i>pattern-based</i> 或 <i>track-based</i>) 的选项		
signoff.create_metal_fill.fill_lower_net_on_adjacent_layer	false	是否在相邻层上、位于网络竖直延展的最小间距范围内插入 fill。
signoff.create_metal_fill.fill_shielded_clock	false	是否为被屏蔽的时钟网络插入 fill。
signoff.create_metal_fill.fix_density_errors	false	是否在插入过程中自动修复密度违例。
signoff.create_metal_fill.space_to_clock_nets	技术文件最小间距的两倍	同层 fill 与时钟网络之间的最小间距。
signoff.create_metal_fill.space_to_key_nets	技术文件最小间距的两倍	同层 fill 与时序关键网络之间的最小间距。
signoff.create_metal_fill.space_to_key_nets_on_adjacent_layer	技术文件最小间距的两倍	相邻层 fill 与时序关键网络之间的最小间距。

8.5.2 执行 Metal Fill 插入

Performing Metal Fill Insertion

signoff_create_metal_fill 可以完成多种插入模式，下面分别介绍：pattern-based 插入、track-based 插入、使用 IC Validator 参数文件定制细节、典型关键尺寸（TCD）结构插入、时序驱动插入，以及增量式插入。

基于 Pattern 的 Metal Fill 插入

Pattern-Based Metal Fill Insertion

Pattern-based 插入是 `signoff_create_metal_fill` 的默认模式，依据代工厂 `runset` 插入金属与 via fill。运行前必须通过 `signoff.create_metal_fill.runset` 指定所用 `runset`。

该模式下命令依次执行：

1. 按前文「读取 Block 的规则」相同的原则把 block 加载进 IC Validator；
2. 移除已有的 metal/via fill——默认移除全 block 范围内的既有 fill（增量式插入见后文）；
3. 在空白区域插入 metal/via fill：默认使用技术文件定义的间距规则，对整个 block 的全部金属/via 层做插入，采用的 hierarchical/flat 模式由 `runset` 文件指定。可通过以下方式调整默认行为：
 - 应用选项 `signoff.create_metal_fill.apply_nondefault_rules`（设为 true 可额外启用 nondefault 间距规则）、`signoff.create_metal_fill.flat`（设为 true 强制 flat 模式）；
 - 命令选项 `-select_layers / -all_runset_layers`（限定层，见下文）、`-coordinates / -excluded_coordinates`（限定区域，见下文）。
4. 从指定 via 层及相关金属层对应的 via 层中，移除「悬空 via fill」（floating via fill，即上下未同时被金属包裹的 via fill）；
5. 将 fill 数据保存进 design view；
6. 把结果文件写入运行目录（见「签核 Metal Fill 结果文件」）。

参见

另见「时序驱动 Metal Fill 插入」一节。

读取 Block 的规则（Metal Fill）

Reading Blocks for Signoff Metal Fill Insertion

规则与「读取 Block 的规则（签核 DRC）」基本一致：默认对顶层与标准单元读 design view，对宏单元/pad 只读 frame view 的 pin 信息；可分别通过 `signoff.physical.merge_stream_files`、`signoff.create_metal_fill.read_layout_views`、`signoff.create_metal_fill.read_design_views` 指定改用 stream 数据、layout view 或 design view，优先级顺序也与签核 DRC 一致（stream 数据 > layout view > design view > frame view）。

指定 Metal Fill 插入的层

Specifying the Layers for Metal Fill Insertion

默认在所有金属布线层插入 metal fill、在所有 via 层插入 via fill。可通过以下选项之一调整：

- `-select_layers`：将插入限定在指定的金属/via 层集合；
- `-all_runset_layers true`：对 `runset` 中定义的所有 fill 层插入（仅适用于 pattern-based 插入）。

指定层之后，命令会：先移除指定层（增量插入时）或全部层（标准插入时）已有的 fill；只在指定层插入新的 fill；并从指定 via 层及与所指金属层相关联的 via 层中移除悬空 via fill。例如指定 `-select_layers {M2}` 时，V1、V2 层上的悬空 via fill 也会被一并移除。

指定 **Metal Fill** 插入的区域

Specifying the Regions for Metal Fill Insertion

默认对整个芯片插入 metal/via fill。可通过以下选项调整：

- `-coordinates`：限定插入区域；
- `-excluded_coordinates`：排除插入区域（若与 `-coordinates` 同时使用，重叠区域不插入）。

坐标格式同前：`{{x1 y1} {x2 y2} ... {xn yn}}`。

注意

传给 IC Validator 的 `METAL_FILL_SELECT_WINDOW` 边界框坐标会自动向外扩大 1 微米，以避免在指定区域边界处产生 DRC 违例；实际插入仍限定在 `-coordinates` 指定的区域内。另外，使用 `-coordinates` 选项时，命令总是采用 flat fill 模式。

指定区域后，命令会：移除该区域（或全部区域，取决于标准/增量模式）已有的 fill；只在指定区域插入新 fill；并移除该区域内的悬空 via fill。例如，移除全部 fill 后只在坐标 (100,150)–(300,200) 的矩形之外重新填充：

```
fc_shell> signoff_create_metal_fill \
    -excluded_coordinates {{100 150} {300 200}}
```

基于 **Track** 的 **Metal Fill** 插入

Track-Based Metal Fill Insertion

Track-based 插入依据技术文件中的属性与规则自动派生出一份 runset，生成与布线 track 对齐的 fill 图形，不使用 `signoff.create_metal_fill.runset` 指定的 runset。与 pattern-based 相比，track-based 方式通常能获得更高的密度、更好的密度控制能力，以及在双重曝光层上更均衡的掩膜分布。

要启用 track-based 插入，将 `signoff_create_metal_fill` 的 `-track_fill` 选项设为非 off 的值。支持三种密度模式：

- **Sparse**（默认）：在信号图形与 fill 图形之间跳过一条 track；
- **Dense**（用 `-fill_all_tracks true` 启用）：不跳过任何 track；

注意

若 block 采用双重曝光工艺且尚未预着色（precolored），启用该模式会增加运行时间。

- **Mixed**：部分层用 sparse、部分层用 dense，需要在 IC Validator 参数文件中分别设置相应参数（见下文）。

Track-based 插入的执行步骤与 pattern-based 类似（加载 block、移除已有 fill、插入新 fill、移除悬空 via fill），主要差异在插入细节：

- 默认依据技术文件的设计规则沿 track 插入，fill 图形长度为 5 微米，宽度等于该层技术文件中定义的 defaultWidth 属性；
- 可用 signoff.create_metal_fill.apply_nondefault_rules 启用 nondefault 间距规则；
- 可用 -track_fill foundry_node 启用特定代工厂的设计规则关键字（可用 signoff_create_metal_fill -track_fill list 查看支持的关键字列表）；
- 可用 -select_layers、-coordinates/-excluded_coordinates 限定层与区域；
- 可在 IC Validator 参数文件中设置 mx_fill_width、mx_min_fill_length、mx_max_fill_length 等参数调整 fill 图形尺寸（见下文「使用 IC Validator 参数文件」）。

若设置了 signoff.create_metal_fill.max_density_threshold，命令还会（可选地）对指定层的 fill 做修剪，以尽量满足最大密度阈值；未设置该选项时不做修剪。

保存 fill 数据时的默认行为：默认把 fill 图形的 data type 设为 0（可用参数文件中的 mx_fill_datatype、viax_fill_datatype 更改）；默认不为 fill 图形设置掩膜约束（若 block 采用双重曝光工艺，可用 -output_colored_fill true 让工具为 fill 图形分配掩膜约束，此时 mask_one 对应 data type 235、mask_two 对应 236，也可通过参数文件中的 mx_fill_datatype_color1/2、viax_fill_datatype_color1/2 自定义）；默认不写出 exclude layer（若代工厂需要，可在参数文件中设置 mx_exclude_layer_datatype）。

结果文件方面，除标准输出文件外，track-based 插入还可用 -report_density 选项输出详细的密度与密度梯度报告（文件名为 prefix_color_balance_and_density_report.txt 与 prefix_fill_density_gradient_report.txt，其中 prefix 可取 on（默认前缀 track_fill）或自定义字符串）：

```
fc_shell> signoff_create_metal_fill -track_fill true \
-report_density my_prefix
```

参见

另见「时序驱动 Metal Fill 插入」一节。

使用 IC Validator 参数文件

Using an IC Validator Parameter File

可以通过 IC Validator 参数文件对 metal fill 插入做更细粒度的定制：

- pattern-based 插入：通过 signoff.create_metal_fill.user_defined_options 把参数文件传给 IC Validator：

```
fc_shell> set_app_options \
-name signoff.create_metal_fill.user_defined_options \
-value {-D INDESIGN_USER_DEFINED_PARAM_FILE=file_name}
```

- track-based 插入：通过 signoff_create_metal_fill 的 -track_fill_parameter_file 选项传入：

```
fc_shell> signoff_create_metal_fill -track_fill true \
-track_fill_parameter_file file_name
```

运行 track-based 插入时，工具会在运行目录中写出一份名为 `track_fill_params.rh` 的参数文件，其中包含各支持参数的默认取值，可据此修改后用于后续运行。

注意

某些行为既可以由 IC Validator 参数控制，也可以由应用选项控制；若两者同时设置，以 IC Validator 参数为准。

表 8.8 列出参数文件中常用的参数（除特别说明外均只适用于 track-based 插入）。参数名中的 `mx` / `vx` 前缀代表技术文件 Layer 段中定义的掩膜名（mask name）；例如若 M11 层在技术文件中的掩膜名为 `metal12`，则相应参数写作 `m12_spare_fill`、`m12_fill_staggering` 等。

表 8.8: IC Validator Metal Fill 插入参数（节选）

参数	默认值	说明
插入模式相关		
<code>mx_sparse_fill</code>	1	是否对该层使用 <code>sparse</code> 模式（1）还是 <code>dense</code> 模式（0）；使用 <code>-fill_all_tracks true</code> 时默认值变为 0。
<code>mx_fill_staggering</code>	0.075 微米	控制 fill 图形的错位排布量。
<code>mx_ignore_route_guide</code>	0	插入时是否忽略（1）该层的布线引导线。
<code>mx_ignore_system_blockage</code>	0	插入时是否忽略（1）该层的系统 <code>blockage</code> 。
<code>mx_via_enclosure</code>	由 <code>ContactCode</code> 派生	<code>via</code> 所需的最小金属包裹量。
尺寸与间距相关		
<code>mx_fill_width</code>	<code>defaultWidth</code>	fill 图形宽度。
<code>mx_min_fill_length</code>	由 <code>minArea</code> 派生	fill 图形最小长度。
<code>mx_max_fill_length</code>	5 微米	fill 图形最大长度；减小该值可提高 via 密度但降低金属密度。
<code>mx_fill2fill_side_spacing</code>	（见技术文件）	fill 图形之间的最小间距。
<code>mx_fill2route_side_spacing</code>	（见技术文件）	fill 图形与网络图形的最小间距。
<code>mx_fill2fill_end_spacing</code>	（见技术文件）	fill 图形之间的最小端到端间距。
<code>mx_fill2Blockage_x/y</code>	（见技术文件）	fill 图形与 <code>blockage</code> 的最小间距。
<code>mx_fill2chipBoundary_x/y</code>	（见技术文件）	fill 图形与芯片边界的最小间距。
<code>vx_iso_via_distance</code>	3.0 微米	判定 via 是否「孤立」时所要求的相邻 via 最大距离。
<code>vx_min_spacing</code>	由 <code>minSpacing</code> 派生	via 之间的最小间距；改小可能引入 DRC 违例，改大会降低 via 密度。

参数	默认值	说明
vx_per_metal	1	单个 metal fill 图形中容纳的 via fill 数上限；调大可提升 via 密度，但可能连通更多 fill 图形，增加邻近信号网络的电容。
<i>Exclude 层相关</i>		
mx_exclude_layer_datatype	-1	指定 exclude 层的 data type；默认值 -1 表示不生成 exclude 层。
<i>输出模式相关</i>		
mx_compress_fill	/ 0	是否压缩（1）该层输出的 fill；使用 -mode add 时始终不压缩。
vx_compress_fill		
mx_fill_datatype	/ 0	插入 fill 图形的 data type。
vx_fill_datatype		
output_colored_fill	0	双重曝光 block 中是否为 fill 图形分配掩膜约束（使用 -output_colored_fill true 时默认为 1）。
<i>密度计算相关</i>		
mx_density_gradient_window	windowSize 或 50 微米	密度梯度检查窗口大小。
mx_max_density_window	50 微米	最大密度检查窗口大小。
mx_min_density	minDensity 或 10%	窗口内允许的最小金属占比。
mx_min_density_window	windowSize 或 50 微米	最小密度检查窗口大小。
mx_window_step_size	窗口大小的一半	密度检查在 x/y 方向的步进大小。

典型关键尺寸 Metal Fill 插入

Typical Critical Dimension Metal Fill Insertion

典型关键尺寸 (typical critical dimension, TCD) 结构由 marker 层与 fill 层图形组成, 用于提升设计良率。启用方法: 在运行 signoff_create_metal_fill 前将 signoff.create_metal_fill.tcd_fill 设为 true。默认对整个 block 的所有金属/via 层插入 TCD 结构, 可用 -select_layers、-coordinates/-excluded_coordinates 调整范围。

注意

TCD Metal Fill 插入不能与时序驱动 Metal Fill 插入同时使用。

参见

另见「移除 Metal Fill」一节。

时序驱动 Metal Fill 插入

Timing-Driven Metal Fill Insertion

时序驱动 metal fill 插入会在指定区域插入 fill，同时避开时序关键网络周边；该功能对 pattern-based 与 track-based 两种模式均适用，默认不开启。启用方式是给 `signoff_create_metal_fill` 加上以下选项之一或两者：

- `-nets`：显式指定关键网络；
- `-timing_preserve_setup_slack_threshold`：依据建立时间裕量阈值自动判定关键网络。阈值不宜设得过大，否则可能导致过多网络被判定为关键网络，反而降低金属密度并留下大片空白区域。

时序驱动插入运行时，命令会：

1. 执行时序分析（含多角多模分析）以最小化对时序的影响；
2. 依据所给选项识别时序关键网络；
3. 移除已有的 metal/via fill（增量式插入方式见前文）；
4. 在指定区域插入 fill，但避开关键网络周边——若要允许在被屏蔽的关键时钟网络上方插入，可将 `signoff.create_metal_fill.fill_shielded_clock` 设为 `true`；
5. 调用 IC Validator 完成插入。默认情况下，同层 fill 与关键网络图形之间的最小间距为技术文件最小间距的两倍（该要求也适用于关键网络在相邻层上的竖直延展），可通过下文「时序驱动 Metal Fill 插入的间距设置」中的应用选项覆盖；

注意

可以限定时序驱动 metal fill 插入的区域（见前文「指定 Metal Fill 插入的区域」），但不能限定层——命令会对所有布线层（或增量模式下的所有改动层）插入。

6. (可选)修复密度违例:时序驱动插入的间距要求有时会引发密度违例,可将 `signoff.create_metal_fill.fix_density` 设为 `true`（默认 `false`）自动修复；
7. 把 fill 数据保存为 fill 单元对象；
8. 把结果文件写入运行目录。

参见

另见「基于 Pattern 的 Metal Fill 插入」「基于 Track 的 Metal Fill 插入」两节。

时序驱动 Metal Fill 插入的间距设置

默认情况下，同层间距取技术文件最小间距的两倍，相邻层间距取技术文件最小间距本身。可通过以下应用选项覆盖：

- `signoff.create_metal_fill.space_to_nets`：同层 fill 与时序关键网络之间的最小间距；
- `signoff.create_metal_fill.space_to_clock_nets`：同层 fill 与（通过 `-nets` 指定的）用户定义时钟网络之间的最小间距；

- `signoff.create_metal_fill.space_to_nets_on_adjacent_layer`: 相邻层 fill 与时序关键网络之间的最小间距;
- `signoff.create_metal_fill.space_to_clock_nets_on_adjacent_layer`: 相邻层 fill 与用户定义时钟网络之间的最小间距。

每个选项均采用如下语法为各层分别指定间距: { {layer1 value1} ... {layern valuen} }, 取值既可以是间距的倍数 (如 4x), 也可以是以微米为单位的绝对距离。

例如, 将 M2、M3 层的同层最小间距设为最小间距的 4 倍:

```
fc_shell> set_app_options \
-name signoff.create_metal_fill.space_to_nets \
-value {{M2 4x} {M3 4x}}
```

或直接指定绝对距离 (M2 为 0.125 微米, M3 为 0.133 微米):

```
fc_shell> set_app_options \
-name signoff.create_metal_fill.space_to_nets \
-value {{M2 0.125} {M3 0.133}}
```

(原书 *Figure 113: Minimum Spacing From Metal Fill to Critical Net Shape*)

请参阅原版 PDF 插图。

图 8.6: Metal Fill 与关键网络之间的最小间距示意

定义密度设计规则

Defining the Density Design Rules

IC Validator 可以检查并修复以下三类密度相关设计规则:

- **最小密度:** 窗口内允许的最小金属占比。启用 `signoff.create_metal_fill.fix_density_errors` 后会检查该规则。默认使用技术文件 `DensityRule` 段中 `minDensity` 属性的设置; 若某层未定义该属性, 则默认取 10%; 也可在 IC Validator 参数文件中用 `mx_min_density` 覆盖。
- **密度梯度:** 相邻密度检查窗口之间允许的最大密度差异百分比。若技术文件中定义了该规则且启用了密度违例修复, 则会被检查; 使用技术文件 `DensityRule` 段中的 `maxGradientDensity` 属性, 若某层未定义则不检查该层的梯度规则。
- **最大开放面积:** 不含任何图形、也不与任何图形相交的正方形区域的最大边长。只有在 IC Validator 参数文件中显式定义时才会检查, 语法为 `mn_max_open_area_rule = value;` (n 为金属层号, value 为正方形边长, 单位微米)。

更多关于密度规则定义的信息, 请参阅 Synopsys Technology File and Routing Rules Reference Manual。

增量式 **Metal Fill** 插入

Incremental Metal Fill Insertion

若此前已用 `signoff_create_metal_fill` 完成过一次插入，可以用增量方式完成以下任务：在特定层或区域追加 fill；替换特定位置已有的 fill；只在改动区域重新做 fill。

在已有 **Fill** 基础上追加

使用 `-mode add` 选项可以不移除已有 fill 就直接插入新 fill，可配合 `-select_layers`（限定层）或 `-coordinates/-excluded_coordinates`（限定区域）控制插入位置。使用这些选项配合 `-mode add` 时，无论 `signoff.create_metal_fill.fix_density_errors` 如何设置，都不会执行密度违例修复。

参见

另见「基于 Pattern 的 Metal Fill 插入」「基于 Track 的 Metal Fill 插入」两节。

替换已有 **Fill**

使用 `-mode replace` 选项可以只对指定位置先移除再插入 fill，通常用于 ECO 之后的局部刷新，须同时指定以下选项之一或多个：`-select_layers`（限定层）、`-coordinates/-excluded_coordinates`（限定区域）、`-nets/-timing_preserve_setup_slack_threshold`（限定网络）。例如，只移除并重新插入 M1、M3 层的 fill，其余层不受影响：

```
fc_shell> signoff_create_metal_fill -mode replace \  
-select_layers {M1 M3}
```

参见

另见「基于 Pattern 的 Metal Fill 插入」「基于 Track 的 Metal Fill 插入」「时序驱动 Metal Fill 插入」三节。

只在改动区域插入

若此前已运行过 `signoff_create_metal_fill`，可用 `-auto_eco true` 将本次插入限定在自上次运行以来发生改动的区域。

注意

仅当自上次 `signoff_create_metal_fill` 运行以来 block 的改动比例低于变化阈值（默认 20%，可用 `signoff.create_metal_fill.auto_eco_threshold_value` 调整）时，才能使用该选项。

使用 `-auto_eco true` 时，工具会自动判定改动的区域与层，移除并重新插入这些位置的 fill，其余既有 fill 保持不变。默认与上一次运行所用的 block 版本比较；如需与其他 block 比较，用 `-pre_eco_design` 指定。

注意

金属层上的改动会触发相邻 via 层上 fill 的移除与重插，以避免金属图形从 ECO 区域移除后残留悬空 via；若某个单元实例发生改动，工具会保守地认为该实例涉及的所有层都发生了变化。

例如，只对自上次运行以来改动的区域移除并重新插入（pattern-based 模式）fill：

```
fc_shell> signoff_create_metal_fill -auto_eco true
```

可用 `-select_layers` 进一步限定层，但仅当所指定的层恰好也是自动检测到的改动层之一时才会生效；不能用 `-coordinates/-excluded_coordinates` 限定区域。使用 `-auto_eco true` 时：不能同时使用 `-mode`、`-all_runset_layers`、`-report_density`；不会在双重曝光层上插入 dense fill（无论 `-fill_all_tracks` 如何设置）；也不会修复密度违例（无论 `signoff.create_metal_fill.fix_density_errors` 如何设置）。

参见

另见「基于 Pattern 的 Metal Fill 插入」「基于 Track 的 Metal Fill 插入」「时序驱动 Metal Fill 插入」三节。

8.5.3 签核 Metal Fill 结果文件

Signoff Metal Fill Result Files

`signoff_create_metal_fill` 把输出文件写入运行目录（`signoff.create_metal_fill.run_dir` 指定，默认 `signoff_fill_run`）；每次运行都会在其中创建一个 `icv_run_#` 子目录存放本次运行产生的文件，主要包括：

- `block.LAYOUT_ERRORS`——检测到的错误详情；
- `block.RESULTS`——运行结果摘要；
- `signoff_create_metal_fill.log`——运行环境摘要；
- `icv_config_out`——顶层 block 与单元库路径信息；
- `layer.map`——本次运行生成的 layer mapping 文件；
- `metal_fill_params.rh`——本次运行所用的 metal fill 参数与选项；
- `metal_fill_compress_params.rh`——存储命令及所用压缩模式（flat 记作 NONE，hierarchical 记作 AUTO）；
- `./run_details` 目录——IC Validator 为本次插入产生的全部原始数据。

8.5.4 查询 Metal Fill

Querying Metal Fill

Metal fill 图形存放在命名为 `FILL_INST_#` 的 fill 单元中：

- 用 `get_fill_cells` 查询 fill 单元。默认只报告顶层设计中的 fill 单元；加上 `-hierarchical` 可报告整个层级中的全部 fill 单元；
- 用 `get_shapes -include_fill` 查询 fill 图形；加上 `-of_objects` 可只返回指定 fill 单元关联的图形。

8.5.5 在 GUI 中查看 Metal Fill

Viewing Metal Fill in the GUI

在版图窗口中查看 metal fill 的步骤：

1. 打开 View Settings 面板：将 Level 设为 1；在 Objects 页签中勾选 Fill Cell 对象与 Area Fill 金属图形的可见性（若列表中看不到 Area Fill，需展开 Route 对象下的 Shape Use 属性）；
2. 打开 Hierarchy Settings 面板：将「View level」设为 1；在「Expanding cell types」中勾选「Fill Cell」。

（原书 *Figure 114: Settings Used to Display Fill Shapes in the GUI*）

请参阅原版 PDF 插图。

图 8.7: 显示 Fill 图形所需的设置

8.5.6 报告金属密度

Reporting the Metal Density

用 `signoff_report_metal_density` 命令报告金属密度信息。计算某一金属层的密度信息时，工具依次按以下优先级确定该层的密度规则与窗口尺寸：

- 密度规则优先级：

1. `signoff.report_metal_density.user_defined_options` 所指参数文件中 `mx_min_density` 参数的设置；
2. 应用选项 `signoff.report_metal_density.min_density` 的设置；
3. 技术文件 `DensityRule` 段中 `minDensity` 属性的设置；
4. 默认值 10%。

- 窗口尺寸优先级：

1. 参数文件中 `mx_min_density_window` 或 `mx_density_gradient_window` 参数的设置；
2. 应用选项 `signoff.report_metal_density.density_window` 或 `signoff.report_metal_density.gradient` 的设置；
3. 技术文件 `DensityRule` 段中 `windowSize` 属性的设置；
4. 默认值 50 微米。

命令的其他默认行为：

- 使用 `fill` 数据做密度计算（可用 `signoff.report_metal_density.fill_view_data` 设为 `read_if_uptodate` 或 `discard` 调整是否改用较新的 design view）；

- 从芯片边界左下角开始，以窗口大小的一半为步长创建密度与梯度检查窗口（可用 `-starting_point` 更改起点，用 `signoff.report_metal_density.density_window_step` 应用选项或参数文件中的 `mx_window_step_size` 参数更改步长，二者都设置时以 IC Validator 参数为准）；
- 对整个 block 计算金属密度（可用 `-coordinates` 限定区域，此时多个区域会合并统计，而非逐一分别报告）；
- 对最小到最大布线层之间的每个金属层计算密度（可用 `-select_layers` 限定层）；
- 默认不生成 heat map（可将 `signoff.report_metal_density.create_heat_maps` 设为 `true` 以生成，供 GUI 查看）；
- 报告文件默认写入当前工作目录下的 `signoff_report_metal_density_run` 目录（可用 `signoff.report_metal_density.run_dir` 更改），报告文件默认命名为 `report_metal_density.txt` 与 `gradient_density_report.txt`（可用 `-output` 更改前缀，例如指定 `-output density.rpt` 时梯度报告会命名为 `density.rpt.gradient`）。

可通过 `signoff.report_metal_density.user_defined_options` 向 IC Validator 命令行追加其他参数。

参见

另见「使用 IC Validator 参数文件」一节。

在 GUI 中查看密度 Heat Map

Viewing Density Heat Maps in the GUI

运行 `signoff_report_metal_density` 之后，可在 GUI 中查看最小密度与梯度密度两类 heat map。

注意

只有当 `signoff.report_metal_density.create_heat_maps` 设为 `true` 时，命令才会生成 heat map。

- 查看最小密度 heat map：依次选择 View > Map > ICV Metal Fill Density。每个色块对应一个密度窗口步进大小，可通过设置 Bin range 来自定义/标准化密度取值区间。
- 查看密度梯度 heat map：依次选择 View > Map > ICV Metal Fill Gradient。每个色条对应一个梯度窗口大小。

8.5.7 移除 Metal Fill

Removing Metal Fill

移除方式取决于移除范围：

- 若要移除整个 block、特定层或特定区域的全部 metal fill，使用 `signoff_create_metal_fill -mode remove`（详见下文）；

(原书 Figure 115–116: Minimum Density Heat Map / Density Gradient Heat Map)

请参阅原版 PDF 插图。

图 8.8: 最小密度与密度梯度 Heat Map 示例

- 若只想移除个别 fill 图形（例如手工修复与 metal fill 相关的 DRC 违例时），使用 `remove_shapes` 或 `remove_fill_cells` 命令；有时也可以通过修改已有 fill 图形来解决该违例（见「修改 Metal Fill」）。

通过 IC Validator 移除 Metal Fill

Removing Metal Fill With the IC Validator Tool

默认情况下，`signoff_create_metal_fill -mode remove` 会移除当前 block 中全部的金属/via fill，包括 TCD 结构。可通过以下方式细化范围：

- `-coordinates`：只移除特定区域的 fill；
- `-select_layers`：只移除特定层的 fill——默认此时不移除 TCD 结构，若需一并移除，须先将 `signoff.create_metal_fill.tcd_fill` 设为 `true`；
- `-nets` / `-timing_preserve_setup_slack_threshold`：只移除关键网络周边的 fill（注意：判定关键网络时不考虑既有 fill，此规则适用于 pattern-based fill，不适用于 track-based fill）；
- `-remove_by_rule`：移除时遵循特定规则，可指定 `ndr`（nondefault 布线规则，适用于两种插入方式）或 `max_density_threshold`（最大密度规则，仅适用于 track-based fill，移除时遵循 `signoff.create_metal_fill.max_density_threshold` 设定的阈值）。

例如，移除时遵循 nondefault 布线规则：

```
fc_shell> signoff_create_metal_fill -mode remove \
-remove_by_rule {ndr}
```

参见

另见「指定 Metal Fill 插入的层」「指定 Metal Fill 插入的区域」两节。

ECO 感知的 Track-Based Metal Fill 自动移除

ECO-Aware Removal of Track Based Metal Fill Shapes

若 ECO 之后某些 track-based metal fill 图形导致了 DRC 违例，可用 `-remove_by_rule drc_auto` 选项自动移除这些违规图形（含由此产生的悬空连接）：

```
fc_shell> signoff_create_metal_fill -remove_by_rule drc_auto
```


8.5.8 修改 Metal Fill

Modifying Metal Fill

有时可以通过手工调整 fill 图形边界、增删 fill 图形来修复某个 DRC 违例：

- 修改 fill 图形边界：用 `set_attribute` 修改其 `bbox` 属性；
- 在顶层 fill 单元中新增 fill 图形：用 `create_shape -shape_use area_fill`；
- 在指定 fill 单元中新增 fill 图形：用 `create_shape -fill_cell`；
- 删除 fill 图形：用 `remove_shapes`。

也可以在 GUI 中直接选中并修改 fill 图形：在 View Settings 面板中选择「Multiple Levels Active」；在 Hierarchy Settings 面板中将「View level」设为至少 2，并在「Expanding cell types」中勾选「Fill Cell」。

8.5.9 执行真实 Metal Fill 提取

Performing Real Metal Fill Extraction

启用真实 metal fill 提取（用于让时序分析考虑实际插入的 fill 图形而非估计值）的步骤：

1. 用 `set_parasitic_parameters` 命令为各时序角关联非仿真（non-emulation）TLUPlus 文件，详见 Fusion Compiler Timing Analysis User Guide；
2. 用以下命令启用真实 metal fill 提取：

```
fc_shell> set_extraction_options \
    -real_metalfill_extraction floating
```

关于 Fusion Compiler 中寄生参数提取的更多信息，请参阅 Fusion Compiler Timing Analysis User Guide。

8.6 自动修复孤立 Via

Automatically Fixing Isolated Vias

孤立 via（isolated via）是指周围没有足够近的相邻 via、不满足工艺要求的 via。检测并修复孤立 via 的一般步骤：

1. 按前述方法搭建 IC Validator 环境；
2. （可选）用 `set_host_options` 启用分布式/多核处理；
3. 设置修复孤立 via 所需的应用选项。至少必须通过 `signoff.fix_isolated_via.isolated_via_max_range` 为各 via 层定义「判定为孤立 via 所允许的最大相邻距离」，语法为：

```
{ {via_layer1 distance1} ... {via_layern distancen} }
```

其中 `via_layer` 使用掩膜名（如 `via1`），`distance` 单位为微米；

4. 运行 `signoff_fix_isolated_via` 命令执行检查与修复。

8.6.1 设置修复孤立 Via 的应用选项

Setting Options for Fixing Isolated Vias

表 8.9: 签核孤立 Via 修复的应用选项

应用选项	默认值	说明
<code>signoff.fix_isolated_via.isolated_via_max_range</code> (必填)	<code>None</code>	各 via 层判定孤立 via 所允许的最大相邻距离（微米）；若在此距离内找不到相邻 via，则该 via 被判定为孤立 via，命令会尝试在该距离内插入一个用于修复的 via。
<code>signoff.fix_isolated_via.avoid_network_types</code>	<code>clock</code>	修复孤立 via 时应避开的网络类型，取值可为 <code>clock</code> 、 <code>pg</code> 、 <code>none</code> 中的一个或多个。
<code>signoff.fix_isolated_via.run_dir</code>	<code>signoff_fix_isolated_via</code>	运行目录。
<code>signoff.fix_isolated_via.user_defined_options</code>		追加到 IC Validator 命令行的额外参数字符串。

8.6.2 运行 `signoff_fix_isolated_via` 命令

Running the `signoff_fix_isolated_via` Command

该命令既可以只做检查，也可以检查并修复。

仅检查孤立 Via

Checking for Isolated Vias

加上 `-check_only true` 选项即可只检查而不修复：

```
fc_shell> signoff_fix_isolated_via -check_only true
```

以检查模式运行时，命令会生成一份汇总报告，列出各 via 层检测到的孤立 via 数量。

检查并修复孤立 Via

Checking and Fixing Isolated Vias

默认情况下，`signoff_fix_isolated_via` 同时执行检查与修复，具体步骤为：

1. 依据 `signoff.fix_isolated_via.isolated_via_max_range` 定义的范围检查孤立 via；
2. （可选）在每个检测到的孤立 via 周围指定范围内，用 `track-based metal fill` 插入方式生成 `dummy fill` 图形，作为后续插入修复 via 的「落脚点」：
 - 默认情况下，命令会忽略已有的 `fill` 数据，重新执行一次 `track-based fill` 插入来生成这些 `dummy fill`；插入时遵循技术文件定义的沿 `track` 设计规则，部分代工厂需要额

外规则时可通过设置相应的 `TRACK_FILL_FOUNDRY_name IC Validator` 变量启用（借助 `signoff.create_metal_fill.user_defined_options` 设置）：

```
fc_shell> set_app_options \
  -name signoff.create_metal_fill.user_defined_options \
  -value {-D TRACK_FILL_FOUNDRY_name}
```

- 若此前已完成过 track-based metal fill 插入，可用 `-update_track_fill true` 复用已有的 fill 图形来修复孤立 via；此时须用 `-track_fill_runset_include_file` 指定初次插入所用的参数文件（即前次运行目录下的 `track_fill_params.rh`）。例如初次插入的运行目录为 `init_fill`：

```
fc_shell> signoff_fix_isolated_via -update_track_fill true \
  -track_fill_runset_include_file init_fill/track_fill_params.rh
```

3. 在指定范围内按以下优先级插入修复 via：（a）在已有的非宽（non-wide）网络图形与 fill 图形之间插入 via；（b）延伸已有非宽网络图形的线端后再插入 via；（c）在宽金属图形与 fill 图形之间插入 via。插入时会遵循布线规则（含双重曝光规则与 nondefault 间距规则），以避免引入新的 DRC 违例；

注意

若 block 布线过于拥挤或过于稀疏，命令可能无法修复全部孤立 via。

4. 移除未被使用的 dummy fill 图形；
- 5.（可选）将更新后的 block 保存到磁盘。默认不自动保存，需要之后手工用 `save_block`；若要在运行结束时自动保存，加上 `-save_design true` 选项；
6. 保存结果到磁盘，包括：
 - 汇总报告：列出修复前后各 via 层的孤立 via 数量；
 - 错误数据文件：默认名为 `signoff_fix_isolated_via.err`（可用 `-error_data` 更改文件名），可在 error browser 中查看，具体见 Fusion Compiler Graphical User Interface User Guide。

Chapter 9

使用 Custom Router 布线

Routing Using Custom Router

注意

本章为学习笔记式整理译文，按原书（V-2023.12，约 p.631–665）第 8 章的目录结构编写，覆盖全部小节的核心概念、命令与选项，但对原书大段表格文字与示例脚本做了归纳、精简与改写，不逐句照搬原文，请以官方 PDF 为准。

Custom Router 是一款支持有网格（gridded）与无网格（gridless）布线的基于形状（shape-based）的布线器，适用于定制数字、混合信号与模拟电路的布线需求。在 Fusion Compiler 环境中，可用 Custom Router 为 block 之间的关键信号创建互连（route），再交由 Fusion Compiler 完成后续物理实现；也可用一组自定义布线约束对关键信号或时钟网做预布线（prerouting）。

Custom Router 在 Fusion Compiler 环境中提供以下能力：

- 批处理模式下的预布线；
- 基于 Tcl 的布线约束管理；
- 依据自定义布线约束的自动布线；
- 用于预布线的混合（hybrid）流程；
- DDR（Double Data Rate）网络布线流程。

Custom Router 与 Fusion Compiler 完全集成，支持 20 nm 及以下先进工艺的设计规则。本章依次介绍：Custom Router 使用前提、布线约束的定义、约束组管理、常用应用选项、route_custom 布线、网络屏蔽、结果检查，以及混合流程与 DDR 流程。更详细的内容请参阅 *Custom Compiler Custom Router User Guide*。

9.1 在 Fusion Compiler 工具中使用 Custom Router

Using Custom Router in the Fusion Compiler Tool

Fusion Compiler 提供一组 Tcl 命令，用于在其环境中运行 Custom Router。这些命令用于定义布线约束：在 block 级别为一条或多条网络添加走线，或添加差分对布线、总线布线、屏蔽等布线特性。

此外还提供一组应用选项（application option），用于全局控制自定义布线的行为。关于约束与应用选项的具体定义方式，见「定义布线约束」与「使用 Custom Router 应用选项」。

`route_custom` 命令会结合已定义的布线约束，以及工艺文件（technology file）与设计本身的信息，调用 Custom Router 执行自动布线，详见「使用 Custom Router 布线」。

Custom Router 支持的约束类型包括：

- 网络屏蔽（net shielding）与差分对（differential pair）；
- 长度匹配（matched length）；
- 可变宽度与间距（variable width and space）；
- Pin 宽度匹配与渐变（tapering）。

Custom Router 支持的 Fusion Compiler 非默认布线规则（nondefault routing rule）包括：布线层与 via、布线宽度与间距、布线网格（routing grid）、组内/组间间距、taper halo、布线阻挡（blockage）、布线走廊（routing corridor）以及屏蔽（shielding）。

图 9.1 给出 Custom Router 的基本流程：在已完成电源地网布线、但时钟树尚未布线的 block 上，结合工艺文件中的设计规则定义布线约束，执行自定义布线，再进行后布线优化，最后分析布线结果。

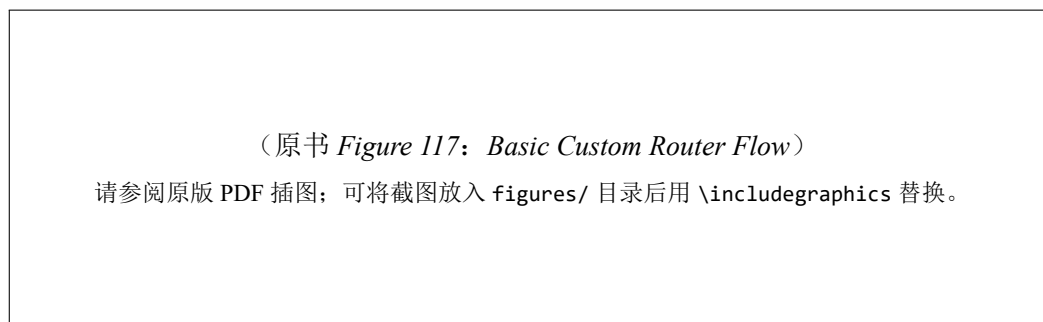


图 9.1: Custom Router 基本流程

9.2 使用 Custom Router 前的准备

Before Using Custom Router

运行 Custom Router 之前，需要完成以下准备工作：

- 确认工艺文件中已正确定义所需的设计规则，例如层与 via 的定义（详见 *Synopsys Technology File and Routing Rules Reference Manual*）；
- 检查设计中的连接关系，并排查 pin 阻挡（pin blockage）问题（详见 *Custom Compiler Custom Router User Guide* 中“Reviewing the Design”相关内容）；

- 确认 block 内的电源、地网已完成布线（详见 *Fusion Compiler Design Planning User Guide*）；
- 按照「检查可布线性」中的方法检查布局的可布线性。

9.3 定义布线约束

Defining Routing Constraints

Fusion Compiler 提供一组全局生效的应用选项来控制自定义布线的结果（见「使用 Custom Router 应用选项」）。除此之外，还可以针对具体网络定义布线约束，这些约束的优先级高于全局应用选项，起到”引导”布线的作用。Custom Router 会同时遵守这些自定义约束与 Fusion Compiler 的非默认布线规则。

注意

针对具体网络的约束必须应用在物理网络（physical net）上。在选取受约束网络时，应使用 `get_nets -physical_context` 以确保返回的是物理网络对象，例如：

```
fc_shell> create_net_shielding \  
-for [get_nets -physical_context pr*] \  
-disabled_layers {M5 M6} -sharing true \  
-layer_gaps {M2 3 M3 5}  
{shielding_4}
```

表 9.1 汇总了用于创建各类 Custom Router 约束组（constraint group）的命令。关于非默认布线规则，见「使用非默认布线规则」；关于约束组的检查与移除，使用 `report_constraint_groups` 与 `remove_constraint_groups`（见「管理约束组」）。

表 9.1: 定义 Custom Router 布线约束的命令

命令	说明
<code>create_bus_routing_style</code>	为一组以单一总线主干（trunk）形式布线的网络指定总线布线风格，见「定义总线布线风格」。
<code>create_differential_group</code>	为布线定义差分组，见「创建差分组」。
<code>create_wire_matching</code>	为一组网络或 pin 到 pin 连接定义匹配走线长度，见「定义匹配走线长度」。
<code>create_length_limit</code>	为一组网络或 pin 到 pin 连接定义最小走线长度，见「定义最小走线长度」。
<code>create_net_shielding</code>	为一组网络或 bundle 定义屏蔽风格，见「在网络上插入屏蔽」。
<code>create_net_priority</code>	决定网络的布线顺序，优先级越高越先布线，见「定义网络优先级」。

9.3.1 定义总线布线风格

Defining the Bus Routing Style

要将一组网络或 bundle 作为单一总线主干（trunk）布线，可用 `create_bus_routing_style` 创建约束组，需要指定：

- 约束组名称：不指定时自动命名为 `bus_style_n`；若重名默认报错，可加 `-force` 覆盖已有同名组；
- 适用对象：通过 `-for` 指定网络、bundle 或拓扑边（topology edge）；
- 约束内容：通过表 9.2 中的选项，按层为主干指定约束，并覆盖各 bit 上的单独设置。

表 9.2: 总线布线风格约束选项

选项	说明
<code>-valid_layers</code>	指定主干可用层列表，工具据此推导最小/最大层。
<code>-layer_spacings</code>	按层指定最小间距，如 {M1 5.0 M2 6.0}。
<code>-layer_widths</code>	按层指定最小宽度，如 {M1 5.0 M2 6.0}。
<code>-shield_placement</code>	主干屏蔽放置方式：default（默认，取自 <code>custom.route.bus_intra_shield_placement</code> ）、 <code>double_interleave</code> （每根线单独屏蔽）、 <code>half_interleave</code> （每两根线共享一次屏蔽）、 <code>interleave</code> （相邻线间插入共享屏蔽）、 <code>outside</code> （仅在主干/组外侧放置屏蔽）。见「总线布线选项」。
<code>-corner_type</code>	总线主干拐角布线方式：auto（默认，自动选取以降低拥塞）、 <code>cross</code> （拐角处各 bit 交叠）、 <code>river</code> （拐角处各 bit 不交叠）。见「总线布线选项」。
<code>-gap</code>	最外侧总线 bit 与 block 内其他图形之间的间距，默认 0。

例如，以下命令为匹配 `pr*` 的网络创建总线布线约束组（未命名，自动得名 `bus_style_1`），屏蔽方式为 `interleave`，M2 层最小间距 3 μm ，M3 层最小间距 5 μm ：

```
fc_shell> create_bus_routing_style \  
-for [get_nets -physical_context pr*] \  
-shield_placement interleave \  
-layer_spacings {M2 3 M3 5}  
{bus_style_1}
```

9.3.2 创建差分组

Creating Differential Groups

使用 `create_differential_group` 可为网络、bundle 或拓扑边创建差分组（differential group）或差分对（differential pair）。Custom Router 会为每条网络配对最近的 pin，并从两个 pin 之间的公共汇聚点开始布线，尽量让距离最近、走线模式最相似的一对网络配成对。

需要指定：

- 名称：不指定时自动命名为 `differential_pair_n` 或 `differential_group_n`；同名默认报错，可用 `-force` 覆盖；

- 组内网络：通过 `-for` 指定；指定两条网络即构成差分对，三条则构成差分组；
- 约束内容：见表 9.3。

表 9.3: 差分组约束选项

选项	说明
<code>-twist_style</code>	绞线 (twist) 方式: <code>diagonal</code> (45° 绞线)、 <code>none</code> (默认, 不绞线)、 <code>orthogonal</code> (90° 绞线)。
<code>-twist_interval</code>	相邻两次绞线间的距离。
<code>-twist_offset</code>	第一次绞线与被连接 pin 之间的距离, 默认 0。
<code>-valid_layers</code>	差分组可用的布线层。
<code>-layer_spacings</code>	按层指定最小间距。
<code>-layer_widths</code>	按层指定最小宽度。
<code>-shield_placement</code>	当组内网络需要屏蔽时, 主干的屏蔽放置方式, 取值同「定义总线布线风格」, 默认为 <code>outside</code> 。见「总线布线选项」。
<code>-gap</code>	最外侧 bit 与其他图形之间的间距 (微米), 默认 0。

例如, 以下命令为 `prn`、`prp` 两条网络创建差分对 (自动命名为 `differential_pair_0`), 不指定绞线方式, 屏蔽方式取默认值:

```
fc_shell> create_differential_group \  
-for [get_nets -physical_context {prn prp}] \  
-shield_placement default  
{differential_pair_0}
```

9.3.3 在网络上插入屏蔽

Inserting Shields on the Nets

使用 `create_net_shielding` 可为选定网络添加屏蔽 (shielding), 以降低串扰或寄生效应。需要指定:

- 名称: 不指定时自动命名为 `shielding_n`; 同名默认报错, 可用 `-force` 覆盖;
- 适用对象: 通过 `-for` 指定网络、`bundle` 或拓扑边;
- 屏蔽风格与按层要求: 见表 9.4。

例如, 以下命令为名称以 `pr` 开头的网络创建屏蔽约束组 (自动命名 `shielding_4`): M2 层最小间距 3 μm , M3 层 5 μm , 允许共享屏蔽, 且 M5、M6 层不可用于屏蔽:

```
fc_shell> create_net_shielding \  
-for [get_nets -physical_context pr*] \  
-disabled_layers {M5 M6} -sharing true -layer_gaps {M2 3 M3 5}  
{shielding_4}
```


表 9.4: 网络屏蔽约束选项（节选）

选项	说明
-shield_net / -shield_net_2	指定作为屏蔽的（次）网络；布单条信号网时分别用于左/下与右/上两侧屏蔽。
-via_defs	可用于连接屏蔽的 via 定义；默认全部可用。
-gap / -max_gap	走线与屏蔽之间的全局最小/最大间距，默认 0。
-width	全局屏蔽宽度，默认 0。
-layer_gaps / -layer_max_gaps / -layer_widths	按层指定最小间距、最大间距、屏蔽宽度，默认 0。
-min_segment	需要屏蔽的最小走线长度，默认 0。
-sharing	组内网络是否可共享屏蔽，取值 unset/true/false。
-group_shield	使用同一套屏蔽图形屏蔽组内全部网络；与 -enclose_pins 互斥。
-enclose_pins / -enclose_vias	屏蔽是否包裹 pin / via，取值 unset/true/false；前者与 -group_shield 互斥。
-disabled_layers	不允许用于屏蔽的层。

9.3.4 定义网络优先级

Defining the Net Priority

使用网络优先级约束可以指定特定网络的布线顺序，优先级高的网络先布线。用 create_net_priority 创建约束组，需要指定：

- 名称：不指定时由命令自动分配；同名默认报错，可用 -force 覆盖；
- 适用对象：通过 -for 指定网络或 bundle；
- 优先级：通过 -priority 指定，取值范围为 -128 到 128（含边界）的整数。

例如，以下命令为匹配 pr* 的网络与 bundle 创建名为 abc 的约束组，优先级设为 15：

```
fc_shell> create_net_priority abc \  
-for [get_nets -physical_context pr*] \  
-priority 15  
{abc}
```

9.3.5 定义最小走线长度

Defining Minimum Wire Lengths

使用 create_length_limit 可为一组网络或 pin 到 pin 连接中的各对象分别定义不同的最小走线长度。此特性常用于对单驱动、多接收端（single driver, multiple receivers）的网络做长度限制布线：

命令执行时，Custom Router 会在后台生成 Steiner 节点，并在驱动端、Steiner 节点与各接收端之间执行点到点布线；每个接收端可以有匹配长度约束，也可以有独立于驱动端的长度约束。

(原书 Figure 118: *Receivers With Individual Wire Length Constraints From Driver D*)
请参阅原版 PDF 插图。

图 9.2: 驱动端 D 带各自独立走线长度约束的多个接收端

命令语法:

```
fc_shell> create_length_limit
  -for objects
  [-min_value float]
  [-exclude pins_or_ports]
  [-driver pin_or_port]
  [-force]
  [intent_name]
```

主要选项:

- -for: 必选，指定需满足约束的驱动 pin/port 或网络组；
- -min_value: 最小走线长度，默认 0；
- -exclude: 从长度匹配约束中排除指定 pin 或 port；
- -driver: 指定驱动 pin 或 port；
- -force: 覆盖已有同名约束；
- intent_name: 约束名称，不指定则自动分配，同名默认报错。

示例 29: 为驱动 D 的 5 个接收端分别创建走线长度约束
针对驱动 D 与 5 个接收端 R1–R5，先为驱动网络指定驱动 pin，再为各接收端 pin 各自设置不同的最小长度（数值仅为示意）:

```
fc_shell> create_length_limit -for [get_nets -physical_context $net] \
  -driver [get_pins -physical_context D/o1]
fc_shell> create_length_limit -for [get_pins -physical_context R1/a] \
  -min_value 38.0
fc_shell> create_length_limit -for [get_pins -physical_context R2/a] \
  -min_value 40.0
fc_shell> create_length_limit -for [get_pins -physical_context R3/a] \
  -min_value 45.0
fc_shell> create_length_limit -for [get_pins -physical_context R4/a] \
  -min_value 42.0
fc_shell> create_length_limit -for [get_pins -physical_context R5/a] \
  -min_value 41.0
```

示例 30：为 **pr*** 网络设置最小走线长度

以下命令为匹配 **pr*** 的网络与 **bundle** 创建名为 **abc** 的约束组，最小走线长度设为 10，即组内每条走线长度均不小于 10 μm ：

```
fc_shell> create_length_limit abc -for [get_nets -physical_context pr*] \
-min_value 10
{abc}
```

（原书 *Figure 119: Minimum Wire Length Constraint for pr* Nets*）

请参阅原版 PDF 插图。

图 9.3: **pr*** 网络的最小走线长度约束

示例 31：为 **a2/A pin** 设置最小走线长度

以下命令为 **a2/A pin** 创建名为 **gcr_length_limit_a2** 的约束组，最小走线长度设为 2000：

```
fc_shell> create_length_limit -for [get_pins -physical_context a2/A] \
-min_value 2000.0 -force gcr_length_limit_a2
```

（原书 *Figure 120: Minimum Wire Length Constraint for the a2/A Pin*）

请参阅原版 PDF 插图。

图 9.4: **a2/A pin** 的最小走线长度约束

9.3.6 定义匹配走线长度

Defining Matching Wire Lengths

使用 **create_wire_matching** 可定义一组网络或 **pin** 到 **pin** 连接，使其走线长度保持一致，从而降低时钟偏斜（**skew**）与时序违例。支持两种匹配方式：

- 基于 **pin**（**pin-based**）：**pin** 之间的走线长度需匹配，典型场景是多个接收端相对同一个驱动端 **D**（或 **D'**）保持长度一致；
- 基于网络（**net-based**）：指定网络的总走线长度需匹配，典型场景是两条网络（如 **pr1**、**pr2**）的总长度需落在给定容差（如 10 μm ）范围内。

命令语法：

```
fc_shell> create_wire_matching
```

(原书 *Figure 121: Receivers With Matched Wire Length From Driver D*)
请参阅原版 PDF 插图。

图 9.5: 驱动端 D 带匹配走线长度的多个接收端

(原书 *Figure 122: Receivers With Matched Wire Length From Driver D'*)
请参阅原版 PDF 插图。

图 9.6: 驱动端 D' 带匹配走线长度的多个接收端

(原书 *Figure 123: Net-Based Wire Length Matching*)
请参阅原版 PDF 插图。

图 9.7: 基于网络的走线长度匹配

```

-for objects
-tolerance float
[-match_type type]
[-exclude objects]
[-relative]
[-driver pin_or_port]
[-force]
[intent_name]

```

主要选项：

- **-for**：必选，指定驱动 pin/port（pin-based）或网络组（net-based）；
- **-tolerance**：长度容差，须 ≥ 0 ；配合 **-relative** 可解释为百分比；
- **-relative**：将 **-tolerance** ≤ 1 的值解释为百分比，须与 **-tolerance** 同时使用；
- **-match_type**：匹配类型，length（默认，比较总长度）或 length_per_layer（按层比较长度）；
- **-exclude**：从匹配约束中排除指定 pin 或 port；
- **-driver**：指定驱动 pin 或 port；
- **-force**：覆盖已有同名约束；
- **intent_name**：约束名称，不指定则自动分配。

布线完成后会生成长度匹配汇总报告，用于排查未满足容差的走线：若 **-for** 指定的是基于 pin 的匹配，会生成 Length Constraint Report（原书 Figure 124）；若是基于网络的匹配，会生成 Routed Length Report（原书 Figure 125）。两种报告均按容差列出实际走线长度与偏差，便于定位超差的连接。

以下按典型场景给出用法示例（数值与对象名仅为示意）：

示例 32/33：多接收端相对驱动 **D**（或 **D'**）的长度匹配

```

% 驱动为固定 pin D/o1，各接收端相对驱动匹配（容差 1%）
fc_shell> create_wire_matching -for [get_nets -physical_context $net] \
    -relative -tolerance 0.01 -driver [get_pins -physical_context D/o1]
fc_shell> create_wire_matching -for [get_pins \
    -physical_context {R1/a R2/a R3/a}] -tolerance 0.01 -relative

% 驱动为变量指定的 pin（如平衡驱动 D'），其余接收端排除驱动本身后互相匹配
fc_shell> create_wire_matching -for [get_nets -physical_context $net] \
    -tolerance 0.01 -relative -driver $driver_full_name
fc_shell> create_wire_matching \
    -for [get_pins -physical_context -of_objects $net] \
    -exclude $driver_full_name -tolerance 0.01 -relative

```

示例 34/35：pr* 网络的绝对/相对容差匹配

```

% 绝对容差 10um
fc_shell> create_wire_matching abc -for [get_nets -physical_context pr*] \
    -tolerance 10
{abc}

```

```
% 相对容差 10%
fc_shell> create_wire_matching def -for [get_nets -physical_context pr*] \
    -tolerance 0.10 -relative
{def}
```

示例 36/37: 按层匹配与排除特定 pin

```
% 对 clk 网络各终端按层比较走线长度, 容差 1%
fc_shell> create_wire_matching -for [get_pins -physical_context \
    -of_objects [get_nets -physical_context {clk}]] \
    -tolerance 0.01 -match_type length_per_layer -relative \
    -force clk_match

% 对 $nets 做匹配, 但排除 u1/a pin, 容差 5%
fc_shell> create_wire_matching -for $nets -tolerance 0.05 \
    -exclude [get_pins -physical_context u1/a] -force clk_match2
```

示例 38: 基于网络的匹配并排除指定 pin, 随后布线

```
set nets [get_nets -physical_context {net1 net1p}]
set_routing_rule -min_routing_layer m7 -max_routing_layer m8 \
    -min_layer_mode allow_pin_connection -max_layer_mode hard $nets
set exclude_pin_names [get_pins -physical_context {i2/a i3/a}]
create_wire_matching -for $nets -tolerance 0.01 -relative \
    -exclude $exclude_pin_names
route_custom -nets $nets
```

(原书 *Figure 126: Net-Based Wire Matching Constraints With Pin Exclusions*)

请参阅原版 PDF 插图。

图 9.8: 排除指定 pin 的基于网络的走线匹配约束

也可以结合 `set_routing_rule` 限定布线层范围, 再定义匹配约束并运行布线, 形成一个完整的“设层 → 设匹配约束 → 布线”流程, 具体做法可参考「使用 DDR 布线流程」中的类似步骤。

9.4 管理约束组

Managing Constraint Groups

创建约束组之后, 可以对其进行报告或移除:

- 报告: 使用 `report_constraint_groups`, 可用 `-type` 指定要报告的约束类型 (默认报告全部类型), 用 `-count` 指定报告对象数量。支持的类型包括: `bus_style` (`create_bus_routing_style`)、`differential_group` 与 `differential_pair` (`create_differential_group`)、`matched_wire`

(create_wire_matching) 、 net_priority (create_net_priority) 、 shielding (create_net_shielding)、 wire_length_limit (create_length_limit)。

- 移除：使用 remove_constraint_groups，指定要移除的约束组名，或用 -all 移除全部。命令会解除相关对象与被移除约束组的关联，并报告实际移除的组数。

```
fc_shell> report_constraint_groups
...
Name                                Type                                Objects
-----
NDR_bus                             bus_style                           bus1
NDR_diff                            differential_pair                    clk0 clk1
shield_diff                          shielding                           clk0 clk1
matchL_con                           matched_wire                         match1

fc_shell> remove_constraint_groups $a
```

9.5 使用 Custom Router 应用选项

Using Custom Routing Application Options

Fusion Compiler 提供一组应用选项来全局控制自定义布线结果；若需针对具体网络覆盖这些全局设置，见「定义布线约束」。

要列出全部自定义布线相关的应用选项：

```
fc_shell> report_app_options custom.route.*
```

若要启用混合流程（hybrid flow），将 custom.route.hybrid_flow 设为 on，该选项会自动为一系列混合流程相关的应用选项设定合适的值。

表 9.5 按功能分组列出主要的自定义布线应用选项，各组的详细说明见后续小节。

以下按功能分组详细说明主要应用选项。

9.5.1 总线布线选项

Bus Routing Options

一组以 custom.route.bus_* 命名的应用选项用于控制总线布线的以下方面：

- 拐角类型（**Corner Type**）： custom.route.bus_corner_type，取值 auto（默认，自动选取以降低拥塞）、cross（拐角处交叠）、river（拐角处不交叠）；
- 组内屏蔽放置（**Intra-shield Placement**）： custom.route.bus_intra_shield_placement，取值含义与「定义总线布线风格」中的 -shield_placement 相同，默认 outside；
- **Pin-主干偏移（Pin-Trunk Offset）**： custom.route.bus_pin_trunk_offset，用于改变总线主干与总线 bit pin 之间的默认间距；
- 主干拆分（**Trunk Splitting**）： custom.route.bus_split_ignore_width 指定触发主干拆分的障碍（如电源网格）宽度阈值，即当遮挡宽度超过该值时把总线主干拆成多段；在此基础上，

`custom.route.bus_split_even_bits` 要求 `custom.route.bus_split_ignore_width` 已设置，用于控制自动拆分时电源/地轨之间是否按偶数根 bit 布线；

- 分支引出 (**Tapoffs**): `custom.route.bus_tap_off_enable` 控制是否将 pin 连接到总线主干上；开启后可用 `custom.route.bus_tap_off_shielding` 为分支引出部分添加屏蔽。

以下示例中，总线主干含 4 根 bit，且已设置 `custom.route.bus_split_ignore_width`；电源、地轨之间的空间只能容纳 3 根 bit。图 9.9(a) 示出 `custom.route.bus_split_even_bits` 关闭时的两种可能结果，图 9.9(b) 示出该选项开启时的结果。

(原书 Figure 127: Splitting Bus Trunk to Route Through Power and Ground Rails)
请参阅原版 PDF 插图。

图 9.9: 总线主干为穿越电源/地轨而拆分

9.5.2 网格贴合选项

Track Adherence Options

使用 `custom.route.layer_grid_mode` 控制走线及其并行屏蔽、并行屏蔽跳线 (jumper)、总线屏蔽是否贴合走线轨道 (track) 或布线网格。取值为 `{{layerName mode}}` 形式的列表，`layerName` 可为具体层名或 **All** (全部层)；`mode` 可取：

- **on**: 贴合轨道，若该层无轨道定义则贴合布线网格；
- **off**: 不贴合轨道或网格，适用于无网格 (gridless) 布线；
- **off_cost <costValue>**: 尽量贴合轨道，数值越大越倾向于保持在轨道/网格上。

若未定义轨道与布线网格却指定了 **on**，Custom Router 会使用内部推导的网格：网格间距为走线宽度加图形间最小间距，偏移量取 0 或间距的一半 (取能容纳更多 pin 的一个)。例如 `{{M2 off} {M3 off_cost 4}}` 表示 M2 层不贴合网格、M3 层以代价值 4 尽量贴合。

9.5.3 差分对选项

Differential-Pair Options

一组 `custom.route.diffpair_*` 应用选项用于控制差分对布线：`custom.route.diffpair_twist_jumper_enable` (是否对组内网络做绞线)、`custom.route.diffpair_twist_jumper_interval` (相邻绞线间距)、`custom.route.diffpair_twist_jumper_offset` (首次绞线与被连接 pin 的距离，默认 0)、`custom.route.diffpair_twist_jumper_style` (绞线方式，取值 `diagonal/none` (默认)/`orthogonal`，含义同「创建差分组」中的 `-twist_style`)。

9.5.4 屏蔽选项

Shielding Options

一组 `custom.route.shield_*` 应用选项用于控制屏蔽行为:

- `custom.route.shield_connect_mesh_overlap`: 在屏蔽与电源网格的每个交叉处创建 via 连接; 若屏蔽与任何电源条均不相交, 则退回默认的四段式连接方式;
- `custom.route.shield_connect_route`: 将屏蔽连接到地, 会产生较长的连接线并可能阻碍后续布线, 故仅在确有需要时开启 (这类屏蔽会被标记为动态屏蔽);
- `custom.route.shield_connect_to_supply`: 将屏蔽连接到最近的电源网格;
- `custom.route.shield_min_open_loop`: 同一层上开放式屏蔽环 (open shield-loop) 的最小总长度, 短于该值则不添加屏蔽;
- `custom.route.shield_min_shield_seg / custom.route.shield_min_signal_seg`: 屏蔽段、需屏蔽走线段的最小长度, 默认均为 0;
- `custom.route.shield_net / custom.route.shield_second_net`: 全局默认的 (次) 屏蔽网络, 含义同 `create_net_shielding` 中的 `-shield_net/-shield_net_2`。

9.5.5 单环长度补偿

Single-Loop Matching

创建单个绕线环 (single loop) 以延长走线、实现长度匹配, 常用于 DDR 布线流程。

(原书 *Figure 128: Adding Single Loops to Extend Wires*)

请参阅原版 PDF 插图。

图 9.10: 通过单环延长走线

相关应用选项: `custom.route.single_loop_match` (开启单环延长)、`custom.route.single_loop_match_max_spacing` / `custom.route.single_loop_match_min_spacing` (环内最大/最小间距)、`custom.route.single_loop_match_offset` (允许环内某段走线使用与其余段不同的层)。

9.5.6 平衡模式选项

Balance Mode Option

`custom.route.balance_mode` 应用选项用于中介层 (interposer) 设计, 减小因走线长度与 via cut 数量差异带来的偏斜。将其设为 `true` 后:

- Custom Router 会在同一网络的两个 μ bump 与一个 C4 bump 之间创建 feedthrough 走线;
- μ bump 与 C4 bump 之间的走线长度保持一致;

- 各 μ bump-C4 bump 连接中的 via cut 数量保持一致。

典型用法是先用 `create_routing_rule/set_routing_rule` 定义各层宽度、via 及层方向（如 M2、M4 走水平，M1、M3、M5 走垂直），再打开 `custom.route.balance_mode`，最后执行 `route_custom`：

```
fc_shell> create_routing_rule skew -widths {M1 6 M2 6 M3 6 M4 6 M5 25} \
    -vias {{VIA12_1cut 3x3 R} {VIA23_1cut 3x3 R} {VIA34_1cut 3x3 R} \
    {VIA45_1cut 2x5 R}}
fc_shell> set_routing_rule $nets_to_route -rule skew \
    -max_routing_layer M5 -min_routing_layer M1
fc_shell> set_app_options -list {custom.route.balance_mode true}
fc_shell> route_custom -nets $nets_to_route
```

9.6 使用 Custom Router 布线

Routing With the Custom Router

执行自定义布线前，需先按照「定义布线约束」创建所需约束，并设置合适的应用选项。之后运行 `route_custom` 即可调用 Custom Router 完成布线。默认对 block 内全部网络布线；如需限定范围，使用 `-nets` 指定网络。

默认情况下，命令运行结束后会清除内部配置数据；若希望在当前会话内继续运行其他 Custom Router 命令并保留缓存数据，可将 `-keep_session` 设为 `true`（默认 `false`）。

典型流程示例（先定义总线风格与屏蔽约束，再设置应用选项，最后布线）：

```
fc_shell> create_bundle -name Bus1 {net0 net1}
fc_shell> create_bus_routing_style -for {Bus1} \
    -valid_layers {M5 M6} -layer_widths {M5 0.4 M6 0.44} -force bus1
fc_shell> create_net_shielding -for {Bus1} -shield_net VSS \
    -layer_gaps 0.21 -layer_widths 0.2 -sharing true -force sh1
fc_shell> set_app_options -name custom.route.bus_corner_type -value river
fc_shell> set_app_options \
    -name custom.route.bus_intra_shield_placement -value interleave
fc_shell> route_custom -nets {net0 net1}
```

类似地，也可以先创建长度匹配约束或差分组约束，再执行 `route_custom`：

```
fc_shell> create_wire_matching -for MatchL1 \
    -match_type length -tolerance 1 -force matchL1_con
fc_shell> route_custom -nets {net1 net2 net3 net4}

fc_shell> create_differential_group -for {net1 net2} \
    -valid_layers {M3 M4} -layer_widths {M3 0.4 M4 0.4} \
    -layer_spacings {M3 0.4 M4 0.4} \
    -twist_style diagonal -twist_interval 80.0 -force Diff
fc_shell> route_custom -nets {net1 net2}
```

9.7 网络屏蔽

Shielding the Nets

在敏感网络周围添加屏蔽可以降低噪声与串扰。执行 `route_custom` 时，工具会依据 `create_net_shielding` 定义的约束自动为相应网络添加屏蔽。

如果需要单独在一次运行中为已布线网络补充屏蔽，可使用 `create_custom_shields`：

```
fc_shell> create_custom_shields -nets {net1 net2}
```

要移除由 `create_custom_shields` 添加的屏蔽，使用 `remove_custom_shields`；该命令同时也会移除 `route_custom` 自动创建的屏蔽。

9.8 检查布线结果

Checking the Routing Results

运行 Custom Router 完成布线后，可通过报告布线结果、生成拥塞图（congestion map）、运行 DRC 检查等方式检查结果的质量。具体检查方法见「分析布线结果」。

9.9 使用混合布线流程

Using a Hybrid Routing Flow

混合流程（hybrid flow）中，先用 Custom Router 完成总线主干布线但暂不完成部分 pin 连接，再用 Zroute 完成这些 pin 连接。这样既能利用 Custom Router 做预布线，又能避免两种布线器在 pin 连接方式上的差异带来的不一致与 DRC 违例。

要启用混合流程，将 `custom.route.skip_connect_pin_type` 设为下列一个或多个取值（默认 none，即不启用混合流程）：

- **auto**：Custom Router 判断各 pin 的可布线性，自动完成可连接 pin 的连接；
- **all**：不完成 block 内任何 pin 的连接；
- **stdcell**：完成除标准单元 pin 之外的全部 pin 连接；
- **io**：完成除 I/O 单元 pin 之外的全部 pin 连接；
- **macro**：完成除 macro pin 之外的全部 pin 连接。

启用混合流程后，Custom Router 默认自行决定走线与未连接 pin 之间预留的间距；若需指定最大距离，可用 `custom.route.distance_to_net_pin`，取值格式为 {net_name distance} 的列表。

典型流程：先用 Custom Router 完成主干布线（跳过全部或部分 pin 连接），再用 `route_eco`、`remove_redundant_shapes` 与 `route_detail -incremental true` 交由 Zroute 完成剩余 pin 连接与清理：

```
fc_shell> set_app_options -name custom.route.skip_connect_pin_type \
    -value all
fc_shell> route_custom -nets {net1 net2}
fc_shell> route_eco -nets {net1 net2}
```

```
fc_shell> remove_redundant_shapes -nets {net1 net2}
fc_shell> route_detail -incremental true
```

若只希望跳过标准单元 pin 的连接（其余 pin 由 Custom Router 完成），可先用 `set_routing_rule` 限定布线层范围，再将 `custom.route.skip_connect_pin_type` 设为 `stdcell`，其余步骤相同。

9.10 使用 DDR 布线流程

Using a DDR Routing Flow

本节介绍如何用 Custom Router 对双倍数据速率（DDR）设计执行布线。图 9.11 给出 DDR 网络与 core 之间建立互连的基本流程：准备输入数据（定义 bundle、定义并应用布线规则）→ 首次自定义布线 → 定义单环延长（设置长度匹配）→ 二次自定义布线 → 插入 buffer 单元 → 运行 ECO 布线（含增量详细布线）→ 修剪冗余图形。

（原书 *Figure 129: A Basic DDR Flow*）

请参阅原版 PDF 插图。

图 9.11: 基本 DDR 布线流程

具体步骤如下：

1. 布线前准备

- 将需要长度匹配的网络分组为 bundle：

```
fc_shell> create_bundle -name GRP1 [get_nets $nets]
```

- 为 DDR 网络创建并应用布线规则，指定初始布线层（应预留至少两个额外层用于后布线长度匹配，例如初始布线用 M4、M5，则后布线匹配应包含 M3–M6）：

```
fc_shell> create_routing_rule DDR -widths {layer value ...} \
    -spacings {layer value ...}
fc_shell> set_routing_rule -rule DDR -min_routing_layer M4 \
    -max_routing_layer M5 $all_ddr_nets
```

- 按需添加布线阻挡，避免 Custom Router 在布局阻挡覆盖的区域内布线，从而减少后续插入 buffer 时出现无法放置的情况。

2. 首次自定义布线：route_custom -nets \$all_ddr_nets。

3. 定义长度匹配相关约束

- 设置单环匹配相关应用选项: `custom.route.single_loop_match` (开启) 、
`custom.route.single_loop_match_min_spacing`、`custom.route.single_loop_match_max_spacing`、
`custom.route.single_loop_match_offset_layer`;
 - 为各 `bundle` 设置匹配约束: `create_wire_matching -for [get_bundles GRP1] -match_type length -tolerance 20 -force match1` (对 GRP2 同理);
 - 用 `custom.route.match_box` 设置匹配布线的边界框 (默认 `{{0 0} {0 0}}`), 设置后布线器只在框内进行匹配布线, 应确保该框覆盖全部可用信道空间)。
4. 执行长度匹配布线: `route_custom -nets [get_bundles {GRP1}]`; 若定义了多个匹配框, 需对每个框分别运行。
 5. 报告不匹配的网络: 可对日志文件搜索关键字 `Mismatch`。
 6. 减少不匹配网络数量, 可选方法包括: 重新分配各组在同一信道内的可用布线层后重跑初始布线; 调大 `custom.route.single_loop_match_max_spacing`; 或允许单环使用不同层 (`custom.route.single_loop_match_offset_layer` 设为 `true`)。
 7. 插入 **buffer** 单元: 使用 `add_buffer_on_route`, 可指定网络/单元前缀便于识别, 并用 `-repeater_distance_length_ratio` 控制插入间隔 (例如总长的 20%)。
 8. 布局合法化: `legalize_placement -cells $ddr_BUF` 与 `check_legality -cells $ddr_BUF` (`-cells` 选项仅标准 `legalizer` 支持)。
 9. 保留时钟网走线形状: 将相关走线与 `via` 的 `shape_use` 属性设为 `user_route`。
 10. 限制后续改动范围: 将相关走线与 `via` 的 `physical_status` 属性设为 `minor_change`。
 11. 运行 **ECO** 布线重新连接网络: `route_eco -nets $all_nets -reroute modified_nets_only`。
 12. 运行增量详细布线修复 **DRC**: 先将 `shape_use` 属性改回 `detail_route`, 再执行 `route_detail -incremental true`, 最后用 `remove_redundant_shapes` 清理冗余图形。
 13. 检查布线结果: 用 `check_routes` 检查 **DRC** 违例 (默认仅检查信号图形之间; 加 `-check_from_user_shapes true` 可扩展到信号图形与用户图形之间); 也可在 GUI 中查看 ECO 网络与 `buffer`, 或报告 DDR 网络与 ECO 网络的长度。

Chapter 10

相对布局物理数据通路

Physical Datapath With Relative Placement

相对布局（relative placement, RP）是 Fusion Compiler 中用于精确控制一组单元之间相对列、行位置关系的布局约束机制。设计者可以把若干单元组织成一个矩阵状的“相对布局组”，在后续粗布局（coarse placement）、优化、时钟树综合与绕线过程中，工具会把该组中的单元当作一个整体来维护其内部拓扑，不会随意打散重排。业界也常把这种能力称为“物理数据通路”（physical datapath）或“结构化布局”（structured placement）。

本章围绕以下主题展开：

- 相对布局技术概述与收益
- 相对布局的整体流程
- 创建相对布局组
- 向组中添加对象（leaf cell、硬宏、层次化子组、blockage 等）
- 相对布局组选项的设置（锚定、对齐、tiling、方向、keepout、固定单元处理、优化限制、legalize 移动量等）
- 修改已有相对布局组的结构
- 为时钟 sink 自动生成相对布局组
- 相对布局组的布局与合法化
- 相对布局组的分析与查询
- 相对布局信息的保存
- 相对布局命令速查

10.1 相对布局技术概述

Introduction to Physical Datapath With Relative Placement

相对布局最常见的应用场景是数据通路（datapath）与寄存器阵列这类具有明显规律性的电路结构，但原则上可以把任意单元纳入相对布局组，从而人为固定一组门级逻辑之间的拓扑关系，直接决定版图上的排布方式。借助这种手段，设计者可以主动探索多种 QoR（Quality of Result）收益，例如缩短关键连线、降低局部拥塞、改善时序、控制寄存器间的时钟偏斜（skew）、减少 via 数量，以及提升良率、降低动态与漏电功耗。

相对布局组一旦创建并添加了单元，就会在网表上隐式生成一个“矩阵结构”，并在随后的物理优化流程中，被当作一个不可拆散的实体来保持内部相对位置——组本身既可以随布局自由浮动（floating），也可以被固定（fixed）在指定坐标。

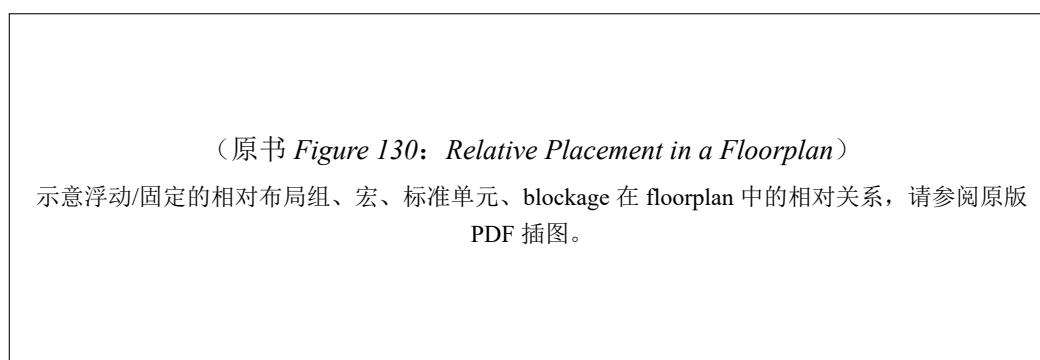


图 10.1: Floorplan 中的相对布局组

10.1.1 相对布局的主要收益

Benefits of Relative Placement

相对布局与具体工艺无关，并且天然有利于提升可绕线性，此外还能带来以下收益：

- 在设计的关键区域缩小布局搜索空间，从而使线长、时序、功耗、面积等 QoR 指标以及拥塞情况更加可预测；
- 在粗布局、优化、时钟树综合与绕线的各个阶段都能保持相对布局关系不被破坏；
- 为迁移旧版本（legacy）设计或第三方 IP 中已验证过的结构化版图提供了一种可靠的维持手段；
- 同时支持扁平（flat）与层次化（hierarchical）设计；
- 允许在保持相对布局拓扑不变的前提下，对组内单元做 sizing（换驱动能力）。

10.2 相对布局流程

Relative Placement Flow

一次典型的相对布局工作，大致按以下步骤展开：

1. 按常规方法完成设计准备工作（库、约束、DEF/网表导入等）。

2. 按需要设置好布局相关约束（例如多电压域相关的布局约束）。
3. 定义相对布局的具体结构与选项：
 - (a) 用 `create_rp_group` 创建相对布局组，见「创建相对布局组」；
 - (b) 用 `add_to_rp_group` 把单元、层次化子组或 `blockage` 加入组中，见「向组中添加对象」；
 - (c) 用 `set_rp_group_options` 设置组的锚定、对齐、tiling、方向等属性，见「设置相对布局组选项」。
4. 执行常规的布局与优化流程（例如 `place_opt`）。
5. 检查相对布局的实际效果，见「分析相对布局组」；若布局结果不理想，可以调整相对布局组的结构或选项，再重新执行布局与优化。

10.3 创建相对布局组

Creating Relative Placement Groups

相对布局组本质上是一份“容器”定义：它把一批 `leaf cell`、下级相对布局组以及 `blockage`，组织到一个由若干行、列构成的矩阵格点中。使用 `create_rp_group` 创建组时，必须用 `-name` 指定组名；可以用 `-columns` 与 `-rows` 指定矩阵的列数与行数，若不指定则默认创建一个 1 行 1 列的组。

例如，创建一个名为 `RP1`、有 6 行 6 列的相对布局组：

```
fc_shell> create_rp_group -name RP1 -columns 6 -rows 6
```

（原书 *Figure 131: Relative Placement Column and Row Positions*）

展示组内位置的行列编号方式，请参阅原版 PDF 插图。

图 10.2: 相对布局组的列、行位置编号

关于组内行列位置的几条基本规则：

- 列编号从 0 开始，0 为最左列；行编号从 0 开始，0 为最底行；
- 一列的宽度由该列中最宽的单元决定，一行的高度由该行中最高的单元决定；
- 矩阵中的位置不要求全部占满——可以只使用其中一部分格点，其余位置保持为空。

默认情况下，相对布局组创建在当前设计（`current design`）中；若需要在其他设计中创建，可用 `-design` 选项指定目标设计名。如果该设计在顶层被多次例化（`instantiate`），那么对其中某一个例化实例所做的相对布局组修改，会同步反映到所有其他例化实例上。

要删除相对布局组，使用 `remove_rp_groups` 命令。

10.4 向组中添加对象

Adding Objects to a Group

创建好相对布局组之后，可以用 `add_to_rp_group` 命令向其中添加以下几类对象：

- Leaf cell（含 physical-only cell），见「添加 Leaf Cell」；
- 硬宏（hard macro），见「添加硬宏单元」；
- 其他相对布局组（层次化嵌套），见「层次化相对布局组」；
- Blockage（占位/留白），见「添加 Blockage」。

添加对象时需要遵守以下限制：

- 目标相对布局组必须已经存在；
- 只能把对象加到组内一个尚为空的位置；
- 每个位置只能容纳一个对象。

要从组中移除对象，使用 `remove_from_rp_group` 命令：可分别用 `-cells`、`-rp_group`、`-blockage` 移除 leaf cell、子组或 blockage。被移除对象原来占用的位置会重新变为空位。

10.4.1 添加 Leaf Cell

Adding Leaf Cells

用 `-cells` 选项配合 `add_to_rp_group` 可以把 leaf cell（含 physical-only cell）加入组中，并用 `-column`、`-row` 指定其在组内矩阵中的位置。

一个 leaf cell 可以跨越多个列或多个行的位置，这称为“单元跨位”（straddling）。跨越的列数、行数分别用 `-num_columns`、`-num_rows` 指定，不设置时二者默认均为 1。例如，把单元 U23 加到位置 (0,0)，且它横跨 2 列、占 1 行：

```
fc_shell> add_to_rp_group rp1 -cells U23 \
    -column 0 -row 0 -num_columns 2 -num_rows 1
```

需要注意，跨位仅适用于 leaf cell，不能用于层次化子组或 blockage。

10.4.2 指定 Leaf Cell 的方向

Specifying Orientations for Leaf Cells

添加 leaf cell 时可以指定其摆放方向；若不指定，工具会自动为其分配一个合法方向。指定方向有两种方式：

- 在 `add_to_rp_group` 中用 `-orientation` 给出一组候选方向，工具会从中挑选一个合法方向；
- 用 `set_attribute` 命令在单元上设置 `rp_orientation` 属性。

10.4.3 添加硬宏单元

Adding Hard Macro Cells

`add_to_rp_group` 也支持把硬宏加入相对布局组。与普通 `leaf cell` 不同的是，添加硬宏时不能使用 `-pin`（指定用于对齐的 `pin`）或 `-orientation`（指定方向）选项。

处理含硬宏的相对布局组，建议按如下步骤操作：

1. 先创建相对布局组、加入含硬宏在内的所有单元，并设置好相关选项；
2. 用 `create_placement -floorplan` 完成一次布局；
3. 用 `set_placement_status fixed` 把组内硬宏的位置固定下来——在继续做后续布局与优化之前，组内硬宏必须已被布局并固定，组内其他单元也必须已经完成布局；
4. 再执行后续的布局与优化。

10.4.4 层次化相对布局组

Adding Relative Placement Groups (Hierarchical RP)

层次化相对布局允许把一个相对布局组作为对象嵌入到另一个组当中，被嵌入的组在父组眼中就像一个普通 `leaf cell` 一样参与排布。

采用层次化结构来组织相对布局约束，通常有以下好处：

- 可以让相对布局的组织方式与 RTL（Verilog/VHDL）的层次结构相呼应，便于维护和理解；
- 便于复用重复出现的电路模式（例如加法器阵列）；
- 能显著减少需要重复编写的相对布局脚本行数；
- 便于对独立模块做整体集成；
- 为搭建复杂的组合形态提供了很大的灵活性。

创建层次化相对布局组

Creating Hierarchical Relative Placement Groups

要把一个组嵌入到另一个组中，在 `add_to_rp_group` 中使用 `-rp_group` 选项指定被嵌入的子组名，并用 `-column`、`-row` 指定它在父组矩阵中的位置。被嵌入的子组必须与父组处于同一设计中。

一个子组一旦被嵌入某个父组，就相当于直接“焊死”在父组内部：同一个子组在同一设计中只能被嵌入一次；但是，包含了某个子组的父组本身，还可以被进一步嵌入到该设计中的另一个组里，或者被例化到另一个设计的组中。

下面的示例构造一个两级层次：先建立三个各含两个单元的底层组 `g_a`、`g_b`、`g_c`，再把它们作为三行嵌入到顶层组 `g_top` 中：

```
create_rp_group -name g_a -columns 2 -rows 1
add_to_rp_group g_a -cells U1 -column 0 -row 0
add_to_rp_group g_a -cells U4 -column 1 -row 0

create_rp_group -name g_b -columns 2 -rows 1
add_to_rp_group g_b -cells U2 -column 0 -row 0
```

```
add_to_rp_group g_b -cells U5 -column 1 -row 0

create_rp_group -name g_c -columns 2 -rows 1
add_to_rp_group g_c -cells U3 -column 0 -row 0
add_to_rp_group g_c -cells U6 -column 1 -row 0

create_rp_group -name g_top -columns 1 -rows 3
add_to_rp_group g_top -rp_group g_a -column 0 -row 0
add_to_rp_group g_top -rp_group g_b -column 0 -row 1
add_to_rp_group g_top -rp_group g_c -column 0 -row 2
```

(原书 *Figure 132: Including Groups in a Hierarchical Group*)

请参阅原版 PDF 插图。

图 10.3: 层次化相对布局组的嵌套结构

层次化结构中的跨位

Using Hierarchical Relative Placement for Straddling

前面提到的“跨位”不仅可以发生在单个 leaf cell 上，配合层次化嵌套还可以构造出既有列跨位又有行跨位的复杂形状：先分别定义只含 leaf cell 的底层组，再定义一个中间层组把它们组合起来，最后再定义一个顶层组，把这个中间层组和另一个额外的 leaf cell 一起嵌入，就可以让该 leaf cell 同时跨越多列。

(原书 *Figure 133/134: Hierarchical Relative Placement Group With Straddling*)

请参阅原版 PDF 插图。

图 10.4: 层次化结构中的行/列跨位组合

利用层次化结构做压缩布局

Using Hierarchical Relative Placement for Compression

默认情况下，相对布局的构造方式是从每个单元的左下角开始对齐排布的。“压缩”(compression)则会去除每一行中多余的空白，使结构更紧凑；压缩之后，各列不再严格对齐，但被压缩区域的利用率会更高。

要构造压缩结构，可以先分别创建若干个各含一行 leaf cell 的底层组，再创建一个父组把这些底层组按列的方式组合起来：

```
create_rp_group -name g_a -columns 2 -rows 1
add_to_rp_group g_a -cells U1 -column 0 -row 0
add_to_rp_group g_a -cells U4 -column 1 -row 0

create_rp_group -name g_b -columns 2 -rows 1
add_to_rp_group g_b -cells U2 -column 0 -row 0
add_to_rp_group g_b -cells U5 -column 1 -row 0

create_rp_group -name g_c -columns 2 -rows 1
add_to_rp_group g_c -cells U3 -column 0 -row 0
add_to_rp_group g_c -cells U6 -column 1 -row 0

create_rp_group -name g_top -columns 1 -rows 3
add_to_rp_group g_top -rp_group g_a -column 0 -row 0
add_to_rp_group g_top -rp_group g_b -column 0 -row 1
add_to_rp_group g_top -rp_group g_c -column 0 -row 2
```

也可以不依赖层次化结构，而是直接在 set_rp_group_options 上用 -tiling_type 选项，对水平方向应用压缩，详见「控制组内 Tiling 方式」。

10.4.5 添加 Blockage

Adding Blockages

用 -blockage 选项配合 add_to_rp_group，可以在相对布局组内部预留一块不允许放置相对布局单元的空间。添加 blockage 时可以指定：

- 位置：用 -column、-row 指定，若不指定则默认放在位置 (0,0)；
- 尺寸：用 -height、-width 指定。若不指定，工具会根据组的 tiling 类型自动确定尺寸——tiling 类型为 bit_slice 时，高度取 site row 高度、宽度取所在列宽度；tiling 类型为 compression 系列时，高度取 site row 高度、宽度取一个 site 的宽度；
- 是否允许与其他相对布局 blockage 或非相对布局对象重叠：用 -allow_overlap 选项。

例如，在 rp1 组的位置 (0,2) 添加一个名为 gap1、高 1 行、宽 5 列的 blockage：

```
fc_shell> add_to_rp_group rp1 -blockage gap1 \
    -column 0 -row 2 -width 5 -height 1
```

10.4.6 在预定义区域内自由添加单元

Adding Cells Within a Predefined Relative Placement Area

如果只想约束一批单元落在组内某个矩形区域中，而不严格规定每个单元的具体格点，可以用 -free_placement 选项配合 add_to_rp_group。使用该选项时必须同时指定：

- 放置区域的原点：-column、-row；
- 放置区域的高度和宽度：-height、-width；

- 该单元自身占用的列数与行数：`-num_columns`、`-num_rows`。

例如，把单元 **U41** 加入一个原点在 (0,0)、高 4、宽 5 的区域中，且该单元自身占 2 列 2 行：

```
fc_shell> add_to_rp_group rp2 -cells U41 -free_placement \  
-column 0 -row 0 -height 4 -width 5 -num_columns 2 -num_rows 2
```

10.5 设置相对布局组选项

Specifying Options for Relative Placement Groups

相对布局组的各类属性统一通过 `set_rp_group_options` 命令设置。表 10.1 汇总了主要选项及其作用，详细用法见后续小节。

表 10.1: `set_rp_group_options` 主要选项一览

选项	作用
<code>-x_offset</code> , <code>-y_offset</code>	指定锚点坐标（见「锚定相对布局组」）
<code>-anchor_corner</code>	指定锚点对应组的哪个角（见「锚定相对布局组」）
<code>-alignment</code>	指定组内单元的对齐方式（见「列内 Leaf Cell 对齐」）
<code>-pin_name</code>	Pin 对齐时使用的对齐 pin 名
<code>-tiling_type</code>	指定 tiling 方式（见「控制组内 Tiling 方式」）
<code>-group_orientation</code>	指定组的方向（见「相对布局组方向」）
<code>-utilization</code>	指定利用率，默认 100%
<code>-place_around_fixed_cells</code>	指定 legalize 时如何处理固定单元（见「处理布局中的固定单元」）
<code>-optimization_restriction</code>	限制组内单元允许的优化类型（见「控制 RP 单元优化」）
<code>-move_effort</code>	控制 legalize 时组的移动幅度（见「控制 Legalize 时的移动量」）

要移除已设置的选项，使用 `remove_rp_group_options` 命令；必须指定组名和至少一个选项，否则该命令不产生任何效果。

10.5.1 锚定相对布局组

Anchoring Relative Placement Groups

默认情况下，工具可以把一个顶层相对布局组放在 `core` 区域内的任意位置。如果需要精确控制某个顶层组的位置，可以用 `set_rp_group_options` 的 `-x_offset`、`-y_offset` 选项对其进行“锚定”。这两个偏移量都是相对 `core` 区域左下角的浮点数（单位微米）。

- 如果 `x`、`y` 坐标都指定了，组会被锚定在该固定点上；
- 如果只指定了其中一个坐标，工具会保持已指定的坐标不变，沿另一个方向自由滑动来确定最终位置。

用 `-anchor_corner` 可以指定用组的哪个角来对齐锚点，取值包括：

- `bottom_left`：以组的左下角为锚点（默认）；
- `bottom_right`：以组的右下角为锚点；

- **top_left**: 以组的左上角为锚点;
- **top_right**: 以组的右上角为锚点;
- **rp_location**: 以组内某个具体位置的元素为锚点, 该位置由 **-anchor_row**、**-anchor_column** 指定, 且该位置必须已放有单元、keepout 或子组。

例如, 把组 **misc1** 的左下角锚定在坐标 (100, 100):

```
fc_shell> set_rp_group_options misc1 -anchor_corner bottom_left \
-x_offset 100 -y_offset 100
```

也可以用组内指定位置的元素来锚定, 例如让 **RP1** 组第 1 列第 2 行处的单元被锚定到坐标 (100, 100):

```
fc_shell> set_rp_group_options RP1 \
-anchor_corner rp_location -anchor_column 1 -anchor_row 2 \
-x_offset 100 -y_offset 100
```

(原书 *Figure 136-138*: 锚点角与 *rp_location* 锚定示意)
请参阅原版 PDF 插图。

图 10.5: 相对布局组的锚定方式

10.5.2 列内 Leaf Cell 对齐

Aligning Leaf Cells Within a Column

组内同一列中的 leaf cell, 可以按以下三种方式对齐: 左对齐 (默认)、右对齐、按 pin 对齐。合理选择对齐方式, 有助于改善时序和可绕线性。

左对齐 (默认)

Aligning by the Left Edges

工具默认按左边缘对齐组内单元; 若要显式声明这一行为, 可使用 **-alignment left**。

右对齐

Aligning by the Right Edges

使用 **-alignment right** 可让组内单元按右边缘对齐。

注意

对于层次化相对布局组, 右对齐 (**bottom-right alignment**) 设置不会沿层次向下传播到子组。

按 Pin 位置对齐

Aligning by Pin Location

使用 `-alignment pin` 配合 `-pin_name`，可以让工具按某个指定 `pin` 的位置对齐该列中的单元。工具会在列内每个单元上查找该 `pin`：若某单元存在该 `pin`，就按该 `pin` 的位置对齐；若某单元不存在该 `pin`，则退化为左对齐，并给出提示信息；若整列中没有任何单元含该 `pin`，则给出告警信息。

```
create_rp_group -name rp1 -columns 1 -rows 4
set_rp_group_options rp1 -alignment pin -pin_name A
add_to_rp_group rp1 -cells U1 -column 0 -row 0
add_to_rp_group rp1 -cells U2 -column 0 -row 1
add_to_rp_group rp1 -cells U3 -column 0 -row 2
add_to_rp_group rp1 -cells U4 -column 0 -row 3
```

按 `pin` 对齐时，工具会尝试不同的单元方向，从中挑选出使该列宽度最小的组合；但如果在 `add_to_rp_group` 中已经用 `-orientation` 显式指定了某单元的方向，工具会尊重该设定，不再自动调整。

组一级设置的对齐 `pin` 会作用于组内所有单元；如果个别单元需要使用不同的对齐 `pin`，可以在 `add_to_rp_group` 添加该单元时用 `-pin_name` 单独覆盖。例如下面的脚本中，组 `rp2` 整体按 `pin A` 对齐，但其中 `I5`、`I6` 两个单元改用 `pin B` 对齐：

```
create_rp_group -name rp2 -columns 1 -rows 6
set_rp_group_options rp2 -alignment pin -pin_name A
add_to_rp_group rp2 -cells I3 -column 0 -row 0
add_to_rp_group rp2 -cells I4 -column 0 -row 1
add_to_rp_group rp2 -cells I5 -column 0 -row 2 -pin_name B
add_to_rp_group rp2 -cells I6 -column 0 -row 3 -pin_name B
add_to_rp_group rp2 -cells I7 -column 0 -row 4
add_to_rp_group rp2 -cells I8 -column 0 -row 5
```

添加对象时覆盖对齐方式

Overriding the Alignment When Adding Objects

在 `add_to_rp_group` 中使用 `-override_alignment` 选项，可以为正在添加的单个对象单独指定与组整体设置不同的对齐方式；但如果该组是按 `pin` 对齐的，则不能用该选项覆盖。

例如，组 `rp1` 整体为右对齐，但其中 `U0` 单独覆盖为左对齐，其余单元维持组的右对齐设置：

```
fc_shell> create_rp_group -name rp1 -columns 1 -rows 4
fc_shell> set_rp_group_options rp1 -alignment right
fc_shell> add_to_rp_group rp1 -cells U0 -column 0 -row 0 \
    -override_alignment left
fc_shell> add_to_rp_group rp1 -cells U1 -column 0 -row 1
fc_shell> add_to_rp_group rp1 -cells U2 -column 0 -row 2
fc_shell> add_to_rp_group rp1 -cells U3 -column 0 -row 3
```

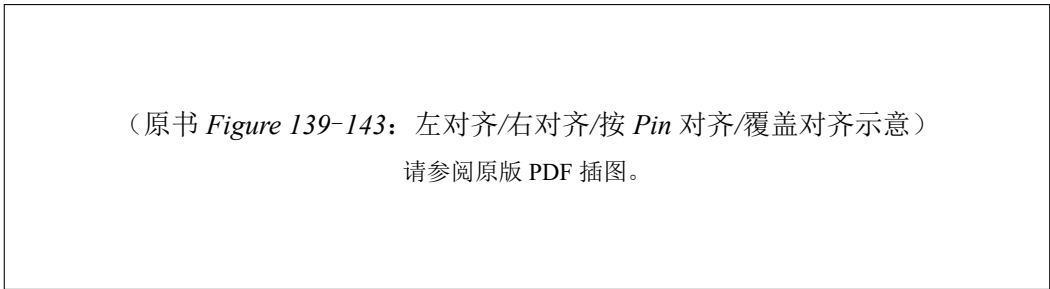



图 10.6: 几种列内对齐方式的效果对比

10.5.3 控制组内 Tiling 方式

Controlling the Tiling Within Relative Placement Groups

组内对象的排布疏密方式，由 `set_rp_group_options` 的 `-tiling_type` 选项控制，可选设置见表 10.2。

表 10.2: `-tiling_type` 选项设置

设置	说明
<code>bit_slice</code> (默认)	按位切片方式排布，同时保持行、列两个方向的对齐；行高取该行最高单元高度，列宽取该列最宽单元宽度
<code>horizontal_compression</code>	每行内水平方向压缩，仍保持行对齐
<code>vertical_compression</code>	每列内垂直方向压缩，仍保持列对齐

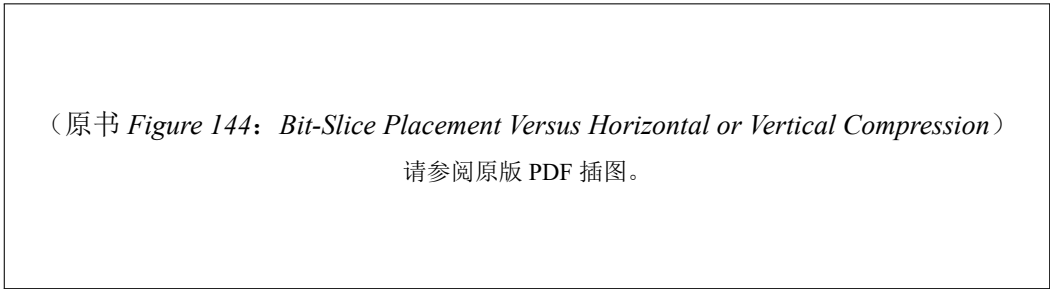


图 10.7: Bit-slice 排布与水平/垂直压缩的对比

需要注意，`-tiling_type` 的设置不会从父组自动传递给子组，每一层需要单独设置。
即使组内含有跨行、跨列的 `leaf cell`，也同样可以应用压缩：

```
fc_shell> add_to_rp_group rp -cells U5 \  
-column 0 -row 0 -num_columns 1 -num_rows 2  
fc_shell> set_rp_group_options rp -tiling_type horizontal_compression
```

10.5.4 相对布局组方向

Specifying the Orientation of Relative Placement Groups

Fusion Compiler 支持以下四种相对布局组方向：

- **R0**: 列方向从左到右、行方向从下到上;
- **R180**: 列方向从右到左、行方向从上到下 (相对 R0 整体旋转 180 度);
- **MY**: 列方向从右到左、行方向从下到上 (相对 R0 沿 Y 轴镜像);
- **MX**: 列方向从左到右、行方向从上到下 (相对 R180 沿 X 轴镜像)。

(原书 *Figure 146: Orientation of Relative Placement Groups*)
请参阅原版 PDF 插图。

图 10.8: 相对布局组的四种方向

工具默认会自动为相对布局组选择能够最小化线长的方向, 也可以用 `set_rp_group_options` 的 `-group_orientation` 选项手动指定:

```
fc_shell> set_rp_group_options [get_rp_groups design::rp] \
    -group_orientation MY
```

对于含有层次化子组的设计, 方向设置会沿层次一路向下传播到最底层。

注意

改变相对布局组的方向时, 之前设定的对齐方式、利用率等约束仍会按原有指定继续保持。

10.5.5 Keepout Margin

Specifying a Keepout Margin

如果希望阻止其他相对布局组紧贴某个特定组放置, 可以用 `set_rp_group_options` 的 `-rp_only_keepout_margin` 选项, 为该组的左、下、右、上四个方向分别指定一个只针对其他相对布局组生效的留白边距。

例如, 为组 `rp1` 设置左右边距 15、上下边距 10:

```
fc_shell> set_rp_group_options rp1 \
    -rp_only_keepout_margin {15 10 15 10}
```

10.5.6 处理布局中的固定单元

Handling Fixed Cells During Relative Placement

`Legalize` 相对布局组时如何对待 `floorplan` 中已固定的单元, 由 `set_rp_group_options` 的 `-place_around_fixed_cells` 选项控制, 取值见表 10.3。

对于层次化相对布局组, 可以给顶层组和下级子组分别设置不同的取值; 若未给某个下级子组单独设置, 则沿用顶层组的取值。

表 10.3: -place_around_fixed_cells 选项设置

设置	说明
standard	围绕固定的标准单元做 legalize，同时避开固定的 physical-only 单元
physical_only	围绕固定的 physical-only 单元做 legalize，同时避开固定的标准单元
all（默认）	同时围绕固定的标准单元与 physical-only 单元做 legalize
none	同时避开固定的标准单元与 physical-only 单元

10.5.7 允许非相对布局单元

Allowing Nonrelative Placement Cells

默认情况下，非相对布局单元在粗布局阶段不允许出现在相对布局组内部，但在优化与 legalize 阶段是允许的。若希望在粗布局阶段就利用某个组内的空闲区域来降低拥塞、改善 QoR，可以用 -allow_non_rp_cells 选项打开该行为：

```
fc_shell> set_rp_group_options rp1 \  
-allow_non_rp_cells
```

如果组内含有通过 -blockage 添加的留白区域，默认这些区域在布局、优化、legalize 各阶段都不允许放置任何单元。若希望连这些 blockage 区域也一并对非相对布局单元开放，可以使用 -allow_non_rp_cells_on_blockages 选项：

```
fc_shell> set_rp_group_options rp2 \  
-allow_non_rp_cells_on_blockages
```

10.5.8 控制相对布局单元的优化

Controlling the Optimization of Relative Placement Cells

对相对布局单元做修改或移动，都可能破坏组内既定的结构；如果某个单元在优化过程中被直接删除，其相对布局信息也会随之丢失。为了在后续优化中保护既有结构，可以用 -optimization_restriction 选项限制允许的优化类型，取值见表 10.4。

表 10.4: -optimization_restriction 选项设置

设置	说明
all_opt	不限制，允许对组内单元做任意优化
size_only	仅允许换驱动能力（sizing）
size_in_place	仅允许原位（in-place）sizing
no_opt	禁止对组内单元做任何优化

对于层次化相对布局组，该选项在顶层的设置会直接向下传播到各级子组，子组自身单独设置的值会被忽略。

10.5.9 控制 Legalize 时的移动量

Controlling Movement When Legalizing Relative Placement Groups

粗布局阶段，工具会为每个顶层相对布局组估算一个初始位置；在随后的 legalize 阶段，`-move_effort` 选项决定该组允许在多大范围内偏离这个初始位置来消除相对布局约束的违例。把该选项从较高的移动力度调低，相当于缩小了为该组搜索合法位置时所考察的区域范围。

10.5.10 处理 Tap Column 与相对布局组重叠

Handling Tap Columns and Relative Placement Group Overlaps

在粗布局阶段，为了避免相对布局组与 tap cell 所在列发生重叠，可以设置应用选项 `place.coarse.no_split_rp_width_threshold`。只有宽度小于相邻 tap column 间距的相对布局组，才能被工具自动挪开以消除重叠；该选项能帮助布局引擎提前获知 tap column 之间的间距信息。此选项主要用于 N5 及更早的工艺节点；N3 及更先进节点下，tap column 间距会被自动计算，无需手动设置。

```
place.coarse.no_split_rp_width_threshold 46
```

10.6 修改相对布局组结构

Changing the Structures of Relative Placement Groups

对已经创建好的相对布局组，可以用 `modify_rp_groups` 命令做结构性调整：

- 用 `-add_rows`、`-add_columns` 增加行或列；
- 用 `-remove_rows`、`-remove_columns` 删除行或列；
- 用 `-flip_row`、`-flip_column` 翻转某一行或某一列；
- 用 `-swap_rows`、`-swap_columns` 交换两行或两列。

例如，交换组 `my_rp_group` 的第 0 列与第 2 列：

```
fc_shell> modify_rp_groups [get_rp_groups my_rp_group] \
    -swap_columns {0 2}
```

再例如，翻转该组的第 1 列：

```
fc_shell> modify_rp_groups [get_rp_groups my_rp_group] \
    -flip_column 1
```

10.7 为时钟 Sink 生成相对布局组

Generating Relative Placement Groups for Clock Sinks

在已完成布局的设计中，可以用 `create_clock_rp_groups` 命令，为时钟 sink 及其驱动单元自动生成相对布局组。之后再执行布局与优化时，工具会依照这些自动生成的约束来放置时钟 sink 及其驱动单元，从而改善可绕线性、降低动态功耗。

(原书 *Figure 147/148: Swapping / Flipping Columns of a Relative Placement Group*)

请参阅原版 PDF 插图。

图 10.9: 列交换与列翻转示意

可以通过以下方式控制参与该自动分组的 sink 范围:

- 用 `-min_sinks`、`-max_sinks` 指定单个驱动单元所驱动 sink 数量的下限与上限 (默认下限为 2、上限为 128);
- 用 `-timing_driven` 选项决定是否排除时序关键 sink (默认会排除所有 slack 为负的 sink);
- 用 `-cells` 指定要考虑的单元范围 (默认考虑所有 sink)。

还可以通过以下方式控制生成的组的规模与形状:

- 用 `-distance` 指定同一组内 sink 之间允许的最大曼哈顿距离 (默认 100 微米), 超出该距离的 sink 会被划入不同的组;
- 用 `-max_rp_rows` 指定组的最大行数 (默认 32);
- 用 `-auto_shape` 让工具依据单元的实际分布情况, 自动决定组的行列数。

运行 `create_clock_rp_groups` 前, 设计必须已经完成布局; 生成相应的时钟相对布局组之后, 需要用 `reset_placement` 重置布局, 再用 `place_opt` 重新执行布局与优化。

10.8 相对布局组的布局与合法化

Performing Placement and Legalization of Relative Placement Groups

10.8.1 存在 Obstruction 时的相对布局

Relative Placement in a Design Containing Obstructions

在布局过程中, 相对布局组会自动避开 DEF 文件中定义的、或者由 `create_placement_blockage` 命令创建的 `placement blockage` (obstruction)。当组的路径上存在这类障碍时, 组可以被拆分成若干段跨越该障碍, 同时仍然维持内部的相对布局关系。

具体处理方式取决于障碍的高度: 如果障碍高度低于某个阈值, 相对布局单元会被垂直方向挪动以避开它; 如果障碍高度超过该阈值, 则整列单元会被水平方向整体挪动。

10.8.2 对已布局设计做相对布局组合法化

Legalizing Relative Placement Groups in a Placed Design

要在一个已完成布局的设计上, 只针对相对布局组做局部改善, 可以用 `legalize_rp_groups` 命令——该命令只 `legalize` 相对布局组, 不涉及非相对布局单元; 也可以指定只对部分相对布局组生效。

(原书 *Figure 149: Relative Placement in a Design Containing Obstructions*)

请参阅原版 PDF 插图。

图 10.10: 存在 Obstruction 时相对布局的处理方式

用 `-prototype` 选项可以做一次快速 `legalize`，此时工具不保证所有合法性约束都被满足，因此该选项更适合在方案摸索阶段使用。

用 `-legalize_over_rp` 选项，可以允许某个相对布局组 `legalize` 时覆盖到其他相对布局组之上。例如，让组 `RP3` 可以合法化到 `RP1`、`RP2` 之上：

```
fc_shell> legalize_rp_groups -legalize_over_rp RP3
```

(原书 *Figure 150: Legalizing the RP3 Relative Placement Group Over Other Groups*)

请参阅原版 PDF 插图。

图 10.11: 允许某相对布局组覆盖其他组做 Legalize

用 `legalize_rp_groups` 对一个或多个相对布局组做完 `legalize` 后，可能会与其他相对布局组或非相对布局单元产生新的重叠。此时应使用 `check_legality` 命令检查是否存在重叠，再用 `legalize_placement` 命令消除剩余的重叠问题。

10.8.3 在已布局设计中新增相对布局组

Creating New Relative Placement Groups in a Placed Design

也可以在一个已经布局完成的设计中新建相对布局组，再用 `place_opt -from final_place` 命令做增量式的相对布局与优化。需要注意，如果被选中纳入同一相对布局组的单元，在初始布局中彼此相距较远，这种增量式相对布局可能会拖累 QoR。

```
fc_shell> create_rp_group -name new_rp -columns 1 -rows 2
fc_shell> add_to_rp_group new_rp -cells U1 -column 0 -row 0
fc_shell> add_to_rp_group new_rp -cells U2 -column 0 -row 1
...
fc_shell> place_opt -from final_place
```

如果是在已布局设计中新建组，并且通过 `set_rp_group_options` 的 `-x_offset`、`-y_offset` 为其指定了锚点，也可以直接用 `legalize_rp_groups` 命令，把该新建组 `legalize` 到指定坐标上。相比重新跑一遍完整的 `place_opt`，这种增量式做法能显著缩短运行时间。

10.9 分析相对布局组

Analyzing Relative Placement Groups

10.9.1 布局前检查约束

Checking Relative Placement Groups Before Placement

在正式运行布局与优化之前，可以用 `check_rp_constraints` 命令，提前检查相对布局约束中可能导致布局失败（无论是致命的还是非致命的）的问题。

```
fc_shell> check_rp_constraints rp_volt1
```

该命令会输出一份摘要，列出被检查的顶层相对布局组总数，以及其中可能无法满足约束的组数；对于存在潜在问题的组，还会给出具体的告警说明（例如某组的高度超过了其所在电压域或独占移动边界的高度限制）。

10.9.2 分析相对布局组的可布局性

Analyzing the Placeability of a Relative Placement Group

在正式布局之前，还可以用 `check_rp_constraints -analyze_placement` 命令，判断某个相对布局组究竟能否被成功放置：

- 用 `-region` 或 `-trials` 选项，把分析范围限制在 `core` 区域中的某个区域，或某个数量的随机采样点内；
- 用 `-no_pdc`、`-no_adv`、`-no_rp_constraints` 选项，分别在分析时忽略物理设计约束、高级设计规则或相对布局约束；
- 用 `-threshold` 选项，只报告那些“可放置比例”低于指定阈值的组——即在被分析的全部采样位置中，该组实际可以合法放置的位置占比低于此阈值时才会被列出。

10.9.3 报告相对布局约束违例

Reporting Relative Placement Constraint Violations

布局与优化完成之后，可以用 `report_rp_groups` 命令识别布局问题和相对布局约束违例。使用该命令时，必须显式指定要分析的组名列表，或者用 `-all` 选项分析全部相对布局组。

默认情况下，该命令会报告以下四类组：已放置且无违例的组、已放置但存在非致命违例的组、因致命违例而放置失败的组，以及尚未放置的组。可以用表 10.5 中的选项，改变默认的报告范围。

表 10.5: report_rp_groups 常用选项

选项	作用
-critical	仅报告因致命相对布局违例而未能放置的组
-non_critical	仅报告已放置但不满足相对布局约束的组
-unplaced	仅报告尚未放置的组
-verbose	输出更详细的信息

该命令也可以在布局与优化之前运行，用来提前确认设计中哪些相对布局组尚未被放置。

10.9.4 查询相对布局组

Querying Relative Placement Groups

用 `get_rp_groups` 命令，可以按对象或属性值查询相关的相对布局组。例如，查询设计中全部相对布局组：

```
fc_shell> get_rp_groups
{RP_TA RP_TCO RP_HO_1 RP_HO_2 RP_HO_3 RP_HO_4 RP_THO}
```

只查询顶层相对布局组，可以加 `-top` 选项：

```
fc_shell> get_rp_groups -top
{RP_TA RP_TCO RP_THO}
```

查询包含某个 leaf cell 的相对布局组，可以用 `-of_objects` 选项：

```
fc_shell> get_rp_groups -of_objects U129
{RP_HO_4}
```

10.9.5 在 GUI 中分析相对布局

Analyzing Relative Placement in the GUI

Fusion Compiler 图形界面提供了专门用于可视化、分析相对布局组的工具，包括 Relative Placement (RP) Groups Visual Mode 与 Relative Placement (RP) Net Connection Visual Mode 两种可视化模式。更详细的用法说明请参阅《Fusion Compiler Graphical User Interface User Guide》中关于 Map 与 Visual Modes 的相关主题。

10.10 保存相对布局信息

Saving Relative Placement Information

当使用 `save_lib` 保存设计库时，相对布局信息会随之自动保存到设计库数据库中。

除此之外，也可以用 `write_rp_groups` 命令，把相对布局组及其所含对象、选项设置导出为一份可重新执行的 Tcl 脚本文件，用 `-file_name` 指定输出文件名。使用该命令时，必须显式指定要导出的相对布局组，或者用 `-all` 导出全部相对布局组。

默认情况下，`write_rp_groups` 只会写出用于创建指定组、以及向其中添加 leaf cell、层次化子组与 blockage 的命令，但不会写出用于生成层次化子组自身内部结构的命令。可以用表 10.6 中的选项调整默认行为。

10.11 相对布局命令汇总

Summary of Relative Placement Commands

表 10.7 汇总了本章涉及的主要相对布局命令及其对应小节，便于查阅。

表 10.6: write_rp_groups 常用选项

选项	作用
-hierarchical	把相对布局层次结构中所有层级的组都写出；若省略该选项，只写出顶层组，不写子组
-create	只写出 create_rp_group 命令
-cell	只写出 add_to_rp_group -cells 命令
-rp_group	只写出 add_to_rp_group -rp_group 命令
-blockage	只写出 add_to_rp_group -blockage 命令

表 10.7: 相对布局主要命令一览

命令	对应小节
create_rp_group, remove_rp_groups	创建相对布局组
add_to_rp_group, remove_from_rp_group	向组中添加对象
set_rp_group_options, remove_rp_group_options	设置相对布局组选项
modify_rp_groups	修改相对布局组结构
create_clock_rp_groups	为时钟 Sink 生成相对布局组
legalize_rp_groups	对已布局设计做相对布局组合法化
check_rp_constraints	布局前检查约束 / 分析可布局性
report_rp_groups	报告相对布局约束违例
get_rp_groups	查询相对布局组
write_rp_groups	保存相对布局信息

参见

关于设计准备、放置约束设置、通用布局与优化流程，请参阅前述各章内容。

Chapter 11

层次化实现

Hierarchical Implementation

本章说明如何在层次化设计（hierarchical design）上执行综合（synthesis）、布局（placement）、优化（optimization）、时钟树综合（clock tree synthesis）、布线（routing）以及后布线优化（postroute optimization）。内容涵盖：

- 使用 Abstract 进行层次化综合
- 使用 ETM 进行层次化综合
- 层次化综合流程的预检查
- 提供块级测试模型（test model）
- 指定块级功耗意图（power intent）
- Abstract 视图概述
- 创建 Abstract 视图
- 报告 Abstract 纳入原因
- 创建 Abstract 之后修改 block
- 创建 Frame 视图
- 在顶层链接 Abstract 视图
- 将 block 从 Abstract 切换为 Design 视图
- 链接带多个 label 的子块
- 从顶层指定 block 的可编辑性
- 使用 Abstract 做顶层收敛前的准备
- 检查含 Abstract 设计的顶层收敛问题
- 使用 Abstract 视图执行顶层收敛

- 创建 ETM 与 ETM cell library
- 在顶层链接 ETM
- 使用 ETM 执行顶层收敛
- 透明层次优化（Transparent Hierarchy Optimization, THO）

11.1 使用 Abstract 进行层次化综合

Performing Hierarchical Synthesis Using Abstracts

基于 Abstract 的层次化综合流程主要包括以下步骤：

1. 在顶层对设计进行划分（partition），并为下层 block 生成设计库（design library），详见「全芯片设计的划分与规划」。
2. 对每个下层 block 进行综合，并生成顶层综合所需的块级信息，详见「综合子块」。
3. 对顶层进行综合，详见「综合顶层」。

参见

- Overview of Abstract Views（Abstract 视图概述）
- Creating Abstract Views（创建 Abstract 视图）
- Creating a Frame View（创建 Frame 视图）

11.1.1 全芯片设计的划分与规划

Partitioning and Planning the Full Chip Design

基于 Abstract 的层次化综合流程包含以下两种情形：

- 子块与顶层设计的 floorplan 尚不可用；
- 子块与顶层设计的 floorplan 已可用。

Floorplan 不可用时的层次化综合流程

Hierarchical Synthesis Flow When Floorplans are not Available

当子块与顶层设计的 floorplan 尚不可用时，可按以下步骤对顶层做划分与规划，并创建后续自底向上（bottom-up）综合所需的下层 block：

1. 读入全芯片设计并施加约束，例如：

```
fc_shell> set REF_LIBS "stdcell.ndm macro.ndm"
fc_shell> create_lib TOP -technology techfile \
    -ref_libs $REF_LIBS
fc_shell> analyze -format verilog $rtl_files
fc_shell> elaborate TOP
fc_shell> set_top_module TOP
```

(原书 Figure 187: Hierarchical Synthesis Flow When Floorplans are not Available)

请参阅原版 PDF 插图。

图 11.1: Floorplan 不可用时的层次化综合流程

```
fc_shell> load_upf fullchip.upf
fc_shell> read_sdc fullchip.sdc
```

2. 识别设计划分并拆分约束，例如：

```
fc_shell> set_budget_options -add_blocks {BLOCK1 BLOCK2}
fc_shell> split_constraints
```

3. 创建带设计信息的子块设计库，并启用按块配置参考库（reference library）的设置，例如：

```
fc_shell> copy_lib -to_lib BLOCK1.nlib -no_design
fc_shell> copy_lib -to_lib BLOCK2.nlib -no_design
fc_shell> set_attribute -objects BLOCK1.nlib \
    -name use_hier_ref_libs -value true
fc_shell> set_attribute -objects BLOCK2.nlib \
    -name use_hier_ref_libs -value true
fc_shell> save_lib -all
```

4. 创建子块设计划分，例如：

```
fc_shell> commit_block -library BLOCK1.nlib BLOCK1
fc_shell> commit_block -library BLOCK2.nlib BLOCK2
fc_shell> save_lib -all
```

5. 加载此前由 split_constraints 为未映射子块与顶层生成的 UPF 与 SDC 约束，例如：

```
fc_shell> set_constraint_mapping_file ./split/mapfile
fc_shell> load_block_constraints -all_blocks -type SDC \
    -type UPF -type CLKNET
fc_shell> save_lib -all
```

6. 运行 compile_fusion，直到子块的逻辑优化（logic optimization）与自动 floorplanning 完成，例如：

```
fc_shell> compile_fusion -to logic_opto
```

7. 运行 compile_fusion，直到顶层设计的工艺映射（technology mapping）完成，例如：

```
fc_shell> compile_fusion -to initial_map
```

8. 执行设计规划（design planning）操作：从 floorplan 初始化、子块与电压域（voltage area）塑形、硬宏与标准单元布局、电源网络创建，直到 pin assignment。
9. 运行 compile_fusion，直到顶层设计的逻辑优化完成，例如：

```
fc_shell> compile_fusion -from logic_opto -to logic_opto
```

10. 仅对顶层做增量标准单元布局，例如：

```
fc_shell> create_placement -floorplan -use_seed_locs
```

11. 执行时序估计（timing estimation）与预算（budgeting），生成块级实现所需的 block budget。
12. 完成设计规划后，打开 elaborated RTL NDM，加载 floorplan、feedthrough 与 budget，重建子块与顶层设计。

Floorplan 可用时的层次化综合流程

Hierarchical Synthesis Flow When Floorplans are Available

当子块与顶层设计的 floorplan 已可用时，可按以下步骤对顶层做划分与规划，并创建后续自底向上综合所需的下层 block：

（原书 *Figure 188: Hierarchical Synthesis Flow When Floorplans are Available*）
请参阅原版 PDF 插图。

图 11.2: Floorplan 可用时的层次化综合流程

1. 读入全芯片设计并施加约束，例如：

```
fc_shell> set REF_LIBS "stdcell.ndm macro.ndm"
fc_shell> create_lib TOP -technology techfile \
    -ref_libs $REF_LIBS
fc_shell> analyze -format verilog $rtl_files
fc_shell> elaborate TOP
fc_shell> set_top_module TOP

fc_shell> load_upf fullchip.upf
fc_shell> read_sdc fullchip.sdc
```

2. 对设计划分拆分约束，例如：

```
fc_shell> set_budget_options -add_blocks {BLOCK1 BLOCK2}
fc_shell> split_constraints
```

3. 创建带设计信息的子块设计库，并启用按块配置参考库的设置，例如：

```
fc_shell> copy_lib -to_lib BLOCK1.nlib -no_design
fc_shell> copy_lib -to_lib BLOCK2.nlib -no_design
fc_shell> set_attribute -objects BLOCK1.nlib \
    -name use_hier_ref_libs -value true
fc_shell> set_attribute -objects BLOCK2.nlib \
    -name use_hier_ref_libs -value true
fc_shell> save_lib -all
```

4. 创建子块设计划分，例如：

```
fc_shell> commit_block -library BLOCK1.nlib BLOCK1
fc_shell> commit_block -library BLOCK2.nlib BLOCK2
fc_shell> save_lib -all
```

5. 加载此前由 `split_constraints` 为未映射子块与顶层生成的 UPF 与 SDC 约束，例如：

```
fc_shell> set_constraint_mapping_file ./split/mapfile
fc_shell> load_block_constraints -all_blocks -type SDC \
    -type UPF -type CLKNET
fc_shell> save_lib -all
```

6. 加载子块与顶层设计的 floorplan。

7. 运行 `compile_fusion`，直到子块与顶层设计的逻辑优化完成，例如：

```
fc_shell> compile_fusion -to logic_opto
```

8. 执行标准单元布局，随后进行 pin assignment、时序估计与预算。

9. 完成设计规划后，打开 elaborated RTL NDM，加载 floorplan、feedthrough 与 budget，重建子块与顶层设计。

注意

各项设计规划操作的细节，请参阅 *Design Planning User Guide*。

11.1.2 综合子块

Synthesizing a Subblock

要综合某个子块并生成顶层综合所需的块级信息，执行以下步骤：

1. 读入「全芯片设计的划分与规划」中 rebuild 步骤后生成的子块设计库。
2. 施加综合该 block 所需的任何块级专用约束与设置。
3. 使用 `set_dft_signal`、`set_scan_configuration` 等命令施加 DFT 设置，并用 `create_test_protocol` 创建测试协议（test protocol）。
4. 插入 DFT 逻辑并综合该 block：

```
fc_shell> compile_fusion -to logic_opto
fc_shell> insert_dft
fc_shell> compile_fusion -from initial_place
```

5. 创建只读 Abstract 视图、Frame 视图与测试协议，并保存 block:

```
fc_shell> create_abstract -read_only
fc_shell> create_frame
fc_shell> write_test_model BLOCK1.ct1
fc_shell> save_block -as BLOCK1/PostSynthesis
```

11.1.3 综合顶层

Synthesizing the Top-Level

要综合顶层，执行以下步骤：

1. 读入「全芯片设计的划分与规划」中 rebuild 步骤后生成的顶层设计。
2. 设置合适选项并链接顶层设计，例如：

```
fc_shell> set_label_switch_list \
    -reference {BLOCK1 BLOCK2} PostCompile
fc_shell> set_attribute -objects {BLOCK1.nlib BLOCK2.nlib} \
    -name use_hier_ref_libs -value true
fc_shell> set_top_module TOP
```

若子块已用 label 保存在设计库中，用 set_label_switch_list 指定要链接的 label；set_top_module 用于链接指定的顶层模块。

3. 施加综合顶层所需的任何专用约束与设置，并读入子块的测试模型，例如：

```
fc_shell> read_test_model BLOCK1.ct1
fc_shell> read_test_model BLOCK2.ct1
fc_shell> create_test_protocol
```

4. 插入 DFT 逻辑并综合顶层设计：

```
fc_shell> compile_fusion -to initial_opto
fc_shell> insert_dft
fc_shell> compile_fusion -from initial_place
```

11.2 使用 ETM 进行层次化综合

Performing Hierarchical Synthesis Using ETMs

在层次化流程中，block 设计以带 Frame 视图的宏（macro）形式表示；已实现的 block 则以提取时序模型（extracted timing model, ETM）的形式，用于设计的顶层实现。

ETM 是设计的 Liberty 模型表示，由 PrimeTime 工具中的 extract_model 命令生成。该命令用相关 Liberty 属性捕获设计的时序与功耗信息，示例如下：


```

library("block1") {
...
comment : "PrimeTime Extracted Model." ;
    voltage_map("nom_voltage", 1.08);
    voltage_map("VDD", 1.08);
...
cell( block1 ) {
timing_model_type : "extracted";
...
pg_pin("VDD") {
    voltage_name : "VDD";
    pg_type : primary_power;
}
...
}

```

本节主题包括:

- 使用 ETM 运行层次化综合可行性分析流程
- 使用 ETM 运行层次化综合流程
- 使用 ETM 的层次化综合示例脚本

参见

- SolvNet 文章 2284429, *Top-level Closure Flow With Extracted Timing Models (ETMs)*
- Creating ETMs and ETM Cell Libraries (创建 ETM 与 ETM cell library)
- Linking to ETMs at the Top Level (在顶层链接 ETM)

11.2.1 使用 ETM 运行层次化综合可行性分析流程

Running the Hierarchical Synthesis Feasibility Analysis Flow Using ETMs

为减少迭代次数并获得更优结果,可在运行层次化综合之前先做可行性分析(feasibility analysis),如图 11.3 所示。

层次化综合可行性流程使用 `compile_fusion` 之后生成的下列块级信息:

- Floorplanning 信息,用于步骤 2 中生成 ETM;
- 功耗意图与测试模型(CTL)信息,用于步骤 4 的顶层综合。

11.2.2 使用 ETM 运行层次化综合流程

Running the Hierarchical Synthesis Flow Using ETMs

按图 11.4 所述步骤运行层次化综合流程。与可行性流程不同,该流程使用 `place and route` 之后生成的下列块级信息:

- Floorplanning 信息,用于步骤 3 中生成 ETM;

(原书 *Figure 189: Hierarchical Synthesis Feasibility Analysis*)

流程示意: 1) 前端工具综合 block (`compile_fusion`); 2) 前端工具创建 ETM (`extract_model`);
3) 前端工具综合顶层 (`compile_fusion`)。请参阅原版 PDF 插图。

图 11.3: 层次化综合可行性分析

- 功耗意图与测试模型 (CTL) 信息, 用于步骤 5 的顶层综合。

(原书 *Figure 190: Hierarchical Synthesis Flow*)

流程示意: 1) 前端综合 block; 2) 后端实现 block; 3) 后端创建 ETM; 4) 前端综合顶层。请参阅原版 PDF 插图。

图 11.4: 层次化综合流程

11.2.3 使用 ETM 的层次化综合示例脚本

Example Script of Hierarchical Synthesis Using ETMs

以下脚本给出层次化综合流程示例:

```
# Specify reference libraries
set REF_LIBS ${std_cell}.ndm
lappend REF_LIBS ${block_design}_etm.ndm
create_lib -tech ${TECH_FILE} -ref_libs $REF_LIBS ${DESIGN_NAME}

# Read the RTL design
analyze -format verilog [list $rtl_list]
elaborate ${DESIGN_NAME}

#ETM.ndm in ref_lib is used to link cell instances
set_top_module ${DESIGN_NAME}

# Load UPF for ETM cell instance
load_upf ${block_design}.upf -scope cell_instance

# Read test model for ETM cell instance
read_test_model ${block_design}.mapped.ctl
```

```
...
compile_fusion -to initial_opto

# Query commands
# List all macro cell instances
get_cells -hierarchical -filter "is_hard_macro == true"
# ref view name is "timing" for cell instances bound to ETM
get_attribute ${CELL} is_etm_moded_cell
```

11.3 层次化综合流程的预检查

Performing Prechecks for Hierarchical Synthesis Flows

要检查层次化综合流程的就绪程度，可使用下列命令：

- `check_design -checks block_ready_for_top`
在 block 综合之后运行，检查已综合的 block 是否已准备好用于顶层综合。
- `check_design -checks hier_pre_compile`
在顶层综合之前运行，检查子块与顶层设计是否已准备好进入层次化流程。

11.4 提供块级测试模型

Providing Block-Level Test Models

在运行层次化综合时，可为顶层设计实现提供子块的测试模型。做法是：在块级综合期间为已映射的 block 生成测试模型，再在顶层综合时将该测试模型注解到子块。例如：

- 在块级综合期间生成测试模型：

```
# Read block RTL and constraints
# Specify DFT configuration
set_dft_signal ...
set_scan_configuration ...
create_test_protocol
...
compile_fusion -to initial_opto
insert_dft
write_test_model -output block_des.mapped.ctl
```

- 在顶层综合中使用块级测试模型：

```
# Read RTL and constraints
# Specify DFT configuration

read_test_model block_des.mapped.ctl
set_dft_signal ...
set_scan_configuration ...
create_test_protocol
```

```
...
compile_fusion -to initial_opto
insert_dft
```

11.5 指定块级功耗意图

Specifying Block-Level Power Intent

在运行层次化综合时，必须为顶层设计实现提供子块的功耗意图。可提供完整的块级 UPF，由工具自动提取功耗意图接口。下图示意名为 U1 的子块；顶层功耗意图脚本用 `load_upf` 加载块级功耗意图。

注意

块级功耗意图规范仅在使用 ETM 的层次化综合中必需。在 Abstract 流程中，UPF 约束由工具自动处理。

- 在 PD_Top 设计中将 U1 作为 ETM
- 顶层功耗意图：

```
create_power_domain PD_TOP -include_scope
create_supply_net VDD_A0
...
load_upf block.upf -scope U1
...
connect_supply_net VDD_A0 -port U1/VDD_A0
```

- 块级功耗意图：block.upf

```
create_power_domain PD_BLK -include_scope
create_power_switch sw1 -domain PD_BLK \
  -input_supply_port {in VDD_A0} \
  -output_supply_port {out VDD_SW} ...
...
add_port_state sw1/out -state {ON 1.08} -state {OFF off}
...
create_pst pst -supplies {VDD_SW ...}
```

11.6 Abstract 视图概述

Overview of Abstract Views

在 Abstract 视图中，block 的门级网表由仅包含所需接口逻辑（interface logic）的部分门级网表建模；其余逻辑全部移除。

图 11.5 显示一个 block 及其 Abstract 视图，其中保留了：

- 各时序路径上输入端口到第一级寄存器之间的逻辑；

- 各时序路径上最后一级寄存器到输出端口之间的逻辑。

与纯组合逻辑输入端口到输出端口时序路径（A 到 X）相关的逻辑也会保留。被保留寄存器的时钟连接同样保留。寄存器到寄存器（register-to-register）逻辑则丢弃。



图 11.5: Block 及其 Abstract 视图

Abstract 视图的接口逻辑包括：

- 从输入端口到寄存器或输出端口的时序路径上的所有 cell、pin 与 net；
- 从寄存器或输入端口到输出端口的时序路径上的所有 cell、pin 与 net；
- 从主时钟（master clock）到生成时钟（generated clock）连接中的任何逻辑；
- 驱动接口寄存器的时钟树，包括时钟树中的任何逻辑；
- 从时钟端口出发的最长与最短时钟路径；
- 属于接口逻辑、且带有时序约束的所有 pin。

除接口逻辑外，Abstract 视图还包含与接口逻辑相关的下列信息：

- 布局信息（placement information）
- 时序约束（timing constraints）
- 时钟树例外（clock tree exceptions）
- 寄生参数信息（parasitic information）
- 悬空 pin（dangling pin）的 transition 与 case value
- 功耗管理单元（power management cell）与 UPF 约束
- Abstract 视图中保留的 net 上的非默认布线规则（nondefault routing rule）关联，以及最小/最大层约束（若有）

11.7 创建 Abstract 视图

Creating Abstract Views

在 Fusion Compiler 中，可在设计流程的多个阶段为 block 创建 Abstract 视图，例如综合、布局、优化、时钟树综合、布线之后等。创建 Abstract 视图之前，请确保顶层所需的 scenario 已创建且处于活动（active）状态。

要为顶层收敛创建 Abstract 视图，使用 `create_abstract` 命令。

- 用 `-timing_level` 控制 Abstract 视图中时序信息的多少，可选设置如下：
 - **boundary**: 创建仅包含连接到各边界端口的边界单元（一层逻辑）的 Abstract。此类 Abstract 还包含 `feedthrough` 数据路径、`feedthrough` 组合时钟路径，以及驱动输出端口的内部时钟逻辑。可用于在顶层修复 DRC 违规。
 - **compact**: 创建紧凑 Abstract，仅包含接口逻辑中关键 `setup/hold` 时序路径的时序信息。这是默认设置，也是顶层设计收敛的首选。
 - **full_interface**: 为全部接口逻辑创建带时序信息的 Abstract。当多重例化块（multiply-instantiated block, MIB）在顶层的约束对各实例不同时，可能需要 `full_interface` Abstract。
- 要为当前 block 的下层 block 创建 Abstract，可用下列方法之一：
 - 用 `-blocks` 并指定 block 名，为特定下层 block 创建 Abstract；
 - 用 `-all_blocks` 为所有下层 block 创建 Abstract。

使用 `-blocks` 或 `-all_blocks` 时，若指定 block 已有 Abstract 视图，默认不会重新创建；要强制重建，使用 `-force_recreate`。
- 要用 `-include_objects` 把特定 net、port、层次 pin 或 leaf pin 纳入布局或时序 Abstract。
- 用 `-read_only` 创建只读 Abstract。
默认情况下，工具创建可编辑（editable）Abstract，可在父 block 上下文中修改；但会把该 block 的 design 视图设为只读，防止对 block 做改动。
若创建只读 Abstract，则该 block 的 design 视图仍可编辑，可继续对其修改。
- 用 `-preserve_block_instances false` 展平物理层次，仅保留下层 block 中相关逻辑。
- 用 `-target_use planning` 或 `-target_use implementation` 指定 Abstract 用于设计规划还是顶层实现。工具会按预期用途对 Abstract 施加相应内部设置。
- 用 `-host_options` 指定分布式处理的主机选项。当用 `-blocks` 或 `-all_blocks` 创建多个 Abstract 视图时，可借此缩短运行时间。

11.7.1 创建含功耗信息的 Abstract

Creating Abstracts With Power Information

要在块级创建含对应 block 功耗信息的 Abstract，按以下步骤操作：

1. 用 `set_scenario_status` 的 `-active true` 激活要进行功耗分析的 scenario，并用下列选项启用功耗分析：
 - `-dynamic_power true`: 动态功耗分析
 - `-leakage_power true`: 漏电功耗分析

功耗信息仅对已启用动态或漏电功耗的活动 scenario 存储。
2. (可选) 通过 `read_saif` 读入 SAIF，或用 `set_switching_activity` 在 net 上注解开关活动（switching activity）。更多说明见 [Annotating the Switching Activity](#)。
3. 将应用选项 `abstract.annotate_power` 设为 `true`，以便在 Abstract 中存储功耗信息。

4. 用 `create_abstract` 为该 block 创建 Abstract。

创建带功耗信息的 Abstract 时，工具只为被移除的逻辑存储功耗信息。

在顶层例化此类 Abstract 并做功耗分析时，工具会：

- 根据顶层看到的开关活动与负载，在上下文中重新计算 Abstract 内接口逻辑的功耗；
- 对被 Abstract 创建时移除的逻辑，使用 Abstract 中已存储的功耗。

要用 `report_power -blocks` 报告顶层例化 Abstract 中的功耗信息。若存在多层物理层次，可用 `-levels` 指定要报告的层数。

11.7.2 为信号电迁移分析创建 Abstract

Creating Abstracts for Signal Electromigration Analysis

要创建可用于顶层信号电迁移 (signal electromigration) 分析的 Abstract, 在运行 `create_abstract` 之前，将应用选项 `abstract.enable_signal_em_analysis` 设为 `true`。

11.7.3 处理多层物理层次

Handling Multiple Levels of Physical Hierarchy

对于多层物理层次设计，若要为内部已例化 Abstract 的 block 再创建 Abstract，须先：

- 将希望用 Abstract 表示的下层 block 绑定到特定 Abstract。

例如，假设设计具有如下多层物理层次：

(原书 *Figure 192: A Design With Multiple Levels of Physical Hierarchy*)

BOT / MID / TOP 层次关系示意，请参阅原版 PDF 插图。

图 11.6: 多层物理层次设计

要为名为 BOT 的 block 创建 Abstract：

```
fc_shell> open_block BOT
fc_shell> create_abstract
```

要为名为 MID 的 block 创建 Abstract：

```
fc_shell> open_block MID
fc_shell> change_abstract -view abstract -references BOT
fc_shell> create_abstract
```

默认情况下，`create_abstract` 保留所有物理层次。要用 `-preserve_block_instances false` 展平物理层次并仅保留下层 block 的相关逻辑，例如：

```
fc_shell> open_block MID
fc_shell> create_abstract -preserve_block_instances false
```

在下层 block 按只读处理的顶层实现流程中使用展平 Abstract。展平 Abstract 可减小顶层网表规模，并简化数据管理——无需把下层 block 放进顶层的参考库列表。

11.7.4 为最小脉宽分析创建 Abstract

Creating Abstracts for Minimum Pulse Width Analysis

在顶层时序 Abstract 中，Fusion Compiler 支持检查最小脉宽（minimum pulse width，MPW）违规。创建 Abstract 时，工具会保留存在 MPW 违规的时钟 sink，及其扇入 cell 与 net。

可用下列应用选项：

- 将 abstract.keep_min_pulse_width_violation_pins 设为 true，检查并保留带 MPW 违规的寄存器及其完整时钟路径。这些寄存器的数据路径不会保留。该选项默认值为 false。
- 将 abstract.min_pulse_width_violation_slack_limit 设为所需 slack，以限制纳入的 MPW 违规范围。这有助于保留尚未违规、但 slack 略为正的 MPW pin。

11.8 报告 Abstract 纳入原因

Reporting Abstract Inclusion Reasons

在 Fusion Compiler 中，create_abstract 会在顶层与块级 Abstract 视图中纳入 cell、net、pin 等网表对象，并报告纳入原因。

要用 report_abstract_inclusion_reason 报告特定网表对象被纳入 Abstract 视图的原因；该命令可覆盖 Abstract 中各层次的对象。

每条原因会显示一个原因码（reason code），对应有效的纳入理由。同一网表对象可有多个原因。

注意

report_abstract_inclusion_reason 不会报告层次 cell，或层次 cell 上 pin 的纳入原因。

可在例化了 Abstract 的顶层设计上做原因报告，无需把 Abstract 设为当前 block。

下例报告 Abstract 视图中 pin 的纳入原因：

```
fc_shell> report_abstract_inclusion_reason [get_pins -of_object \
[get_cells -filter "is_hierarchical==false" -hierarchical *]]

Legend
mv - MV Logic
comp - Compact interface logic
cell - Cell inclusion

-----
Pin name                                Reason(s) for inclusion
-----
AINV_P_260/I                            comp, cell
AINV_P_260/VDD                           cell
```


AINV_P_260/VSS	cell
AINV_P_260/ZN	comp, cell
AINV_P_262/I	comp, cell
AINV_P_262/VDD	cell
AINV_P_262/VSS	cell
AINV_P_262/ZN	comp, cell
AINV_P_263/I	mv, comp, cell

报告开头会显示图例（legend），说明各原因码的详细含义。

11.9 创建 Abstract 之后修改 Block

Making Changes to a Block After Creating an Abstract

用 `create_abstract` 为 block 创建 Abstract 视图时，默认会同时保存该 block 的 Abstract 与 design 视图，并把 design 视图改为只读。

可以打开只读 block 并在内存中修改，但不能用 `save_block` 或 `save_lib` 保存这些修改。要保存对只读 block 的修改，可用下列方法之一：

- 用 `save_block -as` 另存为不同的 block；
- 用 `create_abstract` 为该 block 重新创建 Abstract，工具会自动保存 design 视图；
- 若不再需要 Abstract，用 `remove_abstract` 删除它。这会把 design 视图从只读改回可编辑，随后可用 `save_block` 或 `save_lib` 保存。

若用 `create_abstract -read_only` 创建只读 Abstract，工具不会把该 block 的 design 视图设为只读，后续修改可用 `save_block` 或 `save_lib` 保存，例如：

```
fc_shell> create_abstract -read_only
fc_shell> change_link [get_cells U25] AND2
fc_shell> save_block
```

11.10 创建 Frame 视图

Creating a Frame View

要执行顶层布线（含 virtual routing），每个 Abstract 视图都必须有对应的 Frame 视图。用 `create_frame` 创建 Frame 视图；该命令从 design 视图提取 blockage、pin 与 via 信息。

创建 Frame 视图后，用 `save_lib` 保存设计库。

11.11 在顶层链接 Abstract 视图

Linking to Abstract Views at the Top-Level

要使顶层设计链接到下层 block 的 Abstract 视图，包含该 Abstract 视图的设计库必须是顶层的参考库之一。

例如，假设顶层设计名为 TOP，并例化了两个下层 block BLK1 与 BLK2，如图 11.7 所示。

(原书 *Figure 193: Top-Level Design With Instantiated Blocks*)

TOP 例化 BLK1、BLK2 示意，请参阅原版 PDF 插图。

图 11.7: 例化了 Block 的顶层设计

要链接到 BLK1 与 BLK2 的 Abstract，包含这些 Abstract 视图的设计库必须是 TOP 的参考库。若尚未加入，可用 `set_ref_libs -add` 添加，例如：

```
current_block TOP
set_ref_libs -add ../BLK1/BLK1.nib
set_ref_libs -add ../BLK2/BLK2.nib
```

要用 `report_ref_libs` 报告当前设计的参考库。

链接顶层设计时，若下层 block 已有 Abstract 视图，工具默认链接到该 Abstract。链接下层 block 时各视图的默认优先级为：

1. Abstract 视图
2. Design 视图
3. Frame 视图
4. Outline 视图

11.12 将 Block 从 Abstract 切换为 Design 视图

Changing the Block From Abstract to Design View

要用 `change_abstract`，并通过 `-view` 指定目标视图，把 block 从 Abstract 切换为 design 视图。从 Abstract 切到 design 视图时，应移除现有约束并施加全芯片时序约束。

下例将 BLK1 从 Abstract 切到 design 视图，并施加全芯片时序约束。

1. 移除现有时序约束：

```
fc_shell> remove_scenarios -all
fc_shell> remove_modes -all
fc_shell> remove_corners -all
```

2. 用 `change_abstract` 从 Abstract 切到 design 视图：

```
fc_shell> change_abstract -view design -references BLK1
```

3. 执行全芯片 scenario 创建脚本：

```
fc_shell> source full_chip_scenario_creation.tcl
```

若 block 在其设计库中的 Abstract 视图已更新，可用 `change_abstract -reload` 重新加载新的 Abstract。

要用 `report_abstracts` 报告设计当前链接到的 Abstract 视图。

关于指定参考库以及在顶层链接 Abstract 视图的更多信息，见 SolvNet 文章 2436119，*Setting Up Designs for Hierarchical Place and Route*。

11.13 链接带多个 Label 的子块

Linking to Subblocks With Multiple Labels

在实现流程的不同阶段保存 block 时，可用 `save_block` 的 `-as blockName/labelName` 或 `-label labelName`，为每次保存指定唯一 label。

默认情况下，链接时顶层会链接到具有相同 label 的下层 block。若子块已用多个 label 保存，可用 `set_label_switch_list` 指定顶层应链接到哪个 label。该命令给出链接时按优先级排序的 label 列表。默认作用于所有子块；要用 `-reference` 指定不同的 block。

下例指定顶层应链接到 BLOCK1 与 BLOCK2 的 PostCompile label：

```
fc_shell> set_label_switch_list \
    -reference {BLOCK1 BLOCK2} PostCompile
```

下例指定：在顶层链接任意子块时，PostCompile 优先于 PreCompile：

```
fc_shell> set_label_switch_list {PostCompile PreCompile}
```

可再次使用 `set_label_switch_list` 更新 block 的 label 切换列表，再用 `link_block -rebind` 重新链接。重新链接时注意：

- 若切换列表中的 label 在某子块均不可用，或指定了空切换列表，工具会把该子块链接到其先前已链接的 label；
- 若已移除某子块的全部 label，工具会把该子块链接到其父 block 的 label。

11.14 从顶层指定 Block 的可编辑性

Specifying the Editability of Blocks From the Top-Level

对于带物理层次的设计，可在顶层用 `set_editability` 指定下层 block 是否可修改。

可更改：

- 用 `-blocks` 指定的特定 block；
- 用 `-from_level` 指定从某一物理层次起的全部 block；
- 用 `-to_level` 指定到某一物理层次为止的全部 block。

对于使用只读 Abstract 的顶层实现流程，链接设计后应显式把下层 block 设为只读：

```
fc_shell> set_editability -blocks [get_blocks -hierarchical] \
    -value false
```

默认情况下,若用下列命令之一更改库名或 block 名,工具不会把可编辑性设置从旧参考传播到新参考:

- `change_abstract -lib`
- `link_block -rebind -force`
- `set_reference`

要保留可编辑性设置,在更改库名或 block 名之前,将应用选项 `design.preserve_reference_editability` 设为 `true`。

用 `save_block` 保存层次化设计时,可编辑性设置会被保存。

11.15 使用 Abstract 做顶层收敛前的准备

Preparing for Top-Level Closure With Abstracts

在顶层执行综合、布局、优化、时钟树综合与布线之前,必须完成下列任务:

- 施加顶层时序约束与设置。
可用 `split_constraints` 把芯片级约束拆成独立的顶层与块级约束。
- 用 `set_block_to_top_map` 审查并指定顶层与块级 `mode`、`corner`、`clock` 之间的对应关系。
- 若用于顶层布局与优化的是已经过 CCD 优化的预时钟树实现 block,则 CCD、clock-gate estimation 等引擎在 block 内寄存器或集成时钟门控 (integrated clock gating, ICG) 单元时钟 pin 上动态调整的理想时钟延迟 (ideal clock latency),在顶层可能看不到,从而导致顶层时序不准确。要用下列命令把这些时钟延迟约束从块级提升到顶层:

```
promote_clock_data -latency_offset -auto_clock connected
```

- 施加顶层时钟树综合设置与例外:
 - 若顶层例外未包含下层 block 内 pin 上的 balance point,用下列命令从下层提升 balance point:

```
promote_clock_data -auto_clock connected -balance_points
```

- 若在块级执行了并发时钟与数据 (concurrent clock and data, CCD) 优化,工具会为块级时钟端口推导中位时钟延迟 (median clock latency),以反映 CCD 期间为块级寄存器推导的延迟调整。要用下列命令把这些块级时钟延迟作为块级时钟 pin 的 clock balance point delay 提升:

```
promote_clock_data -auto_clock connected -port_latency
```

- 若下层 block 含 clock mesh,用 `promote_clock_data -mesh_annotations` 提升 mesh 注解 (已注解的 transition 与 delay)。

- 若只需为特定顶层时钟提升时钟数据,使用:

```
promote_clock_data -clocks {top_clock} -port_latency
```

该命令也会提升与时钟无关的例外。若给定顶层时钟在任一 `mode` 中都未映射到任何块级时钟,工具会发出警告。

- 若顶层存在无法映射到块级对应 mode/corner/clock 的活动 mode、corner 或 clock，仍可用下列命令为其他已映射的 mode/corner/clock 提升时钟数据：

```
promote_clock_data -balance_points -force
```

该命令会对未映射的 mode、corner 或 clock 发出错误。

- 施加顶层 UPF 约束；可用 `split_constraints` 从全芯片 UPF 约束生成。
工具会把下层 block 中所需的 UPF 约束提升到顶层。

11.16 检查含 Abstract 设计的顶层收敛问题

Checking Designs With Abstracts for Top-Level-Closure Issues

可用 `check_hier_design` 检查含 Abstract 的层次化设计，查找应用选项一致性问题，以及与顶层收敛相关的设计问题。在执行顶层收敛前识别并修复这些问题，有助于缩短周转时间（turnaround time）。

使用 `check_hier_design` 时可指定：

- 用 `-reference` 指定要检查的参考。若不指定，命令检查物理层次中的所有参考。
- 用 `-stage` 指定检查类型：
 - `-stage timing`：执行时序相关检查。若不指定 `-stage`，默认执行时序相关检查。
 - `-stage pre_placement`：同时执行时序与预布局相关检查。

`check_hier_design` 可检查顶层约束与每个链接到 Abstract 的下层实例约束之间的一致性。启用该功能时，在运行 `check_hier_design` 之前将应用选项 `abstract.check_constraints_consistency` 设为 `true`。

工具在 `create_abstract` 期间会保存预定列表中的 `timer` 应用选项设置。在顶层，`check_hier_design` 会将这些设置与块级设计中的对应设置进行比较。

下列应用选项会在 `create_abstract` 期间自动保存，并由 `check_hier_design` 校验：

- `time.case_analysis_propagate_through_icg`
- `time.case_analysis_sequential_propagation`
- `time.clock_gating_propagate_enable`
- `time.clock_gating_user_setting_only`
- `time.clock_marking`
- `time.clock_reconvergence_pessimism`
- `time.create_clock_no_input_delay`
- `time.crpr_remove_clock_to_data_crp`
- `time.delay_calc_waveform_analysis_mode`
- `time.delay_calculation_style`
- `time.disable_case_analysis_ti_hi_lo`

- `time.disable_clock_gating_checks`
- `time.disable_cond_default_arcs`
- `time.disable_internal_inout_net_arcs`
- `time.disable_recovery_removal_checks`
- `time.edge_specific_source_latency`
- `time.enable_auto_mux_clock_exclusivity`
- `time.enable_ccs_rcv_cap`
- `time.enable_clock_propagation_through_preset_clear`
- `time.enable_clock_propagation_through_three_state_enable_pins`
- `time.enable_clock_to_data_analysis`
- `time.enable_non_sequential_checks`
- `time.enable_preset_clear_arcs`
- `time.enable_si_timing_windows`
- `time.gclock_source_network_num_master_registers`
- `time.special_path_group_precedence`
- `time.use_lib_cell_generated_clock_name`
- `time.use_special_default_path_groups`

关于 `check_hier_design` 识别的问题及修复方法，请参阅对应消息 ID 的 man page。

除生成报告外，`check_hier_design` 还会生成增强消息系统（enhanced messaging system, EMS）数据库，可在 Fusion Compiler GUI 的消息浏览器中查看。应在运行 `check_hier_design` 之前创建 EMS 数据库，例如：

```
fc_shell> create_ems_database check_hier.ems
fc_shell> check_hier_design -stage timing
fc_shell> save_ems_database
```

在 Fusion Compiler GUI 消息浏览器中，可对消息排序、过滤，并链接到对应 man page，如图 11.8 所示。

（原书 *Figure 194: Viewing the EMS Database in the Message Browser*）

请参阅原版 PDF 插图。

图 11.8: 在消息浏览器中查看 EMS 数据库

也可用 `report_ems_database` 以 ASCII 格式输出 EMS 数据库中的信息。

`check_hier_design` 执行的检查同样可在 `check_design` 中使用。可以：

- 用 `check_design -checks hier_timing` 执行专用于顶层收敛的时序检查；
- 用 `check_design -checks timing` 执行全部时序检查（含顶层收敛专用检查）；
- 用 `check_design -checks hier_preplacement` 执行专用于顶层收敛的预布局检查；
- 用 `check_design -checks preplacement` 执行全部预布局检查（含顶层收敛专用检查）。

也可为 `check_design` 生成 EMS 数据库并在 GUI 中查看，方式与 `check_hier_design` 类似。

11.16.1 用 Early Data Check Manager 处理设计数据

Handling Design Data Using the Early Data Check Manager

在识别并修复顶层收敛相关问题的同时，工具允许在存在设计数据违规的情况下探索顶层流程。层次化检查可检测设计中各类不匹配、不完整与不一致的数据。可为层次化检查配置下列策略（policy）以管理设计数据违规：

- **Error:** 不缓解违规，而是发出错误消息。
- **Tolerate:** 做出可预期、可解释的假设以继续流程；不对设计或设置做更改。
- **Repair:** 用一种或多种修复策略缓解违规并记录。修复可包括对设计与设置的更改。
- **Strict:** 按检查所支持的策略，依次尝试 error、tolerate、repair。
- **Lenient:** 按检查所支持的策略，依次尝试 repair、tolerate、error。

注意

对错误的容忍（toleration）与修复（repair）可能影响顶层流程的 QoR。错误容忍与修复主要用于早期设计探索阶段管理数据违规。在实现流程中，应对所有检查使用 strict 策略。

处理早期设计数据的前提

Prerequisites for Handling Early Design Data

工具在 `check_hier_design` 命令内部执行修复，并把记录保存到子块中。对于需要重建 Frame 的修复，还需运行 `save_lib` 把 Frame 保存到磁盘。

要将修复及其记录成功保存到子块，需要下列设置：

1. 以可编辑方式打开包含子块的参考库：

```
open_lib top.nlib -ref_libs_for_edit
```

2. 对需要执行修复的子块启用可编辑性：

```
set_editability -blocks <> -value true
```

若需执行修复，`check_hier_design` 会检查所需可编辑性设置；若未正确设置，会发出 TL-170 错误消息。

注意

工具支持对链接到 `read_only Abstract`（而非可编辑 `Abstract`）的子块执行修复，因为管理不匹配设计数据的机制目标只是让顶层探索流程能继续，子块 `Abstract` 上的修复工作不应合并回实际 `design` 视图。若顶层使用可编辑 `design` 视图，修复工作仍会继续。

早期数据检查、策略与修复策略**Early Data Checks, Policies, and Strategies**

工具可在设计早期阶段识别数据问题与违规。可为预定义检查设置策略与策略配置，以允许、容忍或修复数据。表 11.1 描述了顶层收敛的预定义检查。

其中：

- BOT 表示底层设计；
- MID 表示例化 BOT 的中间层设计；
- TOP 表示例化 MID 的顶层设计。

为早期数据检查设置策略**Setting the Policy for Early Data Checks**

要用 `set_early_data_check_policy` 设置或修改全部检查的策略，或修改数据检查的策略设置。可用下列选项配置策略：

- `-checks`：指定预定义数据检查列表。检查名支持星号通配符（*）。
- `-policy`：按检查所支持的类型指定策略，可为：
 - `error`：若检查不支持 `error` 策略，工具发出错误消息。
 - `tolerate`：若检查不支持 `tolerate` 策略，工具发出错误消息。
 - `repair`：若检查不支持 `repair` 策略，工具发出错误消息。
 - `strict`：按下列顺序设置第一个匹配的策略：1. `error`；2. `tolerate`；3. `repair`。例如，若检查支持 `error` 与 `tolerate`，选择 `strict` 时会设为 `error`。
 - `lenient`：按下列顺序设置第一个匹配的策略：1. `repair`；2. `tolerate`；3. `error`。例如，若检查支持 `error` 与 `tolerate`，选择 `lenient` 时会设为 `tolerate`。

注意

若同时指定 `checks` 与 `policies`，配置支持本节所述、且该应用特定检查所支持的全部策略。但若只指定 `policies` 而不指定 `checks`，则配置仅支持 `strict` 与 `lenient` 策略。

- 也可在 `-policy` 中使用 `if_not_exists` 关键字，仅当当前设计尚未设置策略时才设置。
- `-strategy`：为检查指定要应用的 `strategy`。

表 11.1: 顶层收敛检查、策略与修复策略（原书 Table 64）

Check	说明	Strategy	支持 Policies	支持 References
hier.block.missing_frame_view	检查层次 block 是否缺少 Frame 视图	从已绑定视图重建 Frame	error, tolerate, repair	MID, BOT
hier.block.reference_missing_port_location	检查物理层次边界 pin 位置是否缺失	按连接关系在 block 边界上为 block pin 分配位置	error, tolerate, repair	MID, BOT
hier.block.reference_port_outside_boundary	检查物理层次边界 pin 位置是否在边界外	按连接关系在 block 边界上重新分配位置	error, tolerate, repair	MID, BOT
hier.block.reference_missing_port	检查物理层次参考中是否缺少 port	在参考 block 中创建缺失 port 并更新位置	error, tolerate, repair	MID, BOT
hier.block.instance_bound_to_frame	检查物理层次实例是否绑定到 Frame 视图	—	error, tolerate	TOP
hier.block.instance_with_design_type_macro	检查物理层次实例是否绑定到 macro	—	error, tolerate	TOP
hier.top.estimated_corner	检查顶层是否存在 estimated corner	—	error, tolerate	TOP
hier.block.missing_leaf_cell_location	检查物理层次 leaf cell 位置是否缺失	在 block 边界内为 leaf cell 分配近似位置（不考虑合法性）	error, tolerate, repair	MID, BOT
hier.block.leaf_cell_outside_boundary	检查物理层次 leaf cell 位置是否在边界外	将 leaf cell 重新放到边界内最近位置（不考虑合法性）	error, tolerate, repair	MID, BOT
hier.block.missing_child_block_instance_location	检查指定物理层次中子物理层次实例位置是否缺失	按连接关系在 block 边界内为下层子块分配近似位置（不考虑重叠消除）	error, tolerate, repair	MID
hier.block.child_block_instance_outside_boundary	检查子物理层次实例位置是否在指定物理层次边界外	将子块移到边界内最近位置（不考虑重叠消除）	error, tolerate, repair	MID
hier.block.port_mismatch_between_views	检查物理层次各视图之间是否存在 port 不匹配	以绑定视图为 golden，重建 Frame 以与之匹配	error, tolerate, repair	MID, BOT
hier.block.missing_design_view_from_abstract	检查 Abstract 是否缺少 design 视图	—	error, tolerate	MID, BOT
hier.block.instance_unlinked	检查物理层次参考是否因视图从 Frame/ETM 变为 Abstract/design 而未正确链接到顶层	—	error	TOP
hier.block.abstract_type_non_timing	检查 Abstract 类型是否非 timing	—	error	MID, BOT
hier.block.abstract_target_use_non_implementation	检查 Abstract 的 target use 是否非 implementation	—	error	MID, BOT
hier.block.missing_core_area	检查物理层次是否缺少 core area	—	error	MID, BOT
hier.block.unmapped_logic	检查物理层次中是否存在未映射逻辑	—	error	MID, BOT

注意

-strategy 不适用于层次化检查。

- -references: 为参考子设计指定策略与 strategy。

报告早期数据检查记录

Reporting Early Data Check Records

可用 report_early_data_checks 报告全部检查或特定检查集的策略设置。可用下列选项配置报告:

- -policy: 报告全部检查的配置详情, 显示各检查所选策略与 strategy。与 -hierarchical 合用可报告子块配置。
- -checks: 报告各检查的当前策略、支持的策略、strategy 与帮助文本。检查名支持星号通配符 (*)。与 -hierarchical 合用可报告子块检查。
- -verbose: 报告所有失败检查的摘要、策略、已应用 strategy, 以及带注释的已检查对象。与 -hierarchical 合用可报告子块中失败检查的记录、策略、strategy 与已检查对象。
- -hierarchical: 遍历层次, 报告层次中每个设计的检查、策略、strategy 与失败计数。

下例中, report_early_data_checks -hierarchical -policy 报告顶层与块级设计上层次化检查的策略设置:

```
fc_shell> report_early_data_checks -policy
*****
Report : report_early_data_checks
Design : top
Version: R-2020.09
Date   : Thu Aug 20 23:53:54 2020
*****
Design          Check                                Policy
Strategy
-----
unit_des.nlib:top.design hier.block.instance_bound_to_frame      error
unit_des.nlib:top.design hier.block.instance_unlinked            error
unit_des.nlib:top.design hier.block.instance_with_design_type_macro  error
unit_des.nlib:top.design hier.top.estimated_corner                error
-----
blk              hier.block.abstract_missing_design_view          error
blk              hier.block.abstract_target_use_non_implementation error
blk              hier.block.abstract_type_non_timing              error
```

```
blk                                hier.block.leaf_cell_outside_boundary    error
-----
-----
```

下例中，report_early_data_checks -hierarchical -verbose 报告对已检查对象执行的修复，并给出子块上的策略、strategy 与已检查对象详情：

```
fc_shell> report_early_data_checks -hierarchical -verbose
*****
Design Check                                Policy Strategy Checked  Comment
                                           Object
-----
blk hier.block.missing_leaf_cell_location repair          eco_cell Instance location
                                           updated to
                                           X:150145 Y:69620
-----
```

在运行顶层早期流程时，可能因子块或顶层设计中的设计数据违规而遇到顶层错误或警告。要利用数据检查与策略能力探索早期流程，需要知道设计上所见顶层错误对应的检查名。report_hier_check_description 提供各顶层错误关联检查的信息。下例显示该命令生成的报告：

```
fc_shell> report_hier_check_description
*****
Report : Hier check description
Design : top
Version: R-2020.09
Date   : Fri Aug 21 00:08:32 2020
*****

Legend
  E - error
  R - repair
  T - tolerate
  TOP - Top-Design
  BLK - Respective Block-Ref

-----
-----
Check                                Error  Tolerate Repair Allowed  Allowed
Description                          ID      ID      ID    Policies References
-----
-----
hier.block.missing_frame_view        TL-101  TL-401  TL-501  E|R|T    BLK
Missing frame view
hier.block.abstract_missing_design_view TL-101  TL-401  N/A     E|T      BLK
Missing design view for abstract
hier.block.reference_missing_port_location TL-126  TL-426  TL-526  E|R|T    BLK
Missing location of physical
                                           hierarchy boundary pin
```

hier.block.reference_port_outside_boundary	TL-127	TL-427	TL-527	E R T	BLK
Location of physical hierarchy boundary			pin outside physical hierarchy boundary		

生成早期数据检查记录报告

Generating a Report of Early Data Check Records

可用 `get_early_data_check_records` 从设计生成检查记录报告。与 `-hierarchical` 合用可获取所有子块的检查记录。

报告格式为：

```
check_name@object
```

例如：

```
fc_shell> get_early_data_check_records -hierarchical
{hier.block.missing_leaf_cell_location@eco_cell}
```

可用 `-filter` 按对象类、检查名等参数过滤结果，例如：

```
fc_shell> get_early_data_check_records \
-filter "checked_object_class==port"
{place.port_type_mismatch@clk route.missing_layer_direction@feed_in
route.layer_name_mismatch@out}
```

11.17 使用 Abstract 视图执行顶层收敛

Performing Top-Level Closure With Abstract Views

可先实现各 block，再实现顶层，如图 11.9 所示：



图 11.9: Block 完成后再实现顶层

也可并行实现 block 与顶层，如图 11.10 所示：

在顶层使用 Abstract 时，可用块级支持的命令执行顶层综合、布局、优化、时钟树综合、布线与后布线优化。当前工具在顶层收敛期间不会修改 Abstract 内部，因此可创建并使用只读 Abstract。

(原书 *Figure 196: Top and Block Levels Implemented in Parallel*)
块级与顶层流程并行，中途交换 Abstract/Frame。请参阅原版 PDF 插图。

图 11.10: 顶层与块级并行实现

11.18 创建 ETM 与 ETM Cell Library

Creating ETMs and ETM Cell Libraries

要在 Fusion Compiler 中创建 ETM 与 ETM cell library，见「在 Fusion Compiler 工具中创建 ETM 与 ETM Cell Library」。

要分别在 PrimeTime 与 Library Manager 中创建 ETM 与 ETM cell library，见：

- 在 PrimeTime 工具中创建 ETM
- 在 Library Manager 工具中创建 ETM Cell Library

11.18.1 在 Fusion Compiler 工具中创建 ETM 与 ETM Cell Library

Creating ETMs and ETM Cell Libraries in the Fusion Compiler Tool

不必先在 PrimeTime 中用 `extract_model` 为每个 mode/corner 组合创建 ETM，再在 Library Manager 中把 ETM 与对应物理信息合并成 cell library；可直接在 Fusion Compiler 中用 `extract_model` 一步完成这两步。

下列 Fusion Compiler 脚本为设计的每个 mode 与 corner 创建 ETM，再与对应物理信息合并并创建相应 cell library：

```
# Open the design, create a frame view, and set the PrimeTime options
open block.nlib:block.design
create_frame <options>          ;# extract_model needs Frame for Library
preparation
set_pt_options -pt_exec_path <> -work_dir ETM_work_dir \
               -post_link_script <tcl script with extract_model*
               variables> \

# To use StarRC for Parasitic extraction
set_app_options -name extract.starrrc_mode -value true
set_starrrc_options -config <starrrc_config_file>

# Create the ETM and generate the ETM cell Library
extract_model
```

默认情况下，Fusion Compiler 将生成的 ETM library 写出到：

- 带 label 的 block: [pwd]/ETM_Lib_work_dir/block_name/label_name/
- 不带 label 的 block: [pwd]/ETM_Lib_work_dir/block_name/

11.18.2 在 PrimeTime 工具中创建 ETM

Creating ETMs in the PrimeTime Tool

可在 PrimeTime 中用 `extract_model` 为设计创建 ETM。对于多 corner-多 mode 设计，必须为每个 scenario 施加相应 corner 与 mode 约束并分别创建 ETM。

下列 PrimeTime 脚本为 AMS_BLK 设计的 S3 scenario（由 m1 mode 与 c3 corner 组成）创建 ETM:

```
#Read in the design
read_verilog ./AMS_BLK.v
link

# Apply parasitics
read_parasitics ./AMS_BLK.spef

#For multivoltage designs, apply UPF data and settings
load_upf ./AMD_BLK.upf
set extract_model_include_upf_data true

# Apply the mode (m1) and corner (c3) constraints for the scenario (S3)
source m1_constraints.tcl
source c3_constraints.tcl

# Enable clock latencies for designs with synthesized clock trees
set extract_model_with_clock_latency_arcs true
set extract_model_clock_latency_arcs_include_all_registers false

# Create the ETM
extract_model -library_cell -format db -output AMS_BLK_m1_c3
```

关于在 PrimeTime 中创建 ETM 的更多信息，见 *PrimeTime User Guide* 的 Extracted Timing Models 一章。

11.18.3 在 Library Manager 工具中创建 ETM Cell Library

Creating ETM Cell Libraries in the Library Manager Tool

为每个 mode/corner 组合创建 ETM 后，用 Library Manager 把 ETM 与对应物理信息合并并创建 cell library。更多信息见 *Library Manager User Guide*。

下列 Library Manager 脚本把 AMS_BLK 设计各 mode/corner 的 ETM 与对应物理信息合并，并创建相应 cell library:

```
# Create a Library work space
create_workspace -flow etm_moded AMS_BLK

# Read the physical data (frame view)
read_ndm -views frame AMS_BLK.ndm
```

```
# Read the ETMs for every scenario
read_db -mode_label m1 AMS_BLK_m1_c1.db
read_db -mode_label m1 AMS_BLK_m1_c2.db
read_db -mode_label m1 AMS_BLK_m1_c3.db
read_db -mode_label m2 AMS_BLK_m2_c1.db
read_db -mode_label m2 AMS_BLK_m2_c2.db
read_db -mode_label m2 AMS_BLK_m2_c3.db

# Check the compatibility of the libraries you read in
check_workspace

# Generate cell library
commit_workspace -output AMS_BLK_ETM.ndm
```

创建 ETM 的 cell library 时，可用下列方法之一获取物理数据：

- 用 read_ndm -views frame 读入对应 block 的 Frame 视图；
- 用 read_lef 读入 block 的 LEF 文件；
- 用 read_gds 读入 block 的 GDSII 文件；
- 用 read_oasis 读入 block 的 OASIS 文件。

11.19 在顶层链接 ETM

Linking to ETMs at the Top Level

要在顶层链接 ETM，在创建顶层设计库之前把 ETM library 加入参考库列表，例如：

```
fc_shell> lappend nt_ref_lib "AMS_BLK_ETM.ndm"
fc_shell> create_lib -technology tech.tf -ref_libs $nt_ref_lib TOP
```

包含多个 mode 的 ETM 称为 moded ETM。对于链接到 moded ETM 的 cell 实例，必须用 set_cell_mode 指定所需 mode，例如：

```
fc_shell> current_mode m1
fc_shell> set_cell_mode m1 U1
```

对于链接到 moded ETM 的 cell 实例，若不指定 mode，工具不会激活时序弧、时序检查、生成时钟与 case value。

要用 report_cell_modes 报告特定 cell 实例的 cell mode。

要用 set_reference 把 cell 实例在其 Abstract 视图与 ETM 之间切换。下例把 U1 从 Abstract 切换到 ETM：

```
fc_shell> set_reference -block AMS_BLK_ETM.ndm:AMS_BLK.timing U1
```

下例把 U1 从 ETM 切回 Abstract：

```
fc_shell> set_reference -block AMS_BLK.ndm:AMS_BLK.abstract U1
```

在 ETM 与 Abstract 视图之间切换后，需重新施加顶层时序约束。

11.20 使用 ETM 执行顶层收敛

Performing Top-Level Closure With ETMs

在顶层使用 ETM 之前，必须完成下列任务：

- 施加仅顶层的时序约束与设置。

若由 ETM 表示的 cell 实例含内部时钟，必须从顶层重新施加这些时钟定义。避免引用 block 内部对象的跨边界时序例外。

- 要施加 UPF 约束与设置，必须：

1. 用 `save_upf` 从各块级设计生成块级 UPF；
2. 在顶层，为每个由 ETM 表示的 block 实例加载块级 UPF，并加载仅顶层 UPF：

```
load_upf -scope <instance_name> block.upf
load_upf top.upf
```

- 施加顶层布局约束与设置，例如 hard keepout margin。

在顶层使用 ETM 时，可用块级支持的命令执行综合、顶层布局、优化、时钟树综合、布线与后布线优化。

11.21 透明层次优化

Transparent Hierarchy Optimization

透明层次优化（Transparent Hierarchy Optimization, THO）使工具能透明地优化层次化设计。

THO 允许在遵守物理 block 边界的同时，并发优化顶层与子块，并使用 CCD、AWP、PrimeTime 延迟计算等全部高级优化技术。

在 THO 期间，工具在顶层时序上下文中优化顶层与特定子块（design 视图）。THO 不改变子块形状与物理层次。默认情况下，工具遵守 pin 位置，但提供选项以重新分配 pin 位置以改善 QoR。

由于无需展平设计，可继续用层次化 block 做 signoff: signoff extraction 与 PrimeTime STA 均可层次化运行。此外，后期 ECO 也往往更容易实现。

THO 可用于 `compile_fusion` 的 `final_place` 与 `final_opto` 阶段，以及 `route_opt` 与 `hyper_route_opt`。

（原书 Figure 197: *Transparent Hierarchy Optimization Flow*）

THO 流程: `open_block -ref_libs_for_edit` → 启用 THO 设置 → `compile_fusion -from final_place / route_opt` 或 `hyper_route_opt`。请参阅原版 PDF 插图。

图 11.11: 透明层次优化流程

下列示例流程展示 THO 在整体实现流程中的用法。本例中，THO 作为典型层次化优化流程之上的额外增量优化步骤运行。

THO 流程包括以下步骤：

- 访问库（Accessing Libraries）
- 设置透明层次优化（Setting up Transparent Hierarchy Optimization）
- 在 `compile_fusion` 的 `final_place` 阶段执行 THO
- 在 `compile_fusion` 的 `final_opto` 阶段执行 THO
- `compile_fusion` 之后重新预算（Rebudgeting）
- 在 `route_opt` 或 `hyper_route_opt` 期间执行 THO

11.21.1 访问库

Accessing Libraries

启用 THO 设置之前，必须确保：

- 所有子块库以编辑模式打开。必须用 `open_lib` 的 `-ref_libs_for_edit` 打开父库。
- block 的 design 视图可编辑。

11.21.2 设置透明层次优化

Setting up Transparent Hierarchy Optimization

执行 THO 需要 design 视图。顶层可能正链接到 Abstract 视图；请按「将 Block 从 Abstract 切换为 Design 视图」一节的说明，把要优化的 block 切换为 design 视图。按下列步骤设置 THO：

1. 若在块级 `compile_fusion` 之后执行 THO，必须用下列命令把 CCD useful skew balancing offset 以及集成时钟门控单元的 latency offset 传播到顶层时序上下文：

```
fc_shell> promote_clock_data -auto_clock all -latency_offset
fc_shell> promote_clock_data -auto_clock all -balance_points
```

2. 要启用并对 block 执行 THO，确保要优化的 block 已允许编辑：

```
fc_shell> set_editability -blocks {list of blocks} -value true
```

3. 用下列命令验证 block 可编辑性：

```
fc_shell> report_editability -blocks [get_blocks -hierarchical]
```

4. 在 THO 期间，子块内的拥塞更新、route-driven estimation（RDE）计算以及 ECO 布线等操作可用分布式处理运行。用下列命令进行设置。必须按示例指定自己的队列参数（如 `qsub` 参数）：

注意

若未指定 host options，工具会在主进程中串行运行各子块操作。

```
fc_shell> set_host_options -name for_tho -num_processes 4 -max_cores 16
    -submit_command [list qsub -P queue -pe mt 16 -l mem_free=100G -cwd]
fc_shell> set_hierarchy_options -host_option for_tho -blocks {list of
    blocks}
```

5. 用 `init_hier_optimization` 启用 THO。按阶段使用不同选项，见后续各节。

11.21.3 在 `compile_fusion final_place` 阶段执行 THO

Performing THO During `compile_fusion final_place` Stage

在 `compile_fusion` 的 `final_place` 阶段，可跨顶层与各 block 透明地执行布局。布局发生在各自 block 边界内，同时又是全局的——忽略 pin 位置以获得直通路路径。布局可由线长、时序与拥塞驱动。

THO 提供基于全局布局细化 pin 位置的选项。可在 THO 的粗布局步骤中，为指定 block 集细化 pin 位置。

1. 要细化 pin 位置并允许 pin 移动：

```
fc_shell> set_hierarchy_options -enable_pin_movement true -blocks \
    {list of blocks}
```

2. 要用 `report_hierarchy_options` 报告 `set_hierarchy_options` 所施加的设置。
3. 要用下列命令以 THO 运行 `final_place` 阶段：

```
fc_shell> init_hier_optimization -flow pre_route
fc_shell> compile_fusion -from final_place -to final_place
fc_shell> commit_hier_optimization
```

`init_hier_optimization` 会发出类似下列消息，确认哪些 block 可用于优化：

```
Information: Sub-block u1/a/i1 (Reference: ablock.design) is treated
as modifiable. (HOPT-001)
```

全局布局之后，THO 调用 `place_pins` 创建新的 pin 布局。

11.21.4 在 `compile_fusion final_opto` 阶段执行 THO

Performing THO During `compile_fusion final_opto` Stage

Fusion Compiler 在 `final_opto` 阶段执行透明层次优化，确保所有块间路径与块-顶层路径都像扁平设计一样被优化。这样，CCD 优化可完整处理这些路径上的违规。

```
fc_shell> init_hier_optimization -flow pre_route
fc_shell> compile_fusion -from final_opto -to final_opto
fc_shell> commit_hier_optimization
```

11.21.5 compile_fusion 之后重新预算

Rebudgeting After compile_fusion

在带 THO 的 `compile_fusion` 之后，块接口时序相对各 block 单独实现时会发生变化。由于 block 现已基于顶层时序图景优化，在继续对 block 运行 `clock_opt` 之前，必须为 block 创建更新后的时序约束。THO 期间可能已修改 CCD useful skew balance point offset；这些 offset 也需要变成块上下文专用，以便时钟树综合步骤能将其纳入考虑。

下例展示如何执行预算：

```
fc_shell> set budget_instances "u1/a/i1 u1/b/i2 ..."
fc_shell> set_budget_options -reset -all
fc_shell> set_budget_options -add_blocks $budget_instances
fc_shell> set_app_options -list { plan.budget.use_ccd_latency true }
fc_shell> set_app_options -list { plan.budget.write_hold_budgets false }
fc_shell> set_app_options -list { plan.budget.estimate_timing_mode
false }
fc_shell> compute_budget_constraints -setup_delay -latency_targets actual
-balance false -boundary
fc_shell> write_budgets -output budgets-post-compile -force -nosplit
-include_balance_points -include_pin_latency
```

11.21.6 在 route_opt 或 hyper_route_opt 期间执行 THO

Performing THO During route_opt or hyper_route_opt

在实现流程末尾执行 THO，可获得最佳优化潜力，因为优化的是完整设计。`route_opt` 与 `hyper_route_opt` 对顶层与各 block 一起执行后布线优化。优化后的 ECO 布线会在所有已打开编辑的子块上执行。

THO 优化已完全集成到 `route_opt` 与 `hyper_route_opt`。在单主机上使用 `hyper_route_opt` 或 `route_opt` 的示例如下：

```
fc_shell> init_hier_optimization -flow post_route
fc_shell> route_opt or hyper_route_opt
fc_shell> commit_hier_optimization
```

使用 `route_opt` 时，可在每次 `route_opt` 之后并行执行必须进行的 ECO 布线操作，以缩短整体运行时间。须按「设置透明层次优化」一节设置 host options。`hier_route_eco` 在各 block 中执行 ECO 布线，示例如下：

```
fc_shell> init_hier_optimization -flow post_route \
-detached_hier_route_eco
fc_shell> route_opt
fc_shell> hier_route_eco
fc_shell> commit_hier_optimization
```

注意

以这种方式使用 `hyper_route_opt` 时，工具会忽略该设置，不会分发 ECO 布线。

Chapter 12

RedHawk 与 RedHawk-SC Fusion

RedHawk and RedHawk-SC Fusion

RedHawk™ 与 RedHawk-SC™ 电源完整性（power integrity）解决方案通过 RedHawk Fusion 与 RedHawk-SC Fusion 接口集成到实现流程中。可在电源规划（power planning）与初始布局（initial placement）完成后，在物理实现流程的不同节点使用 RedHawk Fusion 与 RedHawk-SC Fusion 进行轨分析（rail analysis）。从而在详细布线（detail routing）之前发现潜在电源网络问题，显著缩短设计周期周转时间（turnaround time）。

布局完成且 PG mesh 可用后，使用 RedHawk Fusion 或 RedHawk-SC Fusion 对电源与地网络执行压降分析（voltage drop analysis），以计算功耗并检查压降违例。也可在设计流程的其他阶段（例如详细布线之后）执行压降分析。

芯片收尾（chip finishing）完成后，使用 RedHawk Fusion 或 RedHawk-SC 执行 PG 电迁移分析（PG electromigration analysis），以检查电流密度违例。

本章说明如何在环境中使用 RedHawk Fusion 或 RedHawk-SC Fusion 执行轨分析。关于 RedHawk 或 RedHawk-SC 分析流程与命令的详细说明，请参阅 *RedHawk User Manual* 或 *RedHawk-SC User Manual*。

本章包含以下主题：

- 使用 RedHawk-SC Fusion 运行轨分析（Running Rail Analysis Using RedHawk-SC Fusion）
- RedHawk Fusion 与 RedHawk-SC Fusion 概述（An Overview for RedHawk Fusion and RedHawk-SC Fusion）
- 设置可执行程序（Setting Up the Executables）
- 指定 RedHawk 与 RedHawk-SC 工作目录（Specifying RedHawk and RedHawk-SC Working Directories）
- 为轨分析准备设计与输入数据（Preparing Design and Input Data for Rail Analysis）
- 将理想电压源指定为 Tap（Specifying Ideal Voltage Sources as Taps）
- 缺失 Via 与未连接 Pin 检查（Missing Via and Unconnected Pin Checking）
- 使用多个轨场景运行轨分析（Running Rail Analysis with Multiple Rail Scenarios）

- 执行压降分析（Performing Voltage Drop Analysis）
- 执行 PG 电迁移分析（Performing PG Electromigration Analysis）
- 执行最小路径电阻分析（Performing Minimum Path Resistance Analysis）
- 执行有效电阻分析（Performing Effective Resistance Analysis）
- 执行分布式 RedHawk Fusion 轨分析（Performing Distributed RedHawk Fusion Rail Analysis）
- 电压驱动电源开关单元尺寸调整（Voltage Driven Power Switch Cell Sizing）
- 使用宏模型（Working With Macro Models）
- 执行签核分析（Performing Signoff Analysis）
- 写出分析与检查报告（Writing Analysis and Checking Reports）
- 在 GUI 中显示地图（Displaying Maps in the GUI）
- 在 GUI 中显示 ECO 形状（Displaying ECO Shapes in the GUI）
- 电压热点分析（Voltage Hotspot Analysis）
- 查询属性（Querying Attributes）

12.1 使用 RedHawk-SC Fusion 运行轨分析

Running Rail Analysis Using RedHawk-SC Fusion

与 RedHawk Fusion 能力类似，RedHawk-SC Fusion 在 Fusion Compiler 环境中支持门级轨分析与检查能力。要启用 RedHawk-SC Fusion，需同时启用应用选项 `rail.enable_redhawk_sc`，并禁用应用选项 `rail.enable_redhawk`。

关于 RedHawk-SC Fusion 的更多信息，见「设置可执行程序」与「指定 RedHawk 与 RedHawk-SC 工作目录」。

RedHawk Fusion 与 RedHawk-SC 共用同一套轨分析与检查命令，以及用于查看分析结果的同一 GUI（除若干限制外）。表 12.1 比较了在 RedHawk Fusion、RedHawk-SC Fusion 之一或二者中受支持的 `analyze_rail` 选项。

在使用 RedHawk-SC Fusion 执行轨分析之前，必须按「为轨分析准备设计与输入数据」所述指定库与所需输入文件的位置。以下两个应用选项仅在 RedHawk-SC Fusion 中可用：

- `rail.toggle_rate`：为轨分析指定不同单元类型的翻转率（toggle rate）。例如：

```
fc_shell> set_app_options -name rail.toggle_rate \
-value {clock 2.0 data 0.2 combinational 0.15 \
sequential 0.15}
```

- `rail.em_only_tech_file`：指定用于 PG 电迁移分析的电迁移规则文件。当电迁移规则信息定义在与 `rail.tech_file` 所指定技术文件不同的单独技术文件中时，使用此选项。例如：

```
fc_shell> set_app_options -name rail.em_only_tech_file \
-value EM_ONLY.rule
```

表 12.1: RedHawk Fusion 与 RedHawk-SC Fusion 分析功能比较（原书 Table 65）

选项	RedHawk Fusion	RedHawk-SC Fusion
-redhawk_script_file	支持	支持
-voltage_drop	支持	支持
-switching_activity	支持	支持
-electromigration	支持	支持
-power_analysis	支持	支持
-result_name	支持	支持
-script_only	支持	支持
-bg	支持	支持
-min_path_resistance	支持	仅与 -voltage_drop 联用时支持
-effective_resistance	支持	仅与 -voltage_drop 联用时支持
-check_missing_via	支持	仅与 -voltage_drop 联用时支持; 显示缺失 via 检查结果时不遵从 IR 阈值设置
-extra_gsr_option_file	支持	不支持
-multiple_script_files	支持	不支持
-submit_to_other_machines	支持	不支持

12.2 RedHawk Fusion 与 RedHawk-SC Fusion 概述

An Overview for RedHawk Fusion and RedHawk-SC Fusion

RedHawk Fusion 与 RedHawk-SC Fusion 均支持门级轨分析能力，包括静态与动态轨分析。可在 Fusion Compiler 环境中使用二者执行下列类型的检查与分析：

- 缺失 **via** 与未连接 **pin** 形状检查
要检查设计中的缺失 via 或未连接 pin 形状，先使用 `set_missing_via_check_options` 配置检查相关设置，再使用 `analyze_rail -check_missing_via` 执行检查。详见「缺失 Via 与未连接 Pin 检查」。
- 压降分析
使用 `analyze_rail -voltage_drop`，或在 GUI 中选择 Rail > Analyze Rail 并选择“Voltage drop analysis”。将 `-voltage_drop` 设为 `static`、`dynamic`、`dynamic_vcd` 或 `dynamic_vectorless` 以选择分析类型。详见「执行压降分析」。
- **PG** 电迁移分析
使用 `analyze_rail -electromigration`。详见「执行 PG 电迁移分析」。
- 最小路径电阻分析
使用 `analyze_rail -min_path_resistance`。在 RedHawk-SC 分析流程中，必须将 `-voltage_drop` 与 `-min_path_resistance` 一起使用。详见「执行最小路径电阻分析」。

根据所运行的分析类型，工具会生成可在 Fusion Compiler GUI 中查看的可视化显示（地图，maps），以及可在错误浏览器（error browser）中显示的错误数据。这些地图与错误数据有助于在不离开 Fusion Compiler 环境的情况下发现问题区域并确定纠正措施。

注意

Fusion Compiler 工具中的 RedHawk Fusion 不支持 RedHawk 签核分析功能，例如层次化分析，或带集总（lumped）/SPICE package 的动态分析。详见「为轨分析准备设计与输入数据」。

默认情况下，分析多角多模（multicorner-multimode）设计时，RedHawk Fusion 仅分析当前设计场景（design scenario）。关于如何创建并指定用于轨分析的轨场景（rail scenario），见「使用多个轨场景运行轨分析」。

图 12.1 示出典型设计流程中使用这些分析能力的位置。

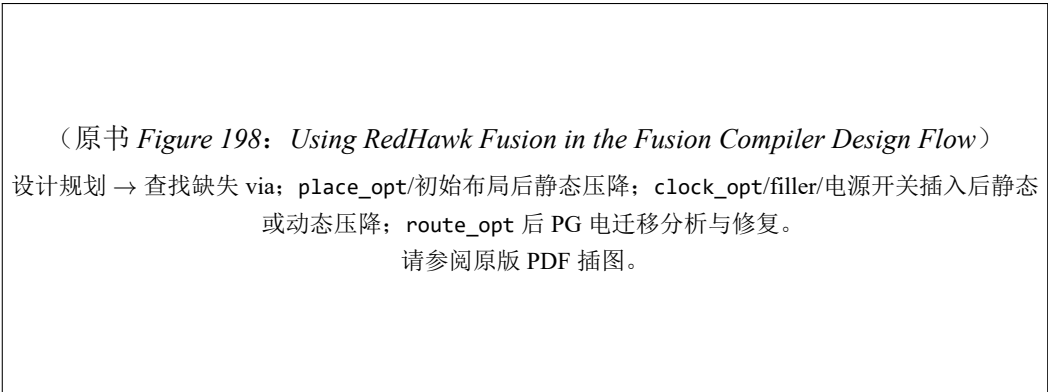


图 12.1: 在 Fusion Compiler 设计流程中使用 RedHawk Fusion

12.2.1 RedHawk Fusion 与 RedHawk-SC Fusion 数据流

RedHawk Fusion and RedHawk-SC Fusion Data Flow

RedHawk/RedHawk-SC Fusion 功能允许在实现阶段执行轨分析。在具备所需输入文件的情况下，Fusion Compiler 工具创建用于调用 RedHawk 工具的 RedHawk 运行脚本与 Global System Requirements（GSR）配置文件（或用于调用 RedHawk-SC 工具的 RedHawk-SC Python 脚本），以运行 PG net 抽取、功耗分析、压降分析与 PG 电迁移分析。

分析完成后，Fusion Compiler 工具使用 RedHawk 或 RedHawk-SC 计算的结果生成分析报告与地图。随后可在 Fusion Compiler GUI 中以图形方式检查热点。还提供错误数据与 ASCII 报告，用于检查超出限值的位置。

图 12.2 说明在 Fusion Compiler 环境中使用 RedHawk Fusion 或 RedHawk-SC Fusion 执行轨分析时的数据流。

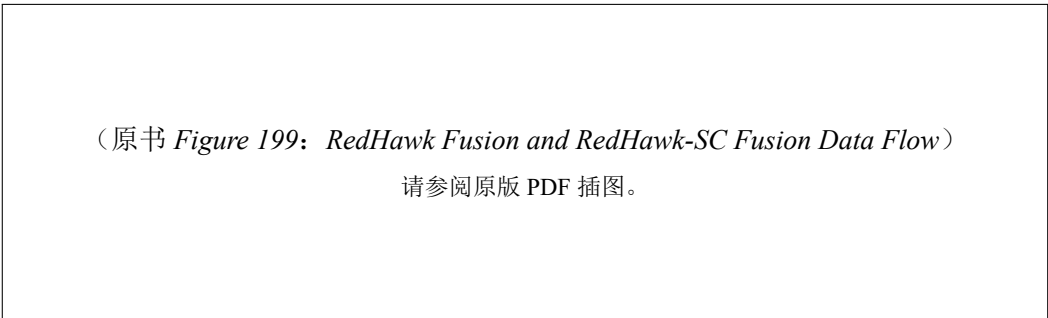


图 12.2: RedHawk Fusion 与 RedHawk-SC Fusion 数据流

12.2.2 RedHawk/RedHawk-SC Fusion 分析流程

RedHawk/RedHawk-SC Fusion Analysis Flow

指定必要的输入与设计数据后，即可对设计执行压降与 PG 电迁移分析。图 12.3 说明基本静态轨分析流程中的步骤。

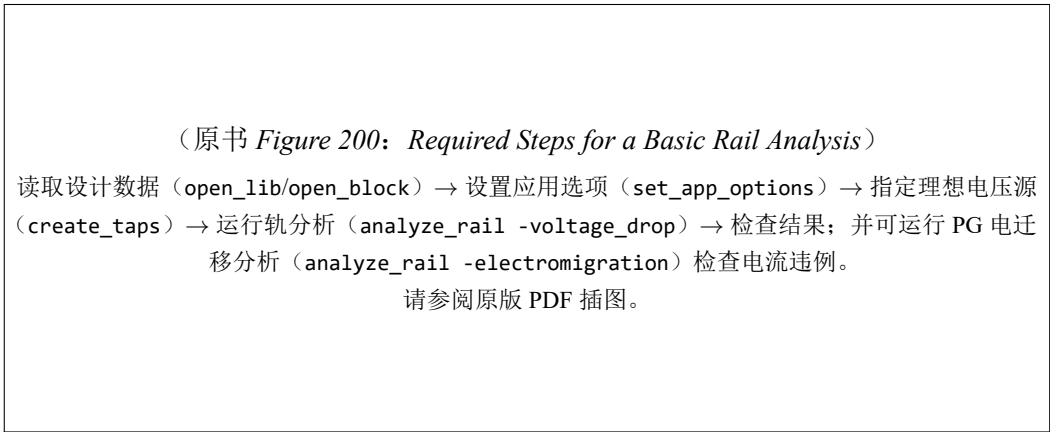


图 12.3: 使用 RedHawk 与 RedHawk-SC Fusion 进行基本轨分析的必要步骤

12.2.3 在后台运行 RedHawk Fusion 命令

Running RedHawk Fusion Commands in the Background

默认情况下，执行 analyze_rail 进行轨分析时，会调用 RedHawk 或 RedHawk-SC 工具以输入数据执行指定分析类型，并在完成后将分析结果加载回轨数据库。在 RedHawk Fusion 或 RedHawk-SC Fusion 进程结束前，无法运行任何 Fusion Compiler 命令。

要在工具于后台运行时执行布局编辑任务，运行 analyze_rail -bg。分析完成后，运行 open_rail_result -back_annotate 将分析结果上传到轨数据库。

默认情况下，RedHawk 或 RedHawk-SC 在分析完成后将结果加载回轨数据库。但若在 analyze_rail 中指定了 -bg，则需运行 open_rail_result -back_annotate，以根据最新分析结果重建轨数据库并创建新的 RAIL_DATABASE 文件。

注意

在设计中移动 net 之后回注 (back-annotate) 先前运行的分析结果时，基于实例的地图会反映更新后的实例位置变化，但寄生地图不会。

打开轨结果时，工具不检测该结果是否在使用 -bg 的情况下生成。若对非 -bg 生成的结果运行 open_rail_result -back_annotate (或反之)，命令可能无法正确工作。

12.2.4 报告并检查后台进程状态

Reporting and Checking the Status of the Background Process

当 RedHawk 或 RedHawk-SC Fusion 进程结束时，工具发出如下消息：

Info: Running REDHAWK_BINARY in background with log file: LOG_FILE

LOG_FILE 保存在 PWD 目录中。

要检查后台 `analyze_rail` 进程是否仍在活动，使用 `report_background_jobs` 命令。

12.3 设置可执行程序

Setting Up the Executables

要运行 RedHawk Fusion 或 RedHawk-SC Fusion 功能，通过设置 `rail.product` 与 `rail.redhawk_path` 应用选项启用功能并指定 RedHawk 或 RedHawk-SC 可执行程序的位置。

例如：

```
fc_shell> set_app_options -name rail.product -value redhawk|redhawk_sc
fc_shell> set_app_options -name rail.redhawk_path \
    -value /tools/RedHawk_Linux64e5_V19.0.2p2/bin
```

必须确保所指定的可执行程序与所用 Fusion Compiler 版本兼容。

参见

指定 RedHawk 与 RedHawk-SC 工作目录

12.4 指定 RedHawk 与 RedHawk-SC 工作目录

Specifying RedHawk and RedHawk-SC Working Directories

轨分析期间，RedHawk/RedHawk-SC Fusion 创建工作目录以存放生成的文件，包括分析日志、脚本与各类输出数据。默认情况下，RedHawk 或 RedHawk-SC 工作目录名为 `RAIL_DATABASE`。要使用不同名称，使用 `rail.database` 应用选项。

数据结构如下：

- **RAIL_CHECKING_DIR:** RedHawk Fusion 保存轨分析期间缺失信息相关报告文件的目录。例如，运行静态轨分析时，工具在该目录中生成：
 - `libcell.apl_current`
 - `libcell.apl_cap`
 - `libcell.missing_liberty`
- **RAIL_DATABASE:** 工具保存并检索功耗计算与轨分析期间生成的结果与日志文件的目录。包含以下子目录：
 - `in-design.redhawk` 或 `in-design.redhawk_sc`：其中含 `design_name.result`，RedHawk 或 RedHawk-SC 在此保存分析结果文件。
 - `design_name`：RedHawk 或 RedHawk-SC 写出分析结果供 Fusion Compiler 检索以显示地图与查询数据的目录。例如，运行 `open_rail_result` 会加载该目录中的数据以便在 GUI 中显示地图。

若分析完成且希望保留数据供稍后检索，需在进行另一次 `analyze_rail` 运行之前，使用 `rail.database` 将现有轨数据保存到另一工作目录。例如：

```
fc_shell> set_app_options -name rail.database \
-value RAIL_DATABASE_STATIC_RUN
```

否则，RedHawk/RedHawk-SC 工具会用新数据覆盖先前生成的轨数据，并发出如下警告：

```
Warning: Rail result still open! Will be overwritten!
```

要避免该警告，在另一次 `analyze_rail` 之前运行 `close_rail_result`，从内存中移除所有轨数据。

12.5 为轨分析准备设计与输入数据

Preparing Design and Input Data for Rail Analysis

在使用 RedHawk/RedHawk-SC Fusion 能力分析设计中的压降与电流密度违例之前，使用 `set_app_options` 指定库与所需输入文件的位置。

表 12.2 对应原书 Table 66，下列应用选项用于在 Fusion Compiler 环境中指定设计与输入数据以运行 RedHawk/RedHawk-SC 轨分析。

表 12.2：用于指定设计与输入数据的应用选项（原书 Table 66）

rail.lib_files

指定 .lib 格式的库文件。例如：

```
fc_shell> set_app_options \
-name rail.lib_files \
-value {test1.lib test2.lib}
```

rail.tech_file

指定用于 PG 抽取与 PG 电迁移分析的 Apache 技术文件。例如：

```
fc_shell> set_app_options \
-name rail.tech_file \
-value ChipTop.tech
```

rail.3dic_enable_die

指定纳入详细分析的 die。例如：

```
fc_shell> set_app_options \
-name rail.3dic_enable_die \
-value {DIE1 true DIE2 true ...}
```

rail.instance_power_file

指定用户定义的 Instance Power File (IPF)，用于向 pin 提供功耗。例如：

```
fc_shell> set_app_options \
-name rail.instance_power_file \
-value {E1 FILE1 DIE2 FILE2 ...}
```

rail.apl_files

指定 Apache APL 文件，包含用于动态轨分析的电流波形与本征寄生。例如：

```
fc_shell> set_app_options \  
-name rail.apl_files -value \  
{cell.current current \  
cell.cdev cap}
```

rail.switch_model_files

指定用于动态轨分析的 RedHawk 开关单元模型文件。例如：

```
fc_shell> set_app_options \  
-name rail.switch_model_files \  
-value {switch.model}
```

rail.macro_models

指定宏模型文件。例如：

```
fc_shell> set_app_options \  
-name rail.macro_models \  
-value {gds_cell1 mm_dir1 \  
gds_cell2 mm_dir2}
```

rail.lef_files

（可选）指定 LEF 文件列表。未指定时，工具使用设计库中的数据生成一份。

rail.def_files

（可选）指定 DEF 文件列表。未指定时，工具使用设计库中的数据生成一份。

rail.pad_files

（可选）指定 pad 位置文件列表。未指定时，需通过运行 `create_taps` 指定理想电压源。

rail.sta_file

（可选）指定静态时序文件（STA），包含最终 slew 与延迟信息。未指定时，工具使用设计库中的数据生成静态时序窗信息。

rail.spef_files

（可选）指定 SPEF 文件列表，包含信号 net 的详细寄生阻性与容性负载数据。未指定时，Fusion Compiler 使用设计库中的数据生成 SPEF 文件。

rail.effective_resistance_instance_file

（可选）指定实例文件，列出用于有效电阻计算的单元实例。

rail.generate_file_type

（可选）指定为 RedHawk Fusion 或 RedHawk-SC Fusion 轨分析生成的脚本文件类型。可用类型：

- `tcl`（默认）：生成 Tcl 格式文件，默认文件名为 `set_user_input.tcl`。
- `python`：生成 Python 格式文件，默认文件名为 `input_files.py`。Python 脚本文件仅在 RedHawk-SC Fusion 轨分析流程中支持。

使用定制 Python 脚本文件运行 RedHawk-SC 轨分析时需要此应用选项。详见「支持 RedHawk-SC 定制 Python 脚本文件」。例如：

```
fc_shell> set_app_options \  
-name rail.generate_file_type -value python
```

rail.generate_file_variables

仅在 RedHawk-SC 中支持。指定定制 Python 脚本用于 RedHawk-SC 轨分析时的首选用户变量。当脚本包含非标准 collateral Python 变量名时需要此选项。支持的文件类型：LEF、DEF、SPEF、TWF、PLOC。例如：

```
fc_shell> set_app_options \  
-name rail.generate_file_variables -value \  
{PLOC ploc LEF lef_files DEF def_files \  
SPEF spef_files TWF tw_files}
```

rail.enable_new_rail_scenario

（可选）启用带多个轨场景的 RedHawk Fusion 或 RedHawk-SC Fusion 轨分析。默认为 false。详见「使用多个轨场景运行轨分析」。

rail.enable_parallel_run_rail_scenario

（可选）在分析流程内启用分布式 RedHawk Fusion 或 RedHawk-SC Fusion 分析。

rail.scenario_name

（可选）指定用于 RedHawk 或 RedHawk-SC Fusion 轨分析的场景。工具在电源完整性流程的 IR 驱动功能中遵从所指定的设计场景，例如 IR 驱动布局、IR 驱动并发时钟与数据优化以及 IR 驱动优化。详见「使用多个轨场景运行轨分析」。

rail.dump_icc2_results_style

指定如何以新的 Ansys 地图样式加载结果（例如 -value 2.0）。

12.5.1 生成轨分析脚本文件

Generating Rail Analysis Script Files

可运行 `analyze_rail -script_only`，根据设计库中的信息生成运行轨分析所需的设置，而不实际执行分析。

该命令将数据写入工作目录。保存到目录的输出文件见表 12.3。

表 12.3: 运行 `analyze_rail -script_only` 时的输出数据（原书 Table 67）

文件名	说明	RH	RH-SC
RedHawk GSR 文件	包含运行 RedHawk Fusion 轨分析的配置设置	是	否
LEF/DEF、SPEF 与 STA 文件	若运行 <code>analyze_rail</code> 前未指定这些输入文件则生成	是	是
RedHawk Fusion 运行脚本（ <code>analyze_rail.tcl</code> ）	包含运行轨分析所需命令	是	否
RedHawk-SC Fusion 运行脚本（ <code>analyze_rail.py</code> ）	包含运行轨分析所需的 Python 命令	否	是

12.5.2 支持 RedHawk-SC 定制 Python 脚本文件

Supporting RedHawk-SC Customized Python Script Files

RedHawk-SC Fusion 提供应用选项与命令以读入用户定制的 Python 脚本文件。工具从设计库生成全部设计 collateral（包括 DEF、LEF、SPEF、TWF 与 PLOC），并将其加入用户定制 Python 脚本，以便在单次 `analyze_rail` 迭代中运行压降分析。用户定制 Python 脚本文件支持多轨场景的轨分析，以及其它 IR 驱动功能（例如 IR 驱动布局或 IR 驱动并发时钟与数据优化）。

要为 RedHawk-SC Fusion 轨分析使用定制 Python 脚本文件：

1. 将脚本文件类型设为 `python`：

```
fc_shell> set_app_options -name rail.generate_file_type -value python
```

2. 设置 `rail.generate_file_variables`，指定要加入用户定制 Python 脚本的输入文件（包括 DEF、LEF、SPEF、TWF 与 PLOC）的变量。若定制 Python 脚本使用与 RedHawk-SC Fusion 相同的值，则无需设置此应用选项。

	DEF	LEF	SPEF	TWF / PLOC
fc_shell 默认名	DEF	LEF	SPEF	TWF / PLOC
RedHawk-SC Fusion 默认变量名	def_files	lef_files	spef_files	tw_files / ploc

例如，若为 DEF 变量指定 `test_def_file` 以匹配定制 Python 脚本中的 DEF 文件：

```
fc_shell> set_app_options -name rail.generate_file_variables \
-value {DEF test_def_files}
```

3. 创建并指定轨场景，以生成要加入定制 Python 脚本的必要输入文件。

运行 `create_rail_scenario` 创建轨场景并将其与当前设计中的设计场景关联。可创建多个轨场景并与同一设计场景关联。

然后运行 `set_rail_scenario -generate_file {LEF | TWF | DEF | SPEF | PLOC}`，指定轨场景以及要从设计库生成的输入文件类型。

注意

使用多角多模技术运行轨分析时，必须在 `set_rail_scenario` 之前使用 `create_rail_scenario` 创建轨场景。

下例创建 `T_low` 与 `T_high` 轨场景并为其生成输入文件：

```
fc_shell> create_rail_scenario -name T_low \
-scenario func_cbest
fc_shell> set_rail_scenario -name T_low \
-generate_file {PLOC DEF SPEF TWF LEF} \
-custom_script_file ./customized.py
fc_shell> create_rail_scenario -name T_high \
-scenario func_cbest
fc_shell> set_rail_scenario -name T_high \
-generate_file {PLOC DEF SPEF TWF LEF} \
-custom_script_file ./customized.py
```

随后可将 `customized.py` 用于多轨场景轨分析或其它 IR 驱动功能。

示例

Examples

网格系统上的多轨场景轨分析（定制 **Python** 脚本）：

```
#Required. Specify the below options to enable multi-rail-scenario rail analysis.
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}

#Required. Set the type of the script file to python.
set_app_options -name rail.generate_file_type -value python

#Use user-defined test_def_files for DEF variable to match
#with customized python.
set_app_options -name rail.generate_file_variables \
    -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c_ -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
    -generate_file {PLOC DEF SPEF TWF LEF} \
    -custom_script_file ./customized.py

set_host_options -submit_protocol sge \
    -submit_command {qsub -V -notify -b y \
    -cwd -j y -P bnormal -l mfree=16G}

analyze_rail -rail_scenario {func.ss_125c func.ff_125c} \
    -submit_to_other_machine
```

本地机器上的多轨场景轨分析：

```
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}
set_app_options -name rail.generate_file_type -value python
set_app_options -name rail.generate_file_variables \
    -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c_ -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
```

```
-generate_file {PLOC DEF SPEF TWF LEF} \
-custom_script_file ./customized.py

set_host_options -submit_command {local}

analyze_rail -rail_scenario {func.ss_125c func.ff_125c}
```

网格系统上的单场景轨分析:

```
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}
set_app_options -name rail.generate_file_type -value python
set_app_options -name rail.generate_file_variables \
    -value {DEF test_def_files }

create_rail_scenario -name func.ff_125c -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
    -generate_file {PLOC DEF SPEF TWF LEF} \
    -custom_script_file ./customized.py

set_host_options -submit_protocol sge \
    -submit_command {qsub -V -notify -b y \
    -cwd -j y -P bnormal -l mfree=16G}

analyze_rail -rail_scenario {func.ff_125c} \
    -submit_to_other_machine
```

网格系统上的 **IR** 驱动布局:

```
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}
set_app_options -name rail.generate_file_type -value python
set_app_options -name rail.generate_file_variables \
    -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
    -generate_file {PLOC DEF SPEF TWF LEF} \
    -custom_script_file ./customized.py

set_host_options -submit_protocol sge \
    -submit_command {qsub -V -notify -b y \
    -cwd -j y -P bnormal -l mfree=16G}
# Enable IR-driven placement before running clock_opt -from final_opto
```

```
set_app_options -name opt.common.power_integrity -value true

clock_opt -from final_opto
```

多角多模设计上网格系统中的并发时钟与数据优化：

```
#Required. Specify the below options to enable MCM-based optimization.
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}
set_app_options -name rail.generate_file_type -value python
set_app_options -name rail.generate_file_variables \
    -value {DEF test_def_files }

create_rail_scenario -name func.ss_125c -scenario func.ss_125c
set_rail_scenario -name func.ss_125c -voltage_drop dynamic

create_rail_scenario -name func.ff_125c -scenario func.ff_125c
set_rail_scenario -name func.ff_125c \
    -generate_file {PLOC DEF SPEF TWF LEF} \
    -custom_script_file ./customized.py

set_host_options -submit_protocol sge \
    -submit_command {qsub -V -notify -b y -cwd -j y \
    -P bnormal -l mfree=16G}
# Enable IR-driven CCD analysis before running route_opt
set_app_options -name opt.common.power_integrity -value true
route_opt
```

参见

指定 RedHawk 与 RedHawk-SC 工作目录

12.6 将理想电压源指定为 Tap

Specifying Ideal Voltage Sources as Taps

Tap 用于对被分析器件所处的外部电压源环境建模；它们不是设计本身的一部分，可视为电压源的虚拟模型。设计中理想电压源与理想电源的位置是获得准确轨分析结果所必需的。

要创建 tap，使用 `create_taps`。可通过多次运行 `create_taps` 创建多个 tap。

使用 `-name` 为所创建的 tap 指定名称。若指定名称已存在于设计中，工具发出警告并用新 tap 替换原 tap。当通过单次调用 `create_taps` 创建多个 tap 时，所有创建的 tap 按 `tap_name_NNN` 命名约定命名，其中 NNN 为唯一整数标识符，`tap_name` 为 `-name` 的字符串参数。

可用下列任一方式创建 tap：

- 将顶层 **PG pin** 当作 tap

当 block 没有可用的物理 pad 或 bump 信息时，使用 `-top_pg`。

使用 `-supply_net` 将 tap 显式关联到从其定义对象所连接的电源（电源或地）net。要隐式关联，运行不带 `-supply_net` 的 `create_taps`。

下例为所有顶层电源 pin 创建 tap：

```
fc_shell> create_taps -supply_net VDD -top_pg
```

- 在绝对坐标处创建 tap

使用 `-layer` 与 `-point` 在指定坐标创建 tap。若指定位置没有电源 net、电源 pin 或 via 形状（即没有到电源网络的导电通路），则尽管 tap 存在，也对轨分析无影响。

若在已有 tap 的位置再定义 tap，工具发出警告并用新 tap 替换原 tap。必须使用 `-supply_net` 指定该 tap 所关联的电源 net。

注意

当 `-point` 与 `-of_objects` 联用时，位置相对于指定对象的原点；否则相对于当前顶层 block 的原点。

下例使用绝对坐标创建 tap。命令检查该坐标位置是否触及任何布局形状：

```
fc_shell> create_taps -supply_net VDD \
  -layer 3 -point {100.0 200.0}
Warning: Tap point (100.0, 200.0) on layer 3 for net VDD does not
touch any polygon (RAIL-305)
```

- 使用现有实例作为 tap

要在指定对象上创建 tap，使用 `create_taps -of_objects`。对象列表中的每一项可以是单元或库单元，或单元与库单元的集合。默认情况下，工具在对象的电源与地 pin 上创建 tap。若指定 `-point`，工具相对于每个对象原点在指定物理位置创建 tap。

在分析已物理例化 pad cell 的顶层设计时，使用现有实例创建 tap。

下例为 `some_pad_cell` 库单元的每个实例创建 tap。工具在相对单元实例原点 {30.4 16.3} 处的 metal1 层为每个实例创建一个 tap：

```
fc_shell> create_taps -of_objects [get_lib_cell */some_pad_cell]\
  -layer metal1 -point {30.4 16.3}
```

下例在匹配 `*power_pad_cell*` 模式的所有单元实例的全部电源与地 pin 上创建 tap：

```
fc_shell> create_taps -of_objects [get_cells -hier \
  *power_pad_cell*]
```

- 导入 tap 文件

若处理的 block 尚未定义 pin，或 pad cell 的位置/设计尚未最终确定，可在 ASCII 文件中定义 tap 位置、层号与电源 net，再用 `create_taps -import` 导入。

支持的 tap 文件格式如下：

```
net_name layer_number [X-coord Y-coord]
```

以分号（;）或井号（#）开头的行视为注释。 x/y 坐标以微米为单位。

例如：

```
fc_shell> exec cat tap_file
VDD 3 500.000 500.000
VSS 5 500.000 500.000
fc_shell> create_taps -import tap_file
```

- 带 package 创建 tap
- 简单 package RLC（电阻–电感–电容）模型为所有电源与地 pad 提供一组固定的电气特性。要在轨分析中使用简单 package RLC 模型，需通过 rail.pad_files 指定定义该简单 RLC 模型的 pad 位置文件。
- 关于简单 package RLC 模型的详细说明，见 *RedHawk User Manual* 相关章节。

注意

在使用简单 package RLC 模型创建 tap 之前，必须使用 rail.allow_redhawk_license_checkout 启用 RedHawk 签核许可证密钥。

12.6.1 验证 Tap 并查找无效 Tap

Validating the Taps and Finding Invalid Taps

创建 tap 时，默认 create_taps 会验证所创建的 tap 是否插入在有效位置。若 tap 未触及任何布局形状，工具发出警告。要在形状外放置 tap 而不发出警告，使用 -nocheck 禁用检查。

```
fc_shell> create_taps -supply_net VDD -layer 3 \
-point {100.0 200.0} -nocheck
```

Tap 属性

Tap Attributes

验证检查期间，工具为每个插入的 tap 对象标记 is_valid_location 属性以指示其有效性。启用验证检查时，有效 tap 为 true，无效 tap 为 false。使用 -nocheck 时，所有 tap 的属性值为 unknown。

工具在后续压降与最小路径电阻分析中更新 tap 属性。属于 PG net 的每个 tap 会对照抽取形状检查。分析完成后，工具根据轨分析结果更新 tap 有效性状态。

下例先列出本命令运行中加入 block 的 tap 对象属性，再检查 Tap_1437 是否通过验证：

```
fc_shell> list_attributes -application -class tap
...
Attribute Name      Object    Type      Properties Constraints
-----
context             tap      string    A          integrity,
                  rail,
                  session

full_name           tap      string    A
is_valid_location   tap      boolean   A
layer_name          tap      string    A
```

```
layer_number      tap      int      A
net               tap      string     A
object_class      tap      string     A
parasitics        tap      string     A
position          tap      coord      A
static_current    tap      double     A
type              tap      string     A
absolute_coord, auto_pg, cell_coord, cell_pg, lib_cell_coord,
lib_cell_pg, signal, terminal, top_pg
fc_shell> get_attribute [get_taps Tap_1437] is_valid_location
true
```

查找无效 Tap

Finding Invalid Taps

要查找未通过有效性检查的 tap，使用 `get_taps` 报告 `is_valid_location` 为 `false` 的 tap。例如：

```
fc_shell> get_attribute [ get_taps Tap_1437 ] is_valid_location
false
```

为无效 Tap 查找替代位置

Finding Substitute Locations for Invalid Taps

若 tap 的指定位置无效，使用 `create_taps` 的 `-snap_distance` 选项为 tap 查找替代位置。命令在同一层上（由 `-layer` 定义）于指定 snap 距离内水平与垂直搜索附近布局形状。tap 被重新插入到距指定位置最小距离内的有效形状角点。

注意

仅可为由 `-point` 与 `-layer` 指定的无效 tap 搜索替代位置。例如：

```
fc_shell> create_taps -point {50,50} -layer {M1} \
-snap_distance 100
Snap tap point from original location (50.000, 50.000) to new
location (90.000, 65.910)
```

如图 12.4 所示，指定的 tap 位置未触及任何布局形状。红色矩形是距指定位置最近的形状，其右下角在所有其它矩形中距离最小。工具将该 tap 重新插入到红色矩形的右下角。

12.6.2 移除 Tap

Removing Taps

要移除先前用 `create_taps` 创建的 tap，使用 `remove_taps`。要移除特定 tap，对 `get_taps` 使用 `-filter`。下例移除类型为 `top_pg` 的全部电源 pin tap：

```
fc_shell> remove_taps [get_taps -filter type==top_pg]
```

(原书 *Figure 201: Finding a Substitute Location for an Invalid Tap*)

Original {x,y} → Snapped tap。请参阅原版 PDF 插图。

图 12.4: 为无效 Tap 查找替代位置

要移除 block 中的全部 tap:

```
fc_shell> remove_taps
```

12.7 缺失 Via 与未连接 Pin 检查

Missing Via and Unconnected Pin Checking

注意

缺失 via 与未连接 pin 检查在 RedHawk Fusion 与 RedHawk-SC Fusion 分析流程中均受支持。

在检查缺失 via 与未连接 pin 之前，确保已按「为轨分析准备设计与输入数据」具备所需输入文件。

可在压降分析期间或单独运行中检查 block 内的缺失 via 或未连接 pin。例如，可在完成电源结构之后、运行 `place_opt` 之前执行检查，以便在轨分析前发现电源网络错误。

默认情况下，RedHawk/RedHawk-SC Fusion 检测下列类型的电源网络错误：

- 未连接 **pin** 形状：未与理想电压源连续物理连接的 **pin** 形状。
- 缺失 **via**：若两个重叠金属形状在其重叠区域内不含 **via**，则视为潜在缺失 **via** 错误。

要检查并修复缺失 via 或未连接 pin：

1. 使用 `set_missing_via_check_options` 定义配置选项，见「设置检查选项」。
2. 使用 `save_block` 保存 block。
3. 使用 `analyze_rail -check_missing_via` 执行检查，见「检查缺失 Via 与未连接 Pin」。在 RedHawk-SC Fusion 流程中，必须将 `-check_missing_via` 与 `-voltage_drop` 一起指定。
4. 通过在错误浏览器中打开生成的错误文件，或将检查结果写出到输出文件，检查结果见「查看错误数据」与「写出分析与检查报告」。
5. 插入 PG via 以修复报告的缺失 via，见「修复缺失 Via」。

12.7.1 设置检查选项

Setting Checking Options

在检查设计中的缺失 via 或未连接 pin 之前，使用 `set_missing_via_check_options` 指定必要选项。

要检查或移除选项设置,分别使用 `report_missing_via_check_options` 或 `remove_missing_via_check_options`。
表 12.4 列出 `set_missing_via_check_options` 的常用选项。各选项详细说明见 man page。

表 12.4: `set_missing_via_check_options` 常用选项（原书 Table 68）

选项	说明
<code>-redhawk_script_file</code>	使用已有 RedHawk 脚本文件进行缺失 via 检查。
<code>-exclude_stack_via</code>	从缺失 via 检查中排除堆叠 via。默认在分析中包含堆叠 via。
<code>-check_valid_vias</code>	报告金属重叠是否能容纳有效 via 模型。
<code>-exclude_cell / -exclude_instance</code>	排除指定单元的所有实例所覆盖区域，或排除单元实例列表。默认在全芯片级执行检查。
<code>-exclude_regions</code>	指定要从缺失 via 检查中排除的矩形区域。
<code>-sort_by_hot_inst</code>	按压降值对缺失 via 排序。默认按所定义层之间的平均 delta 电压排序。
<code>-threshold</code>	设置触发缺失 via 检查的电压阈值。设为 -1 禁用电压检查过滤。默认为 1 V。仅在压降分析期间运行检查时有效。
<code>-min_width</code>	忽略小于指定宽度的线段。
<code>-pitch</code>	指定平行线跨越或触及 GDSII block 时缺失 via 的 pitch。
<code>-gds</code>	标记位于 GDSII block 区域内以及跨越 GDSII block 的布线中的缺失 via。默认忽略位于或重叠已由 GDS2DEF 或 GDSMMX 实用程序处理的 block 的项。
<code>-ignore_inter_met、-toplayer、-bottomlayer</code>	忽略指定顶层与底层之间所有层上的导线。

12.7.2 检查缺失 Via 与未连接 Pin

Checking Missing Vias and Unconnected Pins

使用 `analyze_rail -check_missing_via`，根据 `set_missing_via_check_options` 指定的设置检查 block 中的缺失 via 与未连接 pin。

若希望与压降分析一起运行缺失 via 与未连接 pin 检查，使用 `-voltage_drop`。工具随后在缺失 via 报告中报告几何中找到的所有缺失 via 及其电压值。要设置跨层电压阈值以过滤报告，使用 `set_missing_via_check_options` 的 `-threshold`。可在一次缺失 via 检查中设置多个阈值。

注意

在 RedHawk-SC Fusion 流程中，必须将 `-check_missing_via` 与 `-voltage_drop` 一起指定。

若要在一次 `analyze_rail` 运行中同时报告带与不带电压阈值的缺失 via，先设置期望阈值（例如 0.001），再将值重置为 -1。将 `-threshold` 设为 -1 禁用电压检查。

缺失 via 检查完成后，打开生成的错误数据以在错误浏览器中检查错误。详见「查看错误数据」。
示例 48

```
fc_shell> set_missing_via_check_options -exclude_stack_via \
    -threshold 0.0001
fc_shell> analyze_rail -voltage_drop static -check_missing_via \
    -nets {VDD VSS}
```

上例将阈值设为 0.0001，并比较 via 两端电压（排除堆叠 via），然后在静态轨分析期间运行缺失 via 检查。

RedHawk 脚本文件包含：

```
perform extraction -power -ground
perform analysis -static
mesh vias -report_missing -exclude_stack_via -threshold 0.0001 \
    -o apache.missingVias1
```

示例 49

```
fc_shell> set_missing_via_check_options -exclude_stack_via \
    -threshold -1
fc_shell> analyze_rail -voltage_drop -check_missing_via \
    -nets {VDD VSS}
```

上例将阈值设为 -1 以禁用电压检查，并运行缺失 via 检查。

示例 50

```
fc_shell> set_missing_via_check_options -exclude_stack_via \
    -threshold 0.001
fc_shell> set_missing_via_check_options -exclude_stack_via \
    -threshold 0.002
fc_shell> set_missing_via_check_options -exclude_stack_via \
    -threshold -1
fc_shell> analyze_rail -voltage_drop static -check_missing_via \
    -nets {VDD VSS}
fc_shell> report_rail_result -type missing_vias -supply_nets \
    {VDD VSS} rpt.missing_vias
Information: writing rpt.missing_vias.no_threshold done
Information: writing rpt.missing_vias done
```

上例为缺失 via 检查设置三个不同阈值。工具生成两份报告：`rpt.missing_vias.no_threshold` 包含几何中找到的全部缺失 via；`rpt.missing_vias` 包含两端电压差小于 0.001 与 0.002 的 via。

12.7.3 查看错误数据

Viewing Error Data

RedHawk 或 RedHawk-SC Fusion 将检查错误数据文件（名为 `missing_via.supplyNetName.err`）保存在工作目录中。每个电源 net 生成一个错误数据文件。退出当前会话前，务必保存 block 以在 block 中保留生成的错误数据。

要在错误浏览器中打开错误数据文件，从 Error Browser 对话框选择 **File > Open ...**，然后在 Open Error Data 对话框中选择要打开的错误数据文件（见图 12.5）。

(原书 *Figure 202: Opening Error Data from the Error Browser*)

请参阅原版 PDF 插图。

图 12.5: 从错误浏览器打开错误数据

关于错误浏览器的更多信息，见 *Fusion Compiler Graphical User Interface User Guide* 中的“Using the Error Browser”。

参见

指定 RedHawk 与 RedHawk-SC 工作目录

12.7.4 修复缺失 Via

Fixing Missing Vias

要根据 `analyze_rail` 写出的指定 DRC 错误对象插入 PG via 以修复缺失 via，使用 `fix_pg_missing_vias`。

当工具为缺失 via 错误对象插入 PG via 时，会在错误浏览器中将该错误对象状态设为 **fixed**（见图 12.6），从而可快速检查是否所有缺失 via 均已修复。

(原书 *Figure 203: Error Status After Inserting PG Vias for Missing Vias*)

工具将错误对象标记为 fixed。请参阅原版 PDF 插图。

图 12.6: 为缺失 Via 插入 PG Via 后的错误状态

下例为 `analyze_rail -check_missing_via` 报告的 VDD net 的全部错误对象插入 PG via:

```
fc_shell> set errdm [open_drc_error_data rail_miss_via.VDD.err]
fc_shell> set errs [get_drc_errors -error_data $errdm]
fc_shell> fix_pg_missing_vias -error_data $errdm $errs
```

12.8 使用多个轨场景运行轨分析

Running Rail Analysis with Multiple Rail Scenarios

要控制 RedHawk Fusion 或 RedHawk-SC Fusion 轨分析的场景：

- 使用 `rail.scenario_name` 为轨分析指定不同的设计场景。详见「为轨分析指定设计场景」。
- 可创建轨场景并将其与当前设计中的设计场景关联。从而可对一个或多个轨场景运行优化与轨分析，以识别可能的电源完整性问题。详见「创建并指定用于轨分析的轨场景」。

12.8.1 为轨分析指定设计场景

Specifying a Design Scenario for Rail Analysis

默认情况下，对于多角多模设计，RedHawk 或 RedHawk-SC Fusion 仅分析当前设计场景。要为轨分析指定不同设计场景，使用 `rail.scenario_name`。工具在电源完整性流程的 IR 驱动功能中遵从所指定的设计场景，例如 IR 驱动布局、IR 驱动并发时钟与数据优化以及 IR 驱动优化。

脚本示例：

```
open_lib
open_block

set_app_option -name rail.technology -value 7
set_app_options -name rail.tech_file -value ./Apache.tech
set_app_options -name rail.lib_files -value {\
  A.lib \
  B.lib }
set_app_options -name rail.switch_model_files -value
{./switch_cells.txt }
source ./test_taps.tcl

current_scenario func_max
set_app_options -name rail.scenario_name -value func_typ

analyze_rail -voltage_drop
```

12.8.2 创建并指定用于轨分析的轨场景

Creating and Specifying Rail Scenarios for Rail Analysis

可创建多个轨场景，并将其与当前设计中的设计场景关联。所创建的轨场景保存在设计库中。

要对多个轨场景运行轨分析：

1. 指定下列应用选项以启用多轨场景轨分析：

```
fc_shell> set_app_options -list {
  rail.enable_new_rail_scenario true
  rail.enable_parallel_run_rail_scenario true
}
```


2. 运行 `set_scenario_status -ir_drop`，为指定设计场景启用轨分析：

```
fc_shell> set_scenario_status func_cbest -ir_drop true
```

3. 创建轨场景并将其与已有设计场景关联。用 `set_rail_scenario` 为所创建的轨场景指定轨属性。可为不同目的创建多个轨场景并与同一设计场景关联。

```
fc_shell> create_rail_scenario -name T_low \
    -scenario func_cbest
fc_shell> set_rail_scenario -name T_low \
    -voltage_drop dynamic -extra_gsr_option_file extra.gsr \
    -nets {VDD VSS}
fc_shell> create_rail_scenario -name T_high \
    -scenario func_cbest
fc_shell> set_rail_scenario -name T_high \
    -voltage_drop dynamic -extra_gsr_option_file extra2.gsr \
    -nets {VDD VSS}
```

4. 用 `set_host_options` 指定多核处理设置。例如，在本地主机上运行：

```
fc_shell> set_host_options -submit_command {local}
```

在网格系统上运行：

```
fc_shell> set_host_options -submit_protocol sge \
    -submit_command {qsub -V -b y -cwd -P bnormal -l mfree=16G}
```

5. 用 `analyze_rail` 运行轨分析：

```
fc_shell> analyze_rail -rail_scenario {T_high T_low} \
    -submit_to_other_machines
```

注意

要对 IR 驱动布局或 IR 驱动并发时钟与数据优化使用多轨场景轨分析能力，必须在 `clock_opt` `-from final_opto` 或 `route_opt` 之前设置：

```
fc_shell> set_app_options \
    -name opt.common.power_integrity -value true
```

关于电源完整性分析的更多信息，见 [Enabling the Power Integrity Features](#) 一节。

12.8.3 示例

Examples

对多个轨场景运行轨分析：

```
# Specify options for multi-rail-scenario rail analysis
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
```

```

}

set_host_options -submit_protocol sge \
    -submit_command {qsub -V -notify -b y -cwd -j y \
    -P bnormal -l mfree=16G}

# Custom RedHawk-SC Fusion python script to specify user-defined
# toggle rate for scenario T_custom
create_rail_scenario -name T_custom -scenario func_cbest
set_rail_scenario -name T_custom -custom_script_file custom_sc.py

# Default RedHawk-SC Fusion toggle rate for scenario T_def
create_rail_scenario -name T_def -scenario func_cbest
set_rail_scenario -name T_def -voltage_drop dynamic -nets {VDD VSS}

# Run RedHawk-SC Fusion rail analysis on the specified rail scenario
analyze_rail -rail_scenario {T_custom T_def} -submit_to_other_machines

# (Optional) Open and examine the generated rail results
open_rail_result {T_custom T_def}

```

在电源完整性流程中运行多轨场景轨分析：

```

## Specify RedHawk/RedHawk-SC based options
set_app_options -name rail.enable_redhawk -value true
set_app_options -name rail.enable_redhawk_sc -value false
set_app_options -name rail.redhawk_path -value /test/bin
set_app_options -name rail.tech_file -value design_data/tech/rh.tech
set_app_options -name rail.lib_files -value {
inputs/test/CCSP_test.lib
}

set_host_options -submit_protocol sge -submit_command {qsub -V \
    -notify -b y -cwd -j y -P bnormal -l mfree=16G}

## Specify options for running optimization on multiple rail scenarios
set_app_options -list {
    rail.enable_new_rail_scenario true
    rail.enable_parallel_run_rail_scenario true
}

## Enable IR drop flag for a given scenario
set_scenario_status [current_scenario] -ir_drop true

## Create a rail scenario and specify associated options
create_rail_scenario -name T_low -scenario func_cbest
set_rail_scenario -name T_low -voltage_drop dynamic \
    -extra_gsr_option_file extra.gsr -nets {VDD VSS}

create_rail_scenario -name T_high -scenario func_cbest
set_rail_scenario -name T_high -voltage_drop dynamic \
    -extra_gsr_option_file extra2.gsr -nets {VDD VSS}

## Enable power integrity flow

```

```
set_app_options -name opt.common.power_integrity -value true
```

12.9 执行压降分析

Performing Voltage Drop Analysis

注意

在运行 `analyze_rail` 计算压降与电流违例之前，确保已为各分析模式提供所需输入文件。关于准备输入文件，见「为轨分析准备设计与输入数据」。

压降分析在 RedHawk Fusion 与 RedHawk-SC Fusion 分析流程中均受支持。

要调用 RedHawk/RedHawk-SC 工具计算指定电源与地网络的压降，使用带下列选项的 `analyze_rail`：

- 使用 `-voltage_drop` 指定分析模式：
 - `static`：执行静态压降分析。
 - `dynamic`：执行动态压降分析。
 - `dynamic_vectorless`：执行无 Verilog Change Dump (VCD) 输入的动态压降分析。
 - `dynamic_vcd`：使用 VCD 输入执行动态压降分析。启用时，使用 `-switching_activity` 指定翻转活动文件。
- 使用 `-nets` 指定要分析的电源与地电源 `net`。工具在分析中考虑指定电源 `net` 的全部开关或内部电源 `net`，无需显式指定内部电源 `net`。
- (可选) 使用 `-switching_activity` 指定翻转活动文件。支持的文件格式：VCD、SAIF 与 ICC_ACTIVITY。未指定时，默认使用翻转率进行轨分析。

注意

若使用动态无向量 (`vector-free`) 模式，无需提供翻转活动信息。

语法：

```
-switching_activity {type file_name [strip_path] [start_time  
end_time]}
```

- `VCD`：指定门级仿真生成的 VCD 文件。默认轨分析读取并使用 VCD 文件中的全部时间值。要指定从 VCD 读取的时间窗，使用可选的 `start_time` 与 `end_time`。指定后，工具读取 VCD 中全部时间值，但仅使用指定时间窗进行轨分析。
- `SAIF`：指定一个或多个门级仿真生成的 SAIF 文件。
- `icc_activity`：运行 Fusion Compiler `write_saif`，生成包含用户标注与仿真翻转活动（含状态相关与路径相关信息）的 SAIF 文件，覆盖完整设计且不运行传播引擎。

`strip_path` 参数从对象名开头移除指定字符串，使 VCD 或 SAIF 中的对象名与 `block` 中的对象名兼容。该参数区分大小写。

- (可选) 默认工具使用 RedHawk/RedHawk-SC 功耗分析功能计算指定电源与地网络的功耗。若希望由 Fusion Compiler 生成轨分析所需功耗数据, 将 `-power_analysis` 设为 `icc2`。关于使用 Fusion Compiler `report_power` 运行功耗分析的详细说明见相关文档。
- (可选) 默认 `analyze_rail` 自动生成用于运行 RedHawk 轨分析的 GSR 文件。要指定附加到当前运行所生成 GSR 文件的外部 GSR 文件, 使用 `-extra_gsr_option_file`。
- (可选) 使用 `-redhawk_script_file` 指定先前 `analyze_rail` 运行生成的已有 RedHawk 脚本文件。使用已有脚本文件时, 工具遵从该文件中的设置并忽略所有其它当前启用的选项。例如, 运行 `analyze_rail -voltage_drop static -redhawk_script_file myscript.tcl` 时, 工具忽略 `-voltage_drop` 设置。未指定 `-redhawk_script_file` 时, 必须用 `-nets` 指定要分析的 net。

下例对 VDD 与 VSS net 执行静态压降分析, 并使用 GSR 配置文件中定义的可选设置:

```
fc_shell> analyze_rail -voltage_drop static -nets {VDD VSS} \
-extra_gsr_option_file add_opt.gsr
```

12.9.1 查看压降分析结果

Viewing Voltage Drop Analysis Results

分析完成后, 工具将分析结果与日志保存在工作目录中。关于工作目录, 见「指定 RedHawk 与 RedHawk-SC 工作目录」。

`analyze_rail` 生成下列文件以调用 RedHawk 工具运行轨分析:

- RedHawk 运行脚本 (`analyze_rail.tcl`)
- RedHawk GSR 配置文件 (`*.gsr`)

`analyze_rail` 生成下列文件以调用 RedHawk-SC 工具:

- RedHawk-SC Python 脚本 (`analyze_rail.py`)

轨分析完成时, RedHawk 或 RedHawk-SC 在 `in-design.redhawk/design_name.result` (或 RedHawk-SC 流程中的 `in-design.redhawk_sc/design_name.result`) 目录中生成:

- RedHawk 分析日志 (`analyze_rail.log.*`)
- 时序窗文件 (`*.sta`)
- 信号 SPEF 文件 (`*.spef`)
- 全芯片分析的 DEF/LEF 文件 (`*.def`、`*.lef`)

压降分析完成后, 工具将分析结果 (`*.result`) 保存在工作目录下的 `design_name` 目录中。这些轨分析结果用于显示地图并查询属性, 以确定大压降问题区域。

- 要重新加载轨结果以便在 GUI 中显示地图, 使用 `open_rail_result`:

```
fc_shell> open_rail_result
REDHAWK_RESULT
```

详见「在 GUI 中显示地图」。

- 要写出轨分析结果, 使用 `report_rail_result`。必须用 `-type` 指定结果类型。例如, 要写出当前 block 的压降值, 将 `-type` 设为 `voltage_drop_or_rise`。详见「写出分析与检查报告」。

- 要查询当前轨结果缓存中存储的结果，使用 `get_attribute`。详见「查询属性」。

注意

RedHawk Fusion 不支持在 Fusion Compiler 环境之外由 RedHawk 工具生成的结果。RedHawk `dump icc2_result` 命令仅在 Fusion Compiler 会话内有效。

12.10 执行 PG 电迁移分析

Performing PG Electromigration Analysis

注意

PG 电迁移分析在 RedHawk Fusion 与 RedHawk-SC Fusion 分析流程中均受支持。

要分析 PG net 上的电流密度并识别存在潜在电迁移问题的线段，运行 `analyze_rail -electromigration`。分析结果保存在工作目录中。

要执行电迁移分析：

1. 设置 `rail.tech_file`，指定定义逐层电流密度限值的 Apache 技术文件 (*.tech)。例如：

```
fc_shell> set_app_options -name rail.tech_file \
-value design_data/tech/test.tech
```

2. 运行带下列选项的 `analyze_rail`：

- 使用 `-voltage_drop` 与 `-electromigration` 启用电迁移分析。
- 使用 `-nets` 指定要分析的电源与地电源 net。工具考虑指定电源 net 的全部开关或内部电源 net。
- （可选）使用 `-extra_gsr_option_file` 指定附加到当前运行所生成 GSR 的外部 GSR 文件。
- （可选）使用 `-redhawk_script_file` 指定先前运行生成的已有 RedHawk 脚本。使用已有脚本时，工具遵从脚本设置并忽略其它当前启用选项。未指定 `-redhawk_script_file` 时，必须用 `-nets` 指定 net。

3. 检查分析结果，见「查看 PG 电迁移分析结果」。

12.10.1 查看 PG 电迁移分析结果

Viewing PG Electromigration Analysis Results

PG 电迁移分析完成后，可通过显示 PG 电迁移地图或在错误浏览器中检查生成的错误来检查电流违例。

显示 PG 电迁移地图

Displaying the PG Electromigration Map

PG 电迁移地图是叠加在物理电源 net 上的彩色编码电迁移值可视化显示。

对于静态分析，地图显示平均电迁移值。对于动态分析，地图显示平均、峰值或均方根电迁移值。

图 12.7 示出以不同颜色突出问题区域的电迁移地图示例。使用选项选择要显示的层或 net 以调查问题区域。例如，要检查某一特定 net 的电流值，取消选择全部 net，再从列表中选择要显示的 net。要仅分析某层上的形状，从列表中选择该层。

关于显示电迁移地图的详细步骤，见「在 GUI 中显示地图」。

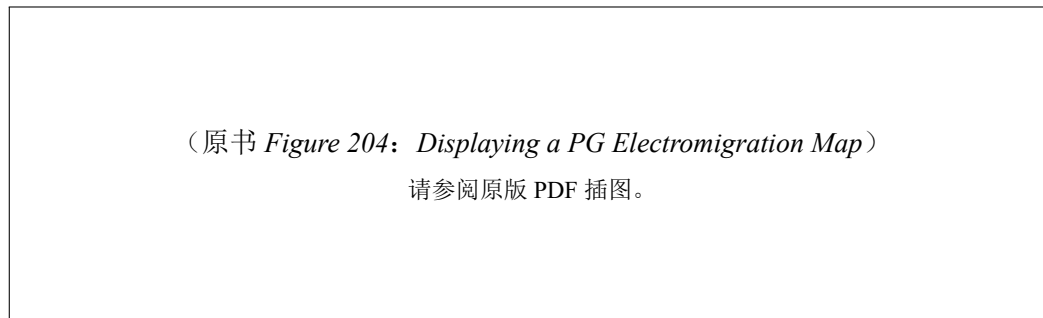


图 12.7: 显示 PG 电迁移地图

检查 PG 电迁移违例

Checking PG Electromigration Violations

PG 电迁移分析完成后，可在错误浏览器中检查生成的错误文件，或将生成的错误写出到输出文件。

工具不会将生成的错误写入工作目录，这意味着若退出工具而不保存 block，错误数据会被删除。若希望在另一会话中检查错误，请确保保存 block 以保留生成的错误数据。

可在错误浏览器中检查生成的错误文件，或将错误写出到 ASCII 文件。

- 要在错误浏览器中检查生成的错误：

1. 在 GUI 中选择 View > Error Browser。
2. 在 Open Error Data 对话框中选择要检查的错误并单击 Open Selected。
3. 在 Error Browser 中检查错误详情（见图 12.8）。

- 要将错误写出到 ASCII 文件，使用下列脚本：

```
set fileName "errors.txt"
set fd [open $fileName w]
set dm [open_drc_error_data rail_pg_em.VDD.err]
set all_errs [get_drc_errors -error_data $dm *]
foreach_in_collection err $all_errs {
    set info [ get_attribute $err error_info];
    puts $fd $info }
close $fd
```

错误文件输出示例：

```
M1 (414.000,546.584 414.344,546.584) em ratio 23585.9% width 0.06
blech
```

```
length 0.000 um current 0.0024963
M1 (414.344,546.584 416.168,546.584) em ratio 23401.3% width 0.06
blech
length 0.000 um current 0.0024768
...
```

(原书 *Figure 205: Opening Generated Errors in the Error Browser*)
请参阅原版 PDF 插图。

图 12.8: 在错误浏览器中打开生成的错误

12.11 执行最小路径电阻分析

Performing Minimum Path Resistance Analysis

注意

最小路径电阻分析在 RedHawk Fusion 与 RedHawk-SC Fusion 分析流程中均受支持。

执行最小路径电阻分析可快速检测设计中电源与地网络的薄弱点。节点的最小路径电阻值是到理想电压源位置 (tap) 的最小阻性路径上的电阻值。理想电压源可以是电源或地 pin、用户定义的 tap 或 package。由于该值仅表示单条路径上的电阻，它不一定等于从该节点到电源或地的有效电阻；相反，它表示节点到电源或地有效电阻的上界。

要执行最小路径电阻分析，运行 `analyze_rail -min_path_resistance`。必须用 `-nets` 指定要分析的电源与地 net。

注意

在 RedHawk-SC 轨分析流程中，必须同时指定 `-voltage_drop` 与 `-min_path_resistance` 以计算最小路径电阻。

可在压降分析之前或期间执行最小路径电阻分析。分析完成后，工具将最小路径电阻值与压降结果写入同一轨结果。

同时执行压降分析与最小路径电阻计算：

```
fc_shell> analyze_rail -nets {VDD VSS} \
-min_path_resistance -voltage_drop static
```

仅执行最小路径电阻分析：

```
fc_shell> analyze_rail -nets {VDD VSS} -min_path_resistance
```

12.11.1 查看最小路径电阻分析结果

Viewing Minimum Path Resistance Analysis Results

分析完成后，运行 `report_rail_result -type minimum_path_resistance`，以显示最小路径电阻地图、将结果写出到输出文件，或查询单元实例或 pin 上的属性。

下例将 VDD net 的计算最小路径电阻写出到 `minres.rpt`：

```
fc_shell> open_rail_result
fc_shell> report_rail_result -type minimum_path_resistance \
    -supply_nets { vdd } minres.rpt
fc_shell> sh cat minres.rpt
FEED0_14/vbp 518.668
INV61/vdd 158.268
ARCR0_1/vbp 145.582
FEED0_5/vdd 156.228
INV11/vbp 518.669
```

要为单元实例写出详细最小电阻路径报告，运行 `report_rail_minimum_path` 并指定要报告的 net 与单元。要在 GUI 中查看该最小电阻路径的逐段电阻，使用 `report_rail_minimum_path` 的 `-error_cell`，将最小路径电阻路径上的几何保存到错误单元。随后可在错误浏览器中打开该错误单元，检查从单元 pin 到最近 tap 的最小电阻路径并追踪途经的每个几何。

下例追踪最小电阻路径上的全部几何并写入名为 `test.err` 的错误单元：

```
fc_shell> open_rail_result
fc_shell> report_rail_minimum_path -cell U19 -net VDD -error_cell
```

如图 12.9 所示，错误浏览器中按层对每个几何分类。单击几何可查看 IR drop、电阻、包围盒等详细信息。在 GUI 中，追踪路径在设计布局中以黄色高亮（见图 12.10）。

（原书 *Figure 206: Displaying Error Cells in the Error Browser*）

请参阅原版 PDF 插图。

图 12.9: 在错误浏览器中显示错误单元

参见

在 GUI 中显示地图；写出分析与检查报告；查询属性

(原书 *Figure 207: Traced Segment is Highlighted in the Design Layout*)

请参阅原版 PDF 插图。

图 12.10: 追踪线段在设计布局中高亮

12.11.2 使用鼠标工具追踪最小路径电阻

Tracing Minimum Path Resistance Using the Mouse Tool

使用最小路径电阻 (MPR) 鼠标工具, 可交互式追踪从指定电源或地 net 到设计中边界节点的最小电阻路径。

要用 MPR 交互式追踪工具追踪计算得到的最小路径电阻:

1. 在 GUI 中选择 Task > Rail, 然后从任务列表选择 Analysis Tools, 打开 Task Assistant 窗口 (见图 12.11)。
2. 在 Task Assistant - Rail 窗口中单击 MPR Mouse Tool。
3. 指定要分析的 net, 单击 OK。

图 12.12 所示的 MPR 交互式鼠标工具提供下列功能:

- 放大/缩小以查看追踪
- 两种操作模式: Add 模式在地图中追踪多条路径; Replace 模式每次在地图中追踪一条路径
- 使用标准查询工具显示追踪信息
- 选择区域以一次追踪多个单元
- 用脚本记录交互操作

(原书 *Figure 208: Invoking the MPR Mouse Tool*)

请参阅原版 PDF 插图。

图 12.11: 调用 MPR 鼠标工具

(原书 *Figure 209: Tracing the Path With the Least Resistance*)

可单击隐藏地图中全部对象，然后选择 Cell。请参阅原版 PDF 插图。

图 12.12: 追踪最小电阻路径

12.12 执行有效电阻分析

Performing Effective Resistance Analysis

注意

有效电阻分析在 RedHawk Fusion 与 RedHawk-SC Fusion 分析流程中均受支持。

要计算指定电源与地 net 的有效电阻，使用 `analyze_rail -effective_resistance`。

要指定用于有效电阻计算的单元实例列表，在文本文件中描述目标实例，然后用 `rail.effective_resistance_instance_file` 指定该文本文件。

例如：

```
fc_shell> sh cat cell_list.txt
u_GS_PRO_65
u_GS_PRO_64
ChipTop_U_compressor_mode/U3

fc_shell> set_app_options \
  -name rail.effective_resistance_instance_file \
  -value cell_list.txt
fc_shell> analyze_rail -nets {VDD VSS} -effective_resistance
```

可在压降分析之前或期间执行有效电阻分析。分析完成后，工具将有效电阻值与压降结果写入同一轨结果。

注意

在 RedHawk-SC 轨分析流程中，必须同时指定 `-effective_resistance` 与 `-voltage_drop` 以计算有效电阻。

计算完成后，工具将结果保存在工作目录中。要将有效电阻值写出到文本文件，使用 `report_rail_result -type effective_resistance`。要查询电阻值，使用 `get_attribute`。

例如：

```
fc_shell> get_attribute [get_pins u_GS_PRO_65/VDD] \
  effective_resistance
```

参见

指定 RedHawk 与 RedHawk-SC 工作目录；写出分析与检查报告

12.13 执行分布式 RedHawk Fusion 轨分析

Performing Distributed RedHawk Fusion Rail Analysis

注意

分布式轨分析功能仅在 RedHawk Fusion 分析流程中受支持。

默认情况下，`analyze_rail` 使用单进程执行 RedHawk 轨分析。为降低内存使用与总运行时间，可使用分布式处理方法，在不同于运行 Fusion Compiler 工具的机器上运行 RedHawk Fusion。

可在同一 Fusion Compiler 会话中向目标 farm 机器提交多个 RedHawk 作业并并行运行。从而仅用一个 Fusion Compiler 许可证即可运行多次 RedHawk 分析。每个 RedHawk 作业需要一个 `SNPS_INDESIGN_RH_RAIL` 许可证密钥。

要检查生成的分析结果，先运行 `open_rail_result` 加载保存在指定目录中的结果。随后可在地图中调查问题区域，并在文本文件或错误视图中检查违例。

一次只能打开一个结果上下文。要加载另一次分析运行生成的轨上下文，先用 `close_rail_result` 关闭当前轨结果，再打开期望的轨结果（见「在 GUI 中显示地图」）。

12.13.1 许可证要求

License Requirement

默认情况下，所需许可证数量等于分布式处理模式中用 `-multiple_script_files` 指定的脚本文件或配置数量。若运行 `analyze_rail` 时可用许可证数少于脚本文件数，命令以错误消息停止。

要在 `analyze_rail` 运行期间持续检查可用的 `SNPS_INDESIGN_RH_RAIL` 许可证，将 `rail.license_checkout_mode` 设为 `wait`。若工具检测到许可证变为可用，会立即重用或检出许可证并向 farm 机器提交脚本。默认 `rail.license_checkout_mode` 为 `nowait`。

分析结束后，运行 `remove_licenses` 释放 `SNPS_INDESIGN_RH_RAIL` 许可证：

```
fc_shell> remove_licenses SNPS_INDESIGN_RH_RAIL
```

如图 12.13 所示，`rail.license_checkout_mode` 设为 `wait`。有六个脚本要提交到 farm 机器，但仅有两个可用 `SNPS_INDESIGN_RH_RAIL` 许可证。此时工具先检出两个许可证并等待其它许可证可用。当某脚本完成作业后，工具立即重用其许可证以提交其它脚本。

12.13.2 启用分布式处理的步骤

Steps to Enable Distributed Processing

要为 RedHawk Fusion 启用分布式处理：

1. 使用 `set_host_options` 定义分布式处理配置。可指定下列一个或多个选项：

(原书 *Figure 210: Waiting for Available Licenses During Distributed RedHawk Fusion Run*)

请参阅原版 PDF 插图。

图 12.13: 分布式 RedHawk Fusion 运行期间等待可用许可证

选项	说明
-submit_protocol	指定提交作业的协议，例如 LSF、GRD 或 SGE。注意：提交作业前，运行 rsh 检查目标 farm 机器是否支持从远程机器接受分布式处理的 rsh 协议。
-target	指定仅提交 RedHawk 作业。
-timeout	指定作业提交超时值。
-submit_command	指定 LSF、GRD、SGE 或定制作业提交程序的完整路径名以及其它所需选项，例如 qsub 或 bsub。
-max_cores	控制每个分布式 worker 作业可使用的核数。默认为 1。

2. 运行带下列选项的 analyze_rail:

选项	说明
-submit_to_other_machines	启用向目标 farm 机器提交 RedHawk 作业。
-multiple_script_files directory_name script_name	指定 RedHawk 脚本以及保存分析结果的目录。

示例 51

```
%> rsh myMachine ls
test1
CSHRC
DESKTOP
> set_host_options "myMachine" -target RedHawk \
    -max_cores 2 -timeout 100
> analyze_rail -submit_to_other_machines \
    -voltage_drop static -nets {VDD VDDG VDDX VDDY VSS}
```

上例先检查机器 myMachine 是否支持 rsh 协议，再仅为 RedHawk Fusion 启用向该机器提交作业。超时值为 100，使用核数为 2。

示例 52

```
> set_host_options -submit_protocol sge \
    -submit_command { qsub -P normal }
> analyze_rail -submit_to_other_machines -voltage_drop \
    static -nets {VDD VSS}
```

上例启用向 SGE farm 机器提交作业。

示例 53

```
> set_host_options -submit_protocol lsf \
  -submit_command {bsub}
> analyze_rail -submit_to_other_machines \
  -multiple_script_files {{result1 RH1.tcl} {result2 RH2.tcl}}

> open_rail_result result1
> close_rail_result
> open_rail_result result2
```

上例用两个 RedHawk 脚本向 LSF farm 机器提交两个 RedHawk 作业，并将保存分析结果的目录指定为 result1 与 result2。

示例 54

```
> set_app_options -name rail.license_checkout_mode \
  -value wait
> set_host_options -submit_protocol sge \
  -submit_command {qsub}
> analyze_rail -submit_to_other_machines \
  -multiple_script_files {{ result1 RH1.tcl} {result2 RH2.tcl} \
  {result3 RH3.tcl}}
```

上例向 SGE farm 机器提交三个 RedHawk 脚本，并在 analyze_rail 运行期间等待可用许可证。

12.14 电压驱动电源开关单元尺寸调整

Voltage Driven Power Switch Cell Sizing

Fusion Compiler 工具可调整电源开关单元尺寸，以降低总功耗并缓解设计中的压降。电源开关单元具有不同阈值电压（VT），分为 high、standard、low 与 ultra-low。高阈值电压的电源单元功耗与漏电流较低，但时序较慢、驱动强度较小；低阈值电压的电源单元时序较快，但功耗与漏电流较高。通过电源开关单元尺寸调整，工具支持不同驱动强度与阈值电压，以降低设计中的 IR（电压）降。

注意

当工具切换电源单元（交换）时，只能交换 footprint 相同但阈值电压不同的单元。这意味着若单元尺寸与 pin 形状不同，则无法交换。

要执行电源开关单元交换，可使用 size_power_switches 设置最大 IR drop 目标。交换电源单元以满足该目标时，工具尝试寻找 VT 更高但 R_{ON} 更低的另一单元，从而降低 IR drop。可用 set_power_switch_resistance 设置工具在交换时考虑的 ON 电阻（ R_{ON} ）值。基于 IR drop 目标与电源单元设定的 R_{ON} ，工具尝试替换电源单元以满足目标压降。

注意

若工具找不到匹配目标压降与 R_{ON} 的合适电源单元，则跳过单元尺寸调整。

要执行电压驱动的单元尺寸调整：

1. 使用 RH Fusion 执行压降分析，获得由相关电源开关驱动的单元的最大压降。详见「执行压降分析」。
2. 使用 `set_power_switch_resistance` 设置单元切换时考虑的 R_{ON} 值限值。提供：
 - Lib cell name 一库单元名。
 - On resistance value 一单元尺寸调整时考虑的 R_{ON} 值限值。可从开关模型文件获取该信息。

例如：

```
fc_shell> set_power_switch_resistance \
saed32hvt_pg_ss0p_v125c/HEAD2X2_HVT 0.015
```

3. 运行 `size_power_switches` 执行单元尺寸切换。提供 `-max_irdrop` 的值，指定由开关驱动的标准单元在电源 net 上允许的最大有效压降。此为必填输入。例如：

```
fc_shell> size_power_switches -max_irdrop 0.05
```

关于进一步细化命令行为的其它非必填输入，见 `size_power_switches` 的 man page。

工具还提供下列命令以进一步协助电源开关电阻：

- `get_power_switch_resistance` 一获取电源开关单元的 R_{ON} 值。
- `reset_power_switch_resistance` 一重置为电源开关单元指定的 R_{ON} 值。
- `report_power_switch_resistance` 一报告所有已设置 R_{ON} 的电源开关库单元。使用 `-verbose` 可显示 footprint 相同的单元组、 R_{ON} 值以及设计中例化的单元数。使用 `-verbose` 时还可使用 `-same_vt`，报告当前库中 VT 相同但驱动强度不同的电源开关单元及其 R_{ON} 。

12.15 使用宏模型

Working With Macro Models

在先进工艺技术的设计中使用存储器与宏单元，以降低内存使用并改善轨分析性能。宏模型在 RedHawk Fusion 与 RedHawk-SC Fusion 分析流程中均受支持。

关于使用宏模型，见下列主题：

- 生成宏模型（Generating Macro Models）
- 创建 Block Context（Creating Block Contexts）

12.15.1 生成宏模型

Generating Macro Models

要在分析层次化设计时考虑存储器 block 内部的电流与功耗分布，使用 `exec_gds2rh` 为存储器 block 生成 RedHawk 宏模型。该命令在底层调用 RedHawk `gds2rh` 实用程序，从存储器 block 抽取电源网格与电流源，抽象级别从用于准确 block 级分析的完全详细模型到用于全芯片动态分析的抽象模型不等。

注意

要使用 RedHawk gds2rh 实用程序创建 RedHawk 宏模型，必须在运行 `exec_gds2rh` 之前用 `rail.redhawk_path` 指定 RedHawk 可执行程序路径。

生成的宏模型保存在当前工作目录或配置文件中指定的目录。要使用生成的宏模型运行轨分析，在运行 `analyze_rail` 之前设置 `rail.macro_models -value {cell11 path1}`。

关于如何运行 RedHawk gds2rh 实用程序，见 *RedHawk User Manual*。

所需配置文件

Required Configuration File

要使用 `gds2rh` 生成 RedHawk 宏模型，必须提供包含下列信息的配置文件：

- GDS II 文件
- 顶层设计名
- 层映射文件
- 电源与地 net 名

每个存储器指定一个配置文件。确保配置文件包含正确的宏模型生成语法，否则工具发出错误消息并终止模型生成过程。

配置文件示例：

```
LEF_FILE {
    ./lef/saed32sram.lef
}
GDS_FILE ./gds/SRAMLP2RW128x16.gds
GDS_MAP_FILE ./redhawk.gdslayermap
OUTPUT_DIRECTORY SRAMLP2RW128x16_output

USE_LEF_PINS_FOR_TRACING 1

VDD_NETS {
    VDD
}
GND_NETS {
    VSS
}
TOP_CELL SRAMLP2RW128x16
```

示例

Examples

为 SRAMLP2RW128x16、SRAMLP2RW32x4、SRAMLP2RW64x32 与 SRAMLP2RW64x8 存储器生成 RedHawk 宏模型：

```
## Specify the path to the RedHawk executable ##
set_app_options -name rail.redhawk_path -value $env(REDHAWK_IMAGE)/bin

## Specify one configuration file per memory ##
exec_gds2rh -config_file SRAMLP2RW128x16.config
exec_gds2rh -config_file SRAMLP2RW32x4.config
exec_gds2rh -config_file SRAMLP2RW64x32.config
exec_gds2rh -config_file SRAMLP2RW64x8.config
```

使用上一示例生成的宏模型运行顶层轨分析：

```
## Run rail analysis at top level ##

open_lib
open_block top
link_block

## Specify rail analysis setting ##
create_taps
set_app_options -name rail.tech_file -value test.tech
set_app_options -name rail.lib_files -value test.lib
...
## Set macro model file ##
set_app_options -name rail.macro_models \
  -value { SRAMLP2RW128x16 ./data_ SRAMLP2RW128x16
           SRAMLP2RW32x4  ./data_ SRAMLP2RW32x4
           SRAMLP2RW64x32 ./data_ SRAMLP2RW64x32
           SRAMLP2RW64x8  ./data_ SRAMLP2RW64x8
         }

## Run rail analysis ##
analyze_rail -voltage_drop static -all_nets

## Check rail results ##
open_rail_result
gui_start
```

12.15.2 创建 Block Context

Creating Block Contexts

宏模型用于缩短先进工艺技术设计的总周转时间。当全芯片签核压降分析在宏中报告违例时，在 block 级修复违例再重跑全芯片分析以验证修复可能耗时较长。

为缩短周转时间，可为宏 block 创建 context 模型。Context 模型描述 block 与全芯片设计之间的关系（含物理与电气信息），使你可通过运行 block 级分析而非全芯片分析来验证修复。

要创建 context 模型，使用 `create_context_for_sub_block`。

要使用生成的 block context 运行压降分析，在运行 `analyze_rail` 之前设置 `rail.pad_files` 与 `rail.block_context_model_file`。

注意

为 block 创建全芯片 context 仅在 RedHawk Fusion 中受支持。

图 12.14 示出 block 中的修复可有效改善顶层设计性能。



图 12.14: 在 Block Context 中修复违例

Block Context 的内容

Content of Block Contexts

`create_context_for_sub_block` 对 block 的全芯片 context 建模，包括物理模型与电气模型。

- 物理模型：描述 block 与全芯片设计之间连接的位置。这些 tap 位置通过追踪 RedHawk 寄生数据得到。物理模型在 `create_context_for_sub_block` 完成后以 `.ploc` 格式生成。
- 电气模型：基于有效电阻与电容的模型。工具使用原生电阻引擎计算有效电阻，用于峰值动态压降分析期间的等效电流计算。默认工具为指定 block 建模全芯片 context。电容数据由 RedHawk 抽取引擎得到。

要为指定 block 创建电气模型，对 `create_context_for_sub_block` 指定 `-electric_model_file`。电气模型以 `.context` 格式写出。

示例

Examples

为 `top/cell_inst_1` block 创建 block context。生成的物理与电气模型分别为 `block.ploc` 与 `block.context`：

```
##Create block context in top block ##
open_block top
create_context_for_sub_block \
  -block_instance top/cell_inst_1 block.ploc \
  -nets {VDD_TOP VSS_TOP} \
  -electric_model_file block.context
close_block

##Run IR analysis with block context information in block-level design ##
open_block block
set_app_options -name rail.pad_files -value block.ploc
```

```
set_app_options -name rail.block_context_model_file \
    -value block.context
analyze_rail -nets {VDD_BLOCK VSS_BLOCK} \
    -voltage_drop static -extra_gsr_option_file extra.gsr
...
```

仅为 top/cell_inst_1 block 创建物理模型:

```
##Create block context in top block ##
open_block top
create_context_for_sub_block \
    -block_instance top/cell_inst_1 block.ploc \
    -nets {VDD_TOP VSS_TOP}
close_block

##Run IR analysis with block context information in block-level design ##
open_block block
set_app_options -name rail.pad_files -value block.ploc
analyze_rail -nets {VDD_BLOCK VSS_BLOCK} \
    -voltage_drop static -extra_gsr_option_file extra.gsr
...
```

12.16 执行签核分析

Performing Signoff Analysis

RedHawk Fusion 或 RedHawk-SC Fusion 不支持 RedHawk 签核分析能力，例如信号电迁移或浪涌电流（inrush current）分析。若拥有 RedHawk 签核许可证，可通过 rail.allow_redhawk_license_checkout 在 Fusion Compiler 环境中启用下列 RedHawk 签核分析功能。未列入下列列表的签核分析功能只能用 RedHawk 独立工具运行。

- 使用定制宏模型的动态分析
- 使用 package 模型的动态分析
- 使用 RTL 级 VCD 文件的分析

要执行 RedHawk 签核分析：

1. 修改 GSR 文件或 RedHawk 运行脚本以包含相关配置设置。
2. 设置下列应用选项以启用 RedHawk 签核功能：

```
fc_shell> set_app_options \
    -name rail.allow_redhawk_license_checkout -value true
```

将 rail.allow_redhawk_license_checkout 设为 true 允许 RedHawk 工具在 Fusion Compiler 内检索相关 RedHawk 许可证以执行签核功能。默认为关闭。

3. 运行带 -extra_gsr_option_file 或 -redhawk_script_file 的 analyze_rail，执行 GSR 或 RedHawk 脚本文件中定义的分析。按需指定其它设置。

注意

无法在 Fusion Compiler GUI 中显示 RedHawk 签核分析结果。

4. 继续分析流程中的其它步骤。

参见

RedHawk Fusion 与 RedHawk-SC Fusion 概述

12.17 写出分析与检查报告

Writing Analysis and Checking Reports

检查或分析完成后，运行 `report_rail_result` 将分析或检查结果写出到文本文件。报告文件中功耗单位为瓦特，电流单位为安培，电压单位为伏特。

要为调试目的从顶层 `RAIL_DATABASE` 显示 `block` 级轨结果，使用 `-top_design` 与 `-block_instance`。详见「显示 Block 级轨结果」。

要仅写出热点区域内单元或几何的轨结果，使用 `get_instance_result` 与 `get_geometry_result`。详见「生成基于实例的分析报告」与「生成基于几何的分析报告」。

使用 `-type` 指定要报告的数据或错误类型。表 12.5 列出支持的数据与错误类型。

使用 `-limit` 限制要报告的元素数量。设为零则按降序将全部元素写出到输出文件。

使用 `-threshold` 过滤写出到报告文件的数据。工具将大于指定阈值的数据写出到输出文件。该选项仅对下列数据类型有效：

- `pg_pin_power`
- `voltage_drop_or_rise`
- `effective_voltage_drop`

使用 `-supply_nets` 将报告限制到指定电源 `net`。该选项仅对下列数据类型有效：

- `effective_voltage_drop`
- `instance_minimum_path_resistance`
- `minimum_path_resistance`
- `pg_pin_power`
- `voltage_drop_or_rise`

在输出报告中，工具以不同格式列出数据。例如：

- 对于 `effective_voltage_drop` 类型：

– 报告静态分析结果时，格式为：

```
cell_instance_pg_pin_name mapped_pg_pin_name supply_net_name
effective_voltage_drop
```

– 报告动态分析结果时，格式为：

表 12.5: report_rail_result 支持的数据与错误类型（原书 Table 69）

类型	说明
effective_voltage_drop	有效电压值，单位伏特。
effective_resistance	有效电阻值。
instance_minimum_path_resistance	每个实例上的最小路径电阻值。
instance_power	每个实例上的功耗值，单位瓦特。
minimum_path_resistance	每个电源或地 pin 上的最小路径电阻值。
missing_vias	不同层上电源 net 布线重叠但无 via 连接的区域。
pg_pin_power	每个电源或地 pin 上的功耗值，单位瓦特。
unconnected_instances	未连接到任何理想电压源的实例。
voltage_drop_or_rise	动态压降或电压上升违例，单位伏特。

```
cell_instance_pg_pin_name mapped_pg_pin_name supply_net_name
  average_effective_voltage_drop_in_tw
  max_effective_voltage_drop_in_tw
  min_effective_voltage_drop_in_tw
  max_effective_voltage_drop
```

- 对于 instance_minimum_path_resistance，结果按 Total_R 值排序，格式为：

```
Supply_net Total_R(ohm) R_to_power(ohm) R_to_ground(ohm)
  Location Pin_name Instance_name
```

- 对于 minimum_path_resistance:

```
full_path_cell_instance_name/pg_pin_name resistance
```

- 对于 missing_vias:

```
net_name via_location_x_y top_metal_layer bottom_via_layer
  delta_voltage
```

- 对于 pg_pin_power:

```
full_path_cell_instance_name pg_pin_name power
```

- 对于 voltage_drop_or_rise:

```
full_path_cell_instance_name/pg_pin_name voltage
```

- 对于 unconnected_instances 错误类型，格式取决于错误条件：

- 当电源与/或地 net 与理想电压源物理断开时：

```
unconnected_type {net_names} instance_name instance bbox
```

- 当电源或地 net 浮空或逻辑浮空但物理仍连接到理想电压源时：

```
unconnected_type {net_names} instance_name:pin_name instance bbox
```

写出前五个 PG pin 功耗值到 power.rpt:

```
fc_shell> open_rail_result
fc_shell> report_rail_result -type pg_pin_power -limit 5 \
    -supply_nets { VSS } power.rpt
fc_shell> sh cat power.rpt
FI2/vss 4.81004e-06
FI6/vss 4.80959e-06
FI3/vss 4.79702e-06
FI4/vss 4.79684e-06
FI1/vss 4.79594e-06
...
```

写出 VDD 与 VSS 的有效压降值到 inst_effvd.rpt:

```
fc_shell> open_rail_result
fc_shell> report_rail_result -type effective_voltage_drop \
    -supply_nets { VDD VSS } inst_effvd.rpt
```

写出单元实例的最小路径电阻值到 inst_minres.rpt:

```
fc_shell> open_rail_result
fc_shell> report_rail_result -type instance_minimum_path_resistance \
    -supply_nets { VDD VSS } inst_minres.rpt
fc_shell> sh cat inst_minres.rpt

Rmax(ohm) = 162.738
Rmin(ohm) = 4.418
=====
Supply_net Total_R(ohm) R_to_power(ohm) R_to_ground(ohm)
Location Pin_name Instance_name
=====
VDD 162.738 152.686 10.052 586.000,
496.235 VDDL dataout_44_u
VSS 156.266 137.963 18.303 544.000,
613.235 VSS GPRs/U2317
...
```

12.17.1 显示 Block 级轨结果

Displaying Block-Level Rail Results

对顶层设计执行轨分析后，工具将分析结果保存在顶层轨数据库中。要定位 block 级设计中报告的问题，运行带 `-top_design` 与 `-block_instance` 的 `open_rail_result`，从顶层轨数据库摘录 block 级实例数据。工具随后基于摘录的 block 级轨分析数据为指定 block 实例显示地图。

注意

block 级轨结果的摘录保存在内存中，退出当前会话时删除。

下例打开 ./RAIL_DATABASE 目录中名为 REDHAWK_RESULT 的轨结果（顶层设计 bit_coin 的轨分析结果），然后为 block 实例 slice_5 创建轨分析数据摘录。注意：此例中必须打开实例 slice_5 的 block 设计，而非顶层设计。

```
fc_shell> open_lib block.nlib
fc_shell> open_block block
fc_shell> set_app_options -name rail.database -value RAIL_DATABASE
fc_shell> open_rail_result REDHAWK_RESULT \
    -top_design bit_coin \
    -block_instance slice_5
fc_shell> report_rail_result
```

12.17.2 生成基于实例的分析报告

Generating Instance-Based Analysis Reports

使用 get_instance_result, 将带轨数据的单元实例写出到输出文本文件, 单位由 report_units 定义。

使用 -type 指定要报告的分析或检查类型。支持的分析类型: effective_voltage_drop、current、effective_resistance、min_path_resistance 与 power。

get_instance_result 的常用选项:

选项	说明
-net	(可选) 指定要报告的 net。未指定时报告全部电源 net 的数据。
-touching	(可选) 报告触及指定矩形区域（格式 {{x1 y1} {x2 y2}}）的单元实例。
-top value	(可选) 报告压降值最大的前 N 个单元实例。
-threshold	(可选) 报告压降值大于指定阈值的单元实例。所有分析类型均支持。
-percentage	(可选) 报告压降值大于理想压降值指定百分比的单元实例。仅 voltage_drop 类型支持。默认为 0.1，即理想电源电压的 10%。
-histogram	(可选) 在报告中包含显示每 net 平均压降值的直方图。
-collection	(可选) 将单元实例报告为单元集合，而非基于列的文本报告。

示例——报告指定区域内的实例压降结果:

```
fc_shell> get_instance_result -net VDD -type voltage_drop \
    -touching {{1610 1968} {1620 1969}}
Isw2/Isw2_pktpro/U179070 -0.053533
Isw2/Isw2_pktpro/U64076 -0.0535949
...
```

参见

电压热点分析

12.17.3 生成基于几何的分析报告

Generating Geometry-Based Analysis Reports

使用 `get_geometry_result`，将带轨结果的几何信息写出到输出文件，单位由 `report_units` 定义。

一次只能指定一个 `net`；指定多于一个 `net` 时显示错误消息。使用 `-type` 指定分析或检查类型。支持的分析类型：`voltage_drop` 与 `min_path_resistance`。

报告格式：

```
geometry bbox_value
```

`get_geometry_result` 的常用选项：

选项	说明
<code>-layer</code>	（可选）指定要报告的层名或层号。未指定时报告全部层。
<code>-touching</code>	（可选）指定矩形区域（格式 <code>{{x1 y1} {x2 y2}}</code> ），报告触及该区域的全部几何层。
<code>-top value</code>	（可选）报告压降值最大的前 N 个几何层。
<code>-threshold</code>	（可选）指定阈值；报告压降值大于该阈值的几何层。
<code>-percentage</code>	（可选）指定百分比；报告压降值大于理想电源电压该百分比的几何层。例如百分比为 0.1 时，报告大于理想电源电压 10% 的几何。
<code>-histogram</code>	（可选）在报告中包含显示每 <code>net</code> 平均压降值的直方图。

示例——写出指定区域内带压降值的几何数据：

```
fc_shell> get_geometry_result -net VDD -type voltage_drop \
    -touching {{1610 1968} {1620 1969}}
{ { 1616.475 1967.000 } { 1617.525 1969.800 } } metal5 0.00448
{ { 1608.810 1946.000 } { 1611.190 1974.000 } } metal7 0.00446
{ { 1608.000 1946.000 } { 1612.000 1974.000 } } metal9 0.00446
```

参见

电压热点分析

12.18 在 GUI 中显示地图

Displaying Maps in the GUI

分析完成后，工具将设计数据与分析结果（`*.result`）保存在 RedHawk 工作目录下的 `in-design.redhawk/design_name.result` 目录中。必须先运行 `open_rail_result` 加载分析结果，然后才能在 Fusion Compiler GUI 中显示地图。

表 12.6 列出 RedHawk Fusion 分析流程中支持的地图类型。

要在 Fusion Compiler GUI 中显示地图：

- 要通过 GUI 加载轨分析结果，选择 `Task > Rail Analysis > 2D Rail Analysis`。出现 `Block Level 2D Rail Analysis` 窗口后，单击 `open_rail_result` 链接。

或者运行 `open_rail_result`，加载分析完成后保存在 RedHawk 工作目录下 `design_name` 目录中的设计数据与分析结果（`*.result`）。

表 12.6: RedHawk Fusion 流程中支持的分析地图（原书 Table 70）

地图类型	说明
Rail voltage drop map	显示压降地图：叠加在物理电源 net 上的彩色编码压降值。静态分析显示平均压降；动态分析显示峰值压降、平均压降或峰值电压上升。注意：即使 Fusion Compiler 库文件中不存在某些层，工具仍在压降地图上显示 LEF 或 DEF 文件中的全部层。
Rail parasitics map	压降分析完成后，显示 block 中给定电源 net 的寄生电阻；显示该 net 任意给定形状的电阻。
Rail power map	压降分析完成后，基于 RedHawk 功耗计算结果显示基于实例的功耗地图或功耗密度地图。基于实例的功耗地图：显示 block 中单元实例的功耗值。功耗密度地图：显示 block 中区域的功耗密度值。
Rail current map	压降分析完成后显示 block 的电流分布。
Rail minimal path resistance map	最小路径电阻分析完成后，显示所选 net 的最小路径电阻值。
Rail electromigration map	静态或动态电迁移分析完成后，显示 block 的电流违例。
Rail instance peak current map	显示 block 中不同实例或区域的峰值电流值。
Rail instance effective voltage drop map	压降分析完成后，显示 block 中给定实例的有效电压值。
Rail instance effective resistance map	有效电阻分析完成后，显示 block 中给定实例的有效电阻值。
Rail instance switch cell	动态分析完成且开关单元模型输入文件可用时，显示 block 中开关单元的电流、有效电压与电压差地图。若无开关模型输入文件，工具将这些开关单元视为黑盒且不显示轨地图。基于实例的电流地图：显示电流如何流经所选开关单元。基于实例的有效电压地图：显示所选开关单元处的有效电压。基于实例的电压差地图：显示所选开关单元两端的电压差。

2. 在 GUI 布局窗口中，选择 View > Map 并选择要显示的地图。

在地图中，问题区域以不同颜色高亮。将指针移到实例上可在 InfoTip 中查看更多信息。默认工具显示 block 中全部实例的信息。要检查某一特定实例的信息，取消选择全部实例，再从列表中选择要显示的实例。

使用地图面板中的选项更改地图显示：

- Value: 设置要显示的地图类型
- Bins: 更改用于显示地图的唯一 bin 数量
- From: 与 To:: 设置功耗密度地图中显示的密度值范围
- Text: 在布局中显示计算值；若值不可见，放大以查看

12.18.1 示例

Examples

下列示例说明如何在 GUI 中显示各类地图：

- 图 12.15: 显示轨实例有效压降地图并检查热点。
- 图 12.16: 显示 block 中开关单元的轨地图并检查所选开关单元的电压信息。
- 图 12.17: 显示 block 的功耗密度地图。

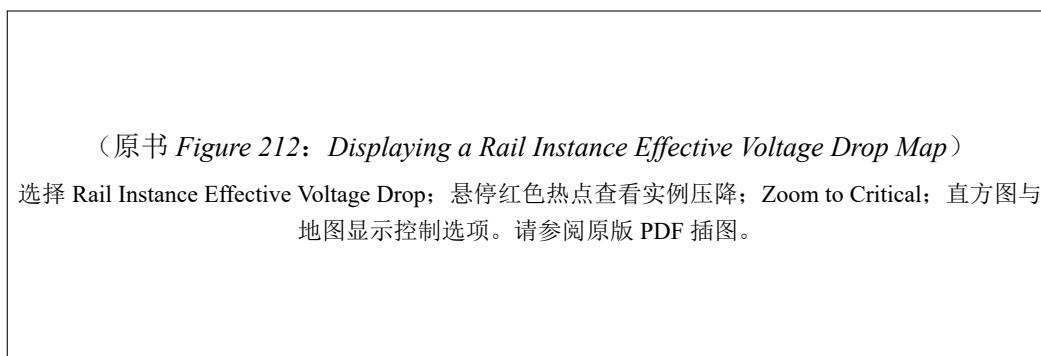


图 12.15: 显示轨实例有效压降地图

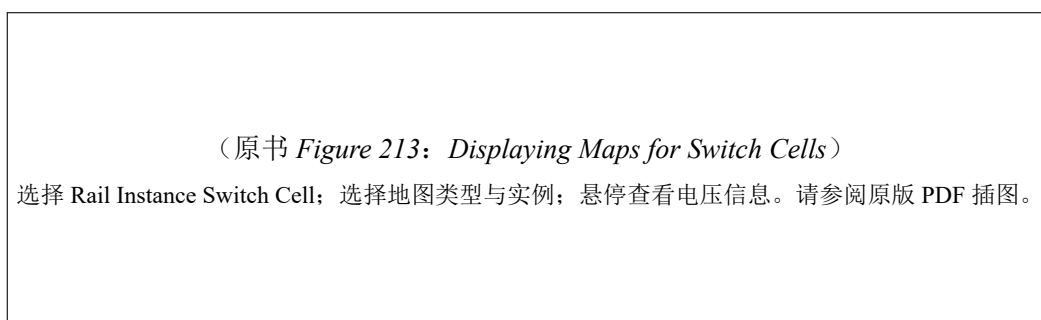


图 12.16: 显示开关单元地图

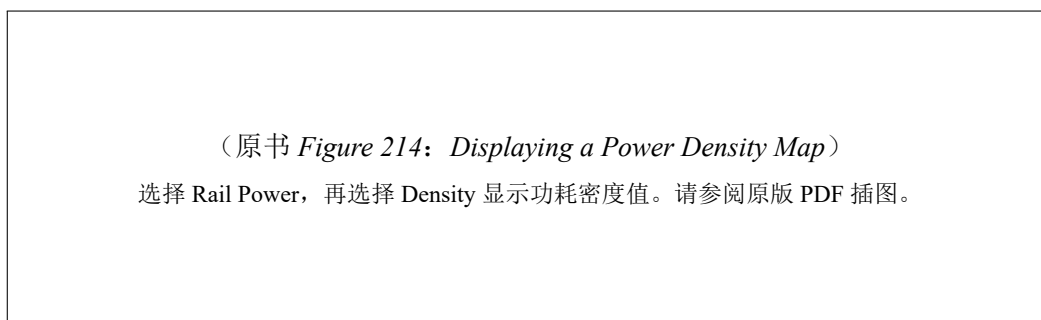


图 12.17: 显示功耗密度地图

参见

指定 RedHawk 与 RedHawk-SC 工作目录

12.19 在 GUI 中显示 ECO 形状

Displaying ECO Shapes in the GUI

当使用 RedHawk `mesh add` 命令在顶层 block 中为当前子 block 添加虚拟 PG mesh 时，默认这些虚拟 PG mesh 不会作为布局对象显示在 GUI 中。因此，这些 GUI 形状无法用布局对象集合命令（例如 `get_shapes`）查询，保存 block 时也不会保存到设计库。

要将 ECO 形状与轨结果一起保存并在 GUI 中显示，启用 `rail.display_eco_shapes`：

```
set_app_options -name rail.display_eco_shapes -value true
```

注意

`mesh add` 命令仅在 RedHawk 中可用，不在 RedHawk-SC 中可用。因此，若同时启用 `rail.enable_redhawk_sc` 与 `rail.display_eco_shapes`，工具会发出错误消息。
要在 GUI 中显示 ECO 形状，必须先将 `rail.allow_redhawk_license_checkout` 设为 `true` 以启用 RedHawk 签核许可证密钥，否则发出错误消息。

要添加虚拟 PG mesh 并在 GUI 中显示所创建的 ECO 形状：

1. 准备包含 `mesh add` 命令的脚本文件（例如 `mesh.tcl`）。
2. 使用 `mesh.tcl` 脚本文件运行设计设置：

```
fc_shell> set_rail_command_options -script_file mesh.tcl \
    -command setup_design -order after_the_command
```

工具生成 RedHawk 脚本文件，并在设计设置之后 `source mesh.tcl`。

3. 使用下列命令执行轨分析：

```
fc_shell> analyze_rail -nets -voltage_drop
```

4. 运行 `open_rail_result` 加载设计数据与分析结果。
5. 运行 `gui_start` 打开 Fusion Compiler GUI。在 GUI 布局窗口中选择 `View > Map` 并选择要显示的地图。

此时可在 GUI 中显示并查询由 RedHawk `mesh add` 创建的 ECO 形状。

12.19.1 脚本示例

Script Examples

使用 `set_rail_command_options` 在 GUI 中显示 ECO 形状：

```

## Open design ##
open_block
link_block
## Specify taps ##
create_taps
## Specify RedHawk Fusion input files or variables ##
set_app_options -name rail.enable_redhawk -value 1
set_app_options -name rail.redhawk_path -value
set_app_options -name rail.disable_timestamps -value true
set_app_options -name rail.display_eco_shapes -value true
set_rail_command_options -script_file mesh.tcl -command \
    setup_design ...
## Analyze##
analyze_rail -voltage_drop ... -nets {VDD VSS}
## Check GUI ##
open_rail_result

```

使用 RedHawk 脚本在 GUI 中显示 ECO 形状:

```

## Open design ##
open_block
link_block
## Specify taps ##
create_taps
## Specify RedHawk Fusion input files or variables ##
set_app_options -name rail.enable_redhawk -value 1
set_app_options -name rail.redhawk_path -value
set_app_options -name rail.disable_timestamps -value true
set_app_options -name rail.display_eco_shapes -value true
## Analyze ##
analyze_rail -voltage_drop ... -nets {VDD VSS} -script_only
## Modify the script.tcl containing mesh add command ##
analyze_rail -redhawk_script_file
## Check GUI ##
open_rail_result

```

12.20 电压热点分析

Voltage Hotspot Analysis

轨分析完成后，分析报告中可能报告数百到数千个压降违例，从而难以识别违例的根本原因。

使用电压热点分析能力，通过将整个芯片划分为小网格框为电源或地 net 生成热点，并报告所生成网格框的轨相关信息。此外，可使用 `get_instance_result` 与 `get_geometry_result` 仅查询特定分析类型的基于单元或基于几何的轨结果。

热点分析提供下列功能：

- 识别高压降 aggressor 的根本原因，例如 aggressor 自身的大电流，或因时序窗重叠导致同时翻转的邻近单元的电流。

- 根据报告确定用于修复电压违例的设计技术，例如选择候选参考单元进行尺寸替换、移动或重定位单元以降低压降，或应用 PG 增强以解决压降问题。

本节包含下列主题：

- 生成热点（Generating Hotspots）
- 报告热点（Reporting Hotspots）
- 移除热点（Removing Hotspots）
- 电压热点分析示例（Voltage Hotspot Analysis Examples）

要仅写出位于热点区域内的单元实例或几何的轨结果，使用 `get_instance_result` 与 `get_geometry_result`。详见「生成基于实例的分析报告」与「生成基于几何的分析报告」。

12.20.1 生成热点

Generating Hotspots

要为电源或地 net 生成压降热点，使用 `generate_hot_spots`。该命令将整个芯片区域按行与列划分为多个网格（例如 100×100）。网格框按每个网格框中的最大有效压降值排序，并用从 0 开始的整数索引。

要基于标准单元 site row 创建行，并用最低金属层上的垂直 PG strap 创建列，使用 `-site_row`。或者，用 `-row` 与 `-column` 指定划分整个区域的行数与列数。

命令为压降值大于理想压降指定百分比的网格创建错误单元。默认为 0.1，即对压降值大于理想电源电压 10% 的网格创建错误单元。错误单元类型为有效压降。

要检查生成的错误单元内容，在错误浏览器中打开错误单元。要检查网格框内容，使用 `report_hot_spots`。要从内存中移除生成的电压热点数据，使用 `remove_hot_spots`。

选项	说明
-net	指定生成热点的 net 名。未指定时使用全部电源 net。
-percentage	指定用于生成热点错误单元的百分比。默认为 0.1。
-site_row integer	默认使用标准 site row 高度将整个区域划分为多行，用最低 PG net 垂直 strap 划分为多列。要使用 site row 高度的倍数创建网格行，指定带整数的 -site_row。例如设为 2 时，每两个 site row 构造一行。
-row	指定划分整个区域的行数。
-column	指定划分整个区域的列数。

参见

报告热点；移除热点；电压热点分析示例

12.20.2 报告热点

Reporting Hotspots

使用 `generate_hot_spots` 在 block 上生成热点后，运行 `report_hot_spots`，报告热点网格框中带电压相关单元属性的单元。电压相关单元属性包括：current、overlap_current、effective_resistance、timing_window、slack、output_load 与 static_power。

使用这些电压相关单元属性识别电压违例的根本原因。例如，可以：

- 使用电流与输出负载电容信息，判断单元的高峰值是否由负载过大引起。若是，拆分输出可能是降低高压降的有效方法。
- 检查并比较当前热点与邻近网格的时序窗，判断单元是否可移到其它网格（目标）。这样做可降低原网格的压降，而不增加目标网格的压降。
- 比较区域内全部单元的 slew 与负载电容，判断高峰值电流是否由陡峭 slew 或大负载电容引起。
- 查找时序窗重叠的单元，识别动态轨分析期间可能同时翻转的单元。

`report_hot_spots` 的常用选项：

选项	说明
<code>-index</code>	按索引列出热点网格。
<code>-summary</code>	报告全部网格的摘要，例如创建的网格数、 <code>block</code> 中的实例数，以及压降值高于理想压降指定百分比的网格数。
<code>-object</code>	指定要报告的对象名；列出包含指定单元实例对象的热点网格。
<code>-verbose</code>	打印详细报告。

按索引列出电压网格框

Listing Voltage Grid Boxes by Index

默认情况下，工具向上下各搜索三行、向左右各搜索三列，以确定当前网格的邻近网格。使用 `-index` 指定用于报告电压热点网格的索引。

索引号最小的网格框具有最高的有效压降值。从而可确定热点区域的大小。例如，若周围网格索引号都较大，而仅少数中心网格索引号小得多，则假定该压降热点为岛状（`island`）。

累积电流

Accumulating Current

对于网格框中的实例，通过累积其邻近单元上基于时序窗的电流来估计重叠电流（`overlap current`）。

命令按时序窗从小到大扫描，并随时间流逝累积电流。利用该信息可判断有多少单元同时翻转，以及为降低压降偏移时序窗时应施加多少时间偏移。

假定设计有三个实例及其电流如下。如下列报告所示，点 2.0 处的累积电流来自 `inst2` 与 `inst3`，不包含 `inst1` 的电流：

```
inst1 [0.0 2.0] current 0.11
inst2 [1.0 2.5] current 0.22
inst3 [2.0 3.0] current 0.33
```

参见

生成热点；移除热点；电压热点分析示例

12.20.3 移除热点

Removing Hotspots

工具将生成的热点数据保存在内存中。若要在另一次运行中为其它 net 生成热点，需先用 `remove_hot_spots` 从内存中移除先前生成的热点数据，再继续另一次热点分析。

例如，若已对 VDD net 运行 `generate_hot_spots`，则必须先移除 VDD net 的热点数据，再为其它 net 生成热点。

参见

生成热点；报告热点；电压热点分析示例

12.20.4 电压热点分析示例

Voltage Hotspot Analysis Examples

本主题提供如何使用热点分析能力识别电压违例根本原因的示例。

时序窗重叠的单元可能同时翻转，从而导致高压降。图 12.18 说明如何识别单元是否具有重叠时序窗。

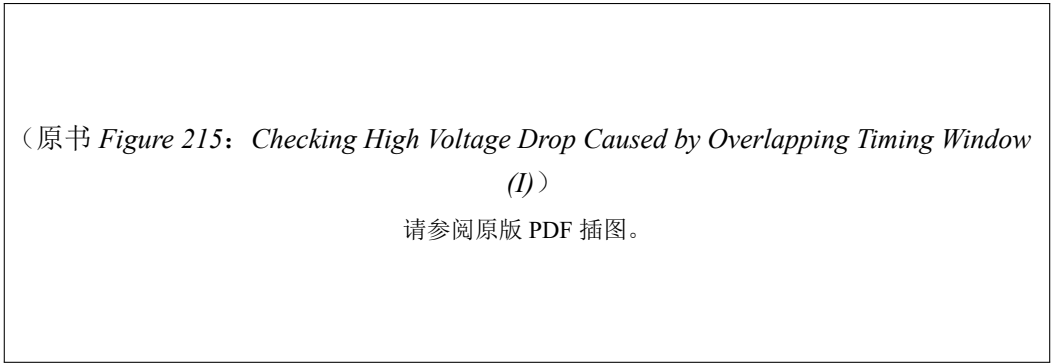


图 12.18: 检查由重叠时序窗引起的高压降 (I)

接着，运行带 `-index` 的 `report_hot_spots`，查找哪些网格具有导致大电流的重叠时序窗。如图 12.19 所示，当 `-index` 设为 5 时，网格 5 报告的总电流较小；当 `-index` 设为 2 时，网格 2 报告的总电流较大。此时，重叠时序窗导致网格 2 中的有效压降较高。

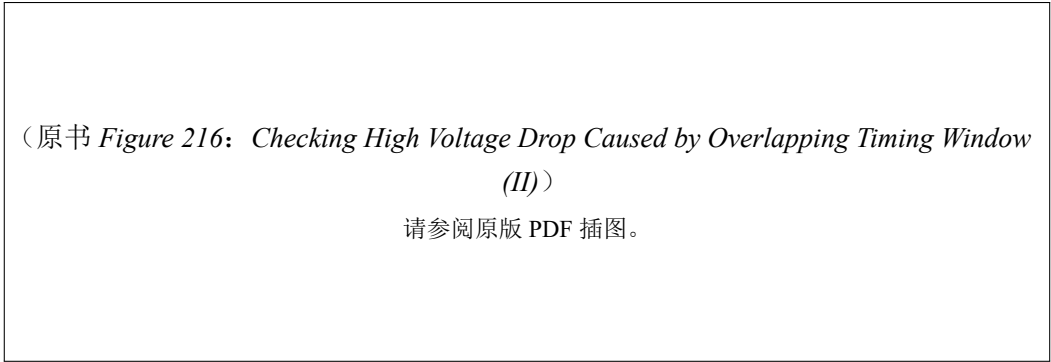


图 12.19: 检查由重叠时序窗引起的高压降 (II)

参见

生成热点；报告热点；移除热点

12.21 查询属性

Querying Attributes

轨分析或缺失 via 检查完成后，使用 `get_attribute` 查询存储在 RedHawk 工作目录中的结果。可查询表 12.7 所示的、专用于轨分析结果的属性。

表 12.7: 轨结果对象属性（原书 Table 71）

对象	属性
Cell	static_power
Pin	static_current; peak_current; static_power; switching_power; leakage_power; internal_power; min_path_resistance; voltage_drop; average_effective_voltage_drop_in_tw; max_effective_voltage_drop (= effective_voltage_drop) ; max_effective_voltage_drop_in_tw; min_effective_voltage_drop_in_tw; effective_resistance

下例检索 cell1A/VDD pin 的 effective_voltage_drop 属性：

```
fc_shell> get_attribute [get_pins cell1A/VDD] effective_voltage_drop
-0.02812498807907104
```

下例检索 cell1A 单元的 static_power 属性：

```
fc_shell> get_attribute [get_cells cell1A] static_power
2.014179968645724e-05
```

Chapter 13

ECO 流程

ECO Flow

工程变更指令（engineering change order，ECO）是对已完成或接近完成的设计所做的增量更改。可使用 ECO 在无需对整个设计重新综合、布局与布线的情况下，修复功能、时序、噪声与串扰违例；也可在保持设计性能的同时实现迟到的设计变更。

下列主题描述 Fusion Compiler 工具支持的各种 ECO 流程，以及在这些流程中执行的任务：

- 时序或功能变更的通用 ECO 流程（Generic ECO Flow for Timing or Functional Changes）
- Freeze Silicon ECO 流程（Freeze Silicon ECO Flow）
- Signoff ECO 流程（Signoff ECO Flow）
- 增量 Signoff ECO 流程（Incremental Signoff ECO Flow）
- ECO Fusion 流程（ECO Fusion Flow）
- ECO Fusion 电源完整性流程（ECO Fusion Power Integrity Flow）
- 手动实例化备用单元（Manually Instantiating Spare Cells）
- 自动添加备用单元（Automatically Adding Spare Cells）
- 添加可编程备用单元（Adding Programmable Spare Cells）
- 使用 `eco_netlist` 命令进行 ECO 更改（Making ECO Changes Using the `eco_netlist` Command）
- 使用网表编辑命令进行 ECO 更改（Making ECO Changes Using Netlist Editing Commands）
- 调整单元尺寸（Resizing Cells）
- 在 net 上添加缓冲器（Adding Buffers on Nets）
- 在已布线 net 上添加缓冲器（Adding Buffers on Routed Nets）
- 优化 net 扇出（Optimizing the Fanout of a Net）
- 报告放置 ECO 单元的可用 site（Reporting Available Sites for Placing ECO Cells）
- 识别并撤销非最优 ECO 更改（Identifying and Reverting Nonoptimal ECO Changes）

- 放置 ECO 单元 (Placing ECO Cells)
- 将 ECO 单元放置并映射到备用单元 (Placing and Mapping ECO Cells to Spare Cells)
- 更新 ECO 单元的电源 net (Updating Supply Nets for ECO Cells)
- 记录对版图所做的更改 (Recording the Changes Made to a Layout)
- 执行成组中继器插入与放置的先决条件检查 (Performing Prerequisite Check for Group Repeater Insertion and Placement)
- 添加一组中继器 (Adding a Group of Repeaters)
- 查询成组中继器 (Querying Group Repeater)
- 交换变体单元 (Swapping Variant Cell)
- 修复多电压违例 (Fixing Multivoltage Violations)

13.1 时序或功能变更的通用 ECO 流程

Generic ECO Flow for Timing or Functional Changes

当可以灵活地添加新单元并移动或删除现有单元时，使用此流程纳入时序或功能 ECO 更改。若设计尚未 tape out，推荐使用此流程。

无约束 (unconstrained) ECO 流程包含下列步骤：

1. 用下列方法之一用 ECO 更改更新设计：
 - 使用 `eco_netlist` 命令，见“使用 `eco_netlist` 命令进行 ECO 更改”。
 - 使用网表编辑 Tcl 命令，见“使用网表编辑命令进行 ECO 更改”。
2. 使用 `place_eco_cells` 命令更新布局，见“放置 ECO 单元”。
3. 向 site 阵列的空白处添加填充单元 (filler cells)，见“插入填充单元 (Inserting Filler Cells)”。为缩短运行时间，在下列情况下使用 `-post_eco` 选项：
 - 用 `create_stdcell_fillers` 命令插入金属填充单元时，工具将所插入的填充单元标记为 post-ECO 单元。
 - 用 `remove_stdcell_fillers_with_violation` 命令移除存在 DRC 违例的填充单元时，工具仅对 post-ECO 单元执行 DRC 检查。
4. 使用 `route_eco` 命令更新布线，见“执行 ECO 布线 (Performing ECO Routing)”，或手动重布受影响的 net。

13.2 Freeze Silicon ECO 流程

Freeze Silicon ECO Flow

若单元布局已固定，且只能更改金属与 via 掩模图形，则使用此流程。若设计已 tape out，并希望避免重新生成整套掩模的费用，推荐使用此流程。

要执行 freeze silicon ECO 流程，块中必须包含备用单元（spare cells）。可在设计流程的任意时刻用下列方法之一向块添加备用单元：

- 手动实例化备用单元，见“手动实例化备用单元”。
- 布局后用 `add_spare_cells` 命令自动添加备用单元，见“自动添加备用单元”。
- 在芯片收尾（chip finishing）阶段用 `create_stdcell_fillers` 添加可编程备用单元，见“添加可编程备用单元”。

Freeze silicon ECO 流程包含下列步骤：

1. 将 `design.eco_freeze_silicon_mode` 应用选项设为 `true`，以在 freeze silicon 模式下启用 ECO 更改。
2. 用下列方法之一用 ECO 更改更新设计：
 - 使用 `eco_netlist` 命令，见“使用 `eco_netlist` 命令进行 ECO 更改”。
 - 使用网表编辑 Tcl 命令，见“使用网表编辑命令进行 ECO 更改”。
3. 使用 `check_freeze_silicon` 命令分析 ECO 单元到备用单元的映射。
4. 使用 `place_freeze_silicon` 命令自动将全部 ECO 更改映射到备用单元，或使用 `map_freeze_silicon` 命令将每个 ECO 单元手动映射到特定备用单元，见“将 ECO 单元放置并映射到备用单元”。
5. 使用 `route_eco` 命令更新布线，见“执行 ECO 布线”，或手动重布受影响的 net。

13.3 Signoff ECO 流程

Signoff ECO Flow

在 Fusion Compiler 中完成布局布线后，若设计仍有时序或设计规则违例，可在 PrimeTime 中修复这些违例；也可在 PrimeTime 中进行功耗或面积回收（power or area recovery）。

若在 PrimeTime 中对设计做了更改，可生成 ECO 变更列表（change list）文件，并利用 Fusion Compiler 的 ECO 能力将这些更改纳入设计，如图 13.1 所示。

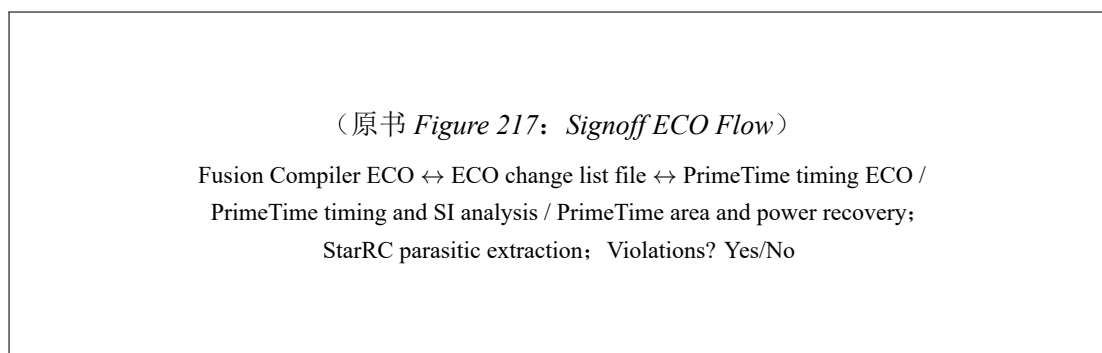


图 13.1: Signoff ECO 流程

纳入 PrimeTime ECO 更改的步骤如下：

1. 通过 source 含有网表编辑命令的 PrimeTime ECO 变更列表 Tcl 文件来更新设计。

2. 使用 `place_eco_cells` 命令更新布局，示例如下：

```
fc_shell> place_eco_cells -legalize_mode minimum_physical_impact \
    -eco_changed_cells -legalize_only -displacement_threshold 10
```

关于 `place_eco_cells` 的更多信息，见“放置 ECO 单元”。

3. 向 site 阵列空白处添加填充单元，见“插入填充单元”。为缩短运行时间，在下列情况下使用 `-post_eco` 选项：

- 用 `create_stdcell_fillers` 插入金属填充单元时，工具将所插入填充单元标记为 post-ECO 单元；
- 用 `remove_stdcell_fillers_with_violation` 移除存在 DRC 违例的填充单元时，工具仅对 post-ECO 单元执行 DRC 检查。

4. 使用 `route_eco` 更新布线，见“执行 ECO 布线”，或手动重布受影响的 net。

13.4 增量 Signoff ECO 流程

Incremental Signoff ECO Flow

在 PrimeTime 中做 ECO 更改并经 Fusion Compiler 无约束 ECO 流程纳入后，可能需要在工具间多次迭代才能达到所需 QoR 目标。为缩短整体周转时间，可用增量 signoff ECO 流程降低每次迭代的运行时间。该流程由 Fusion Compiler 生成增量设计数据，使你能在 StarRC 中做增量提取，并在 PrimeTime 中做增量时序分析与 ECO，如下图所示。

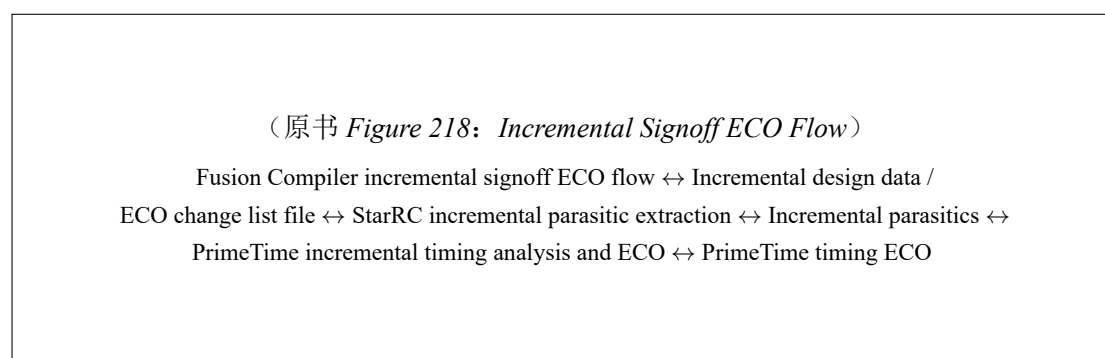


图 13.2: 增量 Signoff ECO 流程

在 Fusion Compiler 内执行增量 signoff ECO 流程时，使用 `record_signoff_eco_changes` 命令。该命令将 PrimeTime ECO 纳入设计库、跟踪对设计所做的全部更改，并生成运行 StarRC 增量提取以及 PrimeTime 增量时序分析与 ECO 所需的增量文件。

Fusion Compiler 增量 signoff ECO 流程包含下列步骤：

1. 使用 `open_block` 打开设计库。
2. 使用 `record_signoff_eco_changes -start -input` 纳入 PrimeTime ECO 更改并开始跟踪对设计的 ECO 更改，示例如下：

```
fc_shell> record_signoff_eco_changes -start -input pt_eco.tcl
```

3. (可选) 使用网表编辑命令对设计做额外时序 ECO 更改。

注意

不要对设计做功能 ECO 更改。若这样做，工具将停止跟踪正在对设计执行的 ECO 更改。

4. 使用 `place_eco_cells` 放置 ECO 单元。
5. 使用 `route_eco` 更新布线，见“执行 ECO 布线”，或手动重布受影响的 `net`。
6. 使用 `record_signoff_eco_changes -stop` 停止跟踪 ECO 更改并完成增量 signoff ECO 流程，示例如下：

```
fc_shell> record_signoff_eco_changes -stop
```

13.5 ECO Fusion 流程

ECO Fusion Flow

Fusion Compiler ECO Fusion 流程允许你在 Fusion Compiler 工具内使用 PrimeTime 物理感知 ECO 能力以及 StarRC 提取能力 (In-Design signoff extraction)。可在实现流程的极晚阶段用此流程修复时序与 DRC 违例，并改善面积与功耗 QoR。

Fusion Compiler ECO Fusion 流程包含下列步骤：

1. 使用 `set_pt_options` 指定在 Fusion Compiler 内运行 PrimeTime ECO 所需的设置。下列示例指定 PrimeTime 可执行文件路径以及分布式处理设置：

```
fc_shell> set_host_options -name pteco_host_option \
    -submit_command "/lsf/bin/bsub -R \"rusage[mem=$MEM]\" \"
fc_shell> set_pt_options -pt_exec /snps_tools/PT/pt_shell \
    -host_option pteco_host_option
```

若当前工作目录中存在 `.synopsys_pt.setup` 文件，Fusion Compiler 会使用其中的信息。也可用 `set_pt_options` 的 `-pre_link_script` 与 `-post_link_script` 指定额外的 PrimeTime 设置。更多信息见 `set_pt_options` 的 man page。

2. 按如下方式设置 StarRC 提取：

(a) 确保已启用 StarRC 提取。StarRC 提取由 `extract.starrrc_mode` 应用选项控制。默认值为 `fusion_adv`。也可设为 `in-design` 或 `none`。若要使用原生提取器 (native extractor)，将该值设为 `none`。

(b) 使用 `set_starrrc_options -config` 指定运行 StarRC 提取的配置文件。

3. 使用 `eco_opt` 命令执行优化。默认情况下，`eco_opt` 使用 PrimeTime ECO 能力修复时序 (setup 与 hold) 与 DRC 违例、移除冗余缓冲器，并改善总功耗 QoR；随后用 Fusion Compiler ECO 布局布线能力纳入这些 ECO 更改。可用 `-type` 选项控制优化类型。下列示例仅用穷尽路径分析 (exhaustive path-based analysis) 修复时序与 DRC 违例：

```
fc_shell> eco_opt -pba_mode exhaustive -types {timing drc}
```

下列示例改善漏电功耗 QoR，但不纳入 ECO 更改，而是生成包含 ECO 更改的文件：

```
fc_shell> eco_opt -pba_mode exhaustive -types {timing drc} \
  -write_change_file_only
```

更多信息见 `eco_opt` 的 man page。

4. 使用 `check_pt_qor` 分析 PrimeTime QoR。更多信息见该命令的 man page。

注意

运行 `eco_opt` 时，Fusion Compiler 会根据其时序分析设置（如 derating 因子、片上变化设置等）指定 PrimeTime 设置。因此，为获得最佳收敛，请确保 postroute 阶段的 Fusion Compiler 时序分析设置与 PrimeTime 一致。可用 `check_consistency_settings` 识别两者设置的差异。

13.6 ECO Fusion 电源完整性流程

ECO Fusion Power Integrity Flow

Fusion Compiler ECO Fusion 电源完整性流程允许你使用 RedHawk Fusion 电源轨分析功能识别电源完整性问题，并用 PrimeTime 物理感知 ECO 能力加以修复。

该流程包含下列步骤：

1. 设置 `rail.database` 应用选项，指定保存 RedHawk Fusion 电源轨分析结果的路径。
2. 使用 `analyze_rail -voltage_drop dynamic_vcd` 或 `analyze_rail -voltage_drop dynamic_vectorless` 执行基于动态向量或不基于向量的压降分析。
3. 使用 `set_pt_options` 指定在 Fusion Compiler 内运行 PrimeTime ECO 所需的设置。
4. 使用 `eco_opt -type power_integrity` 修复已识别的电源完整性问题。

为修复电源完整性（压降）违例，工具在考虑时序与 DRC QoR 的同时对攻击者单元（aggressor cells）进行缩小尺寸（downsize）。但该优化要有效，需满足：

- 攻击者单元不应处于时序或 DRC 关键；
- 单元库中应有更小版本的攻击者单元，且未设置 `dont_use` 或 `dont_touch` 属性。

下列为运行 ECO Fusion 电源完整性流程的示例脚本：

```
set_app_options -name rail.database -value RAIL_DATABASE_BASELINE
analyze_rail -voltage_drop dynamic_vcd -all_nets \
  -switching_activity {VCD ./top.vcd top}
set_pt_options -pt_exec /snps_tools/PT/pt_shell
eco_opt -type power_integrity
```

13.7 手动实例化备用单元

Manually Instantiating Spare Cells

可用下列方法之一在块中手动实例化备用单元：

- 在 Verilog 网表中实例化；
- 使用 `create_cell`、`connect_pins` 等网表编辑命令添加。

手动实例化备用单元时，应当：

- 在逻辑层次中均匀分布备用单元，以便更好地处理块内任意位置的 ECO 更改；
- 将其输入适当地接到电源或地，以防止备用单元产生噪声并消耗功耗。

若单元满足下列条件，工具会自动将其识别为备用单元：

- 不是仅物理单元（`physical-only cell`）；
- 除时钟 `pin` 外，所有输入均未连接或接到逻辑常数；备用单元的时钟 `pin`（若有）可连接到时钟网络；
- 所有输出均未连接。

若在对块执行物理综合之前实例化备用单元，工具会在后续物理综合步骤中放置并合法化这些备用单元。但若在已优化、已布局并合法化的块中实例化备用单元，则须用下列步骤放置并合法化：

1. 使用 `spread_spare_cells` 分散备用单元。默认情况下，该命令在整个 `core` 区域均匀分布并放置全部备用单元。可用下列两种方法之一指定要放置的备用单元：

- 用 `-cells` 放置特定备用单元；
- 用 `-voltage_areas` 放置属于特定电压区域的全部备用单元。

可如下控制备用单元放置：

- 用 `-boundary` 指定放置备用单元的区域；
- 用 `-ignore_blockage_types` 忽略特定类型的布局 `blockage`。默认情况下 `add_spare_cells` 会遵守所有布局 `blockage` 类型；
- 用 `-random_distribution` 忽略当前单元密度并随机放置备用单元；
- 用 `-density_aware_ratio` 指定按单元密度分布放置的备用单元百分比，其余随机放置。默认全部按密度分布放置。例如 `-density_aware_ratio 80` 表示 80% 按密度、20% 随机放置。

2. 使用 `place_eco_cells -legalize_only -cells` 合法化备用单元。

13.8 自动添加备用单元

Automatically Adding Spare Cells

布局与优化之后，可用下列步骤添加备用单元并合法化：

1. 使用 `add_spare_cells` 添加备用单元，并指定下列信息：

- 用 `-cell_name` 指定备用单元名称前缀；
- 用下列两种方法之一指定要插入的备用单元类型与数量：
 - 用 `-lib_cell` 与 `-num_instances` 指定库单元及每种实例数。例如各插入 250 个 AND2 与 OR2；

```
fc_shell> add_spare_cells -cell_name spare \
-lib_cell {AND2 OR2} -num_instances 250
```

- 用 `-num_cells` 为各库单元指定不同实例数。例如插入 200 个 NAND2 与 150 个 NOR2:

```
fc_shell> add_spare_cells -cell_name spare \
-num_cells {NAND2 200 NOR 150}
```

- (可选) 用 `-repetitive_window` 指定在布局区域中重复的放置窗口。例如在重复的 20×20 微米窗口中各添加 15 个 AND2 与 OR2:

```
fc_shell> add_spare_cells -cell_name spare \
-lib_cell {AND2 OR2} -num_instances 15 \
-repetitive_window {20 20}
```

在重复的 25×20 微米窗口中添加 20 个 NAND2 与 15 个 NOR2:

```
fc_shell> add_spare_cells -cell_name spare \
-num_cells {NAND2 20 NOR 15} -repetitive_window {25 20}
```

默认 `add_spare_cells` 在整个设计中均匀分布备用单元。要在特定范围内分布:

- 层次块: 用 `-hier_cell`。工具在包围该层次块全部单元的矩形区域内分布;
- 边界框: 用 `-boundary`;
- 电压区域: 用 `-voltage_areas`。

还可进一步控制放置:

- `-ignore_blockage_types`: 忽略特定类型布局 blockage;
- `-random_distribution`: 忽略密度并随机放置;
- `-density_aware_ratio`: 按密度放置的百分比 (其余随机);
- `-input_pin_connect_type`: 指定备用单元输入 pin 的连接类型, 有效值为 `tie_low`、`tie_high` 或 `open`。

2. 使用 `place_eco_cells -legalize_only -cells` 合法化备用单元。

13.9 添加可编程备用单元

Adding Programmable Spare Cells

若 vendor 提供, Fusion Compiler 支持可编程备用单元 (programmable spare cells), 亦称门阵列填充单元 (gate array filler cells)。这些单元可通过金属掩模更改进行编程以实现 ECO, 从而降低掩模成本并缩短结果交付时间。

插入可编程备用单元的步骤:

1. 通过对标准单元与可编程备用单元对应的库单元设置相同的 `psc_type_id` 属性, 指定标准单元可映射到的可编程备用单元。下列示例指定:

- 名为 BUF1 与 INV1 的标准单元可映射到名为 fill1x 的可编程备用单元（psc_type_id 设为 1）；
- 名为 NAND2 与 NOR2 的标准单元可映射到名为 fill2x 的可编程备用单元（psc_type_id 设为 2）。

```
fc_shell> set_attribute [get_lib_cells fill_lib/fill1x] \
    psc_type_id 1
fc_shell> set_attribute [get_lib_cells stdcell_lib/BUF1] \
    psc_type_id 1
fc_shell> set_attribute [get_lib_cells stdcell_lib/INV1] \
    psc_type_id 1
fc_shell> set_attribute [get_lib_cells fill_lib/fill2x] \
    psc_type_id 2
fc_shell> set_attribute [get_lib_cells stdcell_lib/NAND2] \
    psc_type_id 2
fc_shell> set_attribute [get_lib_cells stdcell_lib/NOR2] \
    psc_type_id 2
```

2. 使用 create_stdcell_fillers 插入可编程备用单元。例如插入 fill1x 与 fill2x:

```
fc_shell> create_stdcell_fillers \
    -lib_cells {fill_lib/fill1x fill_lib/fill2x}
```

在 ECO 流程中，工具根据 psc_type_id 属性设置、单元宽度与电压区域，将 ECO 单元与可编程备用单元交换。若工具从设计中移除 ECO 单元，可重用对应的可编程备用单元。

13.10 使用 eco_netlist 命令进行 ECO 更改

Making ECO Changes Using the eco_netlist Command

可使用 eco_netlist 命令对块进行 ECO 更改。

若使用 freeze silicon ECO 流程，在运行 eco_netlist 之前，须将 design.eco_freeze_silicon_mode 设为 true。

使用 eco_netlist 时，用下列两种方法之一进行 ECO 更改：

- 用 -by_verilog_file 指定包含 ECO 更改的 golden Verilog 网表；
- 用 -block 指定包含 ECO 更改的 golden 块。使用 -block 时，默认：
 - 工具假定 golden 块在当前设计库中；要指定不同设计库，用 -golden_lib；
 - 工具对当前设计做 ECO 更改；要对不同设计更改，用 -working_block；要用 -working_lib 指定含该设计的设计库。

工具将工作设计与输入 Verilog 网表或 golden 设计比较，并生成包含实现功能更改的网表编辑 Tcl 命令的变更文件。必须用 -write_changes 指定输出文件名。

默认情况下，工具忽略下列差异：

- 仅物理单元的差异。要用 -compare_physical_only_cells 考虑这些差异；

- 时序 ECO 更改,如被调整尺寸的单元或被添加/移除的中继器。要用 `-extract_timing_eco_changes` 在功能 ECO 之外同时考虑时序 ECO;
- 电源与地对象的差异。

要对工作设计应用 ECO 更改, `source eco_netlist` 生成的变更文件, 示例如下:

```
fc_shell> eco_netlist -by_verilog_file eco.v \
    -compare_physical_only_cells -write_changes eco_changes.tcl
fc_shell> source eco_changes.tcl
```

13.11 使用网表编辑命令进行 ECO 更改

Making ECO Changes Using Netlist Editing Commands

若 ECO 更改很少, 可用网表编辑命令更新设计。若使用 `freeze silicon` ECO 流程, 在运行网表编辑命令之前, 须将 `design.eco_freeze_silicon_mode` 设为 `true`。

网表编辑命令列表见 *Fusion Compiler Data Model User Guide* 中的 `Common Design Objects` 主题。

13.11.1 使用 ECO 脚本进行网表编辑

Using ECO Scripts for Netlist Editing

Fusion Compiler 支持在设计规划期间使用 ECO 脚本更改网表。可使用:

- `write_split_net_eco`: 将物理多扇出 `net` 的分支上推到顶层;
- `write_push_down_eco`: 将标准单元树下推一层;
- `write_spare_ports_eco`: 在子块上创建备用端口与 `net`。

更多信息见 *Fusion Compiler Design Planning User Guide* 中的 `Generating ECO Scripts for Netlist Editing` 主题。

13.12 调整单元尺寸

Resizing Cells

可用 `size_cell` 调整单元尺寸, 示例如下:

```
fc_shell> size_cell U21 -lib_cell AND2X4
```

若通过将 `design.eco_freeze_silicon_mode` 设为 `true` 启用 `freeze silicon` 模式, 默认情况下 `size_cell` 在调整尺寸前会检查在五倍单位 `site` 高度距离内是否有兼容备用单元。若无兼容备用单元, 则不调整该单元。

- 用 `-max_distance_to_spare_cell` 控制该距离;
- 用 `-not_spare_cell_aware` 禁用此功能。

13.12.1 撤销尺寸调整期间所做的更改

Reverting Changes Made During Resizing

要用 `revert_cell_sizing` 撤销由 `size_cell` 所做的 ECO 更改。用 `-include_adjacent_sized_cells` 可同时包含同一行中与指定单元相邻的已调整尺寸单元。使用该选项时，默认还撤销指定单元两侧紧邻的已调整尺寸单元；若再使用 `-adjacent_cell_distance`，工具会在同一行中递归撤销在指定距离内相邻的已调整尺寸单元。

13.13 在 net 上添加缓冲器

Adding Buffers on Nets

要对连接到指定 pin 的 net 添加缓冲器，使用 `add_buffer` 命令。使用时指定下列选项：

- `-new_net_names`：指定要添加的新 net 名称。添加缓冲器时每个缓冲器须一个 net 名；添加反相器对时每对须两个 net 名。
- `-new_cell_names`：指定要添加的新单元名称。添加缓冲器时每个缓冲器须一个单元名；添加反相器对时每对须两个单元名。
- `-inverter_pair`：添加反相器对而非缓冲器单元。须提供具有反相输出的库单元。
- `-respect_voltage_areas`：在电压区域层次内插入缓冲器，使其物理上位于对应布局中。与 `-snap` 互斥。
- `-no_of_cells`：指定每个 net 要插入的缓冲器单元或反相器对数。
- `-object_list`：指定须缓冲的 net、pin 或端口列表。若其单元已放置，新缓冲器或反相器对会靠近指定 pin 或端口放置。
- `-lib_cell`：指定用作缓冲器的库单元（命名库单元或库单元集合）。此为必需选项，与 `buffer_lib_cell` 互斥。
- `-snap`：将新缓冲器放到目标单元旁，对齐到最近 site，并在需要时翻转缓冲器。

13.14 在已布线 net 上添加缓冲器

Adding Buffers on Routed Nets

要对已完全布线的 net 添加缓冲器或反相器对，使用 `add_buffer_on_route`，见下列主题：

- 指定 net 名称、缓冲器类型及其位置
- 控制缓冲器的添加方式
- 为多电压设计指定设置
- 为 Freeze Silicon ECO 流程指定设置

13.14.1 指定 net 名称、缓冲器类型及其位置

Specifying the Net Names, Buffers Types, and Their Locations

使用 `add_buffer_on_route` 时必须指定：

- 要添加缓冲器的 net；
- 用下列方法之一指定要添加的缓冲器类型以及在 net 上的位置：
 - 按指定配置添加缓冲器
 - 在指定位置添加缓冲器
 - 按指定间隔添加缓冲器
 - 按 net 长度比例的间隔添加缓冲器
 - 按指定图案在总线（bus）上添加缓冲器

按指定配置添加缓冲器

Adding Buffers in a Specified Configuration 要用 `-user_specified_buffers` 按精确配置添加单元及其位置。该选项一次只能在一条 net 上添加单元；可插入不同类型的缓冲器或反相器对，但不能同时混合两者。

对每个添加的单元，用 `{instance_name library_cell_name x y layer_name...}` 格式指定：实例名、所用库单元、精确坐标、以及该位置上已有布线图形所在的层。也可用 `-detect_layer` 让工具自动检测该位置最近的布线图形，而不指定布线层。

下列示例在 net n22 上添加 ECO1（BUF2，位置 (100,70)，连到 M3）与 ECO2（BUF4，位置 (150,70)，连到 M3）：

```
fc_shell> add_buffer_on_route net22 \
-user_specified_buffers {ECO1 BUF2 100 70 M3 ECO2 BUF4 150 70 M3}
```

在指定位置添加缓冲器

Adding Buffers at Specified Locations 要在 net 的特定位置添加缓冲器，指定：

- 用 `-lib_cell` 给出可选库单元列表；
- 用 `-location` 给出精确位置，格式为 `{x1 y1 layer1 ....}`。也可用 `-detect_layer` 自动检测最近布线图形。

此方法一次只能在一条 net 上添加单元。

下列示例在 net1 的 (100, 200) 处添加 BUF1 并接到 M4：

```
fc_shell> add_buffer_on_route net1 -lib_cell BUF1 \
-location {100 200 M4}
```

默认新单元与 net 的名称前缀为 `eco_cell` 与 `eco_net`。可用 `-cell_prefix` 与 `-net_prefix` 分别指定。

按指定间隔添加缓冲器

Adding Buffers at a Specified Interval 要对一条或多条 net 按指定间隔添加缓冲器，指定：

- **-lib_cell**: 可选库单元列表；
- **-first_distance**: 驱动端到第一个缓冲器的距离；
- **-last_distance**: 负载 pin 到最后一个缓冲器的距离。必须与 **-repeater_distance** 或 **-repeater_distance_length_ratio** 一起使用；也可与 **-first_distance** 一起使用。但不能与 **-location**、**-user_specified_buffers**、**-user_specified_bus_buffers** 一起使用。

注意

使用 **-last_distance** 时，`eco.add_buffer_on_route.min_distance_buffer_to_load` 应用选项的值被忽略。

- **-repeater_distance**: 缓冲器之间的间隔。可选地用 **-scaled_by_layer** 为各层指定不同缩放因子，或用 **-scaled_by_width** 以层的默认宽度与实际布线宽度之比作为缩放因子。

示例：在 n2、n5 上每隔 150 微米添加 BUF1，驱动端到第一个中继器 100 微米：

```
fc_shell> add_buffer_on_route {n2 n5} -lib_cell BUF1 \
-first_distance 100 -repeater_distance 150
```

示例：在 n1 上每隔 100 微米添加 BUF2，首距 50、末距 20：

```
fc_shell> add_buffer_on_route -repeater_distance 100 \
-first_distance 50 -last_distance 20 net1 -lib_cell BUF2
```

按 net 长度比例的间隔添加缓冲器

Adding Buffers at an Interval That is a Ratio of the Net Length 指定：

- **-lib_cell**;
- **-first_distance_length_ratio**: 驱动端到第一个缓冲器相对总 net 长度的比例；
- **-repeater_distance_length_ratio**: 缓冲器间距相对总 net 长度的比例。

示例：在 n3、n21、n41 上按总长度 20% 的间隔添加 BUF1，首距为总长度的 10%：

```
fc_shell> add_buffer_on_route {n3 n21 n41} -lib_cell mylib/BUF1 \
-first_distance_length_ratio 0.1 \
-repeater_distance_length_ratio 0.2
```

按指定图案在总线上添加缓冲器

Adding Buffers on a Bus in a Specified Pattern 步骤如下：

1. 使用 `create_eco_bus_buffer_pattern` 为总线各 net 定义缓冲器图案，以避免缓冲器成团与重叠。

下列示例创建图 13.3 所示水平总线缓冲器图案：第一个缓冲器在最上方 net；从左起以距离 2 交错，每三个缓冲器后重复：

表 13.1: 与总线图案相关的命令（原书 Table 72）

目的	命令
创建总线图案	create_eco_bus_buffer_pattern
报告总线图案信息	report_eco_bus_buffer_patterns
获取缓冲器图案集合	get_eco_bus_buffer_patterns
移除总线图案	remove_eco_bus_buffer_patterns

```
fc_shell> create_eco_bus_buffer_pattern -name top_left \  
-first_buffer top -measure_from left -distance 2 -repeat_after 3
```



图 13.3: 水平总线的缓冲器图案

下列示例创建图 13.4 所示垂直总线图案：第一个缓冲器在最右侧 net；从上起交错距离为 2、3、2，然后重复：

```
fc_shell> create_eco_bus_buffer_pattern -name right_top \  
-first_buffer right -measure_from top \  
-user_specified_distance {2 3 2}
```

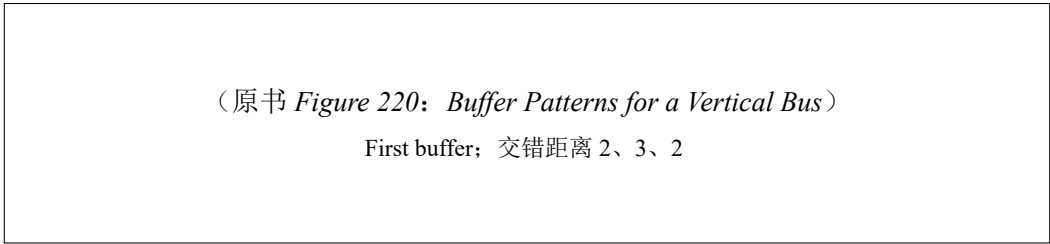


图 13.4: 垂直总线的缓冲器图案

- 2. 使用 add_buffer_on_route -user_specified_bus_buffers 在总线 net 上添加缓冲器。对每组缓冲器用 {buffer_pattern_name library_cell_name x y} 格式指定：图案名、库单元、图案中第一个缓冲器的坐标。

示例：在 data_A 总线上用图案 BP1 添加一组 BUF2，第一个缓冲器在 (100, 70)：

```
fc_shell> add_buffer_on_route [get_net data_A*] \  
-user_specified_bus_buffers {BP1 BUF2 100 70}
```

13.14.2 控制缓冲器的添加方式

Controlling How Buffers are Added

默认情况下，`add_buffer_on_route`：

- 在所有布局 `blockage`、软宏与硬宏之上添加缓冲器。要用 `-dont_allow_insertion_over_cell` 防止在特定宏上放置；或用 `-respect_blockages` 防止在所有 `blockage` 与宏上放置，并可用 `-allow_insertion_over_cell` 列出允许放置的宏。
- 在全局布线与详细布线的 `net` 上都添加缓冲器。要用 `-only_global_routed_nets` 仅在尚未分配 `track` 或详细布线的全局布线 `net` 上添加。
- 在被驱动 `pin` 的最低公共层次添加缓冲器。要用 `-on_top_hierarchy` 在尽可能高的层次添加。
- 若因 `net` 的逻辑与物理拓扑差异而需要创建新端口，则不在该布线段上添加缓冲器。要用 `-punch_port` 通过创建新端口在此类布线段上添加。

13.14.3 为多电压设计指定设置

Specifying Settings for Multivoltage Designs

对于多电压设计：

- 用 `-respect_voltage_areas` 确保遵守主/次电压区域，并在物理位于对应电压区域的逻辑层次中添加单元；
- 用 `-voltage_area_specific_lib_cells` 按 `{va1 lib_cell1 va2 lib_cell2 ...}` 为不同电压区域指定不同库单元（须同时使用 `-lib_cell`）；
- 用 `-select_mv_buffers` 按 `{va1 single_rail_lib_cell1 dual_rail_lib_cell1 ...}` 为不同电压区域指定单轨/双轨库单元（须同时使用 `-lib_cell`）。未指定的电压区域使用 `-lib_cell` 中的单元；默认电压区域名为 `DEFAULT_VA`；
- 用 `-allow_physical_feedthrough_buffer` 允许在指定电压区域的物理 `feedthrough net` 上放置缓冲器（只能与 `-respect_voltage_areas` 或 `-voltage_area_specific_lib_cells` 一起使用，且须为主电压区域）。之后用 `set_eco_power_intention` 更新所添加缓冲器的电源 `net` 与电源域设置；
- 用 `-respect_gas_station` 在指定电源 `net` 的 `gas station` 内添加单元。`Gas station` 是恒定供电区域，用于对穿越掉电电源域的 `net` 进行缓冲。用 `-max_distance_route_to_gas_station` 指定到 `gas station` 的最大允许距离。

13.14.4 为 Freeze Silicon ECO 流程指定设置

Specifying Settings for the Freeze Silicon ECO Flow

当 `design.eco_freeze_silicon_mode` 为 `true` 并在 `freeze silicon` 模式下运行 `add_buffer_on_route` 时，工具在添加缓冲器前检查五倍单位 `site` 高度距离内是否有备用单元；若无则不添加。

- 用 `-max_distance_to_spare_cell` 控制该距离；

- 用 `-not_spare_cell_aware` 禁用此功能。

13.15 优化 net 扇出

Optimizing the Fanout of a Net

在时序 ECO 流程中，可用 `split_fanout` 向 net 添加缓冲器并优化其扇出。必须指定：

- 要用 `-net` 按名称或用 `-driver` 按驱动端指定待优化 net；
- 用 `-lib_cell` 指定库单元；
- 用下列方法之一指定优化方式：
 - `-max_fanout`：该 net 的最大扇出约束；
 - `-load`：要缓冲的负载 pin 或端口；可用 `-hierarchy` 指定添加缓冲器的逻辑层次。

此外还可以：

- 用 `-on_route` 在现有布线拓扑上添加缓冲器（与 `-load` 合用时不能指定 `-hierarchy`）；
- 用 `-max_distance_for_incomplete_route` 对未完全连接到驱动端或负载端子的已布线 net 拆分扇出（指定端子与不完整布线的最大距离）。若在指定距离内找不到或找到多于一条布线段，工具报错；
- 用 `-respect_blockages` 放置缓冲器时避开布局 blockage 与宏；
- 用 `-cell_prefix` 与 `-net_prefix` 指定新单元与 net 的名称前缀（默认 `eco_cell/eco_net`）。

13.16 报告放置 ECO 单元的可用 site

Reporting Available Sites for Placing ECO Cells

可用 `report_cell_feasible_space` 报告可用于放置 ECO 单元的 site。默认报告整个块；可用 `-voltage_areas` 限制到特定电压区域，或用 `-boundary` 限制到特定矩形/多边形区域。

13.17 识别并撤销非最优 ECO 更改

Identifying and Reverting Nonoptimal ECO Changes

可用 `report_eco_physical_changes` 识别由 `size_cell`、`add_buffer` 与 `add_buffer_on_route` 所做的 ECO 更改是否会在 ECO 布局与合法化期间导致大位移。

可用 `revert_eco_changes` 撤销这些命令所做的非最优 ECO 更改；用 `-cells` 指定要撤销的单元。

13.18 放置 ECO 单元

Placing ECO Cells

对于无约束 ECO 流程，使用 `place_eco_cells` 放置 ECO 单元。该命令根据连通性与单元相关延迟推导每个 ECO 单元的布局位置，再将每个 ECO 单元合法化到最近的未占用 site。现有单元保持不动，以尽量减小对其布局的影响。

要放置：

- 全部未放置单元：用 `-unplaced_cells`；
- 特定单元：用 `-cells`；
- 仅因 ECO 操作而更改的单元：用 `-eco_changed_cells`。

若单元的 `eco_change_status` 属性为下列值之一，则视为已更改：`create_cell`、`change_link`、`add_buffer`、`size_cell`。运行 `eco_netlist` 时工具会自动设置该属性；若未使用 `eco_netlist` 更新设计，可用 `set_attribute` 手动设置。

13.18.1 使用 `place_eco_cells` 时控制布局

Controlling Placement When Using the `place_eco_cells` Command

可如下控制 ECO 单元布局：

- `-channel_aware`：使用宏之间的通道区域进行 ECO 布局（默认避开通道）。
 - `-fixed_connection_net_weight`：为连接到固定点（如 I/O pad 或宏）的 net 指定权重。默认权重为 1；大于 1 的整数使 ECO 单元更靠近固定点。
 - 要在 ECO 布局中优先处理特定 net：
 1. 用 `set_eco_placement_net_weight` 指定 net 权重；
 2. 用 `place_eco_cells` 的 `-honor_user_net_weight` 遵守这些权重。
- 用 `report_eco_placement_net_weight` 报告所指定的 net 权重。
- 要为 ECO 单元创建虚拟连接并在布局中使用（而非实际连接）：
 1. 用 `create_virtual_connection` 创建虚拟连接；
 2. 用 `-use_virtual_connection` 在 ECO 布局中使用它们。

```
fc_shell> create_virtual_connection -name VC_0 \
  -pins {U1/Y ECO17/A}
fc_shell> create_virtual_connection -name VC_1 \
  -pins {U2/Y ECO17/B}
fc_shell> create_virtual_connection -name VC_2 \
  -pins {ECO17/Y D17_OUT} -weight 3
fc_shell> place_eco_cells -cells ECO17 -use_virtual_connection
```

用 `remove_virtual_connections` 移除；用 `get_virtual_connections` 查询。

- **-max_fanout**: 忽略连接到 ECO 单元且超过特定阈值的高扇出 net;
- **-ignore_pin_connection**: 忽略 ECO 单元上特定 pin 的连接。

13.18.2 使用 **place_eco_cells** 时控制合法化

Controlling Legalization When Using the **place_eco_cells** Command

放置后, **place_eco_cells** 将每个 ECO 单元合法化到最近未占用 site; 现有单元保持不动。
可用下列方法控制合法化:

- **-no_legalize**: 放置后不合法化;
- **-legalize_only**: 仅执行合法化;
- **-legalize_mode**, 取值为:
 - **free_site_only** (默认): 在空闲 site 上合法化, 不移动既有单元。附近无空闲 site 时 ECO 单元可能有大位移;
 - **allow_move_other_cells**: 通过移动既有单元将 ECO 单元合法化到最近合法位置;
 - **minimum_physical_impact**: 先将可就近合法化到空闲 site 的 ECO 单元合法化; 对其余单元再通过移动既有单元合法化。
- **-displacement_threshold**: 指定合法化位移阈值; 还可与更大的 **-max_displacement_threshold** 合用。

这些位移阈值与 **-legalize_mode** 的配合如下:

- **free_site_only**: 仅合法化在位移阈值内有空闲 site 的 ECO 单元。超过阈值则不合法化, 并创建集合 **epl_legalizer_rejected_cells**。可随后用下式合法化:

```
fc_shell> place_eco_cells \
  -legalize_mode allow_move_other_cells \
  -legalize_only -cells $epl_legalizer_rejected_cells
```

若同时指定 **-max_displacement_threshold**, 还会创建集合 **epl_max_displacement_cells** (为前者子集), 用于识别位移极大、希望拒绝其 ECO 更改的单元。

- **allow_move_other_cells**: 不能指定上述两个位移阈值选项。
- **minimum_physical_impact**: 先合法化阈值内有空闲 site 的单元; 超过阈值的则通过移动既有单元合法化。若还指定 **-max_displacement_threshold**, 超过该最大阈值的单元不合法化, 并放入 **epl_max_displacement_cells**。
- **-remove_filler_references**: 指定合法化期间可移除的填充单元类型。填充单元在非固定布局且与 ECO 单元重叠时被移除; 默认合法化时不移除任何填充单元。
- 将 **place.legalize.enable_advanced_legalizer** 设为 **true** 以启用 Advanced Legalizer, 并在合法化后移除有违例的填充单元。与 **-remove_filler_references** 合用时, 仅移除该选项指定的填充单元。此时 **-min_filler_distance** 被忽略。

13.18.3 以最小物理影响（MPI）放置 ECO 单元

Placing ECO Cells With Minimal Physical Impact (MPI)

为尽量减小对既有布局的扰动，在 `place_eco_cells` 中使用：

- `-legalize_mode minimum_physical_impact`
- `-max_displacement_threshold`
- `-remove_filler_references`

13.19 将 ECO 单元放置并映射到备用单元

Placing and Mapping ECO Cells to Spare Cells

对于 freeze silicon ECO 流程，使用 `place_freeze_silicon` 将 ECO 单元放置并映射到备用单元。默认放置并映射全部 ECO 单元；用 `-cells` 指定特定单元。示例：

```
fc_shell> place_freeze_silicon -cells {EC01 EC02 EC03}
```

该命令将每个 ECO 单元放置并映射到最近的匹配备用单元。因 ECO 更改而删除的单元不会从设计中移除，而是转换为备用单元并保留在设计中。

使用 `place_freeze_silicon` 时可以：

- 用 `-no_spare_cell_swapping` 将 ECO 单元放到目标备用单元旁但不做映射，以便先分析 QoR；
- 用 `-map_spare_cells_only` 仅映射已放到目标备用单元旁的备用单元；
- 用 `-write_map_file` 生成 ECO 单元到备用单元的映射文件；必要时可编辑后用 `map_freeze_silicon` 映射；
- 用 `-min_filler_distance` 指定映射到可编程填充单元的 ECO 单元之间的最小距离。

13.19.1 为可编程备用单元指定映射规则

Specifying Mapping Rules for Programmable Spare Cells

可用 `set_programmable_spare_cell_mapping_rule`（在运行 `place_freeze_silicon` 之前）定义映射规则，以控制：

- 备用单元与 PG net 的重叠；
- 多高度备用单元的拆分与单高度备用单元的合并；
- 不同类型备用单元的兼容性。

用 `report_programmable_spare_cell_mapping_rule` 报告或移除所指定的映射规则。

13.19.2 将 ECO 单元映射到特定备用单元

Mapping ECO Cells to Specific Spare Cells

要用 `map_freeze_silicon` 将 ECO 单元映射到特定备用单元，用 `-eco_cell` 与 `-spare_cell` 指定名称。示例：

```
fc_shell> map_freeze_silicon -eco_cell ECO1 -spare_cell spare_1
```

也可用映射文件，并通过 `-map_file` 指定。映射文件格式为：

```
ECO_cell_1    spare_cell_1
ECO_cell_2    spare_cell_2
...          ...
```

13.19.3 将 ECO 单元映射到逻辑等效备用单元

Mapping ECO Cells to Logically Equivalent Spare Cells

工具映射 ECO 单元时会查找具有相同库单元名的备用单元；若找不到，该 ECO 单元保持未映射。要用下列步骤识别并将此类 ECO 单元映射到逻辑等效（LEQ）备用单元：

1. 用 `check_freeze_silicon` 识别没有匹配备用单元的 ECO 单元；
2. 用 `create_freeze_silicon_leq_change_list` 为这些 ECO 单元查找逻辑等效备用单元。指定：
 - `-cells`：未映射 ECO 单元名称；
 - `-output`：输出文件名。该文件为含网表编辑命令的 Tcl 脚本，将每个 ECO 单元替换为逻辑等效备用单元，可以是：
 - 具有相同逻辑功能但不同库单元名的单个备用单元；
 - 最多两个备用单元的组合，以给出相同逻辑功能。
3. 查看输出 Tcl 文件，确认映射满意，必要时编辑；
4. 在工具中 `source` 该 Tcl 文件，用逻辑等效单元替换 ECO 单元。

13.20 更新 ECO 单元的电源 net

Updating Supply Nets for ECO Cells

要用 `eco_update_supply_net` 更新 ECO 单元的电源 net。默认更新由下列 ECO 命令添加的全部单元的电源 net：

- freeze-silicon 模式下的 `size_cell`
- `add_buffer`
- `add_eco_repeater`
- `split_fanout`
- `add_buffer_on_route`

要对特定单元更新，使用 `eco_update_supply_net -cells`。

13.21 记录对版图所做的更改

Recording the Changes Made to a Layout

可用 `record_layout_editing` 记录对版图所做的更改并生成包含更改的 Tcl 文件，示例如下：

```
fc_shell> record_layout_editing -start
fc_shell> remove_shapes RECT_32_0
fc_shell> record_layout_editing -stop -output layout_changes1.tcl
```

可做下列版图更改：

- 用 Tcl 命令创建或移除版图对象；
- 用 `set_attribute` 设置属性；
- 用 GUI 移动或调整对象大小。

借助此功能，多名用户可并行地对版图不同部分做 ECO 更改。每位用户可用 `record_layout_editing` 输出含其版图更改的 Tcl 文件，再将这些 Tcl 文件全部应用到原始版图。

13.22 执行成组中继器插入与放置的先决条件检查

Performing Prerequisite Check for Group Repeater Insertion and Placement

使用 `place_group_repeater` 或 `add_group_repeater` 的 `-check_prerequisites` 选项，对设计执行先决条件检查，以便在命令流程开始时检测设计错误。

- 对 `place_group_repeater`，与下列选项一起使用以提供检查所需信息：

```
place_group_repeater
  -cells | -repeater_groups
  [-lib_cell_input]
  [-lib_cell_output]
  -check_prerequisites
```

- 对 `add_group_repeater`：

```
add_group_repeater
  -nets | -bundles
  [-lib_cell_input]
  [-lib_cell_output]
  -check_prerequisites
```

先决条件检查在设计与搜索区域中验证下列内容：

- 设计中存在 site 行；
- 每条 net 只有一个驱动端，可有多个负载；
- pin 或端口已分配位置；

- 对于标准单元上的 **pin**，标准单元已放置；
- 超网（supernets）必须已布线；
- 布线端点距搜索区域内的驱动端、负载 **pin** 或端口不得超过 $5\mu\text{m}$ ；
- 搜索区域内驱动端或负载的布线已定义；
- 负载已连接到驱动端；
- 当布线端点开路时，同一 **net** 的其他布线不得出现在该 **pin** 或端口附近；
- 布线必须分配给超网驱动 **pin** 的 **net**；
- 必须为布线创建 **via**；
- 中继器具有物理库单元。

下列示例展示用 **add_group_repeater** 的 **-check_prerequisites**：

```
fc_shell> add_group_repeater -bundles bundle_wo_load \
-lib_cell ref_lib1/libCell1 -lib_cell_input A -lib_cell_output X \
-repeater_distance 10 -check_prerequisite
Error: The net net_wo_load does not have loads. (ECO-329)
Error: The cell driver1 of the pin driver1/X is not placed. (ECO-331)
Error: 0
      Use error_info for more info. (CMD-013)
```

13.23 添加一组中继器

Adding a Group of Repeaters

要用 **add_group_repeater** 添加一组中继器，见下列各节：

- 定义一组中继器
- 将中继器列表分组
- 为中继器组设置约束
- 添加电压区域感知的成组中继器
- 报告分配给中继器组的约束
- 移除中继器组的约束
- 布线前放置成组中继器
- 执行沿布线（**on-route**）的中继器放置
- 为多比特寄存器放置成组中继器
- 指定中继器组位置
- 允许中继器组位于宏之上
- 指定中继器组的切割间距与切割距离

- 指定中继器组的水平与垂直间距
- 指定用作中继器的库单元
- 避免中继器与既有 tap 单元重叠
- 成组中继器插入期间避免串扰
- 预览中继器组
- 取消中继器放置
- 移除中继器组

13.23.1 定义一组中继器

Defining a group of repeaters

要为切割线(cutline)支持定义一组中继器,使用 `set_repeater_group` 或 `create_repeater_groups`。命令用法示例如下:

```
set_repeater_group -group_id group_id \
  -check_connection \
  [-cells cell_list] \
  [-cutline { {{x1 y} {x2 y}} | {{x y1} {x y2}} } ] \
  [-driver_group_id group_id] \
  [-path_drivers pin_port_list] \
  [-path_loads pin_port_list] \
  [-override]
  [-lib_cell_input string]
  [-lib_cell_output string]
  [-clear]
```

13.23.2 将中继器列表分组

Grouping a list of repeaters

要用 `create_repeater_groups` 自动将中继器列表或超网的中继器分组。该命令接受中继器列表、超网列表或超网束(bundles),并返回由 `set_repeater_group` 创建的中继器组列表。支持单负载与多负载超网。

```
create_repeater_groups -supernets supernet_list | \
  -supernet_bundles supernet_bundle_list | \
  -cells cell_list \
  -lib_cells lib_cell_list \
  [-lib_cell_input lib_pin] \
  [-lib_cell_output lib_pin] \
  [-group_pin_spacing spacing] \
  [-ignore_pin_layers]
```

13.23.3 为中继器组设置约束

Setting Constraints for a Group of Repeaters

要用 `set_repeater_group_constraints` 定义中继器组约束。该命令的优先级低于 `add_group_repeater` 与 `place_group_repeater` 中对应的命令选项。

```
set_repeater_group_constraints -type type_list \
  [-horizontal_repeater_spacing spacing] \
  [-vertical_repeater_spacing spacing] \
  [-layer_cutting_distance layer_scale_list]
```

13.23.4 添加电压区域感知的成组中继器

Adding Voltage Area Aware Group Repeaters

电压区域感知的成组中继器根据对应电压区域选择合法库单元，并更新中继器的电源 `net` 以满足多电压约束。可根据设计方法，用 `add_group_repeater` 的特定选项以三种方式添加：

- 仅单轨缓冲器：用 `-va_aware`

```
fc_shell> add_group_repeater -lib_cell <lib_cell> -va_aware
```

- 按电压区域的单轨缓冲器：用 `-voltage_area_specific_lib_cell`

```
fc_shell> add_group_repeater -lib_cell <default_lib_cell> \
  -voltage_area_specific_lib_cell {{VA1 libcell1}} {{VA2 libcell2}}
```

- 单轨与双轨缓冲器：用 `-select_mv_buffer`

```
fc_shell> add_group_repeater -lib_cell <default_lib_cell> \
  -select_mv_buffer {{VA1 SingleRail1 DualRail1}} \
  {{VA1 SingleRail2 DualRail2}}
```

电压区域感知成组中继器根据电压区域与中继器位置选择库单元，有助于解决多电压违例。

13.23.5 报告分配给中继器组的约束

Reporting the Constraints Assigned to a Group of Repeaters

要用 `report_repeater_group_constraints` 报告由 `place_group_repeater`、`add_group_repeater` 或 `set_repeater_group_constraints` 分配的约束。例如：

```
fc_shell> report_repeater_group_constraints
```

13.23.6 移除中继器组的约束

Removing Constraints for a Group of Repeaters

要用 `remove_repeater_group_constraints` 移除已分配的约束。例如：

```
fc_shell> remove_repeater_group_constraints -type type_list
```

13.23.7 布线前放置成组中继器

Placing Group Repeaters Before Routing

用 `preplace_group_repeater`s 在布线前放置成组中继器。按升序或降序放置（默认升序）：升序表示从最低或最左侧可用 track 放到最高或最右侧 track。

注意

运行 `preplace_group_repeater`s 需要 FAA-Base-Beta 许可证。

必须指定单元集合或成组中继器的组 ID。指定单元集合的示例：

```
fc_shell> preplace_group_repeater \
-repeater_group_locations { { $cellGroup1, M2, {720 363.5}, \
north, descending} {$cellGroup2, M2, {920 363.5}, east, ascending} \
{$cellGroup3, M2, {1230 328.5}, east, ascending} \
{$cellGroup4, M2, {1430 328.5}}, east}
```

指定组 ID 的示例：

```
fc_shell> preplace_group_repeater \
-repeater_group_locations { { 1,M2, {720 363.5}, south, ascending} \
{2, M2, {920 363.5},west, ascending} {3, M2, {1230 328.5}, \
south, ascending} {4, M2, {1430 328.5}, west, descending}}
```

13.23.8 执行沿布线的中继器放置

Performing On Route Placement of Repeaters

要用 `place_group_repeater`s 对互连中继器执行沿布线放置：

```
place_group_repeater \
-cells cell_list | -repeater_groups group_id_list \
[-lib_cell_input pin_name] \
[-lib_cell_output pin_name] \
[-max_distance_for_incomplete_route distance] \
[-first_distance distance] \
[-last_distance distance] \
[-blockage_aware] \
[-horizontal_repeater_spacing spacing] \
[-vertical_repeater_spacing spacing] \
[-layer_cutting_distance layer_scale_list]
[-verbose]
```

13.23.9 为多比特寄存器放置成组中继器

Placing Group Repeaters For Multibit Registers

要用带 `-multi_bit` 的 `create_repeater_groups`：


```
create_repeater_groups
-cells | -supernet_bundles | -supernets
-multi_bit
-pin_mapping {{lib_cell1 {inp1 outp1} {inp2 outp2}...}}
```

-multi_bit 根据路径驱动端确定多比特中继器顺序并创建中继器组。当 **net** 未定义布线时，该选项根据位置放置成组中继器。

要验证多比特寄存器放置支持，使用：

```
fc_shell> preplace_group_repeater \
-repeater_group_locations repeater_group_location_list
```

repeater_group_location_list 参数格式为：

```
{{cell_collection, layer, location, potential routing direction, placing
order} ...}
```

或

```
{{group_id, layer, location, potential routing direction, placing order}
...}。
```

13.23.10 指定中继器组位置

Specifying Locations for Repeater Groups

用下列互斥选项之一控制中继器组位置：

- **-repeater_distance**：中继器组之间的距离。支持：
 - **-first_distance**：驱动端到第一个中继器组中心的距离；
 - **-min_distance_repeater_to_load**：最后一个中继器组到负载的最小距离（默认是 **-repeater_distance** 的一半）；
 - **-tolerance**：相对 **-repeater_distance** 的最大增量距离（默认 $5\mu\text{m}$ ）。
- **-relative_distance**：依次指定驱动端到第一组、第一组到第二组、第二组到第三组等的距离；
- **-location**：每个中继器组中心的位置；
- **-number_of_repeater_groups**：沿布线均匀插入的中继器组总数；
- **-cutlines**：定义切割线的位置对。切割线须垂直于布线，中继器组以切割线为中心。

13.23.11 允许中继器组位于宏之上

Allowing Repeater Groups Over Macros

用 **-allow_insertion_over_block** 允许在软宏或硬宏上添加中继器组。

注意

若中继器单元是反相器，须自行确保逻辑正确性。

13.23.12 指定中继器组的切割间距与切割距离

Specifying Cut Space and Cut Distance for Repeater Groups

用下列互斥选项之一控制切割：

- `-layer_cutting_spacing`：布线到驱动 pin 的切割间距，切割从驱动端开始；
- `-layer_cutting_distance`：从中继器中心到输入/输出布线的切割距离。

13.23.13 指定中继器组的水平与垂直间距

Specifying Horizontal and Vertical Spacing for Repeater Groups

用下列互斥选项之一控制间距：

- `-horizontal_repeater_spacing`：按单元 site 宽度指定中继器水平间距；
- `-vertical_repeater_spacing`：按 site 行高度指定中继器垂直间距。

13.23.14 指定用作中继器的库单元

Specifying Library Cells as Repeaters

用 `-lib_cell` 指定用作中继器的库单元，并支持：

- `-lib_cell_input`：库单元有多个输入时要连接的输入 pin；
- `-lib_cell_output`：库单元有多个输出时要连接的输出 pin。

13.23.15 避免中继器与既有 tap 单元重叠

Avoiding Overlapping Repeaters With Existing Tap Cells

用 `-honor_special_cell` 在插入中继器时避免与既有 tap 单元重叠。

13.23.16 成组中继器插入期间避免串扰

Avoiding Crosstalk During Group Repeater Insertion

串扰管理仅在 `create_group_repeater_guidance` 与 `add_group_repeater` 命令流程中受支持。流程如下：

1. 用下列命令查询交错组（interleaving groups）：

```
create_group_repeater_guidance -net \
-number_of_interleaving_groups number
```

默认 `number` 为 1。例如取值为 2 时，一个 `net` 组被分成两个交错 `net` 组。

2. 用下列命令在成组中继器插入期间管理串扰：

```
add_group_repeater
-nets | -bundles
-first_distance_of_net_groups
```

```
{ {group_id1 $first_dist1} {group_id2 $first_dist2} ...}
-repeater_distance $dist
```

或

```
add_group_releaters
-nets | -bundles
-cutlines_of_net_groups
{ {group_id1 $cutline_list1} {group_id2 $cutline_list2} ...}
```

- 若指定 `-first_distance_of_net_groups`，命令按 `net` 上既有组 ID 分组 `net`，并按各 `net` 组指定的首距插入中继器；各组中继器间距相同。
- 若指定 `-cutlines_of_net_groups`，命令按 `net` 既有组 ID 分组，并按各组切割线插入中继器。

13.23.17 预览中继器组

Previewing Repeater Groups

用 `-preview` 在插入前预览中继器组位置。中继器用注解对象表示，位置、宽度与高度与中继器库单元相同，组 ID 为 `eco_preview_id`。

注意

再次预览前，须先用 `gui_remove_annotations` 或 `gui_remove_all_annotations` 移除注解对象。

13.23.18 取消中继器放置

Unplacing the Repeaters

要用 `unplace_group_repeaters` 取消先前的中继器放置：

```
unplace_group_repeaters \
-cells cell_list | -repeater_groups group_id_list \
[-lib_cell_input pin_name] \
[-lib_cell_output pin_name]
```

13.23.19 移除中继器组

Removing Repeater Groups

要用 `remove_eco_repeater` 移除一组中继器。`net` 恢复为其原始名称与布线图形。

13.24 查询成组中继器

Querying Group Repeater

成组中继器查询命令改善了互连中继器规划时的易用性。成组中继器查询：

- 提供查询成组中继器路径信息的能力；

- 支持跨会话查询；
- 支持在成组中继器放置之前查询。

规划期间查询成组中继器时，须熟悉下列关键信息：

- 中继器组的单元；
- 中继器组的切割线；
- 组虚拟连接（group virtual connection）。

若组具有驱动组或路径驱动端，则归类为组虚拟连接。组之间的单元连接取决于组内单元顺序。

下列主题描述该流程中的步骤：

- 执行自动分组流程
- 执行手动分组流程
- 单元输入模式（Cell Input Mode）

13.24.1 执行自动分组流程

Performing Auto Grouping Flow

自动中继器组创建允许为寄存器路径推导虚拟连接。对于中继器组输入模式，`place_group_repeater` 应用基于切割线的放置；须为每组指定切割线。中继器根据切割线与布线的交点放置。

基于目标单元的实际连接构建连通性。

1. 用 `create_repeater_groups`（并指定约束选项）对中继器列表或超网中继器分组。命令自动执行单元分组，并用下列属性分组虚拟连接：`group_repeater_driver`、`group_repeater_loads`、`pin_pair`、`group_id`。
2. 用 `get_attribute`、`report_repeater_group` 与 `get_repeater_path_info` 验证路径。`get_attribute` 报告 `group_repeater_driver` 或 `group_repeater_loads` 的属性；`report_repeater_groups` 报告指定组或全部组的信息，`-verbose` 打开时还报告全部组的中继器路径；`get_repeater_paths_info` 返回包含输入单元的中继器路径上的单元。
3. 用 `set_repeater_group -group_id -cutline` 手动为组指定切割线。
4. 对输入中继器组，用切割线在布线上放置互连中继器。

13.24.2 执行手动分组流程

Performing Manual Grouping Flow

自动分组基于 `create_repeater_groups`。若自动分组结果不满足要求，使用手动分组。

放置前检查目标单元连接。手动分组步骤：

1. 用 `-cells` 对单元分组；单元顺序表示单元连接意图。
2. 用 `-driver_group_id` 或 `-path_drivers` 定义组虚拟连接。
3. 用 `-cutline` 指定切割线。
4. 用 `set_repeater_group` 为每组中继器定义寄存器路径连接（虚拟连接）。

- 放置前用 `set_repeater_group -check_connection` 检查单元连接。若连接不符合预期，重新设置中继器组以得到期望连接。`-check_connection` 还可记录 pin 对；`pin_pair` 用于按指定库单元 pin 名查找单元实际连接。

5. 用 `get_attribute`、`report_repeater_group` 与 `get_repeater_path_info` 验证路径。
6. 用 `set_repeater_group -check_connection` 基于实际连接检查并报告无效路径。
7. 对输入中继器组，用切割线在布线上放置互连中继器。

13.24.3 单元输入模式

Cell Input Mode

单元输入模式允许基于沿布线放置设置距离。默认情况下，一条中继器路径上的中继器按均匀距离放置。该模式允许用 `-first_distance` 或 `-last_distance` 控制驱动端侧或负载侧距离。

1. 用 `place_group_repeater -cells -blockage_aware` 在内部启用自动分组；内部用 `create_repeater_groups -cells` 启用自动分组，并根据距离推导切割线，再用 `set_repeater_group -group_id -cutline` 定义中继器组。放置后可用 `report_repeater_groups` 查询中继器组，用 `get_repeater_paths_info` 查询路径信息。
2. 要用 `place_group_repeater -cells` 基于实际连接建立单元连接并应用基于距离的放置；此时不创建中继器组。

13.25 交换变体单元

Swapping Variant Cell

`add_group_repeater` 与 `place_group_repeater` 命令在成组中继器插入与放置中支持变体单元 (variant cell)。

下列主题描述该流程中的步骤：

- 设置变体单元约束
- 设置应用选项
- 对变体单元分组
- 运行并放置成组中继器
- 故障排除 (Troubleshooting)

13.25.1 设置变体单元约束

Setting Constraints of Variant Cell

设置变体单元约束：

1. 用 `set_repeater_group_constraints -type variant_cell_swapping` 设置用于单元交换的变体。
2. 设置后用 `report_repeater_group_constraints` 查看报告，报告提供：变体单元组的 track 图案、变体 site id、是否尊重不共享时序的单元组、首 track 对齐。

3. 用 `remove_repeater_group_constraints` 根据报告移除约束并复位设置。

13.25.2 设置应用选项

Setting Application Option

在使用 `add_group_repeaters` 与 `place_group_repeaters` 之前，用下列命令设置应用选项：

```
set_app_option -name eco.placement.variant_cell_swapping -value true
```

可根据需要规划变体单元流程。不同变体单元流程中变体参考交换规则不同。默认情况下，`eco_change_legal_reference` 中的变体单元支持会自动检测变体模式与设置。

`eco_change_legal_reference` 检查单元当前参考对放置位置是否合法；若不合法，则尝试从其定义的变体单元组中找到合法参考进行交换。

要对其它单元直接使用变体单元功能，使用 `eco_change_legal_reference`。

对于复杂场景，可手动切换变体模式并调整设置。变体单元支持的新应用选项包括：

- 为不同客户与设计流程设置变体模式：`eco.placement.variant_cell_group_mode`；
- 尊重不共享时序视图的变体单元组：`eco.placement.honor_cell_group_not_sharing_timing`。

13.25.3 对变体单元分组

Grouping Variant Cell

在同一变体单元组中，库单元变体与主变体（`master-variant`）仅在掩模颜色（或小几何）上不同，而这些变体的时序数据相同。

变体单元功能支持：

- `track_pattern` 模式
- `siteId_based` 模式

`track_pattern` 模式

`track_pattern mode` 要在 `track_pattern` 模式中指定用于 `track` 对齐检查的第一金属层，设置 `eco.placement.first_track_alignment_layer`。

图 13.5 显示具有两个变体的单元组。

（原书 *Figure 221: Cell Group With Two Variants*）

图 13.5: 具有两个变体的单元组

根据库单元的首 `track` 偏移与 `pitch` 值，可定义并推导从 0 开始的变体库单元类型。图 13.6 显示变体类型 `id` 定义。

可检测目标单元当前位置的变体类型 `id`，并检查当前参考是否合法；若不合法，则从单元组中选择合法参考并交换。图 13.7 显示对目标单元检查并交换合法参考。

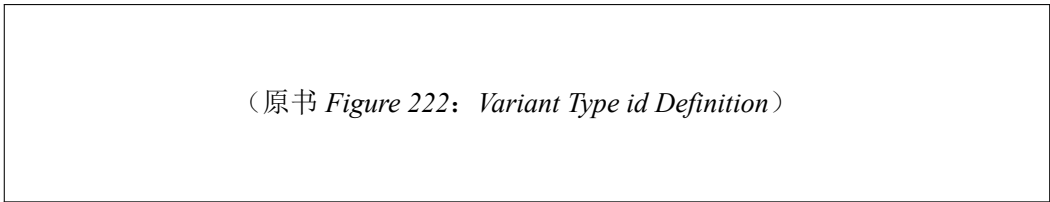


图 13.6: 变体类型 id 定义



图 13.7: 对目标单元检查并交换合法参考

siteId_based 模式

siteId_based mode 变体映射与翻转映射信息在单元组中定义。下列为 `report_cell_groups` 命令的示例。图 13.8 显示映射 id 与翻转映射 id 的单元组定义。

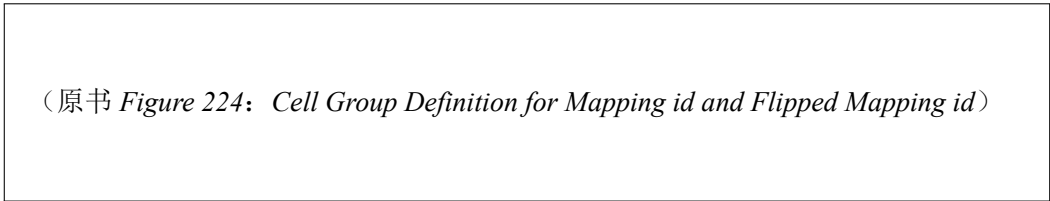


图 13.8: 映射 id 与翻转映射 id 的单元组定义

`siteId_based` 模式有两个子流程：2-Variants 与 N-Variant。图 13.9 显示这两个子流程。每个变体参考都有合法映射 id 与翻转映射 id。图 13.10 显示变体映射与翻转映射 id。对于变体 `siteId`，可先从 `site` 行获取周期与偏移，再用单元放置位置进行计算。图 13.11 显示变体 `siteId` 的计算。

13.25.4 运行并放置成组中继器

Running and Placing Group Repeaters

使用 `add_group_repeaters` 与 `place_group_repeaters` 在命令单元集合中显示变体单元交换的汇总报告。

13.25.5 故障排除

Troubleshooting

表 13.2（原书 Table 73）列出定义成组中继器期间遇到问题的成组中继器放置与插入消息。

(原书 *Figure 225: 2-Variants and N-Variants Sub Flows in siteId_based Mode*)

图 13.9: siteId_based 模式下的 2-Variants 与 N-Variants 子流程

(原书 *Figure 226: Variant Mapping and Flipped Mapping id*)

图 13.10: 变体映射与翻转映射 id

表 13.2: 成组中继器放置与插入故障排除（原书 Table 73）

命令	错误码	消息	排除提示
place_group_repeater	ECOUI-136	Error: The repeater group (%d) is invalid.	用 <code>report_repeater_groups</code> 检查输入组 id 是否存在。
place_group_repeater	ECO-220	Warning: Failed to connect to route for %d object: {%s}.	检查对象的 pin 形状与布线形状。驱动端或负载可以是端口、宏 pin 或已放置单元。
place_group_repeater	ECO-224	Error: The option -repeater_groups cannot be used when eco.placement.eco_enable_pipeline_register_placer is false.	检查对象 pin 形状与布线形状。驱动端或负载可以是端口、宏 pin 或已放置单元。
place_group_repeater	ECO-227	Warning: Failed to place %d repeaters for %d drivers in collection (%s), and failed cells are in collection (%s).	用 <code>query_objects</code> 、 <code>get_ports</code> 、 <code>get_pins</code> 、 <code>get_cells</code> 检查失败的驱动端。

(原书 *Figure 227: Calculation for Variant siteId*)

图 13.11: 变体 siteId 的计算

命令	错误码	消息	排除提示
place_group_repeater	ECO-228	Warning: Some repeaters from path driver (%s) have same repeater group id. The cells belonging to the path are {%s}.	检查同一路径上单元的组 id; 命令会跳过部分中继器。同一路径上单元的组 id 应不同。
place_group_repeater	ECO-229	Warning: Failed to find intersection of cutline %s and route for cell {%s}.	检查切割线与布线交点。
place_group_repeater	ECO-231	Error: The repeater group (%d) has no cutline or invalid cutline.	用 set_repeater_group 为该组设置正确切割线。
place_group_repeater	ECO-232	Error: Cutline {{%s} {%s}} is invalid.	检查切割线格式: {{x1 y} {x2 y}} 或 {{x y1} {x y2}}。
place_group_repeater / unplace_group_repeater	ECO-233	Error: The cells in collection (%s) are invalid since they have no target input pin name (%s) and output pin name (%s).	检查输入/输出 pin 名; 可用 -lib_cell_input/-lib_cell_output (默认分别为 D 与 Q), 或过滤无效单元。
place_group_repeater	ECO-234	Warning: There is no repeater to be placed for the path driver (%s).	检查该路径驱动端是否有待放置中继器。
place_group_repeater	ECO-235	Warning: There is no route on driver net (%s).	检查驱动 net 是否有布线。
place_group_repeater	ECO-236	Warning: There is no target load for path driver (%s).	检查路径驱动端是否有目标负载。
place_group_repeater	ECO-237	Warning: Some cell to be placed has no load for path driver (%s).	检查待放置单元是否缺少负载。
place_group_repeater	ECO-239	Info: %s.	检查放置与连接模式 (cutline/distance; virtual/real connection)。
place_group_repeater / unplace_group_repeater	ECO-240	Error: The option -cells cannot be used when eco.placement.eco_enable_virtual_connection is true.	若需虚拟连接, 改用 -repeater_groups。
place_group_repeater	ECO-241	Warning: The distance based placement does not support multi-fanout path driver(%s).	对多扇出路径驱动端改用基于切割线的放置。
place_group_repeater / unplace_group_repeater	ECO-242	Warning: The cell (%s) has no driver or load (number of driver: %d, number of load: %d).	检查单元驱动端/负载数量。
place_group_repeater	ECO-243	Error: The cells in collection (%s) are invalid since they have no driver or load.	确保待放置单元同时有驱动端与负载, 否则从输入中移除。

命令	错误码	消息	排除提示
place_group_repeater	ECO-244	Warning: The %s %s has no shape.	检查顶层端口或宏 pin 是否有形状。
place_group_repeater	ECO-246	Warning: Skip the path driver (%s) since the driver or load for the path has no shape.	路径驱动端或负载缺少形状时跳过。
place_group_repeater	ECO-251	Info: Place group repeaters successfully.	检查全部输入单元是否成功放置。
place_group_repeater	ECO-252	Warning: Failed to place repeaters for repeater path which driver is %s.	检查该路径驱动端的中继器放置失败原因。
place_group_repeater	ECO-253	Warning: The cutline information %s for cell %s is not on valid route.	检查切割线信息是否落在有效布线上。
place_group_repeater	ECO-254	Warning: Failed to cut route for cell %s.	检查是否未能为该单元切割布线。
place_group_repeater	ECO-255	Warning: Gap %s between driver and nearest route is larger than placing distance %s.	检查到最近 pin 的布线距离是否大于放置距离。
place_group_repeater	ECO-256	Warning: The first/last distance shorter than gap between pin and route.	检查 -first_distance/-last_distance 是否短于 pin 与布线间隙。
place_group_repeater	ECO-257	Warning: Cannot find cutline information for cell %s.	检查该单元是否有切割线信息。
place_group_repeater	ECO-258	Info: The app option about virtual connection is turned on...	确认 eco.placement.eco_enable_virtual_connection 为 true, 并指定了 -repeater_groups; 组内单元保持顺序。
place_group_repeater / unplace_group_repeater	ECO-259	Warning: Number of cells from repeater group inconsistent with drivers from driver group.	检查中继器组单元数与驱动组驱动端数是否一致。
place_group_repeater / unplace_group_repeater	ECO-260	Warning: Number of cells from repeater group inconsistent with loads from load group.	检查中继器组单元数与负载组负载数是否一致。
place_group_repeater / unplace_group_repeater	ECO-261	Warning: Number of path loads less than number of cells in group, so skip these path loads.	路径负载数少于组内单元数时跳过这些路径负载。
place_group_repeater / unplace_group_repeater	ECO-262	Warning: Number of path loads is not an integral multiple of cells in group, so use path loads in order.	路径负载数不是组内单元数的整数倍时按序使用路径负载。

命令	错误码	消息	排除提示
place_group_repeater	ECO-265	Warning: The pin shape {%-s} of %s (%s) is projected to route %s {%-s}.	检查布线是否远离驱动端或负载；工具将最近 pin 形状投影到布线。
unplace_group_repeater	ECO-266	Warning: Cells not placed before cannot be unplaced.	仅由 add_group_repeater 添加并由 place_group_repeater 放置的单元可被取消放置。
unplace_group_repeater	ECO-267	Error: Input and output routes are not merged after unplacing repeater.	取消放置后若布线未对齐或不在同一层则无法合并。
place_group_repeater	ECO-268	Error: Cells are invalid since they are placed before.	先用 unplace_group_repeater 取消放置。
add_group_repeater / place_group_repeater add_group_repeater	ECO-269 / ECO-270	Error: Unable to check the current reference of cell '%s': %s. Error: Unable to find swappable variant reference for cell '%s': %s.	检查报告原因及其他消息；用适当命令检查当前参考是否合法。
add_group_repeater / place_group_repeater add_group_repeater	ECO-271 / ECO-272	Error: Candidate reference and reference of cell are not in the same variant cell group.	检查候选与当前参考的兼容性及其是否同属一个变体单元组。
unplace_group_repeater	ECO-273	Error: The mixed input cells are not supported.	不支持混合由 add_group_repeater 添加与由 place_group_repeater 放置的单元。
unplace_group_repeater	ECO-274	Error: There are no placed cells to be unplaced.	检查是否存在符合条件的已放置单元。
—	ECO-275	Warning: You are deleting cell %s with eco_repeater_group_id %d.	检查是否在删除属于中继器组的单元。
place_group_repeater	ECO-276	Error: Failed to traverse cutlines. The nets may have too many blockages.	检查插入位置；移除 blockage 或找到切割线并用 -repeater_group。
add_group_repeater	ECO-277	Warning: Unable to find free area for repeater on net at location within range.	检查 checkerboard snapping 是否能为中继器找到空闲空间。
set_repeater_group	ECO-278	Warning: You are trying to override cells of repeater group. Use -override if you have to.	覆盖单元时须使用 -override。

命令	错误码	消息	排除提示
create_repeater_group	ECO-278	Error: Get 0 transparent cells from all supernets.	检查输入超网是否没有可分组的透明单元。
create_repeater_group	ECO-279	Error: %s has multiple(%lu) inputs.	检查单元或负载是否有多个输入。

13.26 修复多电压违例

Fixing Multivoltage Violations

ECO 流程中的设计更改有时会导致隔离（isolation）或电压违例。`fix_mv_design` 命令可识别并修复缓冲器树与二极管的违例。

必须与 `fix_mv_design` 一起使用 `-buffer` 或 `-diode` 选项。更多信息见：

- 修复缓冲器（Fixing Buffers）
- 修复二极管（Fixing Diodes）

若 UPF 尚未 `commit`，`fix_mv_design` 在执行任何操作前会自动 `commit` UPF。

该命令对网表所做的更改不保证布局或时序合法。运行后必须检查设计。

13.26.1 修复缓冲器

Fixing Buffers

`fix_mv_design` 的 `-buffer` 选项修复整个设计中缓冲器树内的隔离违例与非法库单元。命令可执行下列更改：

- 最小化双轨缓冲器或反相器的数量；
- 修复缓冲器树中的隔离违例；
- 修复具有非法设置的缓冲器与反相器，包括：
 - 工艺、电压与温度（PVT）设置；
 - 库单元 `purpose` 或 `subset` 设置；
 - 目标库 `subset` 设置；
 - 偏置电压（`bias voltages`）；
- 修复缓冲器电源域与其物理所在电压区域之间的失配。

命令可如下更改网表：

- 交换缓冲器库单元以修复非法设置；
- 将单轨缓冲器换为双轨缓冲器，或将双轨换为单轨；
- 更改双轨缓冲器与反相器的备份电源（`backup supplies`）；
- 更改电平移位器（`level shifter`）的输入或输出电源；

- 用物理或逻辑 feedthrough 修复缓冲器电源域与电压区域（或 gas station 电压区域）之间的失配。

fix_mv_design 保持原始缓冲器或反相器的布局，并遵守与既有缓冲器/反相器关联的中继器策略。此外，命令仅使用电压区域或电压区域分区中可用的电源。

该命令不会插入隔离单元或电平移位器单元来修复既有隔离或电压违例。由命令引起的网表更改不会引入新的隔离或电压违例。

要用 **-from** 与 **net** 或驱动 **pin** 名称指定缓冲器树。若指定 **net** 名，则使用其驱动 **pin**。两种情况下，驱动 **pin** 定义完整或部分缓冲器树。支持通配符。

-lib_cells 指定命令可用作替换单元的单轨与双轨缓冲器或反相器库单元列表。默认情况下，若交换并非修复隔离违例所必需，命令不会在双轨与单轨单元之间交换。但若同时指定 **-minimize_dual_rail**，工具会尽量减少双轨缓冲器的使用以节省面积。

若因 **fix_mv_design** 而插入新库单元，按下列优先级选择：

- 与原始库单元同一库中的单元；
- 与原始库名称相似的库中的单元；
- 与原始库单元名称相似的单元。

默认缓冲器树在电平移位器单元处停止。指定 **-level_shifter** 可允许命令修改电平移位器的输入或输出电源，但不会更改电平移位器库单元。

要用 **-report_only** 仅报告工具建议的更改而不做任何网表更改。

13.26.2 修复二极管

Fixing Diodes

fix_mv_design 的 **-diode** 选项修复整个设计中二极管的电源域与电压区域失配。该命令检查所有二极管，包括由 **create_cell**、**create_diodes**、**add_port_protection_diodes** 以及若干为修复天线违例而插入二极管的布线命令所插入的二极管。

若发现二极管的电源域与电压区域失配，工具用电压区域显式分配对应电源域。若同一电压区域存在多个电源域，工具任选其中一个可用电源域。任一可用电源域均可接受，因为所有二极管单元都是单轨单元，且同一电压区域内所有电源域具有相同的主电源。

要用 **-report_only** 仅获得建议更改报告而不做网表更改；用 **-verbose** 获得详细信息。这是可与 **-diode** 一起使用的仅有两个选项。